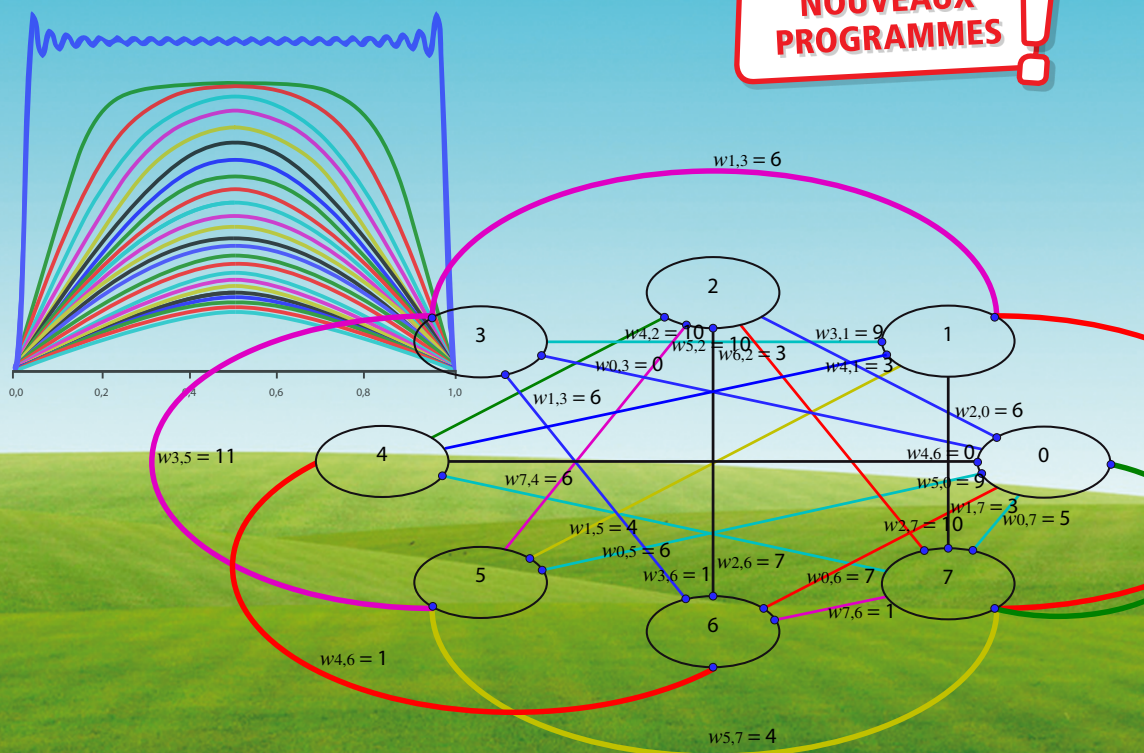


Classes préparatoires scientifiques 1^{re} et 2^e années

INFORMATIQUE

TRONC COMMUN

NOUVEAUX PROGRAMMES



Thierry Audibert
Amar Oussalah



ellipses

CPGE
scientifiques

1^{re} et 2^e années

INFORMATIQUE

TRONC COMMUN

**NOUVEAUX
PROGRAMMES**

Thierry Audibert
Amar Oussalah



Thierry Audibert a été professeur en classes préparatoires scientifiques jusqu'en 2017.
Il travaille depuis à des projets de développement de logiciels.

Amar Oussalah a été maître de conférences en informatique à l'Université de Provence.

Ils remercient chaleureusement Éric Chabert et Catherine Serretiello pour leurs relectures.

ISBN 9782340-048706
©Ellipses Édition Marketing S.A., 2021
8/10 rue la Quintinie 75015 Paris



Le Code de la propriété intellectuelle n'autorisant, aux termes de l'article L. 122-5.2° et 3°a), d'une part, que les « copies ou reproductions strictement réservées à l'usage privé du copiste et non destinées à une utilisation collective », et d'autre part, que les analyses et les courtes citations dans un but d'exemple et d'illustration, « toute représentation ou reproduction intégrale ou partielle faite sans le consentement de l'auteur ou de ses ayants droit ou ayants cause est illicite » (art. L. 122-4).

Cette représentation ou reproduction, par quelque procédé que ce soit constituerait une contrefaçon sanctionnée par les articles L. 335-2 et suivants du Code de la propriété intellectuelle.

www.editions-ellipses.fr

Introduction

Le cours que nous proposons ici est conforme au programme d'informatique de tronc commun des classes préparatoires scientifiques¹. Il peut bien sûr servir hors de ce contexte précis pour toute personne qui voudrait apprendre ou reprendre les bases de l'algorithmique avec Python.

Son organisation en trois parties (et quinze chapitres) est conforme à l'organisation suggérée par ce programme dont la logique nous convient parfaitement.

Nous avons fait le choix, dès la première partie et pour la quasi-totalité des algorithmes étudiés, de mettre en place soit dans le cours, soit dans les exercices, les preuves de programme et l'étude de la complexité. Dans les cas les plus délicats il sera possible de ne se focaliser dans un premier temps que sur la compréhension, la programmation et une vérification empirique de l'algorithme (les tests), pour revenir ensuite sur les aspects plus théoriques au moment de faire la synthèse et de prendre du recul sur ces questions (ce qui est abordé au chapitre huit).

Le premier chapitre reprend l'essentiel du langage Python. Il est conçu pour servir de référence et permet de rendre le manuel auto-suffisant. Si votre maîtrise de Python le permet, vous pourrez entrer directement dans le vif du sujet avec le chapitre deux et les suivants, mais il est conseillé de vérifier régulièrement les spécificités du langage chaque fois qu'un doute ou une anomalie apparaissent.

Il y a dans ce manuel plus de 150 exercices qui sont tous corrigés, ce qui explique son volume. Les scripts et des compléments sont disponibles sur le site des éditions Ellipses et sont accessibles avec le QR-code ci-joint.



1. Il s'agit du programme qui a pris effet en 2021-2022 pour les classes de première année et en 2022-2023 pour les classes de seconde année.

Table des matières

Partie I • Premier semestre

Chapitre 1 • Programmer avec Python	13
1.1 Constantes, identificateurs, variables et affectations	13
1.2 Mots réservés du langage	18
1.3 Types prédéfinis avec Python	19
1.3.1 Types numériques : entiers, flottants, complexes	19
1.3.2 Le type None	20
1.3.3 Le type booléen	21
1.3.4 Les conteneurs	22
1.3.5 Opérations sur les listes, ensembles et dictionnaires	28
1.3.6 Exercices	30
1.4 La programmation et les fonctions avec Python	31
1.4.1 Blocs et indentation	31
1.4.2 Instructions conditionnelles	31
1.4.3 Parcours des objets itérables, boucles for	35
1.4.4 Continue, pass	38
1.4.5 Listes en compréhension	39
1.4.6 Boucle while	40
1.4.7 Break ou pas break ?	44
1.4.8 Procédures et fonctions	45
1.4.9 Compléments : sous-procédures et visibilité des variables	50
1.4.10 Exercices	51
1.5 Modules ou bibliothèques	54
1.5.1 L'instruction import	54
1.5.2 Tableaux (array) de la bibliothèque numpy	56
1.5.3 Les graphiques avec matplotlib	60
1.6 Corrigés des exercices	70
Chapitre 2 • Quelques algorithmes itératifs fondamentaux	95
2.1 Au début était l'arithmétique	96
2.1.1 La division euclidienne des entiers	96
2.1.2 L'écriture binaire d'un entier	98

2.2	Calcul de la moyenne et de la variance	100
2.2.1	Calcul conjoint et analyse	100
2.2.2	Méthode des trapèzes	102
2.3	Algorithmes de recherche séquentielle	103
2.3.1	Listes ou tableaux	103
2.3.2	Recherche d'un élément dans une liste	104
2.3.3	Recherche des plus grands éléments d'une liste	106
2.3.4	Dictionnaires et comptage	110
2.4	Boucles imbriquées	114
2.4.1	Valeurs les plus proches dans un tableau	114
2.4.2	Recherche d'une sous-chaîne dans une chaîne de caractères	115
2.5	Algorithmes dichotomiques	117
2.5.1	Algorithme de recherche dichotomique	117
2.5.2	Exponentiation rapide, version itérative	120
2.6	Corrigés des exercices	122
Chapitre 3	• Récursivité	143
3.1	Introduction	143
3.1.1	Vocabulaire, premiers exemples	143
3.1.2	Quelques dessins de fractales	147
3.2	Mise en garde concernant l'efficacité des programmes récursifs	151
3.3	Mise en oeuvre	157
3.4	Diviser pour régner	160
3.4.1	Recherche de deux points réalisant la plus petite distance	161
3.4.2	Exponentiation rapide, version récursive	163
3.4.3	Recherche dichotomique dans une liste triée	164
3.4.4	Illustration avec des tris	166
3.5	Corrigés des exercices	167
Chapitre 4	• Les tris	189
4.1	Introduction	189
4.2	Tri par insertion	190
4.3	Tri rapide : diviser pour régner	192
4.4	Tri fusion	194
4.5	Tri par insertion dichotomique	196
4.6	Complexité des tris	198
4.7	Sort dans Python	199
4.8	Recherche de la médiane en temps linéaire	200
4.9	Corrigés des exercices	202

Chapitre 5 • Algorithmes gloutons	207
5.1 Introduction	207
5.1.1 Optimisation combinatoire	207
5.1.2 Le principe des algorithmes gloutons	210
5.2 Mise en oeuvre de stratégies gloutonnes	211
5.2.1 Le rendu de monnaie	211
5.2.2 Sélection d'activités	213
5.2.3 Allocations de salles de cours	216
5.3 Bilan	219
5.4 Corrigés des exercices	220
Chapitre 6 • Traitement de l'image	233
6.1 Représentation des images, formats, outils	233
6.1.1 Tableaux à plusieurs dimensions et représentation des images	233
6.1.2 Des fichiers images vers les tableaux de numpy	234
6.2 Dessiner une droite à l'écran	240
6.3 Transformations géométriques d'une image	243
6.3.1 Agrandir : Homothétie d'une image avec $k > 1$	244
6.3.2 Mode d'emploi pour faire tourner une image	245
6.4 Traitements par convolution	249
6.4.1 Filtres linéaires et convolution	249
6.4.2 Quelques effets du filtrage linéaire	255
6.4.3 Détection de contours	256
6.5 Corrigés des exercices	262

Partie II • Deuxième semestre

Chapitre 7 • Calcul numérique : problématique et outils	279
7.1 Représentation des nombres et erreurs de calcul	279
7.1.1 Numérations décimale, binaire, hexadécimale	280
7.1.2 Représentation des entiers sur n bits	282
7.1.3 Les entiers multi-précision de Python	285
7.1.4 Représentation des flottants sur n bits	288
7.1.5 Peut-on calculer avec les flottants ?	293
7.1.6 Exercices	299
7.2 Corrigés des exercices	302

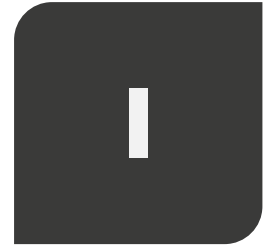
Chapitre 8 • Preuves et complexité des programmes	311
8.1 Spécification d'un algorithme	311
8.1.1 Le vocabulaire	311
8.1.2 Vérifier les pré-conditions et les post-conditions	312
8.1.3 Exemples	315
8.2 Le point sur la notion de preuve d'un algorithme	316
8.3 Le point sur la notion de complexité	321
8.3.1 La place, le temps, la précision	321
8.3.2 Les outils : théorie et pratique	323
8.3.3 Exemples basiques	329
8.3.4 Complexité de l'algorithme d'Euclide	330
8.4 Analyse des programmes récursifs	334
8.5 Exercices	337
8.6 Corrigés des exercices	340
Chapitre 9 • Graphes	365
9.1 Définitions	365
9.2 Listes d'adjacence	367
9.3 Matrices d'adjacence	369
9.4 Parcours en profondeur, composantes connexes	370
9.5 Parcours, versions itératives	375
9.5.1 Piles et files	375
9.5.2 Parcours en largeur, procédure itérative	375
9.5.3 Parcours en profondeur, version itérative	378
9.6 Un algorithme de plus court chemin	381
9.7 Graphes, amas et percolation	386
9.8 Corrigés des exercices	390
Chapitre 10 • Un bref aperçu de la programmation objet	405
10.1 Les concepts de la programmation objet	405
10.2 Une classe pour la structure de graphe	406
10.3 Percolation, une implémentation objet	411

Partie III • Troisième semestre

Chapitre 11 • Bases de données, langage SQL	421
11.1 Introduction	421

11.2	Qu'est ce qu'une base de données relationnelle ?	422
11.2.1	Les relations comme ensembles de p -uplets	422
11.2.2	Modèle relationnel	423
11.3	Algèbre relationnelle	426
11.3.1	La sélection	427
11.3.2	La projection	427
11.3.3	Le produit cartésien de deux tables, le renommage et la jointure	428
11.3.4	La jointure	429
11.3.5	Conflits de noms d'attributs et renommage	430
11.3.6	Union, intersection et différence	430
11.3.7	Récapitulatif et expressions de requêtes avec l'algèbre relationnelle	431
11.4	Langage de manipulation de données, SQL	433
11.4.1	Les interrogations	433
11.4.2	GROUP BY, HAVING et les fonctions d'agrégation	439
11.5	Exercices	442
11.6	SQLite, PostgreSQL, MySQL	445
11.7	Corrigés des exercices	447
Chapitre 12	• À propos des dictionnaires	463
12.1	Dictionnaires ou tableaux associatifs	463
12.1.1	Tableau associatif comme type de données	463
12.1.2	Tableaux associatifs et tables de hachage	466
12.1.3	Quelques fonctions de hachage	467
12.2	Compression de texte : LZ78	469
12.3	Corrigés des exercices	471
12.4	Annexe	474
Chapitre 13	• Programmation dynamique	477
13.1	Premiers exemples	477
13.1.1	Plus longue sous-suite commune	477
13.1.2	Produit de matrices, parenthésages optimaux	482
13.1.3	Problèmes éligibles à la programmation dynamique	486
13.2	Autres exemples	488
13.2.1	Distance d'édition	488
13.2.2	Algorithme de Roy-Floyd-Warshall	490
13.3	Corrigés des exercices	493

Chapitre 14 • Algorithmes pour l'étude des jeux	507
14.1 Jeux sur graphes	507
14.1.1 Exemples de jeux à deux joueurs	507
14.1.2 Représentation par des graphes, vocabulaire	510
14.2 Calcul des attracteurs dans les jeux d'accessibilité	512
14.2.1 Jeu d'accessibilité	512
14.2.2 Attracteurs et pièges	513
14.3 Algorithme du minimax, heuristiques	517
14.3.1 Arbres	517
14.3.2 L'algorithme du minimax	522
14.3.3 L'algorithme du minimax avec heuristiques	524
14.4 Corrigés des exercices	525
14.5 Une classe pour représenter les arbres	535
Chapitre 15 • Algorithmes pour l'étiquetage et la classification	537
15.1 Vocabulaire et définitions	537
15.1.1 Classement et classification automatique	537
15.1.2 Distances, similarités	539
15.1.3 Inertie d'une partition	541
15.1.4 À propos d'intelligence artificielle	543
15.2 Classement supervisé, k-plus proches voisins	544
15.2.1 L'algorithme	544
15.2.2 Les tests	545
15.3 Classification non supervisée, algorithme des k-moyennes	548
15.4 Bibliothèque scikit-learn	552
15.4.1 <i>scikit-learn.datasets</i>	552
15.4.2 <i>k-plus proches voisins avec scikit-learn</i>	553
15.4.3 <i>k-moyennes avec scikit-learn</i>	553
15.4.4 <i>Lexique français anglais (US)</i>	556
15.5 Corrigés des exercices	557
Glossaire de l'informatique générale	569
Bibliographie	581
Index	585



Premier
semestre

Programmer avec Python

Mode de lecture de ce premier chapitre

Ce chapitre regroupe les éléments du langage Python qui permettent la mise en place de l'enseignement de l'informatique de tronc commun. Il est conçu comme un chapitre de référence auquel vous vous référerez quand vous aurez besoin de vérifier ou de chercher des éléments de syntaxe.

Nous ne présenterons qu'une petite partie des primitives disponibles dans la bibliothèque standard et nous travaillerons avec la version 3.8 de Python. Pour en savoir plus, si le besoin s'en fait sentir, vous pourrez utiliser l'aide en ligne avec la fonction `help(...)` ou encore vous référer à la documentation officielle de Python :

<https://docs.python.org/fr/3.8/>

Python est un langage **interprété**¹, dont le typage est dynamique, qui permet la programmation **fonctionnelle**, la programmation **impérative** et la programmation **orientée objet**. Nous n'aborderons que très succinctement cette dernière (au chapitre 10) qui est par ailleurs abondamment illustrée par les objets natifs de Python.

1.1 Constantes, identificateurs, variables et affectations

Un langage de programmation permet de traiter des constantes, des variables, des expressions. Ce qui suit est illustré avec Python et il y aurait beaucoup de ressemblances avec la plupart des autres langages.

- Une **constante** est un objet de valeur connue non modifiable qui peut être traité par le langage². Par exemple, 12, 12.3, -5.1, 0.001, 'bonjour', **True**, **False**, [] (liste vide), [1,2], (1,2), {'a':1, 'b':1}. Chaque constante admet un **type** (entier, flottant,

1. Les mots en gras non définis dans le texte sont souvent expliqués dans le glossaire. En l'occurrence page 573.

2. Dans certains langages, comme en C, on peut affecter une constante à un identificateur qui ne supportera pas d'autre affectation au long de l'exécution du programme. Cet identificateur est déclaré comme constante.

chaîne de caractères, booléen, liste, tuple, dictionnaire, etc.) qui définit les opérations que cette valeur supporte : algébriques sur les nombres, logiques sur les booléens, accès à élément d'indice donné pour les tuples ou les listes, modification ou insertion d'un élément pour les listes.

- Une **variable** est l'association d'un **identificateur** et d'une valeur (ou constante) stockée en mémoire. Un identificateur est une suite de symboles commençant soit par une lettre soit par le signe `_` (underscore) suivi de lettres et/ou de chiffres ou encore de `_` qui doit par ailleurs être et différente des mots réservés du langage (dont la liste est en page 18).
- Cette association est réalisée par l'**affectation** d'une valeur à la variable. C'est l'opération qui rend cette dernière utilisable. Au cours de l'exécution d'un programme Python, une même variable peut prendre des valeurs de différents types³. La syntaxe d'une affectation est `<variable> = <valeur>`.

<pre>>>> x = 7 >>> y = x >>> z = x+1 >>> x, y, z (7, 7, 8) >>> x = y = 12 >>> x, y (12, 12) >>> y = 1 >>> x, y (12, 1)</pre>	<pre>>>> x1 = 8 >>> y, z = x1, 2*x1 >>> x, y, z (12, 8, 16) >>> 34**3 39304 >>> a=_ >>> a 39304</pre>
--	--

Remarque : La variable `"_"` joue un rôle particulier : elle contient la dernière expression évaluée (qui est le contenu de l'**accumulateur**). C'est le `ans()` des calculettes TI ; son usage est réservé aux consoles pour des raisons évidentes de lisibilité et de sécurité du code.

Exercice : On suppose que `x` et `y` ont été préalablement affectées. Que donnent les instructions successives `x = y; y = x; ?` Ont-elles permis de permuter les contenus de `x` et de `y` ?

3. Dans d'autres langages les variables doivent être préalablement déclarées avec leur type qui est celui des constantes qui leur seront affectées ; ce type, contrairement à ce qui se passe avec Python est invariable. On dit que ce typage est **statique**, dans le cas de Python, on parle de **typage dynamique**.

• **Affectation multiple**

Python autorise l’affectation multiple ce qui est illustré dans la colonne de gauche du tableau qui précède. Nous montrons ci-dessous comment on permute, grâce à cela, le contenu de deux variables. La colonne de droite, quant à elle, montre comment on procède à l’aide d’une variable auxiliaire dans un langage sans affectation multiple. Vous devez savoir le faire !

<pre>>>> x,y =10,11 >>> x,y (10, 11) >>> x,y = y,x >>> x,y (11, 10)</pre>	<pre>>>> x,y =10,11 >>> z = x >>> x = y >>> y = z >>> x,y (11, 10)</pre>
---	---

• **L’incrémentation += et ses variantes**

On peut coder l’incrémentation $n = n+1$ de façon plus concise avec $n += 1$ et décliner cela avec les opérateurs arithmétiques $+$, $-$, $*$...

<pre>i = 0 while i < 10: i +=1 print(i)</pre> 1 2 3 4 5 6 7 8 9 10	<pre>i = 10 while i > 0: i -=1 print(i)</pre> 9 8 7 6 5 4 3 2 1 0	<pre>i = 1 while i < 1030: i *=2 print(i)</pre> 2 4 8 16 32 64 128 256 512 1024 2048
--	---	--

• **L’opérateur := (à sauter en première lecture)**

L’opérateur $:=$ permet de réaliser une affectation dans une autre instruction comme on le montre avec une instruction conditionnelle (les deux programmes ont le même

effet, dans le second l'affectation est codée dans l'instruction). Il est présent à partir de la version 3.8 de Python et ne devrait pas vous être indispensable.

```
L = [1,2,3]
n = len(L)
if n < 10:
    print('L a %s éléments, c\'est trop petit!'%(n))

>>> L a 3 éléments, c'est trop petit!

L = [1,2,3]
if (m := len(L)) < 10:
    print('L a %s éléments, c\'est trop petit!'%(m))

>>> L a 3 éléments, c'est trop petit!
```

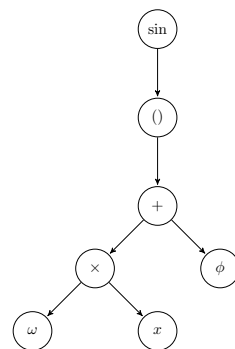
• Une **expression** est une combinaison, construite selon les règles de syntaxe du langage, formée de constantes, variables, opérateurs et fonctions. Par exemple :

- si x, y, a, b, c sont des constantes numériques ou des variables qui ont déjà été affectées de valeurs numériques, $a*x**2+b*x*y+c*y**2$ qui est l'écriture en Python de $ax^2 + bxy + cy^2$ est une expression numérique valide ;
- la liste $[a,b,c,x*y]$ est elle aussi une expression ;
- si la variable x contient une chaîne de caractères, $x + \text{'autre chaîne'}$ est encore une expression valide (l'opérateur $+$ entre deux chaînes désigne la **concaténation** en Python).

Arbre syntaxique associé à une expression

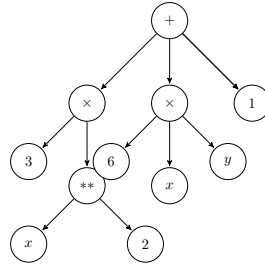
On définit formellement les expressions comme des arbres finis dont les feuilles sont des opérandes (variables ou constantes), les nœuds non terminaux des opérateurs ou des fonctions. Le nombre de sous-arbres dépend de l'**arité** de l'opérateur que le nœud représente.

Par exemple, $\sin(\omega x + \phi)$ est une expression mathématique valide. L'arbre syntaxique qui lui est associé est représenté à droite.



Exercice 1.1 *représentation des expressions par des arbres*

1. Quelle est l'expression associée à l'arbre qui figure à droite ?
2. On y représente une addition, une multiplication avec 3 branches dérivées, pourtant il s'agit d'opérateurs binaires, pourrait-on faire la même chose avec l'élévation à la puissance ?



3. Réécrire cette même expression avec un arbre binaire (dont les nœuds ont au plus deux fils). *Corrigé en 1.1, page 70*

Séparateurs...

- le **point-virgule** sépare deux **instructions** sur une même ligne dans le shell ou dans un script ;
- la **virgule** entre deux **expressions** à l'intérieur de crochets [] est un constructeur de **liste** ;
- la **virgule** entre deux **expressions** à l'intérieur de () ou pas, est un constructeur de **tuple** ;
- les **espaces** et l'**indentation** tiennent lieu de délimiteurs syntaxiques ; nous détaillons cela avec l'apprentissage de la programmation, page 31 ;
- le **double point** n'est pas un séparateur d'instructions, c'est un composant syntaxique des boucles et instructions conditionnelles.

```
>>> a = 123; b = 145
>>> a,b
(123, 145)
>>> a; b
123
145
>>> z = a; a = b; b = z
>>> a,b
(145, 123)
>>> c = 12
SyntaxError: unexpected indent
>>> c = 12
>>> for i in range(0,3):
    print(i);
>>> L =[a,]; L
[145, 123]
```


Illustration dans une console :

- observez la différence entre a ; b et a,b
- soyez attentifs au message d'erreur lorsqu'il y a un espace en début de ligne dans

```
>>> c = 12
```

1.2 Mots réservés du langage

Ce sont les mots du langage standard ; ils ne peuvent servir d'identifiants. Le tableau indique leur contexte d'utilisation et la page où ils sont présentés dans ce cours lorsque c'est le cas.

mot	contexte	page
and	connecteur logique binaire (expressions booléennes)	21
as	associée à import dans ' import package as ...'	55
assert	après évaluation d'une expression booléenne, permet de lever l'exception <code>AssertionError</code>	313
async	non traité ici	
await	non traité ici	
break	provoque l'abandon d'une boucle for ou while	44
class	déclaration d'une classe (programmation orientée objet)	405
continue	dans une boucle, permet de sauter l'étape en cours	38
def	déclaration d'une fonction	45
del	permet d'effacer un ou plusieurs éléments à partir de leurs indices	28, 29
elif	dans une instruction conditionnelle if...elif...else	31
else	dans une instruction conditionnelle if...elif...else	31
except	associé à try pour la gestion des erreurs	...
False	une des deux constantes booléennes	21
finally	associé à try pour la gestion des erreurs	...
for	définit une boucle for ... in...	35
from	' from package import ...'	54
global	déclaration des variables globales dans une fonction	47
if	définit une instruction conditionnelle if...elif...else	31
import	dans ' import package '	54
in	associé à for pour définir une boucle	21
in	opérateur booléen ; relation d'appartenance à un conteneur	21
is	teste l'égalité de deux objets (comme <code>==</code>)	21
lambda	définition d'une fonction par une expression	49
None	voir l'usage avec les définitions de fonctions et procédure	45
nonlocal	gestion de la visibilité d'une variable dans une sous-procédure (à associer à local et global)	50
not	opérateur logique unaire	21
or	connecteur logique binaire (expressions booléennes)	21
pass	instruction vide (pratique en cours de programmation)	38
raise	levée d'une exception	...

return	signale la valeur que retourne une fonction	45
True	une des deux constantes booléennes	21
try	instruction permettant de gérer (capturer) des erreurs ou exceptions	...
while	définit une boucle conditionnelle	40
with	simplification d'écriture ; associé à as (non traité ici)	...
yield	associé à la notion de co-routine (non traité ici)	...

1.3 Types prédéfinis avec Python

1.3.1 Types numériques : entiers, flottants, complexes

• Les entiers

On représente les entiers (éléments de $\mathbb{Z}$) par des objets de type **int** (pour integer ou entier). Les opérations arithmétiques usuelles sont définies comme sur une calculatrice : +, - (relation binaire, soustraction), - (unaire, changement de signe), * (multiplication), ** (élévation à la puissance) ; a/b désigne le quotient dans la **division euclidienne** de a par b, le reste est $a\%b$, **divmod**(a,b) est le couple (q, r) où la relation $a = bq + r, 0 \leq r < b$ définit (q, r) de façon unique.

• Les flottants

On approche les réels par des objets de type **float** (pour float ou flottant). Les constantes de type float ont un affichage décimal (comme 12.3) ou scientifique (comme 2.4379168015552228e-36). Les opérations usuelles sont encore définies : +, - (unaire et binaire), *, /, ** (avec $a * b = e^{b \ln a}$ si b n'est pas entier) ; comme les opérations sur les entiers, elles obéissent aux mêmes **règles de priorité** que celles de vos calculatrices et qui sont nos règles de calcul et de parenthésage habituelles. Les conversions de bon sens⁴ pour les expressions mêlant entiers et flottants sont assurées (a+x avec a entier et x flottant retourne un flottant), la division de deux entiers a/b retourne le quotient approché (on la distinguera donc de l'expression retournant le quotient dans la division euclidienne a//b).

```
>>>a=36789; b=563
>>> a//b
65
>>> divmod(a,b)
(65, 194)
>>> a/b
65.34458259325045
```

```
>>> type(a/b)
<class 'float'>
>>> type(a//b)
<class 'int'>
```

4. Il s'agit de votre bon sens, pas de celui de la machine.

• Les complexes

Les complexes sont représentés par des couples de deux flottants à l'aide du constructeur **complex** avec la syntaxe `complex(x,y)` (x et y flottants) ou encore avec une constante de la forme $1+1j$. Les opérations usuelles sur les complexes sont évidemment **implémentées** : $+$, $-$, $*$, $/$, $**$, parties réelle, imaginaire, conjugué, module...

Le complexe i est noté **1j**, on définira $x + iy$ avec **w=complex(x,y)** ou **w=x+y*1j**.

Construction des complexes et opérations :

- on illustre les deux façons de construire un complexe ;
- on prendra garde aux différentes syntaxes des opérations : **abs(z)** comme une fonction, **z.conjugate()** comme une méthode, **z.real**, **z.imag** comme des champs (ou attributs) de classe. *Ce qui pour le moment paraît être un total désordre prendra tout son sens lorsque nous parlerons de programmation orientée objet.*
- Le module `numpy` propose des fonctions `numpy.real`, `numpy.imag`, `numpy.absolute`, `numpy.conjugate` vectorialisables (nous expliquons cela en section (1.5.2)).

```
>>> z = complex(1,1); z
(1+1j)
>>> w = 1j; w
1j
>>> 1j
1j
>>> 1*j
Traceback (most recent call last):
  File "<pyshell#5>", line 1, in <module>
    1*j
NameError: name 'j' is not defined
>>> z.real
1.0
>>> z.conjugate()
(1-1j)
>>> z*z.conjugate()
(2+0j)
>>> abs(z)
1.4142135623730951
```

1.3.2 Le type None

Python reconnaît un type et un objet **None**. Nous verrons cela page 45 avec la présentation des fonctions.

1.3.3 Le type booléen

Ce type permet d'effectuer les tests. Il comprend deux constantes **True** et **False**, les expressions booléennes sont donc les expressions logiques. Python transforme tout un tas d'objets en booléens : **nous éviterons d'en user**. On construit des expressions booléennes avec les **opérateurs de comparaison** `==`, `<`, `<=`, `!=`, les **connecteurs logiques** **and**, **or**, **not** (négation), **is** (égalité), **in** (appartenance à un conteneur)...

<pre>>>> A="Bonjour";B='Bonjour' >>> A is B True >>> A==B True >>> A!=B False >>> 'on' in A True >>> 'i' not in A True</pre>	<pre>>>>a=1234; b=5678 >>> a<b True >>> a<b or b<a True >>> a+b>b True >>> x=0; x!=0 and a/x>1 # seule la première clause est évaluée False</pre>
--	---

opérateur	évaluation
(P and Q)	P et Q sont des expressions booléennes ; si P est évaluée à True (ou vraie), Q est alors évaluée. Si Q est vraie, (P and Q) est vraie, fausse sinon ; si P est évaluée à False (ou fausse), Q n'est pas évaluée et (P and Q) est fausse.
(P or Q)	P et Q sont des expressions booléennes ; si P est évaluée à True (ou vraie), Q n'est pas évaluée et (P or Q) est vraie. si P est évaluée à False (ou fausse), Q est évaluée et (P or Q) est vraie ssi Q est vraie.
not P	P est une expression booléenne ; négation de P
(E ₁ == E ₂)	teste l'égalité lorsque E ₁ et E ₂ sont des expressions (numériques, booléennes ou autres)
(E ₁ is E ₂)	
(E ₁ != E ₂)	retourne la valeur inverse de (E ₁ is E ₂)
(E ₁ is not E ₂)	E ₁ et E ₂ sont des expressions (numériques, booléennes ou autres)

$(E_1 < E_2)$ $(E_1 \leq E_2)$ $(E_1 > E_2)$ $(E_1 \geq E_2)$	comparaisons (lève une erreur de type si les objets ne peuvent être convertis en objets comparables)
$(e \text{ in } S)$	teste l'appartenance de e à un conteneur ; retourne une erreur de type si S n'est pas un conteneur.

1.3.4 Les conteneurs

On appelle conteneur des objets qui sont susceptibles d'en contenir d'autres. Ce sont, dans le noyau du langage, les chaînes de caractères, les tuples, les listes, les ensembles et les dictionnaires. Ils acceptent tous la syntaxe `<elt> in <conteneur>` qui renvoie une expression booléenne et teste l'appartenance de `<elt>` au conteneur (caractère dans une liste, élément dans les tuples, listes et ensembles, clé dans un dictionnaire). Parmi eux, sont **indexables** les chaînes, tuples et listes, c'est-à-dire que l'on accède à leurs éléments par leur indice avec la syntaxe `X[i]`. Cette notation n'a pas de sens pour un ensemble dont les éléments ne sont pas ordonnées et pour un dictionnaire la notation `D[<clé>]` a un sens différent.

Les termes des objets indexables sont numérotés de 0 à longueur - 1

Les chaînes de caractères (string)

Une chaîne de caractères est une suite de caractères quelconques délimités par deux ' (apostrophes) ou deux " (guillemets) indifféremment. La chaîne vide est '' (ne pas confondre avec un espace ' ').

fonction, opérateur	effet
+	concaténation de deux chaînes de caractères
float()	convertit un nombre ou une chaîne en flottant
int()	convertit un nombre ou une chaîne en entier
str()	convertit un nombre (ou un objet quelconque pour l'affichage par exemple) en une chaîne
ch.count(s)	nombre de sous-chaînes égales à s dans ch
ch.replace(s1,s2)	renvoie une autre chaîne construite en remplaçant la sous-chaîne s1 par s2 dans ch
ch.split()	renvoie la liste des mots de ch séparés par un espace ' ' (par défaut)
ch.strip()	renvoie une autre chaîne construite en supprimant les espaces (et symboles de fin de ligne et tabulations) en début et fin de chaîne
s.join(L)	L : liste de chaînes, s : chaîne (str), renvoie la chaîne L[0]sL[1]s...sL[-1]

On peut déterminer la **longueur** d'une chaîne X (len(X)), en **extraire** des caractères (le premier caractère étant X[0], le dernier X[len(X)-1]); on peut **concaténer** deux chaînes ou plus, convertir une chaîne formée de chiffres en nombre... Un tel objet est de type **str** (pour string ou chaîne).

Chaînes de caractères : opérations et fonctions de base : concaténation, comparaison (is), longueur, indexage	Accès par tranches attention, on peut écrire ch[début : len(ch)], mais pas ch[len(ch)]
<pre>>>> A='Bonjour' >>> type(A) <class 'str'> >>> B="Bonjour" >>> A is B True >>> A+ " "+B 'Bonjour Bonjour' >>> len(A) 7 >>> A[0] 'B' >>> A[1] 'o' >>> A[0],A[1],A[2] ('B', 'o', 'n')</pre>	<pre>>>> A[0:len(A)] 'Bonjour' >>> A[7] Traceback (...): File "<pyshell#56>",line 1, in <module> A[7] IndexError: string index out of range >>> A[6] 'r' >>> int('2') 2 >>> int('23')+7 30 >>> float('23.1') 23.1</pre>

La conversion est par exemple indispensable lorsqu'on propose à l'utilisateur d'un programme d'entrer des nombres au clavier (ils sont lus comme des chaînes de caractères) :

```
x =int(input('x = '))
print(x+2)

x = 2
4
```

```
x =input('x = ')
print(x+2)
>>>
x = 2
Traceback bla bla bla
TypeError: Can't convert 'int' object to str implicitly
```

Tuples et listes

Tuples Regardez bien la dernière ligne : le découpage en tranches (slicing) objetindexable[debut :fin] retourne dans tous les cas les termes de <objet indexable> d'indices début à fin -1 .	Listes On prendra garde à la différence entre la concaténation (+) qui produit une autre liste (ailleurs en mémoire) et les méthodes insert, append, pop... qui modifient L (dont l'adresse mémoire est inchangée).
<pre>>>> t=(12); type(t) <class 'int'> >>> t=(12,); type(t) <class 'tuple'> >>> t=(1,(2,3),4,(5,6,7),8, 'fin') >>> t (1, (2, 3), 4, (5, 6, 7), 8, 'fin') >>> t[0] 1 >>> t[1] (2, 3) >>> len(t) 6 >>> t[6] Traceback ...bla bla bla IndexError: tuple index out of range</pre>	<pre>>>> L=[1,2,4,4] >>> L.append(0) >>> L [1, 2, 4, 4, 0] >>> L.insert(0,777); L [777, 1, 2, 4, 4, 0] >>> 7 in L False >>> 777 in L True >>> L+[567] [777, 1, 2, 4, 4, 0, 567] >>> L [777, 1, 2, 4, 4, 0] >>> L.append(567) >>> L [777, 1, 2, 4, 4, 0, 567] >>> L.pop(4);L 4 [777, 1, 2, 4, 0, 567]</pre>

<pre>>>> t[-1] 'fin' >>> t[-2] 8 >>> t[0:2] (1, (2, 3)) >>> t[0:len(t)] (1, (2, 3), 4, (5, 6, 7), 8, 'fin')</pre>	<pre>>>> L[-1] 567 >>> len(L) 6 >>> L[1:3] [1, 2]</pre>
---	--

Un **tuple** (en français **t-uplet**) : suite d'éléments quelconques (éventuellement d'autres tuples) séparés par une virgule et éventuellement encadrés par des parenthèses. Le tuple vide est défini par (), un singleton doit être déclaré avec une virgule (a,). On ne peut ni ajouter ni retrancher ni modifier les éléments d'un tuple (il est dit **immuable**). On accède aux éléments par indexation comme pour les chaînes et les listes.

Une **liste** est une suite d'éléments quelconques séparés par une virgule et encadrés par des crochets. La liste vide se note []. On peut ajouter en fin de liste, insérer des éléments dans une liste, c'est la différence avec les tuples : une liste est **modifiable** ou **muable**.

Avertissement

On prendra garde aux affectations de la forme **M = L** où *L* est une liste. Après une telle affectation les deux variables pointent vers le même objet en mémoire : une modification de *M* affecte donc *L*. **L'instruction $M = L$ doit donc être évitée sous peine de surprises** comme nous le montrons dans le tableau.

```
>>> L=list(range(0,10)); L
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> M = L; T=L[0:len(L)]
>>> M; T
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

>>> T[0]=1024; T; L
[1024, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```



```
>>> M[0]=1012;  M;L;T
[1012, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[1012, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[1024, 1, 2, 3, 4, 5, 6, 7, 8, 9]

>>> M=None
>>> L
[1012, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> M
>>>
```

Par contre, une affectation **T = L.copy()** ou encore **T = L[0 : len(L)]** crée une **autre** liste en mémoire vers laquelle pointe la variable T et une modification de T n'altère pas la liste L sauf si ses éléments sont eux mêmes des listes ou des conteneurs (auquel cas il vous faudra faire appel à la méthode **deepcopy()** du **module copy**).

Ensembles (set)

Exemples d'opérations sur les ensembles

```
>>> ens1={1,2,8,12,3}
>>> type(ens1)
<class 'set'>
>>> 1 in ens1; 90 in ens1
True
False
>>> ens1
{8, 1, 2, 3, 12}
>>> ens1.add(888); ens1
{1, 2, 3, 8, 12, 888}
>>> ens1.remove(888)
>>> ens1.union({19,32})
{32, 1, 2, 3, 19, 8, 12}
>>> ens1.intersection({1,9,32})
{1}
```

C'est la définition mathématique : collection d'objets non ordonnée, sans répétition. Un ensemble est construit en écrivant une séquence de termes séparés par des virgules

et encadrée par des accolades `{ }` ou bien par conversion à partir d'un objet indexable (tuple, liste, chaîne) grâce à la fonction `set`. Attention, l'ensemble vide est défini par `set()` (`{ }` désigne un dictionnaire vide).

Dictionnaires ou tableaux associatifs

Un dictionnaire est la donnée de couples (clé, valeur). La syntaxe est $\{k_1 : v_1, k_2 : v_2, \dots, k_p : v_p\}$. L'implémentation de l'association clé $\rightarrow$ valeur, qui est nécessairement une **application**, permet la recherche de l'élément associé à la clé en un temps $O(1)$ en moyenne, contre $O(\ln n)$ pour une recherche dans une liste triée et $O(n)$ dans une liste non triée. On étudiera cela plus en détails avec les algorithmes de recherche du chapitre 2 et dans la troisième partie de ce cours. Les vitesses des recherches dans une liste ou dans un dictionnaire sont illustrées page 110.

Exemples d'opérations sur les dictionnaires

```
>>> dic1={1:'a', 2:'b', 3: 'abc', 'x': 'xyz'}

>>> dic1.values()
dict_values(['a', 'b', 'abc', 'xyz'])

>>> dic1.keys()
dict_keys([1, 2, 3, 'x'])

>>> dic1['new']= 667; dic1
{1: 'a', 2: 'b', 3: 'abc', 'new': 667, 'x': 'xyz'}

>>> dic1.pop('x'); dic1
'xyz'
{1: 'a', 2: 'b', 3: 'abc', 'new': 667}
```

• **Objets itérables** ce sont les instances des structures que l'on peut **parcourir** : tuple, listes, chaînes, dictionnaires et ensembles sont itérables (et pas toujours indexables).

```
>>> for e in 'Bonjour': print(e)
B
o
.
.
.
u
r
```

```
>>> for e in {1,2,3}:print(e)
1
2
3
>>> dic1
{1: 'a', 2: 'b', 3: 'abc', 'new': 667}
>>> for e in dic1: print(e)
1
2
3
new
```

1.3.5 Opérations sur les listes, ensembles et dictionnaires

Sans autre forme de procès, quelques exemples d’opérations sur les listes. Certaines des **méthodes** (dont la syntaxe est <liste>.<méthode>()) modifient **s** elle même, ce sont celles qui renvoient **None**, les autres renvoient une liste distincte.

Opération (listes)	Résultat
s = []	liste vide
s[i] = x	l’élément d’indice <i>i</i> est remplacé par <i>x</i>
s[i :j] = t	la tranche des éléments d’indices <i>i</i> à <i>j</i> est remplacée par le contenu de l’itérable <i>t</i>
del s[i :j]	équivalent à s[i : j] = []
s.append(x)	ajoute l’élément <i>x</i> en fin de liste
s.extend(x)	<i>x</i> est un itérable traité comme une liste, concaténation
s.count(x)	nombre d’occurences de <i>x</i>
s.index(x[, i[, j]])	renvoie le plus petit <i>k</i> tq. s[k] == x et i <= k < j
s.insert(i, x)	équivalent à s[i :i] = [x]
s.pop([i])	équivalent à x = s[i] ; del s[i] ; return x
s.remove(x)	équivalent à del s[s.index(x)]
s.reverse()	renverse <i>s</i> en place
s.sort([key[, reverse]])	trie <i>s</i> en place
s.copy()	crée une copie de <i>s</i> (distincte en mémoire)

Opération (set)	Résultat
set()	permet de construire un ensemble : <code>s = set()</code> , l'ensemble vide, <code>s=set({1,2})...</code>
s.add()	ajoute un élément
s.remove()	enlève un élément
len(s)	cardinal de s .
x in s	test d'appartenance
x not in s	...
s.isdisjoint(other)	teste si l'intersection est vide
s.issubset(other)	set <= other teste l'inclusion
set < other	teste l'inclusion stricte
s.issuperset(other)	set >= other teste l'inclusion
set > other	teste l'inclusion stricte
s.union(other, ...)	renvoie un nouvel ensemble, la réunion de s et des arguments
s.intersection(other, ...)	renvoie un nouvel ensemble, l'intersection de s et des arguments
s.difference(other, ...)	renvoie un nouvel ensemble formé des éléments de s qui ne sont pas dans la réunion des arguments
s.copy()	renvoie un nouvel ensemble formé des éléments de s mais qui pourra être modifié indépendamment de s

Opération (dict)	Résultat
D = {'a' : 1, 'b' : 12}, D = {}	permet de définir des dictionnaires
D[key]	renvoie la valeur associée à la clé key, lève une exception si key n'est pas une clé de D
D[key] = value	permet de (re)définir la valeur associée à key (si D est déjà défini)
del D[key]	supprime l'association (key, valeur) dans D. Lève une exception si key n'est pas une clé de D
x in D, x not in D	teste si x est une clé du dictionnaire D
list(D), set(D)	renvoie une liste ou l'ensemble des clés de D
S.keys(), D.values()	itérateurs sur les clés ou les valeurs de D
D.items()	un itérateur sur les couples clé-valeurs
D.copy()	retourne un nouveau dictionnaire qui pourra être modifié indépendamment de D

1.3.6 Exercices

Ce sont là des questions brèves qui doivent vous permettre de vérifier que vous avez mémorisé et compris le fonctionnement des conteneurs. Que vous ayez choisi d'avancer avec méthode ou au contraire en sautant des étapes, il est conseillé d'y revenir après avoir écrit vos premiers programmes.

Exercice 1.2 *indexage*

1. On suppose que la variable `O` est affectée d'un objet indexable non vide de 4 termes ou plus : chaîne, liste, tuple. Que produisent les instructions suivantes :

```
>>> ell = len(O)
>>> O[0]; O[ell-1]
>>> O[ell]
>>> O[0 : ell]
>>> O[0 : 3]
>>> O[-1]
>>> O[ell-1]
>>> O[2 : 3]
>>> O[2 : 2]
```

Corrigé en 1.2, page 71

Exercice 1.3

Dans ce qui suit on suppose que l'on a réalisé l'affectation `L=[0,1,2,3,4]` et quelle est toujours effective.

1. **Vrai ou Faux ?** les instructions suivantes modifient la liste `L`.

```
>>> T = L
>>> T[0] = 12
```

2. **Vrai ou Faux ?** les instructions suivantes modifient la liste `L`.

```
>>> T[0:len(L)] = L
>>> T[0] = 12
```

3. **Vrai ou Faux ?** l'instruction suivante modifie la liste `L`.

```
>>> L.append(-6);
```

Corrigé en 1.3, page 71

1.4 La programmation et les fonctions avec Python

1.4.1 Blocs et indentation

Le mot *indentation* vient de la typographie où il désigne le retrait⁵ d'une ligne par rapport aux précédentes. En programmation il est constamment recommandé d'**indenter** son code de façon à le rendre lisible. La règle générale est la suivante : quand une instruction est composée et qu'elle prend plusieurs lignes, les sous-instructions d'un même niveau sont placées sur des lignes successives et débutent sur une même colonne et en retrait par rapport à l'instruction du niveau supérieur, ce que nous nous efforçons d'illustrer avec un algorithme de calcul du nombre d'occurrences du terme **a** dans une liste **L** :

```
L = [1,2,2,1,3,4,-2,5,3,2,1]
a = 2
c = 0
for e in L :
    if e == a :
        c=c+1
print ('c=',c)
```

Avec Python l'indentation fait office de délimiteur de fin de bloc. La suite d'instructions intérieure à la boucle **for** est en retrait et toutes les lignes y commencent sur une même colonne, la suite d'instructions du sous-bloc correspondant à l'instruction conditionnelle **if** subit une seconde indentation. Le retour à l'indentation précédente marque la fin d'un bloc. Les règles d'indentation en Python sont donc impératives, leur non respect provoque une erreur et entraîne un arrêt de l'exécution du script.

Piège à éviter

Les caractères de tabulation ne sont pas lus comme des espaces : évitons-les !⁶

1.4.2 Instructions conditionnelles

• **Syntaxe** : une instruction conditionnelle dans Python est de l'une des formes suivantes où, en allant de gauche à droite, on ajoute des instructions optionnelles :

5. Une ligne est en retrait si elle débute après un espace au moins situé sous le premier caractère de la précédente (ou au-dessus du premier caractère de la suivante).

6. Selon le système, ils seront lus comme 4 ou 8 espaces par défaut et seront transcrits différemment sur une autre machine. Imaginez ce qui se passera si vous les mélangez avec des espaces !

if condition 1 : {s. instructions 1}	if condition 1 : {s. instructions 1} else : {s. instructions 2}	if condition 1 : {s. instructions 1} elif condition 2 : {s. instructions 2} : elif condition n : {s. instructions n} else : {s. instructions n+1}
--	--	---

Une condition est une expression booléenne qui peut être évaluée comme vraie ou fausse (**True**, **False**), les instructions d'une {suite d'instructions} sont séparées par un retour à la ligne ou un ; Le passage à la ligne systématique est conseillé et l'indentation à respecter scrupuleusement.

• Sémantique

if seul : si la condition est évaluée à **True** la {suite d'instructions} est exécutée, sinon rien ;

if suivi de **else** : si la condition est évaluée à **True** la {suite d'instructions 1} est exécutée, sinon la {suite d'instructions 2} est exécutée ;

if suivie d'une ou plusieurs instructions **elif** : le programme sélectionne successivement les conditions suivant les mots **elif** et, dès qu'une évaluation retourne **True**, la suite des instructions correspondantes est exécutée et l'instruction termine là ; à défaut si **else** : est présente les instructions correspondantes sont exécutées ; dans le cas contraire aucune instruction n'est exécutée.

En conséquence : une des conditions au plus est évaluée à **True**, une seule suite d'instructions au plus est exécutée.

• Un exemple complet

Traisons une équation $ax^2 + bx + c = 0$ dans laquelle les coefficients a, b, c sont des réels (ce seront donc des flottants en machine) fournis par l'utilisateur. Observons que selon que a est nul ou pas il s'agit d'une équation de degré un ou de degré deux. Notre programme devra tenir compte de ces différents cas et sous-cas. C'est d'ailleurs dans un double dessein que nous présentons cet exemple car nous y trouvons à la fois :

- des instructions conditionnelles avec plusieurs clauses et imbriquées, ce qui est l'occasion d'illustrer notre propos du moment ;
- une situation dans laquelle nous savons par expérience que les cas non génériques⁷ sont allègrement oubliés ce qui, en maths, en physique comme en informatique est une cause constante d'erreurs.

7. On dira qu'un cas est générique lorsqu'il englobe toutes les situations sauf un certain nombre de cas particuliers plus rares. Dans le cas présent pour tout réel a sauf pour $a = 0$, l'équation est du second degré : c'est là le cas générique.

```
import numpy as np

a = float(input('coefficients a : '))
b = float(input('coefficients b : '))
c = float(input('coefficients c : '))

print('1\ ' équation  $a x^2 + b x + c = 0$ ')
delta = b**2 - 4*a*c

if a == 0 :
    print('n\ ' est pas une équation du second degré')
    if b != 0 :
        print('la solution est ', str(-c/b))
    elif c == 0 :
        print('a tous ses coefficients nuls, tout nombre est solution.')
    else :
        print('s\ ' écrit '+str(c)+'=0 et n\ ' a pas de solution.')
elif delta == 0 :
    print('est une équation du second degré de discriminant nul')
    print('sa seule solution est ' + str(-b/(2*a)))
elif delta > 0 :
    r = np.sqrt(delta)
    print('est une équation du second degré de discriminant strictement positif')
    print('ses solutions sont ' + str((-b-r)/(2*a)) + ' et ' + str((-b+r)/(2*a)) )
elif delta < 0 :
    r = np.sqrt(-delta)*1j
    print('est une équation du second degré de discriminant strictement négatif')
    print('ses solutions sont les complexes ' + str((-b-r)/(2*a)) + ' et ' + str((-b+r)/(2*a)) )
```

```
>>>
coefficients a :  0
coefficients b :  0
coefficients c :  1
ce n'est pas une équation du second degré
l'équation  1.0=0 n'a pas de solution.
```



```
>>>
coefficients a : 0
coefficients b : 2
coefficients c : 9
ce n'est pas une équation du second degré
la solution est -4.5

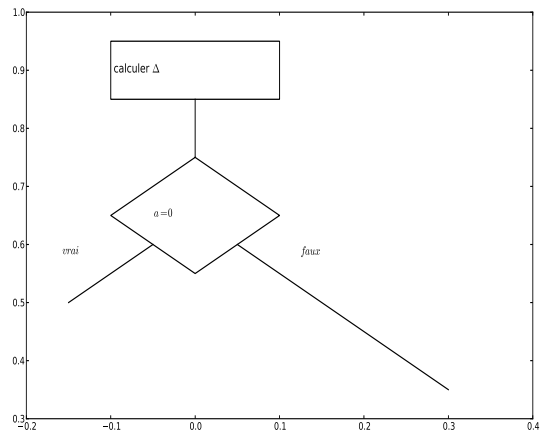
>>>
coefficients a : 1
coefficients b : -3
coefficients c : 2
l'équation  $a x^2 + b x + c = 0$ 
est une équation du second degré de discriminant
strictement positif
ses solutions sont 1.0 et 2.0

>>>
coefficients a : 1
coefficients b : 2
coefficients c : 1
l'équation  $a x^2 + b x + c = 0$ 
est une équation du second degré de discriminant nul
sa seule solution est -1.0

>>>
coefficients a : 1
coefficients b : 1
coefficients c : 1
l'équation  $a x^2 + b x + c = 0$ 
est une équation du second degré de discriminant
strictement négatif
ses solutions sont les complexes  $(-0.5 - 0.866025403784j)$ 
et  $(-0.5 + 0.866025403784j)$ 
```

Exercice 1.4 *diagramme associé au programme précédent*

1. Reprendre le programme précédent et en faire un **organigramme** en complétant la figure placée à la fin de l'exercice.
2. Vérifier que cet organigramme est un arbre dont les feuilles correspondent aux différents cas. Ont-ils tous été testés dans la console ?
3. Peut-on réécrire ce même programme avec une seule instruction conditionnelle ? À quoi ressemblerait alors l'organigramme associé ?



Corrigé 1.4 page 71

1.4.3 Parcours des objets itérables, boucles for

• **Syntaxe** : une boucle **for** en Python réalise une itération sur les éléments d’un conteneur qui est un objet **itérable** (**tuple**, **liste**, **numpy.array**, **range**), sa syntaxe utilise les mots réservés : **for**, **in** et ‘ : ’ :

for e in conteneur : {suite d’instructions}	for e in range (deb,fin,pas) : {suite d’instructions}
--	--

Pour une itération selon un compteur en progression arithmétique, on utilisera la fonction **range** qui retourne un objet qui simule la liste des entiers de la forme $deb + k * pas$ strictement plus petits que *fin* : *e* in range(0,10) permettant d’itérer de 0 à 9).

• **Sémantique** : l’itération parcourt les éléments du conteneur une fois et une seule dans l’ordre où ils apparaissent (rappelons que cet ordre n’est pas prévisible dans un ensemble).

Pour cela il est tout à fait déconseillé, quelque soit le langage de programmation, de modifier un itérateur dans la boucle : les effets sont difficiles à prévoir, la lecture du code malaisée et il y a toujours des solutions de bon sens pour procéder autrement...

- **Opérations standards avec une boucle for** (il va de soi que les a_k , u_0 , f ainsi que n doivent être incarnés pour une programmation effective !)

$\text{somme } \sum_{k=0}^n a_k$	$s = 0$ for k in $\text{range}(0, n+1)$: $s = s + a_k$
$\text{produit } \prod_{k=0}^n a_k$	$p = 1$ for k in $\text{range}(0, n+1)$: $p = p * a_k$
construction d'une liste $[a_1, \dots, a_n]$	$L = []$ for k in $\text{range}(0, n+1)$: $L.append(a_k)$
conjonction : $(a_1 \text{ and } a_2 \text{ and } \dots \text{ and } a_n)$	$v = \text{True}$ for k in $\text{range}(0, n+1)$: $v = v \text{ and } a_k$
disjonction : $(a_1 \text{ or } a_2 \text{ or } \dots \text{ or } a_n)$	$v = \text{False}$ for k in $\text{range}(0, n+1)$: $v = v \text{ or } a_k$
suite récurrente : $u_{n+1} = f(u_n)$	$u = u_0$ for k in $\text{range}(0, n+1)$: $u = f(u)$
récurrence d'ordre deux : $u_{n+2} = f(u_{n+1}, u_n)$	$u, v = u_0, u_1$ for k in $\text{range}(0, n+2)$: $w = v$ $v = f(u, v)$ $u = v$

- **Exemples** : nombre d'occurrences d'un terme dans une liste et la recherche des nombres premiers de 3 à 50 (50 exclu)

```
L=[ 8,12,78,23,4,-4,0,7,12]
a = 12
c = 0
for e in L :
    if e==a :
        c=c+1
print(a,' est présent ', c,' fois dans ', L)
```

```
12  est présent 2 fois
dans [8,12,78,...,0,7,12]
```

```
for n in range(3, 50, 2) :
    prem = True
    for x in range(2, n) :
        if n % x == 0 :
            print(n, '=', x, '*', n//x)
            prem = prem and False
    if prem :
        print(n, ' est premier ;')
```

```
3  est premier ;
5  est premier ;
7  est premier ;
9  = 3 * 3
...
15 = 5 * 3
17 est premier ;
```

- La fonction **enumerate**(*itérable*, *début*) permet le parcours d'un objet itérable en saisissant les couples (index, valeur) :

```
>>>L=[90,34,73,23,1]
>>> for e in enumerate(L):print(e)
(0, 90)
(1, 34)
(2, 73)
(3, 23)
(4, 1)
```

Exercice 1.5 en commençant simplement

Dans les questions qui suivent :

- utiliser **print** pour afficher des instructions ou réponses du programme ;
- utiliser **input** pour afficher une 'invite' à l'écran et récupérer une chaîne de caractères au clavier ;
- penser que les fonctions **str** , **int** ou **float** transforment respectivement un type numérique en chaîne, une chaîne en nombre entier ou flottant lorsque cela a du sens.

1. Écrire un script permettant le calcul de $p!$ pour $p \in \mathbb{N}$ donné par l'utilisateur (utiliser input).

On rappelle que la suite des nombres $p!$ est définie par récurrence, de la façon

$$\text{suivante : } \begin{cases} 0! &= 1 \\ (n+1)! &= n! \times (n+1) \end{cases}$$

par exemple $0! = 1$, $1! = 0! \times 1 = 1$, $2! = 1! \times 2 = 2$, $3! = 2! \times 3 = 6$, ..., $8! = 40320$.

2. Construire la liste des cubes des entiers de 0 à n pour $n \in \mathbb{N}$ donné l'utilisateur. Peut-on faire cela de la même façon avec un tuple ?

3. Écrire un script qui, pour $n \in \mathbb{N}^*$, $a > 0$ donnés par l'utilisateur affiche les n

$$\text{premiers termes de la suite récurrente définie par } \begin{cases} u_0 &= 1 \\ u_{n+1} &= \frac{1}{2} \left(u_n + \frac{a}{u_n} \right) \end{cases}$$

$$\text{puis les } n \text{ premiers termes de la suite définie par } \begin{cases} u_0 &= -1 \\ u_{n+1} &= \frac{1}{2} \left(u_n + \frac{a}{u_n} \right) \end{cases}.$$

On notera r_1 et r_2 les derniers termes calculés pour chacune de ces listes. Évaluer r_i^2 . Justifier simplement que dans le premier cas, tous les termes de la suite sont bien définis.

Corrigé 1.5 page 73

Exercice 1.6 récurrences doubles

$$\text{On considère la suite de Fibonacci définie par } \begin{cases} f_0 &= 0 \\ f_1 &= 1 \\ f_{n+2} &= f_{n+1} + f_n \end{cases}$$

Calculer et afficher ses termes d'indices 2 à n . Justifier que le programme affiche ce qui est demandé.

On commencera par programmer en s'interdisant d'écrire deux affectations simultanées. *Corrigé 1.6 page 74*

1.4.4 Continue, pass

L'instruction **continue** permet, dans une boucle, de sauter l'étape en cours (en passant à la suivante). Elle joue un rôle différent de **pass** qui :

- ne fait rien
- a vocation à se trouver n'importe où dans un programme et pas seulement dans une boucle.

```
for i in range (-4,4):  
    if i==-1 or i==1:  
        continue  
    else:  
        pass  
    print(i)
```

```
>>>  
-4  
-3  
-2  
0  
2  
3
```

Ces deux instructions sont utiles lors de l'élaboration de programmes complexes que l'on teste par parties en ayant déjà mis en place la structure (boucles, fonctions...).

1.4.5 Listes en compréhension

On dit que l'on définit un sous-ensemble de L en compréhension lorsqu'on le caractérise comme étant l'ensemble des éléments de L vérifiant une certaine relation. Par exemple, l'ensemble des éléments de $\mathbb{N}$ qui sont multiples de 3 : $\{n \in \mathbb{N} | 3|n\}$.

De façon analogue, on peut définir sous Python des sous-listes en compréhension : la syntaxe

```
[ x for x in L if C(x) ]
```

permet d'extraire de la liste L la sous-liste des termes qui vérifient la condition $C(x)$ comme ci-dessous les multiples de 3 ou les termes compris entre deux valeurs :

```
>>> L=list(range(0,31))  
>>> [x for x in L if x%3 ==0]  
[0, 3, 6, 9, 12, 15, 18, 21, 24, 27, 30]  
>>> [x for x in L if 13<= x <= 24]  
[13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24]
```

On peut aussi extraire une partie d'un ensemble pour constituer un sous-ensemble,

une liste, etc.

```
>>> S=set(range(0,101))
>>> {x for x in S if x<=31 and x%3==0}
{0, 3, 6, 9, 12, 15, 18, 21, 24, 27, 30}
>>> [x for x in S if 10 <=x <= 31 and x%3==0]
[12, 15, 18, 21, 24, 27, 30]
```

Exercice 1.7

1. Construire en compréhension la liste des cubes de 0 à n .
2. Construire en compréhension la liste des diviseurs de n .
3. On se donne une liste L et un terme x quelconque. Construire en compréhension la liste des indices k tels que $L[k]==x$.

Corrigé 1.7 page 75

Exercice 1.8 *programmation impérative ou fonctionnelle* ?

1. (a) Construire la liste des entiers de 0 à n de deux façons (avec le constructeur **list** et en compréhension).
(b) Une liste L étant donnée, construire en compréhension la liste $L1$ dont les termes sont ceux de L présents consécutivement m fois :

$$[\ell_0, \ell_1, \ell_2, \dots, \ell_{n-1}] \rightarrow [\ell_0, \dots, \ell_0, \ell_1, \dots, \ell_1, \dots, \ell_{n-1}, \dots, \ell_{n-1}]$$

- (c) Une liste L étant donnée, construire en compréhension la liste $L2$ concaténée de L m fois :

$$[\ell_0, \ell_1, \ell_2, \dots, \ell_{n-1}] \rightarrow [\ell_0, \ell_1, \ell_2, \dots, \ell_{n-1}, \dots, \ell_0, \ell_1, \ell_2, \dots, \ell_{n-1}]$$

- (d) Une liste d'entiers positifs ou nuls L étant donnée, construire en compréhension, une liste $L3$ obtenue en remplaçant chaque terme x de L par x occurrences consécutives de x .
2. Reprendre les constructions précédentes, mais en **programmation impérative** avec la méthode **L.append(...)**.

Corrigé 1.8 page 75

1.4.6 Boucle while

- **Syntaxe** : elle est particulièrement simple (comme dans tous les langages) :

```
while condition :
    {suite d'instructions}
```

où la condition est une expression booléenne de la forme $C(x_1, x_2, \dots, x_n)$ où les $(x_i)_i$ sont des variables présentes dans le programme ; elle est donc susceptible d’être modifiée par la {suite d’instructions} qui constitue le **corps** de la boucle.

• **Sémantique** : la condition qui figure après le mot clé **while** est évaluée en entrée de boucle :

- si elle prend la valeur **False** (ou faux), le programme sort de la boucle et passe à l’instruction suivante (s’il y en a une) ; la {suite d’instructions} est donc ignorée ;
- si elle prend la valeur **True** (ou **vrai**), la {suite d’instructions} est exécutée et le programme retourne en début de boucle (où la condition est à nouveau évaluée).

Théorème 1.1 *condition d’arrêt dans une boucle while*

*Considérons une suite d’instructions contenant une boucle **while** de condition*

$C(x_1, x_2, \dots, x_n)$.

- *À la sortie de la boucle, la condition est toujours fausse (la condition d’arrêt est la négation de $C(x_1, x_2, \dots, x_n)$).*
- *Si $C(x_1, x_2, \dots, x_n)$ est vraie à l’entrée de la boucle, et si les instructions qui suivent ne changent pas sa valeur, alors **la boucle ne termine pas** (on dit que le **programme boucle indéfiniment**).*

Avec beaucoup d’à propos, signalons que :

pour **interrompre** un programme on fait : Ctrl-C (windows, Linux, et Mac OS X)

• Premier exemple

On présente ici l’algorithme de division euclidienne des entiers naturels tel qu’il est programmé dans les processeurs, c’est-à-dire sans utiliser la multiplication qui est une opération coûteuse⁸. Pour bien comprendre comment cela fonctionne, vous ferez vous même la division comme elle est décrite ici, avec un papier, un crayon.

Pour $a, b \in \mathbb{N}$, $b \neq 0$ donnés, on veut calculer l’unique couple d’entiers q et r tels que $a = bq + r$ avec $0 \leq r < b$.

- si $a > b$, on retranche b , on note r le résultat et tant que $r \geq b$ on recommence ;
- lorsque $r < b$ nous avons obtenu le reste de la division euclidienne de a par b et le quotient est le nombre d’itérations.

Ainsi, avec $a = 230$ et $b = 13$, les soustractions successives donnent en partant de 230 :

230	217	204	191	178	165	152	139	126
113	100	87	74	61	48	35	22	9

8. C’est ainsi que sont tout d’abord présentées les divisions aux enfants de l’école primaire.


```

a = int (input('a='))
b = int (input('b='))

if b > 0 :
    r = a
    q = 0
    while r >= b :
        r = r-b
        q = q+1
    print('q=', q, 'r=', r)

```

Nous reviendrons sur cet algorithme pour l'analyser plus en détails au début du chapitre 2.

• Deuxième exemple

À la recherche de la plus petite puissance entière de 10, distincte de 0.

```

k, p = 0, 1
while p > 0 :
    print('k =', k, 'et 10**(-'+str(k)+'') = ', p)
    k = k+1
    p = 10**(-k)

```

```

>>>
k = 0 et 10**(-0) = 1
k = 1 et 10**(-1) = 0.1
k = 2 et 10**(-2) = 0.01
k = 3 et 10**(-3) = 0.001
...
k = 322 et 10**(-322) = 1e-322
k = 323 et 10**(-323) = 1e-323

```

Dans cet exemple, nous initialisons les variables k et p à 0 et $10^0 = 1$, puis tant que p est **strictement positif** affichons k, p à l'écran, incrémentons k d'une unité, faisons $p = 10^{-k}$. Sur le plan mathématique, aucune des puissances de p n'est nulle et en précision infinie (représentation exacte des réels) un tel programme ne terminerait pas. En flottants il existe une valeur de k pour laquelle p sera identifié à 0.

Vous avez sous les yeux ce que ce script donne sur une de nos machines (copie partielle). Et sur les vôtres ?

Exercice 1.9 exemples simples

1. Soit n un entier supérieur à 1. Écrire un programme qui détermine le seul entier p , positif ou nul tel que $2^p \leq n < 2^{p+1}$. Encadrer le nombre d'itérations en fonction de $\ln n$.

2. Un critère de divisibilité par 7 s'énonce de la façon suivante :

Soit $m \in \mathbb{N}$, que l'on suppose écrit en base 10, on enlève le chiffre des unités que l'on retranche deux fois au nombre ainsi obtenu. Le nombre m' final est divisible par 7 si et seulement si m l'est.

Exemple :

$31976 \rightarrow 3197 - 2 \times 6 = 3185 \rightarrow 318 - 2 \times 5 = 308 \rightarrow 30 - 2 \times 8 = 14$

Ils sont donc tous divisibles par 7.

(a) Programmer le passage de m à m' (penser à utiliser **divmod**).

(b) Écrire un programme qui demande m à l'utilisateur et qui, à partir de m répète l'opération ainsi décrite tant que le nombre obtenu est supérieur à 7. On fera afficher la suite des nombres ainsi calculés.

Corrigé en 1.9 page 76.

Exercice 1.10 un petit jeu à programmer

```
Ce jeu consiste à deviner un nombre entre 0 et 1000.
Pour abandonner taper 1001
essai n° 1 : 500
trop petit
essai n° 2 : 750
trop petit
essai n° 3 : 825
trop petit
essai n° 4 : 912
trop petit
...
essai n° 9 : 931
trop grand
essai n° 10 : 930
Vous avez réussi en 10 essais
>>>
```

On se propose de réaliser un petit programme qui implémente le jeu présenté dans le tableau qui précède. Les nombres sont introduits par le joueur, le reste étant fait par le script :

1. On veut que le nombre inconnu soit déterminé au hasard entre 0 et 1000. Écrire dans le shell les lignes de commandes permettant de réaliser un tel tirage.
Pour cela utiliser la fonction **randrange** du module **random** (hasard).
2. Écrivez votre script sans autres indications que celles-ci :
 - utiliser **print** pour afficher des instructions ou réponses du programme ;
 - utiliser **input** pour afficher une 'invite' à l'écran et récupérer une chaîne de caractères au clavier ;
 - penser que les fonctions **str** et **int** transforment respectivement un type numérique en chaîne, une chaîne en nombre entier lorsque cela a du sens.
3. Avec indications : diviser votre programme en trois parties :
 - (a) Tirer un nombre inconnu au hasard ; demander un premier essai.
 - (b) Une boucle dont la condition d'arrêt est que le joueur abandonne ou a trouvé la bonne valeur et dont le corps permet d'afficher la réponse.
 - (c) Interpréter la sortie de boucle pour conclure (abandon ou réussite ?).

Corrigé en 1.10, page 76.

1.4.7 Break ou pas break ?

• L'instruction **break** a pour effet d'interrompre la boucle (**for** ou **while**) dans laquelle elle se situe immédiatement. Ce faisant elle enfreint les règles d'une bonne programmation : **son usage est donc fortement déconseillé !**

Exemple : on reprend l'algorithme de recherche des nombres premiers :

- à gauche on teste la divisibilité de chaque n entre 3 et 300 (exclu) par pas de 2 ;
- à droite on interrompt (instruction **break**) la boucle **for** du deuxième niveau dès que l'on rencontre un diviseur de n dans l'instruction conditionnelle ce qui évite des tests inutiles puisqu'on sait que n n'est pas premier.

```
for n in range(3, 300, 2) :
    prem = True
    for x in range(2, n) :
        if n % x == 0 :
            print(n, '=', x, '*', n/x)
            prem = prem and False
    if prem :
        print(n, ' est premier ;')
```

```
for n in range(3, 300, 2) :
    prem = True
    for x in range(2, n) :
        if n % x == 0 :
            print(n, '=', x, '*', n/x)
            prem = False
            break
    if prem :
        print(n, ' est premier ;')
```

<pre>3 est premier ; 5 est premier ; 7 est premier ; 9 = 3 * 3 11 est premier ; 13 est premier ; 15 = 3 * 5 15 = 5 * 3 17 est premier ; : 297 = 3 * 99 297 = 9 * 33 297 = 11 * 27 297 = 27 * 11 297 = 33 * 9 297 = 99 * 3</pre>	<pre>3 est premier ; 5 est premier ; 7 est premier ; 9 = 3 * 3 11 est premier ; 13 est premier ; 15 = 3 * 5 17 est premier ; : 293 est premier ; 295 = 5 * 59 297 = 3 * 99 299 = 13 * 23</pre>
---	--

Exercice : On peut faire tout aussi bien avec une boucle **while** : just do it !

1.4.8 Procédures et fonctions

Nous définirons nos premiers exemples de fonctions dans un script. L'exécution du script charge la fonction dans le shell où elle peut-être utilisée comme illustré ci-dessous. Bien entendu, un script peut contenir à la fois des définitions de fonctions et des appels de fonctions, même si pour des raisons de modularité il sera préférable de séparer, lorsqu'on écrira de plus gros programmes, les définitions des fonctions des scripts les appelant.

• **Premier exemple : factorielles, itérative et récursive**

<pre>def mafact(n) : p = 1 for x in range(1,n+1) : p = p*x return p</pre>	<pre>def mafact(n) : if p == 0 : return 1 else : return p*mafact(p-1)</pre>
---	---

Question : que se passe-t-il à gauche lorsque $p = 0$?

• **Deuxième exemple : calcul d'une moyenne**

La fonction moyenne prend un conteneur itérable à termes numériques en argument et retourne la moyenne de ses termes s'il est non vide, **None** sinon. Les variables n et s sont initialisées à 0 et lors du parcours des éléments de L , n sert de compteur alors que

s contient la somme des éléments déjà rencontrés. Une fois la fonction « moyenne » définie, nous pouvons l'appeler comme toute fonction du noyau de Python.

```
def moyenne(L):
```

```
    n = 0
```

```
    s = 0
```

```
    for x in L:
```

```
        n = n + 1
```

```
        s = s + x
```

```
    if n != 0:
```

```
        return s/n
```

```
    else:
```

```
        return None
```

```
>>> moyenne([1, 2, 3, 4, 5])
```

```
3.0
```

```
>>> moyenne((0, 1, 2, 3, 4, 5))
```

```
2.5
```

```
>>> moyenne([])
```

```
>>> type(moyenne([]))
```

```
<class 'NoneType'>
```

```
>>> moyenne([1j, 2j, 3j])
```

```
2j
```

```
>>> moyenne(range(0, 11))
```

```
5.0
```

```
>>> moyenne({0, 1, 2, 3})
```

```
1.5
```

Lors d'un appel avec un **argument effectif** L qui n'est pas itérable ou dont les termes ne sont pas d'un type numérique, on doit s'attendre à un message d'erreur (boucle **for** ou addition à 0). C'est ce que nous illustrons ici :

```
>>> moyenne(2)
```

```
Traceback ... line 5, in moyenne
```

```
    for x in L:
```

```
TypeError: 'int' object is not iterable
```

```
>>> moyenne((2,))
```

```
2.0
```

```
>>> moyenne(['a', 'b', 'c'])
```

```
Traceback ... line 7, in moyenne
```

```
    s = s + x
```

```
TypeError: unsupported operand type(s) for +:
'int' and 'str'
```

• Variables locales, variables globales

On dit des deux variables n et s présentes dans la fonction `moyenne` qu'elles sont **locales** car elles ne sont pas visibles à l'extérieur de la fonction comme on le voit ci-dessous : un appel qui retourne la moyenne attendue, est aussitôt suivi de l'échec de l'évaluation de n et de s . Ces deux variables n'ayant été définies ni dans le script (à l'extérieur de `moyenne`), ni dans le shell sont inconnues au niveau **global**.⁹

9. Nous ne parlerons pas ici de la durée de vie d'une variable locale.

```
>>> moyenne([1j,2j,3j])
2j
>>> n,x
Traceback (most recent call last):
  File "<pyshell#9>", line 1, in <module>
    n,x
NameError: name 'n' is not defined
```

Une fonction peut aussi faire appel à des **variables globales**, bien que pour des raisons de sûreté du code, cela devrait être au maximum évité. Ces variables sont visibles en dehors de la fonction elle-même et doivent être explicitement déclarées comme telles dans l'en-tête de la fonction. Dans le petit exemple qui suit, chaque appel de la fonction `f` incrémente la variable globale `c` (qui sert donc de compteur des appels à condition d'être initialisée avant le premier appel de notre fonction) et remet `d` à 0 (ce qui ne sert à rien d'autre qu'à vous montrer ce qui se passe) :

```
def f() :
    global c, d
    c = c+1
    d=0
```

On observera au passage que le premier appel sans initialisation préalable de `c` au niveau global conduit à une erreur prévisible. Cela est corrigé pour le deuxième appel.

```
>>> f()
Traceback ... line 17, in f
  c=c+1
NameError: global name 'c' is not defined
>>> c=0;f()
>>> c, d
(1, 0)
>>> d = 123; f()
>>> c, d
(2, 0)
```

• Syntaxe `def`, `return`, `global`

La définition d'une fonction a la forme suivante :

```
def <nom de la fonction> (arg1, ..., argn) :
    instruction1
    ⋮
    instructionp
```

Le nombre d'arguments est un entier naturel quelconque (y compris 0 comme dans l'exemple qui précède), contrairement aux variables locales, les variables globales, si la fonction en utilise, doivent être déclarées :

```
def <nom de la fonction> (arg1, ..., argn) :
    global var1, ..., varm
    instruction1
    ⋮
    instructionp
```

• Paramètres (ou arguments) formels, paramètres effectifs

Définir une fonction n'a pas d'autre effet que d'inscrire sa définition quelque part en mémoire. Seul un appel de la fonction avec des paramètres (ou arguments) effectifs en place des paramètres (ou arguments) formels déclenche la suite des instructions 1 à p . L'instruction **return** suivie d'une expression provoque l'arrêt de la fonction et le retour de l'expression à l'endroit même où la fonction a été appelée (par un script, une autre fonction, etc.). Son emploi est facultatif, en cas d'absence la fonction retourne **None** comme on le vérifie ici :

```
>>> type(f())
<class 'NoneType'>
```

• Affectation à des paramètres formels

Les affectations suivantes lorsque x est une variable formelle sont autorisées. À gauche et à droite les comportements sont différents selon la nature du paramètre. La seule chose à retenir, c'est que l'on ne doit pas programmer comme cela sauf si on veut modifier une liste ou un tableau **en place**, c'est-à-dire sans faire une copie en mémoire.

<pre> def biz(x) : x = 33 + x return x >>> a=1000 >>> biz(a) 1033 >>> a 1000 </pre>	<pre> def bizz(L) : L[0] =33 return L >>> M=[0,1,2,3] >>> bizz(M) [33, 1, 2, 3] >>> M [33, 1, 2, 3] </pre>
---	--

On définit ici deux **fonctions pures** qui prennent une liste en argument et renvoient **une autre liste**, sans modifier l'argument effectif :

<pre> def bizz1(L) : L1 = L.copy() L1[0] =33 return L1 >>> M = [0,1,2,3] >>> bizz1(M) [33, 1, 2, 3] >>> M [0, 1, 2, 3] </pre>	<pre> def bizz1(L) : L1 = L[0:] L1[0] =33 return L1 >>> M = [0,1,2,3] >>> bizz1(M) [33, 1, 2, 3] >>> M [0, 1, 2, 3] </pre>
---	--

• Fonctions lambda

On peut définir rapidement une fonction à partir d'une expression :

```

>>> f=lambda a,b,c :a+10*b+10**2*c
>>> f(1,2,3)
321

```

• Distinction entre fonctions et procédures

Une fonction « pure » prend des arguments et retourne une valeur. Une fois son appel terminé rien n'est modifié (rappelons qu'une affectation $var = f(arg_1, \dots, arg_n)$, qui modifie var , a lieu en dehors de l'exécution de la fonction f). Ce sont les seuls exemples que nous avons développés jusqu'ici et c'est la bonne façon de programmer. Une procédure « pure » quant à elle, modifie quelque chose dans l'environnement (affichage à l'écran, écriture dans un fichier, écriture en mémoire qui perdure une fois la procédure terminée comme une affectation de variable globale, l'ouverture d'une fenêtre graphique, une commande de robot...) et ne retourne pas d'autre valeur que **None**.

On peut encore mixer les deux avec un procédure qui retourne aussi des valeurs pour indiquer que l'opération à réaliser a réussi ou échoué par exemple : ouverture de fichier, communication avec un périphérique, un autre ordinateur, etc. Cela est illustré une première fois dans l'exercice 1.22 avec des procédures graphiques.

1.4.9 Compléments : sous-procédures et visibilité des variables

On peut définir des fonctions ou procédures (de niveau 2) à l'intérieur d'une fonction ou procédure (de niveau 1). Dans ce cas :

- la sous-procédure (de niveau 2) n'est pas visible en dehors de la procédure de niveau 1 ;
- les variables locales (c'est l'option par défaut en Python) de la sous-procédure ne sont pas visibles de l'extérieur : ni de la procédure de niveau 1 et encore moins du niveau global ;
- l'option **nonlocal** dans la procédure de niveau 2 permet de rendre une variable visible dans la procédure de niveau 1 ; elle se différencie de l'option global puisque cette variable n'est toujours pas visible de l'extérieur de la procédure de niveau 1 ;
- les variables globales restent visibles de partout.

```
def niveau1():
    x= 1
    def niveau2() :
        x=2
        print('niveau2, x=:',x)
        return None

    niveau2()
    #appel sous-procédure
    print('niveau1, x=:', x)
    return None

>>> niveau1()
niveau 2, x= : 2
niveau 1, x= : 1
>>> niveau2()
Traceback ... NameError:
name 'niveau2' is not defined
```

```
def niveau1():
    x= 1
    def niveau2() :
        nonlocal x
        x=2
        print('niveau2, x=:', x)
        return None

    niveau2()
    #appel sous-procédure
    print('niveau1, x=:', x)
    return None

>>> niveau1()
niveau 2, x= : 2
niveau 1, x= : 2
>>> x
Traceback...NameError:
name 'x' is not defined
```

1.4.10 Exercices

Exercice 1.11 fonctions et boucles

1. Écrire une fonction qui prend en argument un entier $n > 0$ et retourne (ou renvoie) le seul entier positif ou nul tel que $2^p \leq n < 2^{p+1}$. Expliquer en quoi votre fonction retourne le bon résultat. Encadrer le nombre d'itérations en fonction de $\ln n$.
2. Écrire une fonction qui prend en argument un entier $n \geq 0$ et retourne le seul entier positif ou nul k tel que $\frac{k(k+1)}{2} \leq n < \frac{(k+1)(k+2)}{2}$. Encadrer le nombre d'itérations en fonction de n .

Corrigé en 1.11 page 78

Exercice 1.12 variantes sur les listes

Dans les questions qui suivent la règle du jeu est la suivante :

1. On commence par écrire la fonction demandée avec, comme seules opérations autorisées sur les listes, la création ($L=[0,1,2,1,0]$ ou **list(range(0,n+1))**), l'extraction : $L[i]$, $L[i:j]$ ou $L[i:j:p]$, et l'itération suivant les termes de la liste (**for x in L : instruction**). Comme dans les exemples présentés.

```
>>> L=list(range(0,11)); L
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> L=[0,1,2,3,4,5]
>>> L[0:len(L):2]
[0, 2, 4]
>>> M=L[0:len(L)]
>>> M[0]=17;M;L
[17, 1, 2, 3, 4, 5]
[0, 1, 2, 3, 4, 5]
>>> for x in L: instruction
```

2. On pourra ensuite améliorer la programmation en faisant appel à **L.append(x)**, **len(L)** (et à rien d'autre).
3. Pour chaque fonction programmée, retrouver la méthode du tableau de la page 28 qui réalise la même opération.
4. Les *, **, ***, correspondent à des niveaux de difficulté supplémentaires.

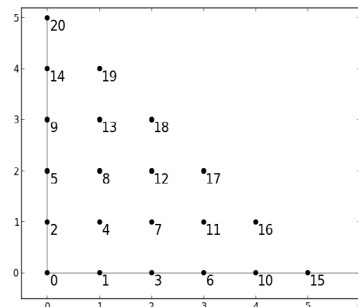
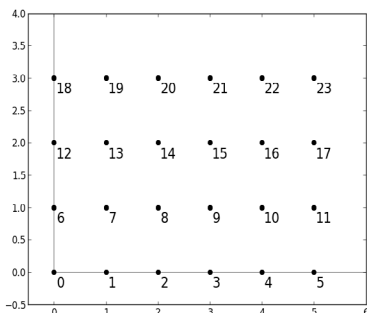
Conseil : on réfléchit avant de programmer et on dispose une feuille et un crayon devant le clavier.

1. Écrire une fonction **longueur(L)** qui prend en arguments une liste L et retourne le nombre de ses termes (0 si elle est vide) ; il s'agit en quelque sorte de reprogrammer **len** que l'on pourra utiliser à bon escient par la suite.

2. Écrire une fonction **appartient**(L, x) qui prend en arguments une liste L et un objet x et retourne **True** ou **False** selon que L contient x ou pas.
3. Écrire une fonction **nb_occurrences**(L, x) qui prend en arguments une liste L et un objet x et retourne le nombre d'occurrences de x dans L .
4. Écrire une fonction **liste_places**(L, x) qui prend en arguments une liste L et un objet x et retourne la liste éventuellement vide des indices des positions de x dans L .
** Essayer de l'écrire en **une ligne**.
5. Écrire une fonction **maximum**(L) qui prend en argument une liste L et retourne le couple formé du plus grand élément de L suivi de la première position dans laquelle il figure.
* Essayer de l'écrire en **une ligne**.
6. Écrire une fonction **places_maximum**(L) qui prend en argument une liste L et retourne le couple formé du plus grand élément de L suivi de la **liste** des positions où il figure.
7. Après avoir relu l'avertissement de la page 25, écrire une fonction ou une procédure **plus_grand_devant**(L) qui prend en argument une liste L et retourne la liste L' obtenue en permutant le plus grand élément de L avec son premier terme,
 - (a) en ne modifiant pas la liste vers laquelle pointe L ;
 - (b) en modifiant la liste vers laquelle pointe L .
8. Écrire une fonction ou une procédure **supprimer**(L, ell) qui prend en arguments deux listes L et ell (qui contient des entiers) et retourne la liste L' obtenue à partir de L en enlevant les termes dont les indices figurent dans ell :
 - (a) en ne modifiant pas la liste vers laquelle pointe L ;
 - (b) *** en modifiant la liste vers laquelle pointe L en commençant par le cas où ell est triée en ordre décroissant (pourquoi ?). Penser à $L[k; k+1] = []$.

Corrigé en 1.12, page 79

Exercice 1.13 numérotations des couples d'entiers...



1. On range $m \times n$ couples d'entiers comme sur la figure de gauche.
 - (a) Écrire une fonction `phi(m, i, j)` qui donne le n° du couple (i, j) .
 - (b) Écrire la fonction `inv_phi(m, k)` qui donne le couple (i, j) de numéro k .
2. On se propose maintenant de programmer la numérotation des couples de $\mathbb{N}^2$ comme sur la figure de droite.
 - (a) Écrire une fonction `psi(i, j)` qui donne le n° du couple (i, j) .
 - (b) Écrire la fonction `inv_psi(k)` qui donne le couple (i, j) de numéro k .

Indications : regarder où se trouvent les couples pour lesquels $i + j = k$ et les couples de n° de la forme $k(k + 1)/2 \dots$

Corrigé en 1.13 page 82.

Exercice 1.14 encore les listes

1. Écrire une fonction `petitsPains(L, m)`¹⁰ qui prend une liste L et un entier $m \geq 1$ en arguments et retourne une nouvelle liste dans laquelle chaque élément de L est remplacé par m clones :

$$[\ell_0, \ell_1, \ell_2, \dots, \ell_{n-1}] \rightarrow [\ell_0, \dots, \ell_0, \ell_1, \dots, \ell_1, \dots, \ell_{n-1}, \dots, \ell_{n-1}].$$

Utiliser la méthode `L.append(...)` comme dans le tableau de la page 36.

2. Réécrire cette même fonction comme une fonction lambda avec des listes en compréhension.

Corrigé en 1.14, page 82

Exercice 1.15

1. Que fait la fonction suivante :


```
lambda L, x: [k for k in range(0, len(L)) if L[k]==x]
```
2. Que fait le script suivant :

```
def NewtonFormel(f, df):
    F = lambda x: x-f(x)/df(x)
    return F
g      = lambda x: x**3-3
dg     = lambda x: 3*x**2
U      = NewtonFormel(g, dg)
```

Corrigé 1.15 page 83.

10. La même affection pour la mythologie du Silmarillion.

1.5 Modules ou bibliothèques

Si au moment où nous écrivons ces lignes, Python est un des langages informatiques les plus utilisés, c'est en grande partie dû, au-delà de ses qualités intrinsèques, à l'abondance et à la qualité de ses modules.

Un **module** (ou bibliothèque logicielle) est un ensemble de programmes dédiés à des applications particulières que l'on peut **importer** pour enrichir le **cœur du langage** (ce qui est à votre disposition lorsque vous ouvrez le programme Python). Certains modules sont livrés avec la **bibliothèque standard**, d'autres devront être installés par vos soins (sur votre machine).

1.5.1 L'instruction import

Le tableau qui suit montre comment fonctionne une importation.

```
>>> sin(0)
... NameError: name 'sin' is not defined

>>> import numpy
>>> sin(0)
... NameError: name 'sin' is not defined
>>> numpy.sin(numpy.pi)
1.2246467991473532e-16
```

La fonction sinus et la constante π ne sont pas définies dans le cœur du langage. Elles sont toutefois implémentées dans la bibliothèque **numpy** que l'on charge avec **import numpy**. En important **numpy** on a accès à ces deux objets que l'on appelle avec **numpy.sin**, **numpy.pi**. Cette syntaxe, avec le préfixe **numpy**, est obligatoire comme on le voit avec la deuxième tentative d'appel, **sin(0)**, postérieure à l'importation de **numpy** et qui échoue quand même.

On peut s'affranchir de cette contrainte en appelant explicitement certains objets du module comme illustré ci-dessous.

```
>>> from numpy import sin, pi
>>> sin(pi)
1.2246467991473532e-16
```

Si on veut écrire facilement **toutes** les fonctions de **numpy**, on peut les appeler avec **from numpy import ***. Mais, attention, l'usage de **from ... import ...** et a fortiori de **from ... import *** vous fera perdre les avantages des **espaces de noms** et une

fonction déjà installée pourrait se trouver remplacée à votre insu par une autre, ce qui peut provoquer des erreurs. Nous ne l'utiliserons pas.

Il est par contre possible de renommer un module comme ci-dessous ce qui permet de simplifier l'écriture.

```
>>> import numpy as np
>>> np.sin(np.pi)
1.2246467991473532e-16
```

Nous présenterons dans ce cours quelques éléments des modules numpy, scipy indispensables pour le calcul numérique et donnerons un aperçu de certains autres modules. Vous devez savoir que le module numpy est appelé par tous les modules qui gèrent du calcul scientifique (scipy, scikitlearn, sympy, etc.)¹¹

Dans la liste qui suit, random, sqlite3, sont dans la bibliothèque standard. Vous devrez installer les autres modules si ce n'est déjà fait.

Module	Aperçus
random	générateurs de variables aléatoires
numpy	les objets du calcul scientifique, page 56 et chapitre 7
scipy	le calcul numérique, vous l'utiliserez dans les autres disciplines
matplotlib	les tracés graphiques, page 60
matplotlib.image	la gestion des images, page 234
sympy	le calcul formel avec Python (utilisé dans SageMaths)
scikitlearn	les algorithmes de l'apprentissage statistique et de l'intelligence artificielle ; chapitre 15
sqlite3	gestion d'une base de données sqlite avec Python

11. Citons la documentation : *NumPy is the fundamental package for scientific computing in Python. It is a Python library that provides a multidimensional array object, various derived objects (such as masked arrays and matrices), and an assortment of routines for fast operations on arrays, including mathematical, logical, shape manipulation, sorting, selecting, I/O, discrete Fourier transforms, basic linear algebra, basic statistical operations, random simulation and much more.*

1.5.2 Tableaux (array) de la bibliothèque numpy

Tous les langages de programmation disposent d'un type **tableau (array)** implémenté de telle sorte que l'occupation de la mémoire soit minimale et que l'accès à un élément à partir de son indice se fasse à temps moyen constant. En contrepartie, la taille du tableau est imposée et les éléments sont de même type (et occupent donc la même place en mémoire). Cela différencie les tableaux des listes, ensembles et autres dictionnaires. Ces différences dans l'implémentation sont de premier abord en boîte noire pour le programmeur. Mais celui-ci devra apprendre à choisir les types de données qu'il utilisera pour optimiser le coût des opérations qu'il sera amené à exécuter.

Le type tableau (array) ne fait pas partie du **cœur** du langage Python. Bien que des tableaux soient définis dans le **module array** de la bibliothèque standard (famille de modules qui sont installés avec Python) nous travaillerons avec les tableaux de la **bibliothèque numpy** que nous devrons donc commencer par importer.

Premières constructions de tableaux : on peut construire des tableaux à partir de conteneurs standards de Python ; les objets sont convertis en un type commun lorsque cela est possible (ce qui est impératif dans un tableau).

```
>>> import numpy
>>> a=numpy.array([1,2,7])
>>> a
array([1, 2, 7])
>>> subdivision = numpy.linspace(0,1,12)
>>> subdivision
array([ 0.          ,  0.09090909,  0.18181818,
        0.27272727,  0.36363636,  0.45454545,
        0.54545455,  0.63636364,  0.72727273,
        0.81818182,  0.90909091,  1.          ])
>>> a =array([1,2,'er'])
>>> a
array(['1', '2', 'er'], dtype='<U2')
>>> a[0]+a[2]
'1er'
>>> type(_)
<class 'str'>
>>> a=numpy.array([1,1j,2-0.1j])
>>> a
array([ 1.+0.j ,  0.+1.j ,  2.-0.1j])

>>> a=numpy.arange(1,12)
>>> a
array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11])
```

```

>>> a=numpy.zeros(12)
>>> a
array([0., 0., 0., 0., 0., 0., 0., 0.,
       0., 0., 0., 0.])
>>> a=numpy.ones((3,3));a
array([[1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.]])
>>> a=numpy.ones((3,3));a
array([[1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.]])
>>> a=numpy.arange(10)
>>> a*2
array([0, 1, 4, 9, 16, 25, 36, 49, 64, 81], dtype=int32)
>>> b=numpy.arange(10);a=numpy.arange(0,100,10)
>>> a,b
(array([0, 10, 20, 30, 40, 50, 60, 70, 80, 90]),
 array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]))
>>> len(a)
10
>>> a[0:len(a):2]
array([0, 20, 40, 60, 80])
>>> a[3]=12345666; a
array([0, 10, 20, 12345666, 40, 50, 60, 70, 80, 90])
>>> a*b
array([0, 10, 40, 90, 160, 250, 360, 490, 640, 810],
      dtype=int32)

```

Une première liste de fonctions à connaître **linspace**, **arange**, **zeros**, **ones**, **len**... On observera que l'on peut indexer par tranche comme avec les conteneurs indexables et surtout **multiplier terme à terme** deux tableaux de même taille comme ici a et b.

• Fonctions vectorialisées de numpy

```

>>> np.cos(np.pi/L)
array([ 8.66025404e-01,  7.07106781e-01,
        5.00000000e-01,  6.12323400e-17, -1.00000000e+00])

```

Le module numpy définit un bon nombre de fonctions mathématiques qui sont **vectorialisées**. Cela signifie qu'une telle fonction $x \rightarrow F(x)$, accepte en arguments des tableaux numériques T et retourne les tableaux $[F(T[1]), \dots, F(T[\ell - 1])]$, si

$\ell = \text{len}(T)$.

```
>>> L=np.array([6,4,3,2,1])
>>> np.pi/L
array([ 0.52359878,  0.78539816,  1.04719755,
        1.57079633,  3.14159265])
```

Vous pouvez également **vectorialiser vos propres fonctions** avec la fonction **vectorize** comme illustré ci-dessous :

```
import numpy as np

def f(x):
    if x>0:
        return 1
    elif x==0:
        return 0
    else:
        return -1

t = np.linspace(-2,2,11)

>>> f(t)
Traceback bla bla bla
ValueError: The truth value of an array...
>>> vf = np.vectorize(f)
>>> vf(t)
array([-1, -1, -1, -1, -1,  1,  1,  1,  1,  1])
```

• **L'avertissement** donné page 25 à propos de l'affectation des listes vaut aussi pour les tableaux.

Exercices

Exercice 1.16 *chargement d'un module*

Quels sont les énoncés vrais, faux, parmi ceux qui suivent ?

1. **Vrai ou Faux ?** Si je charge un module (module1 par exemple) avec `import module1` et si ce module définit la fonction sinus, je peux alors appeler `sin(0)` sans autre forme de procès.
2. **Vrai ou Faux ?** Si je charge un module (module1 par exemple) avec

`import module1` et si ce module définit la fonction sinus, je peux alors appeler `sin(0)` à condition d'utiliser une syntaxe particulière.

3. **Vrai ou Faux ?** Si je charge un module (module1 par exemple) qui contient une définition de la fonction sinus, je n'ai aucun moyen d'appeler `sin(0)` avec cette syntaxe simple, mais je dois impérativement appeler `module1.sin(0)`.

Corrigé en 1.16, page 83

Exercice 1.17 le type array de numpy

1. Quels sont les énoncés vrais, faux, parmi ceux qui suivent ?
 - (a) **Vrai ou Faux ?** Si je définis un tableau A de type `numpy.array` formé d'éléments qui sont de types entier (int), flottant (float) **et** complexe (complex), `A[i]` me renverra des objets du même type que celui que j'y aurais placé.
 - (b) **Vrai ou Faux ?** Si je définis un tableau A de type `numpy.array` formé d'éléments qui sont de types numériques (int, float, complex) **et** chaîne (str), `A[i]` me renverra des objets du même type que celui que j'aurais placé en cette position.
 - (c) **Vrai ou Faux ?** Si je définis un tableau A de type `numpy.array` à partir d'une liste contenant des éléments des trois types numériques int, float, complex, l'instruction `A[i] = 1j` me retourne une erreur si $i \geq \text{len}(A)$ et modifie la liste sans problème si $0 \leq i < \text{len}(A)$.
 - (d) **Vrai ou Faux ?** Si je définis un tableau A de type `numpy.array` formé d'éléments qui sont de type float, l'instruction `A[i] = 1j` me retourne une erreur si $i \geq \text{len}(A)$ et modifie la liste sans problème si $0 \leq i < \text{len}(A)$.
 - (e) **Vrai ou Faux ?** Si je définis un tableau A de type `numpy.array` formé d'éléments qui sont de type float, int, str, l'instruction, `A[i] = 1j` me retourne une erreur si $i \geq \text{len}(A)$ et modifie la liste sans problème si $0 \leq i < \text{len}(A)$.
 - (f) **Vrai ou Faux ?** l'instruction `a=numpy.arange(0,27,3)` retourne un tableau formé des multiples de 3 de 0 à 27.
2. Construire un objet de type `numpy.array`
 - (a) dont les termes sont les multiples de 3 compris entre 0 et 27 ;
 - (b) dont les termes sont les puissances de 2 de $2^0 = 1$ à $2^{12} = 4096$.

Corrigé en 1.17, page 84

1.5.3 Les graphiques avec matplotlib

On pourrait penser que le tracé de figures n'est pas une priorité dans l'apprentissage de l'algorithmique et de la programmation. Il n'en est rien ! Dans bien des cas, les représentations graphiques constituent un outil pour visualiser, conjecturer, déboguer un programme, tout autant qu'un moyen de présenter des résultats. On aura intérêt à développer très tôt dans un projet qui s'y prête les fonctions graphiques parce qu'elles permettront tout au long du processus de construction du programme de vérifier et de tester beaucoup plus rapidement qu'en analysant des sorties numériques. Bien évidemment nous nous cantonnons ici au minimum sans aller nous perdre inutilement dans les usines à gaz que sont (par la force des choses) les bibliothèques graphiques.

Ce qu'il faut savoir avant de commencer

- Les outils pour les tracés graphiques sont dans la bibliothèque **matplotlib** ; nous supposons qu'elle a été installée dans la foulée de numpy (et de scipy).

- Les interfaces graphiques permettant de gérer les figures sont dans le module **pylab** de cette bibliothèque ; c'est lui que nous chargerons avec **numpy** dont il utilise certaines fonctions :

```
import numpy as np
import pylab as pl
```

- La règle pour tous les tracés 2D avec **matplotlib**, est que l'on se donne deux tableaux de même taille X et Y et que l'on trace le **polygone** reliant les points (x_k, y_k) :

pylab.plot(X,Y) relie les points $\begin{bmatrix} x_1 \\ y_1 \end{bmatrix}, \begin{bmatrix} x_2 \\ y_2 \end{bmatrix}, \dots, \begin{bmatrix} x_n \\ y_n \end{bmatrix}$

- Pour tracer une figure nous ferons appel de façon systématique aux notions de base de la géométrie plane : caractérisation des segments, des droites, représentation cartésienne des graphes de fonctions, des courbes paramétrées, expression analytique (éventuellement à l'aide des complexes) des translations, homothéties, symétries et rotations.

Les tableaux qui suivent présentent le code dans la colonne de gauche et le résultat graphique associé dans celle de droite.

```

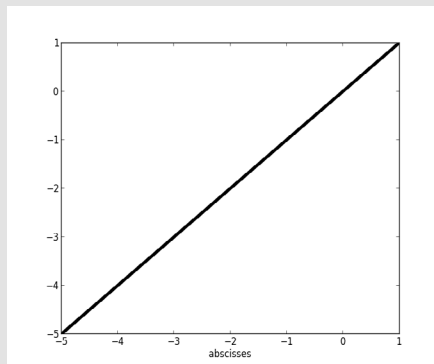
import numpy as np
import pylab as pl

x=np.linspace(1,-5,2)
y=np.linspace(1,-5,2)

pl.xlabel('abscisses')
pl.plot(x,y, linewidth = 4,
        color='black',)
pl.show()
pl.close()

```

Deux points et le tracé du segment dont ils sont les extrémités :



```

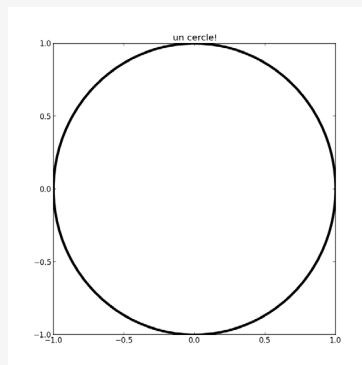
import numpy as np
import pylab as pl

t = np.linspace(0,2*np.pi,101)
xt = np.cos(t)
yt = np.sin(t)

pl.title('un cercle !')
pl.plot(xt,yt,linewidth = 4,
        color='black')
pl.show()
pl.close()

```

Cent points pour le tracé d'un polygone reliant les points de coordonnées $(\cos t_k, \sin t_k)$ du cercle unité :



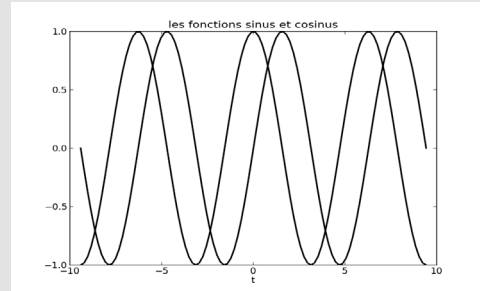
Observer que, comme dans le cours de maths, $(t_k)_{0 \leq k \leq 100}$ est une subdivision de l'intervalle $[0, 2\pi]$ numérotée de 0 à 100 avec $t_0 = 0$, $t_{100} = 2\pi$ et comprenant 101 points, délimitant 100 intervalles.

```
import numpy as np
import pylab as pl

t = np.linspace(-3*np.pi, 3*np.pi, 100)

C,S = np.cos(t), np.sin(t)
pl.title('les fonction sinus et cosinus')
pl.xlabel('t')
pl.plot(t, C, t, S, linewidth= 2,
        color='black')
pl.show()
pl.close()
```

Tracés superposés des fonctions cosinus et sinus :



```
import numpy as np
import pylab as pl

F = pl.figure(figsize=(8,5), dpi=80)
t = np.linspace(-3*np.pi, 3*np.pi, 100)
C,S = np.cos(t), np.sin(t)

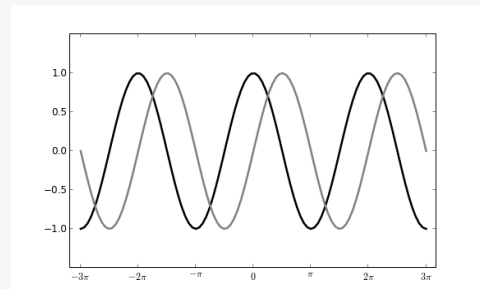
pl.plot(t, C, color="black",
        linewidth=2.5, linestyle="-")
pl.plot(t, S, color="gray",
        linewidth=2.5, linestyle="-")

absRem = np.linspace(-3*np.pi,
                      3*np.pi, 7)
pl.xticks(absRem, [ '$-3\pi$',
                    '$-2\pi$', ..., '$3\pi$' ])

pl.ylim(-1.5,1.5)
pl.yticks(np.linspace(-1,1,5))

pl.show()
```

Pour tracer sur une même figure les deux courbes avec 2 couleurs différentes, deux appels distincts à `pl.plot(...)` avec des options différentes. Pour placer les valeurs formelles -3π , etc. la syntaxe est celle de $\text{\LaTeX}$, le logiciel avec lequel ce livre est écrit.



- Si pour une bonne raison ¹², vous souhaitez en connaître plus sur les possibilités graphiques de Python, la page utile est http://matplotlib.org/api/pyplot_summary.html

12. TIPE par exemple.

• Résumé

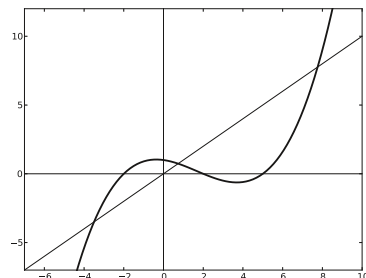
les mots clés	et leur signification
<code>import numpy as np</code>	chargement de la bibliothèque numpy (tableaux et fonctions mathématiques)
<code>import pylab as pl</code>	chargement du module pylab de la bibliothèque matplotlib. Il réunit les fonctions d'interface pour gérer les fenêtres graphiques
<code>numpy.linspace</code>	<code>numpy.linspace(a,b,n)</code> permet de créer un tableau de n points régulièrement espacés de a à b
<code>pylab.plot</code>	génère le tracé dans la figure courante (en écrivant plusieurs instructions <code>plot</code> à la suite on superpose donc les tracés)
<code>pylab.xlim</code> <code>pylab.xticks</code>	précise l'intervalle des abscisses (analogue en <code>ylim</code>) prend en arguments une liste de nombres (abscisses) et une ligne de chaînes de caractères de même taille (étiquette) et permet d'étiqueter les points correspondants sur l'axe des abscisses (analogue en <code>yticks</code>)
<code>pylab.show()</code>	c'est cette instruction qui provoque l'ouverture de la fenêtre graphique qui est le résultat de la superposition des instructions du type précédent.
<code>pylab.close()</code>	permet de fermer la fenêtre graphique avec la souris sans provoquer d'erreur (ne pas l'oublier !)

Exercices

Ne vous laissez pas effrayer par la longueur de certains énoncés. L'exercice 1.21 par exemple est beaucoup plus court qu'il n'y paraît.

Exercice 1.18 *graphiques de base*

1. Tracer sur une même figure les axes des abscisses et des ordonnées dans la fenêtre : $x_{\min} = -7$, $x_{\max} = 10$, $y_{\min} = -7$, $y_{\max} = 12$;
2. Y superposer les graphes de $x \mapsto \frac{1}{20} (x - 2) (x + 2) (x - 5)$ et de $x \mapsto x$ pour obtenir cela :



Corrigé en 1.18 page 86.

Exercice 1.19

1. Écrire un script qui demande à l'utilisateur des nombres $a_1, a_2, b_1, b_2, c_1, c_2$ et trace le triangle ABC .
2. Qu'obtient-on avec le script suivant (papier-crayon) ?

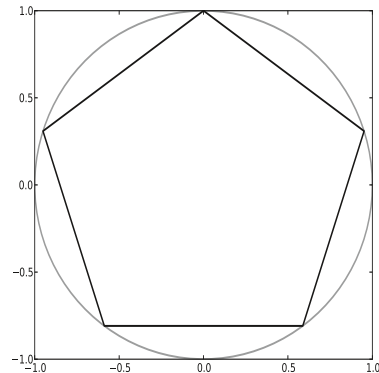
```
import numpy as np
import pylab as pl

T = np.linspace(0, 2*np.pi, 100)
X = 0x*np.ones(100) + r*np.cos(T)
Y = 0y*np.ones(100) + r*np.sin(T)

pl.plot(X1, Y1)
```

3. Écrire un script qui réalise la figure suivante :

- (a) en observant que les sommets sont les points $[\cos(t_k), \sin(t_k)]$ avec $t_k = \frac{\pi}{2} + \frac{2k\pi}{5}$;
- (b) en observant que les sommets ont pour affixes $ie^{+\frac{2ik\pi}{5}}$, $k = 0, 1, 2, 3, 4, 5$ (et 6 ?).



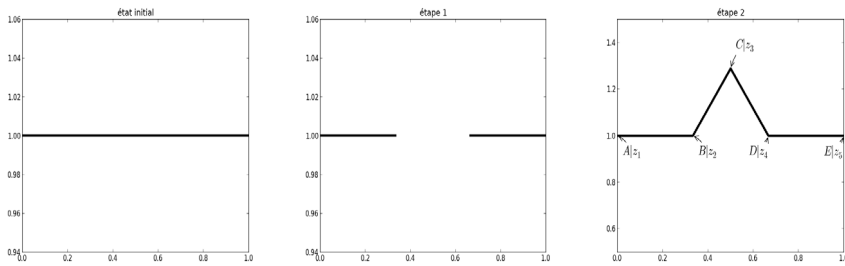
Corrigé en 1.19 page 87.

Exercice 1.20 *tracés de figures, apprentissage de la syntaxe*

On considère un segment $[A,E]$ que l'on souhaite transformer en deux étapes :

- on retire le segment $[B,D]$ dont les extrémités B et D sont situées au tiers et aux deux tiers de $[A,E]$;
- on introduit le point C est l'image de D par la rotation de centre B et d'angle $\pi/3$ et on rajoute à la figure les segments $[B,C]$ et $[C,D]$.

Le but de l'exercice est de tracer ces trois figures. Le code sera écrit dans un script ; on commencera par charger les modules `numpy` et `pylab`.



1. On choisira pour A et B les points du plan d'affixes $z_1 = 1$ et $z_2 = 1 + i$ qui seront donc représentés par les objets de type **complex** 1 et 1+1j. Définir les affixes z_1, z_2, z_4 et z_5 de A,B,D,E. Calculer l'affixe z_3 de C.

Utiliser la constante **pi** et les fonctions **exp** ou **cos** et **sin** de numpy.

2. Tracer le segment [A,E]
3. Tracer la figure de la première étape.
4. Tracer la ligne polygonale [A,B,C,D,E].

Utiliser les fonctions **real()** et **imag()** de numpy.

5. Si votre programme 'plante' à la fermeture, cherchez pourquoi et corrigez.
6. Reprendre les trois figures et augmenter l'épaisseur du trait, la couleur.

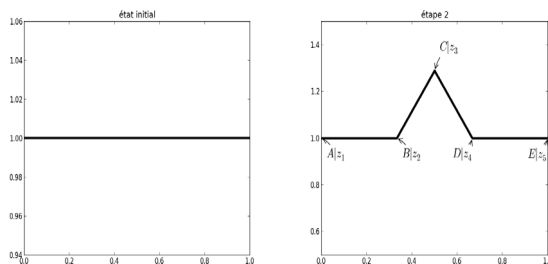
Options **linewidth = ..., color = '...'** ;

Corrigé en 1.20, page 88.

Exercice 1.21 dessinons la plus simple des fractales

On souhaite construire la figure ci-dessous (courbe de Von Koch) dont la construction est définie par récurrence de la façon suivante :

- Une transformation élémentaire consiste à remplacer le segment de la figure de gauche par la ligne polygonale de la figure de droite (dont les 4 cotés ont même longueur).



- Le polygone P_0 est un segment ;
- Le polygone P_{n+1} est construit à partir de P_n en appliquant une transformation élémentaire à chacun de ces cotés.

On utilisera les modules **numpy** et **pylab** pour disposer de π , des fonctions mathématiques (cosinus, sinus, exp) et pour les tracés.

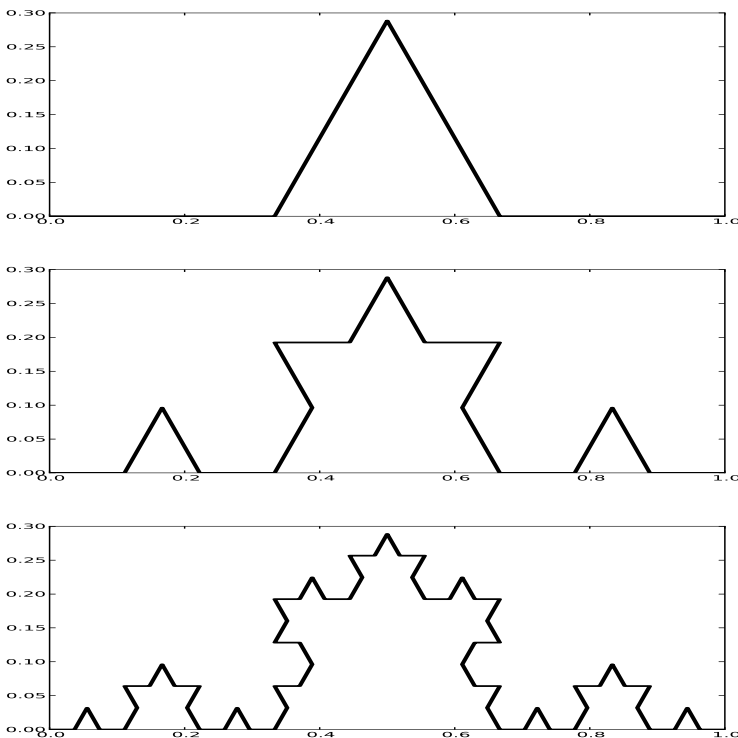
1. Écrire une fonction **transformation élémentaire(za,zb,w)** qui prend en argument trois complexes, za et zb étant les affixes des extrémités d'un segment et

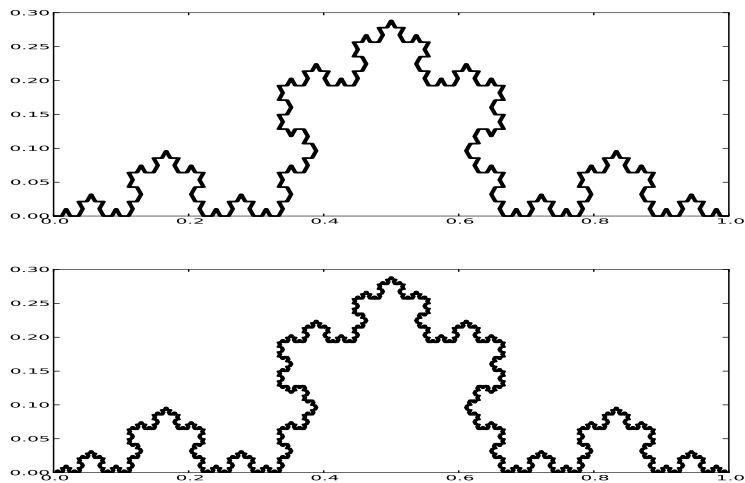
$$\text{retourne la liste } [za, z1, z2, z3] \text{ où } \begin{cases} z2 = za + (zb - za)/3 \\ z4 = za + 2(zb - za)/3 \\ z3 = z2 + w(z4 - z2) \end{cases}$$

Dénombrer les additions, soustractions, multiplications et divisions de votre procédure. Faites en sorte qu'il y ait le moins possible d'opérations.

2. En utilisant la construction de liste proposée comme opération standard dans le tableau de la page 36, écrire une fonction **transformationGlobale(L)** qui prend en argument une liste de complexes $L = [\ell_0, \ell_1, \dots, \ell_{n-1}]$ et retourne une nouvelle liste obtenue en insérant entre deux éléments successifs de L notés $\ell_i = za$ et $\ell_{i+1} = zb$ les points $z1, z2, z3$ de la première question. Comparer la taille de L et celle de la liste ainsi construite.

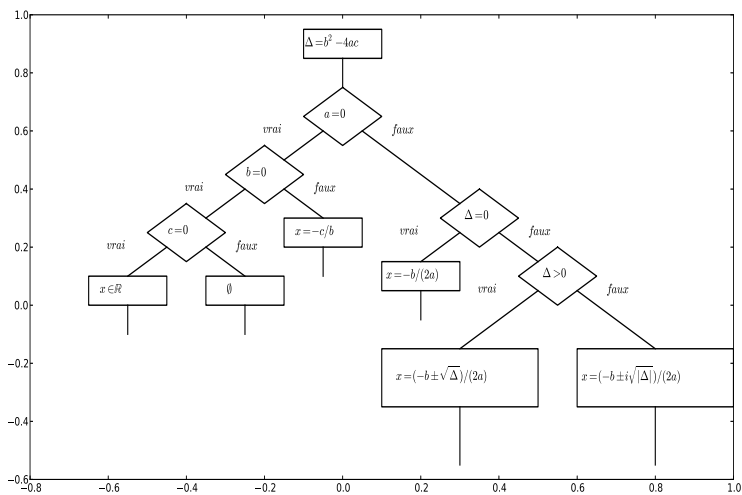
3. Écrire un script qui dessine successivement les figures suivantes à l'écran :





Corrigé en 1.21 page 89.

Exercice 1.22 On se propose ici de construire les outils graphiques qui ont permis de réaliser la figure de l'exercice 1.4 sans aller jusqu'à placer les inscriptions de texte (parce que c'est lourd et moins formateur) :



Pré-requis

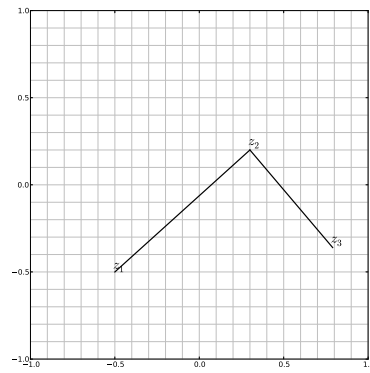
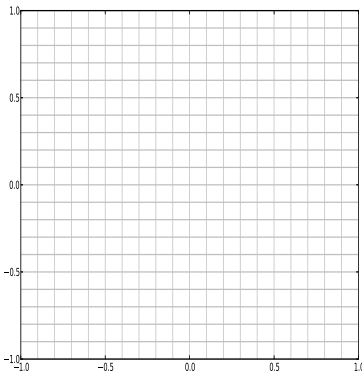
Sur Python : ce mini-problème porte sur les fonctions et procédures, il faudra aussi avoir lu rapidement ce paragraphe qui concerne les graphiques ; les rappels utiles figurent dans l'énoncé.

En géométrie : on aura besoin de l'interprétation géométrique des complexes de terminale ; là aussi on rappelle ce qui est utile.

Important : on dessinera ce que l'on veut obtenir à l'écran avant de programmer quoi que ce soit : on avance avec méthode et réflexion.

On commencera par importer numpy et pylab. On rappelle que l'on commande les tracés superposés avec **pylab.plot**, qu'on les affiche avec **pylab.show()**, et que l'instruction **pylab.close()** est indispensable pour assurer une fermeture correcte après un clic de souris pour fermer la fenêtre graphique.

1. Écrire une procédure **grille**(xmin, xmax, nx, ymin, ymax, ny, g1) qui permet de tracer un quadrillage en gris g1 (color = g1 = 0.75) avec nx lignes verticales et ny lignes horizontales dans la fenêtre $xmin \leq x \leq xmax, \times ymin \leq y \leq ymax$. Cela nous servira à caler les graphiques. On tracera ensuite un quadrillage aux points d'abscisses et ordonnées multiples de 0.1 comme sur la figure à gauche ci-dessous.



Indication : avec **numpy.linspace**(xmin,xmax,nx) on crée...

2. Écrire une procédure **segment**(x1,y1,x2,y2,c,e) qui trace un segment d'extrémités (x1,y1) et (x2,y2).
3. Écrire une fonction **translater**(x,y,u,v) qui prend en argument des flottants x,y,u,v et retourne les coordonnées de l'image du point (x,y) par la translation de vecteur (u,v).
4. Pour cette question il est **indispensable** de prendre un papier, un crayon et de dessiner ce que l'on veut obtenir à l'écran.

Écrire une procédure **rectangle**(x, y, u, v, L1, L2, d, c, e) qui trace un rectangle R tel que

- le point de coordonnées (x,y) est un sommet S_1 de R ;
- le sommet suivant S_2 est à la distance L_1 de S_1 dans la direction et le sens donné par le vecteur (u,v) (penser à le **normer** dans la procédure, c'est ce que l'utilisateur n'aura pas à faire) ;

- on construit un troisième sommet S_3 tel que le coté $[S_2, S_3]$ ait pour longueur L_2 ;
- le rectangle (S_1, S_2, S_3, S_4) est direct si d est la chaîne 'd', indirect sinon.
- c et e sont la couleur et l'épaisseur du trait(facultatif).

Indication maths :

- Si les points M_1, M_2, M_3 ont pour affixes z_1, z_2 et z_3 ,
- si M_3 est l'image de M_1 par la composée $h \circ r$ de la rotation r de centre M_2 et d'angle θ et de l'homothétie de centre M_2 de rapport $\lambda > 0$,

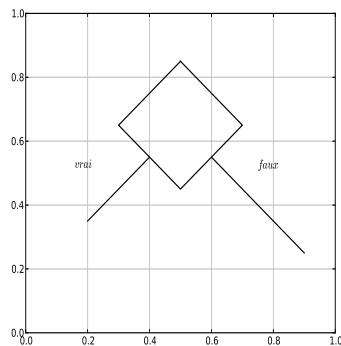
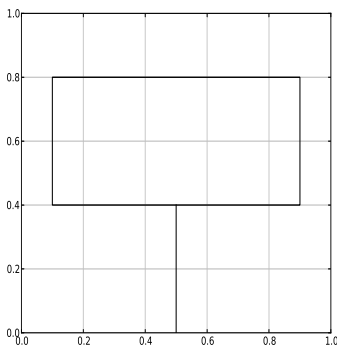
alors : $z_3 = z_2 + \lambda e^{i\theta}(z_1 - z_2)$

Indication Python :

le complexe $z_2 = x_2 + iy_2$ en Python s'écrira **x2 + y2*1j** (avec x2, y2 flottants).

- Écrire une procédure **instruction**(x, y, h) qui prend en argument trois flottants x, y, h et
 - **dessine** à l'écran la figure présentée à gauche ci-dessous (instruction) formée d'un rectangle de longueur $2h$ de largeur h , tel que (x, y) est le point milieu du coté supérieur et d'un segment de longueur h
 - **retourne** le **tuple** des coordonnées (x_1, y_1) de l'extrémité inférieure du segment ;
 - utilise les fonctions `segment` et `translater` déjà programmées.

On voit que ce n'est là, ni une fonction pure, ni une procédure pure. Cela facilitera néanmoins le travail de qui voudra compléter la figure en plaçant l'instruction suivante au bout du segment pointant vers le bas.



- L'instruction conditionnelle sera représentée par un carré (rectangle et losange à la fois) :

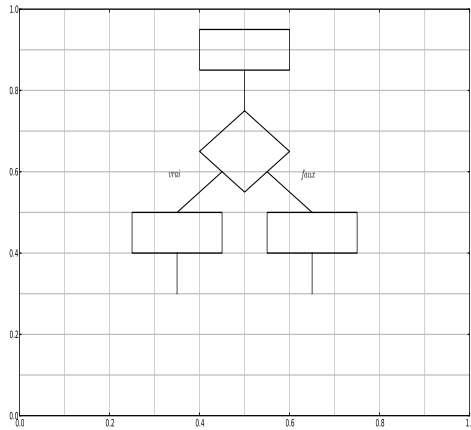
instrCond(x0, y0, h0, cg, cd) qui prend des flottants en arguments avec

- (x_0, y_0) le sommet situé en haut de la figure ;
- h_0 la longueur du coté ;
- cg et cd qui sont les coefficients par lesquels multiplier la longueur h_0 pour obtenir les longueurs des deux segments qui partent à gauche et à droite (vrai ou faux) ; on a besoin de les contrôler pour éviter les télescopes ;

et qui

- **dessine** la figure présentée à droite ci-dessus (instruction conditionnelle) ;
- **retourne** le **tuple** (xg, yg, xd, yd) des coordonnées des extrémités des deux segments, ce qui servira à l'utilisateur pour « rabouter » les instructions suivantes.

7. Vérifier que vous savez « rabouter » les éléments de base :

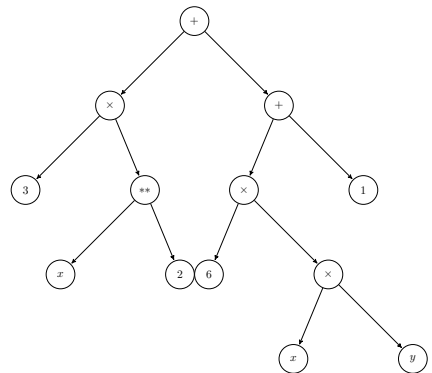


Corrigé en 1.22, page 91.

1.6 Corrigés des exercices

Corrigé de l'exercice n° 1.1

1. Cet arbre représente l'expression : $3x^2 + 6xy + 1$. En effet c'est la somme des trois sous-expressions, chacune représentée par un sous-arbre $3x^2$, $6xy$ et 1.
2. On définit a priori l'addition comme un opérateur binaire (qui prend deux arguments). Mais comme elle est **associative** les expressions $(a + b) + c$ et $a + (b + c)$ sont égales. Ce que l'on écrit par abus de notation $a + b + c$ et dont vous verrez dans ce cours qu'elles ne sont plus identiques quand on ajoute des flottants. L'arbre de l'énoncé traite donc l'addition comme un opérateur n-aire. Cela n'aurait plus de sens avec des opérateurs non associatifs sauf à préciser une convention qui reviendrait à un parenthésage implicite.
3. Un arbre correct serait



Corrigé de l'exercice n° 1.2

- $ell = len(O)$: la longueur de O (nombre de termes), un entier supérieur ou égal à 4 est affecté à ℓ ;
 $O[0]$; $O[ell-1]$: le premier et le dernier éléments de O (**numérotée de 0 à $ell-1$**) ;
 $O[ell]$: une erreur (voir ci-dessus) ;
 $O[0:ell]$ une tranche de 0 à $ell-1$, donc tout l'objet ;
 $O[0:3]$: la sous-liste, le sous-tuple ou la sous-chaîne formée des termes de **n° de 0 à 2 (car le terme indexé par 3 est exclu)**
 $O[-1]$: le dernier terme ;
 $O[ell-1]$: le dernier terme ;
 $O[2:3]$: le troisième terme seul, à savoir $O[2]$ (**car le terme indexé par 3 est exclu de l'itération**)
 $O[2:2]$: la liste vide [], le tuple vide() , la chaîne vide'' (ne retourne pas le terme indexé par 2 car il est exclu de l'itération !)

Corrigé de l'exercice n° 1.3

Après $L=[0,1,2,3,4]$

- Il est Vrai que** la deuxième instruction modifie la liste L

```
>>> T = L
```

```
>>> T[0] = 12
```

 L devient alors (comme M) $[12,1,2,3,4]$
- Il est Faux que** les instructions suivantes modifient la liste L

```
>>> T[0:len(L)] = L
```

```
>>> T[0] = 12
```

 T est une copie de L et la deuxième affectation ne modifie pas L (c'est comme ça dans l'architecture de Python, c'est tout !)
- Il est Vrai que** l'instruction suivante modifie la liste L

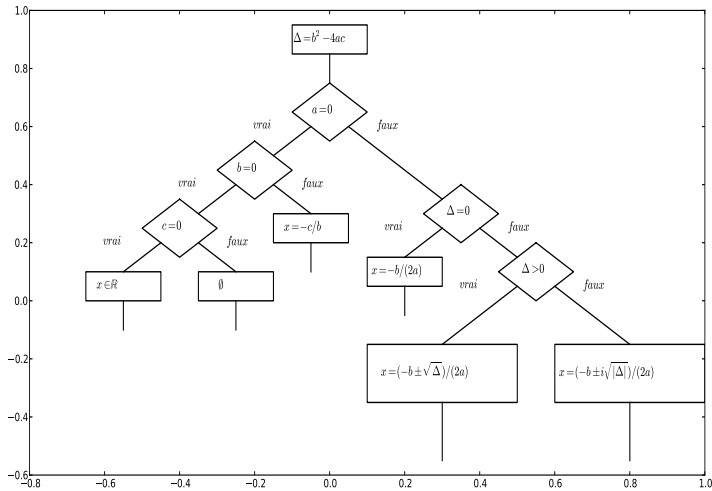
```
>>> L.append(-6) ;
```

 Cette instruction modifie la liste L (mais l'expression elle-même est évaluée à **None** (ce sera plus clair quand vous aurez étudié la programmation orienté objet).

Corrigé de l'exercice n° 1.4

- La solution est donnée par le diagramme suivant¹³ :

13. Sommaire dessiné par un programme Python dont les grandes lignes sont données dans l'exercice 1.22, page 67.



2. C'est un arbre (on peut observer que le graphe symétrique associé est sans boucle) qui admet 5 feuilles (instructions non conditionnelles) correspondant aux 5 cas possibles qui ont tous été testés.
3. Le programme serait le suivant et l'organigramme associé ressemble à un **peigne**

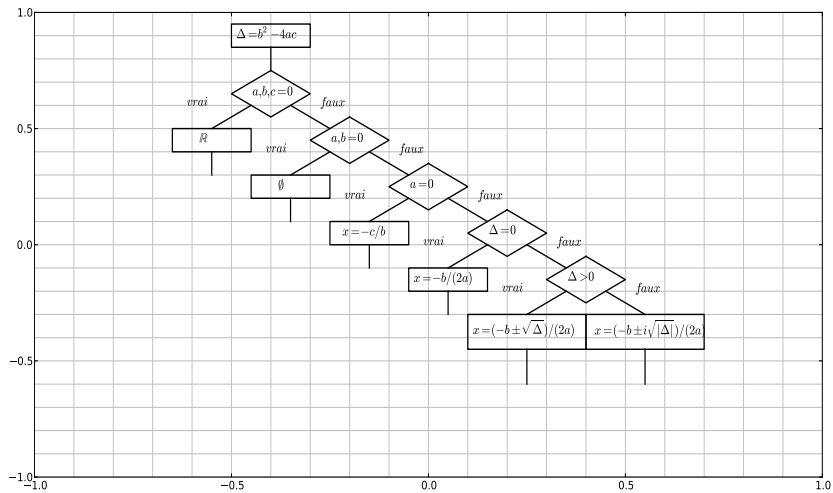
```

delta = b**2 - 4*a*c

if a == b == c == 0 :
    ...
elif a == b == 0 :
    ...
elif a == 0 and c != 0 :
    ...
elif delta == 0 :
    ...
elif delta > 0 :
    ...
else :
    ...

```

Les instructions conditionnelles imbriquées peuvent toujours être remplacées par une instruction **if** suivie d'autant de clauses **elif** que nécessaire le diagramme est alors un **peigne** comme dans notre exemple :



Corrigé de l'exercice n° 1.5

calcul de $p!$ programmé sans instruction conditionnelle car si $n=0$, $\text{range}(1,n+1)$ est vide et il n'y a aucune itération de la boucle for	<pre>n = int(input('n= ')) p = 1 for k in range(1,n+1) : p = k*p print(p)</pre>
--	--

Observons que **L.append(*)** modifie la liste L sans créer de nouvelle liste en mémoire. Par contre, un tuple est **immuable** et on ne pourra le construire terme à terme comme ici : un triplet ne peut être transformé en quadruplet par exemple.

Liste des cubes des entiers de 0 à n On regardera aussi l'exercice 1.7, page 40	<pre>n = int(input('n= ')) L = [] for k in range(0,n+1) : L.append(k**3) print(L)</pre>
--	---

Suite récurrente : on observe que la suite des $(r_1(k))_k$ semble converger vers $\sqrt{a}$ et que $(r_2(k))_k$ semble converger vers $-\sqrt{a}$.

En ce qui concerne la suite de premier terme $r_1(0) = 1$, on démontre par récurrence sur k que $r_1(k) > 0$:

- cela est vrai pour $k = 0$ par construction ;
- ci $r_1(k) > 0$, le terme suivant est $r_1(k+1) = \frac{1}{2} \left(r_1(k) + \frac{a}{r_1(k)} \right)$, à la fois défini (car $r_1(k) \neq 0$ et strictement positif).

Pour ce qui est de la preuve de convergence, de la vitesse de convergence nous laissons cela au cours de maths (c'est là un cas particulier de la méthode de Newton que vous retrouverez en cours de maths et dans les applications numériques).

```
a = float(input('a= '))
n = int(input('n ='))
```

```
r1 = 1
for k in range(1,n+1) :
    r1 = (r1 + a/r1)/2
    print('r1 = ', r1)
```

```
r2 = -1
for k in range(1,n+1) :
    r2 = (r2 + a/r2)/2
    print('r2 = ', r2)
```

```
a= 49
n =6
r1 = 25.0
r1 = 13.48
r1 = 8.557507418397627
r1 = 7.141736912383411
r1 = 7.001406475243939
r1 = 7.000000141269659
r2 = -25.0
r2 = -13.48
r2 = -8.557507418397627
r2 = -7.141736912383411
r2 = -7.001406475243939
r2 = -7.000000141269659
```

Corrigé de l'exercice n° 1.6

Observer l'usage de l'affectation multiple, sans elle, il aurait fallu programmer comme à droite :

```
n = 20
f0, f1 = 1

for k in range(2,n+1) :
    f0, f1 = f1, f0+f1
    print(f1)
```

```
n = 20
f0 = 1
f1 = 1

for k in range(2,n+1) :
    g = f1
    f1 = f0+f1
    f0 = g
    print(f1)
```

Corrigé de l'exercice n° 1.7

1. Cubes de 0 à n :

```
>>> n=212;
>>> [x**3 for x in range(0, int(n**(1/3))+1)]
[0, 1, 8, 27, 64, 125]
```

2. Diviseurs de n :

```
>>> n=212; [x for x in range(1,n+1) if n % x ==0]
[1, 2, 4, 53, 106, 212]
```

3. Liste des indices tels que $L[k]=x$:

```
>>> L = [0,1,2,3,4,5,6,6,5,4,3,2,1,0,-1,-2, -3]
>>> [ k for k in range(0,len(L)) if L[k]==0]
[0, 13]
```

Corrigé de l'exercice n° 1.8

La copie du script pour toute forme de correction :

1. En compréhension

```
L = [x for x in range(0,6)]

L1=[x for x in L for k in range(0,2)]

L2=[x for k in range(0,2) for x in L]

L3=[x for x in L for k in range(0,x)]
```

2. Avec des boucles for explicitées (quasiment identique)

```
T = [0,2,3,1,4]

m, T1= 3, []
for x in L:
    for k in range(0,m):
        T1.append(x)

m, T2= 3, []
for k in range(0,m):
    for x in L:
        T2.append(x)

T3 = []
for x in L:
    for k in range(0,x):
        T3.append(x)
```

Corrigé de l'exercice n° 1.9

1. Comme la boucle termine avec $2^p \leq n < 2^{p+1}$, on a en prenant le log, $p \leq \frac{\ln n}{\ln 2} < p + 1$. Ainsi, $p = \text{Ent}\left(\frac{\ln n}{\ln 2}\right)$

```
n = int(input('n = '))
p = 0
while 2**(p + 1) <= n:
    p = p + 1

print(p)
```

2. (a) On note a_0 le chiffre des unités de m . On a donc $m' = \frac{m - a_0}{10} - 2a_0$.
On sait que `divmod(m,10)` retourne le couple ¹⁴ $(q, r) = (q, a_0)$ formé du quotient et du reste de la division euclidienne de m par 10. Le calcul de m se programme donc :

```
a0 = divmod(m, 10) [1]
(m - a0) // 10 - 2*a0
```

- (b) On écrira donc

le script	l'exécution dans le shell
<pre>m = int(input('m = ')) while m > 7: a0 = divmod(m, 10) [1] m = (m - a0) // 10 - 2*a0 print(m)</pre>	<pre>>>> m = 477557101 47755708 4775554 477547 47740 4774 469 28 -14</pre>

Corrigé de l'exercice n° 1.10

1. Il suffit d'importer la fonction comme illustré page 54 :

```
>>> from random import randrange
>>> randrange(0, 1001)
136
```

14. Tuple à deux éléments.

2. (et 3) Commentons le programme

- première ligne : on importe la fonction **randrange** du module **random**, sa syntaxe est celle de tous les itérables : **randrange(a,b+1)** retourne un nombre aléatoire entre a et b (b+1 n'est jamais tiré) :

```
import random
print('Ce jeu consiste à deviner un nombre entre
0 et 1000.\n Pour abandonner taper 1001')

n = random.randrange(0, 1001)
c = 1
x = int(input('essai n° 1 : '))

while x != n and x != 1001 :
    if x < n :
        print('trop petit')
    else :
        print('trop grand')
    c = c + 1
    x = int(input('essai n° ' + str(c) + ' : '))

if x == n :
    print('Vous avez réussi en ', c, ' essais');
else :
    print("C'était n =", n);
```

- `\n` n'est autre que le caractère de retour à la ligne ; voir la copie de shell dans l'énoncé.
- `c` joue le rôle de compteur des essais ; le premier essai qui sert d'initialisation ayant lieu avant toute entrée dans la boucle, on initialise à `c=1` ;
- `x = int(input('essai n° 1 : '))` : **input**(texte) affiche le texte et récupère la **chaîne** entrée au clavier, **int** convertit cette chaîne en **entier** (il y a une erreur si la chaîne n'est pas convertible) ;
- on arrive enfin au cœur du mini-programme : la condition d'arrêt sera la négation de `(x != n and x != 1001)` à savoir `(x == n or x == 1001)`, c'est-à-dire que le joueur a trouvé **n** **ou** qu'il abandonne ; notre **ou** est exclusif (car l'abandon 1001 n'est jamais une bonne réponse) ;
- le corps de la boucle est sans mystère : affichage de l'indication par le programme avec l'instruction conditionnelle (ici `x` ne peut être égal à `n`), incrémentation du compteur des essais (variable `c`), proposition d'un nouvel essai qui sera testé avant une nouvelle itération éventuelle (observer que `c` est converti en chaîne avec **str**).
- En sortie de boucle, la condition d'arrêt est vérifiée et donc le contenu de `x`

est soit 1001 (abandon) soit la bonne réponse.

Remarque : ce programme ne peut terminer qu'avec une intervention extérieure : le joueur trouve le nombre ou abandonne en tapant 1001. S'il quitte l'ordinateur ou s'endort dans un long sommeil, le programme attend indéfiniment une réponse !

Corrigé de l'exercice n° 1.11

Les deux programmes sont présentés en fin de corrigé.

1. Justifions que le résultat est correct :

- la boucle termine : p est incrémenté à chaque itération et la suite $(2^{p+1})_p$ a pour limite ∞ donc dépasse n à partir d'un certain rang ;
- le critère d'arrêt est $2^{p+1} > n$; comme $n \geq 1$, il y a eu au moins une itération et on avait à l'évaluation précédente $2^p \leq n$. On termine donc avec $2^p \leq n < 2^{p+1}$.

En conséquence $p \ln 2 \leq \ln n < (p+1) \ln 2$, ce qui donne :

$$\frac{\ln n}{\ln 2} - 1 < p \leq \frac{\ln n}{\ln 2}.$$

2. Le principe de la preuve de correction est le même qu'à la question précédente.

Évaluons le nombre d'itérations : en fin de boucle la condition est

$$\frac{k(k+1)}{2} \leq n < \frac{(k+1)(k+2)}{2}.$$

k est donc solution des deux inéquations du second degré : $E_1 : k^2 + k - 2n \leq 0$ et $E_2 : k^2 + 3k - 2 - 2n > 0$. On a donc à la fois

$$k \in \left[\frac{-1 - \sqrt{1+8n}}{2}, \frac{-1 + \sqrt{1+8n}}{2} \right], \quad k \notin \left[\frac{-3 - \sqrt{1+8n}}{2}, \frac{-3 + \sqrt{1+8n}}{2} \right]$$

$$k \text{ est donc le seul entier tel que } \frac{-1 + \sqrt{1+8n}}{2} \leq k < \frac{-3 + \sqrt{1+8n}}{2}.$$

```
def log_entier(n):
    if n<1:
        raise ValueError
    else:
        p=0
        while 2**(p+1) <= n:
            p=p+1
        return p
```

```
def nombre_triangu(n):
    if n < 0 :
        raise ValueError
    else:
        k=0
        while (k+1)*(k+2)//2 <= n:
            k = k+1
        return k
```

Corrigé de l'exercice n° 1.12

1. C'est le pendant de **len(L)**

```
def longueur(L) :
    ell=0
    for x in L :
        ell=ell+1
    return ell
```

2. Appartenance :

```
def appartient(L, x) :
    for e in L :
        if x == e :
            return True
    return False
```

3. Le pendant de la méthode **L.count(x)**

```
def nb_occurrences(L, x) :
    c = 0
    for e in L :
        if x == e : c=c+1
    return c
```

4. On ne peut résister à l'appel de lambda et des constructions en compréhension ; on peut utiliser quasi-indifféremment **len** :

```
liste_places = lambda L,x : [ k for k in range (0, len(L)) if L[k]==x]
```

5. C'est une variante de la fonction **max** de Python qui, quant à elle, retourne un élément et non pas un couple comme la nôtre. Remarquons au passage que, contrairement à **L.count()**, par exemple, **max** n'est pas une méthode de la classe **list** ce qui explique pourquoi elle ne figure pas dans la liste citée dans l'énoncé.

```
def maximum(L) :
    m, pos, k = L[0], 0, 0
    for x in L :
        k = k+1
        if x > m :
            m, pos = x, k
    return m, pos
```

6. Toujours **lambda** et une construction en compréhension :

```
places_max = lambda L : [ k for k in range (0, len(L)) if L[k]==max(L)]
```

ou, avec notre fonction **maximum** qui retourne un couple :

```
places_max = lambda L : [ k for k in range (0, len(L)) if L[k]== maximum(L)[0]]
```

7. Il y a un point délicat dans cette question, celui de l'affectation **M = L** d'une variable pointant vers une liste à une autre variable. Une modification partielle de **M** (**M[0]=...**, par exemple) affecte **L** alors que l'affectation **M = L[0,len(L)]** crée une copie de **L**. D'où une fonction pure (à gauche) et une procédure (à droite) :

Une fonction qui retourne une autre liste sans modifier la liste passée en argument vers laquelle pointe la variable L	Une procédure qui modifie la liste passée en argument (on dit que l'on travaille en place)
<pre>def plus_grand_devant(L) : m, pos = maximum(L) L1 = L[0:len(L)] L1[0], L1[pos] = m, L[0] return L1</pre>	<pre>def plus_grand_devantP(L) : m, pos = maximum(L) L[pos], L[0] = L[0], m return None</pre>

<pre>>>> L=[1,-1,0,9,5,3] >>> plus_grand_devant(L) [9, -1, 0, 1, 5, 3] >>> L [1, -1, 0, 9, 5, 3]</pre>	<pre>>>> L=[1,-1,0,9,5,3] >>> plus_grand_devantP(L) >>> L [9, -1, 0, 1, 5, 3]</pre>
---	--

8. C'est le même problème que le précédent. Une **fonction** qui retourne une autre liste :

```
supprimer = lambda L, ell : [ L[k] for k in range(0, len(L)) if k not in ell]
```

```
>>> L
[0, 1, 2, 3, 4, 5, 6, 6, 5, 4, 3, 2, 1]
>>> supprimer(L, [0,1,2])
[3, 4, 5, 6, 6, 5, 4, 3, 2, 1]
>>> L
[0, 1, 2, 3, 4, 5, 6, 6, 5, 4, 3, 2, 1]
```

*** Une procédure qui modifie L elle-même : le problème est bien plus difficile, en effet on est tenté de faire :

```
def supprimerP(L, ell) :
    #— mauvais choix sauf si ell ↘
    for k in ell :
        L[k:k+1]=[]
    return None
```

Le problème c'est que la suppression d'un terme **k** modifie la numérotation de ceux qui le suivent dans la liste.

Pour éviter tout problème, on commence par supprimer le terme d'indice le plus grand ainsi l'indexation des termes précédent restant inchangée, l'opération qui suit affecte le bon numéro. On n'aura pas de problème si ell est triée dans l'ordre **décroissant** :

```
def supprimerP(L, ell) :
    ell.sort()
    ell.reverse()
    for k in ell :
        L[k:k+1]=[]
    return None
```



```
>>> L
[8, 7, 6, 5, 4, 3, 4, 5]
>>> supprimerP(L, [0,1,2,3])
>>> L
[4, 3, 4, 5]
```

Corrigé de l'exercice n° 1.13

```
def phi(m,i,j):
    return j*m+i

def inv_phi(m, k):
    s = divmod(k, m)
    return (s[0], s[1])
```

```
def psi(i,j):
    s =i+j
    return s*(s+1)//2+j

def inv_psi(n):
    k = 0
    while (k+1)*(k+2)//2 <= n:
        k=k+1
    j = n - k*(k+1)//2
    return (k-j, j)
```

Corrigé de l'exercice n° 1.14 voir aussi l'exercice 1.8 page 40

1. Simple adaptation de la méthode standard de construction d'un objet à partir d'une construction associative en partant d'un élément neutre (ici la liste vide et la concaténation) :

```
def petitsPains(L,m):
    T = []
    for i in range(0,len(L)):
        for j in range(0,m):
            T.append(L[i])
    return T
```

2. Avec une fonction lambda :

```
petitsPainsLambda = lambda L, m : [x for x in L for j in range(0,m)]
```

Corrigé de l'exercice n° 1.15

1. Cette fonction prend deux arguments : une liste L , et x quelconque. Elle retourne la liste définie en compréhension des **indices** des emplacements de x dans L :

```
lambda L, x: [ k for k in range(0, len(L)) if L[k]==x]
```

2. **FctNewtonFormel** prend deux fonctions numériques en arguments et retourne

F telle que $F(x) = x - \frac{f(x)}{df(x)}$.

Remarque : si df est la dérivée de f ,

$$F(x) = x - \frac{f(x)}{f'(x)}$$

est la fonction auxiliaire de la méthode de Newton.

Corrigé de l'exercice n° 1.16

- Il est Faux que :** Si je charge un module (module1 par exemple) avec `import module1` et si ce module définit la fonction sinus, je peux alors appeler `sin(0)` sans autre forme de procès.
En effet, après le chargement `import module1`, il faut encore spécifier le module et rédiger `module1.sin(0)` ; cela est illustré avec le module `numpy` et la fonction `sin`, précisément, dans le tableau de la page 54.
- Il est Vrai que :** Si je charge un module (module1 par exemple) avec `import module1` et si ce module définit la fonction sinus, je peux alors appeler `sin(0)` à condition d'utiliser une syntaxe particulière.
Cette syntaxe est `module1.sin(0)`, on précise ainsi que `sin` fait référence à l'objet (constante, fonction (comme c'est le cas ici)... défini dans le fichier ou module nommé `module1`
- Il est Faux que :** Si je charge un module (module1 par exemple) qui contient une définition de la fonction sinus, je n'ai aucun moyen d'appeler `sin(0)` avec cette syntaxe simple, mais je dois impérativement appeler `module1.sin(0)`.

En effet, j'ai deux possibilités pour un appel direct :

- charger le module avec `from numpy import *` qui permet d'importer tous les objets et de les appeler comme ceux qui sont définis dans le cœur du langage,
- ou encore, ce qui n'a pas été précisé dans le tableau de la page 54 :

```
>>> from numpy import sin, pi
>>> sin(pi)
1.2246467991473532e-16
```

Voir aussi la notion **d'espace de noms** dans le glossaire page 571.

Corrigé de l'exercice n° 1.17

1. Quels sont les énoncés vrais, faux, parmi ceux qui suivent ?

- (a) **Il est Faux que :** Si je définis un tableau A de type `numpy.array` formé d'éléments qui sont de types entier (int), flottant (float) et complexe (complex), `A[i]` me renverra un objet du même type que celui que j'y aurais placé.

Il y a en effet conversion à un type unique (on se retrouve ici avec des complexes) :

```
>>> a=numpy.array([1,2.,0,1+1j,3]); a
array([1.+0.j,2.+0.j,0.+0.j,1.+1.j,3.+0.j])
>>> a[0]
(1+0j)
```

- (b) **Il est Faux que :** Si je définis un tableau A de type `numpy.array` formé d'éléments qui sont de types numériques (int, float, complex) et chaîne (str), `A[i]` me renverra des objets du même type que celui que j'aurais placé en cette position.

Il y a en effet conversion à un type unique (on se retrouve ici avec des chaînes) :

```
>>> a=numpy.array([1,2.,0,1+1j,3,'er']); a
array(['1','2.0','0','(1+', '3','er'],
      dtype='<U3')
>>> a[0]
'1'
>>> a[0]+a[5]
'1er'
```

- (c) **Il est Vrai que :** Si je définis un tableau A de type `numpy.array` à partir d'une liste contenant des éléments des trois types numériques int, float, complex, l'instruction, `A[i] = 1j` me retourne une erreur si $i \geq \text{len}(A)$ et modifie la liste sans problème si $0 \leq i < \text{len}(A)$.

En effet, il y a conversion vers le type complexe :

```
>>> a=numpy.array([1,2.,0,1+1j,3]);a
array([1.+0.j,2.+0.j,0.+0.j,1.+1.j,3.+0.j])
>>> a[0]=1j
>>> a
array([0.+1.j,2.+0.j,0.+0.j,1.+1.j,3.+0.j])
>>> a[7]=12j
Traceback (most recent call last):
  File "<pyshell#10>", line 1, in <module>
    a[7]=12j
IndexError: index out of bounds
```

- (d) **Il est Faux que :** Si je définis un tableau A de type `numpy.array` formé d'éléments qui sont de type `float`, l'instruction, `A[i] = 1j` me retourne une erreur si $i \geq \text{len}(A)$ et modifie la liste sans problème si $0 \leq i < \text{len}(A)$. Il y aura toujours une erreur car le tableau est formé de flottants :

```
>>> a=numpy.array([1,3.2,-45.01]);a
array([ 1. , 3.2 , -45.01])
>>> a[0]=1j
Traceback (most recent call last):
  File "<pyshell#16>", line 1, in <module>
    a[0]=1j
TypeError: can't convert complex to float
```

- (e) **Il est Vrai que :** Si je définis un tableau A de type `numpy.array` formé d'éléments qui sont de type `float`, `int`, `str`, l'instruction, `A[i] = 1j` me retourne une erreur si $i \geq \text{len}(A)$ et modifie la liste sans problème si $0 \leq i < \text{len}(A)$.

En effet, la liste est alors formée de chaînes et `1j` est converti en chaîne avant affectation :

```
>>> a=numpy.array([1,3.2,-45.01, 'str']);a
array(['1', '3.2', '-45', 'str'],
      dtype='<U3')
>>> a[0]=1j
>>> a
array(['1j', '3.2', '-45', 'str'],
      dtype='<U3')
```

- (f) **Il est Faux que :** l'instruction `a=numpy.arange(0,27,3)` retourne un tableau formé des multiples de 3 de 0 à 27.

En effet la borne supérieure n'est pas atteinte, le dernier multiple sera donc 24.

```
>>> a=numpy.arange(0,27,3);a
array([ 0, 3, 6, 9,12,15,18,21,24])
>>> a=numpy.arange(0,28,3);a
array([ 0,3,6,9,12,15,18,21,24 27])
```

2. (a) un objet de type `numpy.array` dont les termes sont les multiples de 3 de 0 à 27 :

voir ci-dessus ;

- (b) un objet de type `numpy.array` dont les termes sont les puissances de 2 de $2^0 = 1$ à $2^{12} = 4096$:

```
>>>a=numpy.arange(0,13);a
array([ 0,1,2,3,4,5,6,7,8,9,10,11,12])
>>> 2**a
array([1,2,4,8,16,32,64,128,256,512,1024,
2048,4096], dtype=int32)
```

Corrigé de l'exercice n° 1.18

```
xmin = -7
xmax = 10
ymin = -7
ymax = 12

pl.xlim(xmin, xmax)
pl.ylim(ymin, ymax)
pl.plot([xmin,xmax], [0,0] , [0,0] ,
        [ymin,ymax],color='black')
pl.show()
pl.close()
```

```

h      = lambda x : (1/20)*(x-2)*(x+2)*(x-5)

xmin = -7
xmax = 10
ymin = -7
ymax = 12
X = np.linspace(xmin, xmax, 100)

pl.xlim(xmin,    xmax)
pl.ylim(ymin,    ymax)
pl.plot([xmin,xmax],[0,0],[0,0],
        [ymin,ymax],color='black')
pl.plot(X, X, X, h(X),
        linewidth = 2.5, color='black')
pl.show()
pl.close()

```

Corrigé de l'exercice n° 1.19

1. On rappelle que

`pl.plot([a1,b1,c1,a1],[a2,b2,c2,a2])` relie les points

$$\begin{bmatrix} a_1 \\ a_2 \end{bmatrix}, \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}, \begin{bmatrix} c_1 \\ c_2 \end{bmatrix}, \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}.$$

On fait donc

```

a1 = float(input('a1 = '))
a2 = float(input('a2 = '))

b1 = float(input('b1 = '))
b2 = float(input('b2 = '))

c1= float(input('c1 = '))
c2= float(input('c2 = '))

pl.plot([a1,b1,c1,a1],[a2,b2,c2,a2])
pl.show()
pl.close()
# ne pas l'oublier pour une
    fermeture correcte de la fenêtre

```

2. Commentaires ligne à ligne :

```
T = np.linspace(0,2*np.pi,100)
```

subdivision de $[0, 2\pi]$, 100 points que l'on note t_k pour la suite ;

```
X = Ox*np.ones(100)
```

tableau (ou vecteur) contenant 100 termes égaux à O_x ;

```
X = Ox*np.ones(100) + r*np.cos(T)
```

on lui ajoute les $r * \cos(t_k)$ ce qui donne 100 termes $O_x + r \cos(t_k)$;

```
Y = Ox*np.ones(100) + r*np.sin(T)
```

100 termes $O_y + r \sin(t_k)$;

`pl.plot(X, Y)` : on obtient le cercle de centre $[O_x, O_y]$ de rayon r .

3. Le pentagone régulier :

```
phi      = 2*np.pi/5
T        = np.linspace(0, 2*np.pi, 100)
X , Y    = np.cos(T), np.sin(T)
pl.plot(X, Y, color='gray', linewidth=2)

L = [np.exp(k*phi*1j)*1j for k in range(0, 6)]
pl.plot(np.real(L), np.imag(L), color='black',
        linewidth=2)

pl.show()
pl.close()
```

Corrigé de l'exercice n° 1.20

On donne le script commenté pour toute forme de corrigé :

```
import numpy as np
import pylab as pl

#-- 2 flottants entre 0 et 1 pour les abscisses
x=np.linspace(0,1,2)
#-- 2 flottants entre 1 et 1 pour les ordonnées
y=np.linspace(1,1,2)

#-- on trace et on affiche [A,B], A=[0,1] et B=[1,1]
pl.plot(x,y,lw=4, color='black')
pl.show()
pl.close()      #--- ne pas l'oublier!
```

```

#-- 2 flottants entre 0 et 1/3 pour les abscisses
x1=np.linspace(0,1/3,2)
#-- 2 flottants entre 1 et 1 pour les ordonnées
y1=np.linspace(1,1,2)

#-- 2 flottants entre 2/3 et 1 pour les ordonnées
x4=np.linspace(2/3,1,2)
#-- 2 flottants entre 1 et 1 pour les ordonnées
y4=np.linspace(1,1,2)

#--- on trace deux figures (ici deux segments)
pl.plot(x4,y4, x1,y1,linewidth =4, color='black')
pl.show()
pl.close()

```

```

#--- le temps des calculs et rotations ---
z1  = 1j
z2  = 1/3+1j
z4  = 2/3+1j
w   = np.exp(1j*np.pi/3)
z3  = z2 +w*(z4-z2)
z5  = 1+1j

Z   = [z1,z2,z3,z4,z5]
X   = np.real(Z)
Y   = np.imag(Z)

#-- pour dire que l'on veut y entre 0.5 et 1.5
pl.ylim([0.5,1.5])

pl.plot(X,Y, lw=4, color='black')
pl.show()
pl.close()

```

Il faut malgré tout activer ses neurones et regarder ce que donne **np.linspace** dans le shell. On peut aussi entrer les listes à la main : $x=[0,1]$, $y=[1,1]$ et faire `plot(x,y)`... À vous de jouer, de faire joujou jusqu'à ce que cela vienne !

Corrigé de l'exercice n° 1.21

1. Sans difficulté : on écrit correctement les relations mathématiques en posant $h = (z_b - z_a)/3$ pour ne faire qu'une seule fois la division par exemple. Chaque appel coûte trois additions, une soustraction, une multiplication et une division.


```
def transformationElementaire(za, zb, w) :
    h = (zb-za)/3
    z2 = za + h
    z4 = z2 + h
    z3 = z2 + h*w
    return za,z2,z3,z4
```

2. • **Maths** : Nous commençons par définir $w = e^{i\pi/3}$. La multiplication de $(z_3 - z_1)$ par w correspond à une rotation d'angle $\pi/3$ et ainsi $(z_3 - z_2) = w(z_4 - z_2)$ ou encore $z_3 = z_2 + w h$.

• La construction de la nouvelle liste est réalisée, à partir d'une liste initialement vide, par adjonctions successives des éléments produits par **transformationElementaire**. Comme la deuxième extrémité du segment n'est pas ajoutée à ce moment-là (quatre adjonctions au lieu de cinq) il ne faut pas oublier le dernier point.

```
import numpy as np
import pylab as pl

def transformationGlobale(L) :
    theta = np.pi/3
    w = np.cos(theta) + 1j*np.sin(theta)
    n = len(L)
    Ls = []
    for i in range(0,n-1) :
        za,z2,z3,z4 = transformationElementaire(L[i], L[i+1], w)
        for z in (za,z2,z3,z4) :
            Ls.append(z)
    Ls.append(L[len(L)-1]) #— le dernier point !
    return Ls
```

3. Avec le script qui suit on construit les polygones $P_1, P_2, \dots, P_6$. Pour les afficher on fait

pl.plot(liste des abscisses, liste des ordonnées)

ce qui, dans le cas présent, est donné par les parties réelles et imaginaires et

pl.plot(np.real(L), np.imag(L), linewidth=3)

```

L=[0,1]
k=0
for x in range(1,7):
    k=k+1
    print(k)
    L = transformationGlobale(L)
    pl.plot(np.real(L), np.imag(L), linewidth=3)
    pl.close()

```

Corrigé de l'exercice n° 1.22

1. **numpy.linspace**(xmin,xmax, nx) crée la liste des points

$$xmin, xmin + \frac{xmax - xmin}{nx}, xmin + 2\frac{xmax - xmin}{nx}, \dots, xmax$$

soit nx + 1 points (à voir comme une **subdivision** de l'intervalle).

```

import numpy as np
import pylab as pl

def grille(xmin, xmax, nx, ymin, ymax, ny, g1):
    X = np.linspace(xmin, xmax, nx)
    Y = np.linspace(ymin, ymax, ny)

    for k in range(0, nx):
        pl.plot([X[k], X[k]], [ymin, ymax], color=str(g1))

    for k in range(0, ny):
        pl.plot([xmin, xmax], [Y[k], Y[k]], color=str(g1))

```

2. Sans commentaire :

```

def segment(x1, y1, x2, y2, c, e):
    pl.plot([x1, x2], [y1, y2], color=c, linewidth=e)

```

3. Sans commentaire :

```

def translater(x, y, u, v):
    return (x+u, y+v)

```

4. — $\frac{\vec{U}}{\|\vec{U}\|}$ est unitaire de même sens et direction que $\vec{U}$ et $\overrightarrow{S_1 S_2} = L_1 \frac{\vec{U}}{\|\vec{U}\|}$;
 — S_2 est donc l'image de S_1 par la translation de vecteur

$$L_1 \left(\frac{u}{\sqrt{u^2 + v^2}}, \frac{v}{\sqrt{u^2 + v^2}} \right);$$

- le rapport des longueurs est $\frac{S_2 S_3}{S_2 S_1} = \frac{L_2}{L_1}$;
- $S_3 = h \circ r(S_1)$ où h est l'homothétie de centre S_2 de rapport $\frac{L_2}{L_1}$ et r la rotation de centre S_2 et d'angle $\frac{\pi}{2}$ si le rectangle est de sens direct, $\frac{-\pi}{2}$ sinon ;
- $S_4 = h' \circ r'(S_1)$ où h' est l'homothétie de centre S_1 de rapport $\frac{L_2}{L_1}$ et $\frac{L_1}{L_2}$ et r' la rotation de centre S_1 et d'angle $\frac{-\pi}{2}$ si le rectangle est de sens direct, $\frac{+\pi}{2}$ sinon ;

```
def rectangle(x, y, u, v, L1, L2, d, c, e) :
```

```
    if d=='d' :
```

```
        i=1j
```

```
    else :
```

```
        i=-1j
```

```
    r = (u**2+v**2)**(1/2) ;
```

```
    u1, v1 = u/r, v/r
```

```
    z1 = x+y*1j
```

```
    z2 = (x+L1*u1)+(y+L1*v1)*1j
```

```
    z3 = z2-(z1-z2)*i*L2/L1
```

```
    z4 = z1+(z2-z1)*i*L2/L1
```

```
    Z = [z1, z2, z3, z4, z1]
```

```
    X = np.real(Z)
```

```
    Y = np.imag(Z)
```

```
    pl.plot(X,Y, color=c, linewidth = e )
```

5. Presque tout le travail est déjà fait, il faut penser à démarrer **rectangle** avec un premier sommet qui est l'image du point d'arrivée de la flèche provenant de l'instruction précédente par la translation de vecteur $(-h, 0)$. Contrairement aux procédures précédentes celle ci retourne une autre valeur que **None** dans le but de faciliter le recollement des instructions.

```
def instruction(x, y, h) :
```

```
    # x,y point entrée flèche
```

```
    rectangle(x-h, y, 1, 0, 2*h, h, 'i', 'black', 1.5)
```

```
    segment(x, y-h, x, y-2*h, 'black', 1.5)
```

```
    return (x,y-2*h)
```

Le dessin de l'énoncé est obtenu avec

```
grille(0 ,1, 6, 0, 1, 6, 0.75)
s1,s2 = instruction(0.5, 0.8, 0.2,' ', 12)
pl.show()
pl.close()
```

6. Explications pas à pas. Faites un dessin sur papier quadrillé pour suivre, comme vous le feriez pour programmer de façon réfléchie.
- On trace un rectangle à partir du point d'entrée (x_0, y_0) de cotés parallèles aux bissectrices du repère (on trouve le deuxième sommet dans la direction $(-1, -1)$ (ou $-\vec{i} - \vec{j}$).
 - Si on veut que la diagonale verticale soit de longueur $2h_0$, on dessinera un carré de côté $\sqrt{2}h_0$; cela explique le `lg = (2**(1/2))*h`.
 - (x_1, y_1) est le point de sortie du segment de gauche, (x_3, y_3) sa deuxième extrémité; on les obtient par translation (toujours en dessinant sur la feuille).
 - Pour (x_2, y_2) et (x_4, y_4) (la parité pour s'y retrouver).
 - Cette procédure retourne le quadruplet (tuple de longueur 4) qui permettra de placer les figures suivantes aux bons endroits.

```
def instrCond(x0, y0, h0, cg, cd) :
    lg = (2**(1/2))*h

    rectangle(x0, y0, -1, -1, lg, lg, 'd', 'black', 1.5)

    x1, y1 = translater(x0, y0, -h/2, -3*h/2)
    x3, y3 = translater(x1, y1, -cg*h, -cg*h)
    segment(x1, y1, x3, y3, 'black', 1.5)

    x2, y2 = translater(x0, y0, h/2, -3*h/2)
    x4, y4 = translater(x2, y2, cd*h, -cd*h)
    segment(x2, y2, x4, y4, 'red', 1.5)

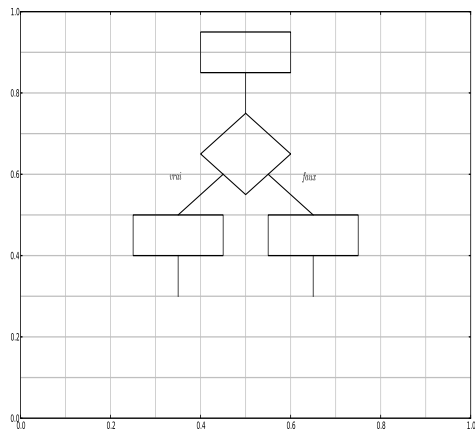
    return x3, y3, x4, y4
```

7. Avec les fonctions qui précèdent on peut faire joujou et on voit bien comment les valeurs retournées par **instruction** (un point) et **instrCond** (deux points) facilitent le travail de l'utilisateur.

```

grille(0 ,1, 6, 0, 1,6, 0.75)
h=0.1
x,y      = instruction(0.5,0.95, h,' ', 12)
a, b, c, d = instrCond(x, y, 0.1, ' ', 12,1,2)
instruction(a, b , h, ' ', 12)
instruction(c,d , h, ' ', 12)
pl.show()
pl.close()

```



Quelques algorithmes itératifs fondamentaux

On expose dans ce chapitre un petit nombre d'algorithmes simples, d'usage universel en informatique générale, que vous devrez connaître, expliquer, adapter et éventuellement analyser.

L'analyse d'un algorithme suppose avant tout que l'on donne sa **spécification**. C'est-à-dire que l'on précise sans entrer dans les détails, sur quelles données il travaille, ce qu'il fait, quels résultats il produit. Nous verrons que c'est plus facile à rédiger s'il s'agit d'une fonction pure.

On prouve ensuite que l'algorithme est conforme à cette spécification, c'est-à-dire que l'on démontre que le résultat produit est celui annoncé. Il ne suffit pas que le programme retourne un résultat, il faut aussi que ce soit le bon !

Lorsque l'algorithme contient une boucle `while`, il ne faudra pas oublier de s'assurer qu'elle (se) termine. C'est la moindre des choses, n'est-ce pas ?

Enfin, et ce sera souvent plus facile, nous chercherons à estimer le nombre des opérations de tel ou tel type que son exécution entraîne. On parle de calcul de **complexité**.

Vous avez deux niveaux de lecture de ce chapitre, ce qui vous permettra d'avancer progressivement. Une première étape consiste à comprendre le fonctionnement des fonctions exposées, à essayer de les adapter pour des problèmes proches. En deuxième lecture, vous reviendrez sur leur analyse. C'est une démarche plus théorique, mais dont l'intérêt est avant tout pratique. La complexité des fonctions confiées aux ordinateurs dans l'industrie demande que les preuves de programmes soient les plus rigoureuses possible avant toute implémentation. Le débogage est long, coûteux, difficile et peut ne jamais déboucher s'il y a eu au départ une erreur de conception.

Nous vous avouons enfin, qu'il nous est arrivé, en rédigeant une preuve de programme pour ce livre, de déceler des bugs que les essais n'avaient pas mis en évidence !

2.1 Au début était l'arithmétique

2.1.1 La division euclidienne des entiers

Nous reprenons ici l'algorithme de la division euclidienne des entiers **naturels** pour introduire les notions de preuve et de complexité de programme. Ces notions seront revues plus en détail dans le chapitre 8.

Division euclidienne des entiers positifs

```
def divEucl(a,b):
    '''
    a, b : int avec a>=0 et b >= 0.
    Renvoie le tuple (q,r) formé du quotient et du
    reste dans la DE de a par b si b >0.
    Ne fait rien et renvoie None sinon.
    '''
    if b >0:
        r = a
        q = 0
        while r>=b:
            r = r-b
            q = q+1
        return q,r
```

Spécification

- La fonction `divEucl(a,b)` prend en arguments deux entiers positifs ou nuls.
- Elle ne fait rien (et retourne `None`) si $b = 0$; si $b \neq 0$, elle retourne le tuple (q, r) tel que $a = bq + r$ avec $0 \leq r < b$.

Correction

Démontrons que ce programme donne toujours le résultat attendu.

Pour cela, notons k le numéro d'une évaluation de la condition $C(r, b) = (r \geq b)$ et montrons par récurrence sur k la proposition

$$\mathcal{P}(k) = \{r \geq 0 \text{ et } a = bq + r\}.$$

Il va de soi qu'ici, a, b, q, r désignent les contenus des variables du programme au moment de la $k^{\text{ième}}$ évaluation de la condition de la boucle. On ne les confondra pas avec le résultat obtenu seulement en fin de programme.

- **Initialisation** Lorsque $k = 1$, nous entrons dans la boucle avec $q = 0$, $r = a \geq 0$ et donc $a = bq + r : \mathcal{P}(0)$ est vraie.
- **Hérédité** Supposons qu'à la $k^{\text{ième}}$ évaluation de $C(r, b)$, l'énoncé $\mathcal{P}(k)$ soit vérifié.
 Si $C(r, b) = (r \geq b)$ est vraie, l'itération remplace r par $r' = r - b \geq 0$ et q par $q' = q + 1$ et on a $bq' + r' = b(q + 1) + (r - b) = bq + r = a$. Ainsi, $\mathcal{P}(k + 1)$ est vraie elle-aussi.
 Si $C(r, b) = (r \geq b)$ est fausse, on sort de la boucle et $\mathcal{P}(k)$ ne subit aucune modification.

Nous avons montré que $\mathcal{P}(k)$ est toujours vraie. Si la boucle termine après k itérations, on aura à la fois $\mathcal{P}(k)$ et la condition d'arrêt $\text{non}(C(r, b)) = (r < b)$ ce qui

$$\text{s'exprime } \begin{cases} a = bq + r \\ 0 \leq r < b \end{cases}$$

Démontrons que ce programme termine.

Notons $r_0 = a$ et si la condition d'entrée dans la boucle **while** est satisfaite au moins une fois, notons r_j la valeur de la variable r à l'issue de la $j^{\text{ième}}$ itération. On obtient une suite d'entiers positifs strictement décroissante. Elle est nécessairement finie.

Complexité

Le nombre de soustractions ou d'itérations est égal à q et $q \leq \lfloor \frac{a}{b} \rfloor$.

Invariant de boucle, variant de boucle (ou fonction de terminaison)

C'était là notre première preuve de programme. Pour la mener à bien, nous avons introduit une propriété $\mathcal{P}(k)$ dont nous avons prouvé par récurrence qu'elle était toujours vérifiée au fil des itérations. $\mathcal{P}(k)$ est ce que l'on appelle un **invariant de boucle**.

Quant à la suite d'entiers positifs $(r_k)_k$ strictement décroissante qui nous a permis de prouver que l'algorithme termine, c'est un **variant de boucle** (ou fonction de terminaison).

Exercice 2.1 division euclidienne dans $\mathbb{Z}$

On définit le quotient et le reste de la division euclidienne de a par $b \neq 0$ dans $\mathbb{Z}$ de

la façon suivante : (q, r) est l'unique couple d'entiers relatifs tel que
$$\begin{cases} a = bq + r \\ 0 \leq r < |b| \end{cases}$$

1. Prendre un papier, un crayon, $a = -123$, $b = -32$. Poser $r = a$. Tant que $r < 0$, ajouter $|b|$ à r .
2. Toujours avec vos papier, crayon, prendre $a = -123$, $b = 32$. Poser $r = a$. Tant que $r < 0$, ajouter $|b|$ à r .

3. * Écrire une fonction `divEuclZ(a,b)` qui prend en arguments deux entiers relatifs a et b et retourne le couple (q, r) lorsque $b \neq 0$, rien sinon (`none`). Séparer les cas $a \geq 0$ et $a < 0$.
4. Trouver un invariant de boucle pour chaque boucle de votre programme.

Corrigé en 2.1 page 122.

2.1.2 L'écriture binaire d'un entier

On sait que tout nombre entier strictement positif peut s'écrire comme somme de puissances de 2.

Théorème 2.1 *écriture binaire d'un entier*

Pour tout nombre entier $n \in \mathbb{N}$, il existe une suite $(c_0, c_1, \dots, c_p, 0, \dots, 0, \dots)$ et une seule vérifiant les propriétés suivantes

1. pour tout indice $i \geq 0$, $c_i \in \{0, 1\}$;
2. il existe un rang $p \in \mathbb{N}$ à partir duquel $c_i = 0$ (ce qui signifie que pour tout indice i , $i \geq p + 1 \Rightarrow c_i = 0$);
3.
$$n = c_0 2^0 + c_1 2^1 + c_2 2^2 + \dots + c_p 2^p = \sum_{i=0}^{\infty} c_i 2^i.$$

On dit que les $(c_i)_i$ sont les **coefficients binaires** (ou par abus de langage, les chiffres binaires) de n et on note $n = c_p \dots c_0(2)$ son **écriture en base 2**. Par exemple

$$37 = 2^5 + 2^2 + 1 = \sum_{k=0}^{\infty} c_k 2^k = 100101_{(2)}$$

Nous montrons comment déterminer l'écriture en base 2 sans faire appel à d'autres opérations que + et -

Exercice 2.2 *écriture binaire d'un entier*

Dans cet exercice les seules opérations arithmétiques admises sont + et -.

1. Que fait le programme ci-contre dans lequel n est un entier supérieur ou égal à 1 ? Donner un invariant de boucle et préciser une relation entre n , i et d après l'arrêt. On exprimera en particulier le lien entre i et $\lfloor \log_2(n) \rfloor$.

```
i, d = 0, 1
while d+d <= n:
    i += 1
    d += d
```

2. On dira que l'exposant i figure dans la décomposition en base 2 de n si $c_i = 1$. Par exemple, seuls 5, 2 et 0 figurent dans la décomposition de $37 = 2^5 + 2^2 + 2^0$. Écrire une fonction `exposants_binaires(n)` qui prend en argument un entier positif n et renvoie la liste strictement décroissante (éventuellement vide) des exposants qui figurent dans la décomposition en base 2 de n .

3. Écrire une fonction `binaire(n)` qui prend en argument un entier positif n et renvoie la liste des chiffres binaires de n .

Par exemple `binaire(37)` renvoie `[1,0,0,1,0,1]`.

Corrigé 2.2 page 122.

La fonction `bin()` de Python

Python dispose d'une fonction native `bin()` qui prend en argument un entier n et renvoie une chaîne débutant par `'0b'` ou `'-0b'` suivie des chiffres binaires de n .

```
>>> bin(37)
'0b100101'
>>> bin(-37)
'-0b100101'
>>> bin(37.)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'float' object cannot be interpreted
as an integer
```

Exercice 2.3 utiliser `bin()`

Écrire en utilisant la fonction native `bin()` une brève fonction qui prend en argument un entier n et renvoie la liste dont le premier terme sera `'+'` ou `'-'` selon le signe de n et les suivants les **coefficients** binaires de n .

Indications : penser à la méthode `split()` des chaînes.

Corrigé 2.3 page 124.

2.2 Calcul de la moyenne et de la variance

Nous montrons ici comment on calcule conjointement la moyenne et la variance d'une suite statistique avec la formule de Huygens en commençant par une suite simple, puis par une suite de valeurs pondérées. Dans la foulée, nous proposons après un bref rappel mathématique, le calcul quasi-identique d'une intégrale par la méthode des trapèzes. Mais nous verrons plus loin (calculs numériques approchés) que la méthode n'est pas numériquement stable.

2.2.1 Calcul conjoint et analyse

• On rappelle que la moyenne d'une suite statistique finie $x = (x_0, x_1, \dots, x_{n-1})$, notée tantôt $\bar{x}$, tantôt $\langle x \rangle$, est définie par

$$\langle x \rangle = \frac{1}{n} \sum_{k=0}^{n-1} x_k.$$

La variance de x est la moyenne du carré de la distance de X à $\langle x \rangle$, à savoir :

$$\text{Var}(x) = \left\langle (x - \langle x \rangle)^2 \right\rangle = \frac{1}{n} \sum_{k=0}^{n-1} (x_k - \langle x \rangle)^2.$$

On ne calcule évidemment jamais la variance à partir de cette définition, mais en observant que pour tout k , $(x_k - \langle x \rangle)^2 = x_k^2 - 2 \langle x \rangle x_k + \langle x \rangle^2$, ce qui en sommant et moyennant donne

$$\left\langle (x - \langle x \rangle)^2 \right\rangle = \frac{1}{n} \sum_{k=0}^{n-1} x_k^2 - \frac{2}{n} \langle x \rangle \sum_{k=0}^{n-1} x_k + \frac{1}{n} \sum_{k=0}^{n-1} \langle x \rangle^2$$

qui se simplifie en la formule de Huygens :

$$\text{Var}(x) = \left\langle (x - \langle x \rangle)^2 \right\rangle = \langle x^2 \rangle - \langle x \rangle^2 \quad (2.2.1)$$

• Lorsque les relevés statistiques sont fournis sous la forme de couples valeur-effectif partiel, $(x_k, n_k)_{0 \leq k \leq p-1}$ (en pratique sous forme d'un tableau), on a par associativité :

$$\langle x \rangle = \frac{1}{n} \sum_{k=0}^{p-1} n_k x_k, \quad \langle x^2 \rangle = \frac{1}{n} \sum_{k=0}^{p-1} n_k x_k^2 \quad \text{avec} \quad n = \sum_{k=0}^{p-1} n_k.$$

Suite simple	Suite pondérée
<pre>def moyVar(X): n = len(X) if n==0: return None else: s1, s2 = 0, 0 for x in X: s1 = s1+x s2 = s2+x*x m = s1/n return m, s2/n-m**2</pre>	<pre>def moyVar(X,N): p1, p2 = len(X), len(N) if p1==0 or p2 !=p1: return None else: s1, s2, n = 0, 0, 0 for k in range(0,p1): n = n+N[k] z = N[k]*X[k] s1 = s1 + z s2 = s2 + z*X[k] m = s1/n return m, s2/n-m**2</pre>

Nous ferons l’analyse du calcul de la moyenne et de la variance pour des données pondérées (colonne de droite).

Spécification

- La fonction `moyVar(X, N)` prend en arguments deux tableaux (ou listes) X et Y contenant respectivement des flottants et des entiers.
- Elle retourne `None` si les listes sont vides ou de taille différentes. Sinon, elle retourne un couple formé de la moyenne et de la variance de la suite de valeurs pondérées $(X[k], N[k])$.

Correction

Cette itération non conditionnelle ne pose pas de problème : $p1$ est la longueur de X et de Y ; les variables $n, s1$ et $s2$ sont initialisée à 0. A l’issue des $p1$ itérations les deux tableaux ont été parcourus dans l’ordre et $s1, s2, n$ contiennent respectivement la somme des éléments de X , la somme des carrés des éléments de X , et la somme des éléments de N ¹.

Complexité

Deux divisions, une soustraction, $3p$ additions, $2p$ multiplications, p étant la longueur

1. Nous n’en dirons pas plus : il n’y a pas lieu de redémontrer ce qui est explicité dans le tableau de la page 36.

des listes. On observera l'usage de la variable z qui permet d'économiser multiplications et élévations au carré, car sans elle on aurait fait dans la boucle :

```
n      = n      + N[k]
s1     = s1     + N[k] * X[k]
s2     = s2     + N[k] * X[k] ** 2
```

2.2.2 Méthode des trapèzes

Calcul approché d'une intégrale, méthode des trapèzes

Soient f une fonction de classe C^3 sur l'intervalle $I[a, b]$ et $t_0 = a < t_1 < t_2 < \dots < t_n = b$, des points formant une subdivision de $[a, b]$.

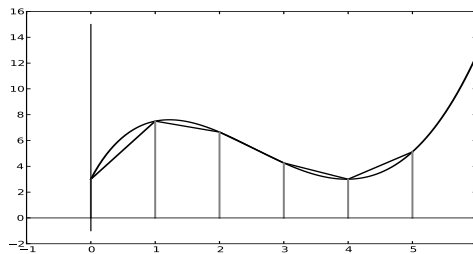
La somme des aires des trapèzes construits comme sur la figure est donnée par la formule :

$$A(f, t_0, \dots, t_n) = \sum_{j=0}^{n-1} (t_{j+1} - t_j) \frac{f(t_{j+1}) + f(t_j)}{2}.$$

On démontre que l'écart entre cette somme et l'intégrale de f sur $[a, b]$ vérifie :

$$\left| \int_a^b f(s) ds - A(f, t_0, \dots, t_n) \right| \leq \frac{h^2}{12} |b - a| M_3$$

où h est le plus grand des $|t_{j+1} - t_j|$, M_3 un majorant de $|f'''(x)|$ sur $[a, b]$.



Exercice 2.4 calcul d'intégrales, méthode des trapèzes

1. Donner une formule exprimant la somme des aires $A(f, t_0, \dots, t_n)$ de la façon la plus adaptée au calcul lorsque les écarts $t_{j+1} - t_j$ sont tous égaux.
2. Écrire une fonction `trapezes(f, a, b, n)` qui prend en arguments une fonction f de type flottant → flottant, deux flottants a et b , un entier n et retourne la valeur approchée de l'intégrale de f sur $[a, b]$ donnée par la méthode des trapèzes pour une subdivision régulière de n intervalles.

3. Tester avec des fonctions dont vous connaissez l'intégrale (polynômes, fonctions trigonométriques...).
4. Dans la réalité, les fonctions sont le plus souvent données par des relevés expérimentaux où l'on connaît des valeurs approchées

$$y_0 \approx f(t_0), y_1 \approx f(t_1), \dots, y_n \approx f(t_n)$$

et rien d'autre. On donc suppose que, comme dans la vraie vie des scientifiques,

- on dispose d'un tableau de flottants $[y_0, y_1, \dots, y_{48000}]$ correspondant à des relevés régulièrement espacés d'un signal $f(t)$ pendant une durée d'une seconde (vous avez enregistré votre voix dans goldwave pendant une seconde par exemple et en avez tiré un fichier de 48000 flottants) ;
- on veut calculer $\int_{t_0}^{t_0+1} f(t)e^{i\omega t} dt$ pour une certaine valeur de $\omega \in \mathbb{R}$ connue.

Proposer une solution pratique.

Indications :

- ne dites pas 'je ne connais pas la fonction' car vous n'en connaîtrez jamais une expression formelle lorsque vous travaillerez sur des fichiers de valeurs expérimentales ;
- ne dites pas non plus 'je suppose que la fonction f est connue et j'utilise le programme précédent,' ce qui pourrait amener votre interlocutrice à penser que vous vous fichez d'elle !
- Voilà à quoi ressemblera le fichier d'enregistrement de votre voix :

```
[ASCII 48000Hz, Channels:2, Samples:332640, Flags:0]
0.000000
...
0.002979
-0.002634
0.003986
```

Corrigé en 2.4 page 124.

2.3 Algorithmes de recherche séquentielle

2.3.1 Listes ou tableaux

Les algorithmes que nous présentons dans cette section traitent indifféremment des listes ou des tableaux unidimensionnels. Les tableaux que nous considérerons, comme dans tout cet ouvrage, sont les **tableaux de la bibliothèque numpy**. Rappelons quelques différences fondamentales entre les listes et les tableaux de numpy :

- On peut étendre une liste et lui ajouter des éléments. Ce n'est pas le cas pour un tableau (de type `numpy.ndarray`) dont nous considérerons que la taille est fixée une fois pour toutes².
- Une liste peut contenir des éléments de types différents. Les tableaux contiennent des éléments qui sont tous convertibles dans un même type (défini avec l'option **nd-type**).
- Nous verrons aussi que le test d'appartenance implémenté dans Python (`x in T`) n'a pas le même coût dans une liste ou dans un tableau (page 110).

L'attribut **size** des tableaux (`T.size`) est un entier (tableau unidimensionnel) ou un tuple d'entiers (tableau multidimensionnels comme ceux qui représentent des matrices). Dans le cas d'un tableau unidimensionnel, `len(T) = T.size`.

2.3.2 Recherche d'un élément dans une liste

Boucle for	Boucle while
<pre>def RechercheF(L,e): ''' L : list numpy.ndarray; ''' r = False for k in L: if e==k: r = True return r</pre>	<pre>def RechercheW(L,e): ''' L : list numpy.ndarray; ''' n, k = len(L), 0 while k < n and e!=L[k]: k=k+1 return k!=n</pre>

• Analysons RechercheF

Correction

`RechercheF(L, e)` retourne le résultat attendu : en effet, toute la liste est parcourue et la variable `r`, initialisée à `False`, reçoit `True` en affectation chaque fois que `e == L[k]` et elle n'est pas modifiée dans les autres cas. Sa valeur booléenne est donc celle de $e \in L$.

Complexité

Un appel à la procédure provoque $n = \text{len}(L)$ itérations dans tous les cas ce qui fait n comparaisons ($e == k$).

2. Nous n'utiliserons donc pas de la méthode `reshape` sauf pour l'instruction de création du tableau.

• Analyses RechercheW

Correction

Montrons tout d'abord que la boucle termine : la condition est évaluée une première fois avec $k = 0$ et k est incrémenté d'une unité à chaque itération. Si la boucle ne terminait pas, la suite des entiers serait parcourue dans l'ordre. Ce n'est pas possible parce qu'alors on aurait réalisé $k = n$.

Montrons que le résultat est conforme à la spécification. Pour cela il nous faut un invariant de boucle. Il ne se lit pas dans le programme (hélas !) mais il existe quand-même. Notons p le numéro de l'évaluation de la condition d'entrée dans la boucle et posons

$$\mathcal{P}(p) = \{k = p - 1 \text{ et } (0 \leq j \leq k - 1 \Rightarrow L[j] \neq e)\}$$

(en langage courant : au moment de la $p^{\text{ième}}$ évaluation, k contient $p - 1$ et pour les indices strictement plus petits $L[j] \neq e$).

- $\mathcal{P}(1)$ est vraie car alors $k = 0$ (initialisation) et il n'y a pas d'indice plus petit ;
- si $\mathcal{P}(p)$ est vraie, soit il n'y a pas d'autre évaluation et on sort de la boucle ; soit il y a une itération de plus ce qui suppose qu'à l'évaluation numéro p , avec $k = p - 1$, on a $L[k] = L[p - 1] \neq e$.

Lors de l'évaluation suivante, de n° $p + 1$ on a, $p = (p + 1) - 1 = k$ et $\mathcal{P}(p + 1)$ reste vraie car on a remplacé k par $k + 1$.

Complexité

Le nombre d'itérations dépend ici de L et de e .

Si e est dans L , il est égal à l'indice de la première position de e dans L augmenté de 1 (on commence à 0) ; sinon c'est n .

On peut aussi s'intéresser au nombre moyen de comparaisons, mais un calcul effectif supposerait que l'on sache attribuer des probabilités aux événements ($e \in L$), ($place(e) = k$)...

• Résumons

- **RechercheF**(L, e) : complexité en n comparaisons $e == k$;
- **RechercheW**(L, e) : complexité dans le meilleur des cas si la liste est non vide : une comparaison $e \neq L[k]$; dans le pire des cas : n comparaisons.
- On considère une liste ou un tableau L et un objet e . On souhaite tout d'abord tester l'appartenance de e à L . Des primitives de Python réalisent ce travail mais notre objectif est de comprendre comment on les réalise. Nous commencerons par deux fonctions qui ont la même spécification :
 - RechercheX(L, e) prend en arguments une liste (ou un tableau) L et un objet e .
 - Elle retourne True si e est élément de L , False sinon.

Exercice 2.5 *adaptation des algorithmes précédents*

- 1. Écrire une fonction `nbreOccurrences(L, e)` qui prend en arguments un tableau ou un conteneur `L`, un objet `e` et retourne le nombre d’occurrences de `e` dans `L`.
- 2. Écrire une fonction `listeOccurrences(L, e)` qui prend en arguments un tableau ou un conteneur `L`, un objet `e` et retourne la liste, éventuellement vide, des positions de `e` dans `L`.
Proposer deux versions : en programmation impérative, et en programmation fonctionnelle avec une lambda fonction qui définit la liste des indices en compréhension.

Corrigé en 2.5 page 126.

Exercice 2.6 *adaptation des algorithmes précédents*

- 1. Écrire une fonction `premOccurrence(L, e)` qui prend en arguments un tableau ou un conteneur `L`, un objet `e` et retourne l’indice de la première occurrence de `e` dans `L` si `e` appartient à `L`, rien sinon.
- 2. Donner une preuve de correction de votre programme (*ie* : montrer qu’il est conforme à la spécification).

Indications :

- (a) Si votre programme contient une instruction conditionnelle, analyser chacun des cas de manière distincte.
- (b) S’il comporte une boucle, prouver qu’elle termine, trouver un invariant de boucle pertinent.

Corrigé en 2.6 page 127.

2.3.3 Recherche des plus grands éléments d’une liste

Nous présentons ici le problème de la recherche du plus grand élément d’une liste ou d’un tableau non trié(e). C’est encore un problème qui admet comme solution un algorithme simple mais dont l’étude et la connaissance sont fondamentales. Il est bien sûr déjà implémenté avec la fonction `max` de Python.

Le maximum	Le maximum avec sa position
<pre>def maximum(L): m = L[0] for k in range(1, len(L)): if L[k] > m: m = L[k] return m</pre>	<pre>def maximum(L): p, m = 0, L[0] for ind, val in enumerate(L): if val > m: p, m = ind, val return m, p</pre>

Nous en proposons deux versions : à gauche, la fonction retourne le maximum de la liste ; à droite, la fonction retourne un couple formé du maximum et de sa première position dans la liste. Analysons la première.

Spécification

La fonction `maximum(L)` du tableau de gauche prend une liste ou un tableau non vide d'entiers ou de flottants et retourne le maximum de la liste.

Correction (dans un cas aussi simple, on peut s'en dispenser)

Considérons l'énoncé $\mathcal{P}(k)$:

au cours de la $k^{\text{ième}}$ itération, au moment de la comparaison dans l'instruction conditionnelle, m contient le maximum de la liste partielle $[L[0], \dots, L[k-1]]$.

- $\mathcal{P}(1)$ est vraie puisque m est initialisée à $L[0]$ et que la première itération est indexée sur $k = 1$;

- si $\mathcal{P}(k)$ est vraie :

- soit $L[k] > m$ et $L[k]$ est donc plus grand que les éléments du début de liste.

Après l'affectation $m = L[k]$, si l'itération suivante a lieu, $\mathcal{P}(k+1)$ est vérifiée.

- soit $L[k] < m$ et m reste le maximum de $[L[0], \dots, L[k]]$; à l'itération suivante, si elle a lieu $\mathcal{P}(k+1)$ est toujours vérifiée.

Complexité

- Nombre de comparaisons : il ne dépend pas du contenu de la liste ; c'est $n - 1$, (n étant la longueur de la liste).

- Nombre d'affectations : dans le pire des cas (lorsque le tableau est trié dans l'ordre croissant) il y a n affectations (initialisation et chaque itération). Dans le meilleur des cas (lorsque le maximum est en début de liste), il n'y a qu'une affectation.

L'écart entre les complexités dans le pire des cas et dans le meilleur des cas nous incite à mettre en avant le nombre moyen d'affectations. Pour cela, nous renvoyons au chapitre 8.

Exercice 2.7 modifications légères

1. Réécrire la fonction `maximum` (avec sa position) sans `enumerate`.
2. Que se passe-t-il si dans la fonction `maximum(L)` du tableau de droite on remplace l'inégalité stricte par une inégalité large ?
3. Modifier la fonction `maximum` pour obtenir la spécification suivante :
La fonction `maximum(L)` prend une liste ou un tableau éventuellement vide d'entiers ou de flottants et retourne `None` si la liste est vide, le maximum de la liste sinon.

Corrigé en 2.7 page 128.

On se propose maintenant d'écrire une fonction qui prend une liste en argument et renvoie ses deux plus grands éléments (ils peuvent être égaux) avec leurs premières positions dans la liste.

Premier et deuxième plus grands éléments d'une liste

```
def recherche_2nd_max(L):
    """
    L : list ou tableau unidimensionnel avec len(L) > 2;
    Renvoie le tuple de tuples (m1,p1), (m2,p2)
    avec m1 >= m2 les deux plus grands éléments
    rencontrés et p1, p2 leurs premières positions dans L.
    """
    assert isinstance(L, list) and len(L) >= 2
    m1, m2, p1, p2 = ?
    if m2 > m1:
        ??
    for i in range(2, len(L)):
        e = L[i]
        if e > m1:
            ???
        elif e > m2:
            ???
    return return (m1, m2), (p1, p2)
```

Exercice 2.8 premier et second maximum

L'encart précédent vous propose un schéma pour une fonction répondant à la spécification.

1. Préparer une batterie de listes avec lesquelles vous testerez votre programme. On cherchera à envisager plusieurs cas : unicité du premier et du deuxième maximum, unicité du premier et plusieurs occurrences du second, plusieurs occurrences du maximum.
2. En vous inspirant de l'algorithme de recherche du plus grand élément, remplacer '?'.
3. Remplacer '??'. A quoi sert cette instruction conditionnelle ?
4. Compléter judicieusement le programme de telle sorte que m1, m2 soient les deux plus grandes valeurs de la liste et p1, p2 les indices des premières occurrences de ces valeurs (si m1 = m2, p1 et p2 sont les indices des deux premières occurrences du maximum).
5. Quel est le nombre maximum de comparaisons pour une liste de longueur n ?
6. Déterminer un invariant de boucle vous permettant de prouver que votre programme est conforme à la spécification. On notera ℓ_i la liste des i premiers éléments de L ($\ell_2 = [L[0], L[1]]$ par exemple), pour formuler cet invariant.

Corrigé 2.8 page 129.

Exercice 2.9 *premier et deuxième plus grand élément d'une liste (algorithme naïf ?)*

1. On veut déterminer les emplacements des deux plus grands éléments d'une liste. On s'autorise des méthodes suivantes :

- `L.pop(i)` qui retourne l'élément `L[i]` et le supprime de la liste ;
- `L.insert(i,x)` qui insère l'élément `x` dans la liste `L` en position d'indice `i`.

- (a) Écrire une fonction `maximum1et2(L)` qui prend en argument une liste `L`, de longueur au moins deux et retourne $((m1, m2), (p1, p2))$ formé du tuple des deux plus grands éléments suivis du tuple des indices de leurs premières occurrences.

On propose la programmation suivante : cette fonction utilise la recherche du maximum (tableau de droite) de la page 106 pour déterminer `m1` et `p1` puis refait la même chose sur la liste privée de son plus grand élément.

Attention : à l'issue de cette procédure, la liste `L` doit se retrouver dans son état initial.

- (b) Déterminer le nombre de comparaisons dans cette fonction. Quelle remarque cela vous inspire-t-il si vous comparez à la complexité de l'algorithme de l'exercice (2.8) ?

Conseils :

- comme toujours avant d'utiliser dans votre programme des fonctions ou méthodes du langage, regardez comment elles fonctionnent avec des exemples que vous choisirez ; vérifiez que tout cela est conforme à la spécification (voir page 28) ;

- une fois que vous maîtrisez les outils, vous pouvez écrire votre algorithme sur feuille ; surveillez sa spécification ;

- vous pourrez ensuite passer à la programmation, pensez à vérifier tous les cas de figure : le premier maximum arrive-t-il avant ou après le second ? Cela change-t-il quelque chose ?

2. Réécrire cette fonction pour un tableau de même spécification que les tableaux de numpy en argument (donc sans `pop()` ni `insert()` puisqu'un **tableau est immuable** : sa taille ne varie pas).

Corrigé en 2.9 page 131.

Exercice 2.10 *tri par sélection, tri de complexité quadratique (inefficace ?)*

On rappelle que si `L` est une liste ou un tableau (numpy.ndarray), lors d'un appel de fonction avec `L` en paramètre, des affectations `L[i] = ..` modifient la liste `L` (voir page 48).

1. Écrire une procédure qui prend un tableau en argument et trie ses éléments dans l'ordre croissant. Cette procédure ne retourne pas de valeur (None par défaut).
Indication : modifier la fonction maximum pour qu'elle recherche le maximum à partir d'une certaine position.

2. Quel est le nombre de comparaisons de cette fonction ?

Corrigé en (2.10) page 132.

Une étude plus systématique des tris est proposée au chapitre 4.

2.3.4 Dictionnaires et comptage

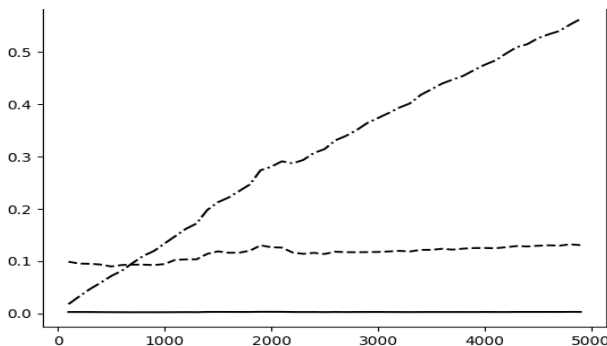
Ce paragraphe est destiné à mettre en avant des situations dans lesquelles l'usage des dictionnaires (ou tableaux associatifs) peut s'avérer d'une grande utilité. Cette question, que nous effleurons ici, sera détaillée et approfondie au chapitre 12.

• Recherche dans une liste, un tableau, un dictionnaire.

Nous commençons par une étude empirique des coûts de la recherche d'un élément dans une liste, un tableau (de numpy) ou **parmi les clés** d'un dictionnaire.

Le graphique ci-dessous, qui permet la comparaison de ces coûts, est issu du script de la page 142 qui :

- Pour chaque entier $n = 100, 200, 300, \dots, 4900$ crée un dictionnaire D (vide), une liste L (vide) et un tableau (numpy.ndarray) de taille n .
- « Tire au sort » n entiers aléatoires compris entre 0 et 10 000, qu'il range dans le tableau, la liste et le dictionnaire (on aura $\text{len}(D) \leq \text{len}(T) = \text{len}(L) = n$, la taille du dictionnaire étant légèrement inférieure à n à cause des répétitions).
- Une fois ces trois conteneurs créés, ce script teste, pour chaque entier k entre 0 et 10 000, l'appartenance de k à la liste, au tableau et au dictionnaire. Les **temps de calculs cumulés** de chaque instruction $k \text{ in } D$, $k \text{ in } T$ ou $k \text{ in } L$ sont relevés et figurent sur le graphique. La ligne continue correspond aux temps de recherche dans les dictionnaires, la ligne formée de - - correspond aux tableaux et la dernière aux listes.



Coûts de 10 000 recherches dans des conteneurs de tailles 100, 200, ..., 4900.

Ce graphique met en évidence un coût moyen linéaire (en $O(n)$) en fonction de la taille de la liste pour la recherche d'un élément, ce qui est conforme à nos études ; constant (en $O(1)$) pour la recherche d'une clé dans un dictionnaire.

Pour ce qui est des tableaux (du type `numpy.ndarray`) l'avantage par rapport aux listes est évident dès que n est assez grand. A noter : les variations locales des courbes sont dues au fait que d'autres processus fonctionnent en même temps que notre script.

• Utilisation d'un dictionnaire en boîte noire

Le script de la page 112 dont l'exercice (2.11) propose l'analyse, nous montre comment on peut utiliser un dictionnaire en boîte noire (le dictionnaire n'existe que localement) pour accélérer (ici considérablement) une recherche.

Exercice 2.11 *première utilisation d'un dictionnaire (ou tableau associatif)*

1. Considérons le script de la colonne gauche de la page 112.

- (a) Quelle est la fonction de l'instruction
`if not q in L: ...` ? Quelle dire de la liste que retourne cet algorithme ?
- (b) Exprimer la probabilité que $q \in L$ au moment du test en fonction de la longueur $n = \text{len}(L)$ à ce moment là et de a et b . Donner un majorant en fonction de N .
- (c) Quelle est la complexité en nombre de comparaisons cachées de la fonction si on suppose qu'à chaque itération on ajoute un élément à la liste ? Cette hypothèse conduit elle à majorer ou à minorer la complexité ?

2. Considérons maintenant le script de la colonne de droite.

- (a) Qu'y a-t-il de changé par rapport au script de la colonne de gauche ? Que penser de l'occupation mémoire ?
- (b) En regardant les temps de calcul, quelle hypothèse feriez vous quant à la complexité du test `q in D` lorsque D est un dictionnaire ?

Corrigé en 2.11 page 133.

Listes aléatoires sans répétition, l'intérêt du dictionnaire

```

import random as rd
import time

def liste_aleatoire(N, a, b):
    L, n = [], 0

    while n < N:
        q = rd.randint(a, b)
        if not q in L:
            L.append(q)
            n += 1
    return L

for k in range(2, 5):
    t0 = time.time()
    liste_aleatoire(
        10**k, 10**9, 10**10)
    t1 = time.time()
    print('k = %s - temps
    de calcul:\n'%(k), t1-t0)

k = 2 - temps de calcul:
0.0004260540008544922
k = 3 - temps de calcul:
0.009874105453491211
k = 4 - temps de calcul:
0.905580997467041
k = 5 - temps de calcul:
82.12676239013672

```

```

import random as rd
import time

def liste_aleatoire_1(N,a,b):
    L, D, n = [], { }, 0
    while n < N:
        q = rd.randint(a, b)
        if not q in D:
            L.append(q)
            D[q] = n
            n += 1
    return L

for k in range(2, 5):
    t0 = time.time()
    liste_aleatoire_1(
        10**k, 10**9, 10**10)
    t1 = time.time()
    print('k = %s - temps
    de calcul:\n'%(k), t1-t0)

k = 2 - temps de calcul:
0.00022459030151367188
k = 3 - temps de calcul:
0.0016520023345947266
k = 4 - temps de calcul:
0.017082929611206055
k = 5 - temps de calcul:
0.18847990036010742
k = 6 - temps de calcul:
1.9241294860839844
k = 7 - temps de calcul:
19.361234188079834

```

• Compter les occurrences des éléments dans une liste

Tri par comptage sur un tableau ou une liste d'entiers

Le tri par comptage prend en argument une liste ou un tableau T d'entiers dont on connaît un minorant m et un majorant M . L'idée est alors de relever les occurrences de chacun des entiers compris entre m et M dans le tableau puis de réserver un autre tableau de même taille que T , que l'on remplit à la volée : le premier élément rencontré est placé en début de $T1$ autant de fois qu'il apparaît dans T et ainsi de suite.

```
def comptage(L):
    """
    L: list ou np.array (à une dimension).
    """
    assert isinstance(L, (np.ndarray, list))
    D = {}
    for i, e in enumerate(L):
        if e in D:
            D[e] += 1
        else:
            D[e] = 1
    return D

def triComptage(T, M):
    """
    T : numpy.ndarray (tableau) ou list d'entiers >=0.
    M : int;
    """
    assert isinstance(T, (np.ndarray, list))
    assert isinstance(M, (np.int64, int))

    T1 = np.zeros(T.size, dtype = int)
    D = comptage(T)
    p, i = 0, 0

    for i in range(0, M+1):
        if i in D:
            for q in range(p, p + D[i]):
                T1[q] = i
            p += D[i]

    return T1
```

Pour compter les occurrences des éléments d'une liste ou d'un tableau, les fréquences de mots dans un texte, il y a tout intérêt tant du point de vue de la simplicité de la

programmation que de l'efficacité du programme à faire un usage systématique des dictionnaires.

C'est l'objet de la fonction `comptage(L)` qui prend une liste ou un tableau en argument et renvoie un dictionnaire { élément de L : nombre d'occurrences de l'élément dans L,... }.

Le tri par comptage permet de trier une liste ou un tableau d'entiers. Il est linéaire (le nombre d'opérations fondamentales est proportionnel à la taille de la liste) mais demande de la mémoire (on réserve un dictionnaire dont la taille est sensiblement celle de la liste) et les pré-conditions sont drastiques : on trie des entiers et on doit connaître un encadrement des termes de la liste (donc éventuellement chercher au préalable le min et le max, ce qui reste linéaire).

2.4 Boucles imbriquées

Les algorithmes opérant sur des listes ou tableaux unidimensionnels que nous avons jusqu'à présent rencontrés dans ce chapitre font intervenir des traitements à la volée, réalisés au fil des itérations d'une unique boucle. On présente ici des parcours où le traitement nécessite lui aussi une boucle. Plusieurs boucles seront alors imbriquées dans les algorithmes correspondants.

2.4.1 Valeurs les plus proches dans un tableau

Recherche des valeurs les plus proches dans une liste (ou un tableau)

Soit L une liste ou un tableau d'entiers de deux éléments ou plus. On recherche un couple (p, q) tel que les valeurs $L[p]$ et $L[q]$ soient les plus proches parmi celles de la liste. Ce qui s'exprime :

- $0 \leq p < q \leq \text{len}(L) - 1$;
- $|L[q] - L[p]| = \inf\{|L[i] - L[j]| \mid 0 \leq i < j \leq \text{len}(L) - 1\}$.

Pour résoudre ce problème on propose la méthode suivante :

- réserver un dictionnaire D ;
- pour chaque indice $0 \leq i \leq \text{len}(L) - 1$, placer dans $D[i]$ le couple $(j, |L[i] - L[j]|)$ qui réalise le minimum de la distance $|L[i] - L[j]|$ lorsque $j > i$.
- une fois ce dictionnaire construit le parcourir pour rechercher le couple (p, q) .

Exercice 2.12 éléments les plus proches dans un tableau

On considère une liste L de longueur supérieure ou égale à 2 dont les éléments sont des entiers. On notera n sa longueur ($n \geq 2$).

1. On initialise un dictionnaire avec $D = \{ \}$; $D[0] = (1, \text{abs}(L[0]-L[1]))$. Écrire un programme qui, parcourant la liste à partir de $L[2]$, permet de placer dans $D[0]$ le couple $(j, \text{abs}(L[0]-L[j]))$ réalisant le minimum de $\text{abs}(L[0]-L[j])$ avec $j \geq 1$.
2. Écrire une fonction qui prend une liste L en argument (vérifiant les pré-conditions énumérées dans le préambule) et renvoie un dictionnaire D dont les clés sont les indices $i = 0, 1, \dots, n-2$, la valeur associée à i étant un couple $(j, \text{abs}(L[i] - L[j]))$ tel que $j > i$ et $\text{abs}(L[i] - L[j])$ soit minimale.
 Votre programme présentera deux boucles, la première indexée sur i variant de 0 à $n-2$, la seconde imbriquée dans la première indexée sur j variant de $i+2$ à $n-1$.
3. Exprimer le nombre d'itérations en fonction de n .
4. Écrire une fonction qui prend une liste L en argument et renvoie un triplet (p, q, m) avec $0 \leq p < q \leq n-1$ et $m = |L[p] - L[q]|$ réalisant le minimum des $|L[i] - L[j]|$ avec $0 \leq i < j \leq n-1$.
 Vous pourrez utiliser la fonction précédente comme fonction auxiliaire ou bien compléter le code pour réaliser la version finale.
5. Exprimer le nombre d'itérations en fonction de n . Cela change-t-il fondamentalement les choses par rapport à la seule construction du dictionnaire ?

Corrigé en 2.12 page 136.

```
[143, 272, 988, 602, 313, 740, 121, 146, 23, 222]
(0, 7, 3)

[737, 516, 641, 117, 987, 883, 544, 229, 369, 644]
(2, 9, 3)

[507, 157, 717, 927, 887, 39, 231, 497, 278, 455]
(0, 7, 10)
```

2.4.2 Recherche d'une sous-chaîne dans une chaîne de caractères

Nous allons étudier dans ce paragraphe quelques algorithmes de traitement de textes, c'est-à-dire de données formées de chaînes de caractères. Les algorithmes que nous mettrons en évidence, en particulier la recherche de mots (ou de motifs) sont d'une grande importance dans des domaines variés : traitements de texte au sens grand public, recherche d'information, logiciel de programmation (compilateurs, interpréteurs), bio-informatique où ils jouent un rôle fondamental dans le décryptage des génomes...

2. Justifier que l'algorithme termine dans tous les cas. Donner une majoration du nombre de comparaisons $w[i] == \text{text}[p+i]$ puis des évaluations de la condition $(i < m \text{ and } w[i] == \text{text}[p+i])$ en fonction de $n = \text{len}(\text{text})$ et de $m = \text{len}(w)$.
3. Est-il possible que p prenne la valeur n ? Cela provoque-t-il une erreur dans l'évaluation $w[i] == \text{text}[p+i]$?
4. On suppose que $w = ''$ (chaîne vide, de longueur 0). Combien y a-t-il d'ajouts à la liste L ?
5. On suppose que $w = \text{text}$. Votre majoration est-elle précise dans ce cas ?
6. Donner une preuve de correction (totale) de l'algorithme.

Corrigé 2.13 page 137.

2.5 Algorithmes dichotomiques

2.5.1 Algorithme de recherche dichotomique

Recherche dichotomique dans une liste triée

```
def rechercheDicho(L,e):
    '''
    L : list , liste triée en ordre croissant;
    e : objet quelconque;
    Renvoie True si e est dans L, False sinon.
    '''
    a, b = 0, len(L)-1
    while a <= b:
        m = (a+b)//2
        if L[m] == e:
            return True
        elif L[m] < e:
            a = m + 1
        else:
            b = m - 1
    return False
```

L'algorithme que nous proposons ici suppose que la liste dans laquelle on recherche un élément est triée. D'une manière ou d'une autre, vous le connaissez déjà : l'idée est que l'on compare l'élément cherché au terme du milieu de la liste, ce qui conduit, tant qu'on ne le trouve pas, à le rechercher soit dans la première partie soit dans la

seconde partie de la liste. Avec une telle méthode le nombre de comparaisons est proportionnel au logarithme de la longueur de la liste. Elle sera reprise au chapitre (4.1) avec l'étude du tri par insertion.

Spécification

La fonction `rechercheDicho(L, e)` prend en arguments une liste ou un tableau de nombres trié dans l'ordre croissant et retourne `True` si $e \in L$, `False` sinon.

Correction et complexité

On peut montrer que cet algorithme termine, que le résultat est bien celui attendu et que le nombre n d'itérations est majoré en fonction du nombre de termes de la liste (ou du tableau L) : $n \leq 1 + \frac{\ln(\text{len}(L))}{\ln 2} = 1 + \log_2(\text{len}(L))$.

Nous en proposons plus loin l'analyse détaillée en exercice. Il est assez facile de montrer que l'algorithme termine, d'évaluer un majorant de sa complexité. Par contre la preuve de la correction, est assez difficile, à la frontière de ce que l'on attendra de vous. Elle est là pour ceux « qui en veulent ». Vous pourrez y revenir en deuxième lecture quand vous vous sentirez plus forts.

Approche empirique

Avant d'entreprendre une éventuelle étude théorique, commençons par observer son fonctionnement lorsqu'il prend en arguments une des listes présentées dans les deux tableaux qui suivent et $e = 22$. On recommencera avec $e = 36 \in L...$

i	0	1	2	3	4
$L[i]$	13	17	21	29	36

et

i	0	1	2	3	4	5
$L[i]$	13	17	21	29	36	40

Avec la première liste, nous aurons successivement :

$k = 0$	$a_0 = 0$	$b_0 = 4$	$m_0 = 2$	$L[2] = 21 < e$
$k = 1$	$a_1 = 3$	$b_1 = 4$	$m_1 = 3$	$L[3] = 29 > e$
$k = 2$	$a_2 = 3$	$b_2 = 2$		

et de façon analogue, la seconde conduit aux itérations

$k = 0$	$a_0 = 0$	$b_0 = 5$	$m_0 = 2$	$L[2] = 21 < e$
$k = 1$	$a_1 = 3$	$b_1 = 5$	$m_1 = 4$	$L[4] = 36 > e$
$k = 2$	$a_2 = 3$	$b_2 = 3$	$m_2 = 3$	$L[3] = 29 > e$
$k = 3$	$a_2 = 3$	$b_2 = 2$		

Exercice 2.14 ★ analyse de recherche dichotomique

Conseil pour les démonstrations : noter a_k, b_k les contenus des variables a et b au moment la $k^{\text{ième}}$ évaluation. On commence donc avec $a_1 = 0, b_1 = \text{len}(L) - 1$.

1. Prouver que l'algorithme termine pour toute liste non vide ou tout tableau L dont les éléments sont triés dans l'ordre croissant. Termine-t-il dans les autres cas ?

2. ★ Pour prouver la correction de l'algorithme montrer que
 - (a) si $e \notin L$, la boucle se poursuit sans interruption forcée jusqu'à la condition d'arrêt.
 - (b) si $e \in L$, la boucle est interrompue avec $L[m] = e$.
3. Comparer le nombre d'itérations à $\frac{\ln(\text{len}(L))}{\ln 2} = \log_2(\text{len}(L))$.

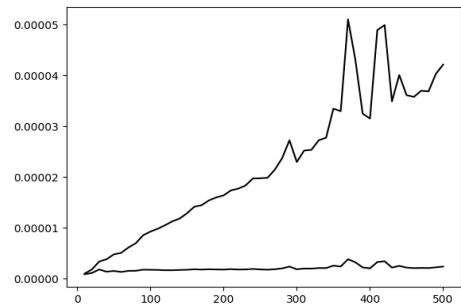
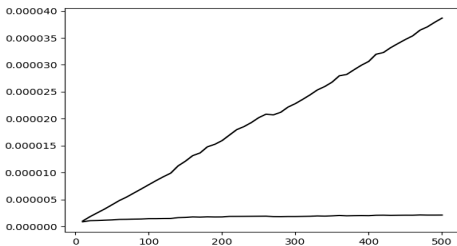
Corrigé en 2.14 page 138.

Exercice 2.15 tests de comparaison de deux algorithmes de recherche

On souhaite ici comparer les temps d'exécution effectifs des algorithmes de recherche séquentielle (vu en 2.3 page 103) et dichotomique.

On note L_n la liste croissante des n entiers impairs de 1 à $2n - 1$.

1. On décide de tester les algorithmes de recherche sur L_n et avec les entiers compris entre 0 et $2n$. Quel est l'intérêt de ce choix ?
2. Écrire une fonction **temps_de_recherche(n)** qui prend un entier $n > 1$ en argument et renvoie le tableau (ou la matrice) à $2n+1$ lignes et 2 colonnes dont la ligne d'indice i contient les temps écoulés pendant les appels `rechercheW( $L_n$ ,  $i$ )` et `rechercheDicho( $L_n$ ,  $i$ )`.
3. Écrire une procédure **temps_moyens()** qui
 - calcule les temps de calcul moyens des $2n + 1$ appels `rechercheW( $L_n$ ,  $i$ )` et `rechercheDicho( $L_n$ ,  $i$ )` pour $0 \leq i \leq 2n$, et pour n variant de 50 à 500 par pas de 50 par exemple ;
 - stocke ces temps moyens dans un tableau dont les lignes sont $[n, \text{tmW}, \text{tmD}]$ où tmW et tmD sont les temps moyens calculés pour la liste L_n ;
 - trace le graphe représentant ces temps de calcul moyens en fonction de n ;



4. Deux appels successifs à cette même procédure ont produit les deux figures ci-dessus. Interpréter le graphique de gauche à la lumière des calculs de complexité que nous avons faits pour `rechercheW()` et `rechercheDicho()`.
Comment expliquer le résultat de la figure de droite obtenu avec un deuxième

appel alors qu'il n'y a rien d'aléatoire *a priori* dans nos algorithmes et dans les choix de L et e pour les tester ?

Conseils : Vous pourrez utiliser `np.zeros((n,p))` pour réserver un tableau (de flottants par défaut) ; `time.time()` pour mesurer les intervalles de temps.

Corrigé en 2.15 page 139.

2.5.2 Exponentiation rapide, version itérative

On propose ici un calcul rapide de puissances entières. Un programme avec une boucle `for` comme ci-dessous conduirait à effectuer n multiplications :

```
def puissance(a,n):
    '''
    a : int ou float;
    n : int avec n>=0.
    '''
    p = 1
    for i in range(0,n):
        p = p*a
    return p
```

On peut réduire considérablement le nombre de ces multiplications, ce qui a une grande importance quand il s'agit de calculer des puissances de très grands nombres, de matrices et d'une façon générale, quand il s'agit de calculer $x \star x \star x \star \dots \star x$ pour une opération associative $\star$.

L'idée sous-jacente est illustrée ici avec le calcul de a^{37}

$$37 = 2^5 + 2^2 + 1 = \sum_{k=0}^{\infty} c_k 2^k = 100101_{(2)}$$

$$a^{37} = \left(\left(\left((a^2)^2 \right)^2 \right)^2 \right)^2 \times (a^2)^2 \times a^1.$$

ce qui permet un calcul avec 9 multiplications au lieu de 37. Notre algorithme reposera sur les deux observations :

1. En parcourant de droite à gauche la liste des chiffres binaires de n on peut faire des élévations au carré successives, qui, pour chacune ne demandent qu'une seule multiplication :

$$a \rightarrow a^2 \rightarrow a^4 \rightarrow a^8 \rightarrow a^{16} \rightarrow a^{32}.$$

2. En partant de $p = 1$, on obtient a^n en multipliant p par la puissance précédemment calculée dès que le chiffre binaire rencontré est égal à 1.

Exponentiation rapide, version itérative

```
def expo_rapide(a,n):
    '''
    a : int|float;
    n : int avec n >= 0.
    Renvoie a^n avec au plus 2log2(n)+2 mult.;
    '''
    L = binaire(n) # liste des chiffres en base 2
    L.reverse()   # ordre inverse
    p, A = 1, a

    for e in L:
        if e == 1:
            p *= A
        A *= A
    return p

>>> expo_rapide(3, 37)
450283905890997363
```

Pour une fonction **binaire()**, voir la section 2.1.2.

Exercice 2.16 analyse de l'exponentiation rapide itérative

On suppose que **binaire(n)** renvoie la liste des chiffres de n en base 2 et qu'un appel ne provoque aucune multiplication.

1. Que renvoie **expo_rapide(a,n)** si $n=0$?
2. Combien un appel **expo_rapide(a, 37)** provoque-t-il de multiplications ?
3. Si $n \in \mathbb{N}^*$, on considère l'entier i tel que $2^i \leq n < 2^{i+1}$. Préciser le nombre de multiplications lorsque $n = 2^i$ et lorsque $n = 2^{i+1} - 1$. On observera que dans les deux cas, $\lfloor \log_2(n) \rfloor = i$. et on exprimera la complexité de cet algorithme en fonction de $\lfloor \log_2(n) \rfloor$.
4. Justifier que l'algorithme termine et qu'il donne le bon résultat.

Corrigé 2.16 page 141.

2.6 Corrigés des exercices

Corrigé de l'exercice n° 2.1

Les deux premières questions sont sans mystère ; on gère le cas $a > 0$ en faisant la division de a par $|b|$ et en observant que $a = bq + r$ ssi $a = -b \times (-q) + r$...

```
def divEuclZ(a,b):
    if b !=0:
        if b < 0:
            b1, sb = -b, -1
        elif b > 0:
            b1, sb = b, 1

        r = a
        q = 0

        if a >=0:
            while r >= b1:
                r = r-b1
                q = q+1
            return q*sb, r
        else:
            while r < 0:
                r = r+b1
                q = q+1
            return -q*sb, r
```

Corrigé de l'exercice n° 2.2

1. A chaque itération $d \leftarrow d + d = 2d$. On a donc comme invariant de boucle $d = 2^{**i}$ puisque avant la première itération, $i = 0$ et $d = 1 = 2^i$. Après la boucle on a donc $2^{**i} = d \leq b < 2^{**}(i+1) = d + d$. On a donc $i = \lfloor \log_2(n) \rfloor$.
2. Avec $n = r = 37$, on a $2^5 \leq 37 < 2^6$ d'où $L = [5]$;
Avec $r = 5$, on a $2^2 \leq 5 < 8$ d'où $L = [5, 2]$;
Avec $r = 1$, on a $2^0 \leq 1 < 2$ d'où $L = [5, 2, 0]$.

```
def exposants_binaires(n):
    '''
    n: int; n >= 0
    Renvoie la liste des exposants qui
    figurent dans l'écriture binaire de n.
    '''
    assert isinstance(n, int) and n >= 0

    r, L = n, []

    while r > 0:
        i, d = 0, 1
        while d+d <= r:
            i += 1
            d += d
            #ici d = 2**i
        # ici d = 2**i <= r < 2d = 2**(i+1)
        L.append(i)
        r = r-d

    return L
```

3.

```
def binaire(n):
    '''
    n: int; n >= 0
    Renvoie la liste des chiffres dans
    l'écriture binaire de n.
    '''
    L = exposants_binaires(n)
    if len(L)==0:
        return [0]
    else:
        B = [0 for i in range(0,L[0]+1)]
        for e in L:
            B[e] = 1
        B.reverse()
        return B
```

Corrigé de l'exercice n° 2.3

Qui dit coefficients, dit entiers dans $\{0, 1\}$ et non pas **chiffres** dans $\{ '0', '1' \}$.

```
def binaire_(n):
    '''
    n: int;
    Renvoie la liste dont le premier terme est
    '+' ou '-' et les suivants la liste des coeffi-
    cients dans l'écriture binaire de n.
    '''
    b = bin(n)
    L = list(b.split('b')[1])
    if b[0] == '-':
        return ['-'] + [int(e) for e in L]
    else:
        return ['+'] + [int(e) for e in L]
```

Corrigé de l'exercice n° 2.4

1. Bien évidemment, lorsque le pas de la subdivision est constant, on regroupera les termes ce qui donne $h = t_{j+1} - t_j = \frac{b-a}{n}$ (n est le nombre d'intervalles entre a et b) et

$$\int_{t_0}^{t_k} f(s) ds \approx h \left(\frac{f(t_0)}{2} + \sum_{j=1}^{k-1} f(t_j) + \frac{f(t_k)}{2} \right) \quad (2.6.1)$$

2. D'où la fonction Python en tout point analogue à celle qui nous a permis de calculer une moyenne :

```
def trapezes(f, a, b, n):
    t, h, s = a, (b-a)/n, (f(a)+f(b))/2
    for k in range(1, n):
        t = t+h
        s = s + f(t)
    return s*h
```

Là encore, le calcul dans la boucle est un standard, il n'y a rien à prouver de plus que ce qui est dans le tableau de la page 36. La complexité est en $n-1$ appels à f ($2n+1$ additions, **une multiplication** et deux divisions par 2 !).

3. On sait que $\int_0^\pi \sin t dt = 2$, $\int_0^\pi \cos t dt = 0$:

```
>>> trapezes(np.sin, 0, np.pi, 100)
1.9998355038874449
>>> trapezes(np.sin, 0, np.pi, 1000)
1.9999983550656877
>>> trapezes(np.cos, 0, np.pi, 1000)
-9.0778753397670667e-15
```

4. Dans la fonction Python qui précède, on suppose que f est donnée comme une fonction de type `float` $\rightarrow$ `float`. Mais si nous disposons de données numérisées, nous ne connaissons de notre fonction qu'un échantillonnage, c'est-à-dire des valeurs $y_0 \approx f(t_0), \dots, y_k \approx f(t_k), \dots$. Le programme précédent est donc inopérant. Pourtant, nous n'avons besoin de rien de plus pour calculer une valeur approchée de l'intégrale de f entre t_0 et t_n avec la formule (2.6.1) puisque seuls les $f(t_k)$ y interviennent.

- On réécrit notre fonction de la façon suivante :

```
def trapezesEchant(Y, a, b):
    n = len(Y) - 1
    t, h, s = a, (b-a)/n, (Y[0]+Y[-1])/2
    for k in range(1, n-1):
        s = s + Y[k]
    return s*h
```

Cette fonction prend en arguments un tableau Y , et deux flottants a et b . On suppose que Y est une suite de relevés pour une subdivision régulière de $[a, b]$. Dans ce cas il y a $n = \text{len}(Y) - 1$ intervalles de longueur $h = (b - a)/n$.

`trapezesEchant(Y, a, b)` retourne donc l'approximation de l'intégrale de la fonction partiellement connue, par la seule formule des trapèzes disponible, celle qui est associée à la subdivision sous-jacente à Y .

- De la même façon, pour calculer $\int_{t_0}^{t_0+1} f(t)e^{i\omega t} dt$ posons $g(t) = f(t)e^{i\omega t}$.

On remplace dans le calcul $f(t_k) \approx y_k$ par $g(t_k) \approx e^{i\omega t_k} y_k$. Ce que nous faisons avec le script suivant pour calculer lorsque $\omega = 1$,

$$\int_0^\pi \sin(t)e^{i\omega t} dt = \frac{i\pi}{2} :$$

```

w    = 1
G = lambda x : np.exp(1j*w*x)
#-- la fonction exponentielle
X = np.linspace(0, np.pi, 10001)
#--- subdivision de [0,pi], 10001 points
Y = np.sin(X)
#--- les sinus
Z  = Y*G(X)
#--- les produits terme à t. par exp(i w x[k])
R1 = trapezesEchant(Z, 0, np.pi)
#--- la formule de calcul sur échantillon

>>> R1
(9.8715779693325545e-08+1.5707963267638831j)
#--- le résultat attendu

```

Corrigé de l'exercice n° 2.5

1. Le pendant de la méthode **L.count(x)** pour les listes :

```

def nbreOccurences(L,e):
    c = 0
    for x in L:
        if x == e:
            c = c+1
    return c

```

2. On peut programmer de façon impérative

```

def listeOccurrences(L,e):
    O = []
    for k in range(0, len(L)):
        if L[k]==e:
            O.append(k)
    return O

```

ou encore fonctionnelle avec une fonction lambda qui retourne une liste en compréhension (attention, pas de passage à la ligne dans l'éditeur, cela renverrait un erreur, mais ici, on n'a pas la place !) :

```
listeOccurrences = lambda L,e:
    [k for k in range(0,len(L)) if L[k]==e]
```

```
>>> L = [0,0,1,-2,3,4,4,0,13]
>>> listeOccurrences(L,13)
[8]
>>> listeOccurrences('Bonjour, bonjour', 'o')
[1, 4, 10, 13]
```

Corrigé de l'exercice n° 2.6

1. On adapte l'algorithme RechercheW sans difficulté en tenant compte explicitement, cette fois-ci, de la liste (ou tableau ou chaîne) vide :

```
def premOccurrence(L,e):
    n = len(L)
    if n==0:
        return None
    else:
        k = 0
        while k < n and L[k] != e:
            k = k+1
        if k == n:
            return None
        else:
            return k
```

On vérifie tout cela. Les trois derniers appels retournent bien None (on pourrait tester le type pour le confirmer).

```

>>> L=[0,1,2,3,4,4,4,90, 12]
>>> premOccurrence(L,12)
8
>>> premOccurrence('Bonjour', 'r')
6
>>> premOccurrence(L,3)
3
>>> premOccurrence(L,90)
7
>>> premOccurrence(L,190)
>>> premOccurrence([],90)
>>> premOccurrence('', 'r')

```

2. Si la liste est vide, $n = 0$ et la fonction retourne la valeur attendue.

Sinon, on entre dans une boucle while qui termine car chaque évaluation de la condition incrémente k d'une unité. La condition $k = n$ sera donc réalisée s'il n'y a pas d'arrêt avant.

Montrons, toujours dans ce second cas, que la fonction fournit le résultat attendu. Pour cela, en notant p le n° de l'évaluation de la condition, définissons comme dans le cours :

$$\mathcal{P}(p) = \{k = p - 1 \text{ et } (0 \leq j \leq k - 1 \Rightarrow L[j] \neq e)\}.$$

On montre alors facilement que $\mathcal{P}(1)$ est vraie. Par ailleurs, si $\mathcal{P}(p)$ est vraie, soit $\mathcal{P}(p+1)$ est vraie, soit la condition est évaluée à faux et la boucle termine. Dans ce cas

-soit $k = n$ et le résultat est None, ce qui est correct car $L[j] \neq e$ si $j \leq k - 1 = n - 1$, e n'est pas dans la liste ;

-soit $k < n$ et $L[k] = e$, $L[j] \neq e$ si $j \leq k - 1$; le résultat est k ce qui est encore correct.

Corrigé de l'exercice n° 2.7

1.

```

def maximum(L):
    p, m = 0, L[0]
    for k in range(1, len(L)):
        if L[k] > m:
            p, m = k, L[k]
    return m, p

```

2. Si nous modifions notre fonction pour écrire

```
def maximum(L):
    p, m = 0, L[0]
    for k in range(1, len(L)):
        if L[k] >= m: \#ici la différence
            p, m = k, L[k]
    return m, p
```

il y aura réaffectation pour chacune des occurrences du maximum rencontrée. p contiendra alors l'indice de la dernière occurrence et non plus de la première.

3. Sans commentaire

```
def maximum(L):
    n = len(L)
    if n == 0:
        return None
    else:
        p, m = 0, L[0]
        for k in range(1, n):
            if L[k] > m:
                p, m = k, L[k]
        return m, p
```

Corrigé de l'exercice n° 2.8

1. Listes strictement croissantes, décroissantes, constantes etc.

```
[12, 13, 14, 15, 16, ..., 20, 21, 22] ---> ((22, 21), (10, 9))
[22, 21, 20, 19, 18, ..., 14, 13, 12] ---> ((22, 21), (0, 1))
[1, 1, 1, 1, 1, 1, ..., 1, 1, 1, 1] ---> ((1, 1), (0, 1))
[1, 23, 41, 3, 23, 67, 67, 10, 1, 67] ---> ((67, 67), (5, 6))
[1, 67, 41, 3, 23, 68, 67, 10, 1, 17] ---> ((68, 67), (5, 1))
[1, 23, 41, 3, 23, 67, 67, 10, 1, 17] ---> ((67, 67), (5, 6))
[1, 23, 41, 3, 23, 67, 67, 10, 1, 170] ---> ((170, 67), (9, 5))
```

2. On écrira la ligne $m1, m2, p1, p2 = L[0], L[1], 0, 1$ pour initialiser.3. On permute éventuellement les m_i et p_i pour assurer $m1 \geq m2$. Ainsi pour une liste de longueur 2, le résultat est correct sans boucle.

4. Voir tableau.

Premier et deuxième plus grands éléments d'une liste, corrigé

```

def recherche_2nd_max(L):
    '''
    L : list avec len(L) > 2;
    Renvoie le tuple de tuples (m1,p1), (m2,p2)
    avec m1 >= m2 les deux plus grands éléments
    rencontrés et p1, p2 leurs premières positions
    dans L.
    '''
    assert isinstance(L, list) and len(L) >= 2

    m1, m2, p1, p2 = L[0], L[1], 0, 1

    if m2 > m1:
        m1, p1, m2, p2 = m2, p2, m1, p1

    for i in range(2, len(L)):
        e = L[i]

        if e > m1:
            m2, p2 = m1, p1
            m1, p1 = e, i

        elif e > m2:
            m2, p2 = e, i

    return (m1, m2), (p1, p2)

```

5. Il y a pour une liste de longueur $n \geq 2$, $2n - 1$ comparaisons.
6. On note ℓ_i la liste des i premiers éléments de L ($\ell_2 = [L[0], L[1]]$ par exemple) et on pose

$$\mathcal{P}(i) = \begin{cases} m1 & = \max(\ell_i) \\ m2 & = \max(\ell_i \text{ privée de } L[p1]) \\ \ell_i[p1] & = m1 \\ \ell_i[p2] & = m2 \\ p1 & \text{est l'indice de la première occurrence de } m1 \\ p2 & \text{est l'indice de la première occurrence de } m2 \text{ si } m2 < m1 \\ p2 & \text{est l'indice de la deuxième occurrence de } m2 \text{ si } m2 = m1 \end{cases}$$

Montrons que $\mathcal{P}(i)$ est un invariant de boucle par récurrence sur $i \geq 2$.

Initialisation : A l'entrée dans la boucle for, on a $i = 2$ et $\mathcal{P}(2)$ est satisfaite.

Hérédité : Supposons $\mathcal{P}(i)$ satisfaite au début d'une itération pour une valeur de $i \geq 2$. Pour une variable x du programme, on notera x sa valeur au début de l'itération et x' sa valeur à la sortie.

On observe que $\ell_{i+1} = \ell_i + [e]$ (avec $e = L[i]$). - Si $e > m1$, c'est strictement le plus grand élément de ℓ_{i+1} rencontré pour la première fois. D'où $m1' = e$ et $m2' = m1$ prend la seconde place. Les p_i restent conformes à la spécification. Si $m1 \geq e > m2$, $m1$ reste le maximum strict, e devient le deuxième max d'où $m2' = e$.

Corrigé de l'exercice n° 2.9

1. Il vous aura fallu, pour programmer cela, regarder ce que font les méthodes `pop()` et `insert()` d'une liste.

```
>>> L1
[1, 2, 3, 4, 0, 17, 89, 100, 3, 26, 0, 99]
>>> m=L1.pop(7)
>>> L1
[1, 2, 3, 4, 0, 17, 89, 3, 26, 0, 99]
>>> L1.insert(7,m)
>>> L1
[1, 2, 3, 4, 0, 17, 89, 100, 3, 26, 0, 99]
```

- (a) Soyez attentifs à la façon dont sont retournés les indices : la liste qui a

```
def maximumlet2(L):
    m1, p1 = maximum(L)

    L.pop(p1)
    # attention: c'est la liste L envoyée
    # en paramètre qui est modifiée
    m2, p2 = maximum(L)
    L.insert(p1,m1)
    # on remet m1 à sa place
    if p2 < p1:
        return (m1, m2), (p1, p2)
    else:
        # attention m2 a été rencontré après m1
        return (m1, m2), (p1, p2+1)
```

permis de trouver le second maximum n'est pas la liste de l'appel : elle a été modifiée. Ainsi $p2$ est la valeur cherchée si ce second max se trouve

avant le premier (il est alors différent du premier) ; sinon, c'est $p2 + 1$ qui est sa position dans la liste d'origine.

- (b) Complexité en nombre de comparaisons : il y a $n - 1$ comparaisons dans le premier appel de maximum (car L est de longueur n) et il y en a $(n - 2)$ dans le second (car L n'a plus, à ce moment, que $n - 1$ éléments). Cela fait au total $2n - 2$ comparaisons avec celle du corps de la fonction. Cette procédure a la même complexité que celle de l'exercice (2.8) en nombre de comparaisons mais elle fait appel aux méthodes `pop` et `insert` qui ne sont pas gratuites.

2. Nous ne pouvons pas faire la même chose avec des tableaux : nous ne pouvons pas enlever un terme par exemple, puisque la taille d'un tableau est immuable (attention, nous ne parlons que des tableaux de `numpy`).

L'idée est alors de repérer le maximum, de le remplacer par un élément strictement plus petit **que n'importe quel autre terme**. Le terme qui a pris sa place ne pourra plus être repéré comme premier ou deuxième maximum, ce qui laisse le champ libre au second plus grand. Observons que si le maximum est présent deux fois, ses deux premières places sont retournées.

```
def maximumlet2T(L):
    m1, p1 = maximum(L)

    if p1 == 0:
        L[p1] = L[1]-1
    else:
        L[p1] = L[0]-1

    m2, p2 = maximum(L)
    L[p1] = m1
    # on replace le maximum
    return (m1, m2), (p1, p2)
```

Corrigé de l'exercice n° 2.10

1. Maximum est réécrite pour commencer la recherche à partir d'un certain rang. Cela permet de trouver le maximum en cherchant dès le début de liste, de le placer à la première place, de chercher le second maximum dans la liste restante, que l'on met en seconde place etc.

```
def maximum(L, deb):
    p, m = deb, L[deb]

    for k in range(deb+1, len(L)):
        if L[k] > m:
            p, m = k, L[k]
    return p, m
```

```
def triNaif(L):
    n = len(L)
    if n >= 2:
        for k in range(0, n-1):
            # inutile de trier le dernier terme!
            p, m = maximum(L, k)
            L[k], L[p] = L[p], L[k]
            # on ne joue pas à ça dans un autre
            # langage!
    return None
```

2. Un appel à `maximum(L,r)` avec une liste de longueur n provoque $n - r - 1$ comparaisons ($r = 0$ est l'indice du premier terme et du premier appel. Le nombre total de comparaisons est

$$\sum_{r=0}^{n-2} (n - r - 1) = \sum_{m=1}^{n-1} m = \frac{(n-1)n}{2}$$

On a posé $m = n - r - 1$ qui varie entre $n - (n - 2) - 1$ et $n - (0) - 1$ (entre parenthèses les bornes extrêmes de r).

Le chapitre (4.1) page 190 est dédié aux tris et nous verrons qu'il en existe de plus efficaces.

Corrigé de l'exercice n° 2.11

1. (a) La fonction `liste_aleatoire(N, a, b)` renvoie, lorsqu'elle termine, une liste de N entiers compris entre a et b **tous distincts**. En effet, n est incrémenté ssi un élément est ajouté à la liste et l'instruction `if not q in L: ...` permet de n'ajouter l'entier aléatoire q à la

liste en construction que s'il n'y figure pas déjà. **La liste contient donc des éléments distincts à chaque étape.**

Remarquons que si $b - a + 1 < N$, le programme boucle (indéfiniment) de façon certaine puisqu'il sera impossible de trouver un nouvel élément qui ne sera pas dans la liste dès qu'elle en contiendra $b - a + 1$. On suppose dans ce qui suit $N \geq b - a + 1 = \text{Card}(\llbracket a, b \rrbracket)$.

- (b) Supposons que pour une certaine itération on ait $n = \text{len}(L)$. La liste contient donc n entiers distincts compris entre a et b . q est choisi aléatoirement dans $\llbracket a, b \rrbracket$ qui contient $b - a + 1$ éléments.

En supposant que la variable aléatoire q suive une loi uniforme (équiprobabilité) sur $\llbracket a, b \rrbracket$, la probabilité que q soit dans L est $\frac{n}{b - a + 1}$ soit :

$$P(q \in L | \text{len}(L) = n) = \frac{n}{b - a + 1} \leq \frac{N}{b - a + 1} \leq 1$$

- (c) Si à chaque itération le résultat du test est positif il y a N itérations exactement. Ce qui donne autant de tests `q not in L` sur des listes $L_0, L_1, \dots, L_k, \dots, L_{N-1}$ de tailles égales à leurs indices : Il y a donc, puisqu'on est dans le pire des cas pour une recherche d'élément dans une liste donnée (l'élément n'y figure pas) :

$$0 + 1 + 2 + \dots + (N - 1) = \frac{(N - 1)N}{2}$$

comparaisons (cachées en ce sens qu'elles sont gérées dans une fonction du noyau).

Évidemment, la probabilité de ce cas extrême qui est la probabilité que tous les tests permettent d'ajouter un élément, les choix de q étant indépendants, sera

$$\prod_{k=1}^{N-1} (1 - P(q \in L | \text{len}(L) = n)) = \prod_{n=0}^{N-1} \left(1 - \frac{n}{b - a + 1}\right).$$

Le cours de maths sur les séries vous permettra de constater que cette probabilité a pour limite 0 quand $n \rightarrow +\infty$.

Nous avons donc minoré le nombre de tests d'appartenance.

En effet, si lors d'une itération l'élément q est dans la liste, le test échouera et se répétera avec la même liste et un autre élément q ce qui **ajoute** itérations et comparaisons par rapport au cas envisagé.

Hors question :

Le temps d'attente du premier test qui réussit suit une loi géométrique de paramètre $p_n = \left(1 - \frac{n}{b-a+1}\right)$ et le temps d'attente ou nombre

moyen de tests à l'étape n est l'espérance : $E(G_n) = \frac{1}{p_n} = \frac{1}{1 - \frac{n}{b-a+1}}$.

En fait, la complexité moyenne sera donnée par la somme des espérances du nombre de tests à chaque étape multipliée par le nombre de comparaisons, à savoir,

$$\sum_{n=1}^{N-1} \frac{n}{p_n} = \sum_{n=1}^{N-1} \frac{n}{1 - \frac{n}{b-a+1}}.$$

Lorsque $b-a \gg N$, cela est proche de $\sum n \dots$

2. Avec un dictionnaire :

- (a) L'algorithme semble sensiblement le même :

On double la liste par un dictionnaire dont les clés sont les entiers ajoutés à la liste, les valeurs leurs indices dans la liste ($D[q] = i$ ajoute $\{q : i, \dots\}$ dans le dictionnaire). La seule différence entre ces deux fonctions est que le test s'effectue sur l'appartenance au dictionnaire et non plus sur l'appartenance à la liste.

L'occupation mémoire est bien sûr plus que doublée (les entiers de la liste sont stockés deux fois : dans la liste et dans les **clés** du dictionnaire, il y a en plus les **valeurs** du dictionnaire qui sont les indices de la liste).

- (b) Les tests d'appartenance à une liste ont disparu, remplacés par les tests d'appartenance d'une clé à un dictionnaire.

Les résultats affichés montrent que lorsque la taille de la liste que l'on veut obtenir est multipliée par 10, **le temps de calcul est sensiblement multiplié par 10**. La complexité de la fonction `liste_aleatoire_1()` est vraisemblablement en $O(N)$.

Cela est conforme à l'affirmation selon laquelle le test d'appartenance à un dictionnaire a un coût en $O(1)$ (ie : majoré par une constante qui ne dépend pas de la taille de la liste).

Corrigé de l'exercice n° 2.12

Recherche des valeurs les plus proches dans une liste (ou un tableau)

```
def plus_proches(L):
    """
    L: list|numpy.ndarray.
    """
    assert isinstance(L, (list, np.ndarray)) \
        and len(L) > 1

    n = len(L)
    # construction du dictionnaire (question 2)
    D = { }

    for i in range(0, n-1):
        D[i] = (i+1, abs(L[i]-L[i+1]))
        for j in range(i+2, n):
            v = abs(L[i]-L[j])
            if D[i][1] > v:
                D[i] = (j, v)

    # recherche (question 4)
    p, q, m = 0, D[0][0], D[0][1]
    for i in range(1, n-1):
        j, v = D[i]
        if m > v:
            p, q, m = i, j, v

    return p, q, m
```

Question 3. La construction du dictionnaire demande

$$\sum_{i=0}^{n-2} (n-i-2) = \sum_{k=0}^{n-2} k = \frac{(n-1)(n-2)}{2} \underset{n \rightarrow +\infty}{\sim} \frac{n^2}{2}$$

itérations (boucle interne).

Question 5. Le parcours du dictionnaire ajoute $n-2$ itérations (et comparaisons). On a toujours une complexité quadratique ($O(n^2)$) car

$$\frac{(n-1)(n-2)}{2} + (n-2) \underset{n \rightarrow +\infty}{\sim} \frac{n^2}{2}$$

Corrigé de l'exercice n° 2.13

1. Lorsque $p = 1$ et $i = 0$, $w[0] = \text{text}[1+0]$ et i est incrémenté.
Lorsque $p = 1$ et $i = 1$, $w[1] = \text{text}[1+1]$ et i est incrémenté.
Lorsque $p = 1$ et $i = 2$, $w[2] = \text{text}[1+2]$ et i est incrémenté.
Lorsque $p = 1$ et $i = 3$, $w[0] \neq \text{text}[1+3]$ et la boucle termine avec $i = 3 < m = 4$.

La fenêtre qui suivra les valeurs de p

X	G	A	T	T	X	X	X	X	X	X	X	X	X	X	X
	G	A	T	C											

2. La boucle intérieure termine car il y a une incrémentation de i à chaque itération et la condition $i = m$ sera remplie. La boucle principale termine pour la même raison (incrémentement de p). L'algorithme termine donc.

Il y a $n - m + 1$ itérations de la boucle principale. La boucle intérieure en provoquera m dès que w apparaîtra comme sous-chaîne de text dans **la fenêtre** $\text{text}[p : p+m]$ et moins dans les autres cas. Cela fait **au plus** $(n - m + 1) \times m$ comparaisons (il n'y a pas de comparaison si $m = 0$, évaluation paresseuse des expressions booléennes).

Il y a par contre au plus $(n - m + 1) \times (m + 1)$ évaluations de la condition complète.

3. Il n'y a une itération dans la boucle interne qu'avec $p + m - 1 < n$. On peut avoir $p = n$ si $m = 0$. Dans ce cas, l'évaluation paresseuse de la condition de cette boucle s'arrête à $i < m$ qui n'est pas vérifiée et $w[0] = \text{text}[n]$ n'est pas évaluée. Dans un langage sans évaluation paresseuse des expressions booléennes, il y aurait eu une levée d'exception !
4. Si w est la chaîne vide, $m = 0$ et la condition $i < m$ n'est jamais remplie. Il n'y a pas d'itération dans la boucle `while` intérieure. Par contre comme $i = m = 0$, la liste L est étendue à chaque itération de la boucle extérieure (la chaîne vide est partout³ !). Donc $n + 1$ itérations de la boucle principale avec une évaluation de la condition.
5. Si $w = \text{text}$, il y a une seule itération pour la boucle principale avec $p = 0$ (puisque $p + n - 1 < n \Leftrightarrow p < 1$). Il y a un test par caractère dans l'évaluation de la boucle intérieure, donc $n = m$ tests. Notre majoration aurait donné $n + 1$.
6. Nous avons déjà montré que l'algorithme termine. Montrons que le résultat est conforme à ce qui est annoncé.

La liste L contient exactement les indices p ajoutés lorsque après la boucle `while` intérieure on a $i == m$. Cette condition est équivalente au fait qu'il y a eu m incrémentations et donc m itérations exactement ou encore que

$$\text{text}[p]\text{text}[p+1]...\text{text}[p+m-1] = w[0]w[1]..w[m-1]$$

3. Cela vous rappelle-t-il quelque chose ?

(c'est-à-dire que la **fenêtre sur text** coïncide avec w). C'est bien là la spécification de notre algorithme. La correction est totale puisque par ailleurs il termine.

Corrigé de l'exercice n° 2.14

- Notons a_k, b_k les contenus des variables a et b au moment de l'évaluation de la condition. La première évaluation a donc lieu avec $a_1 = 0$ et $b_1 = \text{len}(L) - 1$. On note aussi, lorsque cette évaluation est suivie d'une itération, $m_k = (a_k + b_k) // 2$ qui vérifie donc $m_k = \frac{a_k + b_k - r_k}{2}$ avec $r_k = 0, 1$.

Deux cas sont possibles lors du déroulement de l'algorithme :

- Soit le corps de la boucle provoque une sortie et l'algorithme termine.
- Soit nous avons une suite d'affectations $a = m_k + 1$ ou $b = m_k - 1$. Comme à chaque étape $a_k \leq m_k \leq b_k$, on a $b_{k+1} - a_{k+1} < b_k - a_k$. Il s'agit d'une suite d'entiers strictement décroissante, si l'algorithme ne termine pas sur un return, cette suite admettra une valeur négative et la condition d'arrêt sera alors satisfaite.

Nous n'avons pas eu à supposer que la liste était triée : l'algorithme termine donc dans tous les cas.

- Si $e \notin L$, la condition $L[m] == e$ qui permet l'interruption n'est jamais satisfaite.
 - Supposons que $e \in L$.

Comme $a_1 = 0 \leq \text{len}(L) - 1$, il y a au moins une itération. Montrons que l'on entre dans l'itération k avec la propriété

$$\mathcal{P}(k) = \{a_k \leq b_k \text{ et } L[a_k] \leq e \leq L[b_k]\}$$

- C'est évidemment vrai lorsque $k = 1$ (nous nous sommes placés dans ce cas précisément).
- Supposons $\mathcal{P}(k)$ vraie et regardons ce qui se passe lors de l'itération k . On y entre avec $a_k \leq m_k \leq b_k$ et $L[a_k] \leq e \leq L[b_k]$.
- Si $L[m_k] < e$, il vient $a_{k+1} = 1 + m_k$ et $b_{k+1} = b_k$. Nous affirmons au passage qu'il y aura une itération de plus. En effet, la condition d'arrêt serait $a_{k+1} > b_{k+1}$, ce qui signifie que

$$a_k \leq m_k \leq b_k = b_{k+1} < 1 + m_k = a_{k+1}.$$

Cela n'est possible qu'avec $a_k = b_k = m_k$, comme par hypothèse de récurrence $L[a_k] \leq e \leq L[b_k]$, et comme $e \in L$ dans notre cas d'étude, on aurait eu $e = L[m]$. Il y a contradiction.

D'autre part, puisque $L[m_k] < e$, comme $e \in L$, $L[a_{k+1}] \leq e \leq L[b_{k+1}] = L[b_k]$ et dans ce cas $\mathcal{P}(k+1)$ est vraie.

- On raisonne de façon analogue dans le cas $L[m_k] > e$.

Nous avons montré que $\mathcal{P}(k)$ est toujours vraie **lorsque** $e \in L$. Ainsi lorsque $e \in L$, la condition d'arrêt n'est jamais vérifiée, comme l'algorithme termine, on sort de la boucle avec $L[m] == e$ et le résultat est correct.

3. Évaluons le nombre d'itérations dans le pire des cas, c'est-à-dire lorsque l'on termine sur la condition d'arrêt ($e \notin L$). Dans ce cas on a, n étant le nombre d'itérations, $a_n \leq b_n$ et $a_{n+1} > b_{n+1}$, ce qui impose (vérifier) que $a_n = b_n = m_n$.

D'autre part, on a toujours lorsque $1 \leq k \leq n$, $m_k = \frac{a_k + b_k - r_k}{2}$ d'où

$$\begin{cases} b_{k+1} - a_{k+1} = b_k - (m_k + 1) = \frac{b_k - a_k}{2} - \frac{2 - r_k}{2} < \frac{b_k - a_k}{2} \\ \text{ou} \\ b_{k+1} - a_{k+1} = (m_k - 1) - a_k = \frac{b_k - a_k}{2} - \frac{2 + r_k}{2} < \frac{b_k - a_k}{2} \end{cases}$$

On sait alors que l'on aura $b_n = a_n$ dès que $\frac{1}{2^n}(b_1 - a_1) < 1$ soit

$$n > \frac{\ln(b_1 - a_1)}{\ln 2} = \log_2(b_1 - a_1).$$

Corrigé de l'exercice n° 2.15

1. Avec ce choix on teste la recherche de 0 situé à gauche, de $2n$ situé à droite, de tous les termes de la liste (les impairs) et d'éléments qui ne sont pas dans la liste mais situés entre deux termes de la liste (les nombres pairs). Chaque disposition d'un élément par rapport à ceux de la liste est envisagée une fois et une seule. n appels renverront True, $n + 1$ renverront False.

2&3 Les fonctions et procédures :

```
def temps_de_recherche(n):
    '''
    n : int;
    '''
    L = [2*k+1 for k in range(0,n)]
    # L = [1, ..., 2n-1], n éléments
    R = np.zeros((2*n+1, 2))

    m, Ts, Td = 10, 0, 0
```

```

for i in range(0, 2*n+1):
    t0 = time.time()
    for c in range(0, m):
        rechercheW(L, i)
    t1 = time.time()
    Ts = (t1-t0)/m
    t0 = time.time()
    for c in range(0, m):
        rechercheDicho(L, i)
    t1 = time.time()
    Td = (t1-t0)/m
    R[i, :] = [Ts, Td]
return R

```

4. La figure de gauche fait clairement apparaître **une fonction linéaire** en $\text{len}(L) = n$ qui correspond au graphe des temps moyens pour `rechercheW(L,e)` d'après ce que nous savons de sa complexité. L'autre courbe, à la faible croissance, correspond donc aux temps moyens de calculs pour `rechercheDicho(L,e)` qui a une complexité en $O(\ln n)$.

La figure de droite qui, nous dit on, correspond à un autre appel de la même procédure présente des anomalies par rapport à la première ; cela s'explique parce que nous avons mesuré les temps écoulés entre appel et retour de nos fonctions. Or, il se trouve que nous travaillons avec des OS qui permettent à des processus différents de travailler (quasiment) en même temps. Lors de l'appel à `temps_moyens()` qui a produit la figure de droite, un autre processus gourmand demandait de la ressource (en l'occurrence le compilateur `LATEX` avec lequel ce livre est écrit) ce qui explique les temps plus longs dans certains cas.

On aurait donc intérêt à compter les opérations effectives, plutôt que le temps écoulé, pour évaluer nos algorithmes.

```

def temps_moyens():
    N = 50
    M = np.zeros((N, 5))
    for i in range(1, N+1):
        n = 100*i
        R = temps_de_recherche(n)
        tms = sum(R[:, 0]) / (2*n+1)
        tmd = sum(R[:, 1]) / (2*n+1)
        M[i, :] = [n, tms, tmd]

```

```

print(M)
pl.plot(M[:,0], M[:, 1], color='black')
pl.plot(M[:,0], M[:, 3], color='black')
pl.show()
return None

```

Corrigé de l'exercice n° 2.16

1. Si $n = 0$, $L=[0]$ et la fonction renvoie $p = 1$.
2. Avec $n = 37$, on a $L = [1, 0, 0, 1, 0, 1]$ il y a donc 6 multiplication avec la ligne $A = *A$ et 3 de plus lorsque l'on rencontre un coefficient égal à 1. Soit 9 en tout. On pourrait en éviter une en initialisant $p = a$ (pour $n > 0$) mais cela ne présente pas d'intérêt.
3. Si $n = 2^i$, la liste L est de longueur $1 + i = 1 + \lfloor \log_2(n) \rfloor$. Tous ses éléments sont nuls sauf le premier, il y a donc $2 + \lfloor \log_2(n) \rfloor$ multiplications.
Si $n = 2^{i+1} - 1$, la liste L est de même longueur $1 + i = 1 + \lfloor \log_2(n) \rfloor$. Tous ses éléments sont égaux à 1 et il y a $2(1 + \lfloor \log_2(n) \rfloor)$ multiplications. C'est le pire des cas.
4. **Terminaison** : rien à prouver, c'est une boucle for correctement écrite (on ne modifie pas la liste à l'intérieur de la boucle).

Correction : On note i le numéro de l'itération et on pose

$$\mathcal{P}(i) = \{A_i = a^{2^i} \text{ et } p_i = a^{\sum_{j=0}^{i-1} c_j 2^j}\}$$

- Initialisation : avant la boucle, $\mathcal{P}(0)$ est satisfaite car $A_0 = a^1 = a^{2^0}$ et $p_0 = 1 = a^0$.

- Hérédité : On suppose qu'en **début** d'itération $i + 1$, $\mathcal{P}(i)$ est satisfaite.

$A = a^{2^i}$ et $p = a^{\sum_{j=0}^{i-1} c_j 2^j}$ (somme nulle lorsque $i = 0$). En multipliant éventuellement p par $A = a^{2^i}$ on ajoute à l'exposant $c_i 2^i$. En élevant A au carré on sortira avec $A = a^{2^{i+1}}$. On sort donc de l'itération $i + 1$ avec $\mathcal{P}(i + 1)$ satisfaite.

Comparaison des coûts de recherche dans les listes, tableaux et dictionnaires
(voir les représentations graphiques page 110)

```
def comparaisons_in():

    X = list(range(100, 5000, 100))
    TD = np.zeros(len(X))
    TL = np.zeros(len(X))
    TT = np.zeros(len(X))

    for i, n in enumerate(X):
        print(n)
        # on construit et remplit les structures
        D = {}
        L = []
        T = np.zeros(n)

        for j in range(0,n):
            v = random.randint(0, 10**4)
            D[v] = True
            L.append(v)
            T[i] = v
        print(27, len(D), len(L), len(T))

        # on cherche les éléments susceptibles de s'y trouver
        t0 = time.time()
        for k in range(0,10**4):
            k in D
        t1 = time.time()
        TD[i] += t1-t0

        t0 = time.time()
        for k in range(0,10**4):
            k in L
        t1 = time.time()
        TL[i] += t1-t0

        t0 = time.time()
        for k in range(0,10**4):
            k in T
        t1 = time.time()
        TT[i] += t1-t0

    pl.plot(X, TD,      color ='black')
    ...
```

Récurtivité

3.1 Introduction

3.1.1 Vocabulaire, premiers exemples

Définition 3.1 Dans un langage de programmation, une fonction (ou une procédure) *f* est **récursive** si son **exécution** peut provoquer un ou plusieurs appels de *f* elle-même. On distinguera alors les **appels récurtifs** à *f*, qui sont les appels provoqués par l'exécution de *f*, d'un **appel principal**, qui a lieu alors que *f* n'est pas en cours d'exécution.

Un appel récurtif dans une fonction est dit **terminal**, s'il est la dernière instruction avant retour de la fonction.

Un **langage récurtif** est un langage dans lequel on peut programmer des fonctions récurtives ; quasiment tous les langages de programmation sont aujourd'hui récurtifs.

Algorithme d'Euclide	Trace
<pre>def pgcd_rec(a,b): print(a,b) if b==0: return a else: q,r =divmod(a,b) return pgcd_rec(b,r)</pre>	<pre>>>> pgcd(0,23) 0 23 23 0 23 >>> pgcd(6,1024) 6 1024 1024 6 6 4 4 2 2 0 2</pre>

• Voici un exemple de fonction récursive dont nous avons déjà présenté une version itérative avec le calcul du pgcd par l'algorithme d'Euclide. Cette fonction contient une instruction d'affichage qui permet de suivre à la trace le déroulement des appels récursifs.

• Une fonction jouet, pour une récursivité croisée

Une fonction qui dit oui ou non	Une autre fonction qui dit oui ou non
<pre>def est_impair(p): if p == 0 : return False else: return est_pair(p-1)</pre>	<pre>def est_pair(p): if p == 0: return True else: return est_impair(p-1)</pre>

Questions :

1. En quoi ces fonctions sont-elles récursives ?
2. Prouvez qu'un appel à l'une d'entre elles avec comme argument entier $n \geq 0$, termine.
3. Prouvez que la réponse suggérée par le nom de la fonction est correcte.
4. On vient vous dire : j'ai essayé le programme mais il ne s'arrête pas. Qu'en dites-vous ?

Réponses :

1. Un appel principal de l'une d'elles est susceptible de provoquer un appel (que l'on dit alors récursif) à la fonction elle-même.
Par exemple, `est_impair(12)` provoque l'appel `est_pair(11)` qui lui-même provoque l'appel `est_impair(10)`...
2. Un appel principal `f(n)` avec l'une quelconque de ces deux fonctions termine si $n = 0$ et provoque un appel `g(n-1)` avec l'autre. En n étapes exactement un appel avec $n = 0$ terminera.
3. Par récurrence sur n on démontre la propriété

$\mathcal{P}(n) = \{\text{Un appel } f(n) \text{ avec une des deux fonctions renvoie le résultat correct}\}.$

- C'est clair si $n = 0$;

- Si $\mathcal{P}(n)$ est vraie pour un certain entier $n \geq 0$, alors $f(n+1)$ renvoie le résultat $g(n)$ qui est correct par hypothèse de récurrence.

4. Il se peut que le programme ne s'arrête pas : si par exemple on appelle $f(n)$ avec n flottant ou $n \in \mathbb{Z}^-$, la **condition d'arrêt** $n == 0$, ne sera jamais satisfaite. Et la programmation défensive dans tout ça ?

Chacune des fonctions qui précèdent provoque un appel récursif à elle-même. La chronologie des appels en cascade est simple à concevoir et la suite de leurs arguments est, dans chaque cas, strictement décroissante. Cela conduit à un appel avec la condition d'arrêt et l'algorithme termine.

Bien entendu, et ce seront souvent les applications les plus intéressantes, une procédure peut comporter plusieurs appels à elle-même. Cela sera abondamment illustré avec les stratégies « diviser pour régner ».

Voici en attendant, quelques exercices pour vous faire la main.

Exercice 3.1

Réécrire l'algorithme d'Euclide en version récursive avec comme seule opération autorisée la soustraction, comme dans l'exercice (8.17) page 338 en observant que si x et y sont deux *entiers positifs*,

- $\text{pgcd}(x, y) = \text{pgcd}(y, x)$;
- si $x \geq y$, $\text{pgcd}(x, y) = \text{pgcd}(x - y, y)$.

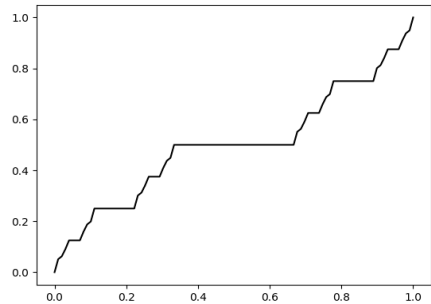
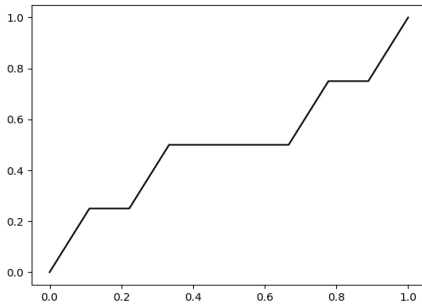
Corrigé en 3.1 page 167

Exercice 3.2 d'après CCINP 2020, épreuve de Maths I, fonction de Cantor-Lebesgue

On note f_0 la fonction (mathématique) définie sur $[0, 1]$ par $f_0(x) = x$. Pour tout entier $n \in \mathbb{N}$, on pose, pour tout $x \in [0, 1]$:

$$f_{n+1}(x) = \begin{cases} \frac{f_n(3x)}{2} & \text{si } x \in \left[0, \frac{1}{3}\right] \\ \frac{1}{2} & \text{si } x \in \left[\frac{1}{3}, \frac{2}{3}\right] \\ \frac{1}{2} + \frac{f_n(3x - 2)}{2} & \text{si } x \in \left[\frac{2}{3}, 1\right]. \end{cases}$$

1. Écrire une **fonction** Python récursive, $\text{cantor}(n, x)$, qui prend un entier $n \geq 0$ et un flottant ou un entier x en arguments et renvoie la valeur de $f_n(x)$.
2. Écrire une **procédure** qui prend un entier n en argument et trace le graphe de f_n .
3. Combien un appel $\text{cantor}(n, x)$ provoque-t-il d'appels récursifs ?



Corrigé en 3.2 page 167

Exercice 3.3 permutations des éléments d'une liste

On rappelle qu'une **permutation** d'un **ensemble** L est une bijection de L sur lui-même.

Nous dirons par extension qu'une liste $L1$ est une permutation de la liste $L0$ de longueur n (et d'éléments distincts ou pas), ssi il existe une permutation ϕ de l'ensemble des indices $\llbracket 0, n-1 \rrbracket$ telle que, pour tout $i \in \llbracket 0, n-1 \rrbracket$, $L1[\phi(i)] = L0[i]$, ie : l'élément de la place i se retrouve en place $\phi(i)$.

1. On suppose que P est une liste des permutations de la liste $[b, c, d]$, par exemple :

$$P = [[b, c, d], [c, b, d], [c, d, b], [b, d, c], [d, b, c], [d, c, b]]$$

Écrire les lignes de codes qui permettent d'obtenir une liste Q des permutations de la liste $[a, b, c, d]$.

On utilisera $\mathbf{q = p.copy()}$ ou $\mathbf{q = p[0:]}$ pour copier une liste sans la modifier, $\mathbf{q.insert(0, ...)}$ pour insérer un élément dans q .

2. Écrire une fonction récursive `permutations(L)`, qui prend en argument une liste L et renvoie la liste des permutations de L .

Par exemple :

```
>>> L = ['a', 'b', 'c'];
>>> for p in permutations(L): print(p)
['a', 'b', 'c']
['b', 'a', 'c']
['b', 'c', 'a']
['a', 'c', 'b']
['c', 'a', 'b']
['c', 'b', 'a']
```

3. Justifier que votre programme termine.
4. On note C_n le nombre d'insertions `q.insert(0, x)` dans votre programme. En admettant la correction du programme, donner une relation de récurrence valide à partir d'un certain rang, puis calculer C_n .

Corrigé en 3.3 page 168.

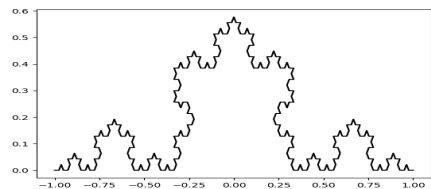
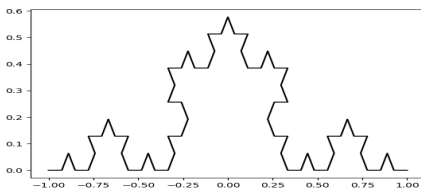
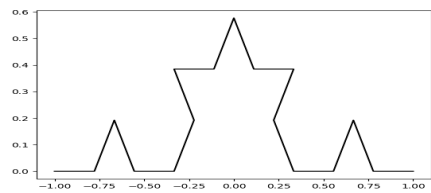
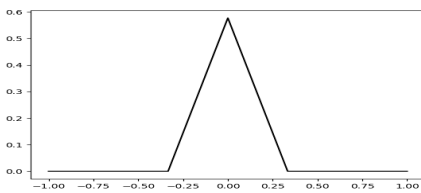
3.1.2 Quelques dessins de fractales

Nous regroupons ici quelques exemples de programmes récursifs permettant de construire des courbes. L'idée générale est la suivante.

- On se donne une liste de segments définissant une **configuration géométrique** (on peut aussi dans le cas d'une ligne polygonale se donner la liste des points).
- On définit une transformation élémentaire qui, à un segment S de la configuration, associe une liste de nouveaux segment construits par **similitudes** à partir des extrémités de S.
- On transforme récursivement la configuration de départ en appliquant la transformation élémentaire à chacun des segments qui la composent.

Exercice 3.4 courbes de Von Koch, version récursive

On reprend ici le tracé des courbes de Von Koch avec une programmation récursive (voir le chapitre 1, page 65 pour une programmation itérative).



Nous allons, pour des raisons de simplicité, représenter les points du plan par des tableaux à deux lignes et une colonne et les matrices par des tableaux à deux lignes et deux colonnes :

```
import numpy as np
A = np.array( [[-1],[0]])
B = np.array( [[ 1],[0]])

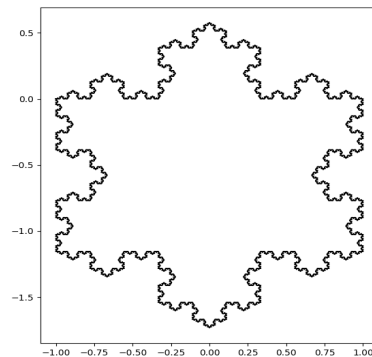
c = numpy.cos(np.pi/3)
s = numpy.sin(np.pi/3)
R = numpy.matrix([[c, -s],[s, c]], dtype=float)
```

et nous commençons par mettre en place des outils pour le tracé de courbes polygonales.

1. Écrire une fonction `is_point(A)` qui prend un tableau défini comme ci-dessus en argument et renvoie `True` s'il est de type `np.ndarray`, de taille deux, et si ses éléments sont des flottants ou des entiers du type `np.int64` (le format des entiers 64 bits de numpy) et `False` sinon.
2. Écrire une procédure `tracer_ligne(L)` qui prend en argument une liste `L` dont les éléments sont des points au sens de la fonction définie dans la question précédente, avec `len(L) > 1` et qui trace la ligne polygonale reliant les points successifs de `L`.
3. Écrire une fonction `transformation_von_koch(A,B)` qui prend en arguments deux points `A` et `B` et retourne le quintuplet (un 5-tuple, donc) `(A, C, D, E, B)` qui correspond à la première figure ; on suppose que les 4 segments sont de même longueur.
4. On transforme une ligne polygonale L_n en appliquant à chacun des ses segments la transformation que l'on vient de définir. La courbe obtenue est notée L_{n+1} . Écrire une fonction récursive `von_koch(L0, n)` qui prend en argument une liste de points L_0 , de longueur supérieure ou égale à 2 et renvoie la liste L_n .

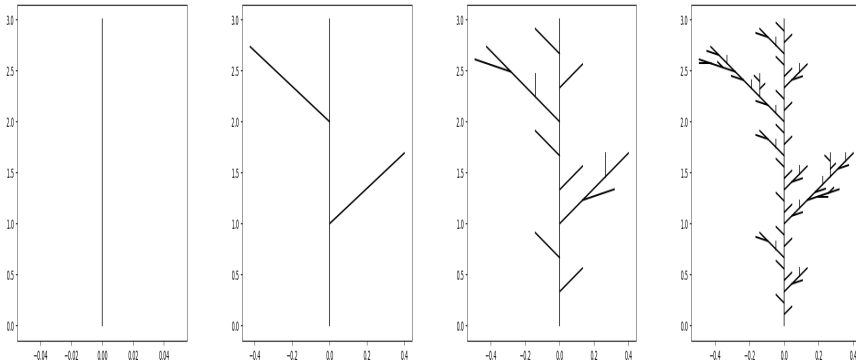
Sur nos figures, L_0 étant le segment $[A,B]$, on a représenté L_1, L_2, L_3 et L_4 .

5. Quel est le nombre d'appels à la fonction `transformation_von_koch()` provoqués par un appel `von_koch(L0, n)` lorsque `len(L0) = m` ?
6. Comment peut-on obtenir la figure ci-contre avec les fonctions et procédures déjà construites dans cet exercice ?



Corrigé en 3.4 page 169.

Exercice 3.5 *exercice vegetal*



On veut réaliser les tracés représentés ci-dessus. La transformation élémentaire que subit chaque segment de la figure n pour obtenir la figure $n + 1$ est visible quand on passe de la première à la deuxième figure.

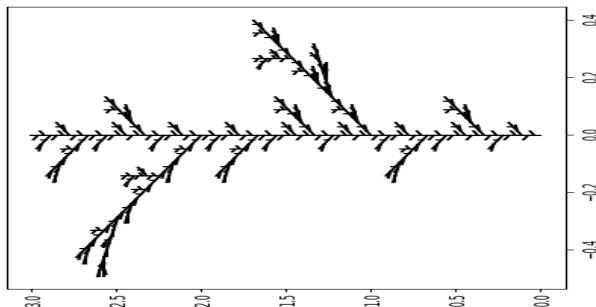
On utilisera les fonctionnalités de numpy comme dans l'exercice 3.4 pour construire points et éventuellement matrices de rotations.

1. Définir, si ce n'est déjà fait la fonction `is_point(A)`. Pensez vous que la procédure `tracer_ligne(L)` conviendrait pour ce problème-ci ?
2. On se propose de définir nos configurations comme des listes de segments. Ainsi, la figure de départ serait $L_0 = [S] = [[A,B]]$ où A et B sont des points définis comme en 3.4.

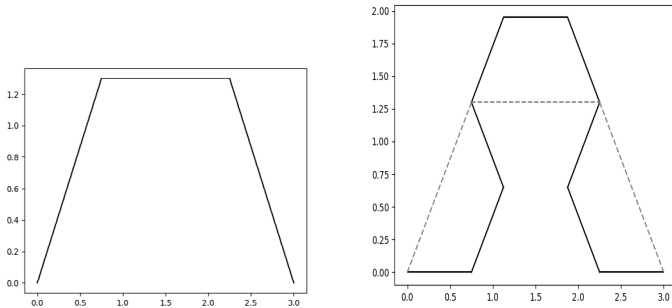
Définir une procédure `tracer_segments(L)` qui prend une liste de segments (donc une liste de listes) et trace la figure composée de ces segments.

3. Définir la fonction `transformation_vegetale(S)` qui prend un segment S en argument, vérifie la précondition, renvoie la liste des segments nécessaires à la construction des figures.
4. Définir une fonction récursive `vegetal(L0, N)` qui renvoie la configuration (liste de segments) construites en n étapes à partir de L_0 .

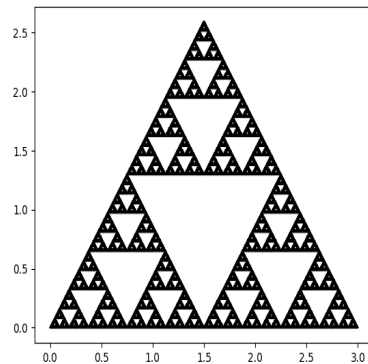
Corrigé en 3.5 page 171.



Exercice 3.6 tapis de Sierpinsky



- On définit toujours nos configurations comme des listes de segments. Ainsi, la figure de départ serait $L_0 = [S] = [[A,B]]$ où A et B sont des points définis comme en 3.4.
Définir une procédure `tracer_segments(L)` qui prend une liste de segments (donc une liste de listes) et trace la figure composée de ces segments.
- Définir une fonction `transformation_sierpinsky(S, theta)` qui prend en arguments un segment S, un flottant theta et renvoie une liste de quatre segments telle que, lorsque $S = [A = [0,0], B = [3,0]]$, et $\theta = \frac{\pi}{3}$, `transformation_sierpinsky(S, theta)` renvoie les segments de la figure de gauche.
- La fonction `transformation_sierpinsky(S, theta)` appliquée successivement à chaque segment de la configuration précédente permet d'obtenir la figure de droite. Comment a-t-on choisi theta dans chaque appel ?
- Écrire une fonction récursive `sierpinsky(L0, n, theta)` qui prend en argument $L = [[A,B]]$ et renvoie la liste des segments permettant de tracer la $n^{ième}$ courbe de la série. On pensera à une bonne alternance de theta, à la fois dans les appels récursifs et dans les appels de `transformation_sierpinsky(S, theta)`. À droite, résultat de l'appel avec $n = 9$.



3.2 Mise en garde concernant l'efficacité des programmes récursifs

L'intérêt d'une programmation récursive est la simplicité de la mise en œuvre lorsque qu'une récurrence simple apparaît dans le problème à modéliser et la facilité avec laquelle on peut conduire les preuves de programmes. Mais une relation simple, programmée telle quelle, peut conduire à une **explosion combinatoire** du nombre des appels récursifs. C'est ce que nous allons illustrer avec le programme qui suit et qui propose de calculer récursivement les termes d'une suite de Fibonacci définie par

$$F_0 = a, F_1 = b, F_n = F_{n-1} + F_{n-2} \text{ si } n \geq 2. \quad (3.2.1)$$

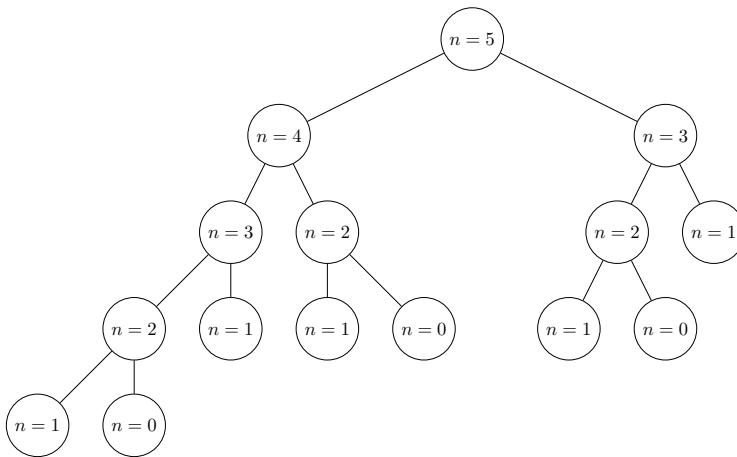
```
def fibol(a,b,n):
    global c
    c =c+1
    print('appel avec n = ', n, ', numéro ', c)
    if n==0:
        return a
    elif n==1:
        return b
    else:
        return fibol(a,b,n-1)+fibol(a,b,n-2)

>>> c = 0;    fibol(1,1,5)
appel avec n = 5 , numéro 1
appel avec n = 4 , numéro 2
appel avec n = 3 , numéro 3
appel avec n = 2 , numéro 4
appel avec n = 1 , numéro 5
appel avec n = 0 , numéro 6
appel avec n = 1 , numéro 7
appel avec n = 2 , numéro 8
appel avec n = 1 , numéro 9
appel avec n = 0 , numéro 10
appel avec n = 3 , numéro 11
appel avec n = 2 , numéro 12
appel avec n = 1 , numéro 13
appel avec n = 0 , numéro 14
appel avec n = 1 , numéro 15
8
```

Notre programme affiche pour chaque appel la valeur de n et le numéro de l'appel.

On y a, en effet, introduit une **variable globale, c , qui nous sert de compteur** du nombre d'appels (nous n'oublierons pas de l'initialiser avant chaque appel).

On voit immédiatement que la fonction est appelée à plusieurs reprises avec les mêmes arguments (a, b ne changent pas). Alors que la version itérative (exercice 1.6 page 38) nous aurait conduit à effectuer 4 itérations, ce programme provoque 15 appels à `fibonacci`, dont la plupart sont redondants. L'arbre qui suit présente l'appel principal suivi des appels récursifs ; si on les suit attentivement à la trace à l'aide de l'affichage précédent, on verra que le parcours est en « profondeur d'abord ».



Cette façon de programmer est donc clairement inefficace (nombre d'appels et occupation mémoire) : pour $n = 30$, par exemple, nous avons $c = 2\,692\,537$.

On peut toutefois réécrire la fonction pour la rendre efficace et c'est ce que nous allons faire en proposant trois méthodes récursives :

- en réécrivant la fonction de telle sorte qu'elle appelle une fonction auxiliaire récursive (qui contient un seul appel, de surcroît **terminal**) ;
- en utilisant l'expression matricielle de la relation de récurrence linéaire d'ordre deux dans l'exercice 3.8 ;
- en utilisant un tableau, ou mieux encore, un dictionnaire pour mémoriser les résultats et éviter les calculs redondants dans l'exercice 3.9 (on parle alors de **mémoïsation**).

★ Nous commençons par la première méthode, plus délicate, mais de portée générale.

L'idée est d'écrire une fonction auxiliaire récursive $G(x, y, n, p)$ qui « transporte » les termes $x = f_{p-1}, y = f_p$ de la suite. Pour cela,

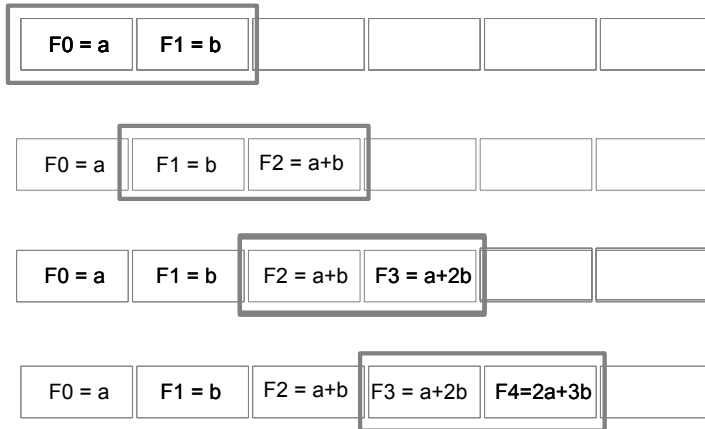
- on initie un appel principal avec $G(a, b, n, 1)$;
- $G(x, y, n, n)$ renvoie y (c'est le critère d'arrêt);
- $G(x, y, n, p)$ appelle $G(y, x + y, n, p + 1)$.

On écrit, comme nous le ferons souvent par la suite, une fonction d'appel $\text{fibonacci}(a, b, n)$ qui utilise cette fonction auxiliaire.

```
def G(x, y, n, p):
    '''
    x, y, n, p :int; avec n >= p.
    Si la précondition: x = F(p-1) et y = F(p) est
    remplie, renvoie y = F(n) et termine avec n==p.
    '''
    if n==p:
        return y
    else:
        return G(y, x+y, n, p+1)

def fibonacci(a, b, n):
    '''
    suite de Fibonacci, appelle G
    '''
    if n==0:
        return a
    elif n==1:
        return b
    else:
        return G(a, b, n, 1)
```

Une preuve de correction est proposée dans l'exercice 3.7. Mais nous allons illustrer un appel principal de la forme $G(a, b, n, 1)$.



La fenêtre glissante, en grisé, indique les valeurs des paramètres formels x et y au cours des appels successifs. On part avec $(x = a = F_0, y = b = F_1, n, p = 1)$, les appels suivants ont pour paramètres $(x = b = F_1, y = a + b = F_2, n, p = 2)$, $(x = a + b = F_2, y = a + 2b = F_3, n, p = 3)$... Lorsque $p = n$, l'algorithme termine avec $y = F_n$.

Exercice 3.7 suites de Fibonacci avec une fonction auxiliaire

Cet exercice fait référence au programme de la page 153.

1. Démontrer par récurrence sur p qu'après un appel principal $G(a, b, n, 1)$ avec comme paramètres $a = f_0, b = f_1$, pour tous les appels récursifs les paramètres effectifs vérifient $x = f_{p-1}, y = f_p$.
2. Montrer que la fonction $\text{fib}(a, b, n)$ termine, produit le résultat attendu. Évaluer le nombre des appels.

Corrigé en 3.7 page 173.

Exercice 3.8 suites de Fibonacci, matrices et exponentiation rapide

Observons que l'on peut réécrire la relation (3.2.1) sous forme matricielle :

$$\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} F_{n-1} \\ F_{n-2} \end{bmatrix} \quad (3.2.2)$$

1. Écrire une relation matricielle entre $\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix}$ et $\begin{bmatrix} F_1 \\ F_0 \end{bmatrix} = \begin{bmatrix} b \\ a \end{bmatrix}$.
2. Écrire une fonction $\text{Puiss}(M, n)$ qui prend pour arguments M une matrice carrée, n un entier naturel et qui renvoie M^n . Cette fonction aura une complexité en $O(\ln n)$ produits matriciels ce que vous justifierez.

On pourra supposer les matrices définies par des instructions analogues à celles qui suivent et utiliser librement la notation $A * B$ pour un produit matriciel...

3. En déduire un calcul rapide de F_n .

*Corrigé en 3.8 page 173***Exercice 3.9** *suites de Fibonacci, utilisation d'un tableau ou d'un dictionnaire*

On se propose de reprendre l'algorithme récursif inefficace présenté en page 151 et de montrer comment, à l'aide d'un tableau, on peut éviter de répéter les calculs inutiles.

Nous utiliserons une particularité du langage Python : lors d'un appel d'une fonction G de la forme $G(T, x)$ dans lequel le paramètre effectif T est une liste, un dictionnaire, un tableau ou une matrice..., *toute modification de T à l'intérieur de la procédure G modifie la variable T dans le processus appelant.*

1. L'idée est la suivante : une fonction d'appel réserve un tableau F , de $n + 1$ valeurs destiné à stocker les f_i au fur et à mesure des calculs. Elle appelle une procédure récursive $G(F, n)$ qui remplit et renvoie $F[n]$ (voir l'encart en fin d'exercice).

```
def fibo(a, b, n):
    if n==0:
        return a
    elif n==1:
        return b
    else:
        F = np.zeros((n+1, ), dtype= int)
        F[0], F[1] = a,b
        G(F,n)
    return F
```

La procédure G évite tout calcul inutile : si lors de l'appel $G(F, n)$, $F[n]$ est déjà calculé elle ne fait rien et renvoie la valeur $F[n]$, sinon elle effectue le calcul récursivement, remplit le tableau et renvoie $F[n]$.

La question : écrire la procédure récursive $G(F, n)$.

2. Ajouter deux compteurs (variables globales) à G pour dénombrer les appels récursifs et les additions. Qu'observera-t-on ?
3. On reprend la même idée (stockage des éléments déjà calculés) mais en utilisant un dictionnaire. Écrire une fonction récursive `fib_dict(a, b, n, D = {})` qui prend en arguments a, b, n : int, un dictionnaire initialisé par défaut comme dictionnaire vide et qui renvoie le $n^{\text{ième}}$ terme de la suite de Fibonacci de premiers termes $f_0 = a$ et $f_1 = b$.

L'appel principal se fait avec D vide, les appels récursifs renvoient directement f_n s'il figure dans le dictionnaire, le calculent et l'ajoutent au dictionnaire sinon.

★ Remarque : L'intérêt de ces méthodes de mémorisation est évident du point de vue des calculs. Du point de vue de la mémoire, il suffirait que `fib` renvoie $F[n]$ plutôt

que F et la place mémoire de F serait immédiatement libérée une fois le processus éteint.

Cette façon de faire est préprogrammée dans certains langages : une option (parfois par défaut) dans les procédures récursives permet de créer en boîte noire un tableau analogue à notre tableau F évitant les calculs redondants.

Corrigé en 3.9 page 174

Mémoïsation et dictionnaires

Vous pourrez mettre en œuvre la technique de **mémoïsation** illustrée dans l'exercice 3.9 en utilisant des **dictionnaires à la place de tableaux** avec les exercices :

- 3.10 page 157 (par lequel on commencera) ;
- 3.11 page 158 sur le nombre de partitions en k parties (d'après un sujet de concours) ;
- 8.15 du chapitre 8, page 311 dans lesquels on construit les sous-listes de longueur p d'une liste donnée.

On rappelle deux choses utiles à ce propos sur le langage Python :

- les fonctions ou procédures de Python admettent des **arguments par défaut** (ils doivent être définis après les autres arguments dans l'en-tête de la fonction) ;
- les contenus des listes, tableaux et dictionnaires envoyés en argument sont affectés par les modifications qui ont lieu dans la fonction ou la procédure.

```
def illustration1(n, x=15):
    '''
    n : int;
    x : int, 15 par défaut.
    '''
    n += 1
    return n + x
```

```
>>> illustration1(17)
33
>>> illustration1(17, 0)
18
>>> illustration1(17, x=0)
18
```

```
def illustration2(n, D):
    '''
    D : dictionary.
    '''
    n += 1
    D[2] = 'hé oui!'
    return None
```

```
>>> n = 6; D = {1: 'a'}
>>> illustration2(n, D)
None
>>> print(D) # D a changé!
{1: 'a', 2: 'hé oui!'}
>>> print(n)
6 # n'a pas changé!
```

3.3 Mise en œuvre

Exercice 3.10 calcul des coefficients binomiaux

On se propose de calculer les coefficients binomiaux à partir de la relation de récurrence : $\binom{n+1}{k+1} = \binom{n}{k} + \binom{n}{k+1}$.

1. Dessiner un arbre où apparaissent les dépendances ou calculs préalables : pour calculer $\binom{n}{k}$ il faut calculer $\binom{n-1}{k-1}$ et $\binom{n-1}{k}$. Que penser d'une fonction récursive qui reposerait sur la formule de récurrence sans adaptation ?
2. Voici un squelette pour une fonction qui calcule ces coefficients sans appel redondant. **On y a juste placé le traitement du dictionnaire.**

```
def coeff_binomial(n, p, D={}):
    ''' mémorisation avec D. '''

    if p > n:
        r =
    elif p == n or p == 0:
        r =
    else:
        # ici 1 <= p <= n-1
        # on va éviter les appels récursifs inutiles
        if n-1 in D and p-1 in D[n-1]:
            r = ...
        else:
            r = ...

        if n-1 in D and p in D[n-1]:
            r += ...
        else:
            r += ...

    # avant de renvoyer le résultat on le stocke
    if n in D:
        D[n][p] = r
    else:
        D[n] = {p : r}
    return r
```

Écrire en vous inspirant de ce schéma, une fonction récursive `coefficient_binomial(n, p, D = {})` qui :

- prend en arguments deux entiers n et p , un dictionnaire (vide par défaut dans l'appel principal) et renvoie le coefficient $\binom{n}{p}$;
- stocke le résultat éventuellement calculé sous la forme d'un dictionnaire dont les clés sont les entiers n et dont les valeurs sont elles-mêmes des dictionnaires de clés $p = 1, 2, \dots, n-1$. Par exemple :
 $D = \{ 4 : \{ 1 : 4, 2 : 6, 3 : 4 \}, 3 : \{ 1 : 3, \dots \}, \dots \}$, d'où $D[4][3] = 4$;

- renvoie immédiatement le résultat dans les cas triviaux et lorsqu'il est déjà renseigné dans le dictionnaire ;
- calcule récursivement le coefficient sinon.

Conseil : relire les rappels sur Python page 156.

Corrigé en 3.10 page 175.

Exercice 3.11 *nombre de partitions en k parties d'un ensemble fini (d'après Centrale PC 2017)*

Cet exercice reprend le sujet de concours reproduit partiellement page 158.

1. Écrire la fonction $S(n, k)$ demandée à la question 3. Les conditions d'arrêt portant exclusivement sur les valeurs conduisant à renvoyer 0 et 1.
2. Minorer sa complexité comme dans la question 4 du problème.
3. Dessiner l'arbre des appels récursifs.
4. Éliminer les appels redondants en utilisant un dictionnaire (comme expliqué dans l'énoncé de l'exercice 3.10).

Corrigé en 3.11 page 176.

Le sujet posé (extraits remaniés - Centrale PC 2017)

Soit E un ensemble non vide. On appelle **partition** de E toute famille $\mathcal{U} = (A_1, \dots, A_k)$ de parties de E telle que :

- chaque A_i est une partie non vide de E ;
- les parties A_i sont deux à deux disjointes, c'est à dire que $i \neq j \Rightarrow A_i \cap A_j = \emptyset$;
- la réunion des A_i forme E tout entier : $E = \bigcup_{i=1}^k A_i$.

Dans tout le problème, pour tout couple d'entiers strictement positifs (n, k) , on note $S(n, k)$ le nombre de partitions de l'ensemble $\llbracket 1, n \rrbracket$ en k parties.

On pose de plus, $S(0, 0) = 1$ et, pour $k > 0, n > 0$, $S(n, 0) = S(0, k) = 0$.

1. Exprimer $S(n, k)$ lorsque $k > n, k = 1$ et $k = n$.
2. Montrer que, pour tous k, n entiers strictement positifs,

$$S(n, k) = S(n - 1, k - 1) + kS(n - 1, k).$$

on pourra distinguer les partitions de $\llbracket 1, n \rrbracket$ selon qu'elles contiennent ou non le singleton $\{n\}$.

3. Rédiger une fonction Python récursive permettant de calculer le nombre $S(n, k)$ par application directe de la formule de récurrence de la question 2.
4. Montrer que pour $n \geq 2$ et $1 \leq k \leq n - 1$, le calcul de $S(n, k)$ par cette fonction récursive nécessite au moins $\binom{n}{k}$ opérations (sommes et produits).

Exercice 3.12 *décompositions et partages d'un entier*

Conseil : Si vous abordez cet exercice tôt dans l'année, vous pouvez le travailler en

deux temps : écrire les fonctions demandées, puis revenir en deuxième lecture, plus tard, pour approfondir complexité et preuves des programmes.

Soit $k, n \in \mathbb{N}^*$. Une *décomposition* de n en k termes est une suite $(x_1, x_2, \dots, x_k)$ d'entiers supérieurs ou égaux à 1 tels que $\sum_{i=1}^k x_i = n$. Une décomposition devient un *partage* si, de plus, c'est une suite *décroissante*.

Observez que les décompositions de $n \geq 1$ en 2 termes sont à l'évidence $(n-1, 1)$, $(n-2, 2)$, ..., $(1, n-1)$, et que l'on obtient une décomposition de n en p termes en concaténant (i) et une décomposition de $(n-i)$ en $p-1$ termes. Par exemple $11 = 7 + (11-7) = 7 + 3 + 1$ est une décomposition (et aussi un partage) de 11 en 3 termes.

1. Écrire une fonction récursive **decompositions**(n, k) qui retourne **une** liste **des** décompositions de n en somme de k termes (supérieurs ou égaux à 1). Votre fonction vérifiera la **pré-condition** $1 \leq p \leq n$ et la **post-condition** sur le résultat (pour tout $\ell \in L$, $\text{len}(\ell) = p$ et $\sum_{i \in \ell} i = n$).
2. On note $D(n, k)$ le nombre de décompositions de n en somme de k termes ($1 \leq k \leq n$). Calculer $D(n, 2)$. Exprimer $D(n, k)$ en fonction des $D(n-i, k-1)$ et en déduire $D(n, 3)$. La conjecture $D(n, k) = \binom{n-1}{k-1}$ vous paraît-elle plausible ?

Si oui, démontrer cette formule en considérant l'ensemble des • ci-dessous : en plaçant judicieusement **deux** barres verticales, on construit une décomposition de 12 en **trois** parties. Ici, $12 = 3 + 4 + 5$.

• • • • • • • • • • • •
• • • | • • • • | • • • • •

3. Prouver la terminaison de votre programme.
4. On note $A(n, k)$ le nombre d'appels récursifs provoqués en cascade par un appel principal **decompositions**(n, k) avec $\text{len}(L) = n \geq 1$. Déterminer une relation exprimant $A(n, k)$ en fonction des $A(n-i, p-1)$. Préciser $A(n, 1)$, $A(n, 2)$ et calculer $A(n, 3)$ en fonction de n .
Ajouter des compteurs à votre fonction pour calculer le nombre des appels récursifs (ca), le nombre d'ajouts d'un élément à une liste (cl pour + ou append) et vérifiez vos calculs pour $D(n, p)$ et $A(n, p)$.
5. Que changer à votre fonction pour qu'elle retourne une liste des partages ?

Corrigé en 3.12 page 178.

Exercice 3.13 calcul récursif de la forme binaire d'un entier

On se propose d'écrire une fonction qui associe à un nombre entier sa représentation binaire. Soit $n \in \mathbb{N}$, un entier naturel, on note $n_{(b)}$ sa représentation en base $b > 1$

qui est la suite de symboles ou chiffres représentant des nombres de zéro à $b - 1$ telle que

$$n_{(b)} = a_p a_{p-1} \dots a_0 \Leftrightarrow n = \sum_{k=0}^p a_k b^k$$

Ainsi, le nombre treize s'écrit en base deux [1,1,0,1] avec les symboles 0 et 1 alors qu'il s'écrit en base dix [1,3] avec des symboles choisis dans $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$. On veut représenter n en base deux par un tableau formé de t chiffres 0 ou 1.

1. Remarques préliminaires

- (a) Vérifier que si $2^p \leq n < 2^{p+1}$, alors $0 \leq (n - 2^p) < 2^p$.
 - (b) Écrire un programme qui pour $n \geq 1$ donné retourne l'unique entier p tel que $2^p \leq n < 2^{p+1}$.
 - (c) Cet entier connu, quelle est la longueur de la liste représentant n en binaire ? Quelle relation cela impose-t-il entre t et p ?
 - (d) En observant que $n = 2^p + (n - 2^p)$ montrer que la liste représentant n est somme de celles représentant 2^p et $n - 2^p$.
2. On commencera par mettre les idées sur le papier avant de programmer. On suppose que le langage choisi permet de construire un tableau de taille t contenant des 0. On fera attention à l'indexation de 0 à $t-1$.
- (a) Décrire les étapes d'une fonction récursive **d2b(n,t)** qui prend en arguments deux entiers n et t (ce dernier supposé tout d'abord suffisamment grand) et retourne un tableau de taille t contenant des 0 et des 1 et représentant la suite des chiffres de n en base 2. Les premiers chiffres peuvent être égaux à 0.
 - (b) Donner un encadrement du nombre d'appel récursifs en fonction de n .
 - (c) Modifier votre procédure de telle sorte que l'appel principal admette sans erreur t quelconque et retourne le tableau de longueur optimale permettant de représenter n en binaire.

Indications : pour créer un tableau sous Python vous pouvez faire
`numpy.zeros(t, int)`

Corrigé en 3.13 page 181.

3.4 Diviser pour régner

Nous présentons dans ce paragraphe l'idée générale d'un principe qui consiste, pour traiter un problème de taille n , à le décomposer récursivement en sous-problèmes. Cela n'a d'intérêt que si le temps cumulé des appels récursifs est inférieur au traitement direct du problème ou encore si le traitement des données ainsi partagées présente des avantages en terme d'occupation mémoire, d'erreurs de calcul. Cette idée est essentielle dans l'étude des tris que nous présentons ci-dessous, dans le calcul rapide de la transformée de Fourier discrète, dans l'algorithme de Strassen pour

le calcul matriciel, en **géométrie algorithmique** pour calculer l'enveloppe convexe d'une famille de points ou pour la recherche de points les plus rapprochés dans un nuage, et dans bien d'autres circonstances.

3.4.1 Recherche de deux points réalisant la plus petite distance

On se propose d'étudier un algorithme permettant de déterminer deux points réalisant la distance (euclidienne) minimale dans un nuage de points. On travaillera en dimension 2.

Description de l'algorithme récursif

On considère L une liste (ou un tableau) de points. On trie ce tableau selon les abscisses puis les ordonnées (ordre lexicographique).

- Si $\text{len}(L) \leq 3$, on emploie la force brute pour calculer le résultat.

- Sinon, on considère les deux sous-listes $L1$ formée des $n/2$ points les plus à gauche et $L2$ la liste complémentaire ; on observe que ces listes restent ordonnées.

- On détermine les solutions des deux sous problèmes. On note δ_0 la plus petite des distances obtenues.

- La solution de notre problème est soit δ_0 , soit une distance entre un point de $L1$ et un point de $L2$ qui sont dans la bande définie par $\mu - \delta_0 \leq x \leq \mu + \delta_0$ avec μ abscisse telle que si $P \in L1$, $x_P \leq \mu$ et si $Q \in L2$, $\mu \leq x_Q$.

Dans les applications pratiques le nuage de points serait représenté par un tableau de numpy à 2 lignes et N colonnes où chaque colonne $T[:, j]$ (avec $0 \leq j \leq N - 1$), représente un point.

Mais, pour nous permettre de nous focaliser sur l'algorithme, nous allons utiliser des listes de listes avec lesquelles l'écriture est plus rapide.

Exercice 3.14 *points plus proches dans un nuage*

On appellera point une liste de deux flottants $L = [x, y]$.

1. Préalable

(a) Écrire une fonction `distance(P,Q)` qui prend deux points en arguments et renvoie leur distance euclidienne.

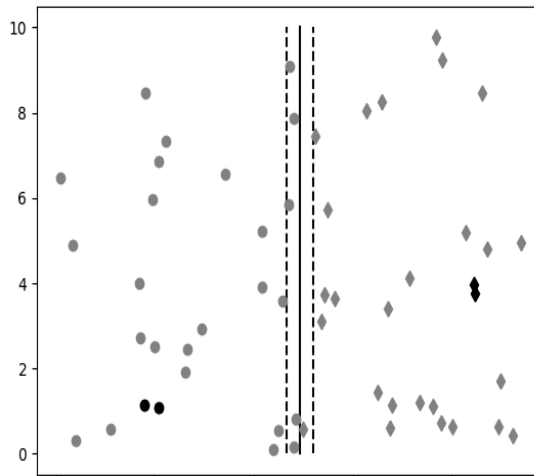
(b) Écrire une fonction `pppBrute(L)` qui prend en argument L , une liste de points, et renvoie un tuple (R,d) tel que R est une liste de deux points de L qui réalisent la distance minimale parmi les couples de points de L et que $d = ||R[0] - R[1]||$.

Combien y a-t-il d'itérations ou de calculs de distances dans votre fonction ?

2. Construire un fonction récursive `ppp(L)` qui prend une liste de points **préala-blement triée** selon l'ordre lexicographique (c'est notre pré-condition) et renvoie un tuple (R,d) avec les mêmes propriétés que le résultat de `pppBrute(L)` (c'est notre post-condition).

Votre fonction mettra en œuvre l'algorithme diviser pour régner présenté ci-dessus. Elle fera appel à `pppBrute(L)` pour des listes de longueurs comprises entre 2 et 3 et pour déterminer si deux points de la bande sont à une distance strictement inférieure aux distances minimales renvoyées par les appels récursifs.

Corrigé en 3.14 page 183.



En noir les points les plus proches dans chaque sous-liste.

Le script qui suit montre comment construire une liste pour vos tests.

```
def nuage_aleatoire(N, x_max, y_max):
    """
    Renvoie une liste représentant N points
    de coordonnées
    tq. 0 <= x <= x_max et 0 <= y <= y_max.
    """
    L = []
    for i in range(0,N):
        x = random.uniform(0, x_max)
        y = random.uniform(0, y_max)
        L.append([x,y])
    L.sort()
    #ce sera par défaut un ordre lexicogr.
    return L
```

3.4.2 Exponentiation rapide, version récursive

Nous avons proposé en (2.5.2) une version itérative de l'exponentiation rapide. Nous avons montré comment réduire le nombre de multiplications pour le calcul d'une puissance entière a^n en utilisant l'écriture binaire de l'entier n .

Nous revenons sur cet algorithme qu'il est plus aisé de programmer de manière récursive. On reprend pour illustrer la méthode le calcul de a^{37} comme nous l'avons fait pour présenter la version itérative, mais nous organisons ici les calculs d'une autre façon. La méthode repose sur les observations qui suivent :

pour calculer a^{37} , on calcule $a^{18} \times a^{18} \times a$
pour calculer a^{18} , on calcule $a^9 \times a^9$
pour calculer a^9 , on calcule $a^4 \times a^4 \times a$
pour calculer a^4 , on calcule $a^2 \times a^2$
pour calculer a^2 , on calcule $a \times a$,

ce qui demande 7 multiplications (au lieu de 36 lorsqu'on multiplie linéairement $a \times a \times a \times \dots \times a$).

Exponentiation rapide	Trace
<pre>def expo(a,n): print(n) if n==0: return 1 else: q, r = divmod(n,2) p = expo(a,q) p = p*p if r==1: p = p*a return p</pre>	<pre>>>> expo(2,37) 37 18 9 4 2 1 0 137438953472 >>> 2**37 137438953472</pre>

L'exercice qui suit, dont l'énoncé est détaillé, propose une première étude de correction et de complexité pour une fonction récursive. Il illustre un principe assez général, selon lequel l'étude d'un algorithme récursif est plus facile que celle d'un algorithme itératif équivalent.

Exercice 3.15 analyse de l'exponentiation rapide récursive

1. Un appel $\text{expo}(a,n)$ termine-t-il toujours ?
2. Prouver que, pour tout $n \in \mathbb{N}$, $\text{expo}(a,n) = a^n$. Vous procéderez par récurrence sur n en observant que l'on peut toujours écrire $n = 2q + r \dots$

3. On veut évaluer $C(n)$ le nombre de multiplications et $A(n)$ le nombre d'appels à *expo* provoqués par un appel principal $\text{expo}(a, n)$ (il ne dépend pas de a). On note toujours $n = 2q + r$ ($r = 0, 1$).
- (a) A-t-on $m \leq n \Rightarrow C(m) \leq C(n)$?
A-t-on $m \leq n \Rightarrow A(m) \leq A(n)$?
 - (b) Montrer que $C(n) = C(q) + u_n$ préciser u_n en fonction de la parité de n (introduire par exemple $(-1)^n$) ou en fonction de r .
 - (c) Montrer que si $n = 2q + r \leq n' = 2q' + r'$, alors, $q < q'$ ou $q = q'$ et $r = 0 < r' = 1$.
 - (d) Calculer $C(2^p)$ et $C(2^p - 1)$.
 - (e) Calculer ou encadrer $A(n)$ et $C(n)$ sachant que $2^p \leq n < 2^{p+1}$.
Exprimer $A(n)$ en fonction de $\log_2(n)$.

Corrigé en 3.15 page 184.

Exercice 3.16 un programme bâclé

Que-pensez vous de cette façon de programmer l'exponentiation rapide :

```
def expo(a,n):
    print(n)
    if n==0:
        return 1
    else:
        q, r = divmod(n,2)
        p = expo(a,q)*expo(a,q)
        if r==1:
            p = p*a
        return p
```

Corrigé en 3.16 page 185.

Exercice 3.17

Réécrire la fonction d'exponentiation rapide *e*, n'ayant recours à aucune variable locale.

Corrigé en 3.17 page 185

3.4.3 Recherche dichotomique dans une liste triée

Nous avons présenté page 117 une version itérative de l'algorithme de recherche dichotomique dans une liste déjà triée. On se propose d'en donner ici une version récursive.

Exercice 3.18 Recherche dichotomique dans une liste triée

1. On veut définir une fonction `est_croissante(L)` qui prend en argument une liste `L` et renvoie `True` ou `False` selon que `L` est ordonnée dans un ordre croissant ou pas.

En écrire une version simplifiée adaptée à l'ordre '`<=`' défini pour certains types en Python (`int`, `float`, `str`,...).

2. Écrire une fonction `est_croissante(L, f)` qui prend en arguments une liste `L` et une fonction `f` où :

- La fonction `f` prend en arguments deux objets `a` et `b` de type `T` (`int`, `float`, `str`, `list`,...) et définit un ordre total sur la classe `T`.

- La fonction `est_croissante(L, f)` renvoie `True` ou `False` selon que les éléments sont ordonnés par $f(a, b) = \text{True}$ ou pas.

3. Écrire une **version récursive de l'algorithme de recherche dichotomique** de x dans `L`, liste triée.

On rappelle le principe de division pour le règne dans ce cas :

- `L` est supposée croissante ;

- On partage `L` en deux sous-listes de taille $\frac{n}{2}$ si n est pair, de tailles $\lfloor \frac{n-1}{2} \rfloor \pm 1$ si n est impair.

- On recherche récursivement x dans celle de ses sous-listes qui est susceptible de contenir x .

Votre fonction aura la **signature** suivante :

`recherche_dichotomiqueR(L, x, f = lambda a,b : a<= b) : booléen, int|None`

et renverra le tuple **True, p** si x est en position `p` dans `L` et le tuple **False, None** sinon. Elle testera par ailleurs la pré-condition : `L` est croissante.

Remarque : La fonction `lambda` qui donne la valeur de `f` par défaut correspond à l'ordre `<=`.

4. Donner une preuve de terminaison de votre programme ; prouver sa correction.
5. Calculer le nombre maximal d'appels récursifs.

Corrigé en 3.18 page 186.

3.4.4 Illustration avec des tris

Tri rapide (quicksort)

Le principe de ce tri est le suivant : pour trier une liste L ,

- si L est vide ou admet un seul élément, elle est triée (c'est notre critère d'arrêt) ;
- sinon
 - on choisit un élément $e \in L$, quelconque, que l'on retire de L ;
 - on construit deux sous-listes : Li formée des éléments inférieurs ou égaux à e , Ls formée des éléments plus grands ;
 - le résultat est la concaténation de Li récursivement triée, de $[e]$, et de Ls elle aussi récursivement triée.

La présentation détaillée et l'analyse figurent page 192 dans le chapitre consacrée aux algorithmes de tris.

Tri fusion

Donnons avant tout le principe de ce tri détaillé page 194 :

- On observe que pour rassembler deux tableaux T_1 et T_2 déjà triés, on peut les interclasser de la façon suivante : on compare les plus petits éléments de chacun d'eux, on place le plus petit des deux dans un nouveau tableau T . On poursuit en comparant successivement les deux plus petits éléments non encore triés de ces tableaux jusqu'à épuisement de l'un d'eux. Il ne reste plus qu'à ajouter les éléments du tableau non vidé, qui sont classés et plus grands que les éléments déjà placés dans T . C'est la fonction $\text{interclassement}(T_1, T_2)$ qui réalise cela.
- Le tri fusion est alors défini de la façon suivante :
 - critère d'arrêt : le tableau a au plus un élément (il est déjà trié) ;
 - si le tableau a plus d'un élément, on appelle récursivement la fonction sur chacun des sous-tableaux et on interclasse leurs copies triées.

3.5 Corrigés des exercices

Corrigé de l'exercice n° 3.1

Euclide avec les seules soustractions :

```
def pgcd(a,b):
    if a < b:
        return pgcd(b,a)
    elif b==0:
        return a
    else:
        return pgcd(a-b,b)
```

Corrigé de l'exercice n° 3.2

```
import numpy as np
import pylab as pl

def Cantor(n, x):
    """
    n : int avec n >= 0;
    x : int|float (entre 0 et 1)
    """
    assert isinstance(x, (int, float)) and x >= 0 and x <= 1
    assert isinstance(n, int) and n >= 0
    if n == 0:
        return x
    elif x >= 0 and x <= 1/3:
        return Cantor(n-1, 3*x)/2
    elif x > 1/3 and x < 2/3:
        return 1/2
    elif x >= 2/3 and x <= 1:
        return (1+ Cantor(n-1, 3*x-2))/2

def tracesCantor(n):
    X = np.linspace(0,1,100)
    Y = [Cantor(n,x) for x in X]
    pl.plot(X,Y)
    pl.show()
```

Corrigé de l'exercice n° 3.3

1. 2.

```

Q = []
for p in P:
    for i in range(0, len(p)+1):
        # on insère pour les n+1 positions: 0 à n incluse!
        q = p.copy()
        ''' attention on n'altère pas p qui va servir
           dans l'itération suivante!'''
        q.insert(i, a)
        Q.append(q)

def permutations(L):
    '''
    L: list.
    Renvoie la liste des permutations de L.
    '''
    global c
    # compteur, initialisé avant l'appel principal
    if len(L) <= 1:
        return [L]
    else:
        a, R = L[0], []
        P = permutations(L[1:])
        for p in P:
            for i in range(0, len(p)+1):
                q = p.copy()
                q.insert(i, a)
                c += 1
                # on compte l'insertion
                R.append(q)
        return R

```

3. Un appel `permutations(L)` avec $\text{len}(L) = n$, termine si $n = 0, 1$.

Supposons que tous les appels avec des listes de longueur n terminent. Un appel avec une liste de longueur $n + 1$ provoque un appel récursif avec une liste $(L[1 :])$ de longueur n qui, par hypothèse de récurrence termine. Comme il est suivi d'un nombre fini d'itérations, la fonction termine.

4. **Complexité** Clairement, $C_0 = C_1 = 0$.

On suppose `permutations(L)` correcte pour toute longueur de liste.

Dans la double boucle pour un appel `permutations(L)` avec $\text{len}(L) \geq 2$, il y a donc $(n-1)!$ itérations de la boucle extérieure et pour chacune de ces itérations, n itérations de la boucle intérieure. Cela fait donc $n \times (n-1)! = n!$ instructions `q.insert()`.

On aura donc $C_n = n! + C_{n-1}$ pour tout $n \geq 2$.

Soit $C_n = n! + (n-1)! + (n-2)! \dots + 2!$ si $n \geq 2$.

Corrigé de l'exercice n° 3.4

1.2.3.4. Les quatre scripts ci-dessous :

```
import numpy as np
import pylab as pl

def is_point(A):
    ''' '''
    r = isinstance(A, np.ndarray)
    r = r and len(A) == 2
    r = r and isinstance(A[0,0], (float, int, np.int64))
    r = r and isinstance(A[1,0], (float, int, np.int64))
    return r

def tracer_ligne(L):
    '''
    L : list; contient des points et len(L) >=2.
    Trace la ligne polygonale.
    '''
    X = [ A[0, 0] for A in L]
    Y = [ A[1, 0] for A in L]
    pl.plot(X,Y)
    pl.show()

def transformation_von_koch(A, B):
    '''
    A, B: points;
    '''
    assert is_point(A) and is_point(B)
    c = np.cos(np.pi/3)
    s = np.sin(np.pi/3)
    R = np.matrix([[c, -s],[s, c]], dtype =float)

    C = A + (1/3)*(B-A)
    E = A + (2/3)*(B-A)
    D = C +R*(E-C)
    #image de E par la rot. de centre C, d'angle pi/3.
    return A, C, D, E, B
```



```
def von_koch(L0, n):
    """
    L0 :list ; len(L0) >=2, L0 est formée de
    tuples(x: float, y: float);
    n : int;
    Renvoie une liste transformée n fois de L0 par
    la transformation de Von Koch.
    """
    assert isinstance(n, int) and n >=0
    if n == 0:
        return L0
    else:
        L1 = [L0[0]]
        for i in range(1, len(L0)):
            A, B = L0[i-1], L0[i]
            A, C, D, E, B = transformation_von_koch(A,B)
            L1 += [C,D,E,B]

        return von_koch(L1, n-1)
```

5. Complexité

Le nombre d’appels **directs** de transformation_von_koch() dans von_koch(L0, n) est égal au nombre des segments soit $m - 1$. Cet appel construit une liste de $4(m - 1)$ segments pour l’appel récursif von_koch(L1, n-1)

paramètres de l'appel	nbre. de segments en entrée	nbre. d'appels directs
$vk(L_0, n)$	$(m - 1)$	$(m - 1)$
$vk(L_1, n - 1)$	$4(m - 1)$	$4(m - 1)$
$vk(L_2, n - 2)$	$4^2(m - 1)$	$4^2(m - 1)$
$\vdots$	$\vdots$	$\vdots$
$vk(L_{n-1}, 1)$	$4^{n-1}(m - 1)$	$4^{n-1}(m - 1)$
$vk(L_n, 0)$	$4^n(m - 1)$	0

Ce qui au total nous fait

$$(m - 1) \sum_{k=0}^{n-1} 4^k = \frac{1}{3}(m - 1)(4^n - 1)$$

appels, ce qui est une **complexité exponentielle** en n .

6. Pour obtenir le flocon, on choisit $L0 = [A, B, C]$ avec ABC équilatéral par exemple.

Corrigé de l'exercice n° 3.5

1. Pour `is_point(A)`, voir le corrigé de l'exercice 3.4. La fonction de tracé de l'exercice 3.4 ne convient pas : elle trace les segments reliant les points successifs et nos figures ne sont pas des lignes polygonales.

2.3.4.

```
def tracer_segments(L, file = None):
    """
    L : list; contient des segments [A,B] où
    A et B sont des points.
    Trace les segments.
    """
    for S in L:
        A, B = tuple(S)
        X = [A[0,0], B[0,0]]
        Y = [A[1,0], B[1,0]]
        pl.plot(X, Y, color = 'black')

    if file == None:
        pl.show()
        pl.close()
    else:
        pl.savefig(directory + file + '.png')
        pl.close()

def transformation_vegetale(S, theta = numpy.pi/6):
    """ S: segment"""

    assert isinstance(S, list) and len(S)==2
    A, B = S[0], S[1]
    assert is_point(A) and is_point(B)

    c = numpy.cos(theta)
    s = numpy.sin(theta)
    R = numpy.matrix([[c, s], [-s, c]], dtype =float)

    C = A + (1/3)*(B-A)
    E = A + (2/3)*(B-A)
    D = C + (0.8)*(R*(E-C))
    F = E + (0.85)*R**(-1)*(B-E)
    return [[A,C], [C,D], [C,E], [E,F], [E,B]]
```

```
def vegetal(L0, n):
    '''
    L0 :list ; len(L0) >=1, L0 est formée de
    segments; un segment est une liste de deux
    points;
    n : int;
    Renvoie une liste transformée n fois de L0 par la
    transformation_vegetale.
    '''
    assert isinstance(n, int) and n >=0
    if n == 0:
        return L0
    else:
        L1 = []
        for S in L0:
            L1 += transformation_vegetale(S)
        return vegetal(L1, n-1)
```

Corrigé de l'exercice n° 3.6

Le code sans autre forme de procès. Toute l'attention doit porter sur l'alternance des signes.

```
def transformation_sierpinsky(S, theta):
    '''
    S: segment;
    Renvoie un liste de 3 segments.
    '''
    assert isinstance(S, list) and len(S)==2
    A, B = S[0], S[1]
    assert is_point(A) and is_point(B)

    c = numpy.cos(theta)
    s = numpy.sin(theta)
    R = numpy.matrix([[c, -s],[s, c]], dtype =float)

    C = A + (1/2)*R*(B-A)
    D = B + (1/2)*R**(-1)*(A-B)
    # changement de sens de rotation!

    return [[A,C],[C,D],[D,B]]
```

```

def sierpinsky(L0, n, theta = numpy.pi/3):
    """
    L0 : list ; len(L0) >=1, L0 est une
    liste de points (ligne polygonale);
    n : int;
    Renvoie une liste de points obtenue par n itérations
    du procédé.
    """
    assert isinstance(n, int) and n >=0
    if n == 0:
        return L0
    else:
        L1 = []
        for i, S in enumerate(L0):
            L1 += transformation_sierpinsky(S, \
                                             (-1)**(i)*theta)
            # Alternance du sens pour chaque segment!

        # Changement du sens pour le premier segment.
        return sierpinsky(L1, n-1, -theta)

```

Corrigé de l'exercice n° 3.7

1. Lorsque l'appel principal pour la fonction auxiliaire G est de la forme $G(f_0, f_1, n, 1)$, on prouve sans difficulté que pour tout $p \in \mathbb{N}$, le $p^{\text{ième}}$ appel à G est de la forme $G(f_{p-1}, f_p, n, p)$. En effet,
 - c'est clair lorsque $p = 1$;
 - supposons que le $p^{\text{ième}}$ appel ait pour paramètres (f_{p-1}, f_p, n, p) si $n = p$ c'est terminé (dernier appel); sinon l'appel suivant admet pour paramètres $y = f_p, x + y = f_{p-1} + f_p = f_{p+1}, n, \text{ et } p + 1$.
 La récurrence est établie.

2. La preuve de correction est immédiate, l'invariant lors d'un appel principal $G(a, b, n, 1)$ est $x = f_{p-1}, y = f_p$ pour l'appel $G(x, y, n, p)$. On termine lorsque $p = n$ et $y = f_p$.

La succession des appels récursifs se fait avec $p = 1, 2, \dots, n$. Il y a donc n appels.

Corrigé de l'exercice n° 3.8

De la relation (3.2.2) on déduit immédiatement $\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1} \begin{bmatrix} F_1 \\ F_0 \end{bmatrix}$. On définit alors une fonction qui calcule les puissances de matrices en collant à l'exponentielle rapide déjà écrite. On est attentif au décalage de l'indice n , on pense à travailler avec des entiers...

```

import numpy as np

def Puiss(M, n):
    if n==0:
        return np.matrix(np.eye(2), dtype=int)
    elif divmod(n,2)[1]==0:
        return Puiss(M, n//2)**2
    else:
        return (Puiss(M,n//2)**2)*M

def fibo_m(a,b, n):
    if n ==0:
        return a
    else:
        M = np.matrix([1,1,1,0], dtype=int).reshape((2,2))
        F0= np.matrix([b,a],dtype=int).reshape(2,1)
        return (Puiss(M, n-1)*F0)[0,0]

```

Corrigé de l'exercice n° 3.9

1. La fonction G sous Python.

```

def G(F,n):

    global c_app, c_add
    c_app += 1 # compte les appels

    if F[n] != 0:
        return F[n]
    else:
        c_add += 1 # compte les additions
        F[n] = G(F, n-1) + G(F, n-2)
    return F[n]

```

2. On initialise avant chaque appel principal dans fibo, $c_{app} = 0$; $c_{add} = 0$;
3. Avec un dictionnaire et sans fonction auxiliaire :

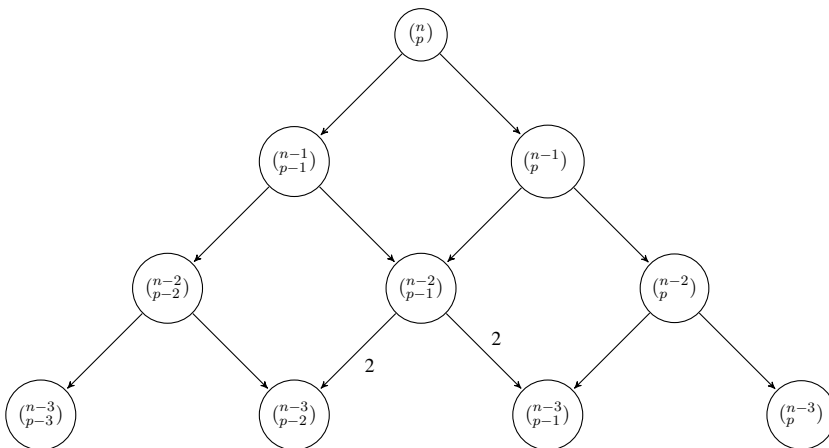
```
def fibo_dict(a, b, n, D = {}):
    global c_app, c_add
    c_app += 1

    if n in D:
        pass
    elif n == 0:
        D[0] = a
    elif n==1:
        D[1] = b
    else:
        D[n] = fibo_dict(a,b,n-1,D)+fibo_dict(a,b,n-2,D)
        c_add += 1
    return D[n]

>>> c_app = c_add = 0
>>> r = fibo_dict(1, 1, 300); print(r, c_app, c_add)
573147844013817084101 199 99
```

Corrigé de l'exercice n° 3.10

1. Un telle fonction ferait des calculs redondants. On observe qu'à la profondeur 3, le calcul de $\binom{n-3}{p-1}$ produirait 3 appels : deux venant du calcul de $\binom{n-2}{p-1}$ lui-même appelé deux fois, et un troisième venant du calcul de $\binom{n-2}{p}$...



2. La fonction complète, avec un compteur pour les appels récursifs :

```
def coeff_binomial(n, p, D={}):
    ''' mémorisation avec D. '''
    global c
    c += 1 # compteur des appels

    if p > n:
        r = 0
    elif p == n or p == 0:
        r = 1
    else:
        #ici 1 <= p <= n-1
        if n-1 in D and p-1 in D[n-1]:
            r = D[n-1][p-1]
        else:
            r = coeff_binomial(n-1, p-1)

        if n-1 in D and p in D[n-1]:
            r += D[n-1][p]
        else:
            r += coeff_binomial(n-1, p)

    if n in D:
        D[n][p] = r
    else:
        D[n] = {p : r}
    return r
```

Corrigé de l'exercice n° 3.11

1.

```
def S(n, k):
    '''
    n, k :int;  n>=0, k >=0.
    Renvoie le nombre de partitions d'un ensemble de n
    éléments en k parties (0 <= k <= n).
    '''
    if k == n:
        return 1
    elif k == 0 or n == 0 or k > n:
        return 0
    else:
        return S(n-1, k-1) + k*S(n-1, k)
```

2. On note $c(n, k)$ le nombre d'opérations lors d'un appel $S(n, k)$.

On a $c(n, n) = c(n, 0) = c(0, k) = 0$ et $c(n, k) = 0$ si $k > n$.

Pour $1 \leq k \leq n-1$, il vient $c(n, k) = c(n-1, k-1) + c(n-1, k) + 2$.

Par récurrence sur $n \geq 2$, montrons la propriété

$$\mathcal{P}(n) = \{ \text{pour tout } k \text{ tel que } 1 \leq k \leq n-1, c(n, k) \geq \binom{n}{k} \}.$$

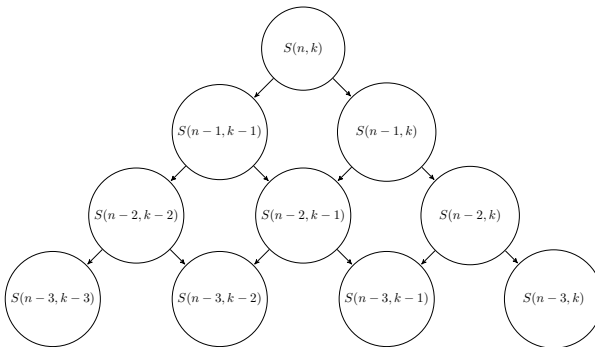
- $\mathcal{P}(2)$ est vérifiée car $c(2, 1) \geq 2 = \binom{2}{1}$.
- Supposons $\mathcal{P}(n)$ satisfaite pour un certain $n \geq 2$. Pour $1 \leq k \leq n$ on a donc :
 $c(n+1, k) = c(n, k-1) + c(n, k) + 2$. Cela nous conduit à envisager les cas :
 - (a) $2 \leq k \leq n-1$, pour lequel on a :

$$c(n+1, k) = c(n, k-1) + c(n, k) + 2 \geq \binom{n}{k-1} + \binom{n}{k} + 2 \geq \binom{n+1}{k} + 2;$$
 - (b) $k = 1$, pour lequel on a :

$$c(n+1, 1) = c(n, 0) + c(n, 1) + 2 \geq 0 + \binom{n}{1} + 2 \geq \binom{n+1}{1};$$
 - (c) et enfin, $k = n$, ce qui donne :

$$c(n+1, n) = c(n, n) + c(n, n-1) + 2 \geq 1 + \binom{n}{n-1} + 2 \geq \binom{n+1}{n} = n+1.$$

3. L'arbre fait apparaître des appels redondants.



4. Utilisation d'un dictionnaire pour réduire les appels récursifs :

```
def S_d(n, k, D = {}):
    """
    n, k :int;
    Renvoie le nombre de partitions d'un ensemble de n
    éléments en k parties (1 <= k <= n).
    """
    global ca, cop
    # appels récursifs et opérations
    ca += 1

    if k==0 or n == 0 or k > n:
        return 0
    elif k == 1 or k == n:
        return 1
```



```
else:
    if n-1 in D and k-1 in D[n-1]:
        r = D[n-1][k-1]
    else:
        ca += 1
        r = S_d(n-1, k-1, D)

    if n-1 in D and k in D[n-1]:
        cop += 2
        r += k*D[n-1][k]
    else:
        ca += 1
        cop += 2
        r += k*S_d(n-1, k, D)

    if n in D:
        D[n][k] = r
    else:
        D[n] = {k : r}
    return r
```

Comparaison grâce aux compteurs (ca : appels récurrents, cop : opérations + et ×) :

appel	résultat	ca	cop	$\binom{n}{p}$
S(15,7)	408741333	6005	6004	6435
S_d(15,7)	408741333	53	48	6435

Corrigé de l'exercice n° 3.12

1. Le script avec une fonction check(L, n, p) pour la post-condition, les compteurs ca et cl (voir dernière question).

```
def check(L, n):
    '''
    L: list de listes.
    Vérifie que chaque élément de L est
    une liste dont la somme des termes est n.
    '''
    for elt in L:
        assert isinstance(elt, list) and len(elt) == p \
            and sum(elt) == n
    return True
```

```
def decompositions(n, p):
    '''      '''
    assert isinstance(p, int) and p>=1 and p <=n

    global ca, cl # compteurs (appels et append)
    ca += 1
    if p==1:
        return [[n]]
    else:
        L = []
        for i in range(1, n-p+2):
            # pour les appels récursifs: 1<= p-1<=n-i
            for ell in decompositions(n-i, p-1):
                L.append([i] + ell)
                cl += 2 # pour append et +

        check(L,n) # post-condition
    return L
```

2. • **Formule de récurrence** : On obtient les décompositions de n en $k \geq 2$ termes, en considérant les décompositions $i + [...]$ où $[...]$ est une décomposition de $n - i$ en $k - 1$ termes. La contrainte sur i est, comme dans notre programme, $1 \leq i \leq n - k + 1$. On a donc

$$D(n, k) = \sum_{i=1}^{n-k+1} D(n-i, k-1).$$

On a clairement $D(n, 1) = 1$ et $D(n, 2) = (n - 1)$. Lorsque $k = 3$, la formule de récurrence donne :

$$D(n, 3) = \sum_{i=1}^{n-2} D(n-i, 2) = \sum_{i=1}^{n-2} (n-i-1) = (n-2) + \dots + 1 = \frac{(n-1)(n-2)}{2}.$$

On retrouve bien là $D(n, 2) = \binom{n-1}{1}$, $D(n, 3) = \binom{n-1}{2}$.

• **Une démonstration sans calcul** nous est donnée par la figure de l'énoncé : pour réaliser une décomposition de n en k parties, il suffit de placer $k - 1$ barres verticales dans les $n - 1$ emplacement entre deux •. Ce qui nous fait $\binom{n-1}{k-1}$ choix possibles.

• Quelle relation remarquable visible sur le triangle de Pascal retrouvons-nous là ?

3. **Terminaison** : Cette fonction termine si $p = 1$. Les appels récursifs se font avec des valeurs effectives $p, p-1, p-h, \dots$. Ces appels ne peuvent excéder la profondeur $p - 1$. Par ailleurs à chaque appel la boucle est finie $(n - p + 1)$ itérations.

4. Nombre d'appels récursifs

Notons $a(n, p)$ le nombre d'appels récursifs directs (de profondeur 1) provoqués par un appel principal avec comme paramètres n et p et $A(n, p)$ le nombre total des appels récursifs induits (ou de profondeur supérieure à 1).

Clairement : $a(n, p) = n - p + 1$ et $A(n, p) = a(n, p) + \sum_{i=1}^{n-p+1} A(n-i, p-1)$.

Il vient alors, $A(n, 1) = 0$, $A(n, 2) = a(n, 2) = n - 1$ et

$$A(n, 3) = (n - 3 + 1) + \sum_{i=1}^{n-1} A(n-i, 2) = (n - 3 + 1) + \sum_{j=2}^{n-1} A(j, 2) = \frac{n^2 - n - 2}{2}$$

Le script pour afficher les résultats et vérifier les calculs des $D(n, p)$ et $A(n, p)$.

<pre>ca, cl = 0, 0 n, p = 11, 3 L = decompositions(n, p) print(n, p, len(L), ca, cl) for ell in L: print(ell)</pre>	<pre>11 3 45 54 180 [1, 1, 9] [1, 2, 8] ... [7, 2, 2] [7, 3, 1] [8, 1, 2] [8, 2, 1] [9, 1, 1]</pre>
--	---

5.

```
def partages(n, p):
    assert isinstance(p, int) and p >= 1 and p <= n
    if p == 1:
        return [[n]]
    else:
        L = []
        for i in range(1, n-p+2):
            for ell in partages2(n-i, p-1):
                if i >= ell[0]:
                    L.append([i] + ell)
        return L

n, p = 20, 3
L = partages(n, p)
```

```

print(n, p, len(L))
n, p  = 20, 3
L      = partages(n, p)

for ell in L:
    print(ell)

```

```

20 3 33
[7, 7, 6]
[8, 6, 6]
[8, 7, 5]
[8, 8, 4]
...
[16, 2, 2]
[16, 3, 1]
[17, 2, 1]
[18, 1, 1]

```

Corrigé de l'exercice n° 3.13

1. (a) C'est immédiat : on retranche 2^p et on a $0 \leq n - 2^p < 2^{p+1} - 2^p = 2^p$.
- (b) Si $n \geq 1$, en écrivant

```

p      = 0
while 2**(p+1) <= n:
    p=p+1

```

au moment de la première évaluation on a $2^p = 2^0 \leq n$.

Invariant de boucle : tant que la condition est satisfaite on a au moment de l'évaluation suivante $2^p \leq n$.

Comme la condition d'arrêt est $2^{p+1} > n$ on est donc assuré d'avoir $2^p \leq n < 2^{p+1}$ en sortie de boucle.

- (c) Si $2^p \leq n < 2^{p+1}$, il nous faut au moins $p + 1$ termes dans la liste pour représenter n .
- (d) On observe que $n = 2^p + (n - 2^p) = 2^p + n_1$ avec $0 \leq n_1 < 2^p$ alors

$$n = 2^p + \sum_{k=0}^{p-1} a_k 2^k = [1, 0, \dots, 0] + [0, a_{p-1}, \dots, a_0]$$

2. (a) Le programme définitif figure en dernière question.
 Si $n = 0$ le problème est résolu avec un tableau de 0 de taille t . C'est la condition d'arrêt.
 Si $n \geq 1$ on calcule p comme indiqué ci-dessus. On réserve un tableau de taille t (on suppose que $t \geq p + 1$). On affecte $b[t-p-1]=1$ (ou $L(t-p)=1$) pour que ce tableau représente n en base 2. On ajoute à ce tableau le tableau résultant de l'appel récursif **d2b**($n - 2^p, t$) qui est un tableau de même taille t représentant $n - 2^p$ en base 2. On sait que c'est le résultat attendu.

- (b) Lorsque $n = 0$; pas d'appel récursif.

Lorsque $n \geq 1$ on sait que les appels successifs se font avec des arguments $n_1, n_2, \dots$ tels que si $2^p \leq n < 2^{p+1}$ on a :

$$0 \leq n_1 < 2^p, 0 \leq n_2 < 2^{p-1}, \dots, 0 \leq n_k < 2^{p-k+1}, \dots$$

Cette suite est strictement décroissante et l'algorithme termine avant que $p - k + 1 = 0$.

Le nombre d'appels récursifs est au plus égal à $k = p + 1$. Pour l'exprimer en fonction de n observons que : $2^p \leq n < 2^{p+1}$, $p \leq \frac{\ln n}{\ln 2} < p + 1$, d'où $k = p + 1 \leq \frac{\ln n}{\ln 2} + 1$.

Bilan Le nombre d'appels récursifs est compris entre 0 (pour $n = 0, 2^p$) et $1 + \frac{\ln n}{\ln 2}$ (pour $n = 2^{p+1} - 1$ ce que l'on vérifie facilement).

- (c) La fonction présentée prend un appel principal avec t quelconque et calcule $t1$ qui sera le même pour tous les appels récursifs.

Supposons que l'appel principal se fasse avec $n \geq 0$ et $t \geq 0$ quelconques. Dès le calcul de p on aura $t1 = \max(t, p + 1)$ de taille suffisante pour contenir tous les chiffres de n . Le premier appel récursif **d2b**($n - 2^p, t1$) ayant pour premier argument un entier plus petit que n ne modifiera pas $t1$ de même que les appels suivants.

```
def d2b(n, t):
    if n==0:
        return np.zeros(t,int)
    else:
        p = 0
        while 2**(p+1)<=n:
            p=p+1
        t1 = max(t,p+1)
        b = np.zeros(t1,int)
        b[t1-1-p] = 1
        return b+d2b(n-2**p,t1)
```

Corrigé de l'exercice n° 3.14

1.

```
def distance(P,Q):
    return ((P[0]-Q[0])**2 + (P[1]-Q[1])**2)**(1/2)

def pppBrute(L):
    n = len(L)
    assert n >= 2
    R0, d0 = L[0:2], distance(L[0], L[1])

    for i in range(1,n):
        for j in range(i+1,n):
            d = distance(L[i], L[j])
            if d < d0:
                R0, d0 = [L[i], L[j]], d
    # (n-2) + (n-3) + ... + 1 = (n-2)(n-1)/2 itérations
    return R0,d0
```

Le nombre d'appels à la fonction distance est ici

$$C_n = 1 + \sum_{1 \leq i < j \leq n-1} 1 = 1 + \frac{(n-2)(n-1)}{2} = O_{n \rightarrow +\infty}(n^2)$$

Nous présentons ci-dessous l'algorithme récursif. **Nous ne l'avons pas complètement optimisé**

Il est en fait possible de réduire le nombre des appels à l'algorithme en force brute pour les points qui sont dans la bande $\mu - \delta_0 \leq x \leq \mu + \delta_0$. On peut en effet démontrer que, **dans une tranche de la bande** de cotés δ et 2δ , il y a au plus 8 points **de notre liste**. Cela permet, pour des points qui sont dans cette bande, de restreindre les calculs de distance aux 7 points qui les suivent et assure une complexité linéaire de cette opération finale ($\leq 7n$). La complexité de la fonction elle-même est en $O(n \ln n)$ (en tenant compte du tri initial sur la liste) mais nous ne le démontrons pas ici.

2.

```
def ppp(L):
    ''' L est supposée triée! '''
    assert len(L) >= 2
    n = len(L)
    if n == 2:
        return L, distance(L[0], L[1])
    elif n == 3:
        return pppBrute(L)
```

```

else:
    L1      = L[0 : n//2]
    L2      = L[n//2 :]
    R1, d1  = ppp(L1)
    R2, d2  = ppp(L2)

    if d1 <= d2:
        R0, d0 = R1, d1
    else:
        R0, d0 = R2, d1

    mu = (L1[-1][0] + L2[0][0])/2
    # abscisse séparatrice
    B = [p for p in L if mu-d0 < p[0] and p[0] < mu+d0]
    # points dans une bande
    if len(B) >= 2:
        R, d = pppBrute(B)
        if d < d0:
            return R, d
    return R0, d0

```

Corrigé de l'exercice n° 3.15

1. Dans une suite d'appels récursifs les arguments sont $n_0 > n_1 > n_2 \dots$. Or, une suite d'entiers naturels strictement décroissante est nécessairement finie.

2. -Le résultat est clair si $n = 0$ (retour immédiat avec $1 = a^0$).

- Supposons le résultat vrai pour les entiers de 0 à $n - 1$ avec $n \geq 0$.

Écrivons $n = 2q + r$ (division euclidienne); l'appel $\text{expo}(a, n)$ provoque un appel récursif $\text{expo}(a, q)$. Par hypothèse de récurrence, celui ci renvoie a^q . Selon que $r = 0$ ou 1 la fonction renvoie alors $p * p = a^{2q} = a^n$ si $r = 0$ ou $p * p * a = a^n$ si $r = 1$.

La récurrence est achevée.

3. Complexité

(a) Clairement $A(n) = A(q) + 1$, donc $A(n) \nearrow$.

Par contre $C(7) = 4$ alors que $C(8) = 3$.

(b) $C(n) = C(q) + u_n$ avec $u_n = 2$ si n est impair et $u_n = 1$ si n est pair.

En effet, **le nombre de multiplications est celui qui apparaît dans le corps de la fonction auquel s'ajoute celui de l'appel récursif!**

Nous l'écrivons $C(n) = C(q) + \frac{3 + (-1)^{n+1}}{2} = C(q) + r$

(c) On suppose que $n = 2q + r < n' = 2q' + r'$. Cela entraîne

$$2(q - q') < r - r' \in \{-1, 0, 1\}.$$

$(q - q')$ étant entier, cela impose $q < q'$ ou $q = q'$ et $r = 0 < r' = 1$.

- (d) Un appel $\text{expo}(a, 2^p)$ produit des appels récursifs avec $n = 2^{p-1}, \dots, 2, 2^0, 0$. Pour chaque appel principal ou récursif, sauf avec $n = 0$, il y a 1 multiplication. Donc $A(2^p) = C(2^p) = p + 1$.

Comme $2^{p+1} - 1 = 2(2^p - 1) + 1$, un appel $\text{expo}(a, 2^{p+1} - 1)$ produit des appels récursifs avec $n = 2^{p-1} + 1, \dots, 2^1 + 1, 1, 0$. Pour chaque appel principal ou récursif, sauf avec $n = 0$ il y a 2 multiplications. Donc $C(2^{p+1} - 1) = 2(p + 1) = 2A(2^{p+1} - 1)$.

- (e) On voit que $A(n) = (p + 1)$ et que $(p + 1) \leq C(n) \leq 2(p + 1)$.

Cela fait $A(n) = 1 + \log_2 n$.

Corrigé de l'exercice n° 3.16

L'instruction $\text{expo}(a, q) * \text{expo}(a, q)$ provoquera autant de multiplications que la procédure naïve :

```
p=1
for i in range(0,n):
    p = p*a
```

Par rapport au programme du cours il y a, cette fois, deux appels récursifs au lieu d'un. Plus rien de rapide !

Corrigé de l'exercice n° 3.17

Exponentielle rapide sans variable locale :

```
def  expol(a,n):
    print(n)
    if n==0:
        return 1
    elif divmod(n,2)[1]==1:
        return expo(a, n//2)**2*a
    else:
        return expo(a, n//2)**2
```


Corrigé de l'exercice n° 3.18

Les codes pour les questions 1., 2. et 3. :

```
def recherche_dichotomiqueR(L, x, f=lambda a,b:a<=b):
    '''...'''
    assert est_croissante(L,f)
    n = len(L)
    if n == 0:
        return False, None
    elif n == 1 and L[0] == x:
        return True, 0
    elif n == 1 and L[0] != x:
        return False, None
    else:
        pivot = L[n//2]
        if x == pivot:
            #f ordre total
            return True, n//2
        elif f(x, pivot):
            L1 = L[0:n//2]
            return recherche_dichotomiqueR(L1, x, f)
        else:
            L2 = L[n//2:n]
            r2 = recherche_dichotomiqueR(L2, x, f)
            if r2[0]:
                return True, r2[1] + n//2
            else:
                return False, None

def est_croissante(L, f = lambda a,b : a<= b):
    '''
    L: list;
    f: fonction à valeur booléennes.
    '''
    if len(L)<=1:
        return True
    else:
        for i in range(1, len(L)):
            if not f(L[i-1], L[i]):
                return False
        return True
```

4. Le programme termine

On note $n = \text{len}(L)$. On procède par récurrence sur n .

- Un appel avec $n \leq 1$ conduit directement à une condition d'arrêt.
- Supposons que la fonction termine pour tout appel avec une liste de longueur inférieure ou égale à n . Un appel avec $n + 1$ provoque un appel avec une liste de tailles

$\lfloor \frac{n+1}{2} \rfloor$ ou $\lceil \frac{n+1}{2} \rceil$ inférieures ou égales à n qui termine (attention, c'est parce que $n \geq 2$ que l'on peut affirmer cela). La fonction termine au plus tard une instruction après ce retour.

Correction

Si $\text{len}(L) \leq 1$, le programme renvoie le résultat correct.

Supposons que ce soit le cas pour des listes de longueurs inférieures ou égales à n et considérons un appel avec une liste de longueur $n + 1$.

Si $x = \text{pivot}$, le résultat est immédiat et correct.

Si $x < \text{pivot}$, il ne peut appartenir à L2, et si on le recherche dans L1 qui est de taille $\leq n$, le résultat sera correct par hypothèse de récurrence.

Si $x > \text{pivot}$ le raisonnement est identique.

5. Complexité

Considérons un appel avec n tel que $2^p \leq n < 2^{p+1}$, on aura alors

$$2^{p-1} \leq \lfloor \frac{n}{2} \rfloor \leq \lceil \frac{n}{2} \rceil \leq 2^p.$$

L'appel récursif suivant se fera avec $2^{p-1} \leq n_1 \leq 2^p$. Il y aura donc au plus $p + 1$ appels. Donc $C_n \leq \log_2(n) + 1$.

Les tris

4.1 Introduction

Les méthodes de tris sont évidemment fondamentales. On dispose de nombreux algorithmes qui se distinguent par leurs complexités en temps ou en place mémoire, par le fait qu'ils sont adaptés au traitement de données qui peuvent être chargés en mémoire vive ou, au contraire, trop volumineuses pour le faire (on parle alors de tri externe, ce qui dépasse nos objectifs).

Dans cette brève introduction nous allons illustrer quelques-uns de ces algorithmes avec des listes ou des tableaux d'entiers (ou de flottants).

En pratique on est amené à trier des objets structurés. Il peut d'agir d'enregistrements dans une base de données, de points comme dans le cas des algorithmes géométriques ; on trie donc sur une combinaison de chaînes de caractères, de nombres, de dates, sur des coordonnées... Ainsi, pour trier une liste de candidats à un concours, *l'ordre lexicographique* prendra en considération successivement les noms, les prénoms, les dates de naissance et enfin, de façon impérative, un identifiant unique par individu, ce qui est indispensable en cas de collision des attributs ou champs précédents. Ce sont les opérations de comparaison, coûteuses et présentes dans chaque itération, que nous prendrons en compte pour évaluer la complexité de ces algorithmes. Nous commençons par préciser le vocabulaire (vous reviendrez sur ces définitions quand vous en aurez besoin, on ne vous demande pas de les retenir avant de rencontrer des exemples).

Définition 4.1 *vocabulaire de base*

• **Clé du tri** : *c'est la relation d'ordre total sur un élément de la structure que l'on trie. Attention, si dans une liste d'enregistrements on trie sur nom-prénom, l'ordre lexicographique qui est un ordre total sur les chaînes de caractères n'induit qu'un pré-ordre total sur les enregistrements (car il peut y avoir des homonymes dans la liste et la relation induite n'est pas anti-symétrique).*

- **Tri en place** : c'est un tri qui lit et écrit directement dans la structure (liste, tableau,...) et qui utilise peu de mémoire supplémentaire.
- **Tri stable** : c'est un tri qui, lorsqu'il rencontre des éléments ayant la même clé dans la structure initiale, les restitue dans le même ordre dans la structure triée (un contre-exemple : le tri par comptage vu en 2.3.4, page 113).
- **Tri interne ou externe** : un tri interne est un algorithme de tri qui traite des structures chargées en **mémoire vive**. Par opposition, un **tri externe** lit, écrit (et déplace) les objets sur un support externe (mémoire de masse).

Nous avons donné deux exemples de tri dans le chapitre 2 : le tri par sélection qui est un tri naïf et nécessite de l'ordre de $\frac{n^2}{2}$ comparaison, le tri par comptage.

4.2 Tri par insertion

On comprend facilement l'idée de ce premier tri en considérant une liste ou un tableau dont les premiers éléments d'indices 0,1, ..., $i - 1$ sont déjà triés. Si on souhaite déplacer l'élément i pour réordonner les éléments d'indices 0 à i , on procède en le comparant aux précédents que l'on fait « remonter » (c'est l'instruction $T[p] = T[p-1]$) dans le programme ci-dessous) jusqu'à ce que la place de $T[i]$ soit trouvée...

Tri par insertion

```
def tri_insertion(T):
    n = len(T)
    for i in range(1, n):
        x = T[i]
        p = i
        while p > 0 and T[p-1] > x:
            T[p] = T[p-1]
            p = p-1
        T[p] = x
    return T
```

Correction

Chaque boucle while termine puisque p y est décrémenté à chaque itération ; comme leur nombre est fini (déterminé par les itérations de la boucle for), le programme termine.

Montrons qu'un invariant de la boucle for est

$$\mathcal{P}(i) = \{[T[0], \dots, T[i-1]] \text{ est triée}\}.$$

- C'est clairement vrai lorsque $i = 1$ (une liste à un élément est triée).
- Supposons qu'avant l'itération n° i , le début de liste soit trié. Observons que la liste n'est modifiée que dans la boucle while. On arrive donc à l'évaluation de la condition avec $[T[0] \leq \dots \leq T[i-1]]$, $x = T[i]$ et $p = i$.
- Si $x \leq T[p-1]$, on sort de la boucle conditionnelle (while) sans itération $[T[0] \leq \dots \leq T[i-1] \leq T[i] = x]$.
- Si $x > T[p-1]$, supposons qu'il y ait j itérations. Les instructions remplacent les termes $T[i], T[i-1], \dots, T[i-j+1]$ par leurs prédécesseurs. Le segment $T[i-j+1], \dots, T[i]$ reste donc trié et après la boucle dont on sort avec $p = i-j$, comme $x = T[i-j] \leq T[i-j+1] \leq \dots \leq T[i]$, et comme le début de la liste n'a pas été modifié, on obtient $[T[0] \leq \dots \leq T[i-1]]$, $x = T[i]$. CQFD.

Complexité dans le pire des cas

Dénombrons les comparaisons *entre éléments du tableau*. Il y a dans tous les cas $n-1$ passages dans la boucle for. Dans la boucle while, *le nombre d'itérations est maximal lorsque T est trié dans l'ordre décroissant*, ce qui conduit à $i-1$ comparaisons. Soit en tout

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} \sim \frac{n^2}{2}.$$

Observons par ailleurs que la fonction travaille directement sur la liste ou le tableau T , ce qui ne demande **aucune occupation supplémentaire de mémoire**.

Exercice 4.1 mesure de la complexité en pratique

On vous propose d'estimer expérimentalement la complexité en moyenne du tri par insertion.

1. Modifier la fonction Python `tri_insertion` en la transformant en une fonction `tri_insertion1(T, cp)` dans laquelle `cp` réalise les comparaisons (`cp(x,y)` renvoie un booléen et joue le rôle de l'option `key` dans la méthode `sort` des liste implémentées dans Python).
2. Placer un compteur (variable globale) dans la fonction `cp` (qui sera incrémenté à chaque appel). Modifier `tri_insertion1` pour que ce compteur soit pris en compte.
3. Définir une fonction construisant des tableaux ou des listes aléatoires de taille donnée `liste_aleat(N)`, et définir un protocole permettant de valider l'hypothèse $\text{cout}_{\text{moyen}}(\text{tri_insertion}(T, cp)) \sim \frac{N^2}{4}$.

Corrigé 4.1 page 202.

4.3 Tri rapide : diviser pour régner

Nous avons page 166 donné les grandes lignes de ce tri. Faisons en ici l'analyse.

Tri rapide (quicksort) sous Python

```
def cp(x,y):
    return x<=y

def quicksort(L, cp):
    if len(L)<=1:
        return L
    else:
        Li , Ls = list([]), list([])
        x = L[len(L)//2]
        L.remove(x)
        for e in L:
            if cp(e,x):
                Li.extend([e])
            else:
                Ls.extend([e])
        return quicksort(Li,cp)+[x]+ quicksort(Ls,cp)
```

Correction

L'algorithme termine puisque les appels récursifs ont pour arguments des listes de longueurs strictement plus petites que $\text{len}(L)$. La condition d'arrêt sera remplie dans tous les cas en un temps fini (l'arbre des appels est au plus de hauteur $\text{len}(L)$).

On montre facilement par récurrence sur la longueur de la liste, que cette fonction retourne une copie triée de L . En effet,

- si $\text{len}(L) = 0, 1$ le résultat est clair ;
- supposons que la propriété soit vraie pour les listes de longueurs $0, 1, \dots, n$. les deux appels récursifs ont comme paramètres des sous-listes de $L \setminus \{e\}$. Elles retournent donc par hypothèse de récurrence deux copies triées de Li et Ls . La concaténation des trois listes est donc triée (e est plus grand que les éléments de Li , plus petit que les éléments de Ls).

Complexité

Nous chercherons naturellement à évaluer le nombre de comparaisons *entre les éléments de la liste* (c'est-à-dire des appels à la fonction cp). Nous ne nous préoccupons pas de la comparaison qui figure dans l'instruction conditionnelle : cela ne change pas l'ordre de grandeur et, selon ce que l'on trie, le coût sera bien moins important

que celui d'une comparaison d'éléments d'une base de données par exemple.

Considérons un appel principal avec une liste de longueur n . Une fois l'élément e (le pivot) choisi, il y a $n - 1$ comparaisons (ou appels à cp). Regardons les deux appels récursifs : si les deux sous-listes ont au moins 2 éléments chacune, il y aura en tout $n - 3$ comparaisons ; si une des deux listes est vide il y aura $n - 2$ comparaisons.

Nous sommes alors en mesure **d'identifier le pire des cas**. En effet, supposons que l'on choisisse systématiquement le plus grand élément de la liste. Les appels récursifs iront deux par deux avec une liste privée d'un élément par rapport à l'argument d'appel et une liste vide. Non seulement la hauteur de l'arbre des appels sera la plus haute possible (égale à n), mais les appels seront les plus coûteux en nombre de comparaisons :

$$(n - 1) + (n - 2) + \dots + 1 = \frac{n(n - 1)}{2} = O(n^2).$$

Cet algorithme, qui date de 1961, est considéré comme un des plus utilisés, des plus implémentés (c'est le tri par défaut de Python et bien d'autres langages ou logiciels). Mais ce n'est sûrement pas à cause de ce piètre résultat. En réalité son intérêt réside (outre sa simplicité) dans sa *complexité en moyenne qui est en $O(n \ln n)$ sur des listes dont les termes sont distincts*. Nous en proposons une étude expérimentale dans l'exercice (4.3).

Exercice 4.2 *tri rapide, variante*

Réécrire le « quicksort » sous Python, en définissant les sous-listes Li et Ls en compréhension.

Corrigé 4.2 page 203.

Exercice 4.3 *vérification expérimentale de la complexité*

On souhaite vérifier expérimentalement la complexité de l'algorithme.

1. Placer un compteur dans la fonction cp (variable globale).
2. Définir un protocole pour évaluer le nombre de comparaisons *en moyenne* à l'aide de ce compteur. Écrire un script le mettant en œuvre.

Indication : construire des listes aléatoires avec la fonction randint du module random pour calculer des complexités en moyenne.

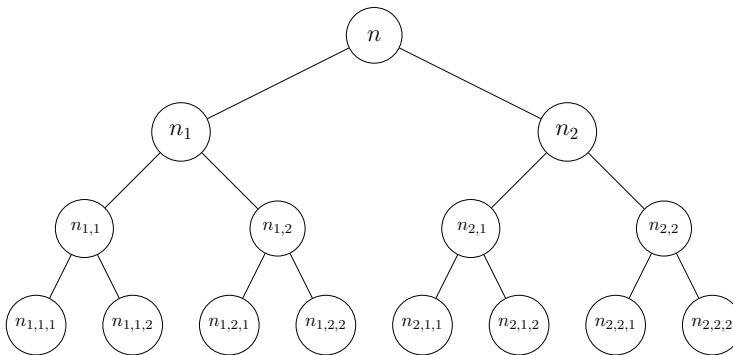
3. On souhaite vérifier le comportement dans le pire des cas. En supposant que le pivot soit, comme dans la fonction proposée, $e = L(\text{len}(L)//2)$, écrire une fonction `la_pire(n)` qui renvoie une liste dont le tri sera en $O(n^2)$.

Indication : observer ce que donne la liste `|1|3|1|4|1|3|1|5|1|3|1|4|1|3|1|` et pour construire une liste analogue, définir une fonction récursive auxiliaire `placer_centre(L, x, d, f)` qui modifie le terme d'indice $\lfloor (d+f)//2 \rfloor$ de L et fait deux appels récursifs `placer_centre(L, x - 1, d, m - 1)` et `placer_centre(L, x + 1, m + 1, f)`...

Corrigé en 4.3 page 204.

4.4 Tri fusion

Comme le tri rapide, c'est un tri qui repose sur une stratégie « diviser pour régner ». Le principe général est donné page 166. Il est conseillé de commencer par le relire. Nous verrons qu'il a une complexité dans tous les cas de $O(n \ln n)$; par contre **ce n'est pas un tri en place** car il réserve un nouveau tableau de même taille que le tableau à trier. Son principe de divisions des données est bien adapté au traitement de celles qui ne tiennent pas en mémoire centrale (il faut alors adapter l'algorithme au type de support).



Un arbre (incomplet) des appels récursifs.

Tri fusion sous Python

```

def tri_fusion(T, cp):
    print('appel avec T = ', T)
    if len(T) <= 1:
        return T
    else:
        n = len(T)
        m = n // 2
        T1 = tri_fusion(T[0:m], cp) #pac
        T2 = tri_fusion(T[m:n], cp)
        return interclassement(T1, T2, cp)
  
```

```

def interclassement(T1, T2, cp):
    """ on suppose que T1,T2 sont des tableaux"""
    i = j = k = 0
    n1, n2, n = len(T1), len(T2), len(T1)+len(T2)
    T = np.array(np.ones(n) , dtype = type(T1[0]) )
    while i < n1 and j < n2:
        if cp(T1[i], T2[j]):
            T[k] = T1[i]
            i, k = i+1, k+1
        else:
            T[k] = T2[j]
            j, k = j+1, k+1

    if i == n1:
        while j < n2:
            T[k] = T2[j]
            j, k = j+1, k+1
    else:
        while i < n1:
            T[k] = T1[i]
            i, k = i+1, k+1
    return T

```

Exercice : Ne peut on faire plus court *dans la syntaxe* de la fonction interclassement et remplacer chacune des deux dernières boucles while par une seule instruction ?

Complexité

Regardons encore le nombre de comparaisons *entre éléments de T*. Ce nombre est compris entre $\min(n_1, n_2)$ et $n_1 + n_2$ lors d'un appel $\text{interclassement}(T_1, T_2)$ ($n_i = \text{len}(T_i)$).

La manière la plus intuitive, nous semble-t-il, pour estimer le nombre de comparaisons dans le pire des cas est de raisonner sur l'arbre des appels récursifs. Nous dirons que l'appel principal est de profondeur 0 et qu'un appel est de profondeur $n+1$ si c'est un appel récursif dans une procédure de profondeur n . Dans la figure qui suit, nous avons $n_1 = \lfloor n/2 \rfloor$, $n_1 + n_2 = n$ et à chaque niveau de l'arbre $\sum \dots \sum n_{i,\dots,k} = n$. Ainsi, le nombre total de comparaisons dans les appels à interclassement est au plus $n \times h$ où h est la profondeur de l'arbre. Il ne reste plus qu'à majorer cette profondeur. Pour cela, encadrons n par deux puissances de 2 : $2^p \leq n < 2^{p+1}$. Il vient alors

$$2^{p-1} \leq n_1 \leq n_2 \leq n_1 + 1 \leq 2^p.$$

Ces divisions s'arrêtent dès que $n_{2,\dots,2} \leq 1 = 2^0$ ce qui fait que la profondeur de l'arbre est majorée par $p + 1 = 1 + \log_2 n$.

Bilan

La complexité en nombre de comparaisons du tri fusion est au plus $(1 + \log_2 n) n = O(n \ln n)$.

Correction

Le programme termine puisque la taille des tableaux passés en argument décroît strictement avec la hauteur des appels.

Pour la correction, raisonnons en remontant dans l'arbre. Démontrons par récurrence sur p la propriété : « *La fonction retourne une copie triée de T si les appels récursifs sont tous de profondeur inférieure ou égale à p .* »

- C'est clair si la profondeur est 0 (tableau ou liste d'au plus un élément).
- Si cela est vrai pour un certain p lorsque la fonction donne naissance à des appels récursifs de profondeur au plus $p + 1$ les deux appels immédiats dans la procédure retournent deux listes triées. Tout repose donc sur la correction de la fonction interclassement, que nous laissons au lecteur !

4.5 Tri par insertion dichotomique

Le principe du tri par insertion dichotomique est le suivant : on part d'une liste L , on construit une **nouvelle liste** triée des éléments de L , Lt en :

- initialisant $Lt = [L[0]]$;
- parcourant la suite des éléments de L , insérant chaque élément de L à sa position dans la liste Lt triée, en construction. Comme la liste en construction est triée, on recherche la place d'un nouvel élément par dichotomie.

On adapte donc l'algorithme vu page 117 pour qu'il retourne la **place** qui doit être celle du nouvel élément dans la liste, ce qui est présenté dans la colonne de gauche du tableau qui suit. La fonction $\text{comp}(x, y)$ définit l'ordre entre deux éléments (elle retourne -1, 0 ou +1 selon que $x < y$, $x = y$, ou $x > y$). Elle est destinée à partir comme argument pour les fonctions de recherche et de tri.

Spécification

La fonction $\text{triDicho}(L, \text{ordre})$ prend en arguments une liste et une fonction associée à un ordre total (telle que $\text{ordre}(x, y) = -1, 0, 1$ selon $x < y$, $x = y$, $x > y$). Elle retourne une liste triée des éléments de L qui reste inchangée.

Recherche dichotomique	Le tri par insertion
<pre>def pos(x,y): return (x < y)-(y < x) def rechDic(L,e, ordre): a, b = 0, len(L)-1 while a <= b: m = (a+b)//2 r = ordre(L[m], e) if r==0: return m elif r == 1: a = m+1 else: b = m -1 return a</pre>	<pre>def triDicho(L, ordre): Lt = [] for x in L: if len(Lt)==0: Lt.append(L[0]) else: p = rechDic(Lt,x,ordre) Lt.insert(p, x) return Lt</pre>

Correction

Tout repose sur la correction de la fonction `rechDic()` qui est une adaptation immédiate de l’algorithme de recherche dichotomique étudié en (2.5.1) et dont la preuve est traitée en exercice page 118. Nous n’y revenons pas.

Complexité

La fonction `triDicho(L,ordre)` réserve une place mémoire égale à celle de L . Sa complexité mémoire est donc $\Theta(N) = \Theta(\text{len}(L))$.

Elle provoque N appels à la fonction `rechDic(Lt,ordre)`. On a vu qu’un tel appel coûte au plus $K \ln(n)$ comparaisons pour une liste de taille n . Ici, au cours des appels successifs, $\text{len}(Lt) = 1, 2, \dots, N - 1$. Cela donne une complexité dans le pire des cas vérifiant

$$\text{coût}_{\text{triDicho}}(L, \text{ordre}) \sim K \sum_{k=2}^{N-1} \ln(k) = O(N \ln N). \tag{4.5.1}$$

En effet, le cours de maths nous apprend que :

$$\sum_{k=1}^N \ln k \underset{n \rightarrow +\infty}{\sim} \int_2^N \ln t \, dt \underset{n \rightarrow +\infty}{\sim} N \ln N.$$

Mise en œuvre

```
L = listeAleat(10,67)
print(' Liste exemple: ', L)
Liste exemple:  [11,26,54,16,10,18,42,7,60,58]

M1 = triDicho(L, pos)
print(' Liste triée: ', M1)
Liste triée:  [7,10,11,16,18,26,42,54,58,60]

M2 = triDicho(L, lambda x,y : pos(y,x))
print(' Liste triée: ', M2)

Liste triée:  [60,58,54,42,26,18,16,11,10,7]
```

4.6 Complexité des tris

Le tri	pire des cas	moyenne	mémoire supplémentaire
tri par sélection	$\sim \frac{n^2}{2}$	$\sim \frac{n^2}{2}$	-
tri par comptage	$O(n)$	$O(n)$	n
tri par insertion	$\sim \frac{n^2}{2}$	$\sim \frac{n^2}{4}$	-
tri rapide	$\sim \frac{n^2}{2}$	$O(n \ln n)$	-
tri fusion	$O(n \ln n)$	$O(n \ln n)$	n
dichotomique	$O(n \ln n)$	$O(n \ln n)$	n

Les trois tris qui n'utilisent pas de mémoire supplémentaire sont les tris *en place* (les éléments sont échangés dans le tableau ou la liste, sur place).

4.7 Sort dans Python

sort, sorted

Les tris sont bien sûr implémentés dans Python et dans tous les langages de programmation, systèmes de calculs et autres tableurs. Les fonctions ou méthodes de tris prennent en arguments la liste ou le tableau à trier et une clé ou une fonction définissant une relation d'*ordre total* ou de **pré-ordre total** sur le type d'éléments contenus dans la liste.

• La classe list et la méthode sort dans Python

```
>>> L=[123,34,-7,2]
>>> L.sort()
>>> L
[-7, 2, 34, 123]

>>> L = [123,34,-7,2]
>>> f = lambda x: -x
>>> L.sort(key =f)
>>> L
[123, 34, 2, -7]
```

• La fonction sorted() dans Python

La *méthode* qui précède réarrange la liste en place alors que sorted est une *fonction* qui retourne un *autre* objet itérable : l'appel

```
sorted(iterable[, key][, reverse])
```

ne modifie pas l'argument principal.

4.8 Recherche de la médiane en temps linéaire

Médiane

Soit $L = [x_0, \dots, x_{n-1}]$ une liste de n valeurs distinctes appartenant à un ensemble totalement ordonné (nombres, chaînes de caractères, enregistrements que l'on peut trier sur une clé etc.). On lui associe la liste triée $\hat{L} = [\hat{x}_0, \dots, \hat{x}_{n-1}]$ formée des mêmes éléments écrits dans l'ordre croissant.

Lorsque $n = 2p+1$ est impair, la *valeur médiane* est la valeur m de la liste pour laquelle il existe p éléments strictement plus petits et p éléments strictement plus grands ($m = x_p$).

Lorsque $n = 2p$ est pair les *valeurs médianes* sont les valeurs $\hat{x}_{p-1}$ et $\hat{x}_p$ (la $p^{\text{ième}}$ et la $(p+1)^{\text{ième}}$ valeurs de la liste ordonnée...

Cette *caractéristique de dispersion* est essentielle en statistique descriptive, mais un calcul rapide de la médiane nous intéresse aussi parce qu'il intervient dans des problèmes de géométrie algorithmique.

Une première idée pour calculer une médiane consiste à trier la liste pour récupérer le ou les éléments médian(s) ce qui a un coût en $O(n \ln n)$. C'est dans les années 70 que les premiers algorithmes linéaires (en $O(n)$) ont été publiés.

L'idée est la suivante : plutôt que calculer exclusivement la médiane, on écrit une fonction récursive $\text{partile}(L, \text{index})$ qui retourne l'élément de $\hat{L}$ d'indice index dans la liste triée. Sous Python c'est l'élément de position $\text{index} + 1$.

Cette fonction :

- retourne la valeur voulue *après avoir trié la liste* si celle-ci comprend de 1 à 5 éléments, ce qui fournit une condition d'arrêt ;
- partitionne la liste L en groupes de 5 éléments dont les médianes sont calculées et stockées dans une liste des médianes, Lm ;
- calcule par un appel récursif $\text{partile}(Lm, \dots)$ la valeur m de la *médiane des médianes* et partitionne la liste L autour de cette médiane en inf , $[m]$, sup (les éléments de inf étant strictement plus petits que m , ceux de sup , strictement plus grands ;
- retourne m si inf contient les éléments indexés jusqu'à $\text{index} - 1$, recherche l'élément d'indice voulu dans inf ou dans sup sinon ;

Exercice 4.4 *un programme qui calcule incidemment la médiane*

On ne confondra pas la position d'un élément dans une liste avec son indice (sous Python $L[1]$ est le deuxième terme d'une liste).

1. Lorsqu'une liste triée est indexée de 0 à $\text{len}(L) - 1$ quels sont selon que $\text{len}(L)$ est pair ou impair les *indices* des valeurs médianes ?
2. Écrire $\text{partile5}(L, \text{index})$ qui retourne l'élément d'indice index de $\hat{L}$ lorsque L une liste non vide d'au plus 5 éléments. Majorer le nombre de comparaisons.
3. Écrire $\text{partager}(L, \text{val})$ qui retourne un triplet $(\ell, \text{inf}, \text{sup})$ dans lequel inf est la sous-liste des éléments strictement plus petits que val , sup celle des éléments strictement plus grands, ℓ la longueur de inf (c'est-à-dire le nombre des éléments strictement plus petits que val dans la liste).

Complexité en nombre de comparaisons ?

4. On entreprend ici une première ébauche de $\text{partile}(L, \text{index})$: si L est de taille ≤ 5 , elle appelle $\text{partile5}(L, \text{index})$; sinon elle partitionne L en paquets de 5 (avec éventuellement un paquet de taille comprise entre 1 et 4) et retourne la liste des *médianes* de ces paquets.
5. Acheter la construction de $\text{partile}(L, \text{index})$ en suivant la description de l'encart.

Cette fonction retourne la valeur médiane unique si $\text{len}(L)$ est impaire, la valeur médiane de droite sinon, lorsqu'on l'appelle avec $\text{partage}(L, \text{len}(L)/2)$.

Attention : la complexité de cette fonction n'est pas linéaire tant que les « paquets » ne sont pas triés en place, ce que nous n'avons pas fait dans le corrigé (peut-être vous ?).

Remarque : ce programme est parfois (souvent) présenté comme un calcul de médiane ; voyez le comme un calcul du $k^{\text{ième}}$ élément de la liste ($k = \text{index} + 1$), ce sera plus clair.

Corrigé en 4.4 page 205.

4.9 Corrigés des exercices

Corrigé de l'exercice n° 4.1

1. On vous propose d'ajouter une fonction de comparaison au tri comparer indifféremment des nombres, de chaînes... dans un ordre ou dans l'ordre inverse. `cp` joue ici le rôle du paramètre `key` de la méthode `sort` de python.

```
def cp(x,y):
    return x<=y

def tri_insertion1(T, cp):
    n =len(T)
    for i in range(1,n):
        x = T[i]
        p = i
        while p > 0 and not(cp(x, T[p-1])):
            T[p] = T[p-1]
            p = p-1
        T[p]=x
    return T
```

2. On ajoute un compteur dans la fonction de comparaison (variable **globale** `c`). On initialise ce compteur à 0 au début de `tri_insertion1`, ce qui permettra de dénombrer les comparaisons.

```
def cp(x,y):
    global c
    c=c+1
    return x<=y

def tri_insertion1(T, cp):
    global c
    c=0

    n =len(T)
    for i in range(1,n):
        x = T[i]
        p = i
        while p > 0 and not(cp(x, T[p-1])):
            T[p] = T[p-1]
            p = p-1
        T[p]=x
    return T
```

3. Les calculs de moyennes (et du temps)

```

liste_aleat =
    lambda n: [random.randint(0,1000) for k in range(0, n)]

def cout_moyen(n, m):
    s = 0
    for i in range(0, m):
        L = liste_aleat(n)
        tri_insertion1(L, cp)
        s = s + c
    return s/m

p = 50
N = 1000
for n in range(1, N, 50):
    t0 = ti.time()
    m = cout_moyen(n,p)
    print('Longueur de liste n = ' + str(n)
        + '\t Nombre moyen d\'appels à cp m = ' + str(m)
        + '\t Rapport m/(n**2) : ' + str(m/n**2)
        + '\tDurée des '+str(p)+' tris: ' + str(time.time()-t0))

```

Corrigé de l'exercice n° 4.2

```

def quicksort2(L, cp):
    if len(L) <= 1:
        return L
    else:
        x = L[len(L)//2]
        L.remove(x)
        Li = [e for e in L if cp(e, x)]
        Ls = [e for e in L if cp(x, e)]
        return quicksort2(Li, cp) + [x] + quicksort2(Ls, cp)

```

Corrigé de l'exercice n° 4.3

1. On insère une ligne standard :

```
def cpc(x,y):
    """avec le compteur"""
    global c
    c =c+1
    return x<=y
```

2. Il y a deux choses à gérer.

Pour approcher le nombre moyen d'appels à cp pour des listes de taille n , il nous faudra faire plusieurs tirages ($p = ?$) de listes aléatoires et calculer une valeur moyenne $X_{n,p}$.

En faisant varier n on pourra ensuite estimer le comportement asymptotique de ce nombre moyen d'appels à cp en fonction de n , par exemple en calculant les quotients $\frac{X_{n,p}}{n \ln n}$ qui devraient rester bornés.

Voilà donc ce que l'on peut faire : `test_moyenne(n, p)` exécute p fois le tri sur des listes aléatoires de taille n et renvoie le nombre moyen d'appel à `cpc` (dans laquelle on a placé un compteur).

On calcule ensuite ces valeurs moyennes et on les compare à $n \ln n \dots$ Les résultats sont dans le tableau.

```
def test_moyenne(n,p):
    global c
    s = 0
    for i in range(0,p):
        L = [rd.randint(0,200) for k in range(0,n)]
        c = 0
        quicksort(L,cpc)
        s = s+c
    return s/p

Nmax = 10
T = np.matrix( np.zeros((3,Nmax), dtype=float))

for k in range(1, Nmax):
    n          = 100*k
    X          = test_moyenne(n,20)
    T[0,k]     = n
    T[1,k]     = X
    T[2,k]     = X/(np.log(n)*n)
```

100.0	200.0	300.0	400.0	500.0	600.0	700.0	800.0
696.9	1615.6	2761.4	4018.0	5281.6	6768.0	8273.55	9736.05
1.513	1.524	1.613	1.676	1.699	1.763	1.804	1.820

L'hypothèse n'est pas invraisemblable. Mais restons prudents, nos listes n'ont pas des éléments distincts...

3. Encore un diviser pour régner :

```
def placer_centre(L, x, d, f):
    """place n entre d et f dans L"""
    if d==f:
        L[d] = x
    elif d<f:
        m = (d+f)//2
        L[m] = x
        placer_centre(L,x-1,d, m-1)
        placer_centre(L,x-1, m+1,f)

def la_pire(n):
    L=[0 for x in range(0,n)]
    placer_centre(L,n,0,n-1)
    return L
```

Corrigé de l'exercice n° 4.4

```
def partile5(L, index):
    L.sort()
    return L[index]

def partager(L, val):
    inf, sup = list([]), list([])
    for e in L:
        if e < val:
            inf.append(e)
        elif e > val:
            sup.append(e)
    return len(inf), inf, sup
```

```

def partile(L, index):
    """index est la position, pas l'indice"""
    if index >= len(L):
        raise ValueError('position hors liste')
    if len(L) <= 5:
        return partile5(L, index)
    else:
        Lm = list([])
        q,r = divmod(len(L),5)
        for k in range(0,q):
            Lm.append(partile5( L[5*k : 5*k+5], 2 ))
        if r>0:
            R = L[5*q: 5*q+r]
            Lm.append(partile5( L[5*q: 5*q+r], r//2 ))

        m5 = partile(Lm, len(Lm)//2) # médiane de Lm
        ell, inf, sup = partager(L, m5)

        if ell == index:
            return m5
        elif ell > index:
            return partile(inf, index)
        else:
            return partile(sup, index-ell-1)

```

Algorithmes gloutons

5.1 Introduction

Nous présentons dans ce chapitre des méthodes de résolution « approchée » de problèmes d'**optimisation combinatoire**, c'est à dire de problèmes de recherche de minimum avec contraintes pour une fonction définie sur un **ensemble fini ou discret** (partie de $\mathbb{N}^n$ ou de $\mathbb{Z}^n$, en général)¹.

Nous illustrons notre propos avec quelques exemples simples et classiques qui nous permettront de préciser la problématique et le formalisme de l'optimisation combinatoire puis de présenter des algorithmes gloutons pour les aborder. Le chapitre 13 (troisième semestre) nous permettra de revenir sur ces questions avec les méthodes exactes/optimales (dans un sens que nous préciserons plus bas) de la programmation dynamique.

5.1.1 Optimisation combinatoire

Les quelques problèmes évoqués ici, vont nous permettre de cerner la notion d'optimisation combinatoire, ce sont aussi ceux avec lesquels nous présenterons les méthodes gloutonnes.

• Le problème du rendu de monnaie

Formulation : Étant donné un système de monnaie (pièces et/ou billets) de valeurs $v_0 < v_1 < \dots < v_{n-1}$, et une somme s , réaliser cette somme avec le plus petit nombre de pièces ou de billets possibles.

Ainsi, avec le système des pièces de la zone euro, soit en centimes d'euros :

$(v_i)_{0 \leq i \leq 7} = (1, 2, 5, 10, 20, 50, 100, 200)$, la somme $s = 731$ est obtenue/réalisée

1. Les méthodes de l'optimisation combinatoire sont fondamentalement différentes des méthodes d'optimisation numérique adaptées aux problèmes dans lesquels on cherche à minimiser une fonction définie sur une partie de $\mathbb{R}^N$ et pour lesquels les outils sont ceux de l'algèbre linéaire, de l'analyse mathématique et de l'analyse numérique.

avec la décomposition

$$731 = 3 \times 200 + 1 \times 20 + 1 \times 10 + 1 \times 1 = 3v_7 + v_3 + v_2 + v_0$$

qui nécessite 6 pièces de monnaie.

Formalisation : Une suite d'entiers strictement croissante $(v_i)_{0 \leq i \leq 7}$ avec $v_0 = 1$ et un entier $s \in \mathbb{N}^*$ étant donnés, déterminer des entiers positifs $(x_i)_{0 \leq i \leq n-1}$ tels que

$$\begin{cases} \sum_{i=0}^{n-1} x_i v_i = s \\ \sum_{i=0}^{n-1} x_i = \text{Min}(\{\sum u_i / \sum u_i v_i = s\}) \end{cases} \quad (5.1.1)$$

• Allocations de salles

On considère une version simplifiée du problème d'allocation de salles pour des cours, réunions ou conférences.

Formulation : Les données : d'une part, des salles, d'autre part, les cours et les conférences dont les horaires sont déterminés et auxquels il faut attribuer des salles. On souhaite utiliser le moins de salles possible.

Formalisation : Pour modéliser le problème on représente chaque cours par un triplet $[c, d, f]$ (c : int, identifiant ou numéro de salle, d : int, heure de début du cours ; f : int, heure de fin du cours avec $8 \leq d < f \leq 19$ par exemple).

Une bonne façon de poursuivre la modélisation est de concevoir les jeux d'allocations possibles sous la forme d'**agendas** qui sont des listes de quadruplets (ou **événements**) $[t, d|f, c, s]$ dont les termes représentent respectivement, l'heure, le statut (début ou fin, 0 ou 1), le n° du cours, le n° de la salle. L'ensemble des agendas valides est clairement une partie de $\mathcal{A} \subset H \times \{0, 1\} \times \llbracket 0, n_S - 1 \rrbracket \times \llbracket 0, n_C - 1 \rrbracket \subset \mathbb{N}^4$ (avec $H = \{8, 9, 10, \dots, 16, 17, 18, 19\}$ ensemble des heures pour un événement, n_S et n_C les nombres de salles et de cours quotidiens).

On cherche donc A (une liste de quadruplets) telle que

$$\begin{cases} A \in \mathcal{A} \\ f(A) = \text{Min}(\{f(X) / X \in \mathcal{A}\}) \end{cases} \quad (5.1.2)$$

La fonction f étant définie comme le nombre de salles distinctes qui apparaissent dans l'agenda A :

$$f(A) = |\{s / \exists e \in A, e[1] = 0 \text{ (début) et } e[3] = s\}|. \quad (5.1.3)$$

Remarques

Ce problème se pose concrètement dans un lycée par exemple : on prépare un emploi du temps, il faut ensuite attribuer des salles aux différents cours. Si l'algorithme ne cherche pas à optimiser le nombre des salles occupées, il ne produira pas de solution réalisable par manque de salles et cela imposera de déplacer les cours ou de pousser les murs.

La situation dans un établissement scolaire est en fait (en apparence) plus complexe :

- les salles ne sont pas échangeables : salles de TP pour la chimie, la physique, les SI, l'informatique, salles multimédia etc.
- on pourra également souhaiter que des cours successifs et qui concernent un même groupe d'élèves (classe) se voient attribuer des salles proches pour minimiser les déplacements.

Formulation générale d'un problème d'optimisation combinatoire

Définition 5.1 problème d'optimisation combinatoire

Un problème d'optimisation combinatoire est défini à partir

- d'un ensemble fini S , (ensemble des solutions réalisables),
- d'une application $f : S \rightarrow \mathbb{R}$,

et consiste à déterminer les éléments $s^* \in S$ vérifiant $f(s^*) = \text{Min}(\{f(s)/s \in S\})$.

Dans les deux exemples que nous avons donnés, les problèmes s'écrivent sans peine sous cette forme. En effet,

- le système (5.1.1) (rendu de monnaie) est de cette forme, avec

$$S = \left\{ (x_i)_i \in \mathbb{N}^n / \sum_{i=0}^{n-1} x_i v_i = s \right\} \text{ et } f(x) = \sum_{i=0}^{n-1} x_i;$$

- le système (5.1.2) (allocation de salle de cours) est lui aussi de cette forme, avec un ensemble de solutions réalisables qui est l'ensemble $S = \mathcal{A}$ des agendas valides (ce sera caractérisé dans l'exercice (5.8) et $f(A)$ est définie par la formule (5.1.3) ;
- le problème du partage d'une ressource unique pour une famille maximale d'activités, défini page 213, se formule lui-aussi de cette façon. On cherche à y **maximiser** le nombre d'activités qui pourront se partager la ressource.

Observons au passage qu'un problème $f(s^*) = \text{Max}(\{f(s)/s \in S\})$ est équivalent au problème $g(s^*) = \text{Min}(\{g(s)/s \in S\})$ avec $g = -f$.

L'ensemble S est défini en compréhension, par une équation ou un système d'équations qu'il nous faudra considérer à défaut de la/le résoudre exhaustivement (d'où le nom d'ensemble des **solutions réalisables**, par opposition aux solutions de notre problème qui sont les **solutions optimales**).

Bien que S soit un ensemble fini, résoudre le problème en force brute, c'est à dire en calculant toutes les valeurs $f(s)$ pour $s \in S$, avant d'en chercher le minimum, est

la plupart du temps rapidement hors de portée. Nous verrons, par exemple, qu'il y a 22 414 façons de rendre la monnaie sur 151 centimes avec le jeu de pièces de la zone euro (soit autant de solutions réalisables). Et dans un lycée de 50 classes avec 75 salles, l'ensemble $H \times \{0, 1\} \times \llbracket 0, n_S - 1 \rrbracket \times \llbracket 0, n_C - 1 \rrbracket$ contient certainement plus de 100 000 éléments (dédoubléments et options obligent) dont, certes, une partie seulement constituera les agendas valides, c'est à dire l'ensemble des solutions réalisables.

5.1.2 Le principe des algorithmes gloutons

Dans les deux problèmes d'optimisation combinatoire que nous avons évoqués il est possible de faire apparaître des étapes dans un processus de résolution :

- Pour le problème de rendu de monnaie on peut décider de **choisir la pièce ou le billet de plus grande valeur** qui permet d'approcher au mieux la somme (par valeur inférieure) avec ces pièces ou billets. On passe ensuite aux billets de valeur inférieure. Avec $V = [1, 2, 5, 10, 20, 50, 100, 200]$ et $s = 731$, on choisira $v = 200$ qui avec $3 \times 200 \leq 731$ donne une approximation par valeurs inférieures de 731. L'étape suivante consiste à approcher 131 avec la plus grande pièce possible. On prend donc $v = 100$, et il reste à réaliser 31... Le nombre d'étapes est ici majoré par le nombre de pièces dans le système.
- Pour le problème d'allocation de salles, on pourra construire un agenda valide, et on l'espère optimal, en commençant par trier la liste des cours en suivant un **ordre croissant sur les heures de fin de cours**. On parcourt ensuite cette liste, et pour chaque cours, on cherche **la première salle libre pendant la durée du cours** (déjà attribuée sur un créneau précédent ou pas). Si cette salle est trouvée, on la retient pour ce cours et on ajoute les deux événements (début et fin du cours) à l'agenda. Sinon, la stratégie aura conduit à un échec. En cas de succès, le nombre d'étapes est clairement égal au nombre de cours. On observera toutefois que les étapes suivent l'ordre de fin des cours.

Définition 5.2

Une **stratégie gloutonne** pour un problème d'optimisation combinatoire consiste à définir une succession d'étapes au cours desquelles on construit progressivement des **solutions réalisables** du problème en faisant des choix :

- qui réduisent au maximum la taille du problème dans l'état où il se trouve (on parle d'*optimum local*) ;
- qui sont *définitifs* (on ne reviendra pas en arrière en fonction de ce que l'on pourrait trouver à l'étape suivante comme on le ferait dans un parcours d'arbre ou de graphe par exemple).

Exercice 5.1 *comprendre la notion de meilleur choix local*

1. On veut réaliser en plusieurs étapes $s > 200$ avec le système de pièces $(v_i)_{0 \leq i \leq 7} = (1, 2, 5, 10, 20, 50, 100, 200)$; une étape consiste à rendre une pièce à la fois. En quoi l'**optimum local** est-il de rendre 200c ? Quelles sont les 3 premiers optimum locaux lorsque $s = 731$?
2. On considère le problème d'allocation de salle. L'agenda est déjà partiellement construit, n cours s'étant déjà vus attribuer des salles. Sans pour autant prouver que la stratégie définie est optimale, pouvez vous en la comparant à une autre, plus naïve, pour attribuer une salle au $(n + 1)^{ième}$ cours, prouver que ce n'est pas la pire ?

Corrigé en 5.1 page 220.

5.2 Mise en œuvre de stratégies gloutonnes**5.2.1 Le rendu de monnaie**

Nous proposons, avec l'exercice (5.2) la mise en œuvre de la stratégie gloutonne définie en 5.1.2 pour le problème du rendu de monnaie qui est formalisé par le système d'équations (5.1.1).

Rappelons qu'à chaque étape de la stratégie gloutonne qui a été proposée, on choisit la pièce ou le billet de plus grande valeur $v \leq s$ (pour « rendre la monnaie »).

- La construction du choix d'étape est proposé à la première question.
- La question 4 vous aide à démontrer que l'algorithme **termine** et fournit dans tous les cas une **solution à l'équation** $\sum x_i v_i = s$.
- La question 5 de cet exercice est importante : on vous invite à vérifier que, selon la suite des valeurs de monnaie (ou système de monnaie), **l'algorithme glouton ne donne pas une solution optimale**. Il ne résout donc pas exactement notre problème d'optimisation pour tous les systèmes de monnaie.

L'exercice (5.3), quant à lui, permettra d'**explorer l'ensemble des solutions réalisables** pour ce même problème.

Exercice 5.2 *mise en œuvre d'une stratégie gloutonne*

1. Définir une fonction `choix_glouton(V, s)` qui prend en argument une liste V (représentant un système de monnaie) et un entier $s > 0$ et renvoie le tuple (i, q, r) où $V[i]$ est la plus grande valeur de V telle que $V[i] \leq s$, et q, r sont le quotient et le reste de la division euclidienne de s par $V[i]$.
On suppose que V vérifie les pré-conditions : $V[0] = 1$ (qui assure qu'il y a une solution admissible) et V est strictement croissante.
2. Écrire une fonction itérative, `rendu_monnaie(V, s)`, dont les arguments vérifient les mêmes hypothèses que dans la question précédente et qui renvoie la liste

des couples $[x_i, V[i]]$ pour lesquels $x_i > 0$, $\sum x_i V[i] = s$, les étapes du rendu étant définies par le choix glouton de la question précédente.

Par exemple, pour $V = (1, 2, 5, 10, 20, 50, 100, 200)$ et $s = 731$, on aura $\text{rendu_monnaie}(V, s) = [[3, 200], [1, 20], [1, 10], [1, 1]]$, ce qui correspond à $((x_i)_i = [1, 0, 0, 1, 1, 0, 0, 3])$.

3. Écrire une fonction récursive, $\text{rendu_monnaieR}(V, s)$ qui fait le même travail.
4. Prouver que $\text{rendu_monnaieR}(V, s)$ termine et que la post-condition $\sum x_i V[i] = s$ est satisfaite.

Notations

On numérotera les appels : $k = 0$ est le n° de l'appel principal et $k = 1, \dots$ les n° des appels récursifs. De la même façon, $r_0 = s$ est la valeur du paramètre formel s lors de l'appel principal, et r_k sa valeur lors de l'appel récursif n° k . Enfin, i_k désignera le plus grand indice pour lequel $V[i_k] \leq r_k$.

5. Majorer le nombre des appels récursifs (ou le nombre d'itérations) de votre fonction.
6. On considère le système de monnaie $V = [1, 3, 4]$. Que donneraient les appels $\text{rendu_monnaieR}(V, 6)$ et/ou $\text{rendu_monnaie}(V, 6)$? Le résultat est-il optimal ?

Corrigé en 5.2 page 220.

Exercice 5.3 solutions réalisables pour le rendu de monnaie

1. Écrire une fonction récursive $\text{realisables_rendu}(V, s)$ qui prend en argument V , une liste strictement croissante d'entiers strictement positifs et un entier $s \geq 1$. Cette fonction renvoie une liste R dont les éléments sont les sous-listes $X = [x_0, \dots, x_{n-1}]$ des solutions réalisables du problème de rendu de monnaie, ie : pour tout $X \in R$, de $\sum x_i V[i] = s$.

Ajouter à votre fonction une assertion pour les pré-conditions, une assertion pour les post-conditions.

Conseils : On pourra remarquer que les cas $s < V[0]$ et $\text{len}(V) = 1$ peuvent servir de condition d'arrêt de la récursivité. On aura intérêt, pour éviter les calculs redondants, à considérer toutes les éventualités pour $x[0]$, et pour chaque possibilité éventuelle à traiter récursivement le système $V[1], \dots, V[n-1]$.

2. Prouver que votre algorithme termine, qu'il ne calcule pas deux fois un même vecteur X . Prouver enfin sa correction.
3. **TP** : On testera avec le système $V_0 = [1, 2, 5, 10, 20, 50, 100, 200]$ (pièces du système euros) et avec $s = 4, 15, 31, 131, 151$. Ce qui nous donne $|R| = 3, 22, 116, 12673, 22414$.

On pourrait aussi tester avec $s = 731$ pour vérifier que la solution renvoyée par l'algorithme glouton est bien optimale, **mais... c'est long !**

Corrigé en 5.3 page 222.

5.2.2 Sélection d'activités

Le problème

Le problème que nous considérons ici est celui de l'attribution d'une ressource unique à des activités multiples (par exemple des cours ou des conférences qui doivent avoir lieu dans une même salle). Chaque activité a une heure de début, une heure de fin.

Formulation : L'objectif est de sélectionner un sous-ensemble **maximal** d'activités susceptibles de partager la ressource sans collision. Il s'agit encore d'un problème d'optimisation combinatoire (déterminer le maximum de f sur $\mathcal{R}$, ensemble fini revient à déterminer le minimum de $g = -f$ ou de $g = M - f$).

Stratégie gloutonne

Une première idée (il y en aura d'autres) est de trier la liste des activités selon l'heure de fin croissante.

On choisit à la première heure $h = h_0$, la première activité parmi celles qui débutent après h et qui finissent le plus tôt. On pose alors $h = f$, heure de fin de cette activité et on réitère le procédé, tant qu'un choix est possible.

Exercice 5.4 ★ formalisation du problème de sélection d'activités

On veut formaliser l'écriture du problème de sélection d'activités pour une ressource unique comme nous l'avons fait en (5.1.1) pour d'autres problèmes. Pour cela on modélise une activité en l'identifiant à une liste de trois éléments $[a, d, f]$ (identifiant ou numéro, heures de début et de fin de l'activité). On suppose qu'il y a N activités à sélectionner.

1. Donner un **exemple d'ensemble** susceptible de contenir les triplets représentant les activités.
2. On note $\mathcal{A}$ un ensemble de N activités parmi lesquelles il faut faire une sélection et $\mathcal{S}$ l'ensemble des suites finies d'au plus N éléments de $\mathcal{A}$. Quel est le nombre d'éléments de $\mathcal{S}$?
3. Caractériser parmi les éléments de cet ensemble, les solutions réalisables. Écrire une fonction `realisable(LA)` qui prend en argument une liste d'activités et renvoie `True` ou `False` selon qu'elle est réalisable ou pas. Majorer sa complexité en fonction de $n = \text{len}(LA)$.
4. On notera $\mathcal{R}$ l'ensemble des solutions réalisables. Définir une fonction sur $\mathcal{S}$ donc le maximum caractérise les solutions optimales.

Corrigé en 5.4 page 224.

Exercice 5.5 sélection d'activités pour une ressource unique

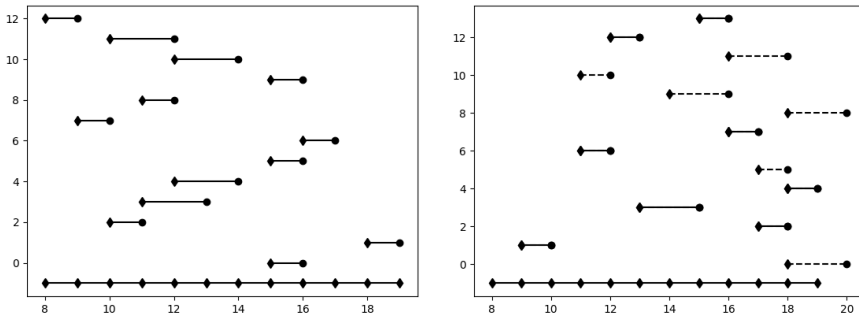
1. Dans le but de faire tourner et tester les scripts qui vont suivre, construire une fonction `generer_activites(N)` qui prend un entier N en argument et retourne une liste de N activités, c'est à dire une liste de triplets $[a, d, f]$ (a : n° de l'activité, d : heure de début, f : heure de fin).

Les activités sont déterminées aléatoirement avec les contraintes suivantes :

- En moyenne 70% d'entre elles durent 1h et 30% durent 2h.
- Les débuts vont d'heure en heure du matin 8h au soir 18h.

Module random : Vous disposez dans la bibliothèque `random` des fonctions `random.random()` qui renvoie un flottant « aléatoire » suivant une loi uniforme sur $[0,1]$ et `random.choice(L)` qui renvoie un élément du conteneur L en faisant un choix équiprobable dans L .

2. Écrire une procédure `dessiner_activites(LA)` qui prend en argument une liste d'activités et en produit une représentation graphique comme sur la figure de gauche ci-dessous.



Indications pour les tracés :

- `pl.plot([8,19], [-1,-1], color='black')` pour tracer une ligne ;
 - `pl.plot(h,-1,'d',color='black')` pour placer le symbole $\blacklozenge$ en position $(h,-1)$;
 - `pl.plot([d, f],[s,s], '--', color='black')` pour une ligne entre les points (d,s) et (f,s) ;
3. Écrire une fonction récursive `choix_glouton_activites(LA, h = 8)` qui prend en arguments LA une liste d'activités et h entier, une heure de début et renvoie la sous-liste de LA formée d'activités compatibles et qui pourront donc partager une ressource commune. Cette fonction suit la démarche gloutonne décrite page 213.
 4. Majorer le nombre des appels provoqués par un appel principal `choix_glouton_activites(LA, h = 8)`.
Justifier par ailleurs que la somme des itérations provoquées par la totalité de ces appels récursifs est constante.

Discuter l'affirmation : la méthode gloutonne pour la sélection d'activité a une complexité en $O(n \ln n)$.

5. Un outil de débogage : Modifier la procédure `dessiner_activites(LA)` pour obtenir `dessiner_activites(LA, AC)`, dans laquelle `AC` est la liste des activités renvoyée par la fonction précédente, qui permettra de visualiser les résultats en représentant en pointillés les activités candidates, en traits pleins les activités retenues (comme sur la figure de droite).

Corrigé en 5.5 page 225.

Une démonstration

Nous allons démontrer que l'algorithme glouton que nous avons mis en place pour la sélection d'activités, renvoie une solution optimale du problème (le nombre d'activités est maximal). Il y a deux choses à prouver :

1. la solution renvoyée est réalisable ;
2. la solution renvoyée est optimale.

Exercice 5.6 *l'algorithme glouton pour la sélection d'activité est exact*

Dans cet exercice on veut montrer que la stratégie définie en 5.1.2 renvoie une solution optimale. La démonstration est indépendante des détails d'une implémentation particulière.

On notera $\mathcal{R}$ l'ensemble des solutions réalisables. Un élément de $\mathcal{R}$ est une suite finie d'activités distinctes $a = (a_0, \dots, a_{n-1})$, sans collision (ou incompatibilité) au niveau des horaires.

1. Justifier que le choix glouton proposé page 213 et dans l'exercice 5.5 construit une solution réalisable. Cela changerait-il avec un autre choix pour le tri initial des activités candidates : heures de début croissantes, durées croissantes ou décroissantes ?
2. On veut ici prouver que la solution fournie par l'algorithme glouton est optimale, c'est à dire que le nombre d'activités programmées (c'est à dire la longueur de la suite) y est maximal.
 - (a) Soient $a = (a_i)_{0 \leq i < p}$ et $a' = (a'_i)_{0 \leq i < q}$ deux éléments de $\mathcal{R}$. On définit $d(a, a')$ de la façon suivante :

$$\begin{cases} d(a, a') = 0, & \text{si } a = a' \\ d(a, a') = 2^{-i}, & \text{si } i \text{ est le premier indice en lequel } a \text{ et } a' \text{ diffèrent.} \end{cases}$$

(ce qui signifie que a_i peut être défini sans que a'_i ne le soit ou que, les deux termes étant définis, $a_i \neq a'_i$).

- i. Que dire de $a[0 : i]$ et $a'[0 : i]$ lorsque $d(a, a') \leq 2^{-i}$?
 - ii. Soient $a, a', a'' \in \mathcal{R}$. Montrer que $d(a, a'') \leq d(a, a') + d(a', a'')$
- (b) ★ On note $g = (g_0, \dots, g_{p-1})$ la suite des activités sélectionnées par l'algorithme glouton. On considère une solution optimale $m = (m_0, \dots, m_{n-1})$. Montrer que si $m \neq g$, on peut construire une autre solution optimale strictement plus proche de g .
- Indication** : Il existe i tel $d(g, m) = 2^{-i}$. Or trois cas se présentent *a priori* lorsque deux solutions réalisables vérifient $d(g, m) = 2^{-i}$.
- $g[i]$ est défini alors que $m[i]$ ne l'est pas.
 - $g[i]$ n'est pas défini alors que $m[i]$ l'est.
 - $g[i]$ et $m[i]$ sont toutes deux définies et $g[i] \neq m[i]$.
- Après avoir justifié que seul le dernier cas est possible, construire en effectuant un remplacement judicieux dans m cette autre suite optimale.
- (c) En déduire que g est optimale.

Corrigé en 5.6 page 226.

5.2.3 Allocations de salles de cours

Nous mettons en œuvre, dans l'exercice qui suit, la stratégie gloutonne définie en 5.1.2. Rappelons qu'il s'agit, une liste de cours avec leurs horaires étant donnée, d'**attribuer des salles aux cours de façon parcimonieuse**.

Exercice 5.7 allocation de salles de cours

1. Dans le but de faire tourner et tester les scripts qui vont suivre, construire une fonction `generer_cours(N)` qui prend un entier N en argument et retourne une liste de N cours, c'est à dire une liste de triplets $[c, d, f]$ (c : n° du cours, d : heure de début, f : heure de fin).

Les cours sont déterminés aléatoirement avec les contraintes suivantes :

- Il y a en moyenne 70% de cours qui durent 1h et 30% de cours qui durent 2h.
- Les cours du matin débutent à 8h au plus tôt, finissent à 12h au plus tard ; ils reprennent à 13h au plus tôt, et terminent à 18h au plus tard. Un cours de 2h ne peut donc débuter ni à 11h ni à 17h.

Module random : Vous disposez dans la bibliothèque `random` des fonctions `random.random()` qui renvoie un flottant « aléatoire » suivant une loi uniforme sur $[0,1]$ et `random.choice(L)` qui renvoie un élément du conteneur L en faisant un choix équiprobable dans L .

2. Écrire une fonction `compatibles(C1,C2)` qui prend en arguments deux listes de deux termes représentant des créneaux horaires, $Ci = [d, f]$ avec $8 \leq d < f \leq 19$, et renvoie un booléen `True` ou `False` selon que l'intersection des intervalles est de longueur nulle ou pas.

3. On suppose que l'algorithme glouton qui va venir dans la question suivante doit déterminer, chaque fois qu'il doit allouer une salle à un cours, la liste (éventuellement vide) des salles déjà attribuées et qui sont occupées sur ce créneau. Pour cela il construit un dictionnaire `planning_salles = { s : [[d,f],[d',f'],...], ... }` dont les clés sont les n° de salles et les valeurs, les listes des créneaux horaires $[d, f]$ pendant lesquelles la salle est occupée.

Écrire une fonction `salles_incompatibles(D, C0)` qui prend en argument un dictionnaire du même type que `planning_salles`, une liste $C0 = [d, f]$ représentant un créneau horaire, et renvoie la liste des clés du dictionnaire `D` qui sont les n° des salles occupées pendant ce créneau.

4. Écrire une fonction `allocations_gloutonnes(LC, S)` qui prend en arguments une liste de cours `LC` (un cours étant défini comme précédemment : $[c, d, f]$) et un entier S , nombre de salles disponibles (chaque salle est représentées par un n° : $0 \leq s < S$).

Cette fonction renvoie un tuple (R, A, s) dans lequel R est un booléen (True ou False selon que tout cours s'est vu attribuer une salle ou pas), A l'agenda correspondant (partiel en cas d'échec à une certaine étape), s l'indice de la dernière salle attribuée.

Le principe d'allocation est celui défini en (5.1.2) page 210 :

- `LC` est triée selon les heures de fin de cours croissantes ;
 - L'algorithme parcourt les cours de la liste `LC` et recherche la première salle disponible à chaque itération ;
 - Lorsque cette salle est trouvée, elle est attribuée au cours courant ; sinon l'algorithme échoue. Les deux événements (début de cours, fin de cours) sont ajoutés à l'agenda.
 - Lorsque le processus termine sans échec, la fonction renvoie (True, A, s).
5. Déterminer le nombre d'appels à la fonction `compatibles(C1, C2)` de votre fonction `allocations_gloutonnes(LC, S)` en fonction du nombre de cours ($n = \text{len}(LC)$). Ces appels sont indirects si vous utilisez comme suggéré par l'énoncé la fonction `salles_incompatibles(D, C0)`.
6. Écrire un programme de visualisation/debuggage de l'agenda qui dessine les créneaux d'occupation des salles comme sur la figure de la page 218.

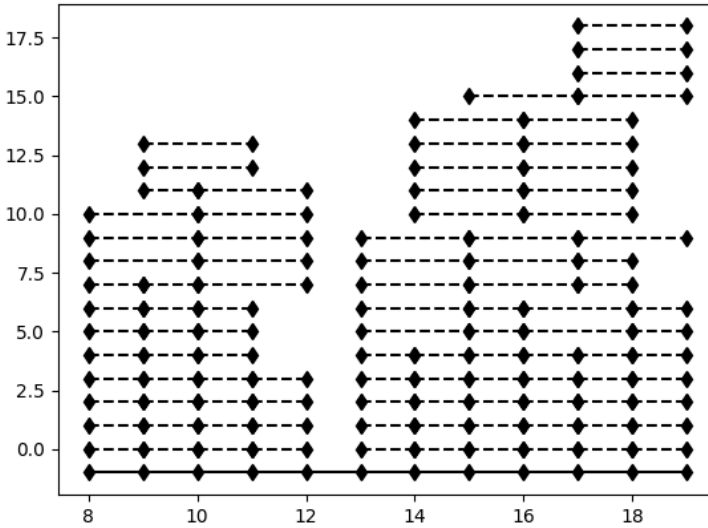
Sur la figure, 19 salles pour 100 cours. Les n° de salle sont en ordonnées, les horaires en abscisses et les cours entre $\blacklozenge - - - \blacklozenge$.

Indications pour les tracés :

- `pl.plot([8,19], [-1,-1], color='black')` trace une ligne ;
- `pl.plot(h,-1,'d',color='black')` pour placer le symbole $\blacklozenge$ en position $(h,-1)$;
- `pl.plot([d, f], [s,s], '--', color='black')` pour une ligne pointillée entre les points (d,s) et (f,s) ;
- Construire un dictionnaire $D = \{c : \{'d' : d, 'f' : f, 's' : s\}, \dots\}$ pour tracer

ensuite les lignes pointillées et les marqueurs de début et fin (à la hauteur de la salle s).

Corrigé en 5.7 page 228.



Exercice 5.8 *allocation de salles de cours, validation (très) partielle de la méthode*
 Pour cet exercice, il n'est pas indispensable d'avoir implémenté le programme, c'est la stratégie définie en 5.1.2 qui y est étudiée et non pas pas le détail de la programmation qui fait l'objet de l'exercice précédent.

On notera $\mathcal{R}$ l'ensemble des solutions réalisables.

1. Observons que montrer que l'algorithme glouton que nous avons défini renvoie une solution réalisable revient à montrer qu'il n'y a pas de collision dans les attributions : on n'attribue jamais la même salle à deux cours qui se chevauchent. Prouver ce résultat en raisonnant par récurrence sur le nombre de cours (on ne se préoccupe pas du nombre de salles disponibles que l'on estime suffisant).
2. On représente les éléments de $\mathcal{R}$ par les listes $m = (m_0, \dots, m_{n-1})$ où m_i est la salle attribuée au cours $n^\circ i$ (les cours étant (ré)ordonnés selon un même ordre croissant des fins de cours qui est **celui choisi par l'algorithme**, on ne se préoccupe pas d'une éventuelle façon de construire m). On note $g = (g_0, \dots, g_{n-1})$ la suite des salles attribuées aux cours par l'algorithme glouton.

(a) Soient m et m' deux éléments de $\mathcal{R}$. On définit $d(m, m')$ de la façon suivante :

$$d(m, m') = \begin{cases} 0 & \text{si } m = m' \\ 2^{-i} & \text{si } i = \min\{j/m_j \neq m'_j\} \end{cases}$$

On considère $m, m', m'' \in \mathcal{R}$.

Que dire de $m[0 : i]$ et $m'[0 : i]$ lorsque $d(m, m') \leq 2^{-i}$?

Montrer que $d(m, m'') \leq d(m, m') + d(m', m'')$

- (b) On considère une solution optimale $m = (m_0, \dots, m_{n-1})$. On suppose que $d(m, g) = 2^{-i}$ et que $g[i]$ n'est pas dans les salles déjà choisies.

Montrer que l'on peut construire une autre solution optimale mais strictement plus proche de g .

- (c) On considère parmi les solutions optimales une solution μ telle que $d(g, \mu)$ soit la plus petite. Que peut-on déduire du nombre de salles affectées jusqu'à l'étape $n^o i$ en laquelle μ et g diffèrent ?

Corrigé en 5.8 page 231.

5.3 Bilan

On retiendra qu'appliquer une méthode gloutonne à un problème d'optimisation combinatoire préalablement décomposé en étapes consiste à faire une succession de choix **optimaux à chaque étape** en vue de réduire une dimension du problème.

Ces stratégies permettent de déterminer des solutions à la fois **réalisables et proches des solutions optimales**. Comme nous l'avons constaté avec la dernière question de l'exercice (5.2), le résultat obtenu n'est pas toujours une solution optimale, même si elle appartient à l'ensemble des solutions réalisables.

Dans le cas où l'on peut prouver que l'algorithme renvoie une solution optimale on dit qu'il est exact. C'est le cas de l'algorithme de sélection d'activités se partageant une ressource unique. La démonstration proposée dans l'exercice (5.6) où l'on montre par **substitution** que l'on peut approcher une solution gloutonne par une solution optimale est fréquente dans ce type de problème.

Lorsque la méthode gloutonne n'est pas exacte ou lorsqu'on ne sait pas si elle l'est, on dit que la méthode est approchée. On utilise de telles méthodes lorsqu'elles offrent de bons résultats en pratique.

Ces méthodes sont motivées par le fait que l'ensemble des solutions réalisables, bien que fini, est souvent trop grand pour une recherche exhaustive ou la mise en œuvre d'un algorithme plus sophistiqué. Dans certains cas on ne pourrait même pas le stocker.

Par ailleurs, les algorithmes gloutons, comme c'est le cas de ceux que nous avons vus ont souvent une complexité quasi-linéaire, c'est à dire en $O(n \ln n)$, comme pour la sélection d'activités, ou polynomiale (comme notre algorithme d'allocation de salles qui peut être amélioré en faisant appel à la théorie des graphes) ce qui est appréciable.

5.4 Corrigés des exercices

Corrigé de l'exercice n° 5.1

1. C'est l'option qui nous rapproche le plus de l'aboutissement $s = 0$. On obtiendra à l'étape suivante $s' = s - 200 > 0$, alors qu'avec toute autre pièce on aurait $s' = s - v_i > s - 200$.

Ainsi avec $s = 731$, on aura trois étapes successives avec 200 de rendu.

2. On pourrait, de façon irréfléchie chercher une salle qui n'a pas encore été attribuée. Cela consommerait bien plus de salles (une par cours dans la journée). Alors que notre stratégie est localement optimale : c'est la plus économe **une fois le classement de la liste opéré** : on essaye de ne pas utiliser de nouvelle salle.

Corrigé de l'exercice n° 5.2

1. 2. 3. Les scripts sont présentés dans les tableaux ci-dessous et page 222. On y a inséré des assertions pour pré et post-conditions.

4. Considérons la fonction récursive pour notre analyse.

Terminaison

Notons k le numéro de l'appel et $r_0 = s, r_1, \dots, r_{p-1}$, les valeurs prises par l'argument s lors des appels principal ($r_0 = s$) et récursifs.

La fonction `choix_glouton(V, s = rk)` calcule et renvoie r_{k+1} qui est le reste de la division euclidienne de r_k par $V[i_k]$. On a donc $r_{k+1} < V[i_k] \leq r_k$. Cette suite est une suite d'entiers positifs strictement décroissante, le programme termine nécessairement sur la condition d'arrêt $r_{p-1} = s = 0$.

Post condition $\sum x_i v_i = s$

Lors de chaque appel $r_k = q_k V[i_k] + r_{k+1}$.

On somme pour $k = 0, \dots, n-1$, on simplifie et comme $r_0 = s$ et $r_n = 0$ on obtient $\sum q_k V[i_k] = s$.

5. On a, dans les appels récursifs $r_0 = s \geq V[i_0] > r_1 \geq V[i_1] > r_2 \geq \dots > r_{p-1}$. On aura au plus $n = |V|$ étapes.

6. Pour le système [4,3,1] la méthode gloutonne proposée renvoie une solution de rendu (comme nous l'avons prouvé) mais **qui n'est pas optimale**.

En effet, partons de $s = 6$. Le choix glouton conduit à se précipiter sur une pièce de 4. Il reste $r = 2$. Le plus petit indice tel que $V[i] \leq 2$ est $i = 2$ avec $V[2] = 1$. Le résultat final, obtenu en deux étapes est $[[1, 4], [2, 1]]$ avec $\sum x_i = 3$ (une pièce de 4, deux pièces de 1). Ce n'est pas optimum car on pouvait faire $[[2, 3]]$ avec $\sum x_i = 2$.

```

def choix_glouton(V, s):
    '''
    s : int, s >= 1 (monnaie à rendre);
    V : list (liste d'entiers: système de monnaie).

    Pré-cond: V strict. croissante, V[0]=1 <= s.
    '''
    k = len(V)-1
    while k > 0 and s < V[k]:
        k -= 1
    # condition d'arrêt: V[k] <= s < V[k+1]
    q, r = divmod(s, V[k])
    return k, q, r

def rendu_monnaie(V, s):
    '''
    s : int, monnaie à rendre;
    V : list (liste d'entiers: système de monnaie).
    Renvoie une liste d'élts. [[xi, si], ...].

    Pré-cond. : V strict. croissante, V[0]=1 <= s.
    Post-cond.: sum(xi*V[i])= s, si in V, 1 <= xi.
    '''
    assert isinstance(V, list) and V[0]==1 and s>= 1

    r, i = s, len(V)
    R = []

    while r > 0 and i > 0:
        i, q, r = choix_glouton(V[0:i], r)
        R.append([q, V[i]])
    # vérification de la post-condition
    assert sum([e[0]*e[1] for e in R]) == s
    return R

```

```

def rendu_monnaieR(V, s):
    '''
    s : int, monnaie à rendre;
    V : list (liste d'entiers: système de monnaie).
    Renvoie une liste d'élts.  [[xi, si], ...].

    Pré-cond. : V strict. croissante et V[0]=1 <= s.
    Post-cond.: sum(xi*si)= s, si in V, 1 <= xi.
    '''
    assert isinstance(V, list) and V[0]==1 and s>= 1

    k, q, r = choix_glouton(V, s)
    if r == 0:
        return [[q, V[k]]]
    else:
        R = rendu_monnaieR(V[0:k], r)
        R.insert(0, [q, V[k]] )
        # vérification de la post-condition
        assert sum([e[0]*e[1] for e in R]) == s
        return R

```

Corrigé de l'exercice n° 5.3

1. Le script figure plus bas.

2. **Terminaison** : Pour un appel avec V non vide, de longueur $n > 0$, l'arrêt est immédiat si $n = 1$. Si $n > 2$ les appels récursifs de profondeur h se font avec des listes de longueurs $n - h$. On atteindra donc la condition d'arrêt $\text{len}(V)=1$.

Non redondance : On le prouve par récurrence sur la longueur de V .

Si $\text{len}(V) = 1$, il n'y a clairement pas de redondance !

Supposons que les vecteurs X calculés dans un appel avec $\text{len}(V) = n$ soient tous distincts.

Pour un appel avec $\text{len}(V) = n + 1$, les vecteurs X qui ont une même valeur $X[0] = k$, ont des coefficients $X[1], \dots$ construits dans un même appel récursif avec $\text{len}(V[1:]) = n$, ils sont distincts par hypothèse de récurrence. L'ensemble de tous les vecteurs l'est aussi.

Correction totale :

Notre programme doit renvoyer un ensemble (l'ensemble $\mathcal{R}$ des solutions réalisables).

On le prouve encore par récurrence sur $n = \text{len}(V)$.

- Clair si $n = 1$.
- On suppose que le programme renvoie une solution correcte pour un certain $n \geq 1$.

Si $\text{len}(V) = n + 1$, les solutions réalisables sont les vecteurs X de taille $n + 1$ tels que $\sum x_i V[i] = s$. Donc $X \in \mathcal{R} \Leftrightarrow \sum_{i=1}^n x_i V[i] = s - x_0 V[0]$.

Par HR l'ensemble des vecteurs $X1 = [x_1, \dots, x_n]$ est bien construit par notre programme. Ce sera aussi le cas pour l'ensemble des éléments de $\mathcal{R}$ puisque le programme colle à la remarque ci-dessus pour le construire.

```
def realisables_rendu(V,s):
    '''
    s : int, s >= 1 (somme à réaliser);
    V : list (liste d'entiers: système de monnaie).
    Renvoie la liste des solutions réalisables
    (liste de 'vecteurs': X = [x0,...,x(n-1)]).

    Pré-cond : s >=1, V strict. croissante, 0 < V[0].
    Post-cond: pour tout X, sum(xi*V[i])= s.
    '''
    assert isinstance(V, list) and len(V) >=1
    assert isinstance(s, int) and s >=1
    R = []
    if s < V[0]:
        return []
    elif len(V) == 1:
        q, r = divmod(s, V[0])
        if r == 0:
            return [[q]]
        else:
            return []
    v, k = V[0], 0
    while k*v < s:
        V1 = V[1:]
        for X1 in realisables_rendu(V1, s-k*v):
            X1.insert(0,k)
            assert sum([X1[i]*V[i]
                        \for i in range(0,len(V))]) == s
            R.append(X1)
        k += 1
    if k*v == s:
        R.append([k] + [0 for i in range(1, len(V))])
    return R
```

La ligne

`assert sum([X1[i]*V[i] for i in range(0,len(V))]) == s`
 garantit que seules des solutions réalisables sont calculées si le programme termine sans lever d'exception. Mais ce n'est qu'une vérification à la volée, pas une preuve pour tous les cas.

3. Vérifications importantes, on ne peut les faire à votre place. On a un beau programme dont la **correction totale** (voir le chapitre 8 pour ces définitions) est prouvée, mais il ne sert pas à grand chose, tant l'ensemble grandit vite !

Corrigé de l'exercice n° 5.4

1. Si les heures de début sont les heures piles de 8h à 18h par exemple et les heures de fin de 9h à 19h, **on pourra** considérer que $[a, d, f] \in \mathbb{N} \times \llbracket 8, 18 \rrbracket \times \llbracket 9, 19 \rrbracket \subset \mathbb{N}^3$.

2. $|\mathcal{A}| = N$ et les suites finies d'éléments de $\mathcal{A}$ sont des k -uplets de $1, 2, \dots, N$ éléments. Donc

$$|\mathcal{S}| = \sum_{k=1}^N \binom{N}{k} \times k! = \sum_{k=1}^N \frac{N!}{(N-k)!}$$

3. Une suite finie de k éléments de $\mathcal{A}$, $([a_i, d_i, f_i])_{0 \leq i \leq k-1}$ est réalisable ssi deux termes distincts sont compatibles (leurs intervalles ne se chevauchent pas) :

$$\forall (i, j) \in \llbracket 0, k-1 \rrbracket, i \neq j \Rightarrow [d_i, f_i] \cap [d_j, f_j] = \emptyset.$$

Ce qui se traduit par

```
def compatibles(A1, A2):
    '''
    A1, A2: list (de la forme [a,d,f] avec d<f).
    '''
    return A2[1]>= A1[2] or A1[1] >= A2[2]

def realisable(LA):
    '''
    LA : list (d'élts. [a,d,f], d < f)
    '''
    n = len(LA)
    for i in range(0, n):
        for j in range(i+1, n):
            if not compatibles(LA[i], LA[j]):
                return False
    return True
```

Le nombre d'appels à compatibles(LA[i], LA[j]) est clairement

$$\sum_{0 \leq i < j \leq n-1} 1 = \frac{n(n-1)}{2}.$$

4. On posera $f(r) = \text{len}(r)$. Soit, si $r = ([a_i, d_i, f_i])_{0 \leq i \leq k-1} \in \mathcal{R}$, $f(r) = k$.

Corrigé de l'exercice n° 5.5

Les scripts sans autre forme de procès.

```
import random
import pylab as pl

def generer_activites(N):
    C = []
    for i in range(0, N):
        u = random.random()
        if u < 0.7:
            duree = 1
        else:
            duree = 2
        d = random.choice(range(8, 19))
        C.append([i, d, d + duree])

    C.sort(key = lambda c: c[2])
    # tri par fins croissantes
    return C

def dessiner_activites(LA):
    pl.plot([8, 19], [-1, -1], color = 'black')
    for h in range(8, 20):
        pl.plot(h, -1, 'd', color = 'black')
    for A in LA:
        c, d, f = tuple(A)
        pl.plot([d, f], [c, c], '--', color = 'black')
        pl.plot(d, c, 'd', color = 'black')
        pl.plot(f, c, 'bo', color = 'black')
    pl.show()
    pl.close()
```



```
def choix_glouton_activites(LA, h = 8):
    '''
    LA: list; liste d'activités [a, d, f];
    h : int; heure à laquelle la ressource est libre.
    Pré-cond: LA est triée selon les fins croissantes.
    '''
    print(len(LA))
    i = 0
    while i < len(LA) and LA[i][1] < h:
        i += 1
    if i == len(LA):
        return []
    else:
        f = LA[i][2]
        return [LA[i]] + choix_glouton_activites(LA[i+1:], f)
```

• Complexité de `choix_glouton_activites(LA, h)`

Notons LA_p, h_p les valeurs des arguments LA et h lors des appels principal ($p = 0$) et récursifs ($p \geq 1$).

Dans le pire des cas les activités de LA sont compatibles entre elles et seront toutes choisies : chaque ajout étant la conséquence d'un unique appel, il y a n .

Par ailleurs, **la boucle while** parcourt LA_{p+1} depuis son premier terme (qui est le successeur de a_p dans LA_0) jusqu'à l'activité a_{p+1} . L'appel suivant se fait avec l'argument $LA = LA[a_{p+1} :]$. LA_0 est donc parcourue par paquets successifs et dis-joints. Il y a en tout n itérations.

Cet algorithme est bien linéaire, mais comme il faut trier la liste avant de la lui soumettre **la méthode gloutonne aura la complexité du tri préalable**, à savoir $O(n \ln)$.

Corrigé de l'exercice n° 5.6

1. On note d_k et f_k les horaires de début et fin de l'activité k , ($0 \leq k \leq n - 1$). Il ne peut y avoir de collision s'il n'y a qu'une activité. D'autre part, par construction, on aura

$$h_0 = d_0 < f_0 = h_1 \leq d_1 < f_1 = h_2 \leq d_2 < f_2 = h_2 \dots$$

et il n'a pas non plus de collision si $n \geq 1$.

Ce qui précède ne tient pas compte de l'ordre dans la liste des activités présentée en entrée pour l'algorithme. Le choix glouton renverra donc toujours une solution réalisable.

2. (a) Dire que $d(a, a') \leq 2^{-i}$ c'est dire que :
- $d(a, a') = 0$, d'où $a = a'$ et $a[0 : i] = a'[0 : i]$;
 - ou
 - il existe $j \geq i$ tel que $d(a, a') = 2^{-j}$ et on a $a[0 : j] = a'[0 : j]$ et a fortiori $a[0 : i] = a'[0 : i]$.

• **Inégalité triangulaire :**

Si $a = a'$ ou $a = a''$ ou $a' = a''$, c'est évident.

Plaçons nous donc dans le cas où les trois éléments (points ?) sont distincts et posons $d(a, a') = 2^{-i}$, $d(a'', a') = 2^{-j}$.

On a alors $a[0 : i] = a'[0 : i]$ et $a''[0 : j] = a'[0 : j]$ et, en notant $k = \min(i, j)$, il vient : $a[0 : k] = a'[0 : k] = a''[0 : k]$.

Donc $d(a, a'') \leq 2^{-k} \leq 2^{-i} + 2^{-j}$. CQFD.

- (b) On considère g la solution gloutonne et m une solution optimale. Si $g \neq m$, il existe i tel que $d(g, m) = 2^{-i}$ avec $g[0 : i] = m[0 : i]$.

Regardons les trois cas qui se présentent :

- i. $g[i]$ est défini alors que $m[i]$ ne l'est pas.

Rappelons que m est optimale donc de longueur maximale, donc $g[i]$ ne peut être défini alors que $m[i]$ ne le serait pas. Cas à exclure.

- ii. $g[i]$ n'est pas défini alors que $m[i]$ l'est.

Comme $m[i]$ débute après h_0 si $i = 0$ ou $h_i = f_{i-1}$ si $i > 0$, il y avait une activité qui était un choix possible pour la stratégie gloutonne. Contradiction également.

- iii. $g[i]$ et $m[i]$ sont toutes deux définies et $g[i] \neq m[i]$.

On peut alors définir m' obtenue en gardant les termes de m sauf le terme $m[i]$ remplacé par $g[i]$. Cela reste une suite réalisable parce que $g[i]$ a été choisie de telle sorte que son heure de fin soit la plus petite possible. L'activité $g[i]$ termine donc avant $m[i]$ et donc avant que $m[i+1]$ ne débute. m et m' ont même longueur, m' est optimale et $d(m, m') \leq 2^{-(i+1)}$.

- (c) Pour toute solution optimale (et il en existe puisque $\mathcal{R}$ est fini), soit $m = g$ soit il existe m' telle que $d(m', g) < d(m, g)$. Comme l'ensemble des solution optimales est fini, le processus de construction de solutions optimales de plus en plus proches de g , donc distinctes, $m \rightarrow m' \rightarrow m'' \rightarrow \dots \rightarrow m^{(k)}$ s'arrêtera avec $m^{(k)} = g$. g est donc bien optimale.

Corrigé de l'exercice n° 5.7

```

import random

def generer_cours(N):
    C = []
    for i in range(0,N):
        u = random.random()
        if u < 0.7:
            duree = 1
            d = random.choice([8,9,10,11,13,14,\
                               15,16,17,18])
        else:
            duree = 2
            d = random.choice([8,9,10,13,14,15,\
                               16,17])
        C.append([i, d, d + duree])

    C.sort(key = lambda c: c[2])
    # tri par fins croissantes
    return C

def dessiner_cours(LC):
    pl.plot([8,19], [-1,-1], color='black')
    for h in range(8,20):
        pl.plot(h, -1, 'd', color='black')
    for C in LC:
        c, d, f = tuple(C)
        pl.plot([d, f], [c, c], '--', color='black')
        pl.plot(d, c, 'd', color='black')
        pl.plot(f, c, 'bo', color='black')
    pl.show()
    pl.close()

def compatibles(C1, C2):
    '''
    C1, C2: list (de la forme [d,f] avec d<f).
    '''
    return C2[0]>= C1[1] or C1[0] >= C2[1]

```

```

def salles_incompatibles(D, C0):
    """
    D : dict. {s : [[d,f], [d',f'],...]};
    C0 : list (C0 = [d, f], heures de début et fin).
    """
    LO = []
    # salles occupées pendant le créneau
    for s in D:
        for C in D[s]:
            # C = [d',f'], créneau d'occ.
            if not compatibles(C, C0):
                LO.append(s)
                break
        #pas la peine de continuer
    return LO

def allocations_gloutonnes(LC, S):
    """
    LC : list, liste de cours [c,d, f];
    S : int, nombre de salles.
    """
    LC.sort(key = lambda c: c[2])
    # on trie les cours par heure de fin croissante
    A, planning_salles = [], {}
    # A est l'agenda, planning_salles décrit en Q3
    for C in LC:
        c, d, f = C[0], C[1], C[2]
        s = 0
        T = salles_incompatibles(planning_salles, [d,f])
        # T liste des salles occupées durant [d, f]
        while s < S and s in T:
            s+= 1
        if s == S:
            return False, A, s
        else:
            A.append([d, 0, c, s])
            A.append([f, 1, c, s])
            if s in planning_salles:
                planning_salles[s].append([d,f])
            else:
                planning_salles[s] = [[d,f]]

    A.sort(key = lambda e: e[0])
    return True, A, s

```

```

def dessiner_agenda(A):
    pl.plot([8,19], [-1,-1], color='black')
    for h in range(8,20):
        pl.plot(h, -1, 'd', color='black')

    D = {}
    for e in A:
        t, w, c, s = tuple(e)
        if not c in D:
            D[c] = {}
            if w == 0:
                D[c]['d'] = t
            else:
                D[c]['f'] = t
        else:
            if w == 0:
                D[c]['d'] = t
            else:
                D[c]['f'] = t
        D[c]['s'] = s

    for c in D:
        d, f, s = D[c]['d'], D[c]['f'], D[c]['s']
        pl.plot([d, f], [s,s], '--', color='black')
        pl.plot(d, s, 'd', color='black')
        pl.plot(f, s, 'd', color='black')

    pl.show()
    pl.close()

```

Complexité

- Dans la fonction `salles_incompatibles(D, C0)`, les appels dépendent du nombre de **valeurs** dans le dictionnaire (et pas du nombre de clés). Dans le pire des cas toutes les salles sont compatibles, les boucles `for` vont à leurs termes (pas d'instruction `break`) : c'est exactement le nombre de valeurs, donc de cours déjà programmés.
- Dans la fonction `allocations_gloutonnes(LC, S)` lorsque le nombre de salles est suffisant, il y aura systématiquement allocation. Lors de l'itération dans la boucle `for` qui se fait avec la liste `LC` n° k il y a déjà eu k affectations de cours qui sont en valeurs dans le dictionnaire `planning_salles`. Un appel à `salles_incompatibles(...)` provoque donc à ce moment au plus k appels à `compatibles(C1,C0)`. Le nombre total d'appels à `compatibles(C1,C0)` est donc majoré par :

$$\sum_{k=0}^{n-1} k = \frac{n(n-1)}{2}$$

Corrigé de l'exercice n° 5.8

1. • S'il n'y a qu'un cours, la méthode gloutonne, attribue une salle. C'est clairement une solution réalisable (et par ailleurs optimale).
 • Supposons que la méthode permette de réaliser des allocations sans collision pour n cours et considérons une liste de $n + 1$ cours. Aux n premiers seront allouées n salles (pas nécessairement distinctes) sans collision. L'algorithme cherche alors une salle libre pour le créneau du $(n+1)^{ième}$ cours. Par définition il choisit la première salle libre sur le créneau en partant de la salle n°0. En attribuant cette salle, il n'y a toujours pas de collision.
2. (a) Dire que $d(m, m') \leq 2^{-i}$ c'est dire que :
 - $d(m, m') = 0$, d'où $m = m'$ et $m[0 : i] = m'[0 : i]$;
 ou
 - il existe $j \geq i$ tel que $d(m, m') = 2^{-j}$ et on a $m[0 : j] = m'[0 : j]$ et a fortiori $m[0 : i] = m'[0 : i]$.

• Inégalité triangulaire

Si $m = m'$ ou $m = m''$ ou $m' = m''$, c'est évident.

On se place donc dans le cas où les trois éléments (points ?) sont distincts :

On pose $d(m, m') = 2^{-i}$, $d(m'', m') = 2^{-j}$.

Alors, $m[0 : i] = m'[0 : i]$ et $m''[0 : j] = m'[0 : j]$.

Notons $k = \min(i, j)$; il vient :

$m[0 : k] = m'[0 : k] = m''[0 : k]$. Donc $d(m, m'') \leq 2^{-k} \leq 2^{-i} + 2^{-j}$.
 CQFD.

- (b) On considère g solution gloutonne et m solution optimale. Si $g \neq m$, il existe i tel que $d(g, m) = 2^{-i}$ avec $g[0 : i] = m[0 : i]$.
 Considérons alors l'allocation $m[i] \neq g[i]$ qui est la première en laquelle g et m diffèrent.
 Si $g_i \notin g[0 : i] = m[0 : i]$, c'est qu'aucune des salles déjà attribuées n'est libre pour le créneau i . Dans ce cas m_i non plus n'est pas dans les salles déjà utilisées. Si on échange dans m les n° des salles g_i et m_i cela ne change rien aux attributions déjà faites, et la liste m' ainsi obtenue reste valide et optimale. C'est alors une liste optimale telle que $dist(g, m') \leq 2^{-(i+1)} < d(g, m)$.
- (c) Si g et μ diffèrent à partir de l'étape i le choix $g[i]$ est celui d'une salle déjà occupée, celui de μ est celui d'une autre salle que $g[i]$ qui est soit déjà occupée, soit une nouvelle salle. A ce moment là, le choix opéré par l'algorithme glouton a toujours été le plus parcimonieux. Nous n'en avons pas pour autant prouvé que le choix définitif sera le choix g .

Traitement de l'image

6.1 Représentation des images, formats, outils

6.1.1 Tableaux à plusieurs dimensions et représentation des images

La représentation informatique des images « colle » à la façon dont elles sont affichées sur les écrans. Ceux-ci sont composés de petits carrés (les pixels) disposés selon une grille et colorés d'une façon homogène. Chaque pixel est repéré par un couple d'entiers et peut être éclairé et coloré ; une image-écran est donc directement associée à un état de cet ensemble.

Cela nous conduit naturellement à représenter une image numérique par un tableau ou une matrice T : chaque terme $T[x, y]$ représente l'état de coloration d'un pixel. De façon très générale, on convient que le terme $T[0, 0]$ est le point situé **en haut et à gauche de l'image** et que $T[i, j]$ est le point situé sur la ligne d'indice i en partant du haut et sur la colonne d'indice j en partant de la gauche.

A titre d'exemple, l'écran que nous utilisons lorsque nous écrivons ces lignes, mesure environ 47 cm de large sur 30 cm de haut avec une résolution de 1680 par 1050 pixels. Il affiche donc à peu près 1250 pixels par cm^2 . Les couleurs y sont par ailleurs codées sur 32 bits (soit 4 octets)¹.

Cela nous amène à réfléchir au contenu de $T[i, j]$, c'est-à-dire aux états du pixel correspondant.

- Pour une image en noir et blanc, il y a deux états possibles et un bit suffira (0 ou 1 selon que le pixel est allumé ou pas).
- Pour une image en niveaux de gris, on **code le plus souvent** les nuances de gris de 0 pour le noir, à 255 pour le blanc ; soit 256 valeurs ou 2^8 bits = 1 octet.
- Pour une image en couleurs, la convention la plus répandue est le codage RGB (red, green, blue) : la couleur est représentée par la superposition des couleurs fondamen-

1. Informations lues sur le logiciel de réglage de la carte graphique gérant cet écran (HP 2229h).

tales choisies ; un premier octet pour le rouge (de 0 à 255), le deuxième pour le vert et le troisième octet pour le bleu. On ajoute à cela un quatrième octet pour coder la transparence du pixel. Cela nous fait bien 4 octets ou 32 bits par pixel comme nous l’avons observé avec notre carte graphique.

Il apparaît alors que notre représentation matricielle habituelle devient insuffisante pour représenter une image en couleur. Il nous faudra donc des tableaux tels que $T[i, j]$ contienne lui-même un tableau.

6.1.2 Des fichiers images vers les tableaux de numpy

Comme les données de tout type, les images sont stockées dans des fichiers. Pour qu’un logiciel puisse traiter une image il faudra que la façon dont les données sont codées lui soit connue. Le format que nous avons suggéré n’est pas le seul : on peut par exemple décider de ne pas coder sur 256⁴ couleurs et transparences, mais de choisir une palette de 16 ou 256 couleurs seulement et, pour chaque pixel, de n’indiquer que la place de la couleur dans la palette ; on peut aussi choisir différents formats de compression des images.

Nous n’allons pas rentrer dans ces détails et ferons appel à des modules spécialisés qui :

- nous permettront de charger une image dans un tableau à trois dimensions ou une hypermatrice (de taille $p_x \times p_y \times 3$ pour une image de $p_x \times p_y$ pixels en RGB par exemple) ;
- nous fourniront des outils de traitement de cette image (mais nous jouerons surtout avec les nôtres) ;
- nous permettront, à partir d’une représentation par des tableaux, d’afficher notre image et de l’enregistrer sous différents formats.

• Le module matplotlib.image avec Python

Le sous-module matplotlib.image de matplotlib, permet de traiter les images avec Python. Ses fonctions essentielles sont données dans le tableau.

fonctions indispensables	brèves descriptions
<code>img=mpimg.imread('*.png')</code>	Charge le fichier dans un tableau de taille $p_x \times p_y \times 3$ pour une image RGB, de taille $p_x \times p_y$ pour une image en niveaux de gris.
<code>imgplot = pl.imshow(img)</code>	Crée l’image dont img est la représentation « matricielle ».
<code>pl.show()</code>	Affiche l’image courante à l’écran.
<code>pl.close()</code>	Prépare une fermeture « propre » de l’image courante d’un clic de souris.

Nous allons commencer par reconnaître le format d'une image lue dans un fichier et chargée avec la fonction `imread(fichier)` de ce module .

On commence comme toujours par importer modules et sous-modules. Dans la suite de ce chapitre nous supposerons systématiquement que ces importations ont été effectuées et nous ne les reproduirons plus.

```
>>> import matplotlib.pyplot as pl
>>> import matplotlib.image as mpimg
>>> import numpy as np
```

- On charge tout d'abord une image au format .jpg (ou .jpeg) avec la fonction `imread()` qui renvoie, comme on peut le constater, un tableau du type `numpy.ndarray` à trois dimensions dont la taille est $960 \times 1280 \times 3$.

```
>>> dir = '/media/.../'
>>> img = mpimg.imread(dir + 'Voilier.jpg')
>>> isinstance(img, np.ndarray)
True
>>> img.ndim
3
>>> img.shape
(960, 1280, 3)
>>> img[0,0,:]
array([21, 15, 19], dtype=uint8)
>>> img[0,0]
array([21, 15, 19], dtype=uint8)
>>> type(img[0,0])
<class 'numpy.ndarray'>
```

Ce tableau correspond à une image de 960×1280 pixels, en couleurs puisque `img[0,0]` qui représente le pixel du haut à gauche de l'image est représenté par un tableau de dimension 1 et de longueur 3 dont les niveaux R,G,B sont 21, 15, 19. Le type des données dans un tableau est uniforme, il est ici précisé dans l'attribut `dtype` (`dtype = uint8`).

Ces nombres sont des entiers codés sur 8 bits (uint8) et prennent des valeurs entre 0 et 255. Soit $2^8 = 256$ valeurs possibles. On prendra garde au fait qu'avec les entiers de type **uint8** on calcule modulo 256.

Par exemple,

$$114 + 171 = 285 = 256 + 29 \equiv 29 [256].$$

Les lignes de console qui figurent à droite montrent les coercitions (conversions de types) conséquences implicites (ou explicite, pour ce qui est de la dernière ligne), d'opérations entre entiers uint8, entiers et flottants.

Ce codage RGB (rouge, vert, bleu) est décrit dans l'exercice 6.1 et aussi dans 6.2 qui reprend le début du sujet d'informatique de tronc commun du concours Centrale-Supélec 2020.

Les entiers uint8

```
>>> x = np.uint8(312)
>>> y = np.uint8(123)
>>> z = x+y; z
179
>>> 2*z
358
>>> d = np.uint8(2)
>>> d*z
102
>>> z*0.1
17.900000000000002
>>> np.uint8(z*0.1)
17
```

• On charge maintenant une image au format .png avec la même fonction `imread()` qui renvoie, comme on peut le constater, un tableau du type `numpy.ndarray` à trois dimensions dont la taille est $960 \times 1280 \times 3$. **Cette fois les valeurs dans le tableau sont des flottants codés sur 32bits.**

```
>>> dir = '/media/.../'
>>> img = mpimg.imread(dir + 'Voilier.png')
>>> isinstance(img, np.ndarray)
True
>>> img.ndim
3
>>> img.shape
(960, 1280, 3)
>>> img[0,0,:]
array([0.11372549, 0.08235294, 0.10196079], dtype=float32)
>>> img[0,0]
array([0.11372549, 0.08235294, 0.10196079], dtype=float32)
>>> type(img[0,0])
<class 'numpy.ndarray'>
```

Exercice 6.1 modifications simples d'une image

Ici, double objectif : vous réfléchissez à la représentation matricielle ou par des tableaux de l'image. Vous êtes par ailleurs confrontés aux entiers modulo 256.

On dispose d'un tableau `ImgC` de taille $(p_x, p_y, 3)$ représentant une image RGB. C'est-à-dire que chaque pixel est représenté par un tableau d'entiers `uint8`, à une dimension et de taille 3 :

```
>>> ImgC[0,0,:]
array([21, 15, 19], dtype=uint8)
```

1. Écrire une fonction `nuance_gris(P)` qui prend en argument un pixel `P` (c'est-à-dire un tableau numpy de dimension 1 et contenant trois entiers de type `numpy.uint8` comme `ImgC[0,0]`) et renvoie une nuance de gris (également de type `numpy.uint8`) pour ce pixel.

À propos des couleurs et du gris

La Commission Internationale de l'Éclairage propose de caractériser l'information de luminance (la valeur de gris) d'un pixel par deux formules :

-couleurs naturelles :

$$\text{Gris} = 0,2125 \cdot \text{Rouge} + 0,7154 \cdot \text{Vert} + 0,0721 \cdot \text{Bleu}$$

- image vue à partir d'un écran vidéo :

$$\text{Gris} = 0,299 \cdot \text{Rouge} + 0,587 \cdot \text{Vert} + 0,114 \cdot \text{Bleu}$$

- On peut aussi prendre comme coefficients 1/3,1/3,1/3 !

2. Écrire une fonction `convertir_en_gris(img)` qui prend un tableau représentant une image en couleur en argument et retourne un tableau du même type représentant cette même image en niveaux de gris.

L'image en nuances de gris ci-dessous est obtenue à partir d'une image en couleurs avec la fonction du corrigé. Essayez avec vos propres photos.

3. Écrire une fonction qui prend un tableau matriciel représentant une image en niveaux de gris et retourne son négatif.



4. * Écrire une fonction `contraste(M, f)` qui prend en arguments un tableau matriciel `M` et une fonction `f`, de $[0, 1]$ (ou de $[0, 255]$) dans lui-même, et retourne une matrice de même taille que `M`, représentant la même image que `M`, plus ou moins contrastée selon `f`.

Vous avez deux types de problèmes à résoudre :

- Discuter de la forme du graphe de f pour tel ou tel résultat. Nous conseillons de traiter une fonction de $[0, 1]$ dans lui-même.

- Gérer la **vectorisation** de votre fonction. Numpy permet en effet de vectoriser, c'est-à-dire d'appliquer une fonction à un tableau ou une matrice et éviter les boucles. Évidemment cela marche avec les fonctions de numpy (`np.log`, `np.cos`, etc.).

Corrigé en 6.1 page 262.

Exercice 6.2 pixels et images

D'après le sujet Informatique, Centrale 2020 (MP, PC, PSI, TSI). Toutes les explications qui reprenaient les éléments du cours n'ont pas été reproduites. A consulter à l'adresse :

<https://www.concours-centrale-supelec.fr/CentraleSupélec/2020/Multi/sujets/I001.pdf>

1. I.A. - Pixels

- (a) On suppose que chacune des trois composantes RGB d'un pixel est représentée par un nombre entier positif ou nul, codé sur 8 bits. Combien de couleurs différentes peut-on représenter avec un tel pixel ?

Dans toute la suite le type pixel désigne un vecteur (tableau numpy à une dimension) d'entiers de type `np.uint8` à trois éléments.

- (b) Donner une instruction permettant de créer un vecteur correspondant à un pixel blanc.

Il est rappelé qu'en Python les opérations d'addition, soustraction, division entière, modulo, élévation à la puissance appliquées à deux opérateurs du même type fournissent un résultat du type de leurs opérandes. (...) L'opérateur division ($/$) entre deux entiers produit toujours un résultat à virgule flottante (...).

- (c) On pose `a = np.uint8(280)` et `b = np.uint8(240)`. Que valent `a`, `b`, `a+b`, `a-b`, `a/b` et `a/b` ?

Les fonctions numpy qui effectuent de manière répétitive des opérations élémentaires, si elles ne garantissent pas l'absence de dépassement de capacité, prennent la précaution d'utiliser pour leurs calculs intermédiaires et leur résultat un type compatible avec le type de base de la plus grande capacité possible. Par exemple le résultat de `np.sum(np.array([100, 200], np.uint8))` est de type `np.uint64` (entier non signé codé sur 64 bits) et vaut bien 300.

- (d) Pour convertir une image en couleurs en niveaux de gris, on peut remplacer chaque pixel par un seul entier, dont la valeur correspond à la meilleure approximation entière de la moyenne des trois composantes RGB du pixel.

Écrire une fonction d'en-tête `def gris(p : pixel) -> np.uint8` : qui calcule le niveau de gris correspondant au pixel `p`.

2. I.B. - Images

Une image en niveaux de gris de taille $w \times h$ (w pixels de large, h pixels de haut) est associée à un tableau d'octets (type `np.uint8`) à deux dimensions, à h lignes et w colonnes. Chaque élément de ce tableau représente le niveau de gris du pixel correspondant. Ainsi le tableau à deux dimensions `img1`, défini par :

```
img1 = np.array([[ 85,  0,  127, 170,  85,  150],
                 [119, 102, 102, 123,  81,  170],
                 [255, 170,  90,  112,  63,  97],
                 [171, 212, 225, 186, 162, 171]],
                 np.uint8)
```

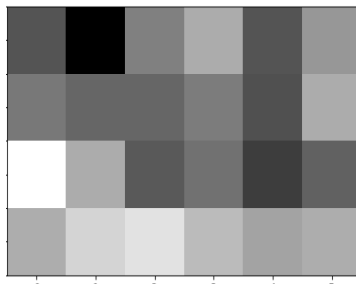
définit une image de taille 6×4 , représentée sur la figure.

Dans toute la suite, on utilise le type `image` pour désigner un tableau d'octets à deux dimensions.

Pour les images en couleurs, on ajoute une dimension au tableau pour représenter les trois composantes d'un pixel.

```
>>> import matplotlib.pyplot as plt
>>> source = plt.imread("surfer.jpg")
>>> source.shape
(3000, 4000, 3)
>>> source[0, 0]
np.array([144, 191, 221], np.uint8)
```

- Ci-dessus l'import d'une image (`surfer.jpg`). Interpréter les valeurs de `source.shape` et de `source[0,0]`.
- Écrire une fonction `convertir_en_gris(a : np.ndarray) -> image` : qui génère une image en niveaux de gris correspondant à la conversion de l'image en couleurs `a`.



6.2 Dessiner une droite à l'écran

Nous présentons dans cette section un des premiers algorithmes publiés dans le domaine de la représentation de droites à l'écran. La question est simple en apparence : on veut tracer un segment sur un écran ou avec une imprimante en gérant deux contraintes, à savoir que le résultat soit le plus proche possible de ce que notre cerveau interprète comme une droite et que la complexité algorithmique soit minimale. Comme un écran d'ordinateur est composé de pixels et que sa structure discrète en fait un réseau² et non pas un plan euclidien, certains problèmes vont se poser.

Cette problématique du passage du continu au discret reviendra dans tout ce chapitre.

- Dans ce qui suit on suppose que notre écran dispose de $p_x \times p_y$ pixels. Le pixel repéré par le couple d'entiers (x, y) est le carré unité $[x, x+1] \times [y, y+1]$ dont les quatre sommets sont les points de coordonnées (x, y) , $(x+1, y)$, $(x+1, y+1)$, $(x, y+1)$. Le pixel $(0, 0)$ se trouve donc dans un coin de l'écran, le pixel $(p_x - 1, p_y - 1)$ occupe le coin opposé.

Rappels de géométrie plane

On suppose le plan de l'image rapporté à un repère orthonormé $(O, \vec{i}, \vec{j})$ et on note (x_a, y_a) le couple des coordonnées d'un point A dans ce repère.

- La droite (AB) passant par deux points A et B distincts a pour équation

$$F(x, y) = (y_b - y_a)(x - x_a) - (x_b - x_a)(y - y_a) = 0 \quad (6.2.1)$$

- Lorsque la droite n'est pas parallèle à $(O, \vec{j})$, $(x_b - x_a) \neq 0$. On peut alors réécrire cette équation sous la forme réduite

$$y = \frac{y_b - y_a}{x_b - x_a}(x - x_a) + y_a = mx + c \quad (6.2.2)$$

et on appelle *pente* de la droite (AB) le quotient $m = \frac{y_b - y_a}{x_b - x_a}$.

- On suppose que la droite n'est pas parallèle à $(O, \vec{j})$: un point P d'abscisse $x_p = x_a + \Delta$ appartient à (AB) ssi $y_p = y_a + m\Delta$. C'est comme cela que l'on dessine sur une feuille quadrillée une droite passant par un point A et dont on connaît la pente : « on avance de Δ unités (carreaux ou cm) en abscisse et on monte (ou descend) de $m \times \Delta$ unités selon les ordonnées ! »
- Soit M un point du plan qui ne se trouve pas sur (AB) . M se trouve à gauche du segment $[A, B]$ lorsqu'on se déplace de A vers B si $F(x, y) < 0$ et à droite si $F(x, y) > 0$. En effet,

$$F(x, y) = \det(\overrightarrow{AM}, \overrightarrow{AB}) = AM \times AB \times \sin(\widehat{AM, AB}). \quad (6.2.3)$$

2. En géométrie, un réseau est un sous-groupe additif « discret » de $\mathbb{R}^n$. Un bon exemple est le sous-groupe $\mathbb{Z}^2 \subset \mathbb{R}^2$.

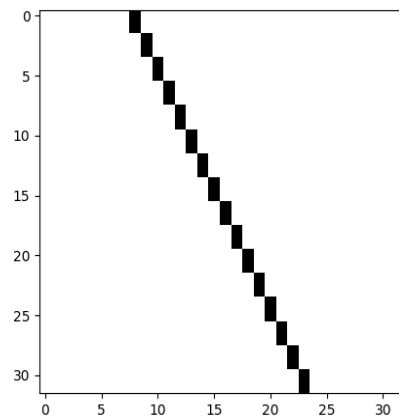
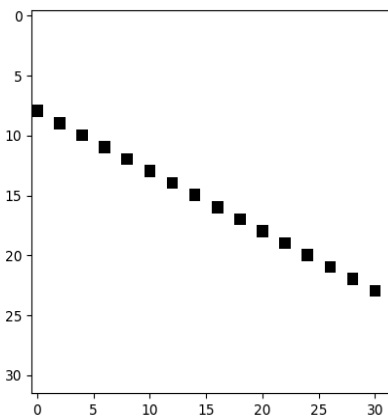
Exercice 6.3 Algorithme du point milieu

L'écran ou l'image est le tableau T réservé à partir de l'instruction

```
T = np.zeros((N, M), dtype = np.uint8) # écran noir
T = T - 1 # écran blanc car -1 = 255 modulo 256.
```

1. Algorithme naïf et algorithme incrémentiel

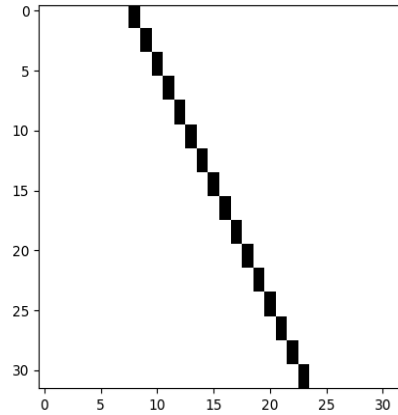
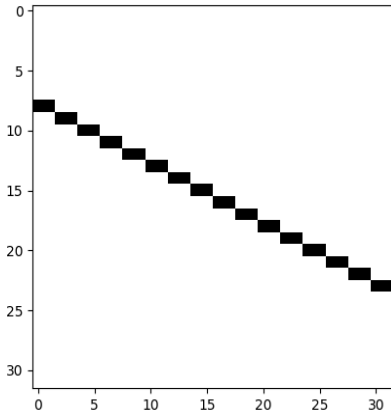
- (a) Écrire une procédure `dessiner_droite1( $T, x_a, y_a, m$ )` qui prend en arguments T , un tableau à 2 dimensions, trois flottants x_a, y_a et m et noircit, pour chaque valeur entière de x entre 0 et $p_x - 1$, le pixel $(x, \text{int}(m(x - x_a) + y_a))$ s'il se trouve dans l'écran. Cette procédure **modifie T en place** (elle peut renvoyer None ou T ainsi modifié).
- (b) Dénombrer les additions, multiplications et comparaisons de flottants de votre programme lors d'un appel avec $y_a = 0$ et $0 \leq m \leq 1$ par exemple.
- (c) Réécrire la procédure de telle sorte qu'il n'y ait plus de multiplication. On notera `dessiner_droite2( $T, x_a, y_a, m$ )` la procédure obtenue.



Deux droites et leurs représentations pixélisées (ici en 32×32 pixels). **Attention**, l'origine est le pixel haut gauche et l'axe des abscisses est l'axe vertical.

- (d) On observe que la qualité de la représentation n'est pas la même selon la valeur de la pente. Dans la figure de gauche $|m| > 1$, alors que dans celle de droite $|m| < 1$ (faire pivoter le livre pour vous en convaincre). Pour remédier à cela on dessinera un pixel par ordonnée si $|m| > 1$.

Écrire un procédure `dessiner_droite3( $T, x_a, y_a, m$ )` qui dessine un pixel proche de la droite pour toute valeur de x si $|m| \leq 1$, et pour toute valeur de y sinon.



Les mêmes droites avec dessiner_droites3().

2. ★ Algorithme du point milieu (Bresenham, 1962)

On se propose maintenant d'écrire un algorithme incrémentiel qui calcule chaque pixel à afficher en fonction du précédent en travaillant exclusivement avec des opérations sur les entiers qui sont, en effet, moins coûteuses.

On supposera que $0 \leq m \leq 1$ et que la droite D que l'on veut pixéliser a pour équation cartésienne

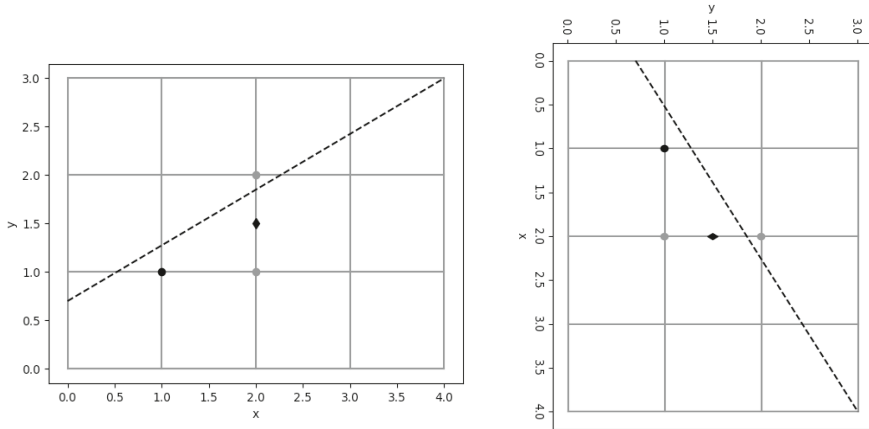
$$F(x, y) = \beta(x - x_a) - \alpha(y - y_a) = \begin{vmatrix} (x - x_a) & \alpha \\ (y - y_a) & \beta \end{vmatrix} = 0 \quad (6.2.4)$$

Pour chaque entier p tel que $0 \leq p = x_p < p_x$, abscisse d'un pixel de l'écran, on veut déterminer un unique pixel (x_p, y_p) appartenant à la représentation discrétisée de D . Observons que le point de D d'abscisse x_p est $(x_p, y_a + m(x_p - x_a))$, y_p sera donc une approximation entière de $y_a + m(x_p - x_a)$.

- Expliquer pourquoi on peut choisir $\alpha > 0$ et $\beta \geq 0$. C'est ce choix que nous ferons dans ce qui suit.
- On suppose que le pixel (x_p, y_p) appartenant à la droite discrétisée est connu. Quels sont les deux valeurs possibles du pixel de cette droite dont l'abscisse est $x_{p+1} = x_p + 1$?
- On note M_{p+1} le milieu de ces deux candidats, Q_{p+1} l'intersection de la droite D avec la verticale d'équation $(x = x_p + 1)$ et on pose

$$d_p = 2 F \left(x_p + 1, y_p + \frac{1}{2} \right).$$

Déterminer la position de M_{p+1} par rapport à Q_{p+1} (au-dessus, au-dessous ? à gauche, à droite ?) en fonction du signe de d_p .



L'algorithme du point milieu consiste à choisir :

$(x_{p+1}, y_{p+1}) = (x_p + 1, y_p + 1)$ si M_{p+1} est au-dessus/à gauche de Q_{p+1}

$(x_{p+1}, y_{p+1}) = (x_p + 1, y_p)$ si M_{p+1} est en-dessous/à droite de Q_{p+1}

- (d) Montrer que lorsque $d_p < 0$, $d_{p+1} - d_p = 2(\beta - \alpha)$ et que si $d_p > 0$, $d_{p+1} - d_p = 2\beta$.
- (e) Justifier que si le point (x_p, y_p) de la droite pixélisée est sur la droite D , alors $d_p = 2\beta - \alpha$ (utiliser la propriété de linéarité à gauche des déterminants 2×2).
- (f) Écrire une procédure `point_milieu(T, x_a, y_a, x_b, y_b, g)` qui implémente cet algorithme pour tracer le segment $[A, B]$ lorsque
- T est un tableau à deux dimensions $p_x \times p_y$ représentant une image (ou un écran) ; $T[0, 0]$ est de type `numpy.uint8` ;
 - A et B ont des coordonnées entières telles que $0 \leq x_a, x_b < p_x$ et $0 \leq y_a, y_b < p_y$;
 - la pente de $[A, B]$ vérifie $0 \leq m \leq 1$;
- et en ne s'autorisant, comme opérations, que l'addition et la soustraction.
- (g) Expliquer comment s'y prendre pour ne travailler qu'avec des additions ou soustractions *sur des entiers*. Calculer la complexité de cette procédure.

Corrigé en 6.3 page 264

6.3 Transformations géométriques d'une image

Nous abordons dans cette section quelques problèmes qui se présentent lorsqu'on veut appliquer des transformations géométriques à une image : l'agrandir, la réduire, la faire pivoter.

6.3.1 Agrandir : Homothétique d'une image avec $k > 1$

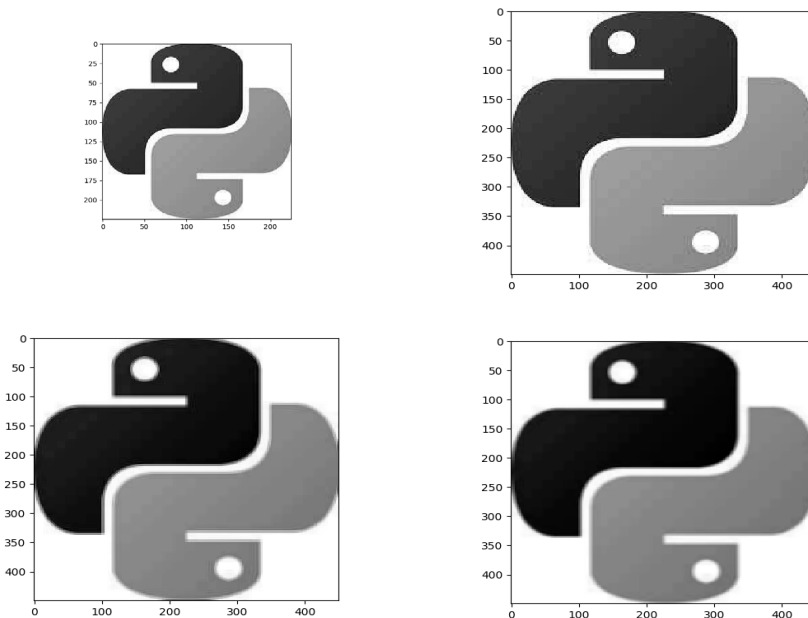
Du point de vue de la géométrie de $\mathbb{R}^2$, agrandir une figure du plan revient à construire son image par une homothétie de rapport $k > 1$.

Par contre, si nous disposons d'une image de $p_x \times p_y$ pixels que nous voulons agrandir pour en faire une image de $kp_x \times kp_y$ pixels avec $k > 1$, entier, l'idée de partir d'un point de l'image d'origine pour placer son homothétique dans l'image d'arrivée est clairement insuffisante : il y a $k^2 p_x p_y$ pixels dans la nouvelle image pour $p_x p_y$ dans l'image d'origine.

Comme dans tous les problèmes de transformation d'image, on procédera en réservant un tableau T_1 pour l'image d'arrivée et pour chacun des points ou pixels de T_1 , on déterminera le point ou pixel de l'image de départ **le plus proche de son antécédent géométrique**. A partir de là, plusieurs options se présentent :

- reporter la couleur de ce pixel dans le pixel de T_1 ;
- reporter une valeur moyenne de pixels avoisinants dans le pixel de T_1 .

La première option produit des images de qualité médiocre puisqu'en moyenne un pixel de l'ensemble de départ sera reproduit k fois à l'identique dans l'image d'arrivée, ce qui produira des effets d'escalier. La seconde option, pour laquelle il y a de nombreuses variantes, est plus coûteuse mais produit un meilleur lissage.



Figures : Le logo Python agrandi sans interpolation, puis avec une moyenne sur 5 points et sur 9 points.

Exercice 6.4 *agrandir une image*

L'écran ou l'image est le tableau T réservé à partir de l'instruction

```
T = np.zeros((N, M), dtype = np.uint8) # écran noir
T = T - 1 # écran blanc car -1 = 255 modulo 256.
```

1. Écrire une fonction `interpoler9(T, x, y)` qui prend en argument un tableau T représentant une image en nuance de gris, x et y deux entiers et renvoie la valeur moyenne du contenu de l'ensemble formé du pixel (x, y) et de ses voisins. On tiendra compte du cas où (x, y) est sur le bord de l'image.

Déterminer la complexité de votre fonction.

2. Écrire une fonction `agrandir(T, k)` qui prend en argument un tableau T représentant une image en nuance de gris (de taille $p_x \times p_y$), k un entier ou un flottant avec $k > 1$ et renvoie un tableau $T1$ de taille $\lfloor kp_x \rfloor \times \lfloor kp_y \rfloor$ agrandissant l'image $T1$ selon la méthode d'interpolation décrite ci-dessus.

Quelle est la complexité de votre fonction ?

Corrigé en 6.4 page 268.

Suivant le même principe on pourra étendre le nombre de pixels qui interviennent pour déterminer la coloration.

6.3.2 Mode d'emploi pour faire tourner une image

Nous nous proposons ici de regarder les problèmes qui se posent lorsqu'on veut faire pivoter une image³ et de proposer quelques solutions.

Le problème

Une image est affichée sur un écran et nous souhaitons la faire pivoter.

Du point de vue géométrique, nous savons transformer une figure en son image par une rotation de centre et d'angle donnés : on sait en effet transformer les points, les segments (les applications affines transforment les segments en segments), les cercles (les homothéties, les isométries et leurs composées transforment en effet les cercles en cercles), etc.

Mais les choses sont différentes sur un écran. Une image à l'écran est un assemblage de petits carrés colorés de façon homogène. Et lorsque nous transformons un tel carré par une rotation nous obtenons bien un autre carré mais il n'y a aucune raison qu'il coïncide avec un autre pixel de l'écran : si l'angle n'est pas un multiple de $\frac{\pi}{2}$ les cotés n'auront aucune des deux directions acceptables. La problématique est la même lorsque l'image est représentée dans un tableau.

Voyons cela de plus près.

3. Le titre est emprunté à l'article [11].

L'approche naïve

On considère une image de $p_x \times p_y$ pixels affichée à l'écran ou stockée dans un tableau T . On choisit un centre et un angle de rotation et on veut représenter la transformée de cette image par la rotation ainsi définie.

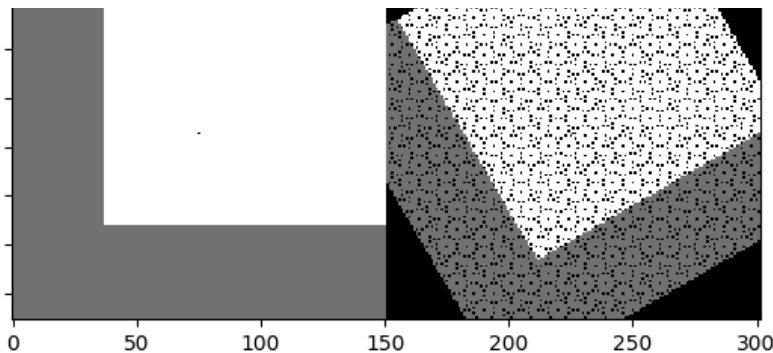
On note $T[i, j]$ la couleur du pixel d'indices (i, j) pour l'image d'origine et $T_1[i, j]$ la couleur du pixel pour l'image transformée. Rappelons que le pixel d'indices (i, j) est le carré unité $[i, i + 1] \times [j, j + 1]$ et non pas un point.

Une première idée est la suivante :

1. On calcule l'image X_1 du point X situé au centre du pixel par la rotation. Ce point est défini par la relation $X_1 = C + R(X - C)$ soit,

$$\begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} x_c \\ y_c \end{bmatrix} + \begin{bmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{bmatrix} \begin{bmatrix} x - x_c \\ y - y_c \end{bmatrix} \quad (6.3.1)$$

2. Si X_1 est un point de l'écran, qui est le rectangle $[0, p_x] \times [0, p_y]$, on recherche le pixel d'indices (i_1, j_1) le « plus proche » de X_1 et on éclaire ce pixel avec la couleur ou la nuance de gris qui est celle du pixel (i, j) dans l'image originale. Ce qui revient à faire $T_1[i_1, j_1] = T[i, j]$.
3. Si X_1 se trouve en dehors de l'écran, on ne fait rien.



Une image de 151×151 pixels et sa transformation par la méthode naïve.

Exercice 6.5 observations à partir d'une figure

La figure représente une image de 151×151 pixels en nuances de gris et sa transformée par la rotation de centre C , de coordonnées $(75, 75)$, et d'angle $\alpha = \frac{\pi}{6}$, avec l'approche naïve décrite ci-dessus. Le tableau T_1 est réservé à partir des instructions

```
N, M = T.shape
T1 = np.zeros((N, M), dtype = np.uint8).
```

On notera img l'image originale (qui occupe le carré de gauche) et img1 l'image contenue dans le tableau T_1 (qui occupe le carré de droite). Le point C correspond au coin haut-gauche du pixel en noir sur img .

1. Pourquoi le « L » grisé et le carré blanc ne se retrouvent-ils pas entièrement dans `img1` ?
2. Que représentent les triangles noirs aux 4 coins de `img1` ? Quels sont leurs angles ?
3. Où se trouve l'image du « L » grisé de l'image de gauche ? L'image du carré blanc ?
4. Comment interprétez-vous la présence de points noirs dans le « L » grisé de l'image de droite ? Et dans le carré blanc ?
5. La figure formée de ces points dépend-elle de l'image d'origine ?

Corrigé en 6.5 page 269.

Pour en savoir plus sur les figures formées par les pixels oubliés, qui pourraient nous mener très loin, on pourra lire l'article [11] paru dans *Images des mathématiques*.

Une tentative moins naïve

On adopte généralement la méthode suivante pour faire pivoter une image sans les jolis dessins parasites dus aux pixels oubliés.

1. Au lieu de partir d'un point de l'image d'origine pour calculer son transformé par la rotation, on choisit un point X_1 centre d'un pixel (i_1, j_1) de l'écran ou de l'image que l'on veut construire et on calcule son antécédent (qui est son transformé X par la rotation inverse).
2. Si X est un point situé dans le cadre de l'image d'origine, on cherche comme précédemment le pixel le plus « proche » de X , soit (i, j) .
3. On colorie alors le pixel (i_1, j_1)
 - soit en posant $T_1[i_1, j_1] = T[i, j]$. Mais cela pose un problème. Lequel ?
 - soit en prenant pour $T_1[i_1, j_1]$ la valeur moyenne des pixels avoisinants.

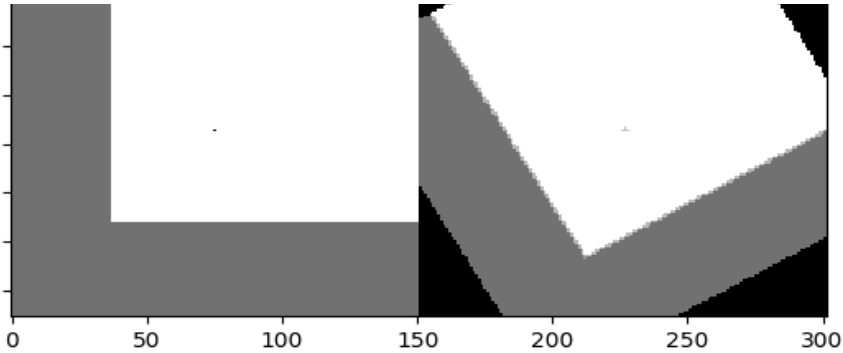
On est alors certain que tous les pixels de la nouvelle figure sont alimentés.
4. Si X est un point hors du cadre de l'image d'origine, on ne fait rien.

Exercice 6.6 *algorithme de rotation*

La figure qui suit représente une image de 151×151 pixels en nuances de gris et sa transformée par la rotation de centre C , de coordonnées $(75, 75)$, et d'angle $\alpha = \frac{\pi}{6}$, avec l'approche décrite ci-dessus. Le tableau T_1 est réservé à partir des instructions

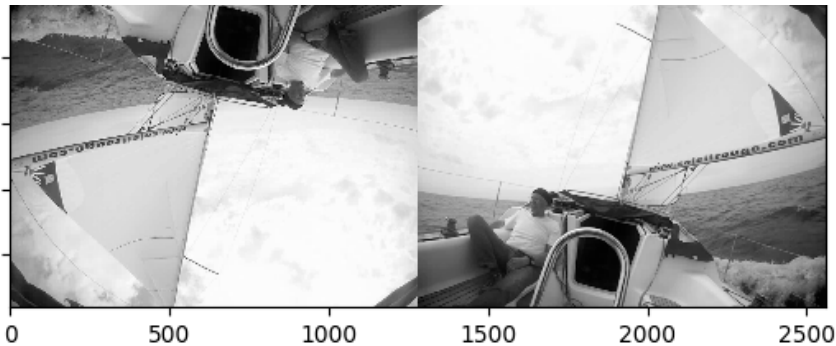
```
N, M = T.shape
T1 = np.zeros((N, 2*M), dtype = np.uint8).
```

On note *img* l'image originale (qui occupe le carré de gauche) et *img1* l'image obtenue par rotation (qui occupe le carré de droite). Le point *C* correspond au coin haut-gauche du pixel en noir sur *img*.



1. Pourquoi le « L » et le carré blanc ne se retrouvent-ils pas entièrement dans *img1* ?
2. Que représentent les triangles noirs aux 4 coins de *img1* ? Quels sont leurs angles ?
3. On entreprend ici l'implémentation de l'algorithme qui a permis de construire la figure.
 - (a) Écrire une fonction `couleur_interpolation(T, i, j, p)` qui prend en arguments une image (ou tableau) *T*, *i*, *j* et *p* des entiers ((i, j) représentant les indices d'un pixel de l'image) et qui renvoie la valeur moyenne du contenu des pixels de la fenêtre $[i - p, i + p] \times [j - p, j + p]$ dans l'image. On tiendra compte des bords de l'image.
 - (b) La formule (6.3.1) de la page 246 donne l'image X_1 d'un point *X* par la rotation de centre *C* et d'angle α . Exprimer de façon analogue *X* en fonction de X_1 .
 - (c) Écrire une fonction `rotation_interpolation(T, C, a)` qui prend en argument un tableau *T* à deux dimensions représentant une image en niveaux de gris, un tableau *C* à une dimension et de taille 2 représentant le centre de la rotation, un flottant *a*, angle de la rotation. Cette fonction renverra un tableau *T1* ayant le même nombre de lignes que *T* et le double de colonnes. La partie gauche de *T1* est égale à *T*, la partie droite contient la figure formée de la rotation de *T* dans le cadre imposé par la taille.

Corrigé en 6.6 page 271.



Au départ la caméra était à l'envers, une rotation de 180° rétablit cette image de 960×1280 pixels.

```
imgC = mpimg.imread(directory + 'Voilier.jpg')
imgG = convertir(imgC) # fonction du Pb. Centrale
N, M = imgG1.shape # 960, 1280
C = np.matrix([[N//2], [M//2]], dtype = int)
imgR = rotation_interpolation(imgG, C, np.pi)
plt.imshow(imgR, cmap='gray') # Tableau à 2 dimensions
plt.savefig(directory + 'RotationImageInterpoleePi.jpg')
plt.show()
```

6.4 Traitements par convolution

6.4.1 Filtres linéaires et convolution

Dans les études précédentes (agrandissements, rotations) nous avons été amenés à considérer le voisinage d'un pixel de l'image d'origine pour « alimenter » un pixel de l'image transformée. Les calculs de couleurs ou de niveaux de gris dépendaient en effet du contenu d'une « fenêtre » autour d'un pixel (comme dans la fonction `interpoler9()` de l'exercice (6.4) page 244 ou avec les méthodes de pivotement d'image décrites page 247).

Nous allons approfondir ces questions et nous commencerons par un exemple de signal à une dimension pour introduire quelques concepts.

Signaux à une dimension

Nous partons d'un problème **jouet**, loin de la complexité réelle du problème industriel, mais qui va nous permettre d'introduire les notions essentielles. On suppose que l'on dispose d'un relevé de fonds marins le long d'une route en vue de déposer un câble. Un sonar a produit un relevé des fonds dépendant continûment du temps qui,

par couplage avec le GPS de bord a permis de construire un relevé des hauteurs de fond à distances régulièrement espacées⁴.

On note $(z_n)_n$ la suite des profondeurs et $(x_n)_n$ la suite des distances le long de la route (on a donc $x_{n+1} - x_n = d$, constant). Comme il s'agit de relevés, le nombre de termes effectifs est fini et les suites seront en pratique indexées sur des intervalles finis de $\mathbb{Z}$.

Dans les définitions qui suivent on pourra considérer que ce sont des suites indexées sur $\mathbb{Z}$, prolongées à gauche et à droite avec des 0 et on ne se préoccupera de **ce qui se passe aux bords** que quand il s'agira de programmer ou de calculer des exemples particuliers.

- On sait qu'il peut y avoir des mesures aberrantes et une première idée (naïve) pour en atténuer le poids est de remplacer $(z_n)_n$ par une suite des valeurs moyennes autour de chaque position. On peut ainsi transformer $z = (z_n)_n$ en posant $T_3(z) = (v_n)_n$ avec

$$T_3(z)_n = v_n = \frac{1}{3}(z_{n-1} + z_n + z_{n+1})$$

Si on veut lisser sur une plus grande distance, on posera plutôt $T_5(z) = (w_n)_n$ avec

$$T_5(z)_n = w_n = \frac{1}{5}(z_{n-2} + z_{n-1} + z_n + z_{n+1} + z_{n+2})$$

- On ne peut pas poser de câble là où les pentes sont trop importantes et on veut donc calculer ces pentes. On peut donc construire une suite $P_1(z) = (p_n)_n$, en posant

$$P_1(z)_n = p_n = \frac{z_{n+1} - z_n}{d} \text{ ou } P_1(z)_n = p_n = z_{n+1} - z_n$$

ce qui nous donne la suite (naïve) des pentes ou une suite **proportionnelle** aux pentes (rappelons que $d = x_{n+1} - x_n$).

Remarquons que si $Z : x \in \mathbb{R} \rightarrow Z(x) \in \mathbb{R}$ est une fonction telle que $Z(x_n) = z_n$ et qui est par ailleurs de classe $\mathcal{C}^2$, on aura avec la formule de Taylor-Young :

$$\begin{cases} Z(x+d) = Z(x) + dZ'(x) + \frac{d^2}{2}Z''(x) + o(d^2) \\ Z(x-d) = Z(x) - dZ'(x) + \frac{d^2}{2}Z''(x) + o(d^2). \end{cases}$$

D'un point de vue pratique, trop peu souligné dans les cours de maths hors analyse numérique,

$$Z(x+d) - Z(x-d) = 2dZ'(x) + o(d^2)$$

ce qui nous donne, pour d suffisamment petit (plusieurs mètres quand même), une meilleure approximation de la dérivée ou de la pente en x .

4. C'est volontairement que nous évitons de parler du temps.

On posera alors $P_2(z) = q = (q_n)_n$ avec

$$P_2(z)_n = q_n = \frac{1}{2d}(z_{n+1} - z_{n-1}) \text{ ou } P_2(z)_n = q_n = (z_{n+1} - z_{n-1})$$

Dans le premier cas on a une approximation de la pente, dans le second une suite proportionnelle à cette approximation.

• Synthèse

1. Les opérateurs T_3, T_5, P_1, P_2 sont tous linéaires, ce qui signifie que si A est l'un d'entre eux, pour des suites $x = (x_n)_n, y = (y_n)_n$ et tout réel λ on a

$$A(x + y) = A(x) + A(y) \text{ et } A(\lambda x) = \lambda A(x).$$

2. Chacun de ces opérateurs commute avec l'opérateur de translation :

$$A(\mathcal{T}_{n_0}(z)) = \mathcal{T}_{n_0}(A(z)).$$

Si $n_0 \in \mathbb{Z}$, l'**opérateur de translation** $\mathcal{T}_{n_0}$ est l'opérateur (linéaire) qui, à une suite $(z_n)_{n \in \mathbb{Z}}$, associe la suite $(z'_n)_{n \in \mathbb{Z}}$ telle que $z'_n = z_{n-n_0}$ (z' est donc la suite obtenue en décalant le signal z de n_0 indices).

3. Pour chacun de ces opérateurs, il existe une suite $(c_n)_n$ telle que, en notant $A(z) = z'$, on a :

$$A(z)_n = z'_n = \sum_{i \in \mathbb{Z}} z_{n-i} c_i.$$

Exemple : On considère T_3 .

$$\text{On définit } c = (c_n)_{n \in \mathbb{Z}} \text{ en posant } \begin{cases} c_{-1} &= 1/3 \\ c_0 &= 1/3 \\ c_1 &= 1/3 \\ c_n &= 0 \text{ si } n \notin \{-1, 0, 1\} \end{cases}$$

Il est clair que $T_3(z)_n = v_n = \sum_{i \in \mathbb{Z}} z_{n-i} c_i = c_{-1} z_{n+1} + c_0 z_n + c_1 z_{n-1}$.

Exercice 6.7 noyaux 1D

Préciser les suites $(c_n)_n$ lorsque $A = T_5, P_1, P_2$.

Attention, vérifiez !

Corrigé en 6.7 page 273.

Définition 6.1 signaux discrets, filtres linéaires, convolution.

- On considère l'espace $\mathcal{S} = \mathbb{R}^{\mathbb{Z}}$ des suites de réels indexées sur $\mathbb{Z}$. C'est l'espace des **signaux discrets**.
- Une application linéaire A de $\mathcal{S}$ dans lui-même est un **filtre linéaire** si elle commute avec l'opérateur de translation.
- Une suite $(c_n)_{n \in \mathbb{Z}} \in \mathcal{S}$ est à **support fini** si tout ses termes sont nuls sauf un nombre fini.

- On appelle **produit de convolution** de $(z_n)_n \in \mathcal{S}$ par $(c_n)_n$ à support fini la suite $z * c$ définie par

$$(z * c)_n = \sum_{i \in \mathbb{Z}} z_{n-i} c_i.$$

- Soit A un filtre linéaire défini par une convolution : $A(z) = z * c$. La suite c est appelée le **noyau de convolution** de A (elle est en fait unique, d'où l'article défini).

Remarque : tout filtre linéaire peut être associé à un noyau de convolution à condition de définir le produit de convolution avec des suites qui ne sont pas nécessairement des suites à support fini ; ce qui nous conduirait à parler de séries convergentes. On laisse cela aux cours de maths de $L_i, i \geq 2$.

Signaux à deux dimensions, images

Nous avons consacré un peu plus de deux pages pour introduire les notions de filtre linéaire, de convolution et de noyau. Nous avons choisi pour cela un exemple joué dans lequel n'intervenait que la longueur (comme dimension au sens physique). Nous sommes prêts à généraliser ces notions en deux dimensions pour travailler avec les images.

Pour faciliter les définitions nous considérerons des suites doubles indexées sur $\mathbb{Z}^2$. On obtient facilement une telle suite en partant d'un tableau à deux dimensions indexé sur $[0, p_x - 1] \times [0, p_y - 1]$ et en l'étendant comme suite double en posant $T[i, j] = t_{i,j} = 0$ si (i, j) sort du cadre.

Définition 6.2 opérateurs de translation, filtres linéaires pour des suites doubles

- On note $\mathcal{S}_2 = \{(z_{i,j})_{(i,j) \in \mathbb{Z}^2}\}$ l'ensemble $\mathbb{R}^{\mathbb{Z}^2}$ des suites doubles indexées sur $\mathbb{Z}^2$. C'est l'espace des **signaux discrets à deux dimensions**.
- Pour tout couple $(n_0, n_1) \in \mathbb{Z}^2$, $\mathcal{T}_{n_0, n_1}$ désigne l'application linéaire de $\mathcal{S}_2$ dans lui-même qui à une suite double indexée sur $\mathbb{Z}^2$, $(z_{i,j})_{(i,j) \in \mathbb{Z}^2}$ associe $(z'_{i,j})_{(i,j) \in \mathbb{Z}^2}$ définie par $z'_{m,n} = z_{m-n_0, n-n_1}$.
On dit que $\mathcal{T}_{n_0, n_1}$ est l'**opérateur de translation** associé à (n_0, n_1) .
- Un **filtre linéaire** est une application linéaire de $\mathcal{S}_2$ dans lui-même qui commute avec les opérateurs de translation.

On généralise donc les notions de convolution et de noyau aux signaux discrets à deux dimensions, c'est-à-dire aux éléments de $\mathcal{S}_2$. Dans la foulée, avec l'exercice (6.8), dans un clin d'œil à M. Jourdain, nous montrons que ce sont des notions que nous avons déjà rencontrées.

Définition 6.3 filtre linéaires, convolution pour des suites doubles

On considère deux suites doubles indexées sur $\mathbb{Z}^2$: $(z_{i,j})_{(i,j) \in \mathbb{Z}^2}$ et $(c_{i,j})_{(i,j) \in \mathbb{Z}^2}$. On suppose que c est à support dans le rectangle (qui est aussi un carré par commodité)

$$[-d, d] \times [-d, d] = \{(i, j) \in \mathbb{Z}^2 / -d \leq i \leq d, -d \leq j \leq d\}.$$

On définit alors le **produit de convolution** $z * c$ de z par c en posant :

$$(z * c)_{n,m} = \sum_{i=-d}^d \sum_{j=-d}^d z_{n-i, m-j} c_{i,j}.$$

Lorsqu'un filtre linéaire $A : S_2 \rightarrow S_2$ est de la forme $A(z) = z * c$, on dit que c est le **noyau de convolution** de A .

Exercice 6.8 noyaux 2D déjà rencontrés

Comme variante de la fonction `interpoler9(T, x, y)` définie dans la question 1 de l'exercice 6.4 de la page 244 on définit la fonction `interpoler4(T, i, j)` qui prend une image T en argument ainsi que deux entiers x et y :

```
def interpoler4(T, x, y):
    '''
    T      : image;
    x, y   : int.
    '''
    assert isinstance(T, np.ndarray) and T.ndim == 2
    N, M   = T.shape
    n, g = 0, 0.

    V = [(-1,-1), (-1,1), (1, -1), (1,1)]
    for v in V:
        if 0 <= x + v[0] and x + v[0] < N \
            and 0 <= y + v[1] and y + v[1] < M:
            n += 1
            g += T[x + v[0], y + v[1]]

    return np.uint8(g/n)
```

1. Que vaut n à l'issue de la boucle lorsque le pixel (x, y) est un coin de l'image, un bord qui n'est pas un coin ou un pixel situé à l'intérieur de l'image ?
2. Montrer que lorsque $1 \leq x \leq p_x - 2$ et $1 \leq y \leq p_y - 2$, `interpoler4()` renvoie le terme d'indices (x, y) du produit de convolution de T (éventuellement étendu à $\mathbb{Z}^2$) par une suite double c dont on précisera les termes non nuls. Que serait le support de c dans le programme ?
3. Reprendre la fonction `interpoler9(T, x, y)` et montrer qu'elle réalise le calcul de $T * c[x, y]$ dès lors que (x, y) est un pixel intérieur à l'image stockée dans le tableau T .

Corrigé en 6.8 page 273.

Représentation d'un noyau par un tableau (ou une matrice)

On représente habituellement les noyaux de convolution par des tableaux (certains disent des matrices carrées) ayant $p \times p$ lignes et colonnes avec p impair. Si c a un support dans le carré $\llbracket -d, d \rrbracket \times \llbracket -d, d \rrbracket$, on pourra alors prendre $p = 2d + 1$.

Par exemple, le noyau c qui permet de remplacer un pixel par la valeur moyenne

des 8 pixels avoisinants sera représenté par $C = \begin{bmatrix} 1/8 & 1/8 & 1/8 \\ 1/8 & 0 & 1/8 \\ 1/8 & 1/8 & 1/8 \end{bmatrix}$ et le produit de

convolution $T * c[x, y]$ sera calculé en lisant dans ce tableau.

Ci-dessous une façon de le définir avec un tableau de numpy :

```
C = np.ones((3,3), dtype = np.uint8)/8
C[1,1] = 0

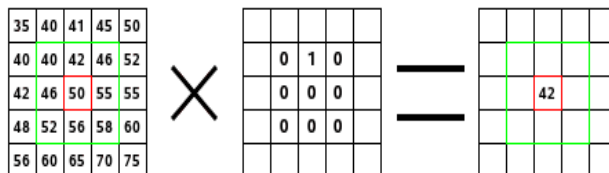
>>> C
[[0.125 0.125 0.125]
 [0.125 0.    0.125]
 [0.125 0.125 0.125]]
```

Avertissement : nous avons choisi de représenter un noyau c par une matrice carrée de dimensions impaires $(2d+1)$ de terme central $C[d, d] = c_{0,0}$ et où $C[d+i, d+j] = c_{i,j}$ pour mieux « coller » à la définition des suites doubles. Dans la littérature les représentations sont souvent symétriques (par symétrie centrale) de la notre. Avec le choix que nous avons fait l'implémentation de la convolution (exercice 6.9) reste plus proche de la définition. Les représentations sont identiques quand les noyaux sont symétriques.



GIMP, par exemple, propose de choisir les filtres en remplissant un tableau à l'aide du menu qui figure à gauche : l'utilisateur définit le noyau de convolution c et le filtre est l'opérateur $T \rightarrow T * c$.

La documentation (voir la page en [10]) propose un exemple pour illustrer la façon dont opère le filtre :



Exercice 6.9 *transformée d'une image par un filtre de convolution*

On se propose d'implémenter le produit de convolution d'une image par un noyau (le filtrage de l'image donc). Le noyau C est représenté par un tableau de taille $(2d + 1) \times (2d + 1)$ et l'image est stockée dans un tableau T de taille $p_x \times p_y$.

```
>>> T = mpimg.imread(directory + 'VoillierNb.jpg')
>>> type(T), T.ndim, T.shape
<class 'numpy.ndarray'> 2 (960, 1280)
```

1. Écrire une fonction `convolution_xy(T, C, d, x, y)` qui prend en arguments un tableau T représentant une image, un tableau C représentant un noyau de convolution et des entiers d, x, y avec les contraintes :
 - C est un tableau carré ayant un nombre impair de lignes et de colonnes ;
 - $d = \frac{p-1}{2}$;
 - la fenêtre de même taille que C , de centre (x, y) est contenue dans T .
 Cette fonction renvoie la valeur $T * C[x, y]$ où $T * C$ est le produit de convolution de T et de C , ou du moins de leurs prolongements à $\mathbb{Z}^2$.
2. Écrire une fonction `convolution(T, C)` qui renvoie le tableau $T1$ dont la partie intérieure $T1[d : p_x - d, d : p_y - d]$ coïncide avec le produit de convolution $T * C$.
3. Modifier cette fonction pour qu'elle renvoie $T1$ dans lequel les pixels du bord sont calculés avec une des méthodes suivantes :
 - On prolonge (virtuellement) T par symétrie par rapport aux bords : $T[-1, y] = T[1, y]$, $T[-d, y] = T[d, y]$, etc.
 - On enroule le tableau : $T[-1, y] = T[p_x - 1, y]$ par exemple ;
 - On tronque C (et on renormalise) ;
 - On garde le pixel de T ...

Corrigé en 6.9 page 273

6.4.2 Quelques effets du filtrage linéaire

Nous présentons ici les effets sur l'image de quelques filtres linéaires définis à partir de produits de convolutions avec des noyaux dont le support est en général de petite taille : 3×3 ou 5×5 .

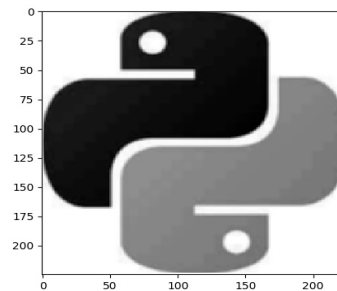
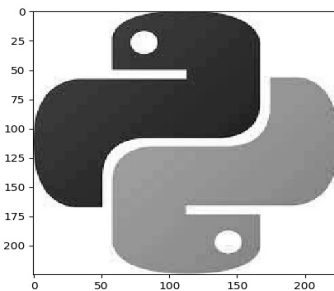
• **Lissage, floutage** : transformer une image avec un noyau uniforme et normalisé (dont la somme des termes est égale à 1) produit un lissage de l'image, c'est-à-dire atténue les variations abruptes et entraîne un floutage des contours.

C'est le cas avec le produit de convolution $T * c$ où $c_{i,j} = \frac{1}{9}$ lorsque $-1 \leq i, j \leq 1$ et $c_{i,j} = 0$ partout ailleurs. Ce filtre remplace chaque pixel par la valeur moyenne des pixels avoisinants. C peut être représenté par le tableau 3×3 ci-contre.

$$\begin{bmatrix} \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \end{bmatrix}$$

Un code faisant appel aux fonctions `convertir_en_gris1()` de l'exercice 6.2 et `convolution()` de l'exercice 6.9 qui permet de lisser et flouter le logo Python.

```
T = mpimg.imread(directory + 'LogoPython.jpeg')
T = convertir_en_gris1(T) # pour un tableau de dim 2
C = np.matrix([[1,1,1],[1, 1, 1],[1,1,1]], dtype = int)/9
T1 = convolution(T, C)
```



• Accentuer les contrastes

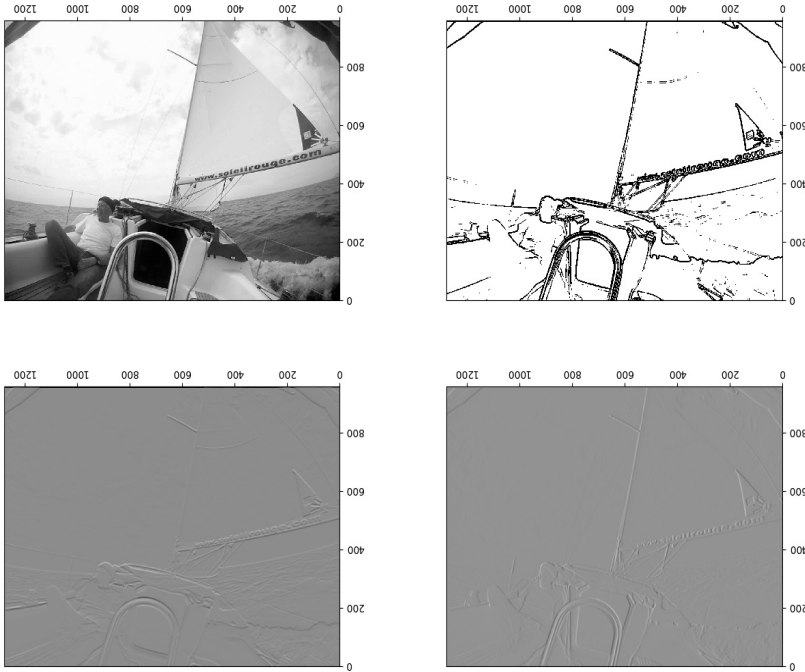
On peut à l'inverse accentuer les contrastes d'une image. C'est le cas avec le produit de convolution $T * c$ où C peut être représenté par le tableau 3×3 ci-contre.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

6.4.3 Détection de contours

Détection de contours, une première approche

On souhaite déterminer les lignes de contour dans une image. Une première approche, naïve, consiste à calculer le *gradient d'intensité lumineuse* entre un pixel et chacun de ses voisins immédiats. Si la norme de ce gradient dépasse un certain seuil, le pixel est noirci dans une copie de l'image, sinon il est laissé en blanc. Dans les images qui suivent, obtenues avec le programme de l'exercice 6.11, les lignes de contours sont détectées en relevant respectivement les gradients nord-sud et ouest-est de l'image.



Dérivées partielles et convolution

Nous avons vu page 250 comment définir la pente ou la dérivée discrète d'un signal à une dimension et, avec l'exercice 6.7, comment la calculer à l'aide d'un noyau de convolution. On procède de la même façon en deux dimensions. Faisons pour cela un aller-retour entre le continu et le discret⁵ : On considère une fonction Z des deux variables réelles x et y et on suppose que les fonctions partielles $x \rightarrow Z(x, y)$ et $y \rightarrow Z(x, y)$ sont « suffisamment régulières » (par exemple de classe C^2) pour que nous puissions écrire :

$$\begin{cases} Z(x+d, y) = Z(x, y) + d \frac{\partial}{\partial x} Z(x, y) + \frac{d^2}{2} \frac{\partial^2}{\partial x^2} Z(x, y) + o(d^2) \\ Z(x-d, y) = Z(x, y) - d \frac{\partial}{\partial x} Z(x, y) + \frac{d^2}{2} \frac{\partial^2}{\partial x^2} Z(x, y) + o(d^2). \end{cases}$$

$$\begin{cases} Z(x, y+d) = Z(x, y) + d \frac{\partial}{\partial y} Z(x, y) + \frac{d^2}{2} \frac{\partial^2}{\partial y^2} Z(x, y) + o(d^2) \\ Z(x, y-d) = Z(x, y) - d \frac{\partial}{\partial y} Z(x, y) + \frac{d^2}{2} \frac{\partial^2}{\partial y^2} Z(x, y) + o(d^2). \end{cases}$$

5. Plus par commodité, parce que vous êtes habitués au continu, plutôt que par raison scientifique.

On obtient des approximations de $\frac{\partial}{\partial x} Z$, (ou de façon analogue, de $\frac{\partial}{\partial y} Z$) avec

$$\frac{\partial}{\partial x} Z(x, y) \approx \frac{1}{d}(Z(x + d, y) - Z(x, y)) \quad (6.4.1)$$

$$\frac{\partial}{\partial x} Z(x, y) \approx \frac{1}{2d}(Z(x + d, y) - Z(x - d, y)) \quad (6.4.2)$$

Lorsque Z qui peut par exemple représenter une image continue (c'est, rassurez vous, une vue de l'esprit) est discrétisée/pixélisée sous la forme d'un tableau T , ces relations deviennent :

$$\frac{\partial}{\partial x} Z(x, y) \approx \frac{1}{d}(T[x + 1, y] - T[x, y]) \quad (6.4.3)$$

$$\frac{\partial}{\partial x} Z(x, y) \approx \frac{1}{2d}(T[x + 1, y] - T[x - 1, y]) \quad (6.4.4)$$

et il est clair que les pentes selon un axe sont obtenues, à un facteur de proportionnalité près, en calculant les produits de convolutions $T * c_1$ et $T * c_2$ où les noyaux c_1, c_2 sont représentés par les tableaux :

$$C_1 = \begin{bmatrix} 0 & 1 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \text{ et } C_2 = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix}$$

De la même façon les **noyaux transposés** donneront les pentes selon l'autre axe :

$$D_1 = \begin{bmatrix} 0 & 0 & 0 \\ 1 & -1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \text{ et } D_2 = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & -1 \\ 0 & 0 & 0 \end{bmatrix}$$

Avertissements

1. On rappelle que dans la représentation pixélisée de l'image, l'axe du premier indice est vertical et dirigé vers le bas, l'axe du second indice est horizontal et dirigé vers la droite.
2. Le choix que nous avons fait pour représenter un noyau (explicité dans l'avertissement détaillé de la page 254) prend ici son importance car les noyaux ne sont pas symétriques.

Ainsi $C2[d + 1, d] = C2[2, 1] = c_{2,1,0}$ est bien le coefficient de $T[x - 1, y - 0]$ dans le produit de convolution :

$$(T * c)_{x,y} = \sum_{i=-d}^d \sum_{j=-d}^d T_{x-i, y-j} c_{i,j}.$$

Gradient, norme du gradient et contour

Définition 6.4 gradient dans le monde continu

Soit Z une fonction de deux variables réelles à valeurs dans $\mathbb{R}$, admettant des dérivées partielles continues, le gradient de Z en (x, y) est le vecteur

$$\begin{bmatrix} \frac{\partial}{\partial x} Z(x, y) \\ \frac{\partial}{\partial y} Z(x, y) \end{bmatrix}.$$

Dans l'étude d'une image pixélisée, T ce sont en fait les écarts entre l'origine d'un pixel et celles des pixels voisins qui nous intéressent (l'origine est située en haut à gauche, au nord-ouest). Ces écarts sont approchés par (ou approchent)

$$d \times \left(\cos \alpha \frac{\partial}{\partial x} Z(x, y) + \sin \alpha \frac{\partial}{\partial y} Z(x, y) \right) \text{ avec } \alpha = \frac{k\pi}{2} \quad (6.4.5)$$

pour les 4 points cardinaux. Comme nous l'avons vu, ces écarts (ou des valeurs proportionnelles) sont obtenus en appliquant un filtre de convolution à l'image ou au tableau T .

Noyau	Direction de l'écart mesuré
$C_2 = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix}$	sud : $T[x + 1, y] - T[x - 1, y]$
$D_2 = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & -1 \\ 0 & 0 & 0 \end{bmatrix}$	est : $T[x, y + 1] - T[x, y - 1]$
$G_x = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$	sud : moyenne sur trois points $x - 1, x, x + 1$.
$G_y = {}^tG_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$	est : moyenne sur trois points $y - 1, y, y + 1$.

Remarque : Lorsque $\alpha = \frac{\pi}{4} + \frac{k\pi}{2}$, ce qui correspond aux directions intermédiaires de la rose des vents, on remplacera d par $\sqrt{2}d$ dans la formule 6.4.5.

Exercice 6.10 filtre de Sobel ; justification des assertions du tableau

- Justifier que $T * c + T * c'$ est le même signal (ou tableau ou image) que $T * (c + c')$.

- Que contient $T * c$ avec $c = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ -1 & 0 & 0 \end{bmatrix}$?

3. Que contient $T * c$ avec $c = \begin{bmatrix} 0 & 2 & 0 \\ 0 & 0 & 0 \\ 0 & -2 & 0 \end{bmatrix}$?

4. Que contient $T * G_x$ avec $G_x = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$?

Que pensez-vous de l'assertion : $T * G_x[i, j]$ contient une approximation de $4 \times d \times \frac{\partial}{\partial x} Z(x_i, y_j)$ (d est ici le côté d'un pixel, par exemple $3 \times 10^{-4} m$ sur mon écran) ?

Corrigé en 6.10 page 274

Définition 6.5 filtres de Sobel

Les filtres de Sobel sont les filtres $T \rightarrow T * G_x$ et $T \rightarrow T * G_y$ associés aux noyaux

$$G_x = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} \text{ et } G_y = {}^t G_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}.$$

Ils renvoient des approximations des composantes du gradient ou des dérivées partielles (respectivement $4 \times d \times \frac{\partial}{\partial x} Z(x_i, y_j)$ et $4 \times d \times \frac{\partial}{\partial y} Z(x_i, y_j)$) lissées par pondération sur 3 pixels alignés selon l'un des axes.

Utilisation des filtres de Sobel pour la détection de contours

On souhaite déterminer les lignes de contour dans une image. L'idée est d'approcher les composantes sud et est du gradient à l'aide des filtres de Sobel. Cela fait on construit une image ne reportant que les pixels pour lesquels la norme de ce gradient dépasse un certain seuil.

Exercice 6.11 gradients, convolutions et contours avec les filtres de Sobel

On suppose que les images sont définies dans des tableaux à deux dimensions en niveaux de gris compris entre 0 et 255. Si vous souhaitez travailler avec des images RGB, il sera facile d'adapter ensuite vos fonctions pour qu'elles travaillent sur des tableaux en trois dimension et de taille $p_x \times p_y \times 3$ (au lieu de $p_x \times p_y$).

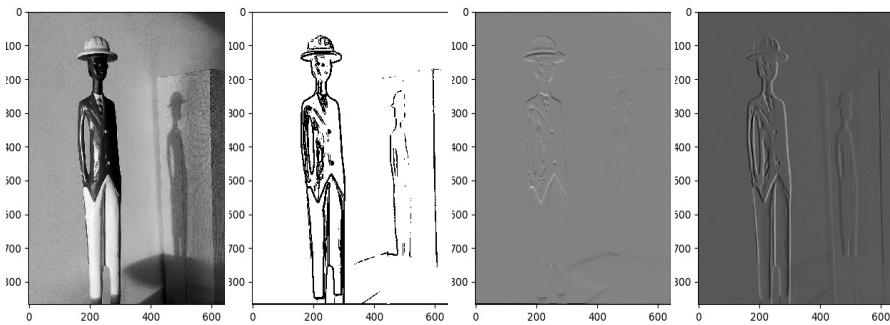
1. Définir les matrices C_lissage, Sobel_x et Sobel_y associées aux noyaux définis page 255 pour le lissage et en 6.5 pour les filtres de Sobel.
2. Pour relever les points en lesquels le gradient est élevé nous serons amenés à calculer une norme du gradient. Il est souvent conseillé d'utiliser pour cela une

des deux normes $\|G\|_\infty = \max(|G_x|, |G_y|)$ ou $\|G\|_1 = |G_x| + |G_y|$. Pourquoi pas la norme euclidienne de $\mathbb{R}^2$?

3. Écrire une fonction `contours_Sobel(T, s)` qui prend en arguments une image T et un seuil s avec s entier et $0 \leq s \leq 255$ et qui renvoie un tableau $T1$ de même taille que T en noir et blanc représentant les contours de l'image T .

Indications

- Utiliser la fonction `convolution(T, C)` définie après la page 273 dans le corrigé de l'exercice 6.9. On prendra garde à ce que les calculs dans `convolution_xy(T, C, x, y)` soient **effectués avec des entiers** (type `int`) plutôt que modulo 256.
- On floutera l'image T avec le filtre associé au noyau `C_lissage`, ce qui revient à calculer $T * C_{\text{lissage}}$.
- On construira les images $G_X = T * G_x$ et $G_Y = T * G_y$ (on pourra utiliser, pour calculer ces trois produits de convolution, la fonction `convolution(T, C)` du même exercice 6.9).
- On pourra alors construire l'image en noir et blanc des contours.



De gauche à droite, l'image d'origine, les contours avec la fonction du corrigé et $s = 60$, les images obtenues avec les noyaux de Sobel G_x et G_y .

4. Suggestions

- (a) Modifier les fonctions `convolution_xy(T, C, x, y)` et `convolution(T, C)` pour qu'elles prennent en compte des images RGB.
- (b) Modifier la fonction `convolution(T, C)` pour qu'elle prenne en compte le bord des images (enrouler, symétriser...).
- (c) On pourrait aussi tenir compte des gradients dans les directions obliques.

Corrigé en 6.11 page 274.

6.5 Corrigés des exercices

Corrigé de l'exercice n° 6.1

1.

```
def nuance_gris1(P):
    '''
    P : numpy.ndarray, [r,g,b] numpy.uint8;
    '''
    g = np.uint8(0.2125*P[0]) +...
    return g
```

2. On pense naturellement à construire terme à terme la matrice dans une double boucle :

```
def convertir_en_gris(img):
    '''
    img : numpy.ndarray (N x M x 3) avec
          img[i,j] : numpy.ndarray dim 1
          et taille 3 d'entiers uint8.
    '''
    assert isinstance(img, np.ndarray) \
        and img.ndim == 3
    N, M, R = img.shape
    imgG = np.zeros((N, M, R), dtype = np.uint8)
    # on réserve un tableau de même dim. et taille
    for n in range(0, N):
        for m in range(0, M):
            g = nuance_gris1(img[n,m])
            imgG[n, m, :] = [g,g,g]
    return imgG
```

Remarque : on peut aussi construire un tableau à deux dimensions qui sera lu avec l'instruction

`img = mpimg.imread('* .png', cmap='gray')`. Pour cela aller voir le corrigé de l'exercice 6.2 (sujet Centrale) page 264 .

3. On peut penser que le noir étant codé par 0 et le blanc par 255, faire

$-g \equiv 256 - g[256]$ permet d'échanger 0 et 255, 122 et 123 etc.

retourne une matrice dont les termes sont des entiers compris entre 0 et 255 qui représente le négatif. C'est bien avec le code `-imgG` que nous avons obtenu le négatif qui figure avec l'énoncé.

Au fait, savez vous encore ce qu'est un négatif d'image à l'heure du tout numérique ? A défaut demandez à vos (grands-)parents.

4. On accentuera le contraste avec une fonction en forme de « S » qui écrase les valeurs inférieures à 1/2 et rapproche de 1 les valeurs supérieures à 1/2, comme celle correspondant au graphe ci-contre, construite à l'aide de $\arctan$ (voir le code).



```
def contraste(img):
    '''
    img : numpy.ndarray (N x M x 3)
    On applique f(x) = atan(20x-10)/pi + 1/2
    '''
    return np.arctan(20*img/255-10)/np.pi + 1/2
```

Le contraste serait, au contraire, diminué avec une fonction s'éloignant rapidement de 0, s'approchant de 1 (pentes importantes en 0 et 1, faible en 1/2). On la construirait alors avec $\tan$ plutôt qu'avec $\arctan$.

Corrigé de l'exercice n° 6.2

1. Partie A

- Pour chaque composante il y a $2^8 = 256$ possibilités soit pour un pixel avec trois composantes $(2^8)^3 = 2^{24}$ couleurs possibles.
- La réponse en première ligne comme dans de l'informatique avec un stylo (le jour du concours). La suite pour éclaircir les idées.

```
>>> P = np.array( [255,255,255] , dtype=np.uint8)
>>> P
array([255, 255, 255], dtype=uint8)
>>> P.shape
(3,)
>>> P.ndim; P[0]
1, 255
```

- On aura $a = 280 \equiv 24[256]$ et $b = 240[256]$.
Donc $a + b \equiv 264 \equiv 8[256]$, $a - b \equiv -216 \equiv 40[256]$.
Par ailleurs $a/b = 24/240 = 0$, (de type `np.uint8`), alors que $a/b = 24/240 = 0.1$ est un flottant.
- On convertit chaque composante en flottant avant de reconverter la somme en `np.uint8`.

```
def gris(P):
    '''
    P : numpy.ndarray, [r,g,b] numpy.uint8;
    '''
    g = np.uint8(P[0]/3) + np.uint8(P[1]/3)\
        + np.uint8(P[2]/3)
    return g
```

2. Partie B

- (a) `source.shape` nous indique que `source` est un tableau à 3 dimensions qui représente une image de 3000×4000 pixels et qui est en couleurs (la troisième dimension possède 3 hyper-colonnes pour R, G et B).

`source[0,0]` est le pixel haut gauche, avec les niveaux 144 pour le rouge etc.

- (b) On crée ici un tableau à deux dimensions au lieu de trois.

Remarque : pour lire un tableau à deux dimensions comme une image en niveaux de gris, c'est

`img = mpimg.imread('*.png', cmap='gray')`.

```
def conversion(a):
    assert isinstance(a, np.ndarray) and a.ndim==3

    N, M, R = a.shape
    imgG = np.zeros((N, M), dtype = np.uint8)

    for n in range(0, N):
        for m in range(0, M):
            imgG[n, m] = gris(a[n, m])
    return imgG
```

Corrigé de l'exercice n° 6.3

1. Algorithme naïf et algorithme incrémentiel

- (a) `T` est modifié en place comme tout tableau, liste ou dictionnaire. On ajoute une option pour le niveau de gris (noir = 0 par défaut).

```
def dessiner_droite1(T, xa, ya, m, g = 0):
    assert isinstance(T, np.ndarray) and T.ndim == 2
    assert isinstance(xa, int) and isinstance(ya, int)
    px, py = T.shape
    for x in range(0, px):
        y = int(m*(x - xa) + ya)
        if 0 <= y and y < py:
            T[x, y] = g
    return T
```

- (b) Les opérations sont dans la boucle for : il y a p_x opérations de chaque. La plus coûteuse est, on le sait la multiplication.
- (c) On s'inspire des rappels de géométrie : dans le sens des x croissants, une incrémentation $x += 1$ conduit à une incrémentation $y += m$ et $x -= 1$, conduit à $y -= m$.

```
def dessiner_droite2(T, xa, ya, m, g=0):

    assert isinstance(T, np.ndarray) and T.ndim == 2
    assert isinstance(xa, int) and isinstance(ya, int)
    px, py = T.shape
    y = ya
    for x in range(xa, px):
        j = int(y)
        if 0 <= j and j < py:
            T[x, j] = g
        y += m
    y = ya-m
    for x in range(xa-1, -1, -1):
        j = int(y)
        if 0 <= j and j < py:
            T[x, j] = g
        y -= m
    return T
```

- (d) Si $|m| \leq 1$ on appelle `dessiner_droite2()`, sinon on reprend le code en remplaçant m par $1/m$ et en inversant les rôles de x et de y .


```

def dessiner_droite3(T, xa, ya, m, g=0):

    assert isinstance(T, np.ndarray) and T.ndim == 2
    assert isinstance(xa, int) and isinstance(ya, int)
    px, py = T.shape

    if abs(m) <= 1:
        return dessiner_droite2(T, xa, ya, m, g)

    else:
        x = xa
        for y in range(ya, py):
            i = int(x)
            if 0 <= i and i < px:
                T[i, y] = g
                x += 1/m

        x = xa - 1/m
        for y in range(ya-1, -1, -1):
            i = int(x)
            if 0 <= i and i < px:
                T[i, y] = g
                x -= 1/m

    return T

```

2. Algorithme du point milieu

- (a) Le vecteur (α, β) est un vecteur directeur de la droite dont la pente est alors $m = \frac{\beta}{\alpha}$ lorsque $\alpha \neq 0$, (par convention $+\infty$ lorsque $\alpha = 0$, cas que nous n'avons pas envisagé). Nous nous sommes placés dans le cas $0 \leq m \leq 1$ et α et β sont du même signe. Quitte à remplacer le vecteur directeur par son opposé, on aura bien $\alpha > 0, \beta \geq 0$. On peut d'ailleurs choisir $(\alpha, \beta) = (1, m)$.

- (b) Les points d'abscisses respectives x_p, x_{p+1} de la droite D que l'on veut représenter sont $\begin{bmatrix} x_p \\ y_a + m(x_p - x_a) \end{bmatrix}$ et $\begin{bmatrix} x_p + 1 \\ y_a + m(x_p - x_a) + m \end{bmatrix}$. On choisit évidemment pour y_p et y_{p+1} les meilleures approximations entières de $y_a + m(x_p - x_a)$ et $y_a + m(x_p - x_a) + m$. Comme $m \in [0, 1]$, on aura $y_{p+1} = y_p$ ou $y_{p+1} = y_p + 1$.

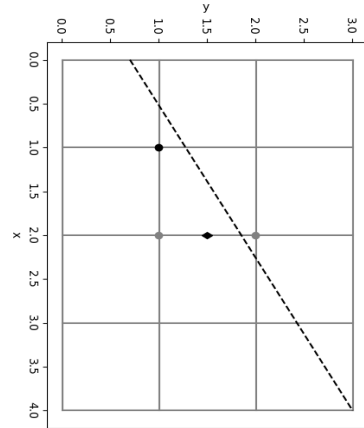
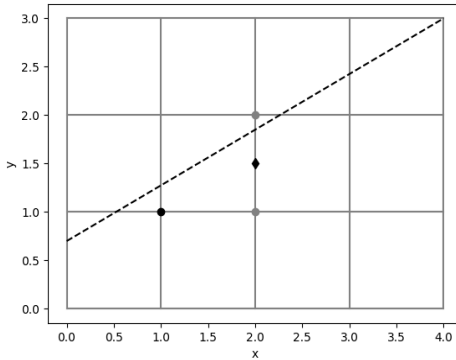
$$\text{Ainsi } \begin{cases} (x_{p+1}, y_{p+1}) = (x_{p+1}, y_p) = (p+1, y_p) \\ \text{ou} \\ (x_{p+1}, y_{p+1}) = (x_{p+1}, y_p + 1) = (p+1, y_p + 1) \end{cases}$$

- (c) Sur la figure M_{p+1} est le point marqué $\blacklozenge$. C'est le point de coordonnées $(x_p + 1, y_p + 1/2)$.

Donc, $d_p = 2F(x_p + 1, y_p + 1/2) = 2\det(\overrightarrow{AM_{p+1}}, \vec{u})$.

M_{p+1} est situé à droite de D quand on se dirige selon les abscisses croissantes puisque le vecteur directeur choisi a pour coordonnées (α, β) avec $\alpha > 0$ (les deux figures sont identiques, à droite la disposition de l'origine et des axes est celle d'un écran). Dans ce cas $F(M_{p+1}) > 0$.

D'une façon générale M_{p+1} est situé à droite ssi $F(M_{p+1}) > 0$ ssi $d_p > 0$.



(d) Premier cas ($d_p < 0$) : $d_{p+1} - d_p = 2 \begin{vmatrix} *+1 & \alpha \\ *+1 & \beta \end{vmatrix} - 2 \begin{vmatrix} * & \alpha \\ * & \beta \end{vmatrix} = 2(\beta - \alpha)$.

Second cas ($d_p > 0$) : $d_{p+1} - d_p = 2 \begin{vmatrix} *+1 & \alpha \\ * & \beta \end{vmatrix} - 2 \begin{vmatrix} * & \alpha \\ * & \beta \end{vmatrix} = 2\beta$

(e) On suppose que le point pixelisé (x_o, y_p) est sur la droite. Alors

$$d_p = 2 \begin{vmatrix} x_p + 1 & \alpha \\ y_p + 1/2 & \beta \end{vmatrix} = 2 \begin{vmatrix} x_p & \alpha \\ y_p & \beta \end{vmatrix} + 2 \begin{vmatrix} 1 & \alpha \\ 1/2 & \beta \end{vmatrix} = 0 + (2\beta - \alpha)$$

(f) Résumons : $d_p = 2 \begin{vmatrix} x_p + 1 & \alpha \\ y_p + 1/2 & \beta \end{vmatrix}$,

- si $d_p < 0$, on choisit $(x_{p+1}, y_{p+1}) = (x_p + 1, y_p + 1)$;

- si $d_p \geq 0$, on choisit $(x_{p+1}, y_{p+1}) = (x_p + 1, y_p)$;

Le code est page 267.

(g) La droite (AB) est la droite passant par A et de vecteur directeur $(\alpha, \beta) = (x_b - x_a, y_b - y_a)$. On peut encore choisir comme vecteur directeur $(1, m)$.

On approche donc m par un quotient $\frac{p}{q}$ et on choisit comme vecteur directeur $(\alpha, \beta) = (q, p)$. On ne travaille alors plus qu'avec des entiers.

L'en-tête sera : `point_milieu(T, xa, ya, q, p)` et il y aura au plus $3p_x$ additions dans la boucle `while`.

```

def point_milieu(T, xa, ya, xb, yb, g = np.uint8(0)):
    '''
    T      : np.ndarray de dim 2; écran ou image;
    xa, ya, xb, yb :int.

    Trace la droite (AB) dans T selon
    l'alg. de Bresenham.
    '''
    assert isinstance(xa, int) and isinstance(ya, int)

    alpha, beta = (xb-xa), (yb-ya)
    assert alpha>0 and beta>= 0 and beta/alpha<= 1

    assert isinstance(T, np.ndarray) and T.ndim == 2
    px, py = T.shape

    ig, id = 2*(beta-alpha), 2*beta
    # pour: incrément cas gauche, cas droite
    x, y, dp = xa, ya, 2*beta-alpha
    # car (xa, ya) est sur la droite

    while x < px and y < py:
        T[x,y] = g
        if dp > 0:
            x, y, dp = x+1, y+1, dp + ig
        else:
            x, dp = x+1, dp + id
    return T

px, py = 64, 64
P = np.matrix(np.zeros((px,py),dtype= np.uint8)-1 )
D = point_milieu(P,0,0,8,4, g = 128)
pl.imshow(D, cmap='gray')
pl.savefig(directory + 'Bresenham_point_milieu1.png')
pl.plot([0, py//2],[0, px], '--', color = 'white')
# on inverse x et y.
pl.show()

```

Corrigé de l'exercice n° 6.4

1. Complexité de la fonction interpoler9() :

soustractions : $1 + 3 = 4$ au plus ;

additions : $1 + 3 + 16 = 20$ au plus (il y a 8 voisins au plus) ;

une division et une conversion.

```
def interpoler9(T, x, y):
    assert isinstance(T, np.ndarray) and T.ndim == 2
    N, M = T.shape
    n, c = 0, 0.

    for u in range(max(0, x-1), min(N, x+2)):
        for v in range(max(0, y-1), min(M, y+2)):
            n += 1
            c += T[u, v]

    return np.uint8(c/n)
```

2. On réserve un tableau que l'on remplit à partir d'une approximation du pixel antécédent et d'une interpolation. L'option nous a servi à produire les deux homothétiques du logo Python (panne d'inspiration).

```
def agrandir(T, k, option):

    assert option in [1, 5, 9]
    assert isinstance(T, np.ndarray) and T.ndim == 2
    N, M = T.shape
    N1, M1 = int(k*N), int(k*M)
    T1 = np.zeros((N1, M1), dtype = np.uint8)-1

    for x1 in range(0, N1):
        for y1 in range(0, M1):
            x, y = int(x1/k), int(y1/k)
            if option == 1:
                T1[x1, y1] = T[x, y]
            elif option == 5:
                T1[x1, y1] = interpoler5(T, x, y)
            elif option == 9:
                T1[x1, y1] = interpoler9(T, x, y)
    return T1

P = mpimg.imread(directory + 'LogoPython.jpeg')
P1 = conversion(P)
P2 = agrandir(P1, 2, 9)
pl.imshow(P2, cmap = 'gray')
pl.show()
pl.close()
```

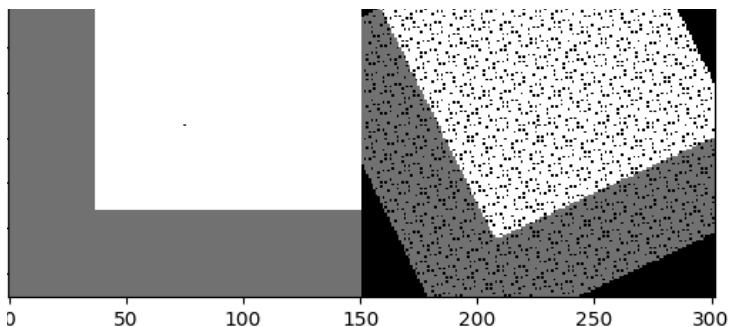
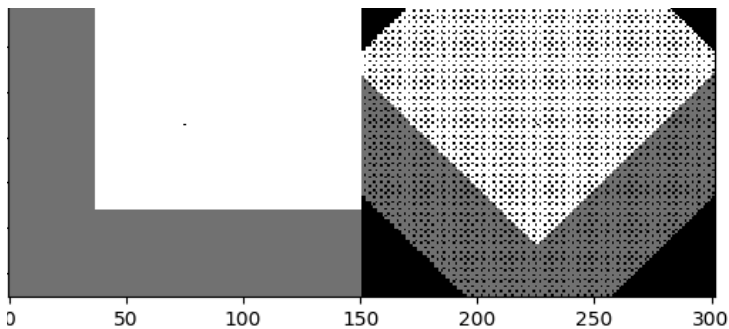
Complexité avec option = X : au plus $k^2 p_x p_y$ appels de `interpolerX()`.

Corrigé de l'exercice n° 6.5

- Après rotation de centre C et d'angle $\frac{\pi}{6}$ certaines parties de l'image du « L » ou du carré sortent de l'écran.

2. Les points de ces quatre triangles sont restés noirs ($T_1[x, y] = 0$) car ils ne correspondent à aucune image X_1 d'un centre de pixel de img .
3. Les images des centres de pixels grisés du « L » (ou blancs du carré) se retrouvent dans les parties grisée et blanche et éclairent les pixels les plus proches. On voit donc les images par rotation du « L » et du carré.
4. Les pixels noirs sont ceux qui n'ont pas été modifiés comme proches d'une image X_1 . Pour les pixels des triangles noirs des 4 coins, cela se comprend bien avec la première question. Pour les pixels présents dans l'image mathématique du « L » ou du carré, cela s'explique par le fait qu'ils n'ont pas été reconnus comme points proches de points X_1 . Cela implique, le nombre de points X_1 calculés et qui se retrouvent dans l'écran étant strictement supérieur aux points éclairés, que certains pixels de img1 ont été éclairés/affectés plusieurs fois par l'algorithme.
5. Ces points ne dépendent pas du contenu de l'image d'origine mais seulement de sa taille, **de l'angle de la rotation** et de la façon dont les flottants sont implémentés (n'oublions pas que ce sont des calculs approchés). On observe que ces pixels injustement oubliés forment des figures remarquables.

Les figures qui suivent, représentent la même image de 151×151 pixels et ses transformations par des rotations d'angles $\pi/4$ et $\pi/7$.



Le code qui a permis de construire la première image :

```
def grille_test1(N,M):
    '''
    N, M : int.
    Renvoie un tableau de type
    numpy.ndarray((N,M), dtype = numpy.uint8).
    '''
    imgG = np.zeros((N, M), dtype = np.uint8)
    imgG += 255
    for i in range((3*N)//4, N):
        imgG[i, :] = 112
    for j in range(0, N//4):
        imgG[:, j] = 112
    imgG[N//2, M//2] = 0 # le centre, pixel noir
    return imgG
```

Le code naïf qui a permis de faire pivoter les figures :

```
def rotation(img, C, a):
    '''
    N, M = img.shape
    img1 = np.zeros((N, 2*M), dtype = np.uint8)

    img1[:, 0:M] = img
    # on garde l'image originale à gauche
    c, s = np.cos(a), np.sin(a)
    R = np.matrix([[c,-s],[s,c]])

    for i in range(0,N):
        for j in range(0,M):
            col = img[i,j] #la couleur
            X = np.matrix([[i+1/2], [j+1/2]])
            X1 = C + R*(X-C)
            i1 = int(X1[0,0])
            j1 = int(X1[1,0])
            if 0 <= i1 < N and 0 <= j1 < M:
                img1[i1, M + j1] = col

    return img1
```

Corrigé de l'exercice n° 6.6

1. Mêmes réponses que dans l'exercice 6.5.
2. Mêmes réponses que dans l'exercice 6.5.
3. (a) On évite de sortir du tableau ou de l'écran avec les fonctions max et min.

```
def couleur_interpolation(T, i, j, p):
    '''
    assert isinstance(T, np.ndarray) \
        and T.ndim == 2

    N, M = T.shape
    n, c = 0, 0.
    for u in range(max(0,i-p), min(N,i+p+1)):
        for v in range(max(0,j-p), min(M,j+p+1)):
            n += 1 # nombre de pixels visités
            c += T[u, v]
    return np.uint8(c/n)
```

(b) Voir plus bas.

(c) Pour implémenter l'algorithme on réserve un tableau T1 destiné à contenir les deux images. Initialisé avec des 0, il contient au départ une image noire.

```
def rotation_interpolation(T, C, a):
    '''
    T      : np.ndarray;
    C      : np.ndarray (centre de la rotation);
    a      : float (0 <= a < 2 pi, angle).
    '''

    assert isinstance(T,np.ndarray) and T.ndim == 2

    N, M      = T.shape
    T1         = np.zeros((N, 2*M), dtype = np.uint8)
    T1[:, 0:M] = T # image d'origine à gauche

    c, s = np.cos(a), np.sin(a)
    iR    = np.matrix([[c, s],[-s,c]])
    #c'est bien l'inverse
    for il in range(0,N):
        for jl in range(0,M):
            X1 = np.matrix([[il+1/2], [jl+1/2]])
            X   = C + iR*(X1-C)
            i   = int(X[0,0])
            j   = int(X[1,0])

            if 0 <= i < N and 0 <= j < M:
                # on s'assure que (i,j) est dans l'image
                T1[i1, M + j1] = \
                    couleur_interpolation(T,i,j,1)

    return T1
```

La question (b) : On obtient X en fonction de X_1 en résolvant la relation $X_1 = C + R(X - C)$ comme une équation d'inconnue X , ce qui donne

$X = C + R^{-1}(X_1 - C)$ soit,

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x_c \\ y_c \end{bmatrix} + \begin{bmatrix} \cos \alpha & \sin \alpha \\ -\sin \alpha & \cos \alpha \end{bmatrix} \begin{bmatrix} x_1 - x_c \\ y_1 - y_c \end{bmatrix} \quad (6.5.1)$$

Corrigé de l'exercice n° 6.7

- Pour T_5 , on aura $c_{-2} = c_{-1} = c_0 = c_1 = c_2 = 1/5$ et $c_n = 0$ dans les autres cas.
- Pour P_1 , on aura $c_0 = -1$, $c_{-1} = +1$ et $c_n = 0$ dans les autres cas.
- Pour P_2 , on aura $c_1 = -1$, $c_{-1} = +1$ et $c_n = 0$ dans les autres cas.

Ne pas oublier que le coefficient de z_{n-i} est c_{+i} .

Corrigé de l'exercice n° 6.8

1. On peut représenter v comme les coordonnées relatives des pixels voisins de (x, y) qui interviennent dans le calcul de la coloration du pixel (x, y) dans la

$$\text{nouvelle image : } \begin{vmatrix} (x-1, y-1) & * & (x-1, y+1) \\ * & (x, y) & * \\ (x+1, y-1) & * & (x+1, y+1) \end{vmatrix}$$

n est incrémenté dès que $(x, y) + v$ est un pixel de l'image, donc :

Lorsque (x, y) est un des quatre coins de l'image, par exemple $(0, 0)$ il n'y aura qu'un seul de ces voisins dans l'image et $n = 1$. Lorsque (x, y) est sur un bord, $n = 2$ et enfin $n = 4$ pour un point intérieur.

2. Dans ce dernier cas g/n contient

$$\frac{1}{4}(T[x-1, y-1] + T[x-1, y+1] + T[x+1, y-1] + T[x+1, y+1]) = T * c(x, y)$$

avec c dont le support est V (du programme) avec $c_{i,j} = 1/4$ lorsque $[i, j] \in V$.

3. interpoler9() (qui figure dans le corrigé de la page 268) utilise un noyau dont le support est exactement $\llbracket -1, 1 \rrbracket \times \llbracket -1, 1 \rrbracket$ et qui vaut $1/9$ en tout point de son support, 0 ailleurs.

Corrigé de l'exercice n° 6.9

```
def convolution_xy(T, C, d, x, y):
    '''
    T : numpy.ndarray, tableau à deux dimensions
        représentant une image;
    C : numpy.ndarray, tableau (2d+1)x(2d+1) à
        deux dimensions représentant un noyau de convolution;
    d : int (tel que C est de taille (2d+1)x(2d+1))
    '''
    g = 0 # couleur ou niveau de gris
    for i in range(-d, d+1):
        for j in range(-d, d+1):
            g += int(T[x+i, y+j]) * C[d-i, d-j]
    return g
```



```

def convolution(T, C):
    '''
    T : numpy.ndarray, tableau à deux dimensions
        représentant une image;
    C : numpy.ndarray, tableau (2d+1)x(2d+1) à deux
        dimensions représentant un noyau de convolution;
        C est de taille (2d+1)x(2d+1).
    '''
    assert isinstance(T, np.ndarray) and T.ndim == 2
    assert isinstance(C, np.ndarray) and C.ndim == 2

    px, py = T.shape
    p, q    = C.shape

    assert p == q and p % 2 == 1
    d       = (p-1)//2

    T1      = np.zeros((px,py), dtype = np.uint8)-1
    # T1[x,y] =255; image blanche.
    for x in range(d, px-d):
        for y in range(d, py-d):
            T1[x,y] = convolution_xy(T, C, d, x, y)
    return T1

```

Corrigé de l'exercice n° 6.10

1. Par définition, $(T * c)_{x,y} = \sum_{i=-d}^d \sum_{j=-d}^d T_{x-i,y-j} c_{i,j}$ donc

$$(T * (c + c'))_{x,y} = \sum_{i=-d}^d \sum_{j=-d}^d T_{x-i,y-j} (c_{i,j} + c'_{i,j}) = (T * c)_{x,y} + (T * c')_{x,y}.$$

2. $(T * c)_{i,j} = T_{i+1,j-1} - T_{i-1,j+1}$

3. $(T * c)_{i,i} = 2T_{i+1,j} - 2T_{i-1,j}$

4. D'après la première question

$$(T * G_x)_{i,j} = (T_{i+1,j+1} - T_{i-1,j+1}) + 2(T_{i+1,j} - T_{i-1,j}) + (T_{i+1,j-1} - T_{i-1,j-1}).$$

qui est une approximation de $4 \times d \times \frac{\partial}{\partial x} T(x, y)$

Corrigé de l'exercice n° 6.11

1. Sous forme de matrices ou de tableaux numpy :

```

C_lissage = np.ones((5,5), dtype = int)/25
Sobel_x   = np.matrix([[1,2,1],[0,0,0],[-1,-2,-1]], \
                        dtype = int)
Sobel_y   = Sobel_x.transpose()

```

2. Avec ces normes, si l'image contient des pixels en entier ou en `np.uint8`, les calculs seront faits avec des entiers. Avec la norme euclidienne on calculera en flottant. Les opérations seront plus longues.
3. La fonction `convolution(T, C)` est présentée deux corrigés plus haut.

```
def contours_Sobel(T, s):
    '''
    T : numpy.ndarray, tableau à deux dimensions
        représentant une image;
    s : int, seuil pour tronquer la norme du gradient.
    '''
    assert isinstance(T, np.ndarray) and T.ndim == 2

    C_lissage = np.ones((5,5), dtype = int)/25
    Sobel_x    = np.matrix([[1,2,1],[0,0,0],[-1,-2,-1]],\
                           dtype = int)

    Sobel_y    = Sobel_x.transpose()

    T0 = convolution(T, C_lissage)
    GX = convolution(T0, Sobel_x)
    GY = convolution(T0, Sobel_y)

    px, py = T.shape
    TC = np.zeros((px,py), dtype = np.uint8)-1
    #image blanche.
    for x in range(0, px):
        for y in range(0, py):
            g = max(abs(int(GX[x,y])), abs(int(GY[x,y])))
            if g > s:
                TC[x,y] = np.uint8(0)
    return TC
```




Deuxième
semestre

Calcul numérique : problématique et outils

Nous exposons dans ce chapitre les idées générales permettant d'aborder le calcul numérique. Nous y verrons avec quels nombres et quelle arithmétique on effectue des calculs exacts ou approchés sur un ordinateur. Cela nous permettra d'expliquer certains phénomènes surprenants : des suites de nombres réels, dont on peut démontrer qu'elles convergent qui, lorsqu'on tente de les programmer, donnent naissance à des processus divergents ; des méthodes qui *a priori* semblent converger et se mettent soudain à déraiser.

7.1 Représentation des nombres et erreurs de calcul

Commençons par une observation toute simple : avec une calculatrice numérique, on exécute en mode formel, puis avec des « flottants » les opérations : $\frac{1}{9} \times 9 - 1$:

$\frac{1}{9} \cdot 9 - 1$	0
$\frac{1}{9} \cdot 9 - 1$	$-1.E-14$
$\frac{1}{9}$	0.111111
$0.11111111111111 \cdot 9$	1.
$0.99999999999999 - 1$	$-1.E-14$
	5/99

1. Calcul en mode formel : $\frac{1}{9} \times 9 - 1 = 0$.

2. Calcul en flottants :

$$\frac{1}{9} \times 9 - 1 \approx -10^{-14};$$

3. $\frac{1}{9} \approx 0.111111$ est un flottant ;

4. L'utilisateur a tapé « ans*9 », qui affiche
1. (un-point). Étrange !

5. L'utilisateur, têtue, a tapé « ans-1 ».

Ce dernier résultat montre que 'ans' = $\frac{1}{9} \times 9$, bien qu'arrondi à 1. à l'affichage précédent est en fait $0.99999999999999 = 1 - 10 \times 10^{-14}$ en mémoire machine.

Nous allons tenter de mesurer les conséquences de l'approximation des réels par des flottants. Pour cela, nous regarderons comment sont représentés les entiers et les réels dans un ordinateur. Le choix d'une représentation des nombres n'est pas (n'est plus, depuis 1982) laissée à la libre inspiration des fabricants de processeurs : elle obéit à des normes que nous allons brièvement introduire après quelques rappels sur la numération.

7.1.1 Numérations décimale, binaire, hexadécimale

Les circuits et mémoires physiques sont des assemblages de transistors ne pouvant prendre que deux états (nous ne disposons pas pour notre part d'ordinateur quantique, et vous non plus pour l'instant). L'information y est donc naturellement stockée sous forme de chaînes de bits (*binary digits* ou chiffres binaires, 0 ou 1) ce qui impose que la représentation des entiers signés ou des flottants repose sur leur écriture binaire. Donnons donc quelques indications sur la façon d'*écrire* ou d'*approcher* les nombres en base 10, 2 ou 16 (écritures décimales, binaires ou hexadécimale). L'écriture en base dix est celle dont nous faisons notre pain quotidien, l'écriture en base deux est, nous l'avons dit, celle qui nous permet de représenter les nombres en mémoire, quant à la représentation hexadécimale elle permet une écriture plus condensée qui se déduit immédiatement de l'écriture binaire, ce qui fait son intérêt en informatique. Soit $n \in \mathbb{N}$, un entier naturel, on note $n_{(b)}$ sa représentation en base $b > 1$ qui est la suite de symboles ou chiffres représentant des nombres de zéro à $b - 1$ telle que

$$n_{(b)} = a_p a_{p-1} \dots a_0 \Leftrightarrow n = \sum_{k=0}^p a_k b^k$$

Ainsi, le nombre deux cent dix neuf s'écrit

- en base dix : [2,1,9] avec des symboles choisis dans $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$;
- en base deux : [1,1,0,1,1,0,1,1] des symboles choisis dans $\{0, 1\}$;
- en base seize : [D,B] avec les symboles choisis parmi les 16 symboles (ou chiffres) de $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$;

On étend ces notations aux nombres de la forme

$$x = \sum_{k=0}^p a_k b^k + \sum_{k=1}^p a_{-k} b^{-k}$$

que l'on écrit en base b : $x_{(b)} = a_p a_{p-1} \dots a_0, a_{-1} a_{-2} \dots a_{-p}$.

Lorsque $b = 10$, ce sont (par définition) les nombres **décimaux** : $\frac{q}{10^p}$, $q \in \mathbb{Z}, p \in \mathbb{N}$.

Lorsque $b = 2$, ce sont les nombres **dyadiques**, de la forme $\frac{q}{2^p}$.

Même si les dyadiques sont tous des décimaux, les deux ensembles sont distincts : $x_{(10)} = 0.1_{(10)}$ par exemple, n'est pas un nombre dyadique car l'équation $\frac{1}{10} = \frac{q}{2^p}$

n'a pas de solution dans $(\mathbb{Z} \times \mathbb{N})^1$, il n'a donc pas de représentation exacte en machine et tout au plus peut on (comme pour tout nombre réel) l'approcher arbitrairement près par des nombres dyadiques :

$$\frac{1}{10} = \frac{1}{2^4} + \frac{1}{2^5} + \frac{1}{2^8} + \frac{1}{2^9} + \frac{1}{2^{12}} + \frac{1}{2^{13}} + \dots$$

Conversions des entiers sous Python

Vous pouvez, sous Python calculer la forme décimale d'un entier exprimé en base 2, 8 ou 16 avec les instructions 0b, 0o, 0x suivie de la chaîne des chiffres décimaux :

>>> 0o10	>>> 0b1010101
8	85
>>> 0x10	>>> 0xAB12F
16	700719
>>> 0b10	>>> 0b123
2	SyntaxError: invalid syntax

L'énoncé qui suit vous indique comment on calcule la représentation binaire d'un entier relatif, puis comment on calcule une expression dyadique approchée d'un décimal.

Exercice 7.1 conversions-« Je ne comprends que ce que je sais programmer »

1. Écrire une fonction `conv10to2Int(n)` qui prend un entier signé (entier relatif) en argument et retourne la liste des ses chiffres en base deux, éventuellement précédée du signe '-'.

```
>>> conv10to2Int(259)
[1, 0, 0, 0, 0, 0, 0, 0, 1, 1]
>>> conv10to2Int(-259)
['-', 1, 0, 0, 0, 0, 0, 0, 0, 1, 1]
```

Conseil : procéder par divisions successives de $|n|$ par 2.

2. On donne ici une fonction qui prend en arguments un décimal x et un flottant $\varepsilon < 1$ et retourne la liste éventuellement signée des chiffres (0 ou 1) d'une approximation dyadique de x à ε près :

1. Elle est équivalente à $2^{p-1} = 5q$ et 5 et 2 sont premiers entre eux nous dirait un arithméticien.


```
def conv10to2(x, eps):
    v      = abs(x)
    e, f    = int(v), v - int(v)
    eps1    = min(eps, 1/2)

    L1      = conv10to2Int(e)
    if f != 0:
        L2    = conv10to2Frac(f, eps1)
        L1.extend(L2)
    if x < 0:
        L1.insert(0, '-')
    return L1
```

Cette fonction fait appel à `conv10to2Int`, objet de la première question et à une fonction `conv10to2Frac( $f$ ,  $eps1$ )` qui, à partir d'une partie fractionnaire décimale, retourne son approximation dyadique à $eps1$ près. Écrire cette fonction.

Méthode : pour déterminer les chiffres dyadiques de f on procède par itération, et, à chaque étape,

- si $2f < 1$ le chiffre est 0, on remplace f par $2f$;
- sinon, le chiffre est 1, on remplace f par $2f - 1$;

```
>>> conv10to2Frac( 0.1, 2**(-30))
['.', 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0,
0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0]
```

3. Écrire une fonction `conv2to10(L)` qui prend en argument une liste représentant un nombre en base 2, contenant éventuellement le symbole '.' et qui renvoie l'écriture décimale associée.

Indication d'ordre syntaxique : utiliser `in` puis `L.index('.')` pour déterminer l'emplacement éventuel du point décimal (ou de la virgule).

Corrigé en 7.1 page 302.

7.1.2 Représentation des entiers sur n bits

Dans la plupart des langages de programmation la représentation des nombres entiers en machine est une chaîne binaire sur n bits (n dépendant évidemment du processeur). L'ensemble des nombres représentés est alors limité. C'était le cas avec les versions Python 2.x (qui ne sont plus « supportées » aujourd'hui).

Avec les versions Python 3.x, les entiers sont codés selon ce standard général (que nous décrivons dans ce paragraphe) tant qu'ils ne dépassent pas une certaine taille. Au delà, le système passe, de façon transparente pour l'utilisateur, à un autre type de représentation que nous évoquerons plus loin (entiers multi-précision).

- La représentation des entiers sur des emplacements ou mots de taille fixe (64 bits aujourd'hui sur les ordinateurs personnels) a pu historiquement adopter les deux conventions que nous décrivons brièvement. C'est la convention du complément à deux qui est devenue le standard.

• **Convention de la valeur signée**

Un chaîne de n bits $b_{n-1}b_{n-2}\dots b_1b_0$ représentant un entier, est interprétée de la façon suivante : le premier bit (ou chiffre binaire), b_{n-1} , est égal à 0 si l'entier est positif et à 1 sinon ; la suite $b_{n-2}\dots b_1b_0$ code alors la valeur absolue.

Conséquences :

- 0 a deux représentations 0000 et 1000 dans le cas d'une représentation sur 4 bits ;
- on représente les entiers compris entre $-(2^{p-1} - 1)$ et $(2^{p-1} - 1)$, intervalle symétrique ;
- il faut un algorithme (en circuit imprimé) pour l'addition de deux nombres de même signe, un autre pour l'addition de nombres de signes opposés ;

• **Convention du complément à 2**

Pour palier les inconvénients de la représentation précédente, on procède de la façon suivante :

- un nombre positif ou nul est représenté par la chaîne $0b_{n-1}b_{n-2}\dots b_1b_0$;
- pour un nombre strictement négatif, on inverse les chiffres de sa valeur absolue et on ajoute 1.

Conséquences :

- le premier bit (bit de poids fort) est le bit de signe (0 si le nombre est positif, 1 sinon) ;
- 0 n'a qu'une représentation ;
- on représente ainsi les entiers de -2^{n-1} à $2^{n-1} - 1$;
- l'addition (ou la soustraction) est codée selon un seul algorithme que nous illustrons brièvement ici (voir aussi l'exercice qui suit).

La flèche verticale pour les deux étapes conduisant au complément à deux : renversement des bits et incrémentation d'une unité...

14	0	0	1	1	1	0
12	0	0	1	1	0	0
↓	1	1	0	0	1	1
-12	1	1	0	1	0	0
14+(-12)	0	0	0	0	1	0

12	0	0	1	1	0	0
14	0	0	1	1	1	0
↓	1	1	0	0	0	1
-14	1	1	0	0	1	0
12+(-14)	1	1	1	1	1	0

À gauche, le résultat est $2_{(10)}$.

À droite résultat est négatif (puisque le premier bit égal à 1), le complément à 2, c'est-à-dire l'opposé est $2_{(10)}$.

• Longueur des chaînes

Le nombre de bits, comme nous l'avons vu, influe directement sur l'intervalle des entiers représentables. Dans certains langages de programmation des entiers codés sur plusieurs longueurs cohabitent, ce qui permet une économie de place mémoire si le programmeur utilise la taille qui lui est strictement nécessaire. C'était le cas de Python avant la version 3.

Exercice 7.2 pour comprendre la notion de complément à deux

On suppose que les entiers sont représentés par des listes de termes choisis dans $\{0, 1\}$.

1. Écrire une fonction `add(L1, L2)` qui réalise l'addition binaire de deux entiers positifs représentés par des listes de chiffres (de tailles quelconques : on pensera à réajuster la longueur des listes).

On pourra commencer par une fonction `add_bits( $b_1, b_2, r_0$ )` qui retourne le couple (b, r_1) résultat (avec retenue) de l'addition des deux bits b_1 et b_2 avec retenue r_0 .

2. Écrire une fonction qui prend une liste de bits en argument et retourne son complément à deux. Cette fonction utilise l'addition.

À partir de cette question on suppose que les listes ont le même nombre de termes et que le premier bit est dévolu au signe du nombre.

Voilà ce que l'on veut :

```
>>> L1=conv10to2Z(567,14)
>>> L2=conv10to2Z(-567,14)
>>> L1;L2
[0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1]
[1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 0, 0, 1]
>>> add_n(L1,L2)
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
>>> L1=conv10to2Z(14,7)
>>> L2=conv10to2Z(-12,7)
>>> L1;L2
[0, 0, 0, 1, 1, 1, 0]
[1, 1, 1, 0, 1, 0, 0]
>>> add_n(L1,L2)
[0, 0, 0, 0, 0, 1, 0]
```

3. Modifier `add` pour qu'elle lève une erreur si les deux listes n'ont pas la même taille. On notera `add_n(L1, L2)` la fonction obtenue. Elle retourne une liste $L1 + L2$ en binaire de même taille que $L1$ et $L2$ et ne tient pas compte de la dernière retenue.
4. Écrire une fonction `conv10to2Z( $x, n$ )` qui prend un entier relatif x et un entier strictement positifs en arguments et retourne la liste de longueur n contenant

les bits de x en binaire avec la convention du complément à 2. Le bit de poids fort est donc dédié au signe.

Cette fonction lève une exception si $|x|$ est trop grand.

5. Écrire une fonction `diff(L1, L2)` qui réalise la soustraction binaire de deux entiers en appelant la fonction précédente.

Corrigé en 7.2 page 303.

7.1.3 Les entiers multi-précision de Python

Comme nous l'avons remarqué dans ce qui précède une représentation des entiers sur 64 bits, avec la convention du complément à 2, permet de représenter tous les entiers n tels que $-2^{63} \leq n \leq 2^{63} - 1$. C'est cette convention qui est adoptée par défaut dans la plupart des langages de programmation. Les scripts qui suivent, exécutés sous Python 3x et avec un processeur 64 bits, montrent que la taille des entiers ne semble pas y être limitée, contrairement à celle des flottants.

```
p = 0
while True:
    print(p, 2.**p)
    p += 1

0      1.0
1      2.0
2      4.0
3      8.0
4     16.0
...
11    2048.0
12   4096.0
...
1022  4.49423283715579e+307
1023  8.98846567431158e+307
Traceback bla bla bla
      print(p, 2.**p)
OverflowError: ... 'Numerical
result out of range'
```

```
p = 0
while True:
    print(p, 2**p)
    p += 1
```

Le script de gauche termine sur une erreur : comme nous le verrons dans la section suivante, le flottant 2^{1024} n'est pas représentable : la représentation des flottants sur 64 bits est limitée, comme celle des entiers. Le script de droite, quant à lui, travaille avec des entiers. Dans le cas où ceux-ci sont représentés sur des mots de taille fixe il doit s'arrêter sur une erreur du même type (avec des mots de 64 bits, dès que $p = 63$ puisque le plus grand nombre représentable est $2^{63} - 1$).

Or, ce n'est pas ce qui se passe. En effet, nous l'avons stoppé avec $p = 16005$ ce qui donne pour la puissance de 2 autant de chiffres binaires. Nous sommes là dans le domaine des **entiers multi-précision** que sait gérer la classe `int` de Python.

Ce choix est évidemment systématique dans les **logiciels de calcul formel** (comme XCas, Maxima, SageMath, écrit en Python et qui utilise le module `Sympy`² et, pour

2. Un résumé des fonctionnalités de `Sympy` est disponible sur le site univienligne.fr d'un des auteurs

citer des logiciels commerciaux, Maple ou Mathematica). Il est plus rare dans les langages de programmation.

Les calculs avec des entiers de tailles illimitées sont indispensables en cryptographie, en arithmétique pour tester des conjectures (on parle parfois de mathématiques expérimentales). L'exercice (7.5) illustre sur un cas d'école l'intérêt des entiers multi-précision : on y exécute des calculs exacts avec des entiers de grandes tailles pour évaluer des expressions pour lesquelles le calcul approché avec des flottants provoque une telle accumulation d'erreurs qu'ils n'ont plus aucune fiabilité.

Représentation des entiers multi précision

Comme la taille des nombres que l'on représente n'est pas limitée *a priori* il faudra les représenter par des listes dont la longueur varie avec celle du nombre. Expliquons dans les grandes lignes UNE façon de procéder (qui dépend des systèmes et n'est pas standardisée pour l'instant).

- On écrit les entiers dans une base β : $X = \sum_{k=0}^p b_k \beta^k$ (avec $0 \leq b_k < \beta$).
- On représente X par la liste des chiffres $[b_0, b_1, \dots, b_p]$.
Ainsi, en base 10, $X = 30987$ serait représenté par la liste $[7, 8, 9, 0, 3]$.
- Les chiffres $(b_k)_k$ sont stockés dans des registres machine et on choisit la base β la plus grand possible qui permettra ce stockage. Cela dépend évidemment du processeur.
- Il ne reste plus qu'à choisir les algorithmes pour réaliser additions, soustractions et multiplications. Pour l'addition et la soustraction les techniques ne diffèrent pas fondamentalement de celles de l'exercice 7.2.

Complexité de la multiplication des entiers multi-précision

Jusqu'à présent, pour estimer la complexité de certaines opérations arithmétiques ou numériques, nous nous sommes contentés de calculer ou de majorer le nombre de certaines opérations (par exemple, pour déterminer la complexité de calculs matriciels, nous avons compté les seules multiplications, beaucoup plus coûteuses que les additions). Ce choix avait du sens pour les calculs avec des entiers de taille fixe ou des flottants de taille fixe puisque chaque opération arithmétique a un coût constant. Lorsque la taille du nombre est arbitraire, **la complexité d'une opération varie avec cette taille**. Il ne suffit plus de dénombrer les opérations, il faut aussi estimer la taille des nombres qui entreront en jeu autant pour ce qui est de la place occupée en mémoire que pour le temps de calcul. Cela dépasse évidemment nos objectifs, mais l'exercice qui suit devrait permettre de saisir les enjeux.

Exercice 7.3 multiplication des entiers

Par commodité nous parlerons de « petits entiers » ou mots machines pour les entiers représentés sur n bits et de « grands entiers » pour les entiers multi-précision.

1. Combien faut-il de chiffres pour représenter l'entier naturel $X \geq 1$ en base β ?
Nous appellerons ce nombre de chiffres **la taille de X** .

2. Addition : on considère ici un langage $\mathcal{L}$ qui ne connaît que les petits entiers, sur 64 bits par exemple. On décide de représenter dans ce langage les entiers multi-précision par les listes des leurs chiffres en base β . Cela suppose que β est lui-même un petit entier.
- (a) Écrire en Python une fonction `addition(X, Y, beta)` qui prend en arguments deux listes X et Y de petits entiers représentant des nombres en base β et renvoie la liste représentant leur somme.
Cette fonction respectera la contrainte du langage $\mathcal{L}$ en ne manipulant que des petits entiers.
 - (b) Quelle est la condition sur β pour que la contrainte soit respectée ?
 - (c) Exprimer la complexité de cette fonction en fonction de la taille de X et de Y ?
3. Multiplication naïve : pour multiplier deux grands entiers $X = \sum_{i=0}^p b_i \beta^i$ et $Y = \sum_{j=0}^q c_j \beta^j$ écrits en base β , l'algorithme naïf, part de la formule

$$Z = X \times Y = \sum_{k=0}^{p+q} \left(\sum_{i+j=k} b_i c_j \right) \beta^k$$

- (a) Écrire une fonction `multiplicationPG(a, p, X, beta)` qui prend en arguments a, p deux petits entiers ($0 \leq a \leq \beta - 1$), X une liste représentant un entier multi-précision en base β et $beta = \beta$, qui renvoie la liste Z des chiffres de $a\beta^p X$ en base β . Cette fonction respectera la contrainte du langage $\mathcal{L}$ en ne manipulant que des petits entiers.
- (b) Quelle est la condition sur β pour que la contrainte soit respectée ?
- (c) Exprimer la complexité de cette fonction en fonction de la taille de X .
- (d) Utiliser cette fonction ainsi que `addition(X,Y,beta)` pour écrire `multiplication[X,Y, beta)` qui renvoie la liste Z représentant le produit des deux grands entiers X et Y . Quelle est la complexité de cette multiplication naïve ?

Corrigé en 7.3 page 304.

Remarque : cet exercice montre comment représenter et construire l'addition et la multiplication des entiers multi-précision quand on dispose déjà de celle des petits entiers. La problématique est identique lorsqu'on veut construire ces opérations sur les entiers représentés sur n bits dans un mot machine. Dans ce cas on travaille sur les chiffres binaires.

Algorithmes pour la multiplication

L'algorithme naïf pour la multiplication de deux entiers de tailles m et n a un coût en $O(m \times n)$. Depuis les années 60 du siècle précédent, des algorithmes plus rapides ont été conçus et aujourd'hui les systèmes de calcul formels implémentent des algorithmes sous-quadratiques. Mots clés : algorithmes de Karatsuba, transformée de Fourier rapide, bibliothèque GMP, SymPy..

7.1.4 Représentation des flottants sur n bits

Remarque préalable : nous noterons, dans ce qui suit, le séparateur décimal '.' (usage anglo-saxon) et non pas ',' (usage francophone), tout en parlant de « virgule » !

Représentation en virgule flottante normalisée

On appelle *nombre à virgule flottante* (ou flottant) un nombre de la forme

$$x = s \times m \times b^e$$

où $s \in \{-1, 1\}$ est le signe de x , m sa mantisse, e l'exposant (entier relatif) et b la base dans laquelle tout cela est exprimé. Par exemple, en base 10 ou en base 2, on écrira tout aussi bien :

$$x_{(10)} = +5.6875 \times 10^0 = +568.75 \times 10^{-2} \dots \quad (7.1.1)$$

$$x_{(2)} = +101.1011 \times 2^0 = +1011011 \times 2^{-5} \quad (7.1.2)$$

Une représentation en *virgule flottante normalisée* est une représentation dans laquelle la mantisse est de la forme $b_0.b_{-1}b_{-2}\dots$ avec b_0 non nul (cela impose $x \neq 0$). En reprenant les exemples précédents nous aurons $x_{(10)} = +5.6875 \times 10^0$ ou $x_{(2)} = +1.011011 \times 2^2$.

On observera qu'en base 2, le premier chiffre avant la virgule dans la mantisse est toujours égal à 1.

La norme IEEE 754

Cette norme définit un format pour la représentation des flottants binaires ainsi que les opérations arithmétiques sur ces nombres (et en particulier leurs modes d'arrondis). Elle définit 4 formats dont la double précision sur 64 bits et la double précision étendue (sur 79 bits). Nous ne la décrirons que partiellement en nous restreignant aux grandes lignes du format double précision (64 bits) et en commençant par la représentation des nombres normalisés.

Pour comprendre ce qui suit, il faudra faire attention à distinguer les chiffres de l'écriture binaire de $x = \pm m \times 2^e$ (comme dans la formule (7.1.2)), des bits qui servent

à représenter ce même nombre en mémoire. Comme nous le verrons il y a quelques décalages : mineur pour la mantisse et un peu plus compliqué pour l'exposant. Rappelons enfin qu'en écriture binaire $2^{10} = 10\,000\,000\,000$, $2^{10} - 1 = 1\,111\,111\,111$.

- Les nombres en virgule flottante normalisée sont codés sur 64 bits

nombre normalisé	signe	exposant décalé	mantisse
$x = \pm 1.b_{-1} \dots b_{-52} \times 2^e$	un bit (n° 63)	$e + 1023$ sur 11 bits	sur 52 bits

- Le bit de poids fort (n° 63) définit le signe : il est égal à 0 pour les nombres positifs, à 1 pour les négatifs ;
 - Les 52 bits de n° 0 à 51 servent à coder la mantisse. Première différence entre la représentation en base 2 et la représentation en mémoire : le premier chiffre de la mantisse, qui est toujours égal à 1 dans une représentation normalisée en base 2, n'est pas représenté (on parle de *bit caché ou implicite*). Pour le reste ce sont bien les b_{-k} que l'on retrouve en mémoire.
 - Les 11 bits de n° 52 à 62 servent à représenter l'exposant de la façon suivante : si x est non nul et a pour exposant e compris entre -1022 et $2^{10} - 1 = 1023$, on ajoute 1023 à e et c'est l'écriture binaire de ce nombre que l'on représente en mémoire (et non pas celle de e lui-même) ; observons que dans la plage de 11 bits dévolue à l'exposant, $00..0 = 0$, et $11\,111\,111\,111_{(2)} = 2^{11} - 1 = 2047_{(10)}$ ne sont pas encore utilisés.
- Les autres « nombres » représentés
 - On représente 0 avec une mantisse égale à 0 et un exposant égal à -1023 (0, après décalage). Il y a donc *deux façons de représenter zéro* puisque le bit de poids fort (donnant le signe) peut prendre les deux valeurs 0 ou 1 (les 63 autres bits sont à zéro).
 - Pour représenter des nombres plus petits que les nombres normalisés, donc que 2^{-1022} , on décide qu'une chaîne pour laquelle les bits d'exposant sont nuls et les bits de mantisse non tous nuls est associée au nombre décimal $\pm 0.b_{-1} \dots b_{-52} \times 10^{-1022}$.
On parle de nombres dénormalisés. Nous expliquons leur intérêt plus loin.
 - On définit deux infinis, $\pm\infty$, avec les 11 bits dévolus à l'exposant pour les nombres normalisés représentant $2047_{(10)} = 2^{10} + 1023_{(10)}$ (ils sont donc tous égaux à 1) et les bits de la mantisse égaux à 0.

On voit donc que toutes les configurations possibles pour ces 11 bits sont maintenant occupées. Mais ce n'est pas fini : on convient de coder avec un exposant décalé à $2047_{(10)}$, et une mantisse non nulle des indicateurs d'erreurs NAN (not a number)³.

- Conséquences
 - Le plus petit nombre positif *normalisé* représentable par un flottant dans cette norme est $x = +1.0 \times 2^{-1022}$. Le bit de poids fort est égal à 0 ($s = +1$); son

3. Détails hors programme.

exposant -1022 est décalé vers 1, les 11 bits correspondants sont donc $0...01$. Sa mantisse est $m_{(2)} = 1.0...0$ et, comme le premier chiffre est toujours 1 et n'est pas représenté, les bits de n° 12 à 63 sont tous égaux à 0.

- Le plus petit nombre positif *dénormalisé* représentable dans cette norme est $x = +0.0...01 \times 2^{-1022} = 2^{-1074}$. Son bit de poids fort est égal à 0 ($s = +1$); son exposant -1022 est décalé vers 0, les 11 bits correspondants sont donc $0...000$; la mantisse est $m_{(2)} = 0.0...01$. Le premier 0 n'est pas représenté, la mantisse est représentée par la chaîne formée de 51 zéros suivis de 1.

Le tableau qui suit illustre ceci avec Python.

Python 3 et la norme IEEE 754

```
p = 0
while 2**(-p) != 0:
    print(p, 2**(-p))
    p=p+1

0    1
1    0.5
2    0.25
...
614  1.4708983551653345e-185
...
1073 1e-323
1074 5e-324
```

- La représentation du plus grand nombre (il est normalisé) a pour bit de signe 0, pour bits d'exposant décalé : 11 111 111 110; sa mantisse est représentée par la chaîne formée de 52 bits égaux à 1. On a donc

$$m = \sum_{k=0}^{52} 2^{-k} = \frac{1 - 2^{-53}}{1 - 2^{-1}} = 2 - 2^{-52}.$$

C'est donc $x = +m \times 2^{(2^{11}-1)-1024} = (2 - 2^{-52}) \times 2^{1023} = 2^{1024} - 2^{971}$.

Pour une vérification expérimentale, voir le tableau de la page 292.

Arrondis et calculs

Grâce aux deux infinis, considérés comme des nombres, pour tout réel x au sens des mathématiciens, il existe deux flottants (représentables en machine) consécutifs x_- et x_+ tels que

$$x_- \leq x \leq x_+.$$

On dit que x_- est **l'arrondi** de x vers $-\infty$, et que x_+ est son arrondi vers $+\infty$. On définit encore l'arrondi vers 0, l'arrondi au plus proche (avec une règle permettant de choisir entre x_- et x_+ lorsque x est au milieu).

Une fois le mode d'arrondi choisi, on peut définir une fonction $fl : x \in \mathbb{R} \rightarrow fl(x)$ où $fl(x) \in \{x_+ \text{ ou } x_-\}$ est défini d'une façon et d'une seule parmi les quatre possibles. Le résultat d'une opération sur les flottants peut alors être parfaitement spécifié.

En résumé

exposant décalé	qui peut-il être ?
$e' = 0$	$x = +0, -0$ ou un nombre dénormalisé $x = \pm 0.m'2^{-1022}$
$e' = 1$	$x = \pm 1.m'2^{-1022}$
$1 \leq e' \leq 2046$	$x = \pm 1.m'2^{e'-1023}$
$e' = 2046$	$x = \pm 1.m'2^{1023}$
$e' = 2047$	$x = \pm \infty$ si $m = 0$, NaN sinon (pour messages d'erreur)

Questions rapides

Vérifiez votre compréhension en répondant à ces questions brèves :

1. Un flottant est codé avec les 11 bits de l'exposant nuls ; que peut-il-être ?
2. Un flottant est codé avec les 11 bits de l'exposant égaux à 1 ; que peut-il-être ?
3. Un flottant est codé avec les 11 bits de l'exposant correspondant à l'écriture binaire de $1024_{(2)}$; que peut-il-être ? préciser des intervalles le contenant.
4. Un flottant est codé avec les 11 bits de l'exposant correspondant à l'écriture binaire de $1053_{(2)}$; que peut-il-être ?
5. À quoi sert le code Python de la page 290 ? Permet-il de vérifier les affirmations qui le précèdent ?
6. Quel est le plus grand flottant représentable selon IEEE 754 sur 64 bits ?
7. Que devient-il si on lui ajoute 1 ?
8. ★ Le code Python suivant est analogue à celui du cours. Permet-il de déterminer expérimentalement la plus grande puissance de 2 représentable par un flottant ? Termine-t-il ?

```
p=0
while 2**(p+1) !=2**p:
    print (p,2**p)
    p=p+1
```

Si vous l'avez programmé, avez-vous dit « mais oui, bien sûr ! » ou restez-vous perplexe ?

9. Quel sont les nombres représentés en mémoire par :

signe	exposant décalé	mantisse
1	2047 en base 2 (sur 11 bits)	52 bits égaux à zéros
1	2046 en base 2 (sur 11 bits)	52 bits égaux à un
0	345 en base 2 (sur 11 bits)	52 bits égaux à zéros
0	2 en base 2 (sur 11 bits)	52 bits égaux à zéros
0	11 zéros	52 bits égaux à zéros

10. Quels sont les flottants dont la représentation de l'exposant en mémoire est 0 ? Combien y en a-t-il ?
11. Quels sont le prédécesseur et le successeur immédiats de 1 ? Quel est l'écart entre 1 et chacun d'eux ?
12. Quels sont le prédécesseur et le successeur immédiats de $x = 2^{-1074}$? Quel est l'écart entre x et chacun d'eux ?

Python 3 et la norme IEE 754

À la recherche de la plus grande puissance de 2 représentable (Python 3.2 OS 64 bits) :

```
p = 0
while float(2**p) :
    print(p)
    p = p+1
...
1022
1023
Traceback ... int too large to convert to float
```

À la recherche de la plus petite valeur de p telle que $2^{1024} - 2^p$ soit représentable :

```
p = 999
d = 2**1024
while float(d - 2**p) != float('Infinity') :
    print('raffinons ', p)
    p=p-1
...
raffinons 972
raffinons 971
Traceback (most recent call last):
long int too large to convert to float
```

7.1.5 Peut on calculer avec les flottants ?

Nous allons revenir au problème qui a motivé notre étude et reprendre quelques calculs numériques avec les flottants. Observons tout d'abord que les nombre-machines sont des dyadiques et que l'affichage sous Python nous présente des décimaux. Cela suppose plusieurs transformations :

- conversion approchée des décimaux x_i donnés en entrée vers les flottants $fl(x_i)$;
- une succession d'opérations qui retourne un ou plusieurs flottants

$$y = fl(Algo(fl(x_1), \dots, fl(x_p)));$$

- et enfin conversion de ces flottants en décimaux.

Nous n'insisterons pas sur ces opérations de conversion qui ne sont pas source d'accumulation d'erreurs, mais sur les calculs eux-mêmes.

• **Calculs de sommes** $\sum_{k=0}^{n-1} \frac{1}{n}$

Calculons avec $n = 10$. Observons tout d'abord que $0.1 = \frac{1}{10}$ n'admet pas de représentation exacte en flottant : notre programme de l'exercice (7.1) nous permet de conjecturer que

$$\frac{1}{10} = \sum_{k=1}^{\infty} \left(\frac{1}{2^{4k}} + \frac{1}{2^{4k+1}} \right) = 0.0001100110011.._{(2)}$$

ce que que l'on démontre ensuite facilement en calculant les sommes pour k variant de 1 à n et passant à la limite. En conséquence $fl(0.1) = 0.1 - \varepsilon \neq 0.1$ où $\varepsilon = 0.1 - fl(0.1)$ est l'**erreur d'arrondi**. Par accumulation, on a $fl(s) \neq fl(1.0)$.

```

s=0.
for i in range(0,10):
    s=s+1/10
print(s)
print(s==1.)
print(s - 1.)
>>>
0.9999999999999999
False
-1.1102230246251565e-16

```

La situation est différente pour $\frac{1}{16} = (1.0 \times 10^{-100})_{(2)}$, pour lequel $fl(x) = x$:

```

s=0.                                >>>
for i in range(0,16):               1.0
    s=s+1/16                         True
print(s)                            0.0
print(s==1.)
print(s - 1.)

```

• Non associativité de l'addition des flottants

<pre> >>> a=-10**30; b=-a; c=1. >>> (a+b)+c 1.0 >>> a+(b+c) 0.0 >>> b+c 1e+30 </pre>	<pre> k = 0 a = 10.**k b, c = -a, 1.0 while (a+b)+c ==a+(b+c): k = k+1 a = 10.**k b,c = -a, 1. </pre>
--	---

Questions

1. Comment expliquer le résultat de la colonne de gauche où nous voyons que $(a + b) + c = 1.0$, alors que $a + (b + c) = 0.0$.
2. Cela aurait-il changé quelque chose si nous avions écrit $c = 1$ au lieu de $c = 1.0$ sous Python comme ici ?
3. La boucle du script de droite termine-t-elle ? Si oui, avec quelle valeur pour k ?

Réponses

1. Le calcul de $(a + b)$ est sans mystère ; par contre les flottants b et c n'ayant pas le même ordre de grandeur, tout change : b est un entier dont l'écriture demande 100 chiffres binaires (donné par $\ln(|b|)/\ln(2)$) on aura donc

$$fl(b) = -1.b_{-1}..b_{-52} \times 10_{(2)}^{100}$$

Pour calculer la somme avec $c = 1.0 \times 10_{(2)}^0$ le processeur aligne les deux nombres avec un exposant commun ce qui repousse le seul chiffre significatif de c au delà du 52^e bit : il disparaît donc et $(b + c)$ et b sont confondus (plus exactement $fl(fl(b) + fl(c)) = fl(b)$). C'est le **phénomène d'absorption**.

Illustrons le avec une addition décimale : supposons que l'on ne dispose que de 10 cases pour placer des chiffres et sommons $x = 7,891273641 \times 10^7$ et $y = 4.947653589 \times 10^{-1}$

$x =$	7	.	8	9	1	2	7	3	6	4	1	10^7
$y =$	4	.	9	4	7	6	5	3	5	8	9	10^{-3}

Quelle que soit la façon de procéder, en stockant le résultat avec une mantisse à 9 chiffres, x et $x + y$ seront confondus.

2. Certes oui ! $c = 1$ sous Python 3.x est un entier et les calculs sont faits en précision illimitée comme sous un logiciel de calcul formel.
3. Elle terminera dès que le chiffre significatif de c sera « relégué » au delà du 52^e bit de la mantisse. Cette frontière est atteinte avec $a = 10^{16}$, puisque $\ln 10^{15} / \ln 2 < 52 < \ln 10^{16} / \ln 2$.

- Équation du second degré

On veut calculer les racines d'une équation du second degré $ax^2 + bx + c = 0$ pour laquelle $\Delta > 0$. Un algorithme naïf transpose simplement la formule :

```
def racinesNaif(a,b,c):
    D = b**2-4*a*c
    if D>0:
        s=D**(1/2)
        return (-b-s)/(2*a),
            (-b+s)/(2*a)
```

Lorsque nous calculons avec $a > 0, b = \frac{1}{a}, c = -a$, nous obtenons :

a	x_1	x_2	$x_1 \times x_2$
0.7	-2.44912535699	0.408309030464	-1.0
0.07	-204.086532535	0.00489988235664	-0.999999999999
0.007	-20408.1633143	4.89999994865e-05	-0.999999991921
0.0007	-2040816.32653	4.8999027058e-07	-0.999980144041
7e-05	-204081632.653	1.29927814539e-08	-2.65158805182
7e-06	-20408163265.3	0.0	-0
7e-07	-2.04081632653e+12	0.0	-0

Les solutions sont, dans le monde des nombres réels,

$$x_1 = \frac{-1/a - \sqrt{1/a^2 + 4a^2}}{2a}, \quad x_2 = \frac{-1/a + \sqrt{1/a^2 + 4a^2}}{2a}.$$

Leur produit est égal à $c/a = -1$, ce qui est loin d'être le cas pour toutes les valeurs retournées. On observe que x_2 est calculée comme somme de deux flottants de signes opposés dont les valeurs absolues sont proches : il y a cette fois effacement

des chiffres significatifs puisque les bits de poids fort, identiques, sont **effacés** dans le calcul. On parle de **phénomène d’annulation**, ou, par anglicisme, de **phénomène de cancellation**. Le calcul de x_1 , dans le cas présent, ne devrait pas poser de problème. Nous allons le vérifier en modifiant notre mode de calcul.

Nous reprogrammons notre algorithme de façon à ne calculer que celle des deux racines qui ne comporte pas de termes de signes opposés et déduisons l’autre de la relation : $x_1 x_2 = c/a$. Le résultat est plus probant, les solutions sont correctes : il suffit de remarquer que $x_1 = x_2 \approx -b/a$ et bien sûr que $x_1 x_2 = c/a$.

```
def racines2(a,b,c):
    b1 = b/2
    D1 = b1**2-a*c
    if D1>0:
        s=D1**(1/2)
        if b1>0:
            x1 = (-b1-s)/a
            return x1, c/(a*x1)
        else:
            x2 = (-b1+s)/a
            return c/(a*x2), x2
```

a	x_1	x_2	$x_1 \times x_2$
0.7	-2.44912535699	0.408309030464	-1.0
0.07	-204.086532535	0.00489988235665	-1.0
0.007	-20408.1633143	4.89999998824e-05	-1.0
0.0007	-2040816.32653	4.9e-07	-1.0
7e-05	-204081632.653	4.9e-09	-1.0
7e-06	-20408163265.3	4.9e-11	-1.0
7e-07	-2.04081632653e+12	4.9e-13	-1.0
7e-08	-2.04081632653e+14	4.9e-15	-1.0

• Dérivation numérique

De nombreux algorithmes font intervenir les dérivées ou dérivées partielles d’une fonction numérique d’une ou plusieurs variables : méthode de Newton pour la recherche de solutions de $f(x) = 0$, méthodes d’optimisation... Lorsque la fonction n’est pas donnée par une formule explicite (tableau de données) ou lorsque cette formule est trop complexe (on peut rencontrer des expressions comportant des centaines de termes), il nous faudra évaluer numériquement cette dérivée. On peut penser à l’approcher par son taux de variation

$$\frac{d}{dx}f(x) \approx \frac{f(x+h)-f(x)}{h}$$

dont on sait qu’il est, dans le monde des réels, d’autant plus proche de la dérivée que h est proche de 0.

Regardons ce que cela donne dans le monde des flottants et choisissons pour cela une fonction simple, $f(x) = x^2$ dont la dérivée est connue et comparons :

Comparaison du taux de variation et de la dérivée

```
def derivee(f, x, h):
    return (f(x+h) - f(x))/h

def test_derivation(f, df, x):
    L = []
    for i in range(0,17):
        h = 10**(-i)
        L.append([h, derivee(f, x, h), df(x)])
    return L

f = lambda x: x**2
df = lambda x: 2*x
L = test_derivation(f, df, 4)
```

Le code qui précède nous donne les résultats qui figurent dans le tableau de droite. On observe que l’écart entre l’approximation du taux de variation $\frac{f(x+h) - f(x)}{h}$ et la valeur $2x$ atteint un optimum en $h = 10^{-8}$. L’ordre de grandeur de h est alors $\sqrt{r}$ où r est la précision de la machine. On peut expliquer cela en étudiant cet écart (voir [2]). On retiendra que pour calculer une dérivée le choix de h doit être de l’ordre $\sqrt{r}$ et que dans la plupart des cas (calculs d’intégrale résolution d’équations différentielles...), vouloir améliorer la précision d’un calcul en réduisant les pas conduit à faire exploser les erreurs d’arrondis.

h	approximation	2x
1	9.0	8
0.1	8.099999999999987	8
0.01	8.0099999999999806	8
0.001	8.00100000000037	8
0.0001	8.00009999998963	8
1e-05	8.00000999952033	8
1e-06	8.000000999430767	8
1e-07	8.000000129015916	8
1e-08	7.999999951380232	8
1e-09	8.000000661922968	8
1e-10	8.000000661922968	8
1e-11	8.000000661922968	8
1e-12	8.000711204658728	8
1e-13	8.029132914089132	8
1e-14	7.815970093361102	8
1e-15	7.105427357601001	8
1e-16	0.0	8

Polynômes de Rump

Le comportement d'un algorithme ou le résultat d'un simple calcul peuvent être très différents en arithmétique flottante de ce qu'ils seraient avec des nombres réels. Ces différences ne sautent pas aux yeux de prime abord, elles apparaissent dans certaines configurations seulement. Cela les rend plus difficilement prévisibles. Au fil des recherches sur ces questions sont apparus des objets permettant de mettre en évidence ce type de problèmes : suites convergentes dans le monde des nombres réels qui divergent dans le monde des flottants (exercice (7.4)), fonctions pour lesquelles les algorithmes d'optimisation divergent numériquement alors qu'ils convergent dans le monde des nombres réels...

Les polynômes de Rump (polynômes en x , fonctions rationnelles en y)

$$R(x, y) = \frac{1335}{4}y^6 + (11x^2y^2 - y^6 - 121y^4 - 2)x^2 + \frac{11}{2}y^8 + \frac{x}{2y} \quad (7.1.3)$$

que l'on étudie dans l'exercice (7.5), illustrent eux aussi ce genre de situations. Nous allons comparer les résultats de calculs exacts conduits avec le module de calcul formel **Sympy** à des calculs approchés avec des flottants.

• Calcul formel avec Sympy

```
import sympy as sp

def Rump(x, y):
    a = sp.Rational(1335, 4)
    b = sp.Rational(11, 2)
    return a*y**6+(11*x**2*y**2-y**6-121*y**4-2)*x**2\
           + b*y**8 +x/(2*y)

>>> a = sp.Rational(1335, 4); print(a)
1335/4
>>> X = sp.symbols('X')
>>> Y = sp.symbols('Y')
>>> R = Rump(X,Y); print(R, float(R))
-54767/66192 -0.8273960599468214
```

Commentaire : Sympy nous a permis de définir des rationnels, les variables locales a et b dans la fonction Python $Rump(x,y)$, la variable a dans le shell qui s'affiche comme $\frac{1335}{4}$, et des entiers X et Y .

Cela permet le calcul formel

$$\text{Rump}(77617, 33096) = \frac{-54767}{66192} \approx -0.8273960599468214.$$

• Calcul approché sous Python

```
def rump(x, y):
    a = 1335/4
    b = 11/2 # ce sont des flottants
    return a*y**6 + (11*x**2*y**2 - y**6 - 121*y**4 - 2)*x**2 \
           + b*y**8 + x/(2*y)

>>> x = 77617; y = 33096
>>> r = rump(x, y); print(r)
1.1805916207174113e+21
```

Commentaire : Le calcul sous Python se déroule du début à la fin avec des flottants. On voit que le résultat dans le monde des flottants est sans rapport avec celui que l'on obtient dans le monde des rationnels ou des réels.

7.1.6 Exercices

Exercice 7.4 une bien désagréable suite récurrente

On considère les suites numériques $(u_n)_n$ et $(f_n)_n$ telles que

$$\begin{cases} u_0 = 2, u_1 = -4 \\ u_{n+1} = 111 - \frac{1130}{u_n} + \frac{3000}{u_{n-1}u_n} \end{cases} \quad \text{et} \quad f_n = \frac{4 \times 5^{n+1} - 3 \times 6^{n+1}}{4 \times 5^n - 3 \times 6^n}.$$

1. Écrire une fonction $U(n)$, qui prend en argument un entier n et retourne u_n .
2. Écrire une fonction $F(n)$, qui prend en argument un entier n et retourne f_n .
3. Comparer les résultats pour n compris entre 0 et 30.
4. On peut démontrer (et ce n'est pas notre propos) que ces deux suites sont égales. Quelle est, de la fonction U ou de la fonction F , celle qui vous paraît donner les résultats les plus corrects ?

Corrigé en 7.4 page 306.

Exercice 7.5 polynômes de Rump

On reprend la fonction rationnelle de Rump défini par la formule (7.1.3) :

$$R(x, y) = \frac{1335}{4}y^6 + (11x^2y^2 - y^6 - 121y^4 - 2)x^2 + \frac{11}{2}y^8 + \frac{x}{2y}$$

Nous avons vu que l'évaluation de ce polynôme en x dans le monde des flottants pouvait se révéler étonnamment fantaisiste. On se propose de construire quelques outils pour explorer la situation. Nous allons travailler avec des entiers « illimités » de Python (ce qui suppose une version 3.3 ou supérieure !).

Ici $\text{rump}(x, y)$ désigne la fonction Python présentée dans l'encart de la page 299.

1. Nous souhaitons calculer $Rump(x, y)$ avec par exemple $y = 33096$ ce qui suppose de calculer y^8 . Ce nombre est-il représentable de façon exacte sur 64 bits ?
2. Pour profiter de la possibilité de travailler sur les entiers multi-précision, observons que le polynôme s'écrit encore

$$\frac{1}{4} (22y^8 - 4y^6x^2 + 1335y^6 - 484y^4x^2 + 44y^2x^4 - 8x^2) + \frac{x}{2y}$$

Écrire une fonction $F(x, y)$ qui permet le calcul exact du facteur entre parenthèses lorsque x et y sont des entiers.

En déduire une fonction $rump1(x, y)$ qui calculera pour (x, y) entiers, $R(x, y)$ avec une précision raisonnable. Comment justifierez-vous que votre calcul est fiable ?

Calculer $rump1(x, y)$ lorsque $x = 77617, y = 33096$, et comparer à ce que donne $rump(x, y)$.

3. On souhaite explorer numériquement la situation.
Pour cela on répète l'expérience qui consiste à tirer au hasard deux entiers (p, q) (attention : $q \neq 0$) et calculer $R(p, q)$ avec chacune de nos fonctions. On évalue l'écart entre les valeurs qu'elles retournent.
 - (a) Utiliser **randint** du module **random** pour écrire une fonction qui retourne un couple d'entiers $(p, q) \in \mathbb{Z} \times \mathbb{Z}^*$. On fera progressivement évoluer la borne supérieure selon les résultats expérimentaux.
 - (b) Écrire ensuite une fonction qui lance un certain nombre de tirages et stocke les résultats dans une liste dont les éléments sont sous la forme $[(p, q), z_1, z_2, e]$ (z_1 et z_2 les valeurs obtenues, $e = |z_1 - z_2|$ l'écart).
 - (c) Tracer une « carte » colorée en fonction de la valeur de l'écart (plot avec des $\bullet$).

Corrigé en 7.5 page 307.

Exercice 7.6 approximation laborieuse de $\ln 2$

Il n'est pas indispensable, pour entreprendre cet exercice d'informatique numérique d'avoir vu les séries en cours de maths : nous donnons ici tout ce qu'il faut savoir pour l'aborder.

L'objectif est d'évaluer de façon pertinente les termes de la série numérique définie par

$$H_n = \sum_{k=1}^n \frac{(-1)^{k-1}}{k} = 1 - \frac{1}{2} + \frac{1}{3} - \dots + \frac{(-1)^{n-1}}{n}.$$

On démontre et nous admettons que :

pour tous $m, n \in \mathbb{N}^*$, si $m \leq n$, $H_{2m} \leq H_{2n} \leq \ln 2 \leq H_{2n+1} \leq H_{2m+1}$.

pour tout $n \in \mathbb{N}^*$, $|H_n - \ln 2| \leq \frac{1}{n+1}$.

1. Nous proposons dans la colonne de gauche deux fonctions $H1(n)$ et $H2(n)$. Y aurait-il des différences entre $H1(2n)$ et $H2(n)$ dans le monde des nombres réels ?

<pre>import numpy as np import time as t def H1(n): s = 0 for k in range(1,n+1): s = s+(-1)**(k+1)/k return s def H2(n): s = 0 for p in range(1,n+1): s = s+1/((2*p-1)*p) return s/2</pre>	<pre>n = int(input('n= ')) for p in range(n, n +2): t0 = t.time() print(H1(2*p)) print(H2(p)) print(H1(2*p+1)) print(np.log(2)) print('t= ',t.time()-t0) print('-----')</pre>
--	--

2. Calculer le nombre maximal de chiffres *décimaux* significatifs pour les flottants en IEE 754 (la mantisse de $fl(x)$ étant codée sur 52+1 bits).
3. Le script de droite donne les résultats présentés ci-dessous. Sont-ils conformes à ce que l'on sait des propriétés de cette suite dans le monde des réels ? Pensez-vous qu'une (ou deux) des suites de flottants $(H1(2n))_n$ et $(H2(n))_n$ soit stationnaire à partir du rang 10^8 ?

<pre>>>> n= 1000 0.6928972430599403 0.6928972430599386 0.6933969931848778 0.69314718056 t= 0.046000003814697266 ----- 0.6928974926853773 0.6928974926853756 0.6933967438086923 0.69314718056 t= 0.016000032424926758</pre>	<pre>>>> n= 100000000 0.6931471780606475 0.6931471773731642 0.6931471830606475 0.69314718056 t= 294.029000043869 ----- 0.6931471780606475 0.6931471773731642 0.6931471830606475 0.69314718056 t= 295.69799995422363</pre>
---	--

7.2 Corrigés des exercices

Corrigé de l'exercice n° 7.1

```
def conv10to2Int(n):
    v      =      abs(n)
    L, p   =      list([]), 0
    if v==0:
        L.append(0)
    else:
        while v > 0:
            v, r = divmod(v, 2)
            L.insert(0, r)
            if n < 0:
                L.insert(0, '-')
        return L
```

```
def conv10to2Frac(f, eps):
    p, g = 0, f
    L     = list(['.'])
    while 2**(-p) > eps:
        p = p+1
        if 2*f < 1:
            L.append(0)
            f = 2*f
        else:
            L.append(1)
            f = 2*f-1
    return L
```

```
def conv2to10(L):
    pt = '.'
    p = len(L)
    if pt in L:
        p = L.index(pt)
    e = [ L[k]*2**(p-1-k) for k in range(0,p) ]
    f = [ L[k]*2**(p-k) for k in range(p+1,len(L)) ]
    return sum(e)+sum(f)
```

Corrigé de l'exercice n° 7.2

La fonction `add` permet de faire la somme de deux entiers positifs de taille quelconque puisqu'elle ajuste la plus petite des listes de bits en la faisant précéder de zéros.

```
def addbits(b1,b2,r):
    """ajout de b1 et b2 avec retenue"""
    """retourne (bit courant, retenue)"""
    c = b1+b2+r
    if c <= 1:
        return (c, 0)
    elif c==2:
        return (0, 1)
    else:
        return (1,1)
#-----

def add(L1,L2):
    n1, n2 =len(L1), len(L2)
    if n1<n2:
        for i in range(0,n2-n1):
            L1.insert(0,0)
    elif n2 < n1:
        for i in range(0,n1-n2):
            L2.insert(0,0)
    L = list([])
    retenue = 0
    for i in range(len(L1),0,-1):
        bnew, retenue = addbits(L1[i-1], L2[i-1], retenue)
        L.insert(0, bnew)
    if retenue ==1:
        L.insert(0, retenue)
    return L
#-----

def compla2(L):
    """complément à 2 d'une liste"""
    CL=list([])
    for i in L :
        if i ==0:
            CL.append(1)
        else:
            CL.append(0)
    return add(CL, [1])
```

```

def add_n(L1,L2):
    """L1, L2 de même taille """
    n1, n2 =len(L1), len(L2)
    assert n1 == n2

    L = list([])
    retenue = 0
    for i in range(len(L1),0,-1):
        bnew, retenue = addbits(L1[i-1], L2[i-1], retenue)
        L.insert(0, bnew)
    assert retenue == 0
    return L

def conv10to2Z(x, n):
    """Conversions des relatifs"""
    L = conv10to2Int(abs(x))
    assert len(L) <= n-1

    p = len(L)
    while p<n:
        L.insert(0,0)
        p = p+1

    if x >=0:
        return L
    else:
        return compla2(L)

```

Corrigé de l'exercice n° 7.3

1. Notons p l'indice du plus grand chiffre non nul de X en base β . On a

$$\beta^p \leq X = \sum_{k=0}^p b_k \beta^k \leq \sum_{k=0}^p (\beta - 1) \beta^k = \beta^{p+1} - 1$$

$$\beta^p \leq X < \beta^{p+1}$$

$$p \leq \log_{\beta} X < p + 1$$

$$\log_{\beta} X - 1 < p \leq \log_{\beta} X$$

Comme le dernier intervalle d'encadrement est de longueur strictement plus petite que 1, il ne contient qu'un seul entier et $p = \lfloor \log_{\beta} X \rfloor$. Le nombre de chiffres pour représenter X en base β est donc $\lfloor \log_{\beta} X \rfloor + 1$.

2. Dans le script qui suit, chaque chiffre ou terme des listes X ou Y est majoré par $\beta - 1$. $X[k] + Y[k] + q$ sera majorée par $2\beta - 2 + q \leq 2\beta - 1$ car ici $q = 0$ ou 1 (pourquoi ?). La contrainte sur β est donc $\beta \leq M/2 + 1/2$ où M est le plus grand des entiers représentables dans le langage.

La complexité est clairement linéaire en $\max(n, n)$ soit $O(\max(\log_\beta X, \log_\beta Y))$.

```
def addition(X, Y, beta):
    '''
    X, Y : list;
    '''
    m, n = len(X), len(Y)
    if m > n:
        X, Y = Y, X
    Z, q = [], 0 # q est la retenue
    for k in range(0, min(m,n)):
        z = X[k] + Y[k] + q # q est la retenue
        q, r = divmod(z, beta)
        Z.append(r)
    for k in range(min(m,n), max(m,n)):
        z = Y[k] + q # q est la retenue
        q, r = divmod(z, beta)
        Z.append(r)
    if q > 0:
        Z.append(q)
    return Z
```

3. Dans le cas de cette fonction $a \leq \beta - 1$ et les « chiffres » $X[i]$ sont aussi majorés par $\beta - 1$; $z = a * c + q \leq (\beta - 1)^2 + q \leq (\beta - 1)^2 + \beta - 1 < \beta^2$.

Ainsi si $\beta^2 < M$, les entiers manipulés dans l'algorithme resteront représentables dans $\mathcal{L}$. Par ailleurs la complexité de la fonction est linéaire en $\text{len}(X) = \lfloor \log_\beta(x) \rfloor = 1$.

```
def multiplicationPG(a, p, X, beta):
    assert a < beta
    Z, q = [0 for i in range(0,p)], 0
    for c in X:
        z = a*c + q
        q, r = divmod(z, beta)
        Z.append(r)
    if q > 0:
        Z.append(q)
    return Z
```

`multiplication(X,Y, beta)` appelle `multiplicationPG(*, *, Y, beta)` $\text{len}(X)$ fois et `addition(P, *, beta)` $\text{len}(X)-1$ fois. la complexité est donc de $\text{len}(X) \times \text{len}(Y)$ multiplications de petits entiers.


```
def multiplication(X, Y, beta):
    '''
    X, Y : list
    '''
    assert isinstance(X, list) and len(X) > 0
    assert isinstance(Y, list) and len(Y) > 0

    P = multiplicationPG(X[0], 0, Y, beta)
    for p in range(1, len(X)):
        P = addition(P, \
                     multiplicationPG(X[p], p, Y, beta), beta)
    return P
```

Corrigé de l'exercice n° 7.4

```
def U(n):
    '''
    n: int
    '''
    u, v = 2, -4
    if n == 0:
        return u
    elif n == 1:
        return v
    else:
        for i in range(1, n):
            u, v = v, 111-1130/v + 3000/(u*v)
        return v

def F(n):
    c, s = 5**n, 6**n
    return (4*5*c-3*6*s)/(4*c -3*s)

def test_UF(N):
    L = []
    for n in range(0, N+1):
        L.append([n, U(n), F(n)])
    return L
```

Si, comme l'énoncé nous le dit, ces suites sont identiques, c'est le programme $U(n)$ qui accumule visiblement les erreurs de calcul. En effet, il semble converger vers 100 (point fixe numérique si on poursuit) alors que $F(n)$ semble avoir, comme la suite de réels qu'elle est sensée représenter, une limite égale à 6 (mettre en facteur 6^n au numérateur et au dénominateur) :

n	U(n)	F(n)
0	2	2.0
1	-4	-4.0
2	18.5	18.5
3	9.378378378378379	9.378378378378379
4	7.801152737752169	7.801152737752162
5	7.154414480975333	7.154414480975249
6	6.806784736924811	6.806784736923633
7	6.592632768721792	6.592632768704439
...	...	...
14	6.120248704570159	6.11588306655108
15	6.166086559598099	6.094739439333681
16	7.235021165534931	6.077722304847243
17	22.062078463525793	6.063940322499809
18	78.57557488787224	6.052721761016152
19	98.34950312216536	6.043552110189269
20	99.8985692661829	6.036031881081857
21	99.99387098890278	6.029847325023902
...	...	...
28	99.99999999998246	6.008154378912229
29	99.9999999999893	6.006786093031206
30	99.9999999999993	6.00564868877142

Corrigé de l'exercice n° 7.5

1. Le calcul avec des entiers codés sur 64 bits serait impossible puisque

$$33096^8 = 1439474789212538429291115400277262336 > 2^{63} - 1 = 9223372036854775807$$

Le calcul en flottants utilise l'approximation :

$$33096^8 = 1.4394747892125385e + 36.$$

2. Écriture naïve, vérification pour des petites valeurs de x et de y .

```
def rump(x,y):
    return (1335/4)*y**6
        + (11*x**2*y**2-y**6-121*y**4-2)*x**2
        + (11/2)*y**8 + x/(2*y)

>>> rump(1,1)
226.75
>>> a = 77617
>>> b = 33096
>>> rump(a,b)
1.1805916207174113e+21
```

Comme $R(x, y)$ s'écrit

$$\frac{1}{4} (22y^8 - 4y^6x^2 + 1335y^6 - 484y^4x^2 + 44y^2x^4 - 8x^2) + \frac{x}{2y}$$

l'idée est de calculer

$$F(x, y) = 22y^8 - 4y^6x^2 + 1335y^6 - 484y^4x^2 + 44y^2x^4 - 8x^2$$

qui donnera **sur les entiers de Python 3.x** le calcul exact (sauf dépassement de la pile en cours de calcul), puis, $R(x, y) = \frac{1}{4} \left(F(x, y) + \frac{2x}{y} \right)$. Seul le dernier calcul se fera en flottants. Vérifions :

```
def F(x,y):
    return 22*y**8+1335*y**6-4*x**2*y**6
        -484*x**2*y**4+44*x**4*y**2-8*x**2

def rump1(x,y):
    return (F(x,y)+2*x/y)/4
```

```
>>> rump(1,1)
226.75
>>> rump1(1,1)
226.75
>>> a, b = 77617, 33096
>>> rump(a,b)
1.1805916207174113e+21
>>> rump1(a,b)
-0.8273960599468213
```

Ce calcul est fiable puisque le calcul $F(a, b)$ est **exact** et on sait que le reste des calculs se fera sans problème (on ajoute $2a/b$ et on divise par 4).

3.

```
from random import randint
import pylab as pl

def couple(d):
    '''
    retourne un couple aléatoire d'entiers relatifs
    (p,q) tels que -d<=p,q<=d et q != 0
    '''
    p,q = randint(-d,d), randint(-d,d)
    while q==0:
        q=randint(-d,d)
    return (p,q)
```

```
def cartographie(d, N):
    ''' Construit une liste de N élts. de la forme:
    [(p,q), z1, z2, |z1-z2|]
    avec: (p,q) point de [-d,d]x[-d,d]
           z1 = rump1(p,q) et z2 = rump(p,q) '''
    L = list([ [(0,1), rump1(0,1), rump(0,1), \
                    (rump1(0,1) - rump(0,1))] ])
    for k in range(0,N):
        p, q = couple(d)
        z1, z2 = rump1(p,q), rump(p,q)
        L.append([(p, q), z1, z2, abs(z1-z2)])
    return L
```

```
def dessiner(carte):
    '''carte est une liste retournée par cartographie'''
    for e in carte:
        p,q = e[0]
        if e[3] < 0.001:
            pl.plot([p,p], [q,q], 'ro', color = 'grey')
        else:
            pl.plot([p,p], [q,q], 'ro', color = 'black')
    pl.show()
    pl.close()
```

Corrigé de l'exercice n° 7.6

1. Dans le monde des réels (celui des bisounours où tout va bien, même si parfois le cours d'analyse peut vous paraître bien subtil), nous avons

$$H_{2n} = \sum_{k=1}^{2n} \frac{(-1)^{k+1}}{k} = \sum_{p=1}^n \left(\frac{1}{2p-1} - \frac{1}{2p} \right) = \frac{1}{2} \sum_{p=1}^n \frac{1}{p(2p-1)}.$$

Les deux appels $H1(2n)$ et $H2(n)$ retournent les mêmes valeurs dans le monde des nombres réels.

2. Si $10^n \leq m < 10^{n+1}$ x s'écrit avec $n+1$ chiffres en base 10 avec $n = \lfloor \ln m / \ln 10 \rfloor$. Le nombre de chiffres de m en base 2 vérifie $2^p \leq m < 2^{p+1}$ et ici $p+1 = 53 = \lfloor \ln m / \ln 2 \rfloor$. On en déduit

$$n+1 = \frac{\ln m}{\ln 10} = \frac{\ln m \ln 2}{\ln 2 \ln 10} = 53 \frac{\ln 2}{\ln 10} \approx 16.9$$

3. Dans le tableau des résultats, $H1(2n)$ et $H2(n)$ qui seraient égaux dans $\mathbb{R}$ montrent des différences dues à la façon dont sont menés les calculs. Mais on garde l'encadrement $H_{2n} \leq \ln 2 \leq H_{2n+1} \dots$

À partir de $n = \frac{10^{-8}}{\sqrt{2}}$ la sommation d'un terme supplémentaire $\frac{1}{p(2p-1)}$ dont le premier chiffre binaire significatif sera au delà du 54^e chiffre après la virgule ne changera pas la valeur $H2(n)$ qui est constante à partir de ce rang dans le monde des flottants.

Pour ce qui est de $H1(n)$ on la modifie en ajoutant à un nombre dont l'écriture binaire est 0.1... un nombre dont le premier chiffre binaire significatif est en 25^e position au moins... La somme sera modifiée jusqu'à $n = 10^{16}$ environ. Une estimation précise des erreurs d'arrondi serait certainement délicate.

Preuves et complexité des programmes

La partie I de ce manuel vous a permis de rencontrer des algorithmes généraux qui constituent un socle à partir duquel vous allez étendre progressivement vos compétences : concevoir la solution d'un nouveau problème, repérer les éléments qui vous assureront que votre solution est correcte, évaluer son coût...

Dans ce chapitre nous proposons une synthèse sur l'analyse des programmes à travers l'étude de quelques exemples et en précisant le vocabulaire pour vous permettre de consolider ces acquis. Vous pourrez ensuite revisiter les thèmes déjà abordés dans la première partie de ce cours et reprendre les preuves de programmes ou les études de complexité que vous n'auriez pas menées au bout dans une première lecture.

8.1 Spécification d'un algorithme

8.1.1 Le vocabulaire

Définition 8.1 *signature et spécification d'une fonction ou d'un algorithme,*

Établir la **signature** d'une fonction ou d'un algorithme consiste à

- la nommer (ou le nommer) ;
- préciser ses arguments (ou paramètres formels) et leurs types ;
- préciser le type des valeurs renvoyées par la fonction.

Spécifier un algorithme consiste à préciser sa signature et à lui ajouter :

- Les **pré-conditions** que ses arguments doivent satisfaire ;
- les **post-conditions** portant sur ces valeurs.

• Exemples

1. La signature d'une fonction qui prend comme arguments deux entiers et renvoie le pgcd de ces deux entiers est donc, en adoptant les types de Python, $\text{pgcd} : (a : \text{int}, b : \text{int}) \rightarrow \text{int}$;

La spécification d'une telle fonction suppose que l'on précise une pré-condition, par exemple $a > b \geq 0$ et une post-condition, à savoir que la valeur renvoyée est bien le pgcd de a et de b .

2. La spécification d'une fonction ne dit rien sur les moyens, elle ne décrit pas un algorithme.

Ainsi, les deux fonctions pgcd étudiées dans les exercices 8.11 et 8.17 ont les mêmes signatures et les mêmes spécifications, pourtant les algorithmes mis en place ne sont pas les mêmes.

3. La signature d'une fonction qui prend comme arguments deux entiers et qui renvoie un triplet (d, u, v) où d est le pgcd de a et b et u, v est un couple d'entiers tels que $au + bv = d$ pourrait être

$$(a : \text{int}, b : \text{int}) \rightarrow (d : \text{int}, u : \text{int}, v : \text{int}).$$

La valeur de retour est un tuple (triplet) d'entiers.

Une pré-condition est $a > b \geq 0$ et une post-condition est d est le pgcd de a et b et $ua + bv = d$. Cela nous donne encore une spécification.

Il est impératif de programmer en renseignant de façon systématique la signature de vos fonctions et si possible leur spécification. Si, dans ce livre, nous ne le faisons pas toujours en présentant un algorithme ou dans les corrigés des exercices c'est une question de place et parce que **signature et spécification figurent déjà** dans l'énoncé ou dans la description de l'algorithme.

8.1.2 Vérifier les pré-conditions et les post-conditions

Un programme se construit en plusieurs étapes et il peut être utile de vérifier chaque composant au fur et à mesure de l'élaboration de l'ensemble.

• La vérification du type des arguments.

Bien que Python adopte un **typage dynamique** (ie : le type d'une variable est déterminé au moment de l'affectation) la vérification systématique du type des arguments d'une fonction est une bonne habitude.

On dispose pour cela de la fonction à valeurs booléennes **isinstance(e, t)** dont la signature est $isinstance : (e : \text{objet}, t : \text{type} | \text{tuple_de_types}) \rightarrow \text{bool}$

et qui renvoie True ssi e est une expression dont le type est soit le second argument soit un élément du tuple donné en second argument.

```
>>> type(12)
<class 'int'>
>>> type('chaine')
<class 'str'>
>>> isinstance('bonjour!', str)
True
>>> isinstance('bonjour!', int)
False
>>> isinstance('bonjour!', (int, str))
True
```

• La vérification des pré-conditions et post-conditions

On dispose pour cela de l'instruction **assert** dont la syntaxe est `assert condition`. Si *condition* est évaluée à True, le programme continue, sinon une exception est levée et le programme est interrompu.

```
def pgcd(a,b):
    '''
    a, b :int avec la pré-condition
        a>=0 et b>= 0 et (a>0 ou b >0);
    Renvoie le pgcd de a et de b.
    '''
    # vérification de la pré-condition
    assert isinstance(a, int) and isinstance(b, int)
    assert a>=0 and b>=0 and (a>0 or b>0)

    r0 , r1 = a, b
    while r1 !=0:
        r0, r1 = r1, divmod(r0, r1)[1]
    return r0

>>> pgcd(12,7)
1
>>> pgcd(7,12)
1
>>> pgcd(13, 0)
13
```



```
>>> pgcd(0, 0)
Traceback (most recent call last):
  File ".../essais.py", line 23, in <module>
    print(pgcd(0,0))
  File ".../essais.py", line 11, in pgcd
    assert a>=0 and b>=0 and(a>0 or b>0)
AssertionError
```

Il est clair que cette instruction ne doit figurer que dans un programme en construction ou pendant sa phase de test. Elle devra être remplacée soit par une gestion des erreurs (**try except (else) (finally)**), ce qui n'est pas au programme) soit par une étude de cas dans un programme définitif.

Exercice 8.1 *deboggage*

On rencontre un problème dans un programme qui renvoie des listes de listes (par exemple les sous-listes de taille k d'une liste donnée) : les sous-listes que ce programme retourne n'ont pas toujours le même nombre d'éléments que ce qui est attendu. Pour déboguer, on veut arrêter le programme au moment où le problème apparaît pour la première fois.

1. Écrire une fonction `check(L, n)` qui prend une liste `L` et un entier positif `n` en argument, renvoie `True` si tous les éléments de `L` sont des listes de longueur n . Il lève l'exception `AssertionError` sinon.
2. On voudrait visualiser l'origine du problème avant le clash final. C'est là que **try ... except** joue son rôle.

Écrire une fonction `check1(L,P, message)` (`message` : str) qui réalise le test précédent dans une clause `try` et **rattrape** l'exception (ou erreur) `AssertionError` pour signaler (avec `print()`) qu'une erreur est décelée (elle affichera aussi le message donné en argument (par exemple 'Test1 demandé ligne 1067 a échoué'). Cette fonction renverra `False` en cas d'erreur éventuellement.

Corrigé en 8.1 page 340.

8.1.3 Exemples

Exercice 8.2 D'après CCINP MP - Maths 1 - 2020.

Le problème dont l'énoncé figure partiellement page 316 introduit le développement ternaire (en base 3) d'un réel $x \in [0, 1[$ avant de poser quelques questions de programmation. Nous reprenons les questions d'informatique pour illustrer la nécessité d'une programmation défensive qui ne pouvait apparaître sur le papier.

1. Voici une réponse proposée à la question 3 de ce problème.

```
def flottantVersTernaire(n, x):
    p, L = 1, []
    for i in range(1, n+1):
        u = p*x # ???
        t = int(3*u)-3*int(u)
        L.append(t)
        p = 3*p
    return L

>>> flottantVersTernaire(4, 0.5)
[1, 1, 1, 1]
```

- (a) Compléter le code pour donner la spécification du programme.
- (b) Déterminer la valeur de u lors de la $i^{\text{ème}}$ itération. Commentez le code pour le faire savoir au lecteur.
- (c) Que donne ce programme si on appelle
 $L = \text{flottantVersTernaire}(5, 1)$?
- (d) Utiliser une ou plusieurs instructions `assert` pour interrompre ce programme lorsque les pré-conditions sur n et sur x ne sont pas remplies.
- (e) On a montré avec la question 1 de ce problème que dans le monde des réels les $t_n(x)$ sont dans $\{0, 1, 2\}$. Mais si nous appelons
 $L = \text{flottantVersTernaire}(50, 1/3)$, nous obtenons une liste dont les premiers termes nous semblent corrects alors que tout se gâte après le $36^{\text{ème}}$ (qui est égal à -8 !) :
 $[1, 0, 0, 0, \dots, -8, 0, 64, 256, 0, 2048, 0, 0, 65536, 0, 524288, 1048576, -8388608, 0]$
 - Comment peut on expliquer ce comportement ?
 - Utilisez une instruction `assert` pour stopper le programme dès que le résultat ne vérifie plus les post-conditions.

2. Écrire la fonction `ternaireVersFlottant(ell)` dans le même esprit (spécification, commentaires et usage de `assert` pour vérifier les pré-conditions).

Corrigé 8.2 page 341.

Le sujet posé (questions 6 à 9)

1. Montrer que si $x \in [0, 1[$ et $n \in \mathbb{N}^*$,

$$t_n(x) = \lfloor 3^n x \rfloor - 3 \lfloor 3^{n-1} x \rfloor \in \{0, 1, 2\}.$$

2. Montrer que les suites $(x_n)_{n \in \mathbb{N}^*}$ et $(y_n)_{n \in \mathbb{N}^*}$ définies par

$$x_n = \frac{\lfloor 3^n x \rfloor}{3^n} \text{ et } y_n = x_n + \frac{1}{3^n}$$

sont adjacentes et que

$$\sum_{n=1}^{\infty} \frac{t_n(x)}{3^n} = x.$$

3. Écrire en langage Python une fonction `flottantVersTernaire(n, x)` d'arguments un entier naturel n et un flottant x et qui renvoie sous forme de liste les n premiers chiffres $t_1(x), t_2(x), \dots, t_n(x)$ définis dans la question précédente du développement ternaire de x .

Par exemple `flottantVersTernaire(4, 0.5)` renvoie `[1, 1, 1, 1]`.

4. Si $\ell = [\ell_1, \dots, \ell_n]$ est une suite finie d'entiers de $\{0, 1, 2\}$ on pose $\sigma(\ell) = \sum_{k=1}^n \frac{\ell_k}{3^k}$.

Écrire en langage Python une fonction `ternaireVersFlottant(l)` qui prend comme argument une liste d'entiers appartenant à $\{0, 1, 2\}$ et renvoie le flottant représentant $\sigma(\ell)$.

Par exemple, `ternaireVersFlottant([1, 1, 1, 1])` renvoie `0.493827.....`

8.2 Le point sur la notion de preuve d'un algorithme

Nous avons vu comment prouver qu'un programme termine et qu'il est conforme à sa spécification. Dans la plupart des exemples que nous avons rencontrés, nous avons établi la preuve par récurrence sur le numéro d'une itération (ou d'un appel récursif). Dans le cas d'une boucle `for` dont l'itérateur n'est pas modifié par le processus, la situation est en général claire (le tableau de la page 36 donne une liste de situations où la boucle `for` est une traduction directe de la récurrence et où il n'y a rien à démontrer). Dans le cas d'une boucle `while`, la preuve peut par contre être délicate : un invariant de boucle ou une preuve de terminaison ne sont pas toujours faciles à trouver.

Le programme `syracuse(a0)` présenté ci-dessous illustre bien cette difficulté. Il calcule les termes successifs de la suite récurrente définie par $u_0 = a_0 \in \mathbb{N}^*$ et

$$\begin{cases} u_{n+1} = u_n/2 & \text{si } n \text{ est pair,} \\ u_{n+1} = 3u_n + 1 & \text{si } n \text{ est impair.} \end{cases} \quad (8.2.1)$$

et s'arrête dès qu'il rencontre un terme égal à 1.

Alors qu'en pratique, la suite $(u_n)_n$ semble ultimement périodique et se terminer par le cycle 4,2,1, personne à ce jour (nous sommes en 2021), ne sait le démontrer et on ne sait donc pas non plus montrer que le programme termine :

Temps de vol d'une suite de Syracuse

```
def syracuse(a0):
    '''
    a0: int; a0 >0.
    ???
    '''
    u, i = a0, 0
    while u !=1:
        q, r = divmod(u, 2)
        i = i+1
        if r == 0 :
            u =q
        else:
            u =3*u+1
    return i
```

Le problème mathématique sous-jacent, à savoir, démontrer que *pour tout entier strictement positif, il existe un rang n tel que pour tout entier $p \geq 0$,*

$$u_{n+3p} = 4, u_{n+3p+1} = 2, u_{n+3p+2} = 1,$$

est toujours ouvert bien qu'il ait été posé dès 1937 par le mathématicien allemand Lothar Collatz¹.

Pour vous permettre de vous entraîner, nous allons tout simplement vous soumettre quelques courts problèmes et algorithmes avec lesquels vous vous exercerez à construire des (in)variants de boucles, des preuves de correction et de terminaison. Nous commençons par préciser le vocabulaire.

1. Depuis la première édition de ce livre, une avancée remarquable a été obtenue par Terence Tao, qui semble malgré tout encore penser que la résolution du problème n'est pas pour demain.

Définition 8.2 *vocabulaire*

- **Variant de boucle (ou fonction de terminaison)** : c'est une fonction F positive dont les arguments sont les variables associées à l'algorithme et qui croît ou décroît strictement à chaque itération. Un variant de boucle permet de prouver qu'un algorithme termine²
- **Correction partielle d'un algorithme** : un algorithme est partiellement correct s'il est conforme à sa spécification dès lors qu'il termine.
- **Correction totale d'un algorithme** : un algorithme est correct s'il termine et s'il est partiellement correct.
- **Invariant de boucle** : c'est une propriété logique $\mathcal{P}$ portant sur l'ensemble des variables de l'algorithme, qui est vraie avant une boucle et reste vraie après chaque itération. Les invariants de boucle permettent de prouver la correction (partielle) des algorithmes itératifs (ou récursifs).

Exercice 8.3 *suite de Syracuse-Collatz*

On s'intéresse au programme « Temps de vol d'une suite de Syracuse »

1. Pouvez vous en donner une spécification ?
2. Donner un invariant de boucle et prouver sa correction partielle.

Corrigé en 8.3 page 342.

Exercice 8.4 *divisibilité par 7*

On considère le procédé suivant :

Soit $m \in \mathbb{N}$, que l'on suppose écrit en base 10. On enlève à l'écriture décimale de m le chiffre des unités et on le retranche deux fois au nombre ainsi obtenu. On note m' le résultat de cette opération.

Par exemple :

$$31976 \rightarrow 3197 - 2 \times 6 = 3185 \rightarrow 318 - 2 \times 5 = 308 \rightarrow 30 - 2 \times 8 = 14.$$

1. Écrire une fonction Python qui réalise l'opération $m \rightarrow m'$ ainsi décrite (avec la fonction `divmod` par exemple). On la notera f .
2. Prouver que le mini-programme qui suit termine lors d'un appel avec m entier.

```
def critere7(m):
    while m > 99:
        m = f(m)
    return m
```

3. Démontrer que `critere7(m)` renvoie un nombre divisible par 7 ssi m est lui-même divisible par 7.

Corrigé en 8.4 page 343.

2. En pratique nous considérerons une fonction $F(k)$ dépendant de l'état du système après l'itération k .

Exercice 8.5 *le jeu de Nim*

On considère une variante du jeu de Nim dont la règle est la suivante :

- on dispose N jetons identiques sur une table ;
- deux joueurs A et B jouent tour à tour ;
- chacun d'eux doit, lorsque c'est son tour, prélever soit un, soit deux, soit trois jetons ;
- le joueur qui enlève le dernier jeton a perdu.

La structure d'un programme simulant une partie est la suivante :

```
def Nim(n)
    N = n
    while N != 0:
        N = Ajoue(N)
        if N == 0:
            r = 'B gagne'
        else:
            N = Bjoue(N)
            if N == 0:
                r = 'A gagne'
    return r
```

1. Prouver que, quelles que soient les fonctions Ajoue et Bjoue, si les règles sont respectées, la procédure termine.
2. Donner les valeurs de N pour lesquelles Ajoue peut être programmée de telle sorte que Ajoue(N) réalise son dernier appel et que le joueur A gagne la partie.
3. Prouver qu'il existe une fonction Ajoue qui garantit que pour certaines valeurs de n A gagne (on écrira Ajoue et on prouvera l'existence d'un **invariant de boucle**).
4. Programmer Bjoue pour que le joueur B gagne dans les autres cas (et quelque soit la façon de programmer Ajoue).

Corrigé en 8.5 page 343.

Algorithme de Floyd

On considère un système dynamique discret, c'est-à-dire une suite $(x_n)_{n \in \mathbb{N}}$ d'éléments d'un ensemble E qui vérifie une relation de récurrence $x_{n+1} = f(x_n)$ pour une certaine fonction $f : E \rightarrow E$.

On dit qu'une telle suite est ultimement périodique de période $p \in \mathbb{N}^*$ s'il existe un entier N tel que, pour $n \geq N$, $x_{n+p} = x_n$.

L'algorithme de Floyd que nous allons étudier dans l'exercice qui suit,

- prend en entrées une fonction $f : E \rightarrow E$ et un élément $x_0 \in E$ définissant une suite récurrente ultimement périodique ;
- retourne un entier i tel que $x_{2i} = x_i$.

Description

Initialisation :

$i = 1$; $xi = f(x_0)$; $x2i = f(f(x_0))$;

Tant que $xi \neq x2i$:

$i = i + 1$; $xi = f(x_i)$; $x2i = f(f(x_{2i}))$;

Exercice 8.6 analyse de l'algorithme de Floyd

1. Pouvez vous donner un exemple de couple (f, x_0) pour lequel
 - la suite n'est pas périodique et le programme s'arrête ?
 - le programme ne s'arrête pas ?
2. Dans l'algorithme tel qu'il est décrit ci-dessus, préciser les contenus des variables xi et $x2i$ au moment de l'évaluation de la condition $C = (xi \neq x2i)$ en fonction de k , le nombre d'itérations de la boucle déjà réalisées.
3. Montrer que si la suite $(x_n)_{n \geq N}$ est périodique de période p , alors pour tout $n \geq N$, et pour tout $k \in \mathbb{N}$, $x_{n+kp} = x_n$.
En déduire que si la suite $(x_n)_n$ est ultimement périodique l'algorithme termine.
4. Justifier que lorsque l'algorithme termine i est un multiple de p .
5. Écrire une fonction Python, $\text{Floyd}(f, x_0)$, qui implémente l'algorithme de Floyd. On pourra la tester avec les fonctions suivantes :
 - (a) f est l'application qui à $x \in \llbracket 0, 22 \rrbracket$ associe le reste dans la division euclidienne de $x^2 + 1$ par 23.
 - (b) g l'application qui à $x \in \llbracket 0, 100 \rrbracket$ associe le reste dans la division de $x^3 - x^2 + 1$ par 101...

On choisira pour chacune d'elles différents premiers termes x_0 . On pourra construire une fonction $\text{iterees}(f, x_0, q)$ qui retourne la liste $[x_0, x_1, \dots, x_q]$ pour vérifier.

Corrigé en 8.6 page 345.

8.3 Le point sur la notion de complexité

Nous avons vu que la façon de programmer pouvait avoir une incidence notable sur le temps de calcul et sur la gestion de la mémoire. Nous revenons sur ces problèmes de complexité des algorithmes en présentant quelques outils et quelques exemples fondamentaux. Les deux amusettes qui nous servent de prélude présentent des situations extrêmes qui devraient nous faire réfléchir à ce qu'il est possible de faire.

8.3.1 La place, le temps, la précision

• Pas la place pour le nombre d'Avogadro

Commençons par un bref rappel de physique-chimie pour lycéens :

- une mole³ d'atomes contient environ $6,02 \times 10^{23}$ atomes (constante d'Avogadro) ;
- une mole d'un gaz parfait à la température de 20°C et à une pression de 1 013,25hPa occupe un volume de 24ℓ.

Imaginons que nous voulions simuler le comportement de particules d'un gaz parfait enfermées dans un volume de 1ℓ sous ces mêmes conditions de pression et de température. On souhaiterait, avant de commencer tout calcul, placer dans un fichier les positions et les vitesses initiales $p = (p_x, p_y, p_z)$ et $v = (v_x, v_y, v_z)$ de chaque particule.

Pour des raisons de place, on pense à répartir ces données en sous-fichiers partagés sur les disques durs des ordinateurs personnels de chaque individu sur terre⁴ :

- nous savons qu'un flottant en double précision est codé sur 64 bits (norme IEEE 754) ;
- nous devons donc disposer pour stocker ces données de

$$\frac{1}{24} \times 6,02 \times 10^{23} \times 6 \times 64 \text{bits} \approx 9.632 \times 10^{24} \text{bits} ;$$

- nous sommes sur la terre à peu près 7×10^9 habitants ;
- un octet est une suite de 8 bits, et un To (Tera octet) contient 10^{12} octets = 8×10^{12} bits ;
- chacun d'entre nous devrait donc disposer en moyenne d'environ 172 To.

Si des optimistes pensent que c'est pour bientôt, ils peuvent toujours réfléchir à la mémoire vive nécessaire pour effectuer les calculs.

• Pas le temps pour la constante d'Euler⁵.

Considérons la suite $(H_n)_n$ définie par

$$H_n = \sum_{k=1}^n \frac{1}{k} = \left(1 + \frac{1}{2} + \dots + \frac{1}{n} \right)$$

3. La mole est la quantité de matière d'un système contenant autant d'entités élémentaires qu'il y a d'atomes dans 12 grammes de carbone 12.

4. Analogue à ce que fait le projet SETI@home pour le partage des calculs.

5. Voir aussi l'exercice 8.16, page 337.

Une étude mathématique rapide que nous ne reprenons pas ici permet d'établir que les suites $(a_n)_n$ et $(b_n)_n$ définies par $\begin{cases} a_n = H_n - \ln(n+1) \\ b_n = H_n - \ln(n) \end{cases}$ sont adjacentes et ont la même limite $\gamma \in \mathbb{R}$, qui est, pour tout $n \geq 1$, encadrée par a_n et b_n : $a_n \leq \gamma \leq b_n$.

Nous pouvons donc, en théorie du moins, connaître un encadrement de γ , que l'on appelle constante d'Euler, avec la précision que l'on veut : il suffit d'observer que $b_n - a_n = \ln\left(1 + \frac{1}{n}\right) \xrightarrow{n \rightarrow \infty} 0$. Ainsi, dès que nous aurons $b_n - a_n \leq 10^{-3}$, nous aurons connaissance de la troisième décimale de γ à une unité près.

L'encart qui suit présente une fonction constanteEuler(e) qui, pour e donné en argument, retourne le triplet formé du premier indice n pour lequel

$$b_n - a_n = \ln\left(1 + \frac{1}{n}\right) \leq e,$$

de a_n et de b_n . Nous complétons le script par quelques lignes permettant à l'utilisateur de faire afficher résultats obtenus et temps de calculs.

Nous importons le module `scipy` pour la fonction **log** (logarithme népérien que nous notons habituellement $\ln$) et le module `time` pour le chronomètre : **time.time()**.

Au moment de l'essayer, rappelez vous pourtant que l'on interrompt un programme avec `Ctrl-C`.

```
import scipy as sp
import time

def constanteEuler(e) :
    n, s = 1, 1
    while sp.log(1+1/n)>e :
        n = n+1
        s = s+1/n
    return n, s-sp.log(n+1), s-sp.log(n)

n=int( input('n= ') )
for k in range(0,n) :
    t0 = time.time()
    print(constanteEuler(10**(-k)))
    print('temps du dernier calcul : ', t.time()-t0)
```

Le tableau montre que le temps de calcul augmente comme une suite géométrique de raison 10 lorsque l'argument est $e = 10^{-k}$.

k	n	a_n	b_n	t_n
0.0	1.0	0.30685281944	1.0	0.0
1.0	10.0	0.53107298117	0.626383160974	0.0
2.0	100.0	0.572257000798	0.582207331652	0.0
3.0	1000.0	0.576716081235	0.577715581568	0.0780000686646
4.0	10000.0	0.577165669068	0.577265664068	0.155999898911
5.0	100000.0	0.577210664943	0.577220664893	1.65400004387
6.0	1000000.0	0.577215164901	0.577216164901	16.5039999485
7.0	10000000.0	0.577215614899	0.577215714899	166.032999992

Était-ce prévisible ?

Le cours de maths de terminale nous apprend que $\lim_{x \rightarrow 0} \frac{\ln(1+x)}{x} = 1$ et on peut montrer en étudiant les fonctions différences qu'en fait pour $x \geq 0$,

$$x - \frac{x^2}{2} \leq \ln(1+x) \leq x.$$

Pour une précision de $e = 10^{-k}$ il nous faudra donc à peu près 10^k opérations et vouloir une décimale de plus demande **une multiplication par 10 du nombre de termes et du nombre d'opérations, donc du temps de calcul**. C'est bien ce que l'on observe⁶.

Question Quel temps faudrait-il pour déterminer à l'unité près la vingtième décimale de γ avec ce programme et le même ordinateur, en admettant que les calculs aient un sens ?

Réponse : environ 51 millions d'années !

• La précision ?

Rappelons simplement qu'en supposant que le temps ne nous soit pas compté l'algorithme qui suit termine dans le monde des flottants.

```
i=1
while 10**(-i) != 0:
    i=i+1
    print(i)
```

8.3.2 Les outils : théorie et pratique

On peut définir plusieurs critères pour évaluer la complexité en temps ou en place mémoire d'un algorithme opérant sur un ensemble $\mathcal{D}$ de données. En pratique, pour

6. Au moment où nous lançons ce programme pour construire le tableau, un anti-virus s'est mis en route. Nous nous en sommes aperçu parce que les temps de calcul ne suivaient pas une progression géométrique. Les résultats du tableau affiché sont bien sûr obtenus anti-virus bloqué.

la complexité temporelle, on choisit de dénombrer ou d'*estimer* (c'est-à-dire encadrer, donner un équivalent, comparer...) des opérations particulières ou certains appels à des fonctions auxiliaires choisis de telle sorte que l'évaluation obtenue soit proportionnelle au temps d'exécution du programme (on parlera alors d'*opérations fondamentales*). Nous verrons cela plus en détails sur quelques exemples.

Définition 8.3 complexités

On considère un algorithme ou une fonction $\mathcal{A}$ opérant sur des données (ou arguments) $d \in \mathcal{D}$. On suppose qu'il existe une fonction $d \in \mathcal{D} \rightarrow \text{taille}(d) \in \mathbb{N}$ et on note $\mathcal{D}_n = \{d \in \mathcal{D} / \text{taille}(d) = n\}$.

Une opération fondamentale étant choisie pour $\mathcal{A}$, le coût (ou la complexité) de l'appel $\mathcal{A}(d)$, pour $d \in \mathcal{D}$, est le nombre d'opérations fondamentales qu'il provoque. On définit alors

- $\max_{\mathcal{A}}(n) = \max\{\text{coût}_{\mathcal{A}}(d) / d \in \mathcal{D}_n\}$, la complexité dans le pire des cas sur les données de taille n ;
- $\min_{\mathcal{A}}(n) = \min\{\text{coût}_{\mathcal{A}}(d) / d \in \mathcal{D}_n\}$, la complexité dans le meilleur des cas (ou dans le cas le plus favorable) sur les données de taille n ;
- lorsque ces complexités ont des ordres de grandeur différents, la notion de complexité en moyenne est plus significative. On la définit en posant

$$\text{moy}_{\mathcal{A}}(n) = \sum_{d \in \mathcal{D}_n} P_n(d) \text{coût}_{\mathcal{A}}(d)$$

où $P_n(d)$ est la probabilité que d soit envoyé comme paramètre à $\mathcal{A}$ avec, bien sûr, $P_n(d) = \frac{1}{\text{card}(\mathcal{D}_n)}$ si les appels sont équiprobables (ce qui est rarement le cas).

Retour sur quelques algorithmes déjà étudiés

1. Division euclidienne de deux entiers

Pour la fonction réalisant la division euclidienne vue page 96, nous pouvons choisir comme *opération fondamentale* l'itération (il y a, pour chaque itération, une comparaison, une soustraction et une incrémentation). Un appel $\text{divEucl}(a, b)$ provoque $\left\lfloor \frac{a}{b} \right\rfloor$ itérations. Si nous considérons que a est la *taille* de la donnée $d = (a, b) \in \mathbb{N} \times \mathbb{N}^*$, le coût dans le pire des cas est a . Et dans le meilleur des cas, il est égal à 1 (respectivement quand $b = 1$ et $b = a$.)

2. Recherche séquentielle dans une liste non triée

Lors de l'étude de l'algorithme de recherche d'un élément x dans une liste L : RechercheW(L, x) (page 103), nous avons choisi comme *taille d'un paramètre* $d = (L, x)$, la longueur de la liste L et pour *opérations fondamentales* les comparaisons $e \neq L[k]$, ce qui revient (à une unité près peut-être) à dénombrer les itérations.

Dans le pire des cas la complexité en nombre de comparaisons est n (cas où x n'est pas dans la liste) et elle est égale à 1 si x est le premier terme de la liste.

Nous étudions plus loin la complexité en moyenne.

Question : notre « à une unité près peut-être » est-il justifié ou preuve de laisser-aller de notre part ?

3. Recherche du plus grand élément d'une liste non triée

On se réfère au programme de la page 106.

- Si l'opération fondamentale choisie est la comparaison, si la taille prise en considération pour les données est la longueur de la liste, alors, *la complexité en comparaisons la même dans le meilleur et dans le pire des cas* : elle est égale à $n - 1$ pour une liste de longueur n .

- Si, par contre, l'opération fondamentale choisie est l'affectation nous avons vu que dans le pire des cas (lorsque le tableau est trié dans l'ordre croissant), il y a n affectations (initialisation et chaque itération) alors que, dans le meilleur des cas (lorsque le maximum est en début de liste), il n'y a qu'une seule affectation.

4. Calcul de p tel que $2^p \leq n < 2^{p+1}$ avec le programme :

```
def log_entier(n):
    if n < 1:
        raise ValueError
    else:
        p = 0
        e = 1
        while e <= n:
            e = 2*e # ou mieux: e += e
            p = p+1
        return p
```

- On choisit pour taille des données le nombre n lui-même et pour opération fondamentale la multiplication.

Un invariant de boucle est $2^p = e$ et le programme termine avec $n < e = 2^p$ (pourquoi termine-t-il ?) ; p est par ailleurs le nombre exact d'itérations donc de multiplications par 2.

On a $2^{p-1} \leq n < e = 2^p$, et $\frac{\ln n}{\ln 2} < p \leq \frac{\ln n}{\ln 2} + 1$. La complexité est donc au plus $\frac{\ln n}{\ln 2} + 1 = \log_2(n) + 1$.

Relations de comparaison, rappels

Commençons par un petit rappel et un complément du cours de maths sur les relations de comparaisons. Elles vont nous permettre de classer les algorithmes en fonction de leur complexité.

Définition 8.4 relations de comparaison au voisinage de l'infini

Soient $(u_n)_n$ et $(v_n)_n$ deux suites numériques. On suppose qu'il existe un rang à partir duquel v_n ne s'annule pas⁷ (ie : il existe $n_0 \in \mathbb{N}$ tel que pour tout $n \geq n_0$, $v_n \neq 0$).

- On dit que $(u_n)_n$ est dominée par $(v_n)_n$ (au voisinage de $+\infty$) ssi il existe une constante $C > 0$ et un rang n_0 tels que, pour $n \geq n_0$, $\left| \frac{u_n}{v_n} \right| \leq C$, ce que l'on note $u_n \underset{n \rightarrow +\infty}{=} O(v_n)$.
- On dit que $(u_n)_n$ et $(v_n)_n$ ont le même ordre de grandeur asymptotique ssi $(u_n)_n$ est dominée par $(v_n)_n$ et $(v_n)_n$ est dominée par $(u_n)_n$. On note alors $u_n \underset{n \rightarrow +\infty}{=} \Theta(v_n)$.
- On dit que $(u_n)_n$ est négligeable devant $(v_n)_n$ (au voisinage de $+\infty$) et on note $u_n = o(v_n)$ ssi $\lim_{n \rightarrow +\infty} \frac{u_n}{v_n} = 0$.
- On dit que $(u_n)_n$ est équivalente à $(v_n)_n$ (au voisinage de $+\infty$) et on note $u_n \sim v_n$ ssi $\lim_{n \rightarrow +\infty} \frac{u_n}{v_n} = 1$.

algorithme	opération	estimation
Calcul de p tel que $2^p \leq n < 2^{p+1}$	*	$\sim \log_2(n)$
Recherche dichotomique dans L (de longueur n)	comparaison	$O(\ln n)$
Recherche séquentielle (liste non triée) ; pire des cas	comparaison	$\sim n$
Recherche séquentielle (liste non triée) en moyenne	comparaison	$\sim n/2$
Division euclidienne de a par b	-	$O(a)$
Tri d'une liste par sélection (exercice 2.10 p. 109)	comparaison	$\sim n^2/2$
Multiplication de deux matrices carrées de taille n	*	$= n^3$
Résol. d'un système linéaire de taille n (m. de Gauss)	*	$\Theta(n^3)$
Calcul d'un déterminant de taille n (m. de Gauss)	*	$\Theta(n^3)$
Tracé d'une courbe de Von Koch (exercice 3.4 p. 147)	tracé de segment	$O(4^n)$.

Exercice 8.7 vérifier, conjecturer, apprendre la prudence...

L'objectif est simple : étudier ces relations du point de vue numérique et informatique pour se préparer à explorer expérimentalement le comportement asymptotiques des algorithmes. Au passage, cela devrait vous aider à mieux comprendre la signification mathématique de ces notions.

1. On prouve sans peine que $3n^3 + 6n \sim 3n^3$. Écrire un script qui permet de le visualiser (numériquement).
2. La formule de Stirling que vous rencontrerez dans votre cours de maths s'écrit $n! \sim \left(\frac{n}{e}\right)^n \sqrt{2\pi n}$. Écrire un script permettant de la vérifier.
3. Comparaison des puissances et des factorielles
 - (a) Écrire un script permettant de visualiser les quotients $\frac{10^n}{n!}$ pour n variant de 1 à 10. L'hypothèse $10^n = o(n!)$ semble-t-elle confirmée ?

7. Attention, la définition dans un cours de maths sera plus générale.

- (b) Écrire un script permettant de visualiser les quotients $\frac{10^n}{n!}$ pour n variant de 1 à 100 par pas de 20. L'hypothèse $10^n = o(n!)$ semble-t-elle confirmée ?

Corrigé en 8.7 page 347.

Mesure pratique de la complexité : le compteur

Montrons avec un exemple simple comment on peut utiliser un *compteur* pour dénombrer les opérations d'un certain type lors de l'exécution d'un programme. Nous avons au chapitre précédent étudié la complexité de l'algorithme de recherche du maximum en nombre d'affectations dans le pire des cas et dans le meilleur des cas. Pour étudier la complexité en moyenne pour cette même opération fondamentale,

- on modifie la fonction en introduisant un compteur, c'est-à-dire une variable globale que l'on incrémente à chaque affectation ;
- on écrit ensuite un script pour construire, pour n variant (de d à f par pas de p) un certain nombre de listes aléatoires de longueur n et on détermine le nombre moyen d'affectations pour chaque valeur de n ;
- on teste (*en calculant des quotients*) différentes hypothèses quant au comportement asymptotique ;
- il ne reste plus ensuite qu'à démontrer !

Le maximum	Le maximum avec un compteur
<pre>def maximum(L): m = L[0] for k in range(1, len(L)): if L[k] > m: m = L[k] return m</pre>	<pre>def maximum(L): global c m = L[0] for k in range(1, len(L)): if L[k] > m: m = L[k] c = c+1 return m</pre>

Exercice 8.8 *exercice technique (et utile)*

On se propose d'écrire un script permettant les calculs de complexité en moyenne pour le nombre d'affectations. Nous commençons par présenter les outils dont nous aurons besoin.

- La fonction **randint** du module **random** avec la syntaxe
`random.randint(a, b)`
retourne un nombre aléatoire k tel que $a \leq k \leq b$. La loi « est » uniforme sur $\llbracket a, b \rrbracket$

- Les fonctions **matrix** et **zeros** du module **numpy** permettent de construire une matrice ou un tableau de ℓ lignes et 3 colonnes par exemple, avec la syntaxe `A = np.matrix(np.zeros((ell,3), dtype = float))`
1. Écrire un mini script qui permet de construire une liste de longueur n et dont les termes sont choisis aléatoirement entre 0 et $10n$ par exemple.
 2. Pour n fixé ($n = 10$ par exemple), écrire un script qui construit successivement q listes aléatoires de longueur n , ($q = 10$ par exemple) calcule et affiche le nombre moyen d'affectations dans la fonction **maximum**(L).
 3. Une fois ces préparatifs faits, écrire un script qui pour n variant de 50 à 1000 par pas de 50, calcule le nombre moyen d'affectations sur $q = 1000$ calculs de maximums sur des listes aléatoires de longueur n . Présenter les résultats dans une matrice (ou tableau) dont chaque ligne contiendra n , le nombre moyen m_n d'affectations sur les q listes de longueur n et un quotient $\frac{m_n}{?}$ qui vous permettra de tester les hypothèses $m_n \sim n$, $m_n \sim n/2$, $m_n \sim \ln n$... Laquelle vous paraît-elle la plus plausible ?
 4. On montre que sur des listes dont *tous les éléments sont distincts*, $m_n = H_n = \sum_{k=1}^n \frac{1}{k}$ (somme partielle de la série harmonique). La fonction **random.sample** de syntaxe `random.sample(range(0,1000),12)` permet de faire un tirage sans remise. Le résultat est-il plus probant ?

Corrigé en 8.8 page 349.

Mesure pratique du temps de calcul : le chronomètre

Il peut s'avérer utile d'estimer expérimentalement le temps de calcul d'une fonction ou d'un programme. Les études que nous avons menées sur quelques exemples remarquables nous permettent le plus souvent d'obtenir un équivalent, une domination voire un ordre de grandeur en fonction de la taille des arguments. Nous savons par exemple que telle méthode opérant sur un objet de taille N demande un nombre d'opérations équivalent à N^3 et nous voulons lancer le programme à partir d'un gros fichier dont nous connaissons la taille.

Il nous est alors possible en mesurant le temps de calcul avec des données de plus petite taille, de déduire celui que prendra le calcul que nous voulons mener à bien. Concrètement : prendre un café en attendant le résultat, partir en week-end, nous en remettre comme nous l'avons fait page 323 à ceux qui viendront après l'extinction de notre espèce... Pour cela on dispose de la fonction **time** du module **time**.

```
import time

t0 = time.time()
for i in range(0,1000):
    i*i
print(time.time()-t0)

>>> 0.00035452842712402344
```

8.3.3 Exemples basiques

Algorithme de Horner pour le calcul d'expressions polynomiales

Vous avez déjà rencontré la méthode de Horner dans vos années lycéennes. Elle consiste à réorganiser les calculs polynomiaux pour réduire le nombre de multiplications : pour calculer $P(X) = a_n X^n + a_{n-1} X^{n-1} + \dots + a_1 X + a_0$, on procède en calculant successivement

$$r_0 = a_n, \quad r_1 = r_0 X + a_{n-1}, \dots, r_n = r_{n-1} X + a_0.$$

Elle présente aussi l'avantage de réduire la taille des calculs intermédiaires ce qui permet dans certains cas d'éviter des dépassements de capacité.

Exercice 8.9 *algorithme de Hörner pour le calcul des polynômes*

1. Écrire une fonction `horner(L, x)` qui prend en arguments une liste L et un nombre x et retourne la valeur en x du polynôme P dont les coefficients sont $a_i = L[i]$ ($P(X) = \sum a_i X^i$).
2. Préciser le nombre d'additions et de multiplications.
3. Prouver la correction (ie : on calcule bien $P(X)$).

Corrigé en 8.9 page 350.

Exercice 8.10 *complexité du produit usuel de deux matrices*

On définit le produit de deux matrices $A \in \mathcal{M}_{n,p}(\mathbb{K})$ et $B \in \mathcal{M}_{p,q}(\mathbb{K})$ comme étant la matrice de $\mathcal{M}_{n,q}(\mathbb{K})$ telle que

$$m_{i,j} = \sum_{k=1}^p a_{i,k} b_{k,j}.$$

On suppose que l'on stocke ces matrices dans des tableaux A et B dont les termes sont accessibles et modifiables avec des instructions $A[i, j]$, pour appeler le terme de

la ligne i et de la colonne j , $A[i, j] = \dots$ pour le modifier.

Sans détailler le script qui permet de réserver un tableau pour stocker $C = A \times B$ et de le remplir, dire comment on calcule la matrice produit et combien il y a de multiplications de deux flottants.

Corrigé en 8.10 page 350.

- On peut programmer le produit matriciel de façon récursive, avec une stratégie « diviser pour régner », pour effectuer un produit matriciel en $O(n^{2.81})$ multiplications (méthode de Strassen).
- Le problème 8.18 page 338 propose une restructuration des calculs qui permet de réduire la complexité d'un facteur $\frac{1}{8}$.

8.3.4 Complexité de l'algorithme d'Euclide

Cette section est un complément dédié à un algorithme qui date de plus de 2300 ans. On y rencontre, crescendo, quatre niveaux de difficulté : la programmation de l'algorithme d'Euclide, son extension au calcul des coefficients de Bezout, les preuves de correction et enfin le calcul de complexité (théorème de Lamé). Nous sommes, en abordant ces questions, au cœur de l'algorithmique, du XXI^{ème} siècle, tant par les méthodes mises en œuvre pour la preuve de programme et le calcul de sa complexité, que pour les applications. Sans calcul de pgcd, sans la rapidité de l'algorithme, aucun système de calcul formel calculant avec des rationnels ou avec des polynômes ne pourrait utilement exister.

• Calcul du pgcd, analyse de l'algorithme d'Euclide

Exercice 8.11 l'algorithme d'Euclide

1. Soient a et $b \neq 0$ deux entiers naturels, q et r le quotient et le reste dans la division euclidienne de a par b caractérisés par $a = bq + r$ et $0 \leq r < b$. Justifier que (a, b) et (b, r) ont les mêmes diviseurs.
2. On définit un algorithme de la façon suivante :

Entrées : a, b entiers positifs ou nuls ; $a \neq 0$ ou $b \neq 0$;

Initialisation : $r0 = a, r1 = b, i = 1$.

Tant que $r1 \neq 0$,

- Calculer (q, r) le couple formé du quotient et du reste de la division euclidienne de $r0$ par $r1$.
- Faire $r0 = r1, r1 = r$.

- (a) Montrer que ce programme termine.
- (b) Déterminer un invariant de boucle.
- (c) Justifier que, lorsque la boucle termine, $r0 = \text{pgcd}(a, b)$.

3. Implémenter cet algorithme sous Python ou vérifier que le programme qui suit en est une implémentation fidèle.

Corrigé en 8.11 page 351.

Algorithme d'Euclide, version basique

```
def pgcd(a,b):
    '''a, b :int avec la pré-condition
        a>=0 et b>= 0 et (a>0 ou b >0);
        Renvoie le pgcd de a et de b. '''
    r0 , r1 = a, b
    while r1 !=0:
        r0, r1 = r1, divmod(r0, r1)[1]
    return r0
```

• Identité de Bezout et algorithme d'Euclide étendu

Nous allons démontrer, grâce à une adaptation de l'algorithme d'Euclide, que si $\text{pgcd}(a, b) = d$, il existe un couple $(u, v) \in \mathbb{Z}^2$ tel que $au + bv = d$.

On écrit dans la foulée une fonction $\text{Bezout}(a, b)$ qui prend en arguments deux entiers naturels a et b ($b \neq 0$) et retourne un tuple (d, u, v) où $d = \text{pgcd}(a, b)$ et u, v vérifient $au + bv = d$.

Exercice 8.12 l'algorithme d'Euclide étendu

On considère l'algorithme de calcul du pgcd de a et de b tel qu'il est décrit dans l'exercice (8.11) ou programmé sous Python ci-dessus.

On note r_{i-1} et r_i les contenus de $r0$ et $r1$ lors de la $i^{\text{ième}}$ évaluation de la condition, (q_i, r_{i+1}) les quotient et reste calculés lors de la $i^{\text{ième}}$ itération dans la boucle, si elle a lieu (attention, à ne pas confondre r_1, r_0 avec $r1 = b$ et $r0 = a$!). On a donc :

$$\begin{aligned} r_0 &= q_1 r_1 + r_2 \\ r_1 &= q_2 r_2 + r_3 \\ r_2 &= q_3 r_3 + r_4 \\ &\vdots \end{aligned}$$

1. Montrer que l'on peut écrire une relation matricielle entre les termes de la suite finie $(r_i)_i$:

$$\begin{bmatrix} r_i \\ r_{i+1} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & -q_i \end{bmatrix} \begin{bmatrix} r_{i-1} \\ r_i \end{bmatrix} \quad (8.3.1)$$

2. En déduire qu'il existe des entiers u et v tels que $\text{pgcd}(a, b) = au + bv$.

3. Écrire un programme Python qui prend a et b en arguments et retourne un triplet (d, u, v) tel que $d = \text{pgcd}(a, b) = au + bv$.

Indications

Pour accéder aux matrices, charger **numpy** ; utiliser **numpy.matrix** pour construire une matrice à partir d'une liste, puis la méthode **reshape(2,2)** pour la restructurer en matrice 2×2 ; la multiplication des matrices, quant à elle, est tout simplement notée $*$: `np.matrix([0,1,1,-q]).reshape(2,2)`

Corrigé en 8.12 page 351.

Avec les deux exercices qui précèdent, nous avons prouvé le

Théorème 8.1 *algorithme d'Euclide, identité de Bezout*

- Si a et b sont deux entiers naturels, la suite définie par l'algorithme :

- $r_0 = a, r_1 = b$,

- tant que $r_{i+1} \neq 0$, r_{i+2} est le reste de la DE de r_i par r_{i+1} ,

est finie. Cela signifie que l'algorithme termine. La dernière itération (de numéro n) donne $r_{n+1} = 0$ et r_n , le dernier reste non nul, est le pgcd de a et b .

- Il existe u, v entiers relatifs tels que $\text{pgcd}(a, b) = au + bv$ (identité de Bezout).

Algorithme d'Euclide étendu au calcul des coefficients de Bezout

```
import numpy as np

def Bezout(a,b):
    r0 , r1, m = a, b, np.identity(2)

    while r1 !=0:
        r0, (q, r1) = r1, divmod(r0, r1)
        m = np.matrix([0,1,1,-q]).reshape(2,2)*m

    return r0, m[0,0], m[0,1]
```

★ **Complexité de l'algorithme d'Euclide, théorème de Lamé**

Nous abordons, avec le théorème de Lamé qui date de 1845, le calcul de la complexité de l'algorithme d'Euclide. En comparant les restes successifs aux termes d'une suite de Fibonacci, nous pouvons majorer le nombre n d'itérations en fonction de $\ln(\max(a, b))$. Bien qu'elle ait fait l'objet de sujets de concours de tous niveaux, on peut considérer cette étude comme difficile au niveau L2. Nous la présentons car il s'agit d'un résultat fondamental et parmi les plus anciens concernant la complexité.

Suites de Fibonacci

Une suite de Fibonacci est une suite telle que $F_{n+2} = F_{n+1} + F_n$ et dont les premiers termes F_0, F_1 sont des entiers naturels.

On sait que les solutions d'une telle équation de récurrence sont les combinaisons linéaires des suites géométriques $(q_1^n)_n, (q_2^n)_n$, q_1 et q_2 étant les solutions de l'équation caractéristique $x^2 - x - 1 = 0$. On écrit donc $F_n = \alpha q_1^n + \beta q_2^n$ et l'on détermine α et β en fonction de F_0, F_1 en identifiant les deux premiers termes. Il vient donc

$$F_n = \left(\frac{q_2 F_0 - F_1}{q_2 - q_1} \right) q_1^n + \left(\frac{q_1 F_0 - F_1}{q_1 - q_2} \right) q_2^n \quad (8.3.2)$$

où l'on a : $q_1 = \frac{1 + \sqrt{5}}{2}$, $q_2 = \frac{1 - \sqrt{5}}{2}$ et $q_1 - q_2 = \sqrt{5}$.

Exercice 8.13 *théorème de Lamé*

On se propose ici d'évaluer le nombre d'opérations réalisées par l'algorithme d'Euclide lors d'une exécution avec comme données $a > b$. On note $(F_n)_n$ la suite de Fibonacci pour laquelle $F_0 = 0$ et $F_1 = 1$.

1. On suppose que $a > b$ et on note $r_0 = a$, $r_1 = b$, et tant que $r_i \neq 0$, $r_{i-1} = r_i q_i + r_{i+1}$ avec $0 \leq r_{i+1} < r_i$. La suite obtenue est celle des restes dans l'algorithme d'Euclide. Montrer que si n est le nombre d'itérations de l'algorithme d'Euclide, alors

$$a \geq F_{n+1}, \quad b \geq F_n.$$

Indication : écrire en colonne 1 la succession des opérations de l'algorithme d'Euclide (on notera r_n le dernier reste non nul); écrire en colonne 2, en commençant par la dernière ligne et après les avoir justifiées, des inégalités vérifiées par les $G_k = r_{n-k+1}$.

r_0	$=$	$q_1 r_1 + r_2$	G_{n+1}	$\geq$	$G_n + G_{n-1}$
r_1	$=$	$q_2 r_2 + r_3$	G_n	$\geq$	$G_{n-1} + G_{n-2}$
r_2	$=$	$q_3 r_3 + r_4$	G_{n-1}	$\geq$	$G_{n-2} + G_{n-3}$
$\vdots$			$\vdots$		
r_{i-1}	$=$	$q_i r_i + r_{i+1}$	G_{n-i+2}	$\geq$	$G_{n-i+1} + G_{n-i}$
$\vdots$			$\vdots$		
r_{n-2}	$=$	$q_{n-1} r_{n-1} + r_n$	G_3	$\geq$	$G_2 + G_1$
r_{n-1}	$=$	$q_n r_n + r_{n+1}$	G_2	$\geq$	$G_1 + G_0$

2. En déduire une majoration de n qui est le nombre de divisions euclidiennes, en fonction des données a et b .

Corrigé en 8.13 page 352.

Théorème 8.2 *Lamé (1845)*

Le nombre n de divisions euclidiennes (ou itérations) dans l'algorithme d'Euclide calculant le pgcd de a et de b avec $a \geq b > 0$ vérifie : $n \leq \frac{\ln(\sqrt{5}b + 1)}{\ln q_1}$.

8.4 Analyse des programmes récur­sifs

Nous sommes confrontés lors de l'écriture et de l'analyse des programmes récur­sifs aux mêmes trois problèmes que pour l'ensemble des programmes.

1. Le programme termine-t-il ?
2. Est-il conforme à sa spécification ?
3. Quelle est sa complexité ?

Nous avons eu l'occasion de présenter ces questions en détails au chapitre 3 (Récursivité) et avec l'étude des tris récur­sifs. Nous y revenons avec l'exercice 8.14 qui est représentatif des preuves de correction dans une fonction récur­sive et l'exercice 8.15 permet de réviser les techniques de mémorisation abordées au chapitre 3 et propose des calculs de complexité qu'il faut avoir vus.

Exercice 8.14 *factorielle partagée*

1. Que calcule la fonction suivante lors des appels $FF(0, 1)$, $FF(0, 2)$, $FF(0, 3)$, ..., $FF(0, 6)$? Dessiner l'arbre des appels. Conjecture ?
2. Pourquoi termine-t-elle ?
3. Peut on prouver ce que l'on a affirmé dans la première question ?

```
def FF(a,b):  
    if b <=a:  
        raise ValueError('b doit être supérieur...')  
    elif b == a+1:  
        return b  
    else:  
        c=(a+b)//2  
        return FF(a,c)*FF(c,b)
```

Corrigé en 8.14 page 352.

Exercice 8.15 *la construction des parties à k éléments*

1. Rappeler la démonstration combinatoire (sans calculs) de la formule de Pascal⁸ :

$$\binom{n+1}{k+1} = \binom{n}{k} + \binom{n}{k+1}$$

2. Montrer que le programme `parties(L, k)` présenté page 336 termine quelque soit la liste L , si $k \geq 0$.
3. Donner sa spécification et démontrer sa correction.
4. Dessiner l'arbre des appels récursifs en partant d'un appel principal `parties(L, k)`. Est-il possible de réduire le nombre des appels récursifs ?
5. Réécrire ce programme en utilisant un dictionnaire, **vide lors de l'appel principal** `partiesD(L, k, D)`. Ce dictionnaire sera rempli au fur et à mesure des retours et sera consulté pour éviter de calculer plusieurs fois les mêmes parties. On pourra le concevoir comme un dictionnaire de dictionnaires. La clé n par exemple est associée à un dictionnaire dont les clés sont $0, 1, \dots, k$, associées aux résultats : $D[n][k]$ est la liste des parties à k éléments :

$$D = \left\{ \begin{array}{l} n \\ \vdots \\ n-1 \end{array} \right. : \left\{ \begin{array}{l} 0 : [], \\ 1 : [[\ell_0], [\ell_1], \dots, [\ell_{n-1}]], \\ 2 : [[\ell_0, \ell_1], [\ell_0, \ell_2], \dots, [\ell_{n-2}, \ell_{n-1}]], \\ \vdots \\ n : \dots, \end{array} \right.$$

$$\left. \begin{array}{l} \\ \\ n-1 \end{array} \right. : \left\{ \begin{array}{l} 0 : [], \\ 1 : [[\ell_1], [\ell_2], \dots, [\ell_{n-1}]], \\ 2 : [[\ell_1, \ell_2], \dots, [\ell_{n-2}, \ell_{n-1}]], \\ \vdots \\ n-1 : \dots, \end{array} \right.$$

Conseil : relire les rappels page 156 du chapitre 3.

6. Comparer la complexité de votre fonction à celle du programme initial en considérant le nombre d'appels récursifs. Vous observerez, en regardant l'arbre que vous avez dessiné, que les appels récursifs de profondeur h sont considérablement réduits. Donner une évaluation dans le pire des cas.

Tester le nombre de ces appels en plaçant un compteur dans vos fonctions.

Corrigé en 8.15 page 353.

8. Le nom d'une formule ou d'un théorème n'est pas toujours celui du premier découvreur ou inventeur. Vous savez sans doute que cette formule apparaît dans des textes chinois, indiens ou arabes depuis les XI^e et XII^e siècles.

Programme proposé à l'étude dans l'exercice 8.15

```
def parties(L, k):
    """
    L : liste;
    k : int avec k>=0.
    Renvoie ?????
    """
    assert isinstance(k, int) and k>=0
    n = len(L)

    if k > n :
        # pas de partie à plus de n élts.
        return []
    elif k==0:
        # partie vide, seule solution.
        return [[]]
    elif k ==1:
        # singletons
        return [[a] for a in L]
    else :
        a = L[0]
        L1 = parties(L[1:n], k-1)
        L2 = parties(L[1:n], k)

        R = []
        for p in L1:
            R.append(p + [a])
        for q in L2:
            R.append(q)

    return R
```

8.5 Exercices

Exercice 8.16 calcul du nombre d'itérations, preuve de programme

On considère la suite $(s_n)_n$ des nombres rationnels définie par

$$s_n = \sum_{k=1}^n \frac{1}{k} = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}.$$

1. Écrire un script qui réalise les opérations suivantes :

(a) il demande de façon interactive un flottant au clavier en affichant l'invite

```
>>>
```

```
entrer x =
```

(b) il détermine le premier entier N tel que $s_N > x$ et affiche le résultat :

```
>>>
```

```
entrer x = 6
```

```
avec N = 227 , s = 6.004366708345567
```

On le testera avec $p = 5, 10, \dots$ et si on est satisfait on essaiera avec $x = 17.4$ par exemple.

2. On demande maintenant de

(a) démontrer par récurrence sur n que chacun des deux algorithmes qui suivent terminerait en donnant le résultat attendu **si on pouvait calculer en précision infinie**, sachant que dans le cours de maths on démontre que

$$\lim_{n \rightarrow +\infty} s_n = +\infty;$$

(b) déterminer pour chacun d'eux une valeur de l'entier n au delà de laquelle la valeur du **flottant** s est constante. Quelles remarques peut on faire ?

(c) calculer avec précision, le nombre d'additions, de divisions qu'entraîne la réalisation de chacun d'eux en fonction de N, la valeur affichée dans les cas où ils terminent.

```
x = float(input('entrer x = '))
s = 1
n = 1
while <= x:
    n = n+1
    s = 1
    for j in range(2,n+1):
        s = s+1/j
print('avec N = ', n , 's = ', s)
```



```
x = float(input('entrer x = '))
s = 1
n = 1
while s <= x:
    n = n+1
    s = s+1/n
print('avec N =', n, 's = ', s)
```

3. Le programme qui suit fait-il autre chose que les précédents ? En quoi est-il préférable ?

```
import numpy as np
x = float( input('entrer x = ' ) )

n = 1
while np.sum(np.array(range(1,n+1))**(-1.)) < x
    n = n+1

print('avec N =', n, 's = ', \
      np.sum(np.array(range(1,n+1))**(-1.)) )
```

Corrigé en 8.16, page 357.

Exercice 8.17 encore l'algorithme d'Euclide

L'algorithme d'Euclide tel que nous l'avons représenté dans ce chapitre fait appel à la division euclidienne. Nous allons reprendre la fonction `pgcd` pour qu'elle n'utilise que la soustraction comme opération arithmétique.

- Observer que si x et y sont deux entiers positifs, dont l'un est non nul,
 - $\text{pgcd}(x, y) = \text{pgcd}(y, x)$;
 - si $x \geq y$, $\text{pgcd}(x, y) = \text{pgcd}(x - y, y)$.
 En déduire une fonction `pgcd` ne faisant intervenir que la soustraction comme opération.
- Démontrer la correction de cette fonction.

Corrigé en 8.17 page 359.

Exercice 8.18 réduction de la complexité du produit de deux matrices

On se propose dans cet exercice de calculer la complexité d'un algorithme naïf du produit matriciel, puis à l'aide d'une restructuration des calculs algébriques, d'en réduire la complexité.

Dans cet exercice d'informatique numérique, on considère des vecteurs de $\mathbb{R}^N$ et des matrices carrées appartenant à $\mathcal{M}_N(\mathbb{R})$ et on sera amené à travailler avec $N = 2n$, pair.

1. On suppose que A et B sont deux matrices de $\mathcal{M}_N(\mathbb{R})$ qui sont représentées sous Python par des matrices de flottants (avec les fonctions de la bibliothèque `numpy`).

- (a) Donner la définition mathématique du produit matriciel $C = A \times B$.
 (b) Écrire un programme (script ou fonction, ce n'est pas le problème) qui prend A et B en arguments et effectue le calcul de C en transcrivant cette formule, sans appel à quelque fonctions prédéfinies que ce soit, le but est d'étudier la complexité du produit.

On supposera C réservée par :

```
C = numpy.matrix((numpy.zeros((N,N), dtype = float))
```

et on rappelle que l'on obtient $C_{i,j}$ en écrivant $C[i,j]$, que l'on actualise la valeur de $C[i,j]$ par une affectation $C[i,j] = \dots$

- (c) Donner en fonction de N le nombre de multiplications de deux flottants que provoque votre script (ou l'appel de votre fonction).
 2. On définit une application $f : \mathbb{R}^{2n} \rightarrow \mathbb{R}^{2n}$ en posant, pour $X \in \mathbb{R}^{2n}$,

$$f(X) = \sum_{i=1}^n x_{2i-1} x_{2i}.$$

- (a) Montrer que, pour $X, Y \in \mathbb{R}^{2n}$, on a :

$${}^tX Y = \sum_{i=1}^{2n} x_i y_i = \sum_{k=1}^n (x_{2k} + y_{2k-1})(y_{2k} + x_{2k-1}) - f(X) - f(Y) \quad (8.5.1)$$

- (b) Soient A et B deux matrices carrées de réels ou de flottants ayant $N = 2n$ lignes et colonnes.
 On note A_i la colonne d'indice i de A et B^j la colonne d'indice j de B .
 Écrire un programme qui calcule et stocke tous les $f(A_i)$ et tous les $f(B^j)$.
 Vous gérerez les différences d'indexation entre le cours de maths et les usages en Python.
 (c) Préciser avec soin le nombre de multiplications de deux flottants provoquées par ces calculs.
 3. (a) Modifier le programme de calcul de $C = AB$ en utilisant la formule (8.5.1) et en supposant qu'avant de calculer les $C[i,j]$ vous disposez des résultats du script de la question (2b).
 (b) Déterminer avec soin le nombre total de multiplications de deux flottants que nécessite votre programme.

Corrigé en 8.18 page 360.

8.6 Corrigés des exercices

Corrigé de l'exercice n° 8.1

```
def check(L, p):
    '''      '''
    assert isinstance(L, list)
    # on vérifie que L est une liste
    for elt in L:
        assert isinstance(elt, list) and len(elt) == p
        # on vérifie que elt est une liste à p élts.
    return True
```

```
>>> check([[0],[1,2]], 1)
Traceback (most recent call last):
  File ".../parties.py", line 62, in <module>
    check([[0],[1,2]], 1)
  File ".../parties.py", line 41, in check
    assert isinstance(ell, list) and len(ell) == p
AssertionError
```

```
def check1(L, p, m = None):
    '''      '''
    try:
        assert isinstance(L, list)
        for ell in L:
            assert isinstance(ell, list) and len(ell) == p
        return True # tout a bien marché
    except AssertionError:
        print('AssertionError déclenchée avec L = %s, p = %s.\n'
              '%(L,p))

        if m!=None:
            print(m)
        return False
```

```
>>> check1([[0],[1,2]], 1, 'simple test')
AssertionError déclenchée avec L = [[0], [1, 2]], p = 1.
Simple test.
```

Corrigé de l'exercice n° 8.2

1. (a) Voir le programme ci-dessous.

(b) À l'entrée de l'itération i , p contient 3^{i-1} donc $u = 3^{i-1}x$. On sort de l'itération avec $t = t_i(x)$ si le calcul est exact et $p = 3^i$.

(c) Si $x = 1$, on aura $t = 0$ à chaque étape. On est par ailleurs en dehors du domaine de spécification de la fonction (ce qui explique que l'on ne retrouve pas les chiffres du développement en base 3 de x), la **pré-condition** sur x n'est pas vérifiée.

(d) Voir le programme ci-dessous.

(e) La ligne `t = int(3*u)-3*int(u)` calcule $\lfloor 3^i x \rfloor - 3\lfloor 3^{i-1} x \rfloor$ mais pour 3^i suffisamment grand les erreurs d'arrondis vont devenir significatives et t ne vérifiera plus la **post-condition** $t \in \{0, 1, 2\}$.

```
def flottantVersTernaire(n, x):
    """
    n entier >= 1;
    x flottant dans [0,1[
    Renvoie la liste des chiffres ternaires de x.
    Teste les arguments (pré-condition)
    Teste les termes de la liste (post-condition).
    """
    assert isinstance(n, int) and n > 0
    assert isinstance(x, (int,float)) and x>=0 and x < 1

    p, L = 1, []
    # p pour les puissances de 3
    # L pour recevoir la suite des chiffres

    for i in range(1, n+1):
        u = p*x # alias 3**(i-1)*x
        t = int(3*u)-3*int(u)
        assert t in {0,1,2} # post-condition sur t
        L.append(t)
        p = 3*p # alias 3**i
    return L

>>> flottantVersTernaire(4, 0.5)
[1,1,1,1]
>>> flottantVersTernaire(4, 1)
Traceback...
assert isinstance(x, (int,float)) and x >= 0 and x < 1
AssertionError
```

2.

```
def ternaireVersFlottant(ell):
    """
    ell : liste d'entiers dans {0,1,2}
    Renvoie un flottant représentant x dont ell est la
    liste des chiffres ternaires.
    Teste la pré-condition sur ell (type et éléments).
    """
    assert isinstance(ell, list)

    s, i, p = 0, 0, 3
    while i < len(ell):
        u = ell[i]
        assert u in {0,1,2}
        # test de chaque élt. à la volée

        s += ell[i]/p
        p = 3*p
        i += 1
    return s
>>> ternaireVersFlottant([1,1,1,1])
0.49382716049382713
```

Corrigé de l'exercice n° 8.3

1. Le programme syracuse prend en argument un entier $a_0 > 0$, calcule les termes successifs de la suite de Collatz de premier terme $u_0 = a_0$ et s'arrête lorsqu'un terme est égal à 1. Dans ce cas la fonction renvoie l'indice de ce terme.
2. On suppose que le programme termine.

Si $a_0 = 1$, il n'y a pas d'itération et le programme renvoie $i = 0$ qui est bien l'indice du premier terme égal à 1.

Sinon, il est clair que i numérote les itérations. Notons u_0 le contenu de u avant itération et, pour $i \geq 1$, u_i le contenu de i après l'itération n° i .

Posons $\mathcal{P}(i) = (u_i \text{ est le terme d'indice } i \text{ de la suite (8.2.1)})$.

Cette propriété est vraie pour $i = 0$, puisque u est initialisée avec a_0 ; il alors suffit de constater que chaque incrémentation de i est accompagnée d'une ré-affectation de u qui colle à la relation de récurrence (8.2.1)

$\mathcal{P}$ est donc bien un invariant de boucle et comme le programme ne termine qu'avec la condition $u = 1$, i est bien l'indice du premier terme tel que $u_i = 1$. Nous avons bien une preuve de **correction partielle**.

Jusqu'à présent ce programme s'est toujours terminé mais vous savez pourquoi nous ne vous avons pas demandé de preuve de terminaison.

Corrigé de l'exercice n° 8.4

1. La transformation $m \rightarrow f(m)$ est définie sur $\mathbb{N}$, à valeur dans $[-18, +\infty[\subset \mathbb{Z}$.

```
def f(m):
    ''' m : int , m >= 0. '''
    assert isinstance(m, int) and m >= 0

    q, r = divmod(m, 10)
    return q-2*r
```

2. La fonction f vérifie $f(m) \leq \frac{1}{10}m \leq m$, pour tout $m \in \mathbb{N}$, l'égalité n'ayant lieu que si $m = 0$ et dans ce cas il n'y a pas d'itération.

On note m_0 la valeur du paramètre effectif et m_k (notre variant de boucle) la valeur de m à la k -ième évaluation. $(m_k)_k$ est strictement décroissante et la condition d'arrêt sera satisfaite dès que $k \geq \frac{\ln(m_0/99)}{\ln 10}$.

3. On observe tout d'abord que $7|m \Leftrightarrow 7|f(m)$.

En effet, avec $m = 10q + r$, il vient $f(m) = \frac{m-r}{10} - 2r = \frac{m-21r}{10}$.

Or,

$$7|m \Leftrightarrow 7|(m-21r) \Leftrightarrow 7|10f(m)$$

et comme 7 et 10 sont premiers entre eux, ceci est équivalent à $7|f(m)$.

Nous tenons alors notre invariant de boucle, c'est la propriété

$$\mathcal{P}(k) = (7|m_k \Leftrightarrow 7|m).$$

Corrigé de l'exercice n° 8.5

- Notons N_k la valeur de N à la $k^{\text{ième}}$ évaluation de la condition. Si $N_k = 0$, la boucle termine évidemment. Sinon, les règles du jeu étant respectées on aura $0 \leq N'_k = \text{Ajoue}(N_k) \leq N_k - 1$. Si cette valeur est nulle on a $N_{k+1} = 0$ après cette itération et le programme termine. Sinon, $0 \leq N_{k+1} = \text{Bjoue}(N'_k) \leq N'_k - 1 \leq N_k - 2$. La suite d'entiers naturels $(N_k)_k$ est strictement décroissante et atteint 0 en un nombre fini d'étapes.
- Pour $N = 4, 3, 2$, si $\text{Ajoue}(N)$ retourne 1, ce qui est toujours possible en retirant 1, 2, ou 3 jetons, c'en est fini de B qui perd au coup suivant. Si par contre $N = 1$, A perd car il doit retirer le dernier jeton ; si $N \geq 5$, A devra laisser au moins 2 jetons et la partie ne s'arrêtera pas nécessairement au coup suivant.
- Si $N \neq 4p + 1$ ($N \neq 1[4]$) on détermine $\text{Ajoue}(N)$ de la façon suivante :

$$\begin{cases} N = 4p & \rightarrow \text{Ajoue}(N) = N - 3 = 4(p-1) + 1 \\ N = 4p + 2 & \rightarrow \text{Ajoue}(N) = N - 1 = 4p + 1 \\ N = 4p + 3 & \rightarrow \text{Ajoue}(N) = N - 2 = 4p + 1 \end{cases}.$$

Quoi que fasse B par la suite, en retranchant 1, 2 ou 3 à N il rendra une valeur différente de 1 modulo 4. Ainsi, pour une programmation convenable de A joue, dès lors que la proposition

$$\{A \text{ joue avec } N \neq 1[4]\}$$

est vérifiée une fois, elle devient **un invariant de boucle**. Comme la suite des valeurs de N décroît strictement une des valeurs $N = 4, 3, 2$ sera présentée à A qui retournera la valeur 1, fatale à B .

4. B est sûr de gagner s'il peut laisser sur la table $4p + 1$ jetons. En effet, dans ce cas A laisse un nombre de jetons non congru à 1 modulo 4 et si on définit B joue comme on a défini A joue, alors $\{B \text{ joue avec } N \neq 1[4]\}$ est aussi un invariant de boucle.

Résumons :

- (a) En définissant A joue comme dans le programme ci-dessous et quelque soit la façon d'écrire B joue,

$$\{A \text{ joue avec } N \neq 1[4]\}$$

est un invariant de boucle lorsque la condition est vérifiée une fois ce qui garantit que A gagne.

- (b) En définissant B joue de la même façon, et quelque soit la façon de programmer A joue, $\{B \text{ joue avec } N \neq 1[4]\}$ devient un invariant de boucle dès qu'elle est vérifiée une fois, qui garantit que B gagne.
- (c) Lorsque les deux fonctions A joue et B joue sont définies de cette façon, le résultat est déterminé par le nombre initial de jetons.

```
import random as rd

def Ajoue(N):
    r = divmod(N, 4)[1]
    if r == 0:
        return N-3
    elif r == 2:
        return N-1
    elif r == 3:
        return N-2
    else:
        return max(N- int(1+3*rd.random()), 0)

Bjoue = Ajoue
```

```

def Nim(n) :
    N = n
    while N != 0:
        N = Ajoue(N)
        print('A joue ', N)
        if N == 0:
            r = 'B gagne'
        else:
            N = Bjoue(N)
            print('B joue ', N)
            if N == 0:
                r = 'A gagne'
    return r

```

Corrigé de l'exercice n° 8.6

- On choisit $f(x) = x + 1$.
 - Lorsque $x_0 = 0$, on a $x_2 = 2x_1$. Le programme s'arrête mais la suite n'est pas ultimement périodique.
 - Lorsque $x_0 = 1$, $x_n = n + 1 \neq x_{2n} = 2n + 1$ pour $n \geq 1$. Le programme ne s'arrête pas.
- On note k le nombre d'itérations déjà réalisées au moment de l'évaluation comme nous le suggère l'énoncé. On montre la proposition suivante

$$\mathcal{P}(k) = \{A \text{ l'instant où } C \text{ est évaluée, } x_i = x_{k+1} \text{ et } x_{2i} = x_{2k+2}\}.$$

- Clairement, lorsque $k = 0$, l'initialisation a produit $x_i = f(x_0) = x_1$ et $x_{2i} = f(f(x_0)) = f(x_1) = x_2$; $\mathcal{P}(0)$ est donc vérifiée.
- Supposons que $\mathcal{P}(k)$ soit vérifiée et que la $(k+1)$ -ième itération ait eu lieu. Alors au moment de l'évaluation, par hypothèse de récurrence, $x_i = x_{k+1}$ et $x_{2i} = x_{2k+2}$. L'itération nouvelle produit les affectations $x_i = f(x_{k+1}) = x_{k+2}$ et $x_{2i} = f(f(x_{2k+2})) = x_{2k+4}$. $\mathcal{P}(k+1)$ est à son tour vérifiée.

La propriété $\mathcal{P}$ est donc un invariant de boucle.

- Dire que $(x_n)_{n \geq N}$ est p -périodique signifie que pour tout $n \geq N$, $x_n = x_{n+p} = x_{n+2p}$ etc... On a bien le résultat demandé; dans ce cas, pour k suffisamment grand, on a $n = kp \geq N$ et $x_{n+kp} = x_{2n} = x_n$. Comme dans l'algorithme, la suite des indices i décrit des entiers **successifs** en partant de 0, la **condition d'arrêt** sera réalisée avec le premier entier i tel que $x_i = x_{2i}$.

Ce programme termine dès que la suite est ultimement périodique. Mais pour autant, nous l'avons vu, il ne termine pas toujours.

4. L'algorithme termine avec $xi = x_i$ et $x2i = x_{2i}$. On a alors pour tout $j \in \mathbb{N}$,

$$x_{i+j} = f^{(j)}(x_i) = f^{(j)}(x_{2i}) = x_{(i+j)+i}$$

ce qui donne en notant $i + j = n$, $x_n = x_{n+i}$ et i est une période (mais pas nécessairement la plus petite).

5. Le programme :

```
def Floyd(f, x0):
    i, xi, x2i = 1, f(x0), f(f(x0))
    while xi != x2i:
        i, xi, x2i = i+1, f(xi), f(f(x2i))
    return i

def iterees(f, x0, q):
    L = list([x0])
    for i in range(0, q):
        L.extend([f(L[-1])])
    return(L)
```

La mise en œuvre :

- (a) f est l'application qui à $x \in [0, 22]$ associe le reste dans la division euclidienne de $x^2 + 1$ par 23 (ou pour les MP : $f : x \in \mathbb{Z}/23\mathbb{Z} \rightarrow x^2 + 1 \in \mathbb{Z}/23\mathbb{Z}$).

```
f = lambda x : divmod(x**2 + 1, 23)[1]
```

Dans le cas présent, on trouve pour $x_0 = 0, 1, 2, \dots, 23$, $i = 6, 4$ ou 2 ce que confirme l'étude exhaustive :

x_0	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	p
0	1	2	5	3	10	9	13	9	13	6
1	2	5	3	10	9	13	9	13	9	6
2	5	3	10	9	13	9	13	9	13	4
3	10	9	13	9	13	9	13	9	13	2
4	17	14	13	9	13	9	13	9	13	4
5	3	10	9	13	9	13	9	13	9	4
6	14	13	9	13	9	13	9	13	9	2
7	4	17	14	13	9	13	9	13	9	4
8	19	17	14	13	9	13	9	13	9	4
9	13	9	13	9	13	9	13	9	13	2
10	9	13	9	13	9	13	9	13	9	2
11	7	4	17	14	13	9	13	9	13	6

12	7	4	17	14	13	9	13	9	13	6
13	9	13	9	13	9	13	9	13	9	2
14	13	9	13	9	13	9	13	9	13	2
x_0	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	p
15	19	17	14	13	9	13	9	13	9	4
16	4	17	14	13	9	13	9	13	9	4
17	14	13	9	13	9	13	9	13	9	2
18	3	10	9	13	9	13	9	13	9	4
19	17	14	13	9	13	9	13	9	13	4
20	10	9	13	9	13	9	13	9	13	2
21	5	3	10	9	13	9	13	9	13	4
22	2	5	3	10	9	13	9	13	9	6

Corrigé de l'exercice n° 8.7

1. On affichera les quotients pour vérifier qu'ils sont compatibles avec l'idée que la limite est 1

```
for n in range(1,100,5):
    print('n=',n,' (3n^3+6n) / (3n^3) =', (3*n**3+6*n) / (3*n**3) )
for n in range(1,1000,50):
    print('n=',n,' (3n^3+6n) / (3n^3) =', (3*n**3+6*n) / (3*n**3) )
>>>
n= 1 (3n^3+6n) / (3n^3) = 3.0
n= 6 (3n^3+6n) / (3n^3) = 1.0555555555555556
n= 11 (3n^3+6n) / (3n^3) = 1.0165289256198347
n= 16 (3n^3+6n) / (3n^3) = 1.0078125

n= 91 (3n^3+6n) / (3n^3) = 1.0002415167250331
n= 96 (3n^3+6n) / (3n^3) = 1.0002170138888888
```

2. Même principe, on stocke les résultats dans une matrice...

n	$n!$	$\left(\frac{n}{e}\right)^n \sqrt{2\pi n}$	quotient $n!/\dots$
10	3628800.0	3598695.61874	1.00836535913
20	2.43290200818e+18	2.42278684676e+18	1.00417501087
30	2.65252859812e+32	2.64517095923e+32	1.00278153624
40	8.15915283248e+47	8.14217264495e+47	1.00208546149
50	3.04140932017e+64	3.03634459394e+64	1.00166803407
60	8.32098711274e+81	8.30943831498e+81	1.00138984096
70	1.197857167e+100	1.19643200473e+100	1.00119117698
80	7.15694570463e+118	7.14949447318e+118	1.00104220396
90	1.48571596448e+138	1.48434094389e+138	1.00092635091
100	9.33262154439e+157	9.32484762527e+157	1.00083367787

```
import numpy as np
Stirling = lambda n: (n/np.exp(1))**n*(2*np.pi*n)**(1/2)
def fact(n):
    p = 1
    for i in range(1,n+1):
        p = p*i
    return p

A = np.matrix(np.zeros((11,4),dtype = float))
c = 0
for n in range(10,101, 10):
    c=c+1
    A[c, :] = [n, float(fact(n)), Stirling(n),
               float(fact(n))/Stirling(n)]
```

3. Les quotients : il semble bien tout d’abord que $n!$ soit bien plus petit que 10^n ...

n	$n!$	10^n	$n!/10^n$
1.0	1.0	10.0	0.1
2.0	2.0	100.0	0.02
3.0	6.0	1000.0	0.006
4.0	24.0	10000.0	0.0024
5.0	120.0	100000.0	0.0012
6.0	720.0	1000000.0	0.00072
7.0	5040.0	10000000.0	0.000504
8.0	40320.0	100000000.0	0.0004032
9.0	362880.0	1000000000.0	0.00036288
10.0	3628800.0	10000000000.0	0.00036288

Mais tout change si on va plus loin et l’on pourra conjecturer que $10^n = o(n!)$ (ce qui se prouve assez facilement : penser que l’on avance dans la suite des factorielles en multipliant par des nombres de plus en plus grands alors que l’on multiplie par des constantes dans une suite géométrique...).

n	$n!$	10^n	$n!/10^n$
1.0	1.0	10.0	0.1
11.0	39916800.0	100000000000.0	0.000399168
21.0	5.10909421717e+19	1e+21	0.0510909421717
31.0	8.22283865418e+33	1e+31	822.283865418
41.0	3.34525266132e+49	1e+41	334525266.132
51.0	1.55111875329e+66	1e+51	1.55111875329e+15
61.0	5.07580213877e+83	1e+61	5.07580213877e+22
71.0	8.50478588568e+101	1e+71	8.50478588568e+30
81.0	5.79712602075e+120	1e+81	5.79712602075e+39
91.0	1.35200152768e+140	1e+91	1.35200152768e+49

Corrigé de l'exercice n° 8.8

```
import random as rd
import numpy as np

def maximum(L):
    global c
    m = L[0]
    for k in range(1, len(L)):
        if L[k] > m:
            m = L[k]
            c = c+1
    return m

d = 50
p = 50
f = 1000
q = 1000

A = np.matrix(np.zeros(((f+1-d)//p+1,3), dtype = float))

ell = 0
for n in range(d, f+1, p):
    m = 0
    c = 0
    for e in range(0,q):
        #L = [rd.randint( 0,10*n) for k in range(0,n)]
        L = rd.sample(range(0, n),n)
        maximum(L)
    m = c/q
    A[ell,:] = [float(n),float(m),float(m/(np.log(n)))]
    ell = ell +1

print(A)
```

Résultats : l'hypothèse $m_n \sim \ln n$ est vraisemblable. Comme on le dit dans la dernière question on a en fait $m_n = H_n$ (voir page 321 et votre cours de maths (année 1 ou année 2) pour des propriétés de la série harmonique $(H_n)_n$.

La démonstration d'un tel résultat est assez difficile et fait appel à la notion de série génératrice que vous étudiez peut-être en cours de maths en deuxième année selon votre filière.

n	m_n	$\frac{m_n}{\ln n}$
50.0	3.512	0.897745231847
100.0	4.11	0.892475160311
150.0	4.495	0.897091832905
200.0	4.964	0.936901219119
$\vdots$	$\vdots$	$\vdots$
850.0	6.444	0.955340875564
900.0	6.369	0.936287913536
950.0	6.376	0.929925669292
1000.0	6.483	0.938510375393

Corrigé de l'exercice n° 8.9

1. Une liste de n termes représente un polynôme de degré $n - 1$.

```
def horner(P, x):
    n = len(P)-1
    r = P[n]
    for i in range(0, n):
        r = r*x+P[n-i-1]
    return r

P = [1, 2, -1, 3]
f = lambda x: 3*x**3 -x**2+ 2*x+ 1
```

2. Le nombre d'additions et de multiplication est égal à n (nombre d'itérations).
 3. Correction par récurrence sur le degré d du polynôme :

- Initialisation, $d = 0$: si $p(X) = a_0$ est de degré 0, $P = [a_0]$, $n = 0$ et il n'y a pas d'itération et la fonction retourne $r = a_0 = P[0]$.
- Supposons le résultat vrai pour les polynômes de degré d (associés aux listes de $d + 1$ éléments).

Soit P liste de $d + 2$ éléments. La sous liste $[P[1], \dots, P[d + 1]]$ est associée à $q(X) = \sum_{k=1}^{d+1} P[k]X^{k-1}$ et les d premières itérations calculent

$$r = q(x) = \sum_{k=1}^{d+1} P[k]x^{k-1}.$$

La dernière calcule

$$r * x + P[0] = x * \sum_{k=1}^{d+1} P[k]x^{k-1} + P[0].$$

On retrouve bien $p(x)$.

Corrigé de l'exercice n° 8.10

```
C = np.matrix(np.zeros((n,q)) # on réserve une matrice

for i in range(0,n):
    for j in range(0, q):
        for k in range(0,p):
            C[i, j] += A[i, k]*B[k, j]
```

Trois boucle for imbriquées : sur les deux indices de la matrice, puis pour calculer le terme défini par $c_{i,j} = \sum_{k=1}^p a_{i,k} b_{k,j}$:

Le dénombrement est immédiat : $n \times p \times q$ soit n^3 pour des matrices carrées.

Corrigé de l'exercice n° 8.11

1. Nous avons $a = bq + r$.

Si $d|a$ et $d|b$, il existe des entiers k, k' , tels que $a = kd$ et $b = k'd$. Dans ce cas $d|b$ et $d|r = a - bq = d(k - k'q)$. Tout diviseur commun à a et b est diviseur commun à b et r .

Réciproquement, si $d|b$ et $d|r$, il divise aussi $a = bq + r$.

2. (a) Notons i le numéro de l'itération (s'il y en a une), r_{i-1} et r_i les contenus des variables $r0$ et $r1$ lors de l'évaluation qui l'autorise, (q_i, r_{i+1}) les quotient et reste obtenus lors de la division correspondante.

Attention, **on ne confondra pas** $r1, r0$ avec $r_1 = b, r_0 = a$!

On a $r_{i-1} = r_i q_i + r_{i+1}$ et $0 \leq r_{i+1} < r_i$. Ainsi, à partir du rang 1, la suite d'entiers positifs $(r_i)_i$ est strictement décroissante et l'algorithme termine (sinon, contradiction).

- (b) Comme $r_{i-1} = r_i q_i + r_{i+1}$ un invariant de boucle est (question 1) :

$$\text{pgcd}(r_{i-1}, r_i) = \text{pgcd}(r_i, r_{i+1}).$$

- (c) On termine avec $r_1 = 0$, or, $r_1 = b$ ou bien r_1 est le dernier reste calculé s'il y a des itérations. Comme l'invariant de boucle est

$$\text{pgcd}(r_{i-1}, r_i) = \text{pgcd}(r_i, r_{i+1}),$$

nous aurons toujours, si le dernier reste non nul est $r_1 = r_n$,

$$\text{pgcd}(r_0, r_1) = \text{pgcd}(r_{i-1}, r_i) = \text{pgcd}(r_n, r_{n+1} = 0) = r_n.$$

3. On se contente de rappeler que $\text{divmod}(a, b)$ retourne le couple (tuple à deux éléments) formé du quotient et du reste dans la division euclidienne de a par b .

Corrigé de l'exercice n° 8.12

1. La relation (8.3.1) est une simple réécriture de la division euclidienne de r_{i-1} par r_i .
2. S'il y a n itérations, le dernier reste non nul est $d = \text{pgcd}(a, b) = r_n$ et l'on a :

$$\begin{bmatrix} r_n \\ r_{n+1} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & -q_n \end{bmatrix} \begin{bmatrix} r_{n-1} \\ r_n \end{bmatrix} \quad (8.6.1)$$

$$\begin{bmatrix} d \\ 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & -q_n \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & -q_{n-1} \end{bmatrix} \cdots \begin{bmatrix} 0 & 1 \\ 1 & -q_1 \end{bmatrix} \begin{bmatrix} r_0 \\ r_1 \end{bmatrix} \quad (8.6.2)$$

Le produit de ces matrices s'écrit $\begin{bmatrix} u & v \\ * & * \end{bmatrix}$ et l'on a bien la relation de Bezout en lisant la première ligne de l'égalité entre matrices :

$$\begin{bmatrix} d \\ 0 \end{bmatrix} = \begin{bmatrix} u & v \\ * & * \end{bmatrix} \begin{bmatrix} r_0 \\ r_1 \end{bmatrix} = \begin{bmatrix} u & v \\ * & * \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix}.$$

3. Le programme Python : c'est celui du tableau de la page 332

Corrigé de l'exercice n° 8.13

- Poser $a = r_0, b = r_1$, conduit à numérotter la suite finie $(r_i)_i$ de telle sorte que, lorsqu'il y a n itérations ou divisions euclidiennes (la première calculant r_2), $r_{n+1} = 0$ et $d = r_n \neq 0$.

Rénumérotions avec méthode les $(r_i)_i$ en partant du dernier :

r_{n+1}	r_n	r_{n-1}	$\dots$	r_k	$\dots$	r_2	r_1	r_0
G_0	G_1	G_2	$\dots$	G_{n+1-k}	$\dots$	G_{n-1}	G_n	G_{n+1}

(ce qui se fait sans problème en repérant l'invariant : « la somme des indices est $n + 1$ »). On réécrit alors

$$r_{i-1} = q_i r_i + r_{i+1} \text{ en } G_{n+2-i} = q_i G_{n+1-i} + G_{n-i}$$

ce qui donne, puisque $q_i \geq 1$ (car la suite des restes est strictement monotone),

$$G_{k+2} = q_i G_{k+1} + G_k \geq G_{k+1} + G_k.$$

Comme $G_0 = 0, G_1 = r_n \geq 1$, on vérifie sans peine que pour tout $k \in [0, n+1]$, $G_k \geq F_k$ où F_k est le $k^{\text{ième}}$ terme de la suite de Fibonacci définie par $F_0 = 0, F_1 = 1$.

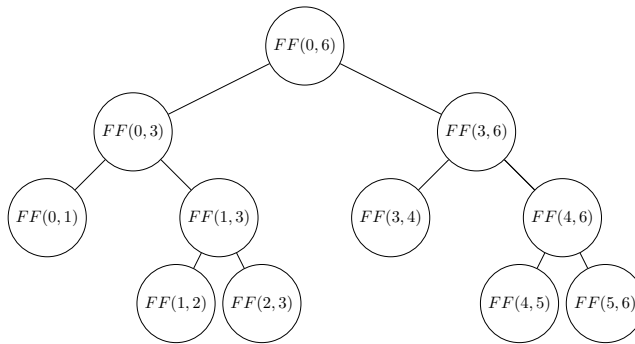
- Il vient en particulier, avec la formule (8.3.2) (et en observant que $q_1 > 0 > q_2$ et que pour deux réels quelconques $|x \pm y| \geq |x| - |y|$)

$$a \geq b = G_n \geq F_n = \frac{q_1^n - q_2^n}{q_1 - q_2} \geq \frac{q_1^n - 1}{q_1 - q_2} = \frac{q_1^n - 1}{\sqrt{5}}$$

On aura donc $q_1^n \leq \sqrt{5}b + 1$ et $n \leq \frac{\ln(\sqrt{5}b + 1)}{\ln q_1}$.

Corrigé de l'exercice n° 8.14

- Avec $a = 0, b = 1$ on tombe sur le critère d'arrêt (valeur renvoyée : 1).
Avec $a = 0, b = 2$, deux appels récurrents terminaux $FF(0, 1)$ et $FF(1, 2)$ qui renvoient 1 et 2. Donc $FF(0, 2) = 2$.
Enfin, avec $a = 0, b = 3$, les appels récurrents sont $FF(0, 1)$ et $FF(1, 3)$ qui appelle $FF(1, 2)$ et $FF(2, 3)$. Le résultat est $6=3!$
La conjecture est plus visible si on dessine l'arbre des appels : $FF(0, b) = b!$ dès que $b \geq 1$.



2. Comme dans l'exponentiation rapide, les appels récursifs, si $a < b$, sont tels que $c - a \leq b - c < b - a$.

Raisonnons alors sur la *hauteur de l'arbre* : nous dirons qu'un appel est de hauteur 0 s'il est terminal et qu'un appel est de hauteur $n + 1$ si la hauteur maximale des appels récursifs qu'il provoque est de hauteur n .

Comme l'écart entre les paramètres effectifs a et b décroît strictement, la hauteur de l'arbre ne dépassera pas $b - a - 1$. L'arbre est donc fini.

3. Preuve de correction :

On note $\mathcal{P}(p)$ la propriété « Pour tout $a \in \mathbb{N}$, $FF(a, a + p)$ est le produit des entiers $a + 1, \dots, a + p$. »

La preuve par récurrence sur $p \geq 1$ est sans problème.

L'intérêt de cette fonction est qu'elle permet de calculer les factorielles en évitant de multiplier des nombres de tailles trop différentes.

Corrigé de l'exercice n° 8.15

1. Soient $n \in \mathbb{N}$, et $k \in \mathbb{N}$. Soit encore E , un ensemble à $n + 1$ éléments, dont nous choisissons un élément a . Les parties à $k + 1$ éléments sont dans deux classes disjointes :

- celle formée des sous-ensembles qui ont $k + 1$ éléments dans $E \setminus \{a\}$ et donc ne contiennent pas a . Il y en a $\binom{n}{k+1}$.

- celle formée des sous-ensembles qui ont $k + 1$ éléments et contiennent a . Il y en a $\binom{n}{k}$. On les obtient en effet en prélevant k éléments dans $E \setminus \{a\}$ et en leur adjoignant a .

On a donc $\binom{n+1}{k+1} = \binom{n}{k+1} + \binom{n}{k}$

2. Ce programme prend en argument une liste et un entier $k \geq 0$. Il termine clairement lorsque $k > n$, $k = 0, 1$.

Nous allons prouver qu'il termine en considérant la longueur $n = \text{len}(L)$ lors de l'appel.

- Si $n = 0$, alors, soit $k = 0$, soit $k > N$ et le programme termine directement.
- Supposons que le programme termine avec toute liste de longueur n et tout $k \geq 0$. Un appel avec une liste L de longueur $n + 1$ produit deux appels

récur­sifs avec une liste de longueur n (L privée de son premier élément). Ces appels terminent donc par hypothèse de récurrence et il est clair que la fonction termine ensuite après les deux boucles `for`.

3. **Spécification** : Cette fonction renvoie une liste vide ou une liste de listes. Cette liste correspond à l'ensemble des parties à k éléments de L .

Preuve de correction par récurrence sur $n = \text{len}(L)$.

On note $\mathcal{P}(n)$ la propriété :

« La fonction renvoie une liste des parties à k éléments de L pour tout $k \in \mathbb{N}$. »

- Le résultat est vrai pour $n = 0$, en effet, si $k = 0$ la fonction renvoie `[]` (seul l'ensemble vide est solution) ; si $k > 0$ la fonction renvoie la liste vide : il n'y a pas de solution.

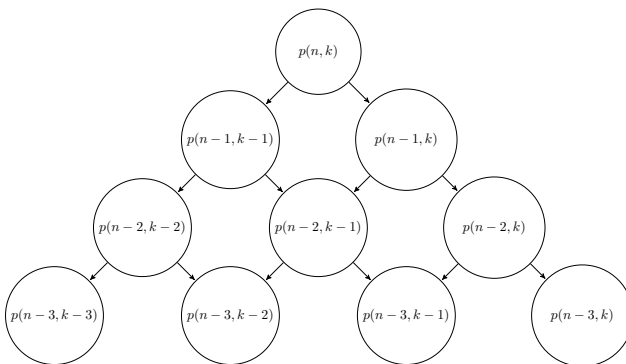
- Supposons le résultat établi pour un certain n .

$L1$ et $L2$ sont alors les parties à $k - 1$ et k éléments dans L privée de a . En ajoutant a aux éléments de $L1$, on a clairement les parties à $k + 1$ éléments de L qui contiennent a . R est la liste formée de la réunion des éléments ainsi obtenus et des éléments de $L1$. C'est bien la liste des parties à k éléments de L .

Observation : ce programme construit un ensemble en suivant la démonstration combinatoire de la question 1. Comme souvent la démonstration d'une formule (ici on répartit les sous-ensembles à deux éléments en deux classes), la construction du programme et sa preuve vont de pair.

4. Dans la fonction elle-même, la liste L est amputée de son premier élément. Cette liste n'apparaîtra plus dans aucun appel récursif de niveau inférieur. Par contre, comme c'est toujours le premier élément qui est enlevé, les appels de niveaux 1, 2, 3, ... font intervenir les même listes, respectivement, $L[1, :]$, $L[2, :]$, $L[3, :]$, ...

Regardons l'arbre de ces appels (sur la figure, $p(n - 3, k - 1)$ représente l'appel parties($L[3, :], k - 1$) et l'arbre est transformé en **graphe des appels**). Il apparaît alors que les appels de niveau h à partir de $p(n, k)$, sont redondants sauf $p(n - h, k - h)$ et $p(n - h, k)$.



5. On reprend le programme avec un dictionnaire pour la mémorisation :

```

def partiesD(L, k, D ):
    ''' Ln: list; k:int, avec k >= 0; D:dictionnaire. '''
    global c1
    c1 += 1 # compteur pour les appels récursifs (sauf 1)

    n    = len(L)
    if n not in D:
        D[n] = {}

    elif k in D[n]:
        return D[n][k] # pas de calculs redondants

    if k > n :
        # pas de partie à plus de n élts.
        D[n][k] = []
        return []

    elif k == n:
        D[n][n] = [L]
        return [L]

    elif k == 0:
        # partie vide, seule solution.
        D[n][0] = [[]]
        return [[]]

    elif k == 1:
        # singletons.
        D[n][1] = [[a] for a in L]
        return D[n][1]

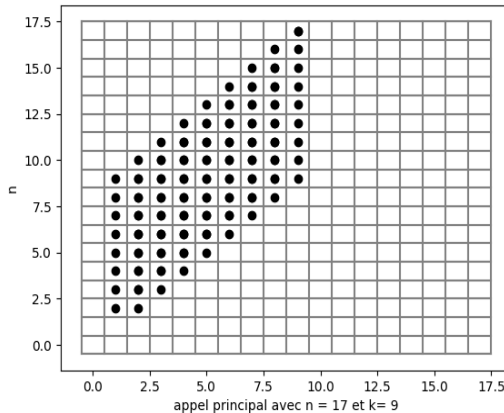
    else:
        assert k >= 2 and k <= n-1
        # précondition pour ce qui suit: 2 <= k <= n-1.
        a    = L[0]
        Ea   = L[1:n]
        L1   = partiesD(Ea, k-1, D)
        L2   = partiesD(Ea, k, D)
        R    = []
        for p in L1:
            R.append(q + [a])
        for q in L2:
            R.append(q)
        D[n][k] = R # on stocke le résultat
        return R

```

À titre de comparaison, le décompte, des appels récursifs provoqués par parties(L, k) et partiesD(L, k) avec des listes L de 3 à 17 éléments et $k = \text{len}(L)//2+1$, grâce à des variables globales c0 et c1 qui jouent le rôle de compteurs dans nos deux fonctions.

<pre> L = ['a', 'b', 'c', 'd', ... , 'p', for n in range(3, len(L)+1): L1 = L[0:n] k = n//2+1 c0 = 0 R = parties(L1, k) c1 = 0 R1 = partiesD(L1, k, {}) print('n= %s, k = %s, c0 = %s' print('n= %s, k = %s, c1 = %s\ </pre>	<pre> n= 3, k = 2, c0 = 5 n= 3, k = 2, c1 = 3 ... n= 11, k = 6, c0 = 923 n= 11, k = 6, c1 = 51 n= 12, k = 7, c0 = 1847 n= 12, k = 7, c1 = 61 ... n= 16, k = 9, c0 = 25739 n= 16, k = 9, c1 = 113 n= 17, k = 9, c0 = 48619 n= 17, k = 9, c1 = 129 </pre>
---	---

6. Complexité



On visualise ici les 129 appels provoqués par l'appel principalD(17, 9).

Supposons que $2 \leq k \leq n-1$. Nous pouvons visualiser les appels récursifs sur l'arbre de la page 354.

- A la profondeur $h = 1$, il y en a 2, pour les deux fonctions.
- à la profondeur $h = 2$, il y en a 2^2 pour la fonction initiale et **3 au plus** pour la fonction partiesD ;
- à la profondeur $h = 3$, il y en a 2^3 pour la fonction initiale et **4 au plus** pour la fonction partiesD ;

Si on tient compte du fait qu'il n'y a pas d'appel avec $k < 0$, le nombre d'appels récursifs à la profondeur h est donc $\min(h+1, k)$. Le nombre total d'appels est donc

majoré par

$$A(n, k) = \min(2, k) + \min(3, k) + \dots + \min(n-1, k) \leq (n-1)k.$$

Corrigé de l'exercice n° 8.16

1. (a) Rien de sorcier, il faut juste penser que l'objet lu avec **input** est une chaîne de caractères (par exemple '6' ou '6.45' et qu'il faut la convertir en flottant avec le constructeur **float()** on écrira donc :

```
x = float(input('entrer x = '))
```

- (b) La solution est dans le programme de gauche, comme on va le justifier immédiatement.
2. (a) **Premier algorithme** : notons simplement C la condition $C(x,s)=(s \leq x)$ et raisonnons par récurrence pour prouver à la fois que le programme termine et qu'il donne le résultat attendu.
 - Initialisation : à l'entrée dans la boucle, donc à la première évaluation de la condition C, n contient 1, s contient $s_1 = 1$;
 - Hérédité : supposons qu'au moment de la $k^{\text{ième}}$ évaluation de C, n contienne k et s contienne s_k , alors, soit la condition d'arrêt ($\text{non}(C) = (s > x)$) est réalisée et le programme termine, soit n est incrémenté et s subit l'affectation $s=s+1/n$. A l'évaluation suivante, n contient $k+1$ et s contient $s_k + \frac{1}{k+1} = s_{k+1}$.

En conséquence, nous avons montré que lorsque k est le nombre d'évaluations de la condition C, n contient k et s contient s_k . Par ailleurs, si comme nous l'apprend le cours de maths, $\lim s_k = +\infty$, alors il existe un plus petit entier k_0 pour lequel $s_{k_0} > x$. Pour les entiers qui précèdent (s'il y en a) la condition C est satisfaite, arrivé à $n = k_0$, le programme termine. Le contenu de n est la valeur k_0 attendue puisque la condition d'arrêt est vérifiée et que pour $k_0 - 1$ elle ne l'était pas (y compris si $k_0 = 1$, ce que l'on obtiendrait pour $x \leq 1$).

Second algorithme : pour ce programme, on constate que s est réinitialisé à chaque itération dans la boucle **while** et que la suite d'instructions

```
s = 1
for j in range(2,n+1):
    s = s+1/j
```

réalise très exactement le calcul de s_n comme nous l'avons signalé dans le cours. La condition évaluée avant chaque itération éventuelle de la boucle **while** est donc $(s_{k+1} \leq x)$, k étant le nombre d'itérations. Ce programme retourne les mêmes résultats que le précédent.

- (b) Le programme de présentation de la boucle `while`, page 42 montre qu'à partir d'un certain rang le flottant $10 * (-k)$ est évalué à 0. pour la machine. A partir de ce rang s n'est plus modifiée, le programme ne termine pas car la suite des flottants (s_k) est stationnaire (et elle converge!).
- (c) **Dénombrement des opérations, premier programme :** on a vu que le nombre d'itérations est $N-1$ lorsque la boucle `while` termine et que n contient N . Il y a donc $2(N-1)$ additions et $2(N-1)$ divisions.

Dénombrement des opérations, second programme : Le nombre d'itérations dans la boucle `while` est le même, mais la $n^{\text{ième}}$ itération réalise $(n-1)$ additions et divisions dans la boucle `for` (on commence l'itération à $j=2$, elle s'arrête à $j=n$), il y a donc en tout

$$\sum_{n=1}^{N-1} (n-1) = \sum_{n=0}^{N-2} k = \frac{1}{2}(N-2)(N-1) \sim \frac{N^2}{2}$$

divisions et $N-1$ additions de plus.

Ces deux programmes qui font le même calcul ne sont donc pas du tout équivalents. Le second répète inutilement des calculs identiques.

3. L'expression `np.array(range(1, n+1)) ** (-1.)` calcule bien la somme $\sum_{k=1}^n \frac{1}{k}$, en effet :

```
(0) np.array(range(1, n+1))
```

construit un tableau formé des entiers de 1 à n (on rappelle que `range(deb, fin)` décrit un intervalle ouvert à droite).

```
(1) np.array(range(1, n+1)) ** (-1.)
```

élève chaque élément du tableau à la puissance -1 ce qui donne

```
[1, 0.5, 0.3333..]
```

(attention avec un entier -1 à la place du flottant -1. on obtient `[1,0,...,0]`).

Enfin,

```
(2) np.sum(np.array(range(1, n+1)) ** (-1.))
```

est la somme du tableau précédent. En conséquence, les divisions et additions sont cachées dans les opérations (1) et (2). Rien de mieux dans le fond que le mauvais programme proposé dans la colonne de droite bien que la fonction `sum` soit plus rapide qu'une boucle `for`.

Corrigé de l'exercice n° 8.17

1. Les deux observations de l'énoncé conduisent à l'algorithme

```
def pgcd_soustr(a,b):
    '''
    a, b :int avec la pré-condition
        a>=0 et b>= 0 et (a>0 ou b >0);
    Renvoie le pgcd de a et de b.
    '''
    # vérification de la pré-condition
    assert isinstance(a,int) and isinstance(b,int)
    assert a>=0 and b>=0 and(a>0 or b>0)

    if a < b:
        a, b = b, a
    while b !=0:
        a = a-b
        if a<b:
            a,b=b,a
    return a
```

2. Nous avons à prouver que l'algorithme termine et qu'il retourne le pgcd de a et de b .

• Terminaison

Notons a_0 et b_0 les contenus des variables a et b avant la première itération.

Notons pour $i \geq 1$, a_i et b_i les contenus des variables a et b après l'itération n° i , si elle existe.

Qu'il y ait permutation ou pas, $\{a_i, b_i\} = \{a_{i-1} - b_{i-1}, b_{i-1}\}$. Posons alors $F(i) = a_i + b_i$. Il est clair que s'il y a une itération au moins

$$F(i) = a_i + b_i = a_{i-1} < F(i-1).$$

Nous tenons notre **variant de boucle**, l'algorithme termine.

• Correction (totale)

Si un des arguments effectifs, a ou b , est nul, alors $b_0 = 0$ (après permutation éventuelle de a et de b) et la fonction renvoie $a_0 = \text{pgcd}(a_0, 0)$.

Sinon, un invariant de boucle est $\mathcal{P}(i) = (\text{pgcd}(a, b) = \text{pgcd}(a_i, b_i))$.

- c'est clair si $i=0$.

- supposons que $\mathcal{P}(i-1)$ soit vraie, comme $\{a_i, b_i\} = \{a_{i-1} - b_{i-1}, b_{i-1}\}$ on a $\text{pgcd}(a_i, b_i) = \text{pgcd}(a_{i-1} - b_{i-1}, b_{i-1}) = \text{pgcd}(a_{i-1}, b_{i-1}) = \text{pgcd}(a, b)$, donc $\mathcal{P}(i)$.

Corrigé de l'exercice n° 8.18

1. (a) On rappelle que l'on définit le produit de deux matrices $A \in \mathcal{M}_{n,p}(\mathbb{K})$ et $B \in \mathcal{M}_{p,q}(\mathbb{K})$ comme étant la matrice $C \in \mathcal{M}_{n,q}(\mathbb{K})$ telle que

$$c_{i,j} = \sum_{k=1}^p a_{i,k} b_{k,j}.$$

```
(b) C = np.matrix(np.zeros((n,q), dtype = float))
for i in range(0,n):
    for j in range(0,q):
        for k in range(0,p):
            C[i,j] += A[i,k]*B[k,j]
```

- (c) Trois boucles for imbriquées : sur les deux indices de la matrice, puis pour calculer le terme défini par

$$c_{i,j} = \sum_{k=1}^p a_{i,k} b_{k,j}$$

ce qui s'écrit en Python, avec le décalage d'indices tel que $c_{i+1,j+1} = C[i,j]$,

$$C[i,j] = \sum_{k=0}^{p-1} A[i,k] B[k,j]$$

Le dénombrement est immédiat : $n \times q \times p$ soit n^3 multiplications pour des matrices carrées.

2. (a) Partons de $\spadesuit = \sum_{k=1}^n (x_{2k} + y_{2k-1})(y_{2k} + x_{2k-1})$

$$\begin{aligned} \spadesuit &= \sum_{k=1}^n x_{2k-1}x_{2k} + y_{2k-1}y_{2k} + x_{2k}y_{2k} + x_{2k-1}y_{2k-1} \\ &= \sum_{k=1}^n x_{2k-1}x_{2k} + \sum_{i=1}^{2n} y_{2k-1}y_{2k} + \sum_{k=1}^n x_{2k}y_{2k} + x_{2k-1}y_{2k-1} \\ &= f(X) + f(Y) + \langle X|Y \rangle \end{aligned}$$

Ce qui donne bien,

$${}^tXY = \langle X|Y \rangle = \sum_{i=1}^{2n} x_i y_i = \sum_{k=1}^n (x_{2k} + y_{2k-1})(y_{2k} + x_{2k-1}) - f(X) - f(Y) \quad (8.6.3)$$

- (b) On commence par programmer la fonction F qui peut prendre en argument une matrice ligne ou colonne (`numpy.matrix`) ou une liste. On teste l'ensemble des situations et on transforme l'objet en liste pour un traitement unique.

```
import numpy as np
import sympy as sp # explications plus bas.

def F(T):
    '''
    T :liste|matrice ligne|matrice colonne
    de longueur ou dimension paire.
    '''
    if isinstance(T, (np.matrix, sp.Matrix))\
        and len(T.shape) == 2:
        p, q = T.shape
        if p == 1 and q%2 == 0:
            X = [T[0,j] for j in range(0,q)]
            n = q//2
        elif q == 1 and p%2 == 0:
            X = [T[i,0] for i in range(0,p)]
            n = p//2
        else:
            print('Vecteur de taille paire attendu\
                  à la place de \'n%s.%'(T))
            return None

    elif isinstance(T,list) :
        if not len(T)%2 ==0:
            print('Liste ou vecteur de taille paire\
                  attendu à la place de %s.%'(T))
            return None
        else:
            X = T
            n = len(T)//2
    else:
        print('Type non traité pour %s (de type %s)'\
              %(T, type(T)))
        return None

    s = 0
    for i in range(0,n):
        s += X[2*i]*X[2*i+1]
    print(s)
    return s
```

C'est une situation dans laquelle une programmation défensive s'impose. Évidemment cela fait beaucoup de code pour trois lignes de calcul.

Rappelons que si nous extrayons une ligne ou une colonne d'une matrice de numpy, nous obtiendrons une matrice à une ligne et N colonnes (ou à N lignes et une colonnes). Nous traitons le cas des listes parce que c'est pratique pour les tests.

```
def stockeFA(A):
    '''
    A: matrice carrée de taille paire N = 2n;
    Renvoie une liste: [f(A0), ...f(A(n-1)) ]
    construite sur les lignes de A
    '''
    if isinstance(A, (np.matrix, sp.Matrix)) \
        and len(A.shape) == 2:
        p, q = A.shape
        if p == q and p%2 == 0:
            n = p//2
        else:
            print('On attend un matrice carrée de taille \
                paire. Ici p=%, q=%s'%(p,q))
            return None
    else:
        print('On attend un matrice. Or type(M) = %s'\
            %(type(A)))
        return None
    L = []
    for i in range(0,2*n):
        L.append(F(A[i,:]))
        # A[i,:] est la ligne i de A.
    return L

def stockeFAFB(A, B):
    '''les lignes de B sont les colonnes de tB.'''
    return stockeFA(A, stockeFA(B.transpose()))
```

- (c) Le nombre de * dans la fonction F est clairement $n = \frac{N}{2}$;

Dans stockeFA, il n'y en a pas si on remplace $2*n$ par p (ou $q = N$) en dehors de celles provoquées par les $2*n$ appels à FA ce qui en fait donc $2n^2$.

Un appel de stockeFAFB en provoque donc $4n^2$.

3. (a) Revenons alors au calcul de $C = AB$. Chaque terme $C[i, j] = A_i B^j$ s'exprime avec la formule :

$$\sum_{k=0}^{n-1} (A[i, 2k+1] + B[2k, j]) * (B[2k+1, j] * A[i, 2k]) - f(A_i) - f(B^j).$$

```

def produitAB(A,B):
    '''
    '''
    def verifierMatrice(A):
        if isinstance(A, (np.matrix, sp.Matrix)) \
            and len(A.shape) == 2:
            p, q = A.shape
            if p == q and p%2 == 0:
                n = p//2
                return n
            else:
                print('On attend un matrice carrée de \
                    taille paire. Ici p=%, q=%s'%(p,q))
                return None
        else:
            print('On attend une matrice. \
                Or type(M) = %s'%(type(A)))
            return None

    n = verifierMatrice(A)
    m = verifierMatrice(B)

    if n == None or m == None:
        return None
    elif n != m:
        print('Les matrices ne sont pas de la même taille!')
        return None
    elif isinstance(A, np.matrix) and isinstance(B,np.matrix):
        C = np.matrix(np.zeros((2*n, 2*n)))
    else:
        #isinstance(A, sp.Matrix) or isinstance(B,sp.Matrix):
        C = sp.zeros(2*n, 2*n)

    LA, LB = stockeFAFB(A, B)
    for i in range(0,2*n):
        for j in range(0,2*n):
            for k in range(0,n):
                C[i,j] += \
                    (A[i,k+k+1]+B[k+k,j])*(B[k+k+1,j]+A[i, k+k])
                # on prend garde de ne pas rajouter de
                # multiplication dans les indices
            C[i,j] -= LA[i] + LB[j]

    return C

```

- (b) Complexité du programme : comme $f(A)$ et $f(B)$ sont déjà calculés, le nombre de multiplication est $n^3 + 4n^2 \sim n^3$ au lieu de $N^3 = 8n^3$ avec une transcription directe de la formule du produit comme dans la première question.

Nous avons écrit ces fonctions de telle sorte qu'elle puissent prendre en compte des matrices de numpy et des matrices de **sympy**. Ce module de calcul formel nous per-

met dans le cas présent de vérifier facilement que nous avons bien transcrit nos formules.

```
>>> A = np.matrix(np.ones((4,4), dtype = int) ).reshape(4,4)
>>> B = np.matrix(np.ones((4,4), dtype = int) ).reshape(4,4)

>>> produitAB(A,B)
[[4. 4. 4. 4.]
 [4. 4. 4. 4.]
 [4. 4. 4. 4.]
 [4. 4. 4. 4.]]

>>> As = sp.Matrix( [['x00', 'x01', 'x02', 'x03'],\
                    ['x10', 'x11', 'x12', 'x13'],\
                    ['x20', 'x21', 'x22', 'x23'],\
                    ['x30', 'x31', 'x32', 'x33']])

>>> Bs = sp.Matrix( [['y00', 'y01', 'y02', 'y03'],\
                    ['y10', 'y11', 'y12', 'y13'],\
                    ['y20', 'y21', 'y22', 'y23'],\
                    ['y30', 'y31', 'y32', 'y33']])

>>> C = produitAB(As,Bs)
>>> C[0,0].expand()
x00*y00 + x01*y10 + x02*y20 + x03*y30
>>> C.expand()
Matrix([[x00*y00 + x01*y10 + x02*y20 + x03*y30,
        ...,
        x30*y03 + x31*y13 + x32*y23 + x33*y33]])
```

Graphes

Ce chapitre propose quelques définitions et algorithmes fondamentaux concernant les graphes. L'étude des graphes a pris son essor au $XIX^{ième}$ siècle¹ et s'est considérablement développée au cours des $XX^{ième}$ et $XXI^{ième}$ siècles. Ses problématiques qui sont du domaine des mathématiques pures et de l'informatique sont alimentées par les besoins de l'industrie ou des réseaux de télécommunications, la physique, la chimie ou encore l'économie et les sciences sociales.

9.1 Définitions

Définition 9.1 *vocabulaire minimal des graphes (non orientés)*

Un **graphe non orienté** est un couple $G = (S, A)$ dans lequel S est un ensemble non vide (dont les éléments sont appelés **sommets** ou **nœuds** du graphe) et A un ensemble de parties à deux éléments de S (dont les éléments sont appelés **arêtes** du graphe).

Deux sommets sont **adjacents** ssi il existe une arête qui les relie.

Le **degré** d'un sommet est égal au nombre d'arêtes dont il est extrémité. On le notera $d^o(i)$.

Une **chaîne** reliant $a \in S$ à $b \in S$ est une suite d'éléments de A , $\{i_1, i_2\}, \{i_2, i_3\}, \dots, \{i_{n-1}, i_n\}$ telle que $i_1 = a, i_n = b$.

Le **sous-graphe** de $G = (S, A)$ **induit** par l'ensemble $S' \subset S$ est le graphe $G' = (S', A_{S'})$ dont les arêtes sont les arêtes de A qui relient deux éléments de S' .

Une **composante connexe** de G est un sous-graphe dans lequel deux sommets distincts sont toujours reliés par un chemin et qui est **maximal** pour cette propriété. Un graphe est **connexe** s'il possède une unique composante connexe.

- Dans ce qui suit, nous considérerons des graphes sans **boucle** (c'est-à-dire, sans sommet relié à lui-même) et dans lesquels deux sommets sont reliés par au plus une

1. Quoiqu'on la fasse traditionnellement remonter à 1735 avec la résolution informelle par Euler du problème des sept ponts de Königsberg.

arête (dans le cas contraire il s'agit de multigraphes).

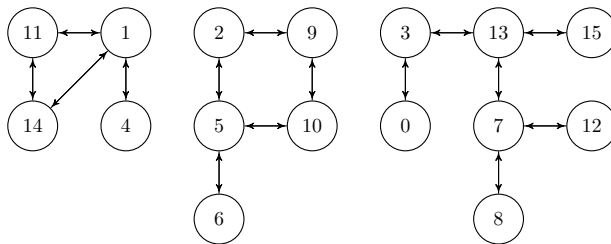


Illustration : On représente ici un graphe formé de 16 sommets (numérotés de 0 à 15) et dont les arêtes sont représentées par des flèches à deux points. Il existe par exemple une chaîne de longueur 3 reliant le sommet 3 au sommet 12 : $\{3, 13\}$, $\{13, 7\}$, $\{7, 12\}$.

Il est clair que les composantes connexes de ce graphe, facilement identifiables, forment une **partition** de l'ensemble S (voir à ce propos l'exercice 9.1 ci-dessous).

Exercice 9.1 composantes connexes et classes d'équivalence

Montrer que la relation entre sommets d'un graphe non orienté G , définie par $R(a, b) = \ll a = b \text{ ou il existe une chaîne reliant } a \text{ à } b \gg$, est une relation d'équivalence et que ses classes d'équivalence sont les (ensembles de sommets associés aux) composantes connexes du graphe.

Une conséquence est qu'à tout graphe non orienté on peut associer une relation d'équivalence dont les classes d'équivalence sont les composantes connexes du graphe. Ces composantes connexes forment donc une partition de l'ensemble S .

Corrigé en 9.1 page 390.

On définit de façon analogue des **graphes orientés** pour lesquels on parlera d'arcs (et non plus d'arêtes), de chemins (et non plus de chaînes). Les degrés sortant et entrant, notés respectivement $d_+^\circ(x)$ et $d_-^\circ(x)$ viennent compléter la notion de degré d'un sommet.

Définition 9.2 vocabulaire minimal des graphes orientés

Un **graphe orienté** est un couple d'ensembles (S, A) avec $A \subset S^2$.

Les éléments de A que l'on appelle alors des **arcs** sont donc des couples (i, j) (et non plus des paires $\{i, j\}$) de sommets. On pourra aussi les noter $i \rightarrow j$ et parler d'**extrémités** initiale et terminale.

Un **chemin** reliant $a \in S$ à $b \in S$ est une suite d'éléments de A , $(i_1, i_2), (i_2, i_3), \dots, (i_{n-1}, i_n)$ telle que $i_1 = a, i_n = b$. Un **circuit** est un chemin dont l'origine et l'extrémité sont égales.

On appelle **degré sortant** d'un sommet i le nombre d'arcs $(i, j) \in A$. Ce degré sortant est noté $d_+^\circ(x)$. Le degré entrant est défini de façon analogue.

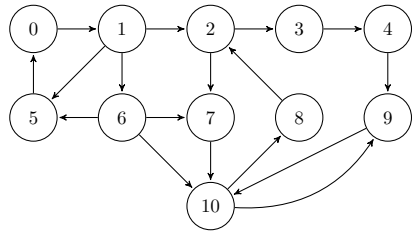
Une **composante fortement connexe** d'un graphe orienté G est un sous-graphe de G dans lequel « pour tout couple (u, v) de nœuds distincts, il existe un chemin de u à

v » et qui est **maximal** pour cette propriété. Un graphe orienté est **fortement connexe** lorsqu'il possède une unique composante fortement connexe.

Les composantes fortement connexes d'un graphe orienté forment une partition de l'ensemble de ses sommets comme on le déduit de l'exercice qui suit.

Exercice 9.2 composantes fortement connexes et classes d'équivalence

1. Déterminer les composantes fortement connexes du graphe orienté de la figure ci-contre.



2. Soit $G = (S, A)$ un graphe orienté.
 - (a) Vérifier que la relation $R(u, v) = \ll u = v \text{ ou il existe un chemin de } u \text{ à } v \text{ et un chemin de } v \text{ à } u \gg$ est une relation d'équivalence définie sur $S \times S$.
 - (b) Quelles sont ses classes d'équivalences ?
3. **Vrai ou faux ?** On suppose que $a \neq b$.
 - Si a et b sont dans une même composante fortement connexe de G , alors il existe un chemin de b vers a et un chemin de a vers b .
 - Si a et b sont dans une même composante fortement connexe alors il existe un circuit qui passe par a et b .
 - Si a et b sont dans une même composante fortement connexe alors il existe un **circuit élémentaire** (ie : qui ne passe pas deux fois par le même sommet) qui passe par a et b .
 - S'il existe un circuit qui passe par a et b alors, a et b sont dans une même composante fortement connexe.

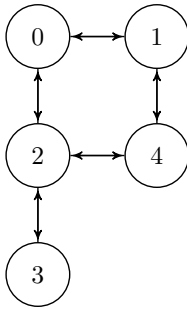
Corrigé en 9.2 page 391.

- On peut encore associer des poids aux arêtes ou aux arcs (que l'on note alors (i, j, w_{ij})). On parle dans ce cas de **graphes pondérés** ou de **graphes valués**.
- Nous décrirons deux façons d'implémenter les structures de graphes (orientés ou pas) : **listes d'adjacence** par et par **matrices d'adjacence**. C'est à partir de la dernière représentation que nous illustrerons la plupart de nos algorithmes.

9.2 Listes d'adjacence

Définition 9.3

Soit $G = (S, A)$ un graphe éventuellement orienté et pondéré. Une représentation par **listes d'adjacence** consiste à associer à chaque sommet du graphe la liste de ses successeurs (ou voisins)



$$\left\{ \begin{array}{l} 0 : [1, 2] \\ 1 : [4, 0] \\ 2 : [0, 4, 3] \\ 3 : [2] \\ 4 : [1, 2] \end{array} \right.$$

Ici avec un tableau associatif dont les clés sont les sommets, les valeurs les voisins.

La classe `graphes_non_orientes()` décrite au chapitre 10, section (10.2), adopte cette représentation des listes d'adjacence avec un tableau associatif.

Exercice 9.3 construction de listes d'adjacence

1. Définir une fonction qui prend en argument une liste des sommets et une liste d'arêtes d'un graphe non orienté et renvoie le tableau associatif ou dictionnaire dont les clés représentent les sommets et les valeurs associées les listes de voisins : `{ sommet : [a,b,c],...}`
2. Écrire une fonction qui prend le tableau renvoyé par la fonction ci-dessus définie et renvoie un tableau associatif de la forme `{ sommet : degré,...}`.
3. On souhaite une représentation dans laquelle toutes les informations sont présentes dans un seul dictionnaire.

$$\left\{ \begin{array}{l} \text{sommet}_0 : \left\{ \begin{array}{ll} 'd-' & : \\ 'd+' & : \\ 'successeurs' & : [[s_0, w_0], [s_1, w_1], ..] \\ & \vdots \end{array} \right. \\ \\ \text{sommet}_1 : \left\{ \begin{array}{ll} 'd-' & : \\ 'd+' & : \\ 'successeurs' & : [[s_0, w_0], [s_1, w_1], ..] \\ & \vdots \end{array} \right. \\ \\ \vdots \end{array} \right.$$

Écrire une fonction `graphe_dictionnaire(S,A)` qui prend en argument une liste des sommets `S` et une liste d'arcs ou d'arêtes `A` d'un graphe orienté et valué et renvoie le dictionnaire de dictionnaires.

Prévoyez qu'en recopiant les listes, on peut oublier des sommets qui vont apparaître dans les arcs, faire des doublons dans la liste des arcs, insérer des triplets

au lieu de conteneurs de longueur deux etc. Programmation paranoïaque attendue.

On testera avec le graphe orienté de la page 367.

Corrigé en 9.3 page 392.

9.3 Matrices d'adjacence

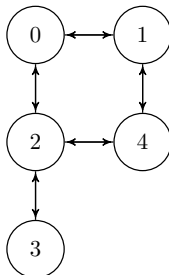
Par souci de simplicité dans tout ce qui suit, cours et programmes, on identifie l'ensemble S des sommets à un intervalle $\llbracket 0, n - 1 \rrbracket$.

Définition 9.4 *matrices associées à un graphe*

La **matrice d'adjacence** associée au graphe non orienté $G = (S, A)$ est la matrice

(symétrique) $A_G \in \mathcal{M}_n(\mathbb{R})$ définie par
$$\begin{cases} a_{ij} = a_{ji} = 1 & \text{si } \{i, j\} \in A; \\ a_{ij} = 0 & \text{sinon.} \end{cases}$$

Un graphe non orienté à 5 sommets et 5 arêtes avec sa matrice d'adjacence :



$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

Définition 9.5 *matrices associées à un graphe orienté et pondéré*

— Si $G = (S, A)$ est un **graphe orienté**, on lui associe la matrice d'adjacence

A_G définie par
$$\begin{cases} a_{ij} = 1 & \text{si } (i, j) \in A; \\ a_{ij} = 0 & \text{sinon.} \end{cases}$$

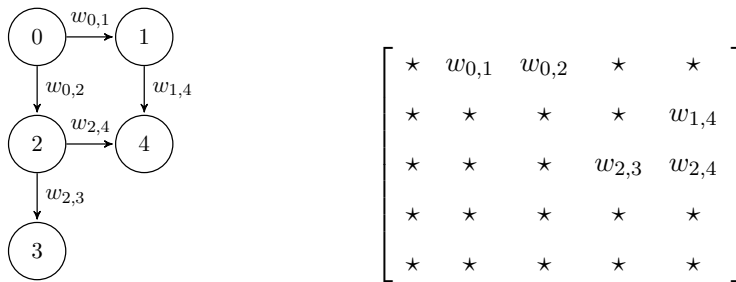
— Si $G = (S, A)$ est un **graphe orienté et pondéré** (ou **valué**), on lui associe la

matrice d'adjacence A_G définie par
$$\begin{cases} a_{ij} = w_{i,j} & \text{si } (i, j, w_{i,j}) \in A, \\ a_{ij} = \star & \text{sinon;} \end{cases} \quad \text{où :}$$

- w_{ij} est le poids associé à l'arc (i, j) ,

- $\star$ est une constante différente de tous les w_{ij} .

Un graphe orienté et pondéré à 5 sommets et 5 arcs avec une matrice d'adjacence :



Exercice 9.4 construction de matrices d'adjacence

Il serait assez pénible de construire nos matrices à la main. On suppose donc qu'un graphe est donné par une liste de sommets S et une liste A d'arêtes. Par exemple, pour le dernier graphe orienté :

$S1 = [k \text{ for } k \text{ in range}(0, 5)]$

$A1 = [[0, 1], [1, 4], [0, 2], [4, 2], [2, 3]]$.

On se propose de construire les matrices d'adjacence à partir de données de ce type.

1. Écrire une fonction `matriceAdjacenceNO( $S, A$ )` qui prend en arguments une liste S des entiers de 0 à n et une liste A dont les termes sont des couples de S et retourne la matrice d'adjacence du graphe non orienté ainsi défini.
2. Réfléchir à la façon de représenter un graphe orienté et pondéré et écrire une fonction `matriceAdjacenceGO( $S, A$ )` qui prend des arguments S et A comme précédemment et retourne la matrice d'adjacence du **graphe orienté** ainsi défini. On veillera en particulier au choix du terme correspondant à l'absence d'arc.

Corrigé en 9.4 page 394.

9.4 Parcours en profondeur, composantes connexes

L'idée du parcours en profondeur est de partir d'un sommet, de le cocher, de choisir un sommet adjacent que l'on coche à son tour et de recommencer. Quand, arrivé à un point où on ne peut poursuivre car tous les sommets adjacents sont cochés, on retourne en arrière et on recommence. On peut jouer à cela avec le graphe de la page 366.

Pour implémenter cet algorithme, on définit deux procédures, « `parcoursPR` » et « `parcoursProfondeur` ».

parcoursPR assure un traitement partiel : lorsque i est le sommet donné en argument, pour chaque sommet adjacent à i , elle réalise un appel récursif à `parcoursPR`. Le sommet i lui-même est traité après tous ces appels récursifs par $f(i)$.

L'instruction print permet de visualiser les appels récursifs et de suivre pas à pas l'algorithme en marche (tableau suivant). A sa place on pourrait assurer un traitement du nœud avant d'explorer les nœuds adjacents.

parcoursProfondeur assure les appels principaux de la procédure précédente en explorant tous les sommets qui ne sont pas marqués. Alors que **parcoursPR** ne permet pas d'explorer au-delà de la composante connexe de *i*, **parcoursProfondeur** permet de « sauter » d'une composante à l'autre. Le traitement des nœuds est réalisé après l'exploration en profondeur.

parcoursPR et parcoursProfondeur (procédure principale)

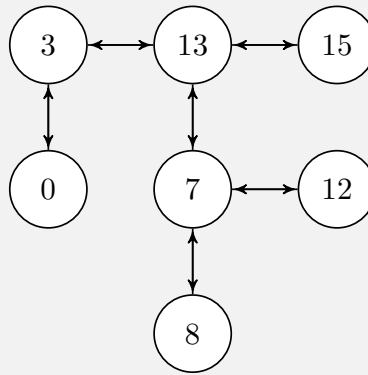
```
def parcoursPR(M, n, i, f, marque):
    '''
    M: matrice d'adjacence, n: sa taille,
    i: noeud à partir duquel lancer l'exploration;
    marque est une liste (vide lors de l'appel
    principal).
    '''
    marque.append(i)
    for j in range(0,n):
        if M[i,j] != 0 and j not in marque:
            print('appel récursif avec j = %s'%(j))
            parcoursPR(M, n, j, f, marque)
    f(i)
    return None

def parcoursProfondeur(M, f):
    n, p = M.shape
    if n != p:
        print('Erreur, on attend une matrice carrée!')
        return None
    marque = []
    for i in range(0,n):
        if i not in marque:
            print('appel principal avec i = %s'%(i))
            parcoursPR(M, n, i, f, marque)
    return None
```

N'oublions pas que, comme *marque*, initialement définie dans **parcoursProfondeur**, est une liste, elle **sera modifiée en place** (il n'en est pas fait de copie).

Un appel principal de `parcoursPR` avec $i = 0$ pour le graphe qui figure à droite (c'est la composante connexe de 0 dans le graphe de la page 366). Les nombres seuls sur la ligne correspondent au traitement postfixe $f(i)$ avec `f = lambda i: print('Traitement final du sommet %s'%(i))`. Il ne vous reste plus qu'à suivre le parcours avec un crayon sans pour autant abîmer ce beau livre.

```
>>> parcoursPR(m0, 16, 0, f, [])
Appel récursif avec j = 3
Appel récursif avec j = 13
Appel récursif avec j = 7
Appel récursif avec j = 8
Traitement final du sommet 8
Appel récursif avec j = 12
Traitement final du sommet 12
Traitement final du sommet 7
Appel récursif avec j = 15
Traitement final du sommet 15
Traitement final du sommet 13
Traitement final du sommet 3
Traitement final du sommet 0
[0, 3, 13, 7, 8, 12, 15]
```



Spécification

parcoursPR($M, n, i, f, marque$) prend en arguments la matrice d'adjacence d'un graphe (S, A) , un entier n ($n = |S|$), un sommet du graphe $i \in S$, une fonction f définie sur S , et un ensemble $marque$.

Elle retourne la réunion de $marque$ et de l'ensemble des sommets j figurant en paramètres dans les appels récursifs.

parcoursProfondeur(M, f) prend en arguments une matrice d'adjacence et une fonction f définie sur S . L'appel assure que chaque sommet du graphe sera traité (par f) une fois et une seule.

Correction de l'algorithme

• Intéressons nous tout d'abord à `parcoursPR` et procédons par étapes. Numérotions la suite des appels de `parcoursPR` et notons i_k, m_k les valeurs effectives des paramètres i et $marque$ dans le $k^{\text{ième}}$ appel.

1. La suite $(m_k)_k$ est strictement croissante ($m_k \subsetneq m_{k+1}$)

Preuve : l'appel $k + 1$, s'il existe se fera avec $m_{k+1} = m_k \cup i_k$ que ce soit un

appel récursif dans `parcoursPR` elle-même ou un appel de même niveau (donc situé dans une même boucle `for` de `parcoursProfondeur` ou de `parcoursPR`).

2. *Après un appel effectif à `parcoursPR` avec comme paramètres effectifs $i = i_k$ et $\text{marque} = m_k$, marque contient la réunion de $m_k \cup \{i_k\}$ et de l'ensemble des sommets s pour lesquels un appel récursif avec $i = s$ a été provoqué.*

Preuve :

Il est clair que marque n'est modifiée qu'au début de **`parcoursPR`**. Comme cette procédure ne fait pas d'autre appels éventuels que des appels récursifs à elle-même, marque se voit adjoindre les sommets $i_k, \dots, i_{k+p}$ correspondant à ces appels récursifs et eux seuls pendant la durée de vie du $k^{\text{ième}}$ appel.

3. *Un appel `parcoursPR(M, n, i*, f, marque)` provoque au plus un appel récursif avec $i = s$ comme paramètre effectif si $s \neq i^*$.*

Preuve :

La suite des (m_k) est strictement croissante ; si le premier appel avec $i = s$ porte le n° k , $s \in m_{k+p}$ pour tout $p \geq 1$. Aucun autre appel ne peut se faire avec s (en place du paramètre formel i) car la condition `j not in marque` n'est plus jamais remplie pour $j = s$ que ce soit dans `parcoursPR` ou dans `parcoursProfondeur`.

4. *Soit `parcoursPR(M, n, i, [])`, un appel effectif à `parcoursPR`, avec marque égal à la liste vide. Alors, pour tout sommet relié par un chemin à i , cet appel provoque un traitement « $f(j)$ » et un seul. Par ailleurs, à l'issue de l'appel, marque est une liste des sommets connectés à i (la composante connexe de i).*

Preuve :

L'unicité est établie avec le lemme précédent.

Nous allons montrer par récurrence la propriété

$$\mathcal{P}(\ell) = \{ \text{Pour tout sommet } s \text{ relié à } i \text{ par un chemin de longueur } \ell, \\ \text{parcoursPR}(M, n, i, []) \text{ provoque un appel } f(s). \}$$

- Initialisation : si $\ell = 1$, s est adjacent à i_0 . Une itération dans la boucle `for` fera $j = s$. A ce moment là, si s n'est pas marqué il y aura un appel avec $j = s$ comme paramètre. A la fin de la boucle ce voisin est dans marque ce qui implique qu'il y a eu un appel avec s comme paramètre qui s'est terminé avec $f(s)$.

- Supposons que le résultat soit établi pour des sommets à distance ℓ de i et considérons alors un sommet s relié à i par un chemin que l'on notera

$$i = i_0 \rightarrow i_1 \rightarrow \dots \rightarrow i_\ell \rightarrow i_{\ell+1} = s.$$

L'hypothèse de récurrence nous assure qu'il y aura un appel avec $i = i_\ell$. Comme s est adjacent à i_ℓ , une itération dans la boucle `for` de cet appel (avec $i = i_\ell$) se fera $j = s$.

- Un appel à la procédure **parcoursProfondeur** assure le traitement unique par f de chaque sommet.

Preuve

On commence par observer que la démonstration du point 4 est inchangée si on remplace $\square$ dans l'appel par une liste ne contenant aucun sommet connecté à i .

La boucle for dans **parcoursProfondeur** démarre avec $marque = \square$. La première itération affecte la composante connexe de $i = 0$ à $marque$. L'appel suivant se fera avec un sommet j n'appartenant pas à $marque$ et tous les sommets connectés à j seront traités par f une fois et une seule.

En poursuivant dans la boucle on traite tous les sommets avec un seul appel à **parcoursPR** par composante connexe.

Complexité de la procédure

- le nombre d'appels à **parcoursPR** et donc de marquages et d'appels à f est exactement le nombre $|S|$ de sommets du graphe.
- le nombre de tests dans **parcoursPR**($M, n, i, f, marque$) est égal à n et comme il y a un appel et un seul par sommet, le nombre de ces tests (appartenance à $marque$) est n^2 exactement.

Au final notre algorithme a une complexité en $\Theta(n^2)$.

Remarque : On fait mieux avec une représentation des graphes par des **listes d'adjacence** puisqu'alors l'algorithme de recherche en profondeur a une complexité en $\Theta(\max(n, p))$, p étant le degré maximal d'un sommet du graphe.

Composantes connexes

Exercice 9.5 composantes connexes, adaptation de **parcoursPR** et de **parcoursProfondeur**

On a observé (et démontré) qu'un appel **parcoursPR**($M, n, i, f, \square$) provoque un traitement de chaque élément de la composante connexe de i dans G . On souhaite modifier **parcoursProfondeur**() pour construire une liste des composantes connexes du graphe.

1. Construire une fonction **composantes_connexes**(M, n) qui retourne une liste des composantes connexes du graphe dont M est la matrice d'adjacence. Cette fonction renverra donc une liste de listes si nous représentons les composantes par des listes.

Elle pourra faire appel à une fonction récursive

composante($M, n, i, marque, comp$) qui parcourt le graphe pour construire **en place** la composante connexe de i dans la liste $comp$ et modifie en place la liste $marque$.

2. Tester avec le graphe non connexe de la page 366.

Corrigé en 9.5 page 395.

9.5 Parcours, versions itératives

On se propose de donner un algorithme de parcours en largeur d'un graphe sous forme itérative en utilisant une file d'attente. Ce sera l'occasion de reprendre le parcours en profondeur pour en donner une version itérative qui fera appel à une structure de pile.

9.5.1 Piles et files

Les piles et les files sont des structures dans lesquelles on peut exclusivement insérer ou retirer un élément, déterminer si une instance est vide ou pas.

On rappelle que dans une **structure de pile** c'est le dernier élément introduit qui est retiré (LIFO pour « last in, first out ») alors que dans une **file d'attente** (FIFO pour « first in, first out »), c'est le premier entré qui est retiré. Les listes de Python avec leurs méthodes pop, append et insert permettent de les simuler facilement :

Une pile (FIFO)	Une file (LIFO)
<pre>>>> P = [] >>> P.append('a') >>> P.append('b') >>> P ['a', 'b'] >>> P.pop() b >>> P ['a'] >>> len(P) > 0 True</pre>	<pre>>>> F = [] >>> F.insert(0, 'a') >>> F.insert(0, 'b') >>> F ['b', 'a'] >>> F.pop() a >>> F ['b'] >>> len(F) == 0 False</pre>

On résume

- pour simuler une pile : append, pop et len ;
- pour simuler une file : insert(0,...), pop et len.

9.5.2 Parcours en largeur, procédure itérative

L'idée d'un algorithme de parcours en largeur consiste, à partir d'un sommet de départ s, à visiter ses voisins et à les traiter au fur et à mesure avant de visiter leurs descendants. Il se prête à une programmation itérative utilisant une structure de file dans laquelle on place s et ses voisins non marqués pour explorer leur descendance après les avoir traités.

Nous n'en produirons pas de preuve détaillée et nous nous contenterons d'énoncer quelques résultats.

Une lecture attentive de ce qui suit (ou une exploration personnelle équivalente) est par contre indispensable pour comprendre son fonctionnement.

- un appel `parcoursL(M, n, i, f, marque)` avec un dictionnaire, `marque`, dont toutes les valeurs sont initialisées à `True`, assure un traitement unique par `f` de tous les sommets de la **composante connexe** de `i` dans `G`, et d'eux seuls ;
- la première itération dans la boucle `while` commence avec une file qui ne contient que le sommet `i` (argument effectif de la fonction) et qui est donc tout de suite vidée ; après la boucle `for` elle contient les voisins de `i` ;
- une seconde itération permet d'assurer le traitement du deuxième sommet rencontré (qui est un voisin de `i`) ; on empile ensuite les voisins de ce sommet qui n'ont pas déjà été rencontrés/marqués ;
- les itérations suivantes assurent donc traitement, désenfilage, enfilage des voisins rencontrés pour la première fois des sommets.

parcoursL

```
def parcoursL(M, n, i, f, marque):  
  
    marque[i] = True  
    print('\nRencontre de i = %s'%(i))  
    F = []  
    F.insert(0,i)  
  
    while len(F) > 0:  
        j = F.pop()  
        # le dernier entré sort de la file  
        f(j)  
        for k in range(0,n):  
            if M[j,k] != 0 and not marque[k]:  
                print('Rencontre de %s'%(k))  
                marque[k] = True  
                F.insert(0,k)  
  
    return None
```

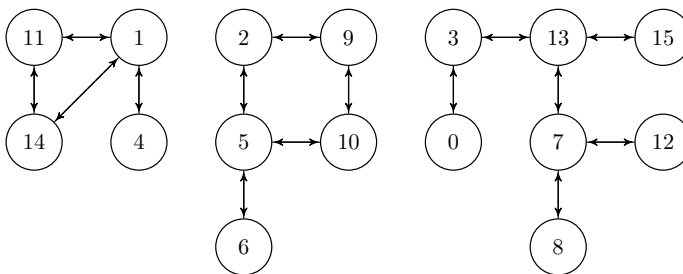
Parcours en largeur, procédure principale

```
def parcoursLargeur(M, f):
    '''
    M: matrice d'adjacence d'un graphe;
    n : integer tel que M soit n x n.
    '''
    n, p = M.shape
    if n != p:
        print('Erreur, on attend une matrice carrée!')
        return None
    marque = {}
    for i in range(0,n):
        marque[i] = False

    for i in range(0,n):
        if not marque[i]:
            parcoursL(M, n, i, f, marque)

    return None
```

- Illustration avec notre graphe fétiche et ses trois composantes connexes.



```
f = lambda i: print('\t- traitement du sommet %s'%(i))
parcoursLargeur(M, 16, f)
```

L'appel de `parcoursLargeur()` sur ce graphe permet de visualiser l'ordre du parcours, de distinguer les sommets qui sont un paramètre de `parcoursL` appelée par `parcours-`

Largeur() ou rencontrés lors d'une itération de la boucle while. On distingue trois appels de `parcoursL`, un par composante connexe.

```
Appel de parcoursL avec i = 0
  - traitement du sommet 0
- rencontre avec j = 3
  - traitement du sommet 3
- rencontre avec j = 13
  - traitement du sommet 13
- rencontre avec j = 7
- rencontre avec j = 15
  - traitement du sommet 7
- rencontre avec j = 8
- rencontre avec j = 12
  - traitement du sommet 15
  - traitement du sommet 8
  - traitement du sommet 12

Appel de parcoursL avec i = 1
  - traitement du sommet 1
- rencontre avec j = 4
- rencontre avec j = 11
- rencontre avec j = 14
  - traitement du sommet 4
  - traitement du sommet 11
  - traitement du sommet 14

Appel de parcoursL avec i = 2
  - traitement du sommet 2
- rencontre avec j = 5
- rencontre avec j = 9
  - traitement du sommet 5
- rencontre avec j = 6
- rencontre avec j = 10
  - traitement du sommet 9
  - traitement du sommet 6
  - traitement du sommet 10
```

9.5.3 Parcours en profondeur, version itérative

Nous reprenons ici le parcours en profondeur pour en proposer une version itérative. Comme nous l'avons vu, cet algorithme se prête bien à une écriture récursive ce

qui en facilite d'ailleurs la preuve. Mais il est souvent intéressant de disposer d'une version itérative d'un algorithme pour des raisons d'efficacité.

Vous construirez, dans l'exercice qui suit, un algorithme itératif de parcours en profondeur d'un graphe. Comme pour l'algorithme de parcours en largeur, donné en 9.5.2 page 375, nous adopterons d'une procédure **parcoursProfondeurIter**(M, f) appelant une procédure **parcoursPI**(M, n, s, f, marque) qui détermine le parcours (sous certaines conditions) d'une composante connexe.

Parcours en profondeur, procédure d'appel

```
def parcoursProfondeurIter(M, f):
    '''
    M: matrice d'adjacence d'un graphe;
    f : fonction opérant sur les entiers/sommets.
    '''
    n, p = M.shape
    if n != p:
        print('Erreur, on attend une matrice carrée!')
        return None

    marque = {}
    for i in range(0,n):
        marque[i] = False

    for i in range(0,n):
        if not marque[i]:
            parcoursPI(M, n, i, f, marque)

    return None
```

Exercice 9.6 *parcours en profondeur, version itérative*

1. Nous utiliserons une pile (LIFO) dans la procédure **parcoursPI**. Nous aurons besoin de l'initialiser, d'y ajouter des éléments, de dépiler. Présentez cela en utilisant une liste et ses méthodes **append**, **pop** exclusivement..

La méthode **pop()** renvoie le sommet et modifie la pile. Comment pourrait on récupérer la valeur du sommet de la pile en restituant à celle-ci son état ?

2. Dans le morceau de code qui suit, M est une matrice d'adjacence, marque est un tableau associatif comme dans les procédures de parcours déjà rencontrées (parcours profondeur récursif ou parcours en largeur itératif), i désigne un sommet du graphe.

```

while j < n :
    if not marque[j] and M[i,j] != 0:
        P.append(j)
        marque[j] = True
        print('- rencontre avec %s'%(j))
        i, j      = j, 0
    else:
        j += 1

```

- (a) Décrire le comportement de ce bloc lorsque $i = 0, j = 0, P = [x, y, z, \dots]$, les valeurs de `marque` sont toutes à `False` sauf **`marque[0] = True`** et `M` représente le graphe de la page 377. On pourra s'arrêter lorsque le sommet 8 sera empilé.
- (b) En quoi cela fait-il penser à un parcours en profondeur ?
3. Compléter la procédure ; pour chaque série ??? remplacée, justifier votre choix. Où allez vous placer le traitement des sommets par `f` (aucun ??? ne le signale) ?

```

def parcoursPI(M, n, s, f, marque):
    print('\nAppel avec %s'%(s))
    i      = ???
    marque[i] = ???
    ???
    ???

    j = ???
    while len(P) > 0:
        while j < n :
            if not marque[j] and M[i,j] != 0:
                P.append(j)
                marque[j] = True
                print('- rencontre avec %s'%(j))
                i, j      = j, 0
            else:
                j += 1
        j = ???
    if len(P) > 0:
        i = ???
        ???

```

9.6 Un algorithme de plus court chemin

On présente ici le problème de recherche du plus court chemin entre deux sommets d'un graphe orienté et pondéré par des poids positifs. Ce problème dont une solution proche de celle que nous présentons ici est due à E. Dijkstra (1959) a de nombreuses applications : recherche du chemin de plus courte longueur ou le moins coûteux entre deux lieux, calculs de routage dans un réseau de télécommunications etc. Après avoir précisé les définitions, nous commençons par une *description informelle* de cet algorithme, qui, à partir d'un sommet s , permet de calculer de proche en proche, pour chaque autre sommet x , un meilleur chemin de s à x .

Définition 9.6

Dans un graphe orienté et pondéré on considère un chemin de s à x (c'est-à-dire une suite finie d'arcs $\gamma = (i_0, i_1), (i_1, i_2), \dots, (i_{n-1}, i_n)$ telle que $s = i_0$ et $i_n = x$). On appelle coût cumulé (ou encore longueur) de ce chemin, la somme des coûts des arcs

$$\text{qui le composent : } \text{coût}(\gamma) = \sum_{k=0}^{n-1} w_{i_k, i_{k+1}}.$$

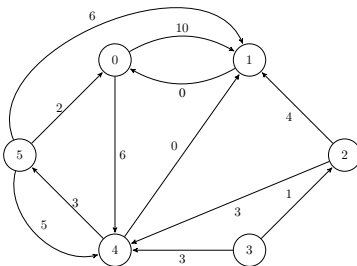
Un plus court chemin de s à x est un chemin de coût (ou de longueur) minimum.

Quelques observations et notations

- Dans ce qui suit les sommets d'un graphe seront numérotés de 0 à $n-1$ et identifiés à leurs numéros.

- Un graphe (S, A) , orienté et pondéré par des poids positifs, sera représenté par une matrice d'adjacence dans laquelle un terme $m_{i,j}$ est maximal si et seulement si il n'y a pas d'arc de i à j . Pour les besoins de la recherche de plus courts chemins, nous prendrons pour ce maximum une valeur strictement supérieure à $n \times \max w_{i,j}$, ce qui garantit que tous les chemins ne comportant pas de cycle (ne passant pas deux fois par un même sommet) ont une longueur inférieure à ce nombre.

Ainsi, dans le graphe qui suit, le poids maximum est égal à 10 et, dans sa matrice d'adjacence, le maximum $m_{0,0} = 66$ (notre infini à nous) vérifie bien $m_{0,0} > n \times \max_{(i,j) \in A} w_{i,j}$.



66	10	66	66	6	66
0	66	66	66	66	66
66	4	66	66	3	66
66	66	1	66	3	66
66	0	66	66	66	3
2	6	66	66	5	66

Nous noterons tout simplement *infini* cette valeur dans nos programmes.

• Observons qu'on peut représenter plusieurs chemins *partant de* $s \in S$ *et qui n'ont pas d'étape (ou de sommet) en commun*, par un seul tableau indexé de 0 à $|S| - 1$ tel que

$$\begin{cases} t_{pred}[i] &= p \text{ si } (p, i) \text{ appartient à un de ces chemins} \\ t_{pred}[s] &= s. \end{cases}$$

Exercice 9.7 *préliminaire conseillé avant de poursuivre la lecture*

1. Le tableau $t_{pred} = [1, 2, 2, 3, 2, 4]$ correspond-il à cette définition ? Quels chemins représente-t-il ?
2. Est-il compatible avec le graphe de la figure précédente ?
3. Écrire une fonction `remonterChemin( $t_p, j$ )` qui prend un argument un tableau formé de cette façon, un sommet $j \in \llbracket 0, n - 1 \rrbracket$ et retourne une liste représentant le chemin arrivant à j qui est codé dans t_p .

Corrigé en 9.7 page 399.

Algorithme de Dijkstra, description informelle

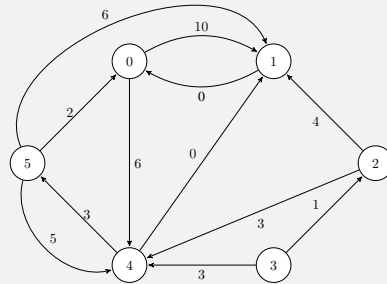
L'algorithme de Dijkstra repose sur les deux observations suivantes :

- si $i_0 \rightsquigarrow i_1 \rightsquigarrow \dots \rightsquigarrow i_k$ est un plus court chemin de $P = i_0$ à $Q = i_k$, alors le chemin extrait $i_0 \rightsquigarrow i_1 \rightsquigarrow \dots \rightsquigarrow i_j$ est un plus court chemin de P à i_j ($1 \leq j \leq k$);
- on peut représenter *des* plus courts chemins de P aux autres sommets du graphe $(S, \mathcal{A})$ par un tableau dans lequel $t_p[i]$ est l'indice du prédécesseur de i dans le plus court chemin (voir ci-dessus).

Algorithme de Dijkstra

- L'algorithme prend en arguments un graphe orienté et valué (représenté par une matrice d'adjacence), un sommet s .

Il retourne pour chaque sommet i du graphe, un plus court chemin de s à i .



- Ces chemins seront représentés par deux tableaux t_p, t_d dans lesquels $t_p[i]$ est le prédécesseur de i dans un plus court chemin de s à i , $t_d[i]$ est la longueur dans ce chemin. On notera $\eta^+(x)$ les sommets extrémités d'un arc d'origine x .
- Tout au long du processus, A et C sont deux ensembles tels que $A \cup C = S$ et $A \cap C = \emptyset$ alors que a est le sommet de C que l'on s'apprête à transférer vers A .

- On initialise avec $A = \emptyset, C = S$. On marque dans tp les seuls chemins directs de s à x avec une distance infinie, sauf pour s lui-même qui est à distance 0.
- Tant que la liste C n'est pas vide,
 - on trie C dans le sens croissant des longueurs $td[c]$.
 - On choisit pour a le premier sommet de C ainsi triée.
 - Pour tout sommet $c \in B = C \cap \eta^+(a)$, si la distance $s \rightsquigarrow a \rightsquigarrow c$ est inférieure à la distance déjà marquée (dans $td[c]$), on remplace cette dernière par $td[a] + w_{a,c}$ et on marque a comme prédécesseur de c .
 - a est enlevé à C , ajouté à A .

Remarque : les notations A, B, C sont celles de l'article original de Dijkstra (lien sur le site dédié à ce cours).

• Illustrations :

- Exécuter cet algorithme en partant de $s = 0$ sur le graphe représenté dans l'encart.

départ	arrivée	longueur	étapes
0	0	0	[0]
0	1	6	[0, 4, 1]
0	2	∞	
0	3	∞	
0	4	6	[0, 4]
0	5	9	[0, 4, 5]

- Exécuter cet algorithme en partant de $s = 3$.

départ	arrivée	longueur	étapes
3	0	3	[3, 4, 1, 0]
3	1	3	[3, 4, 1]
3	2	1	[3, 2]
3	3	0	[3]
3	4	3	[3, 4]
3	5	6	[3, 4, 5]

- Existe-t-il un chemin de 4 à 2 ?

départ	arrivée	longueur	étapes
4	0	0	[4, 1, 0]
4	1	0	[4, 1]
4	2	∞	
4	3	∞	
4	4	0	[4]
4	5	3	[4, 5]

- On remplace dans ce même graphe le poids de l'arc $(5, 4)$ par -5. Y a-t-il un chemin de plus courte longueur de 3 à 4 par exemple ?
- Modifier le poids de l'arc $(3, 2)$ de telle sorte que la plus courte longueur de 3 à 0 soit réalisée par plusieurs chemins. Quelle est alors la distance de 3 à 0 ?

• Nous pouvons maintenant implémenter notre algorithme.

Algorithme de Dijkstra sous Python

```
def Dijkstra(M, s):
    '''
    M matrice d'adj. d'un graphe positivement valué
    s sommet à partir duquel on calcule les plus
    courts chemins.
    '''
    N, infini = len(M[:,0]), M[0,0]

    #--- réservation des ensembles et tableaux
    A, C = [], list(range(0,N))
    td = np.ones(N, dtype = type(infini)) * infini
    td[s] = 0
    tp = np.ones(N, dtype=int) * s

    while C != []:
        C.sort(key = lambda i: td[i])
        a = C[0]
        for c in C:
            if td[a] + M[a,c] < td[c]:
                td[c] = td[a] + M[a,c]
                tp[c] = a

        A.append(a)
        C.remove(a)

    return s, tp, td, infini
```

Correction

L'algorithme termine en $N = |S|$ itérations puisqu'à chaque étape l'ensemble C perd un élément exactement au profit de A ; il est clair qu'à tout instant $A \cup C = S$ et $A \cap C = \emptyset$.

On note A_k, C_k, a_k les contenus des variables A, C, a à la $k^{\text{ième}}$ évaluation de la condition et on pose $B_k = \eta^+(a_k) \setminus A_k$.

Prouvons par récurrence sur k , qu'à l'itération k , pour tout $x \in S$, $td[x]$ contient soit la plus petite longueur des chemins de s à x passant par A_k , soit ∞ , si un tel chemin

n'existe pas.

- Lorsque $k = 1$, $A_1 = \emptyset$, $C_1 = S$, $a_1 = s$ et pour chaque élément de B_1 la première itération inscrit la plus courte distance de $s = a_1$ à c en passant par $A_2 = \{s\}$ (ou laisse $td[x] = \infty$).
- On suppose le résultat établi pour un certain k . Au cours de l'itération $k + 1$, $td[x]$ ne sera modifiée que si le chemin $s \curvearrowright a_{k+1} \curvearrowright x$ est plus court que le chemin déjà inscrit. On aura bien à la fin de l'itération dans $td[x]$ la plus courte longueur de s à x passant par $A_{k+1} = A_k \cup \{a_k\}$.

Lorsque l'algorithme termine, on a $A_{k+1} = S$ avec $C_{k+1} = \emptyset$, ce qui établit la correction de l'algorithme de Dijkstra.

9.7 Graphes, amas et percolation

Dans cette section nous partons d'un problème expérimental : on organise dans une boîte ou entre deux plaques de plexiglas des billes de même taille dont une proportion p seulement est conductrice. La répartition des billes conductrices est laissée au hasard. Existera-t-il une chaîne conductrice entre le fond et le dessus de la boîte ou de la plaque ?

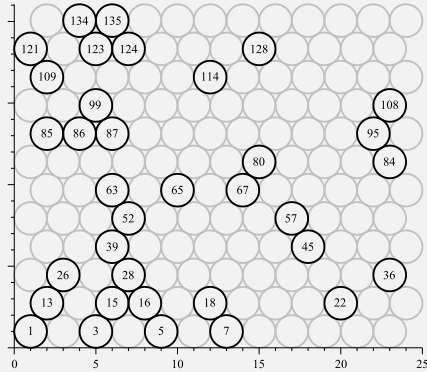
Nous nous proposons d'aborder ce problème simple par **simulation**. Nous associerons assez naturellement à chaque configuration effective un graphe dont le calcul des composantes connexes permettra de répondre à la question.

Ce problème simple fait apparaître un phénomène de **percolation** ou de changement brusque d'état du système. Nous nous en saisirons au passage pour illustrer, au chapitre 10, les avantages de la **programmation objet** dans ce type de programme où interviennent des objets de natures diverses : les graphes et leurs algorithmes, des billes et des configurations qu'il faut modéliser.

Le concept de percolation, dont une définition mathématique a été proposée par Braodbent et Hammersley en 1956, a émergé de l'étude statistique de systèmes constitués d'un grand nombre d'objets reliés entre eux. Dans l'article [9], paru dans *La Recherche*, Pierre Gilles de Gennes donne de nombreuses situations expérimentales (pannes dans les réseaux de télécommunication, taille des amas dans les alliages cristallins binaires, physique des semi-conducteurs, propagation des épidémies...) qui mettent en évidence le passage rapide d'un état à un autre (on parle de transition de percolation). L'article du mathématicien Harry Kesten [12] donne un résumé du point de vue mathématique. Pour préciser notre propos considérons une boîte contenant des billes de même taille, les unes en métal conducteur, les autres en plastique, donc non conductrices.

Mise en évidence d'un seuil de percolation

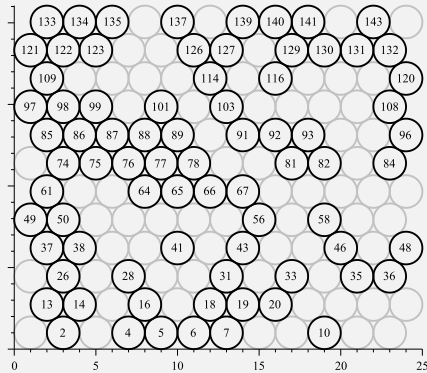
- On choisit $p \in]0, 1[$ et $n \in \mathbb{N}$.
- On remplit m fois de suite une boîte avec $N = n^d$ billes que l'on laisse tomber une à une en choisissant avec une probabilité p une boule conductrice.
- On relève la proportion $q_m(n, p)$ d'expériences dans laquelle il y a **percolation**, c'est-à-dire qu'il y a un **amas** de billes conductrices reliant le haut et le bas de la boîte.



Des amas sans percolation

Pour n fixé on fait varier p et on relève les différentes valeurs moyennes $q_m(n, p)$. On constate alors, comme dans tous les phénomènes cités par Pierre Gille de Gennes, qu'en deçà d'une certaine valeur critique p_c , la probabilité qu'il y ait percolation est nulle, qu'au delà elle se rapproche très rapidement de 1.

Bien évidemment la transition est d'autant plus rapide que n est grand. On peut démontrer que lorsque n tend vers l'infini, la suite de fonctions $p \rightarrow q(n, p)$ converge simplement vers une fonction échelon.



Un amas réalise la percolation

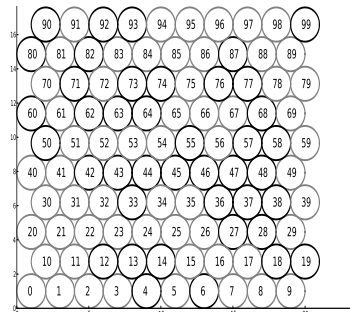
Exercice 9.8 mise en évidence d'un seuil de percolation par simulation

On construit ici les outils logiciels pour simuler l'expérience décrite dans l'encart qui précède. On travaille, comme toujours en deux temps : papier-crayon pour décrire sommairement les structures de données et préciser les algorithmes, puis programmation effective.

On « placera » les boules en quinconce comme sur les figures et sur une seule épaisseur (par exemple coincées entre deux panneaux de plexiglas). Nous avons donc ici $d = 2$ et $N = n^2$ boules.

1. Écrire une fonction $F(p)$ qui retourne True avec la probabilité p et False avec la probabilité $(1 - p)$.

Écrire une fonction $\text{configuration}(n, p)$ qui simule un tirage du tableau de n^2 boules, chaque boule ayant une probabilité p d'être conductrice. Cette procédure retourne une liste dont les termes représentent des boules par un objet $[i, x, y, r, c]$ où i est un entier (le numéro), x, y, r sont des flottants (coordonnées du centre, rayon), c est un booléen indiquant si la boule est conductrice ou pas.



Les boules seront numérotées comme sur la figure de droite et configuration peut faire appel à F ou en reprendre l'idée ; au choix.

2. On suppose donc les boules placées dans une liste sous la forme $[i, x, y, r, c]$. Écrire une fonction qui, à partir de la liste construite par la fonction configuration, retourne un graphe (S, A) dont les sommets sont les numéros des boules conductrices, les arêtes les couples (i, j) tels que les boules i et j sont toutes deux conductrices et en contact.

S est donc un ensemble ou une liste de numéros, A une liste de couples. On ne demande pas encore à ce niveau de finaliser en optant pour une matrice d'adjacence ou pour une liste d'adjacence.

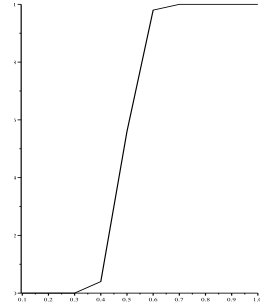
On pourra observer que, le graphe n'étant pas orienté, on peut se contenter de rechercher les arêtes orientées $\rightarrow$, $\nearrow$ et $\nwarrow$ pour éviter des redondances. Dans cette optique, écrire une fonction $\text{voisins_hd}(n, k)$ qui retourne un ensemble formé d'au plus trois numéros de boules voisines de la boule n° k (voisines de droite, de la ligne du haut à gauche, de la ligne du haut à droite). Évidemment, votre fonction tiendra compte de la parité du numéro de ligne de la boule k , du fait que c'est la dernière ligne, que k se situe à l'intérieur ou en bord de ligne etc.

3. On appelle **amas** une **composante connexe** de ce graphe. On suppose que l'on dispose d'une fonction (ou d'une méthode) qui pour un graphe donné dont pour l'instant, la représentation (matrice d'adjacence ou liste d'adjacence) n'importe pas, retourne une liste de composantes connexes (chaque composante est un ensemble). À l'aide des outils déjà construits, écrire une fonction, qui pour une

configuration donnée, retourne True ou False selon qu'il y a percolation ou pas. On ne reprogrammera pas d'algorithme de tri qui sont dans tous les langages.

4. Proposer un programme qui réalise m tirages répétés pour différentes valeurs de p entre 0 et 1 et calcule le nombre moyen de succès pour ensuite déterminer un encadrement du seuil de percolation.

La figure de droite montre ce que pourrait donner un tel programme : fréquences des succès sur m tirages pour différentes valeurs de p . On voit déjà apparaître une transition qui serait, espère-t-on plus marquée avec de plus grandes valeurs de N .



5. Nombres de sommets et d'arêtes du graphe.

- Quel est le nombre moyen de sommets dans le graphe ?
- Combien y-a-t-il de contacts (conducteurs ou pas) entre les boules ? Aidez vous du dessin. En déduire une majoration du nombre d'arêtes du graphe.

★ On se propose d'être plus précis :

- On considère une boule i non située sur un bord ou un coin. Donner la loi du nombre de contacts électriques conditionnée à l'événement $C_i = (i \text{ est conductrice})$. En déduire la loi non conditionnelle de N_i , la variable aléatoire égale au nombre de contacts conducteurs directs entre i et ses voisines ainsi que son espérance.
- Donner pour les boules situées sur un bord un **encadrement** de l'espérance de N_i .
- En déduire un **équivalent** de l'espérance de la variable aléatoire A égale au nombre d'arêtes du graphe en fonction de n et de p .

Corrigé en 9.8 page 399.

Le chapitre 10 donne une implémentation objet de la structure de graphe et propose un script complet pour la simulation du phénomène de percolation en dimension 2, telle qu'elle est suggérée dans ce problème.

9.8 Corrigés des exercices

Corrigé de l'exercice n° 9.1 .

• On suppose que G est un graphe non orienté. La relation $R(a,b) = \ll a = b \text{ ou il existe une chaîne reliant } a \text{ à } b \gg$

- est **réflexive** puisque de la forme $a = b$ ou $P(a,b)$;
- est **symétrique**, car
 - si $R(a,b)$ est vérifiée c'est que $a = b$ et donc $R(b,a)$ est vraie,
 - s'il existe un chemin $(\{a, i_1\}, \{i_1, i_2\}, \dots, \{i_{p-1}, b\})$ reliant a à b , le p -uplet $(\{i_{p-1}, b\}, \dots, \{i_1, i_2\}, \{a, i_1\}) = (\{b, i_{p-1}\}, \dots, \{i_2, i_1\}, \{i_1, a\})$ est alors une chaîne reliant b à a dans G et $R(b,a)$ est vraie.
- Montrons qu'elle est **transitive**. On suppose pour cela que $R(a,b)$ et $R(b,c)$ sont vérifiées.

Dans le cas où il existe une chaîne reliant a à b et une chaîne reliant b à c , la concaténation des deux chaînes fournit une chaîne de a à c et $R(a,c)$ est vérifiée. Dans le cas où $R(a,b)$ est vérifiée avec $a = b$, $R(a,c) = R(b,c)$ est encore vérifiée. Lorsque $R(b,c)$ est vérifiée avec $b = c$, alors $R(a,c) = R(a,b)$ est encore vraie.

R est donc une relation d'équivalence.

• Montrons que les classes d'équivalences de R coïncident avec les ensembles de sommets des composantes connexes du graphe.

- Soit $C \subset S$ une classe d'équivalence pour R .
Si a et b sont dans C , $R(a,b)$ est vraie. Si de plus $a \neq b$, il existe une chaîne de G reliant a à b .
Notons $P(C)$ la propriété « pour tout $(a,b) \in C$ avec $a \neq b$, il existe une chaîne de G reliant a à b » et montrons par l'absurde que C est un ensemble maximal pour cette propriété.
Si ce n'était pas le cas il existerait $c \notin C$ relié par une chaîne à $a \in C$. Par définition de R , $R(a,c)$ serait vraie et on aurait $c \in C$. Contradiction.
Ainsi, toute classe d'équivalence pour R est un ensemble maximal pour la propriété P . Or une chaîne de G qui relie deux éléments distincts a et b de C est aussi une chaîne du sous-graphe construit sur C , à savoir (C, A_C) (avec les notations des définitions (9.1)). En effet, toutes les extrémités d'une arête de cette chaîne sont dans même classe C que a et b .

Bilan : pour toute classe d'équivalence C pour la relation R , le sous-graphe (C, A_C) de G est une composante connexe de G .

- Considérons maintenant une composante connexe de G que nous noterons $(S', A_{S'})$ et $a \in S'$.
Pour tout élément $b \in S'$, soit $a = b$, soit il existe une chaîne reliant a à b . Dans les deux cas $R(a,b)$ est vérifiée et b est dans la composante connexe $\bar{a}$ de a . Ainsi $S' \subset \bar{a}$.
Si $c \in \bar{a}$, il existe une chaîne qui relie a à c . Tout élément de S' est également relié à c qui est donc dans S' puisque S' est maximal pour P . On a donc

$\bar{a} \subset S'$.

Bilan : Pour toute composante connexe $(S', A_{S'})$ de G , S' est une classe d'équivalence pour R .

Corrigé de l'exercice n° 9.2

1. Les composantes fortement connexes sont $\{0, 1, 6, 5\}$, $\{2, 7, 10, 8, 9, 3, 4\}$.
2. (a) Soit $R(u, v) = \ll u = v$ ou il existe un chemin de u à v et un chemin de v à $u \gg$.
Cette relation est clairement réflexive, symétrique, transitive (en concaténant les chemins).
- (b) On montre, comme dans l'exercice 9.1, que toute classe d'équivalence pour R est une composante fortement connexe de G , et que, réciproquement, toute composante connexe est la classe d'équivalence d'un quelconque de ses sommets.
3. - VRAI : Si a et b sont dans une même composante fortement connexe de G , alors il existe un chemin de b vers a et un chemin de a vers b .

- VRAI : Si a et b sont dans une même composante fortement connexe alors il existe un circuit qui passe par a et b . Puisqu'il existe un chemin de a vers b et un chemin de b vers a .

- Il est FAUX que : Si a et b sont dans une même composante fortement connexe alors il existe un circuit élémentaire (ie : qui ne passe pas deux fois par le même sommet) qui passe par a et b .

Contre exemple avec le graphe $G = (S, A)$ où $S = \{a, b, c\}$ et $A = \{(a, b), (b, c), (c, b), (b, a)\}$. Le seul circuit passant par a et c passe deux fois par b .

- VRAI : S'il existe un circuit qui passe par a et b alors, a et b sont dans une même composante fortement connexe.

En effet ce circuit sera de la forme :

$(i_0, i_1), \dots, (i_{p-1}, i_p = a), (i_p = a, i_{p+1}), \dots, (i_{q-1}, i_q = b), (i_q = b, i_{q+1}), \dots, (i_{n-1}, i_0 = c)$ et les deux chemins $(i_p = a, i_{p+1}), \dots, (i_{q-1}, i_q = b)$ et $(i_q = b, i_{q+1}), \dots, (i_{n-1}, i_0 = c), (c = i_0, i_1), \dots, (i_{p-1}, i_p = a)$ relient a à b et b à a . Ce qui place ces deux sommets dans une même composante ou classe d'équivalence.

Corrigé de l'exercice n° 9.3

On corrige la dernière question. La procédure de construction du dictionnaire implémentant une liste d'adjacence tient compte d'erreurs éventuelles. Nous en avons intentionnellement glissé dans la liste des sommets.

```
def graphe_dictionnaire(S, A):
    ''' S: list (liste des sommets d'un graphe);
        A: list (liste des ares;  conteneurs de lg. 2);
        Renvoie un dictionnaire {sommets : {},...}
    '''
    D = {}
    for s in S:
        D[s] = {'d+': 0, 'd-' : 0, 'S' : []}

    V = []
    #permet de vérifier la présence de doublons dans A
    for a in A:
        try:
            o, e = tuple(a)
            if a in V:
                print('doublon dans la liste A avec a=%s'%(a))
                continue # itération suivante dans la boucle
            else:
                V.append(a)
        except ValueError:
            print('Erreur sur l\'arête %s, conteneur de
                  long. 2 attendu.'%(a))
            continue # itération suivante dans la boucle

    if o in D:
        #actualisé
        D[o]['S'].append(e)
        D[o]['d+'] += 1
    else:
        #initialisé
        print('noeud %s ajouté'%(o))
        D[o] = {'d+': 1, 'd-' : 0, 'S' : [e]}

    if e in D :
        #actualisé
        D[e]['d-'] += -1
    else:
        #initialisé
        print('noeud %s ajouté'%(e))
        D[e] = {'d+': 0, 'd-' : -1, 'S' : []}

    return D
```

```

S = [i for i in range(0,10)]
A = [[0,1], [1,2], [1,5], [1,6], [2,3], [2,7], \
      [3,4], [4,9], [5,0], [6,5], [6,7], [6,10], \
      [7,10], [8,2], [9,10], \
      [10,9], [10,8,5], [10,8], [10,8] ]

```

```

D = graphe_dictionnaire(S, A)
presentation_dict(D)

```

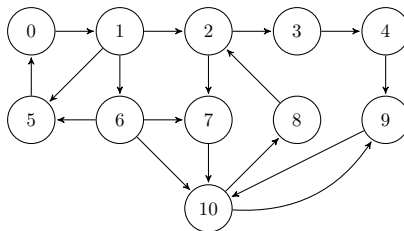
Noeud 10 ajouté.

Erreur sur l'arête [10, 8, 5], conteneur de long. 2 attendu.
 Doublet dans la liste A avec a=[10, 8]. Ignoré.

```

0 :
{
    d+ : 1
    d- : -1
    S  : [1]
}
1 :
{
    d+ : 3
    d- : -1
    S  : [2, 5, 6]
}
...
}
10 :
{
    d+ : 2
    d- : -3
    S  : [9, 8]
}

```



Corrigé de l'exercice n° 9.4

1. On suppose que S et A sont comme dans l'exemple, une liste des numéros des sommets de 0 à n et une liste des arêtes. Après l'appel de numpy pour les matrices, on définit la fonction qui construit la matrice d'adjacence :

```
import numpy as np

def matriceAdjacenceGNO(S,A):
    n      = len(S)
    m      = np.zeros((n,n), dtype = 'int')
    for a in A:
        [i,j] = a
        m[i,j] = m[j,i] = 1
    return m
```

La matrice $m1$ obtenue avec les instructions qui suivent est affichée page 369

```
S1 = [k for k in range(0,5)]
A1 = [[0,1],[1,4],[0,2],[4,2],[2,3]]
m1 = matriceAdjacenceNO(S1,A1)
```

2. On pourra choisir de représenter les arcs pondérés par des listes $[i, j, w_{i,j}]$. Pour remplir la matrice d'adjacence il nous faut déterminer un terme qui ne soit pas un poids et dont la présence signalera qu'il n'y a pas d'arc de i à j . Ce peut être une constante comme le plus grand entier du système, NAN ou encore $1 + \max_{i,j} |w_{i,j}| \dots$ Dans ce dernier cas, ce terme varie avec le graphe.

```
def matriceAdjacenceGO(S,A):
    n      = len(S)
    star   = max([abs(a[2]) for a in A])+1
    # le max de la liste des valeurs absolues
    # des poids, plus 1
    m      = np.ones((n,n), dtype = 'int') *star
    for a in A:
        [i,j,w] = a
        m[i,j] = w
    return star, m
```

Exemple : on obtient $\star = 42$ et la matrice

$$\begin{bmatrix} 42 & 17 & -41 & 42 & 42 \\ 42 & 42 & 42 & 42 & 23 \\ 42 & 42 & 42 & -7 & 37 \\ 42 & 42 & 42 & 42 & 42 \\ 42 & 42 & 42 & 42 & 42 \end{bmatrix} \text{ avec}$$

le script suivant :

```
S2 = [k for k in range(0,5)]
A2 = [[0,1,17],[1,4,23],[0,2,-41],[2,4,37],[2,3,-7]]
star, m2 = matriceAdjacenceGO(S2,A2)
```

Corrigé de l'exercice n° 9.5

1. On démarre avec deux listes, **marque** et **liste_composantes** qui contiendra les différentes composantes renvoyées par les appels principaux de composante. Il faut suivre la trace et vérifier sur un dessin si vous voulez comprendre !

```
def composantes_connexes(M, n):
    '''
    M: matrice d'adjacence d'un graphe;
    n : integer tel que M soit de taille n x n.
    Renvoie la liste des composantes connexes du
    graphe associé à M.
    '''
    marque = [] #sommets visités;
    liste_composantes = [] #liste des composantes;

    for i in range(0,n):
        if i not in marque:
            comp = composante(M, n, i, marque, [])
            liste_composantes.append(comp)
            # la composante a été parcourue
    return liste_composantes

def composante(M, n , i, marque, comp):
    '''
    comp : liste vide lors de l'appel principal
           (composante de i en construction);
    marque : liste des sommets déjà visités;
    Renvoie la composante de connexe de i.
    '''
    comp.append(i)
    marque.append(i)
    for j in range(0,n):
        if M[i,j] != 0 and j not in marque:
            composante(M, n, j, marque, comp)
    return comp
```

2. Ce que cela donne lorsque m0 est la matrice d'adjacence du graphe de la page 366.

```
>>>composantes_connexes(m0, 16, f)
[[0, 3, 13, 7, 8, 12, 15], [1, 4, 11, 14], [2, 5, 6, 10, 9]]
```

Corrigé de l'exercice n° 9.6

1. P = [] # initialiser
 P.append(i) # empiler

 i = P.pop() # dépiler
 P.append(i) # remettre en l'état

2. (a) Nous allons décrire pas à pas le comportement de cette boucle et donner l'état après l'itération.
 En début de boucle $i = 1, j = 0, P = [x, y, z, \dots]$. Nous listons
 - i. Comme $\text{marque}[0] = \text{True}$, on fera $j += 1$ d'où $i = 0, j = 1, P = [x, y, z, \dots]$.
 - ii. Comme $M[0,1] = 0$, on fera $j += 1$ d'où $i = 0, j = 2, P = [x, y, z, \dots]$.
 - iii. Comme $M[0,2] = 0$, on fera $j += 1$ d'où $i = 0, j = 3, P = [x, y, z, \dots]$.
 - iv. Comme $M[0,3] = 1$, on fera $P.append(3)$, $\text{marque}[3] = \text{True}$, $i, j = j, 0$ d'où $i = 3, j = 0, P = [3, x, y, z, \dots]$.
 - v. Comme $M[3,0] = 0$, on fera $j += 1$ d'où $i = 3, j = 1, P = [3, x, y, z, \dots]$.
 - vi. Comme $M[3,j] = 0$ pour $j=1,2,\dots,12$ on aura $i = 3, j = 13, P = [3, x, y, z, \dots]$.
 - vii. Comme $M[3,13] = 1$, on fera $P.append(13)$, $\text{marque}[13] = \text{True}$, $i, j = j, 0$ d'où $i = 13, j = 0, P = [13, 3, x, y, z, \dots]$.
 - viii. Comme $M[3, j] = 0$ ou $\text{marque}[j] = \text{True}$ pour $j = 0, \dots, 6$, on fera $j += 1$ d'où $i = 4, j = 7, P = [4, x, y, z, \dots]$.
 - ix. Comme $M[4,7] = 1$, on fera $P.append(7)$, $\text{marque}[7] = \text{True}$, $i, j = j, 0$ d'où $i = 7, j = 0, P = [7, 13, 3, x, y, z, \dots]$.
 - x. Comme $M[7,j] = 0$ ou $\text{marque}[j] = \text{True}$, pour $j = 1,2,\dots,7$, on fera $j += 1$ d'où $i = 7, j = 8, P = [7, 13, 3, x, y, z, \dots]$.
 - xi. Comme $M[7,8] = 1$, on fera $P.append(8)$, $\text{marque}[8] = \text{True}$, $i, j = j, 0$ d'où $i = 7, j = 0, P = [8, 7, 13, 3, x, y, z, \dots]$.
 - xii. Comme $M[8,j] = 0$ pour tout sommet non marqué la boucle terminera.
 - xiii. Ouf !

- (b) Le cheminement nous conduit, partant du sommet $s = 0$, à chercher le premier voisin qui remplacera s pour poursuivre. Sans circuit on s'éloigne de s . Il devient clair qu'après 8

3. La procédure complétée avec une trace lors d'un appel sur le graphe qui nous a servi pour illustrer nos algorithmes de parcours. On comparera à la trace laissée par l'algorithme de parcours en largeur 378.

```

def parcoursPI(M, n, s, f, marque):
    '''
    M : matrice d'adjacence d'un graphe;
    n : int, nombre de l. et col. de M;
    s : int; sommet;
    f : fonction opérant sur les entiers/sommets;
    marque : dictionnaire {sommet: booléen} .

    Réalise un parcours en profondeur de la composante
    connexe de i, dès lors que marque[s] = False
    pour les sommets de cette composante.
    Algorithme itératif.
    '''
    print('\nAppel avec %s'%(s))
    i = s # le sommet de départ
    marque[i] = True # on le marque
    P = []
    P.append(i) # initialisation de la pile

    j = 0 # premier sommet à tester
    while len(P) > 0:
        while j < n :
            if not marque[j] and M[i,j] != 0:
                P.append(j)
                marque[j] = True
                print('- rencontre avec %s'%(j))
                i, j = j, 0
            else:
                j += 1
        # c'est bien un parcours en profondeur
        j = P.pop()
        f(j)
        if len(P) > 0:
            i = P.pop()
            P.append(i)

```

```
f = lambda i : print('\t- traitement du sommet %s'%(i))
parcoursProfondeurIter(M, f)
```

Appel de parcoursPI avec 0

```
- rencontre avec 3
- rencontre avec 13
- rencontre avec 7
- rencontre avec 8
    - traitement du sommet 8
- rencontre avec 12
    - traitement du sommet 12
    - traitement du sommet 7
- rencontre avec 15
    - traitement du sommet 15
    - traitement du sommet 13
    - traitement du sommet 3
    - traitement du sommet 0
```

Appel de parcoursPI avec 1

```
- rencontre avec 4
    - traitement du sommet 4
- rencontre avec 11
- rencontre avec 14
    - traitement du sommet 14
    - traitement du sommet 11
    - traitement du sommet 1
```

Appel de parcoursPI avec 2

```
- rencontre avec 5
- rencontre avec 6
    - traitement du sommet 6
- rencontre avec 10
- rencontre avec 9
    - traitement du sommet 9
    - traitement du sommet 10
    - traitement du sommet 5
    - traitement du sommet 2
```

Corrigé de l'exercice n° 9.7

1. A la main pour comprendre, avec $t_{pred} = [1, 2, -1, -1, 2, 4]$, on réécrit

i	0	1	2	3	4	5
$t_{pred}[i]$	1	2	2	3	2	4

et 0 a pour prédécesseur 1 qui a pour prédécesseur 2 qui n'en a pas : $2 \rightarrow 1 \rightarrow 0$ est donc un chemin de 2 à 0. De la même façon $2 \rightarrow 4 \rightarrow 5$ est un chemin de 2 à 5. Les autres chemins en partant de 2 sont les sous-chemins de ces deux là.

2. Ce sont dans le graphe qui nous sert de cobaye des chemins de plus courtes longueurs en partant de 2.
3. tp est un tableau de prédécesseurs, que $tp[p] = p$ quand p n'a pas de prédécesseur, $last$ est l'extrémité du chemin à reconstituer :

```
def remonterChemin(tp, last):
    """tp ..., last ..."""
    """tp[s]= s quand s n'a pas de préd..."""
    p = last
    chemin = [p]
    while tp[p] != p:
        p = tp[p]
        chemin.insert(0,p)
    return chemin
```

Corrigé de l'exercice n° 9.8

*On corrige dans les grandes lignes, en précisant la méthode sans trop entrer dans les détails d'implémentation. Le script complet se trouve dans le chapitre 10 écrit en **syntaxe objet** ce qui ne change pas les algorithmes, rend la syntaxe un peu plus longue mais se révèle bien plus pratique à l'usage !*

1. La fonction `random()` du module `random` suit une loi uniforme sur $[0, 1]$ pour laquelle $P([0, p]) = p$. On obtient une loi de Bernoulli de paramètre p en faisant :

```
import random as rd

def F(p):
    if rd.random() < p:
        return True
    else:
        return False
```

Il s'agit, pour construire une configuration, de construire n^2 boules, en précisant leur numéro et leur centre. Nous avons là un petit problème de géométrie du collège :

- le n° de la boule de la ligne ℓ et de la colonne c est $c+n \times \ell$ (en commençant avec $\ell = 0, c = 0$);
- lorsque le n° de ligne est pair (de 0 à $[n/2]$) les centres ont pour coordonnées $x = r + c \times d, y = r + \sqrt{3}r \times \ell$ (d'une rangée à l'autre, le centre « monte » d'une hauteur de triangle équilatéral de coté r);
- lorsque le n° de ligne est impair (de 1 à $[n/2]$), $x = d + d * c, y = r + \sqrt{3}r \times \ell$;

Cela nous conduit à construire la liste des boules de la façon suivante (r affecté etc.). La première idée est de faire :

```
L = []
for ell in range(0, n, 2):
    for col in range(0,n):
        L.append([col+ell*n, r+d*col, r+ell*h, r, F(p)])

for ell in range(1, n, 2):
    for col in range(0,n):
        L.append([col+ell*n, d+d*col, r+ell*h, r, F(p)])
```

Si on veut que l'ordre des boules corresponde à leur numérotation, ce qui pourrait s'avérer plus pratique pour la suite, on écrira plutôt :

```
L = []
for ell in range(0, n):
    if ell%2 == 0:
        dx = r
    else:
        dx = d
    for col in range(0,n):
        L.append([col+ell*n, dx+d*col, r+ell*h, r, F(p)])
```

2. Construction de (S, G) à partir de L :

L'idée est de procéder en deux temps : on parcourt la liste L de toutes les boules, on retient les boules pour lesquelles $cond == True$. On peut le réaliser en compréhension ou avec une boucle for.

Une deuxième boucle for parcourant la liste des sommets (boules conductrices) recherche parmi les (au plus) trois voisines à droite ou au dessus, possibles,

les boules également conductrices et ajoute un couple $[b, v]$ à la liste des arêtes. On aura au préalable défini la fonction $\text{voisines_hd}(n, k)$: On écrit ici l'algorithme de construction des sommets et arrêtes (selon la représentation $b.\text{numero}$ pourra être $b[0]$, $b.\text{cond}$, $b[4]$, etc.

```
def voisines(n, k):
    ell, c = divmod(k, self)
    if ell == n-1:
        if c == n-1:
            return []
        else:
            return [k+1]
    elif divmod(ell, 2)[1] == 0:
        if c == 0:
            return [k+1, k+n]
        elif c == n-1:
            return [k+n, k+n-1]
        else:
            return [k+1, k+n, k+n-1]
    else:
        if c == n-1:
            return [k+n]
        else:
            return [k+1, k+n, k+n+1]
```

```
def graphe(L):
    sommets = []
    for b in L:
        if b[4]:
            sommets.append(L[nb][0])
    arretes = []
    for nb in sommets:
        for nv in voisines(n, L[nb][0]):
            if L[nv][4]:
                arretes.append([nb, nv])
    return sommets, arretes
```

En notation objet, lorsque les boules sont implémentées comme des classes, $b[0]$ et $L[nb][0]$ deviennent $b.\text{numero}$ et $L[nb].\text{numero}$, $b[4]$ et $L[nb][4]$ deviennent $b.\text{cond}$ et $L[nb].\text{cond}$ etc.

3. Nous proposons l'idée suivante : pour chaque amas ou composante (obtenue comme composante connexe du graphe (S, A)) trier sur la clé y et regarder si

les première et dernière boules ont des ordonnées identiques à celles des boules $L[0]$ et $L[n^2]$ par exemple. Si cela est vérifié pour un amas au moins, il y a une jonction entre le haut et la bas de la boîte..

A savoir : la fonction **sorted** permet de trier un objet (liste, chaîne etc.) avec un argument optionnel : la clé pour le tri. Dans notre implémentation objet, une boule est un objet avec comme attributs `b.numero`, `b.x`, `b.y`, `b.rayon`, `b.cond(uctrice)` et nous avons simplement trié les amas avec

```
sorted(C.listeBoules, key =lambda b: b.y)
```

Avec une représentation par liste $b = [nb, x, y, r, c]$ on aurait pu faire :

```
sorted(L, key =lambda b: b[2])
```

4. Une boucle sur une subdivision P de $[0, 1]$ pour les différentes valeurs de p . Une boucle emboîtée dans la précédente pour réaliser m tirages aléatoires de configurations avec p comme paramètre. On stocke la fréquence des succès dans F de même taille que P . On trace ensuite `pl.plot(P, F)` qui dépend de n et de m . Le paramètre décisif est bien sûr n (à condition de prendre m suffisamment grand - éviter $m = 1, 2, 3!$).
5. Probabilités et dénombrements :

- (a) Nombre moyen de sommets du graphe : le nombre de sommets du graphe est l'espérance de la loi du nombre de billes conductrices. Cette variable suit une loi binomiale de paramètres n^2 et p . On a donc $E(|S|) = n^2 p$ et $Var(|S|) = n^2 p(1 - p)$. Nous supposons évidemment ici que les fonctions du module `random` nous donnent des variables aléatoires parfaites !
- (b) Nombre de contacts : pour chaque ligne, il y a n boules et $n - 1$ contacts horizontaux. Pour chaque ligne, sauf la dernière, il y a $2(n - 1)$ contacts bas-haut (droite ou gauche). Ce qui, au total, donne :

$$|A| \leq n(n - 1) + 2(n - 1)^2 = (n - 1)(3n - 1) \underset{n \rightarrow +\infty}{\sim} 3n^2.$$

- (c) Notons N_i le nombre de contacts conducteurs ayant pour origine la boule i située à l'intérieur de la configuration. Lorsque C_i est réalisé, N_i est égal au nombre de voisines de la boule $n^\circ i$ qui sont aussi conductrices. Il y a 6 voisines et les événements C_j étant indépendants, la loi conditionnelle est une loi binomiale :

$$P(N_i = k | C_i) = \binom{6}{k} p^k (1 - p)^{(6-k)}.$$

En distinguant selon que $k = 0$ ou $k = 1, 2, 3, 4, 5, 6$:

$$P(N_i = 0) = P(N_i = 0 | \bar{C}_i)P(\bar{C}_i) + P(N_i = 0 | C_i)P(C_i) \quad (9.8.1)$$

$$= (1 - p) + p(1 - p)^6 \quad (9.8.2)$$

$$P(N_i = k) = P(N_i = k | \bar{C}_i)P(\bar{C}_i) + P(N_i = k | C_i)P(C_i) \quad (9.8.3)$$

$$= 0 + p \binom{6}{k} p^k (1 - p)^{(6-k)} \quad (9.8.4)$$

$$E(N_i) = 0 + p \sum_{k=1}^6 k P(N_i = k | C_i) = 6p^2 \quad (9.8.5)$$

(d) Pour une boule située au bord, $2 \leq N_i \leq 5$ et $E(N_i) \leq 5$.

(e) On observe que la somme des N_i est le nombre des arcs orientés du graphe. On a donc $|A| = \frac{1}{2} \sum_{j=1}^{n^2} N_j$ et :

$$E(|A|) = \frac{1}{2} \left(\sum_{j \in \text{interieur}} E(N_j) + \sum_{j \in \text{bord}} E(N_j) \right) \sim 3p^2 n^2. \quad (9.8.6)$$

En effet, $\sum_{j \in \text{bord}} E(N_j) \leq \frac{5}{2}(4n - 2) = o\left(\sum_{j \in \text{interieur}} E(N_j)\right)$.

Un bref aperçu de la programmation objet

L'objectif est ici de montrer l'intérêt de la programmation objet et de l'illustrer avec le problème 9.8 du chapitre 9 qui s'y prête particulièrement bien. Quoique cette notion ne figure pas au programme de l'informatique de tronc commun, une clarification des concepts qui la sous-tendent vous permettra de mieux comprendre et gérer les subtilités de la syntaxe de Python dans lequel *tout est objet*. Cela constituera une ouverture vers un **paradigme de programmation** que vous ne pourrez pas ignorer à terme. Et si par ailleurs votre TIPE vous conduit à modéliser des situations complexes, ce chapitre pourrait vous aider à structurer vos programmes en construisant vos propres classes.

10.1 Les concepts de la programmation objet

Les structures fondamentales de l'informatique, chaînes de caractères, listes, tableaux, dictionnaires, piles ou files, graphes, sont intrinsèquement liées aux algorithmes qui permettent de les manipuler. La **programmation objet** consiste à définir des objets informatiques de telle sorte que les éléments constitutifs d'une structure et les fonctions associées soient regroupés dans une même entité.

La définition d'une telle entité est une **classe**, les éléments constitutifs de sa structure sont les **attributs** de la classe (ce seront les données de ses instances), ses fonctions ou procédures sont ses **méthodes** ; les **instances** de ces classes sont les **objets** eux-mêmes. Un langage orienté objet est un langage qui permet la définition de classes comme par exemple C++, OCaml, Python, Java et bien d'autres.

Syntaxe objet de Python vue par l'utilisateur

Dans le tableau qui suit les variables `ch` et `L` contiennent respectivement des objets de type `'str'` (string) ou `'list'`, c'est à dire des **instances des classes `'str'` ou `'list'`**.

Dans les deux cas, la fonction `len` : $C \rightarrow \text{Integers}$ (où C contient la plupart des classes de conteneurs) s'applique à ces objets. C'est la syntaxe usuelle en program-

mation lorsqu'on appelle une fonction, proche de la syntaxe mathématique.

La syntaxe est différente lorsque nous appelons les **méthodes** count, upper, replace de la classe 'str' ou sort, append de la classe 'list' : elle est alors de la forme

`nom_instance.nom_méthode(arguments) .`

Cette notation a la même signification que nom_méthode(nom_instance, arguments) mais offre des avantages indéniables pour la sécurisation et l'organisation du code ou quand il s'agit de modéliser des situations informatiques (voir ci-dessous avec la classe graphe_non_oriente) ou d'un domaine d'application particulier (voir la modélisation objet de notre expérience se simulation).

chaînes de caractères	listes
<pre>>>> ch = 'Chaine de caractères' >>> type(ch) <class 'str'> >>> len(ch) 20 >>> ch.count('e') 3 >>> ch1 = ch.replace('i', 'î') >>> ch1 'Chaîne de caractères' >>> ch1.upper() 'CHAÎNE DE CARACTÈRES' >>> ch1.split() ['Chaîne', 'de', 'caractères']</pre>	<pre>>>> L = [1, 5, 3, 16, 7] >>> type(L) <class 'list'> >>> len(L) 5 >>> L.sort() >>> L [1, 3, 5, 7, 16] >>> print(L.sort()) None >>> L.append(45) >>> L [1, 3, 5, 7, 16, 45]</pre>

10.2 Une classe pour la structure de graphe

Nous avons choisi d'implémenter la structure de graphe non orienté dans une classe Python. La représentation par **liste d'adjacence** aura la forme d'un **tableau associatif** {sommet : liste des voisins }.

Cet attribut sera initialisé dans le **constructeur** de la classe et modifié par la suite lors des ajouts éventuels de sommets ou d'arêtes.

Avant de décrire cette classe plus avant, dressons un inventaire des éléments que nous voulons lui ajouter. Les *underscore* dans les définitions signifient que les attributs ou méthodes concernés ne seront pas appelés en dehors de la définition de la classe. Cette distinction entre méthodes ou attributs privés et publiques est une **convention**

en Python. Dans la plupart des autres langages orientés objet, la distinction entre attributs ou méthodes **privées et publiques** est impérative.

Attributs	
self.voisins	dictionnaire {sommet : liste des voisins}
self.__marque__	dictionnaire {sommet : booléens, ...} pour les parcours

Méthodes	
self.__init__(L,A)	constructeur
self.__initialise_voisins__(S, A)	initialise un dictionnaire {sommet : liste des voisins} envoyé au constructeur ;
self.ajoute_sommet(s)	ajoute le sommet dans self.voisins, renvoie None ;
self.ajoute_arete(a)	ajoute l'arete dans self.voisins, met à jour les sommets, renvoie None ;
self.__parcours__(s, fonction)	parcours en profondeur à partir du sommet s ;
self.parcours(fonction)	parcours en profondeur du graphe ; traitement avec fonction ; appelle self.__parcours__(s, fonction) ;
self.__composante__(s)	renvoie la composante connexe de s ;
self.composantes_connexes()	renvoie la liste des composantes connexes du graphe, appelle self.__composante__(s).

Le constructeur de la classe En Python le constructeur d'une classe est la méthode `__init__(...)` qui est implicitement appelée quand on crée un objet ou une instance de la classe. Le mot clé **self** permet de désigner les attributs ou méthodes de la classe au moment où on la définit. Il sera remplacé par une instance lors des appels effectifs.

Classe graphe_non_oriente, constructeur et syntaxe objet
<pre>class graphe_non_oriente: def __init__(self, S = [], A = []): ''' S : liste de sommets; A : liste d'arêtes (conteneurs de longueur 2); ''' self.voisins = self.__initialise_voisins__(S, A)</pre>

Usage : on crée une instance `g` de `graphe_non_oriente`, on appelle attributs et méthodes

```
>>> L = list(range(0,16))
>>> A = [{1,11}, {1,4}, {1,14}, {11,14}, {2,9},
          {9,10}, {10,5}, {5,2}, {5,6}, {3,13},
          {13,15}, {13,7}, {7,12}, {8,7}, {0,3}]
>>> g = graphe_non_oriente(L,A)
>>> g.voisins
{0: [3], 1: [11, 4, 14], 2: [9, 5], 3: [13, 0], 4: [1], ...,
 11: [1, 14], 13: [3, 15, 7], 14: [1, 11], 15: [13]}
>>> g.composantes_connexes()
[[15, 12, 8, 7, 13, 3, 0], [14, 11, 4, 1],[6, 5, 10, 9, 2]]
```

```
def ajoute_arete(self, a):
    '''
    Ajoute une nouvelle arête. Ajoute ses extrémités aux
    clés de self.voisins si elles n'y figurent pas déjà.
    Affiche un avertissement si a n'est pas un conteneur
    de longueur 2 ou si a est déjà représenté.
    '''
    try:
        e, f = tuple(a)
    except ValueError::
        print('Attention, ajoute_arete attend un
              conteneur de longueur 2 et a reçu %s.'%(a))
        return None
    if e not in self.voisins:
        self.voisins[e] = []
    if f not in self.voisins[e]:
        self.voisins[e].append(f)
    else:
        print('Warning: l\'arête %s figure déjà.'%(a))
        return None
    if f not in self.voisins :
        self.voisins[f] = []
    if e not in self.voisins[f]:
        self.voisins[f].append(e)
    else:
        print('Warning: l\'arête %s figure déjà.'%(a))
        return None
```

```
def ajoute_sommet(self,s):
    '''
    Ajoute un nouveau sommet, affiche un avertissement
    si s est déjà un sommet.
    '''
    if s in self.voisins:
        print('%s est déjà dans la liste des \
              sommets! '%(s))
    else:
        self.voisins[s] = []
```

```
def __initialise_voisins__(self, S, A):
    """
    Renvoie un dictionnaire {sommet: liste des voisins}
    """
    V = {}
    for s in S:
        V[s] = []

    for a in A:
        try:
            e, f = tuple(a)
        except TypeError:
            print('Attention, __initialise_voisins__
                  attend un conteneur (de longueur 2)
                  et a reçu %s.'%(a))
            return None
        if e not in V :
            V[e] = []
        if f not in V[e]:
            V[e].append(f)
        else:
            print('Warning: l\'arête %s figure deux
                  fois dans la liste'%(a))
        if f not in V :
            V[f] = []
        if e not in V[f]:
            V[f].append(e)
        else:
            print('Warning: l\'arête %s figure deux fois
                  dans la liste'%(a))
    return V
```


La méthode `composantes_connexes()` renvoie la liste des composantes connexes du graphe et implémente une adaptation de l'algorithme de parcours en profondeur. Elle ne prend pas d'autre argument que `self`, ce qui signifie qu'un appel effectif aura la forme

`nom_instance.composantes_connexes()`.

```
def composantes_connexes(self):
    '''
    Renvoie la liste des composantes connexes du graphe.
    Adaptation de l'algorithme de parcours en profondeur.
    '''
    self.__marques__ = {}
    for s in self.voisins:
        self.__marques__[s] = False

    L = []
    for s, v in self.__marques__.items():
        if not v:
            L.append(self.__composante__(s, []))

    return L

def __composante__(self, s, C):
    '''
    s : sommet du graphe;
    C : liste (composante de s en construction, appel
    principal avec C= []).
    Renvoie la composante connexe de s;
    '''
    self.__marques__[s] = True

    for e in self.voisins[s]:
        if not self.__marques__[e]:
            self.__composante__(e, C)
    C.append(s)
    return C
```

L'attribut `self.__marques__` joue le rôle de la variable *marque* dans le code de la page 371. Mais comme ici sa visibilité ne dépasse pas l'instance de la classe, la sécurité du code n'en est pas altérée (quand on parle d'**encapsulation** dans la POO, on se réfère plus à cette étanchéité entre les données de plusieurs objets qu'à l'organisation du code).

Cette classe implémente également le parcours en profondeur **`self.parcours(fonction)`** et fait de `self.__marques__` les mêmes usages que `self.composantes_connexes()` qui

en est une proche adaptation.

```
def parcours(self, fonction):
    '''
    Parcours en profondeur du graphe. Traitement des
    sommets avec fonction.
    '''
    self.__marques__ = {}
    for s in self.voisins:
        self.__marques__[s] = False

    for s, v in self.__marques__.items():
        if not v:
            self.__parcours__(s, fonction)

def __parcours__(self, s, fonction):
    '''
    Parcours en profondeur du graphe à partir du
    sommet s; appelé par self.parcours(fonction).
    '''
    self.__marques__[s] = True
    for e in self.voisins[s]:
        if not self.__marques__[e]:
            self.__parcours__(e, fonction)
    fonction(s)
```

10.3 Percolation, une implémentation objet

Nous passons maintenant à un autre type de modélisation objet dans laquelle la correspondance entre les objets physiques et les structures qui les représentent est particulièrement naturelle.

Nous allons définir une classe pour représenter des boules qui pourront s'auto-dessiner puis une classe pour les configurations qui pourront s'auto-construire. Cette dernière classe disposera d'une méthode pour construire le graphe des contacts sur l'ensemble des boules conductrices et donc, en calculant les composantes de ce graphe, vérifier qu'un amas relie le haut et le bas de la boîte.

Une classe pour représenter les boules

```

class boule:

    def __init__(self, i, x, y, p, r):
        """
        Définit les attributs i, x, y, r et self.conductrice
        qui est True avec la probabilité p, False avec
        la probabilité 1-p.
        """
        self.numero = i
        self.x      = x
        self.y      = y
        self.r      = r
        self.conductrice = self.__conductrice__(p)

    def __conductrice__(self, p):
        """
        p: flottant dans [0,1].
        Renvoie True avec la probabilité p, False avec ...
        """
        u = random.uniform(0, 1)
        if u < p:
            return True
        else:
            return False

    def dessine(self, f = 15):
        """
        Prépare le tracé de la boule (cercle de centre
        (self.x, self.y), de rayon self.r) et de son numéro;
        couleur 'black' pour les boules conductrices.
        Le tracé sera effectif avec l'appel pl.show().
        """
        if self.conductrice:
            col = 'black'
        else:
            col = 'lightgrey'
        X = np.linspace(0, 2*np.pi, 100)
        pl.plot(self.x + self.r*np.cos(X),
                self.y + self.r*np.sin(X), color = col)
        pl.annotate(str(self.numero) , xy = \
            (self.x- self.r/3, self.y - self.r/3), fontsize = f)

```

Méthodes et attributs d’une classe pour représenter les configurations

Attributs	
self.liste_boules	la liste des n^2 boules (instances de la classe boule) construites dans le constructeur self.__init__(n,p) (de configuration) ;
self.n	integer, hauteur de la boîte représentée par la configuration ;
self.r	diamètre des boules.

Méthodes	
self.__init__(n , p)	constructeur n : integer, hauteur de la boîte qui contient n^2 boules ; p flottant entre 0 et 1, probabilité pour qu’une boule soit conductrice ;
self.dessine()	dessine les n^2 boules de la liste ;
self.graphe_conductrices()	renvoie une instance de graphe_non_orienté construite avec comme liste des sommets la liste des boules conductrices et arêtes les paires de boules conductrices en contact direct ;
self.jonction()	renvoie un booléen selon qu’une composante connexe ou amas de billes conductrices relie le haut et le bas de la boîte ;
self.__numeros_voisines_h	renvoie la liste des n° des billes en contact avec b, situées au-dessus où à droite. Utile pour construire le graphe des contacts conducteurs.

Une classe pour représenter les configurations

```
class configuration:

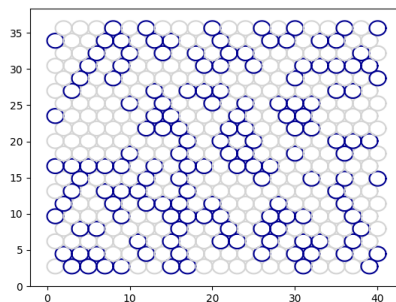
    def __init__(self, n, p):
        '''
        Construction d'un empilement de n x n boules qui
        seront conductrices avec une probabilité p.
        '''
        self.n = n
        self.r = 1
        rr3 = self.r*np.sqrt(3)

        self.liste_boules, k, = [], 0
        x, y = self.r, self.r
        for ell in range(0, n):
            # première boule de la ligne
            if ell%2 == 0:
                x = self.r
            else:
                x = 2*self.r
            y += rr3
            for c in range(0, n):
                b = boule(k, x, y, p, 1)
                self.liste_boules.append(b)
                x+= 2*self.r
                k += 1

    def dessine(self):
        for b in self.liste_boules:
            b.dessine()
        pl.show()
        pl.close()
```

Ces éléments suffisent d'ores et déjà pour produire la figure de droite avec le code

```
C = configuration(20, 0.4)
C.dessine()
```



```
def __numeros_voisines_hd__(self, b):
    '''
    Renvoie la liste des numéros des boules situées à
    droite ou au dessus de la boule b et en contact
    direct avec elle.
    '''
    n      = self.n
    k      = b.numero
    ell, c = divmod(k, n)

    if ell == n-1 and c == n-1:
        return []

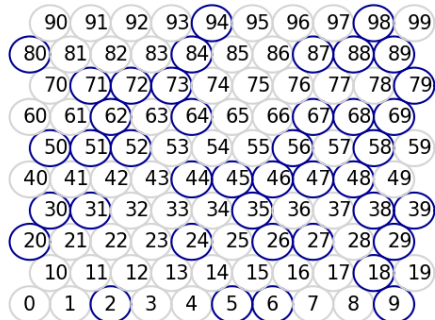
    elif ell == n-1:
        return [k+1]

    elif ell%2 == 0:
        if c == 0:
            return [k+1, k+n]
        elif c == n-1:
            return [k+n, k+n-1]
        else:
            return [k+1, k+n, k+n-1]

    elif c == n-1:
        return [k+n]
    else:
        return [k+1, k+n, k+n+1]
```

Sur la figure, la numérotation des billes à laquelle la méthode `__numeros_voisines_hd__(b)` se réfère.

Cette méthode, à usage interne, permet de construire le graphe non orienté des billes conductrices.



```

def graphe_conductrices(self):
    '''
    Construit le graphe dont l'ensemble des sommets est
    l'ensemble des boules conductrices et l'ensemble des
    arêtes l'ensemble des paires de boules conductrices
    en contact direct.
    '''
    sommets = [b for b in self.liste_boules \
                if b.conductrice ]
    numeros = [b.numero for b in self.liste_boules \
                if b.conductrice ]
    G        = gno.graphe_non_oriente(numeros, [])
    # on a fait import graphes_non_orientes as gno
    # les noeuds du graphe sont les numéros des billes
    # conductrices

    for b in sommets:
        for v in self.__numeros_voisines_hd__(b):
            b1 = self.liste_boules[v]
            if b1.conductrice:
                G.ajoute_arete({b.numero,b1.numero})
    return G

def jonction(self):
    '''
    Return True ou False selon qu'un amas réalise la
    jonction entre le haut et le bas de la boîte.
    '''
    G = self.graphe_conductrices()
    C = G.composantes_connexes()

    b0 = self.liste_boules[0]
    bN = self.liste_boules[self.n**2-1]

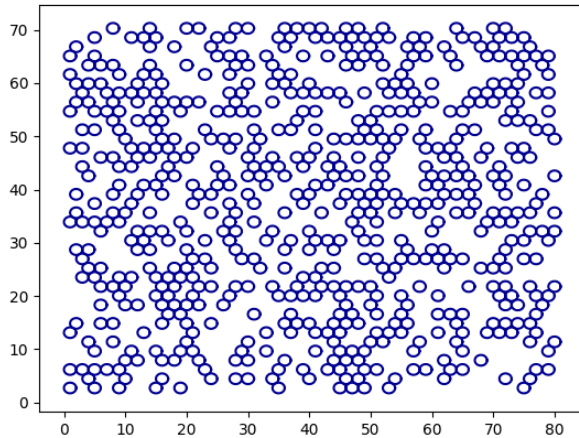
    for comp in C:
        comp_b = [self.liste_boules[i] for i in comp]
        comp_b.sort(key = lambda b: b.y)
        if abs(comp_b[0].y - b0.y) < 0.01 \
            and abs( comp_b[-1].y - bN.y) < 0.01:
            return True
    return False

```

Le code qui suit construit une configuration avec $40 \times 40 = 1600$ boules et $p = 0.40$. La méthode `graphe_conductrices()` permet de construire le graphe des seules boules conductrices. La méthode parcourt de ce graphe est appelée avec une fonction lambda qui assure le tracé de chacune des boules du graphe produit la figure qui suit en

appelant la méthode `dessine()` de la classe `boule`.

```
C = configuration(40, 0.4)
g = C.graphe_conductrices()
g.parcours(lambda i: C.liste_boules[i].dessine())
pl.show()
pl.close()
```



Billes conductrices lors d'un tirage de 1600 billes avec $p = 0.4$.

La classe `percolation` dévolue à l'organisation des simulations

On définit enfin une courte classe **`percolation`** dont le constructeur prend en arguments la hauteur `h_boite` de la boîte (ou le nombre de lignes de boules), le nombre `n_tirages` des tirages (aléatoires) de configurations que l'on veut réaliser pour chaque valeur de p , et la liste des valeurs de p à prendre en considération.

Cette classe réalise donc les tirages demandés, calcule le nombre moyens de jonctions réalisées pour chaque valeur de p et affiche une représentation graphique des résultats (proportion de jonctions réalisées pour chaque valeur de p).

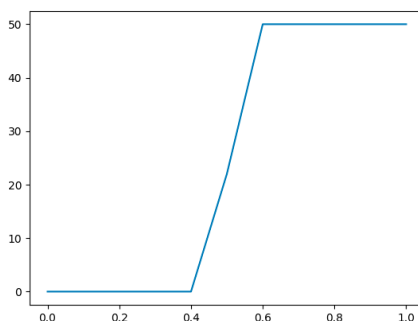
```
perc = percolation(30, 50)
print(perc.resultats)

>>> {0.0: 0, 0.1: 0, 0.2: 0, 0.3: 0, 0.4: 1, 0.5: 28, \
      0.6: 50, 0.7: 50, 0.8: 50, 0.9: 50, 1.0: 50}
```


Le graphique donnant la proportion de succès (un amas relie le haut et le bas) en fonction de p généré par le code ci-dessus.

On observe le brusque changement d'état qui sera d'autant plus marqué que n est grand. À voir ce graphique, on peut penser que le seuil critique est proche de 0,5.

C'est justement ce que le mathématicien Harry Kesten a démontré : en dimension 2, $p_c = 1/2$ (vous pourrez consulter [12]).



```
class percolation:

    def __init__(self, h_boite,
                  n_tirages,
                  P = [p/10 for p in range(0,11)]):

        self.n          = h_boite
        self.tirages     = n_tirages
        self.resultats = {}
        R = []
        for p in P:
            r = self.__succes__(p)
            self.resultats[p] = r
            R.append(r)
        pl.plot(P, R)
        pl.show()
        pl.close()

    def __succes__(self, p):
        '''
        Réalise self.tirages répétitions de l'expérience
        avec p flottant compris entre 0 et 1, fixé.
        '''
        s = 0
        for k in range(0, self.tirages):
            C = configuration(self.n, p)
            if C.jonction():
                s += 1
        return s/self.tirages
```



Troisième
semestre

Bases de données, langage SQL

11.1 Introduction

Vous savez déjà à quoi ressemble un tableur : c'est un logiciel (OpenOffice par exemple) permettant de faire simplement des traitements élémentaires de données alphanumériques stockées dans les cellules d'un tableau à deux entrées (ou feuille de calcul). Chaque cellule est référencée ou indexée par l'identification de sa ligne et de sa colonne : par exemple C-13 ou CW-13 (lorsqu'il y a plus de 26 colonnes), désignera le contenu de la cellule située colonne C (ou colonne CW), ligne 13. Pour faciliter la lecture, on peut placer, le plus souvent en première ligne, des chaînes de caractères rappelant la signification des éléments de la colonne.

On peut ainsi représenter des données assez volumineuses dans des fichiers textes, les charger pour les traiter dans un tableur qui dispose de fonctions diverses : statistiques, tris, accès aux cellules etc. *Cette façon de représenter des données, efficace pour des traitements statistiques simples, atteint rapidement ses limites lorsqu'il s'agit de représenter des entités distinctes ayant des relations complexes et nous allons voir comment on gère ces situations avec des bases de données relationnelles.*

• Une micro-base comme exemple

L'un d'entre nous a un jour voulu ajouter quelques fonctionnalités au site de sa classe :

- permettre aux enseignants d'adresser un courrier à l'ensemble des élèves de l'année en cours (ainsi qu'aux élèves des promotions précédentes à l'occasion de forum, journées portes ouvertes, etc...);
- permettre aux interrogateurs d'entrer les notes qu'ils avaient attribuées en « colle »;
- permettre aux professeurs et aux élèves de les consulter;
- permettre aux enseignants de retrouver les notes par interrogateur, par élève, par semaine d'attribution... et automatiser tous ces calculs, par exemple programmer la construction et l'envoi par courrier électronique d'un fichier csv contenant les informations utiles à date fixe.

Se posent alors un certain nombre de questions :

- comment identifier les données principales ?
- comment les représenter, les structurer ?
- comment exprimer les relations entre elles ?
- comment les exploiter, les rechercher une fois qu'elles ont été enregistrées dans des fichiers ?

La première idée serait de constituer une table avec l'ensemble des élèves et des informations spécifiques : nom, prénom, adresse mél etc... On peut souhaiter, habitués que nous sommes à utiliser des tableurs, ajouter des colonnes pour placer des notes (et combien ?). Là, survient un premier problème : comment, pour une note donnée savoir quel interrogateur l'a attribuée ? Pour un traitement automatisé, on imagine mal devoir déterminer, à partir du calendrier des interrogations orales, qui interrogeait ce jour là, alors que des changements auraient pu intervenir à plusieurs reprises : deux élèves qui permutent, un interrogateur qui en remplace un autre etc... Le mic-mac est déjà assuré dans une situation où le nombre des données est réduit si nous ne nous décidons pas à séparer les données de natures différentes. Nous avons dans ce cas précis décidé de représenter dans des tables distinctes des données associées à des *entités* distinctes. Une table pour les élèves, une table pour les interrogateurs, une table pour les matières, *une table encore pour les notes...*

Vous vous demandez comment seront gérées les informations en apparence dispersées entre ces tables, si considérer que l'on peut placer des notes dans une table à elles seules dévolues n'est pas du délire... Excellentes questions : c'est précisément l'objet de ce chapitre que de vous donner les moyens d'y répondre.

Et pour cela, nous allons faire un petit détour vers le vocabulaire de base de la théorie (naïve) des ensembles.

11.2 Qu'est ce qu'une base de données relationnelle ?

11.2.1 Les relations comme ensembles de p -uplets

Les notions qui permettent de concevoir ce qu'est une base de données relationnelle sont, en effet, les notions élémentaires de la théorie des ensembles que nous allons définir avant de les revisiter du point de vue qui nous intéresse dans ce chapitre.

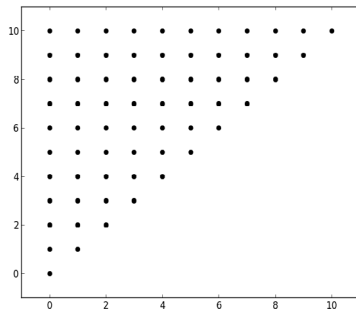
Définition 11.1 relations

Soient $D_1, D_2, \dots, D_n$ (avec $n = 1$ éventuellement), des ensembles.

Le *produit cartésien* de ces ensembles est l'ensemble, noté $\prod_{i=1}^n D_i$, des n -uplets $(t_1, \dots, t_n)$ tels que, pour tout i compris entre 1 et n , $t_i \in D_i$.

Une *relation* $\mathcal{R}$ définie sur $\prod_{i=1}^n D_i$ est un *sous-ensemble* de $\prod_{i=1}^n D_i$. On note usuellement $\mathcal{R}(a_1, \dots, a_n)$ pour signifier que le n -uplet $(a_1, \dots, a_n) \in \mathcal{R}$.

Deux relations que vous connaissez déjà



dividende	diviseur	quotient	reste
3	6	0	3
7	3	2	1
5	4	1	1
4	2	2	0
2	4	0	2
3	5	0	3
5	6	0	5
7	6	1	1
6	4	1	2

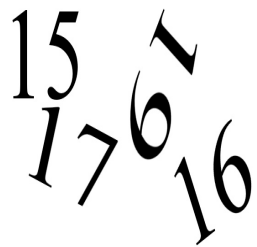
- L'ensemble des points de la figure de gauche est la relation $\mathcal{R}(i, j) = (i \leq j)$ dans le pavé $\llbracket 0, 10 \rrbracket^2$ de $\mathbb{N}^2$. On a $(1, 2) \in \mathcal{R}$ alors que $(2, 1) \notin \mathcal{R}$ par exemple. Nous savons distinguer les termes d'un couple appartenant à la relation en les nommant : le « plus petit », le « plus grand ».
- Le tableau, quant à lui, donne *un ensemble* de quadruplets appartenant (ou vérifiant) la relation $\mathcal{R}(a, b, q, r)$ définie comme l'ensemble des éléments $(a, b, q, r) \in \mathbb{N} \times \mathbb{N}^* \times \mathbb{N} \times \mathbb{N}$ tels que $a = bq + r$ et $0 \leq r < b$. Nous savons distinguer les termes d'un quadruplet appartenant à cette relation en les nommant « dividende », « diviseur », « quotient » et « reste ».

11.2.2 Modèle relationnel

Nous sommes maintenant en mesure de revenir à notre vague intention de représenter des notes dans un tableau. Il nous faudra pour cela bien formuler notre problème et préciser ce que nous entendons par table de notes.

Table « Notes »

id	valeur	élève	matière	prof	date
	15				
	19				
	17				
	16				
	18				



Si une note est un nombre entier compris entre 0 et 20 par exemple, cela risque fort de ressembler à la figure de droite et de ne servir à rien ! Par contre, si ce qui nous intéresse est une *entité* constituée de la *valeur* de la note, de l'*élève* à qui elle a été attribuée, de la *matière* dans laquelle elle a été attribuée, du *professeur* qui l'a attribuée, de la *date* à laquelle elle a été attribuée, alors, une note est un *p-uplet* et l'ensemble des notes est une relation qui peut être représentée par une table dont chaque ligne est un hexa-uplet.

L'ensemble de ces hexa-uplets définit une *relation*. Les diverses entités que nous voulons représenter dans notre base seront de la même façon associées à des tables de ce type. Pour les élèves, par exemple, nous préciserons un certain nombre d'*attributs* :

Table « Élèves »

id	nom	prénom	mél	naissance	lv 1	lv 2	option	groupe	passé

Identifiant

Vous avez observé que nous avons introduit dans nos deux tables une colonne « id » (pour « identifiant »). En effet, supposons que l'on veuille écrire un programme qui récupère les notes d'un ou de plusieurs élèves pour calculer les moyennes ou les présenter à qui interroge la base. Si dans la colonne « élève » de la table « Notes », (que nous noterons « Notes.élève ») un élève est repéré par son nom, ou même par nom-prénom, on rencontrera vite des difficultés : homonymies, fautes de frappe etc... Pour éviter cela, c'est l'identifiant (ici tout simplement un nombre unique attribué par programme lorsque l'entité est installée dans la base : le n° d'inscription dans la base) qui figure dans la colonne « Notes.élèves ». Il n'y a plus de risque d'erreur.

Votre n° INSEE par exemple est l'identifiant qui vous est associé dans la base de données de la sécurité sociale. Il a une structure puisque les premiers chiffres sont associés à certaines de vos caractéristiques : sexe à la naissance, lieu et date de naissance ; le reste est attribué dans l'ordre des inscriptions.

La question de la lisibilité de la table ne se pose pas : en effet, l'accès à une base de donnée est totalement interfacé et l'utilisateur y accède à travers plusieurs couches logicielles. Dans notre exemple,

- l'utilisateur final (par exemple le professeur qui veut donner ses notes) accède à un site en ligne, donne son nom et mot de passe, un programme interroge la base de données pour vérifier les droits d'accès, lui présente un formulaire avec des cases à cocher, des noms et prénoms en clair, l'avertit s'il y a un risque d'homonymie... Au pire l'utilisateur se trompe de date, de nom,... mais en tout cas, il ne peut rien casser !
- le programmeur du site écrit un programme qui permet l'accès à la base de donnée (s'y connecte, lit et écrit dans cette base). Il programme dans le langage de son choix (en l'occurrence du php, mais ce pourrait être du Python, après installation du module MySQL-python et importation du sous-module MySQLdb) ; il n'a toujours pas à connaître les identifiants, seulement les noms des colonnes *attributs* ;
- ce même programmeur, peut aussi accéder directement à la base soit en programmant dans un langage de gestion de base de données (le standard est SQL) soit à l'aide d'une interface adaptée. C'est avec MySql comme système

de base de données et PhpMyAdmin comme interface que nous apprendrons à gérer cette phase. Le reste n'est pas à notre programme.

Revenons donc aux définitions.

Nous sommes maintenant en mesure de définir le vocabulaire associé à la description d'une base de données relationnelle.

Définition 11.2 *bases de données relationnelles...*

Une *base de données* est un ensemble *structuré* de données informatiques (chaînes, valeurs numériques, dates...) dans lequel

- les données sont enregistrées sur un support permanent ;
- les données ne figurent qu'une fois (il n'y a pas redondance de l'information) ;
- chaque objet ou entité possède un identifiant unique.

Une *base de données relationnelle* est une base de données formée d'un ensemble de tables. Une *table* est un ensemble de p -uplets ; du point de vue mathématique et informatique *c'est* une relation définie sur un produit cartésien $\prod_i \mathcal{D}_i$ où le *domaine* $\mathcal{D}_i$ est l'ensemble des valeurs prise par l'attribut de la $i^{ième}$ colonne. L'ordre des p -uplets est donc indifférent et il n'y a pas de répétition.

terme	définition
relation sur $\prod_i \mathcal{D}_i$	sous-ensemble $\mathcal{R} \subset \prod_i \mathcal{D}_i$ du produit cartésien ;
table	<i>ensemble</i> de p -uplets (ou tuples) représentant une relation : <i>table</i> et <i>relation</i> sont des termes synonymes ;
attribut	nom désignant une colonne dans une table (les attributs d'une même table sont distincts). Exemples : dividende, diviseur, quotient, reste ; ou encore : identifiant, nom, prénom, mél, etc...
domaine	le domaine de valeurs de l'attribut A_i de la $i^{ième}$ colonne est l'ensemble $\mathcal{D}_i$ aussi noté $\mathcal{D}(A_i)$; Exemples : entiers, intervalle d'entiers ; flottants ; chaîne de caractère ;
p -uplet (ou ligne) d'une relation	un élément de la relation $\mathcal{R}$ donc de $\prod_i \mathcal{D}_i$ est représenté par une ligne dans la table ;
clé d'une relation	un sous-ensemble <i>minimal</i> d'attributs $\{A_{i_1}, \dots, A_{i_k}\}$ tel que $(t_{i_1}, \dots, t_{i_k}) = (s_{i_1}, \dots, s_{i_k}) \Rightarrow t = s$ Une clé formée d'un seul attribut peut jouer le rôle d'identifiant ; mais on préférera un identifiant qui n'est pas une propriété (neutre ou non sémantique) ;
clé primaire	une clé choisie parmi les clés possibles et signalée comme telle ;
identifiant	une clé primaire formée d'un attribut unique ; il est recommandé de le choisir non sémantique : c'est à dire introduit au seul effet d'être une clé ;

clé étrangère	attribut dont le domaine est l'ensemble des identifiants d'une autre table. Exemple : dans notre table « Notes », les attributs « Notes.élève », « Notes.professeur », seront des clés étrangères (identifiants d'une ligne d'autres tables) ;
schéma d'une relation	c'est l'expression $\mathcal{R}(A_1 : \mathcal{D}_1, ..., A_p : \mathcal{D}_p)$ qui spécifie une table ou relation ;
schéma d'une base de données relationnelle ;	c'est l'ensemble des schémas des relations qui constituent la base (une base de données relationnelles est un ensemble de tables (ou de relations).

Exercice 11.1 à propos des définitions

On reprend les tables « Notes » et « Élèves » de la page 423.

1. Sachant que dans la base de données qui les contient, sont aussi définies une table « Professeur » et une table « Matières » donner pour chacune des deux premières son schéma de relation.

Pour définir les domaines, on pourra utiliser les types suivants : INT (pour les entiers) ; VARCHAR(*n*) (pour les chaînes d'au plus *n* caractères) ; DATE qui permet de stocker des dates sous la forme 'AAA-MM-JJ' ; on peut aussi faire appel au type ENUM (<nom> = ENUM('xxx', 'yyyy', ..., 'zzzz') qui permet de définir une liste dans laquelle un attribut pourra être instancié...).

Préciser les attributs qui sont des clés primaires ou des clés étrangères.

2. Écrire le schéma relationnel de la table « Matières » qui doit permettre de retrouver le professeur qui est en charge du cours, la page web associée à la matière, le mot de passe permettant de la modifier...

Préciser les clés étrangères.

Écrire enfin le schéma relationnel d'une table « Professeur » permettant d'enregistrer les données concernant les interrogateurs qui doivent pouvoir accéder au site de la classe avec leur mot de passe, récupérer ce passe par mél lorsqu'ils l'auront perdu etc...

Préciser les clés étrangères.

3. Pour chaque table, discuter de l'existence éventuelle d'une clé sémantique.

Préciser sur un graphe, dans lequel figureront toutes les tables, la relation « un attribut de T_1 est une clé étrangère qui pointe vers T_2 ».

Corrigé en 11.1 page 447.

11.3 Algèbre relationnelle

Une base de données étant constituée, encore faut-il l'interroger. Ce sera la fonction des langages d'interrogation de bases de données que nous étudierons plus loin avec les variantes de SQL (pour Structured Query Langage).

Mais, de la même façon que nous avons présenté l’objet formel qui nous a permis de définir les tables et les bases de données relationnelles, nous allons définir les opérations sur ces relations qui seront ultérieurement traduites dans un langage particulier. Nous illustrerons ces définitions de façon à éclairer leur intérêt pratique.

11.3.1 La sélection

L’opération de *sélection* permet d’extraire d’une table les p –uplets qui vérifient une certaine propriété F . On note σ_F cette application $\sigma_F : \mathcal{R} \rightarrow \sigma_F(\mathcal{R})$ qui, à une relation associe une autre relation.

Exemple

Revenons à notre table « Notes » et décidons d’en extraire les lignes pour lesquelles $\text{Notes.valeur} > 16$. Le critère est la proposition $F(t) = (t[\text{valeur}] > 16)$ et $\sigma_F(\text{Notes})$ est la sous table formée des lignes (ou hexa-uplets) pour lesquelles $F(t)$ est vraie.

$\mathcal{R} =$

id	valeur	élève	matière	prof	date
	15				
	19				
	17				
	16				
	14				
	13				
	18				

$\downarrow \sigma_F$

$\sigma_F(\mathcal{R}) =$

id	valeur	élève	matière	prof	date
	19				
	17				
	16				
	18				

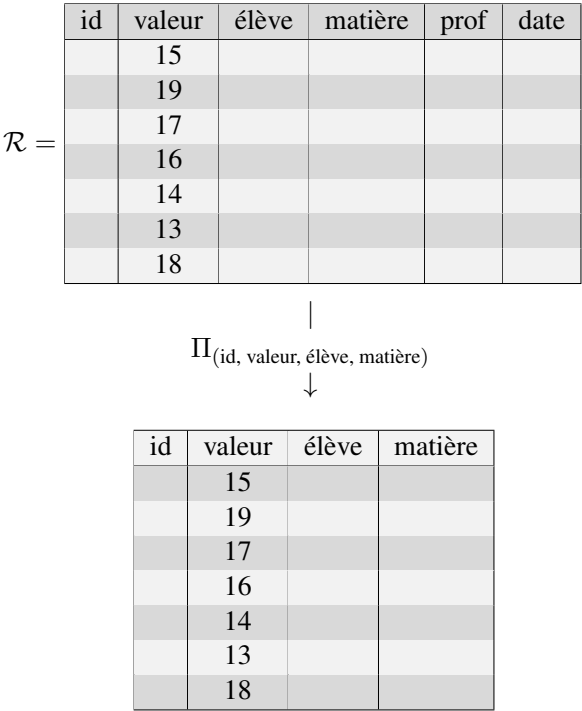
11.3.2 La projection

La projection consiste elle-aussi en une extraction de table, mais il ne s’agit plus ici d’extraire un sous-ensemble appartenant au même produit cartésien. On conserve les *colonnes* correspondant à certains attributs $(A_{k_1}, \dots, A_{k_p})$:

$$\Pi_{(A_{k_1}, \dots, A_{k_p})} : \mathcal{R} \rightarrow \Pi_{(A_{k_1}, \dots, A_{k_p})}(\mathcal{R})$$

Exemple

Toujours avec notre table « Notes », nous décidons que savoir qui a posé la note, et quand, ne nous intéresse plus. La projection sur les attributs Notes.id , Notes.valeur , Notes.élèves et Notes.matière nous fournit la sous-table que nous trouvons pertinente :



11.3.3 Le produit cartésien de deux tables, le renommage et la jointure

Deux tables $\mathcal{R}_1 \subset \prod_i \mathcal{D}_i$ et $\mathcal{R}_2 \subset \prod_j \mathcal{D}'_j$ admettent, comme tout couple d'ensembles, un *produit cartésien* qui est l'ensemble des couples (t', t'') où $t' \in \mathcal{R}_1$ et $t'' \in \mathcal{R}_2$. Un couple $t = (t', t'')$ est donc un élément de $\prod_i \mathcal{D}_i \times \prod_j \mathcal{D}'_j$, ce qui nous donne les attributs et le schéma relationnel de la relation ainsi obtenue.

Exemple

Considérons nos tables « Élèves » et « Notes ». Quel est leur produit cartésien ? A quoi ressemblent ses éléments ? Combien y en a-t-il ?

Concrètement, pour une classe de 48 élèves qui ont chaque semaine deux notes d'oral, il y aura au bout de 30 semaines, 2880 notes. Le produit des deux tables contiendra donc $48 \times 2880 = 138240$ lignes (dont la plupart n'ont aucun intérêt).

Chaque ligne est de la forme $t = (t', t'')$; comme t' a 10 attributs et t'' , qui est une instance de Notes en possède 6, cela nous fait 16 attributs dans t . Notre table comprendra donc 2 211 840 cellules.

La nouvelle table a un schéma relationnel dans lequel il faudra distinguer le premier attribut Élèves.id, issu de la table Élèves, du onzième qui est Notes.id... Nous traiterons ce problème plus loin avec le **renommage**.

Regardons plus avant à quoi ressemble notre nouvelle table. Chaque ligne y est formée de la concaténation de deux éléments t' et t'' des tables « Élèves » et « Notes ». Les deux lignes que nous mettons partiellement en évidence parmi les 138 240 sont

des 16-uplets obtenus par concaténation de t' associé à Sophie Germain dans la table « Élèves » et de t''_1 et t''_2 qui sont deux notes dont la première est attribuée à l'élève d'identifiant n° 17 (qui n'est donc pas Sophie Germain), alors que la seconde est attribuée à Sophie Germain (d'identifiant n° 29 dans la base).

id	nom	prénom	...	passee	id	valeur	élève	matière	...
⋮	⋮	⋮			⋮	⋮	⋮		
29	Germain	Sophie			1309	12	17		
⋮	⋮	⋮			⋮	⋮	⋮		
29	Germain	Sophie			1456	20	29		
⋮	⋮	⋮			⋮	⋮	⋮		

Il va de soi que, quelles que soient les applications que nous envisageons, bien des lignes dans ce produit cartésien sont vides de sens. Si nous voulons conserver les seuls couples dans lesquels l'élève est le même à gauche et à droite, il nous suffira de faire une **sélection** sur le critère $t'.id = t''.élève$ ce que nous exprimons

$$\sigma_{(\mathcal{R}'.id=\mathcal{R}''.\text{élève})}(\mathcal{R}' \times \mathcal{R}'')$$

C'est cette opération, que nous appellerons **jointure**, qui justifie que l'on s'intéresse au produit cartésien dans le modèle relationnel.

11.3.4 La jointure

On appelle donc **jointure** l'opération composée d'un produit cartésien et d'une sélection. On la note $\bowtie$:

$$(\mathcal{R}, \mathcal{R}'') \rightarrow \sigma_F(\mathcal{R}' \times \mathcal{R}'') = \mathcal{R}' \bowtie_F \mathcal{R}''.$$

Dans la plupart des cas le critère de sélection dans une jointure sera l'égalité entre une clé primaire ou un identifiant et une clé étrangère comme dans l'exemple qui précède.

Exemple

Le résultat de la jointure $\sigma_{(\mathcal{R}'.id=\mathcal{R}''.\text{élève})}(\mathcal{R}' \times \mathcal{R}'') = \mathcal{R}' \bowtie_{(\mathcal{R}'.id=\mathcal{R}''.\text{élève})} \mathcal{R}''$ donnerait

id	nom	prénom	...	passee	id	valeur	élève	matière	...
⋮	⋮	⋮			⋮	⋮	⋮		
29	Germain	Sophie			1456	20	29		
⋮	⋮	⋮			⋮	⋮	⋮		

11.3.5 Conflits de noms d’attributs et renommage

Nous avons vu un exemple de produit cartésien $\mathcal{R}' \times \mathcal{R}''$ dans lequel les deux relations $\mathcal{R}'$ et $\mathcal{R}''$ possédaient un attribut portant le même nom. Cette situation qui est (la plus) fréquente pose un problème. En effet, nous nous sommes jusqu’à présent contentés de *décrire* un modèle de représentation des données. Mais dans la perspective de *manipulations* des données avec des outils (langages, interfaces) conviviaux, on souhaite évidemment donner des noms qui ont du sens aux attributs (d’où les conflits potentiels) et les interroger en conservant ces noms.

- La première solution sera de préfixer dans la nouvelle table les attributs par le nom de la table d’origine. Notre table $\text{Élèves} \times \text{Notes}$ deviendrait alors en écrivant E.<attribut> plutôt que Élève.<attribut> :

E.id	E.nom	E.prénom	...	N.id	N.valeur	N.élève	N.matière	...
⋮	⋮	⋮		⋮	⋮	⋮		
29	Germain	Sophie		1456	20	29		
⋮	⋮	⋮		⋮	⋮	⋮		

- Une deuxième solution consiste à renommer les attributs en conflit de nom. Un opérateur est prévu à cet effet, noté ρ . Nous noterons

$$\rho(A_{k_1} \rightarrow A'_{k_1}, \dots, A_{k_p} \rightarrow A'_{k_p}) : \mathcal{R} \rightarrow \rho(A_{k_1} \rightarrow A'_{k_1}, \dots, A_{k_p} \rightarrow A'_{k_p})(\mathcal{R}).$$

Nous pouvons considérer l’expression $(A_{k_1} \rightarrow A'_{k_1}, \dots, A_{k_p} \rightarrow A'_{k_p})$ comme la *substitution* à opérer sur *certain*s noms d’attributs.

11.3.6 Union, intersection et différence

Il s’agit là des opérations ensemblistes usuelles des mathématiques. En ce qui concerne les tables on considérera la réunion, l’intersection ou la différence de tables *ayant le même schéma relationnel*. Le résultat de l’opération est alors une table de même schéma que les précédentes : *même nombre d’attributs, mêmes noms d’attributs et mêmes domaines*.

La différence nous sera indispensable pour certaines interrogations de bases de données sur un critère de la forme $Q = \text{non}(P)$.

• Complémentaire d’un ensemble

On souhaite déterminer les entités de la base qui ne vérifient pas une certaine association, par exemple les élèves qui n’ont pas de notes entre deux dates. On détermine alors, puisque cette information est présente dans la table « Notes », à l’aide d’une jointure suivie d’une projection, ceux des élèves qui ont une note au moins entre des deux dates, la différence nous donne la liste de ceux qui n’en ont pas.

• **Quantificateur universel**

De façon analogue, pour une recherche de la forme $\{x/\forall P, P(x)\}$ on cherchera d’abord $\{x/\exists P, nonP(x)\}$. C’est par exemple le cas si on recherche les élèves dont toutes les notes sont supérieures à une certaine valeur...

11.3.7 Récapitulatif et expressions de requêtes avec l’algèbre relationnelle

Nous pouvons retenir que le modèle relationnel consiste à représenter les données par des *tables* qui, du point de vue mathématique, sont des *relations*. Les opérateurs de l’algèbre relationnelle permettent, par des manipulations algébriques sur ces ensembles, d’extraire l’information contenue dans ces tables.

C’est à cela qu’est dévolue cette section. Nous verrons ensuite que ces spécifications (les données, les opérateurs) sont déjà implémentées dans des langages de manipulation et d’interrogation de bases de données que nous présenterons dans la section suivante.

opérateur	notation	spécification
sélection	$\sigma_F(\mathcal{R})$	sélectionne les <i>lignes</i> satisfaisant au critère F
projection	$\Pi_{(A_{k_1}, \dots, A_{k_p})}(\mathcal{R})$	sélectionne les <i>colonnes</i> dont les attributs A_{k_i} sont spécifiés
produit cartésien	$\mathcal{R}' \times \mathcal{R}''$	produit cartésien des deux ensembles, à associer à la jointure
jointure	$\mathcal{R}' \bowtie_F \mathcal{R}''$	composé du produit cartésien et d’une sélection selon le critère $F : \sigma_F(\mathcal{R}' \times \mathcal{R}'')$
union	$\mathcal{R}' \cup \mathcal{R}''$	union ensembliste de $\mathcal{R}'$ et de $\mathcal{R}''$
intersection	$\mathcal{R}' \cap \mathcal{R}''$	réunion ensembliste
différence	$\mathcal{R}' - \mathcal{R}''$	ensemble des éléments de $\mathcal{R}'$ qui n’appartiennent pas à $\mathcal{R}''$
renommage	$\rho_{(A_{k_1} \rightarrow B_{k_1}, \dots)}$	change le noms de certains attributs

Exemples de formules associées à une requête en langage courant

- **Projection** : table des identifiants, noms et prénoms des élèves

$$\Pi_{(id, nom, prénom)}(Élève)$$

- **Sélection, projection** : table des identifiants des élèves qui ont la note 20 une fois au moins

$$\Pi_{(élève)}(\sigma_{(valeur=20)}(Notes))$$

- **Sélection, jointure, projection** : table des identifiants, noms et prénoms des élèves qui ont eu la note 20 une fois au moins

On part de la sélection des notes égales à 20 :

$$\sigma_{(valeur=20)}(Notes)$$

On réalise une jointure entre cette table et la table « Élève »

$$\begin{array}{c} Élève \\ \bowtie \\ (Élève.id=Notes.id) \end{array} \sigma_{(valeur=20)}(Notes)$$

Il ne reste plus qu'à projeter sur id, nom et prénom, plutôt de de renommer, on précise la table lorsque les noms d'attributs (ici id) sont en conflit :

$$\Pi_{(id,nom,prénom)} \left(\begin{array}{c} Élève \\ \bowtie \\ (Élève.id=Notes.N.id) \end{array} \mathcal{R} \right)$$

• **Sélection, jointure, différence** : liste des identifiants des élèves qui n'ont jamais eu moins de 15.

On commence par sélectionner les notes de valeurs inférieures à 15 :

$$\sigma_{(valeur<15)}(Notes).$$

On réalise ensuite la jointure

$$\mathcal{R} = \begin{array}{c} Élève \\ \bowtie \\ (Élève.id=Notes.id) \end{array} \sigma_{(valeur<15)}(Notes)$$

En projetant sur les attributs de la table élève, on obtient une sous-table formée des élèves ayant failli une fois au moins

$$\mathcal{R}' = \Pi_{(...)}(\mathcal{R})$$

On fait enfin la différence

$$\mathcal{R} = Élève - \mathcal{R}'.$$

Exercice 11.2 requêtes en algèbre relationnelle

On travaille toujours avec nos deux tables « Élèves » et « Notes » (visibles page 424). Exprimer avec les opérateurs de l'algèbre relationnelle, les requêtes suivantes (ie : extraire ou créer les sous-tables satisfaisant le critère).

1. La table des élèves dont le nom de famille est Germain.
2. La table des noms et prénoms des élèves dont le nom de famille est Germain.
3. La table des notes de l'élève E1 (donné par son identifiant).
4. La table des identifiants des élèves qui ont une note inférieure à 10 entre le jour J1 et le jour J2 (donnés au format DATE).
5. La table des noms, prénoms et identifiants des élèves dont une note est inférieure à 10 entre le jour J1 et le jour J2 (donnés au format DATE).
6. La table des élèves (id, nom, prénom) qui n'ont pas de note entre les jours J1 et J2.
7. La table des élèves dont toutes les notes sont inférieures à 11 entre le jour J1 et le jour J2 (donnés au format DATE).

Corrigé en 11.2 page 448.

11.4 Langage de manipulation de données, SQL

Une fois les notions de base de données relationnelle et d’algèbre relationnelle définies, et avant de chercher à les implémenter (ce que nous ne ferons pas), il est plus que souhaitable de disposer d’un langage permettant de définir et de manipuler les données. De tels langages ont évidemment été conçus dans les années 1970 parallèlement à l’élaboration de ces concepts.

L’un d’entre eux, SQL (pour Structured Query Language), s’est aujourd’hui imposé comme un standard. Normalisé par l’ISO il inclut à la fois un langage de **définition de données** et un langage de **manipulation de données**. SQL permet une traduction syntaxique de l’algèbre relationnelle avec quelques extensions.

Nous allons pour vous le faire découvrir (dans le dialecte de MySQL), reprendre les interrogations que nous avons formulées page 431 dans le formalisme l’algèbre relationnelle et montrer comment on les traduit en langage SQL. Mais attention, commencez par lire et comprendre le paragraphe précédent avant de vous lancer dans ce qui suit : tenter de faire sans comprendre serait une perte de temps.

11.4.1 Les interrogations

Les « clauses » qui permettent de structurer les interrogations dans une base de données sont SELECT ... FROM ... WHERE

SELECT	Cette « clause » correspond à l’opérateur de <i>projection</i> : on y déclare une liste d’attributs appartenant aux tables précisées avec la clause FROM
FROM	Cette clause permet la déclaration de la liste des tables nécessaires à la requête.
WHERE	Cette clause permet de définir les prédicats de <i>sélection</i> ; ces prédicats sont formulés selon la syntaxe de la logique propositionnelle.
JOIN . . . ON	Assure la jointure de deux tables sur (ON) un critère de jointure (nous montrerons d’abord comment réaliser une jointure avec les trois clauses qui précédent).
NOT IN	Permet d’exprimer la négation, de réaliser la différence
ORDER BY	Permet de trier sur la valeur des attributs passés en arguments

UNION	réunion de deux
INTERSECT	intersection de deux
EXCEPT	différence
RENAME TABLE...TO	pour renommer une table

Commençons comme précédemment, avec les requêtes simples, projection et sélection.

- **Projection** : listes des identifiants noms et prénoms des élèves (éléments de la table « Élèves »)

```
SELECT id, nom, prénom FROM 'élèves'
```

On peut afficher la totalité de la table avec l’opérateur *

```
SELECT * FROM 'élèves'
```

Il est possible avec LIMIT de limiter le nombre de lignes affichées à la fois à l’écran, ici, 30 lignes à partir de la ligne 0 :

```
SELECT id, nom, prénom FROM 'élèves'
LIMIT 0 , 30
```

- **Sélection** : liste des lignes correspondant aux notes qui ont pour valeur 20

```
SELECT * FROM 'notes' WHERE valeur = 20
```

En ne choisissant pas les attributs, c’est une sélection pure que nous avons opéré.

- **Sélection et projection** : liste des identifiants des élèves qui ont la note 20 une fois au moins

```
SELECT élève FROM 'notes' WHERE valeur = 20
```

Comme cette requête affiche des doublons (une clé étrangère est affichée autant de fois qu'il y a de notes attribuées à l'élève correspondant), ce qui n'est pas la spécification de la sélection, on peut avec la clause **DISTINCT** imposer une occurrence au plus de chaque entité

```
SELECT DISTINCT élève FROM Notes WHERE valeur = 20  
LIMIT 0 , 30
```

• **Jointure, sélection, projection** : liste des identifiants, noms et prénoms des élèves qui ont eu la note 20 une fois au moins :

En sélectionnant les couples du produit cartésien élèves× notes, satisfaisant le critère élèves.id = notes.élève, on réalise une jointure :

```
SELECT * FROM élèves, notes  
WHERE élèves.id = notes.élève
```

On sélectionne ensuite sur le critère supplémentaire notes.valeur = 20, ce qui est une sélection sur la jointure ; enfin, en sélectionnant l'attribut élèves.nom, on réalise une projection qui aura pour but de n'afficher que la liste des noms.

```
SELECT élèves.nom FROM élèves, notes  
WHERE élèves.id = notes.élève AND notes.valeur = 20  
LIMIT 0 , 30
```

Il est évidemment préférable de n'afficher les noms qu'une fois par élève

```
SELECT DISTINCT élèves.nom FROM élèves, notes  
WHERE élèves.id = notes.élève AND notes.valeur = 20
```

• **La même requête, avec JOIN ON** : la clause **JOIN** permet de réaliser une jointure sur (ON) un critère particulier. La syntaxe est alors celle de l'algèbre relationnelle :

```
SELECT DISTINCT élèves.nom
FROM élèves JOIN notes ON élèves.id = notes.élève
WHERE valeur = 20
```

- **Sélection, différence** : liste des identifiants des élèves qui n'ont jamais eu moins de 15.

Commençons par appeler la liste des identifiants des élèves qui ont eu au moins une note inférieure à 15 : en sélectionnant sur *valeur* < 15 nous faisons une sélection, en sélectionnant l'attribut (ou la colonne) *notes.élève*, nous faisons une projection.

```
SELECT DISTINCT notes.élève FROM 'notes'
WHERE valeur < 15
LIMIT 0 , 30
```

On relève ensuite la liste des élèves qui ont toutes leurs notes au dessus de 15 en faisant la différence obtenue grâce à la relation NOT IN

```
SELECT DISTINCT élève FROM notes
WHERE
    élève NOT IN (
        SELECT notes.élève
        FROM 'notes'
        WHERE valeur < 15)
```

- **Sélection, jointure, différence** : liste des noms et prénoms des élèves qui n'ont jamais eu moins de 15.

Cette fois, il nous faut travailler sur deux tables, puisqu'à la différence de la requête précédente, les noms et prénoms ne figurent pas dans la table « Notes » mais seulement dans la table « Élève ». Nous commencerons par une jointure pour

```
SELECT DISTINCT élèves.nom, élèves.prénom
FROM élèves, notes
WHERE élèves.id = notes.élève
```

On extrait ensuite de cette table, obtenue par jointure, la sous-table obtenue avec le critère supplémentaire sur la valeur de la note :

```
SELECT élèves.nom, élèves.prénom
FROM élèves, notes
WHERE élèves.id = notes.élève
      AND notes.valeur < 15
```

On reprend alors notre première jointure en sélectionnant à l'aide de NOT IN, les lignes **qui ne sont pas** dans la sous-table que nous venons de construire (qui passe en deuxième argument de la relation NOT IN) :

```
SELECT DISTINCT élèves.nom, élèves.prénom
FROM élèves, notes
WHERE élèves.id = notes.élève
      AND (élèves.nom, élèves.prénom) NOT IN
          (SELECT élèves.nom, élèves.prénom
           FROM élèves, notes
           WHERE élèves.id = notes.élève
                 AND notes.valeur <15
          )
```

• **La même requête, avec JOIN ON :**

```
SELECT DISTINCT élèves.nom
FROM élèves
JOIN notes ON élèves.id = notes.élève
WHERE (élèves.nom, élèves.prénom )
      NOT IN (
        SELECT élèves.nom, élèves.prénom
        FROM élèves
        JOIN notes ON élèves.id = notes.élève
        WHERE notes.valeur <15  )
```

• **Jointures, réunion :** liste des noms des élèves qui ont eu, au moins une fois, moins de 10 dans la matière 1 **ou** dans la matière 2

```
SELECT élèves.nom
FROM notes, élèves
WHERE notes.élève = élèves.id
      AND notes.matière =1
      AND notes.valeur <10

UNION

SELECT élèves.nom
FROM notes, élèves
WHERE notes.valeur <10
      AND notes.élève = élèves.id
      AND notes.matière = 2
```

- Notre seul exemple de renommage (très pratique et sans difficulté logique)

```
RENAME TABLE `exemple_cpge`.`matières` TO
           `exemple_cpge`.`matières` ;
```

• **Ajoutons des relations à notre base de données :** dans les exercices et exemples qui suivent, nous supposerons que la micro-base de données sur laquelle nous avons travaillé jusqu'à présent, comprend, outre « Élèves » et « Notes », deux autres tables : « Matières » et « Professeurs ».

Matières

id	nom	professeur	page web

Les schémas relationnels (déjà proposés dans l'exercice 11.1, page 426) se déduisent facilement de la lecture des deux tables qui suivent.

Professeurs

id	nom	prénom	mél	matière 1	matière 2	passé

Exercice 11.3 jointures et SQL

Toujours avec notre mini-base d'illustration, et en utilisant FROM ... JOIN...ON, préciser les requêtes suivantes :

1. table des noms, prénoms et notes ;
2. table des noms, prénoms, notes et matière précisée par son nom ;
indication : vous avez ici trois tables à gérer conjointement, commencer par écrire la requête en langage relationnel.

Corrigé en 11.3 page 449.

Exercice 11.4 requêtes et SQL

On reprend à l'identique les interrogations de l'exercice (11.2) de la page 432.

Pour chaque question, réécrire la requête dans le langage de l'algèbre relationnelle et dans le langage SQL tel qu'il est présenté dans ce paragraphe.

On pourra utiliser BETWEEN pour les dates.

Corrigé en 11.4 page 450.

Exercice 11.5 formulations abstraites

Traduire les formules d'algèbre relationnelle qui suivent avec les fonctions SQL du cours.

1. $\Pi_{(A_i, A_j, A_k)} \mathcal{R}$.
2. $\sigma_{(A_i \geq 110 \text{ et } A_i \leq 120)} \mathcal{R}$
3. $\Pi_{(nom)} \circ \sigma_{(date \leq J2 \text{ et } date \geq J1)} (\mathcal{R})$
4. $\mathcal{R}_1 \bowtie_{(\mathcal{R}_1.A_i = \mathcal{R}_2.B_j)} \mathcal{R}_2$
 - (a) avec JOIN ... ON
 - (b) et sans utiliser JOIN...ON
5. $\{t' \in \mathcal{R}_1 / \exists t'' \in \mathcal{R}_2, F(t', t'')\}$ ou $F(t', t'')$ exprime une relation entre les p -uplets t' et t'' .
 - (a) Exprimer cet ensemble en algèbre relationnelle ;
 - (b) Exprimer la requête en SQL
6. Mêmes questions avec $\{t' \in \mathcal{R}_1 / \forall t'' \in \mathcal{R}_2, F(t', t'')\}$

Corrigé en 11.5 page 452.

11.4.2 GROUP BY, HAVING et les fonctions d'agrégation

Les requêtes que nous avons formulées dans l'algèbre relationnelle et éventuellement traduites en (My)SQL, portaient toutes sur des p -uplets. Leur formulation était de la forme « rechercher les éléments d'une table $\mathcal{R}$ tels que... ». Le langage SQL permet d'exprimer des conditions portant sur des *ensembles* de p -uplets. Il fournit pour cela des *fonctions d'agrégation* qui portent sur des sous-ensembles de données (c'est là le sens de l'expression « données agrégées », c'est à dire vues dans leur ensemble à travers leurs nombres comme les quarante voleurs, leurs moyennes, maxima ou minima etc...). SQL dispose de deux clauses supplémentaires GROUP...BY permettant de partitionner la relation (au sens de partitionner un ensemble) et HAVING.

GROUP . . . BY	Cette clause permet de <i>partitionner</i> le résultat d'une requête et éventuellement d'appliquer ensuite une fonction d'agrégation sur les parties obtenues.
ORDER . . . BY	Cette clause permet de trier les résultats
HAVING ()	Permet d'exprimer les conditions portant sur une fonction d'agrégation (prend donc, après une clause GROUP BY, la place qui serait celle de WHERE après SELECT FROM)
COUNT ()	Cette fonction d'agrégation calcule les effectifs de chaque sous-ensemble de la partition
MAX () , MIN ()	Maximum et minimum de l'attribut en argument pour chaque sous-ensemble de la partition (types numériques)
SUM () , AVG ()	Somme et moyenne de l'attribut en argument pour chaque sous-ensemble de la partition

Exemples de requêtes portant sur des données agrégées

- Nombre total de notes

```
SELECT COUNT( * ) FROM `notes`
```

- Nombre total de notes, moyenne de la classe

```
SELECT COUNT( * ) , AVG( valeur ) FROM `notes`
```

- Les valeurs des notes et le nombre d'occurrences pour chaque valeur

```
SELECT valeur, COUNT( valeur ) FROM Notes GROUP BY valeur
```

- Noms des élèves et leurs moyennes

```
SELECT nom, AVG( valeur ) FROM élèves JOIN notes
      ON élèves.id = notes.élève
GROUP BY Notes.élève
```

- Noms, matières et moyennes par matière pour chaque élève

On aura ici à regrouper trois tables, ce que nous faisons avec deux jointures :

```
SELECT * FROM
  (élèves JOIN notes ON élèves.id = notes.élève )
  JOIN matières ON notes.matière = matières.id
```

avant de poursuivre en regroupant par élève et par matière :

```
SELECT * FROM
  (élèves JOIN notes ON élèves.id = notes.élève )
  JOIN matières ON notes.matière = matières.id
GROUP BY élèves.id, matières.id
```

le tout suivi d'une projection et d'un appel à la fonction d'agrégation calculant la moyenne de chaque sous-ensemble de la partition (dont les paquets sont formés des notes attribuées à un même élève dans une même matière) :

```
SELECT élèves.nom, matières.nom, AVG(notes.valeur)
FROM (élèves JOIN notes ON élèves.id = notes.élève )
JOIN matières ON notes.matière = matières.id
GROUP BY élèves.id, matières.id
```

- Table des noms, prénoms, matière, et moyenne dans la matière des élèves y ayant une note supérieure à 15

```
SELECT élèves.nom, matières.nom, AVG( notes.valeur) FROM
  (élèves JOIN notes ON élèves.id = notes.élève))
JOIN matières ON notes.matière = matières.id
GROUP BY élèves.id, matières.id
HAVING AVG( notes.valeur ) >= 15
```

Après les jointures consécutives sur les trois tables, on regroupe les notes par identité d'élève et de matière (un ensemble de la partition est formé des notes d'un élève particulier dans une matière) et on sélectionne sur une condition portant sur la moyenne des valeurs dans un paquet.

11.5 Exercices

Exercice 11.6 requêtes¹

On dispose d'une base de données concernant les astres et planètes de notre galaxie qui contient cinq tables. Son schéma de base de données relationnelle est le suivant :

- Astre (nomAstre, diamètre)
- Planète (nomAstre, nomPlanète, diamètre, masse, tempsRévolution)
- Astrophysicien (nom, prénom, pays)
- Astéroïde (nomAstéroïde, masse, nom, prénom)
- Collision (nomAstre, nomPlanète, nomAstéroïde, date)

Un astre (par exemple le Soleil) est identifié par un nom et possède un diamètre.

Une planète (par exemple la Terre) est identifiée par un nom et le nom de l'astre autour duquel elle tourne. Elle possède un diamètre, une masse et un temps de révolution autour de son astre (la Terre met 365 jours environ pour faire le tour du soleil).

Un astrophysicien est identifié par un nom et un prénom.

Un astéroïde est identifié par un nom et caractérisé par une masse et le nom et le prénom de l'astrophysicien qui l'a découvert. Nom et prénom constituent une clé étrangère sur la relation Astrophysicien.

Enfin, une collision est identifiée par le nom d'une planète, le nom de l'astre autour duquel la planète tourne (nomAstre) et le nom d'un astéroïde (nomAstéroïde). Elle est caractérisée par la date de collision.

1. Pour chaque table ou relation, préciser une clé primaire *suggérée par la description qui a été donnée*. Cela signifie que l'on ne modifie pas la structure imposée et que l'on fait (à tort) confiance aux astronomes quant à leur nomenclature. Ces clés regroupent parfois plusieurs attributs.
2. De la même façon, préciser les clés étrangères.
3. Exprimer chacune des deux questions qui suivent en algèbre relationnelle puis en SQL.
 - (a) Quels sont les astres dont le diamètre dépasse $10^6 km$?
 - (b) Quelles sont les planètes qui tournent autour du soleil et dont le temps de révolution dépasse 500 jours ?
4. Exprimer les trois requêtes suivantes en SQL :
 - (a) Quels sont les astrophysiciens qui ont découvert un astéroïde au moins ?
 - (b) Quels sont les astrophysiciens qui n'en ont découvert aucun ?
 - (c) Quels sont les astres (nomAstre) dont toutes les planètes sont concernées par des collisions.

5. Exprimer les trois requêtes suivantes en SQL :

1. Très largement inspiré d'un sujet 2007 de TélécomSud-Paris.

- (a) Pour chaque astrophysicien, donner son prénom, son nom et le nombre d'astéroïdes qu'il a découvert.
- (b) Quels sont les astres possédant le maximum de planètes ?
- (c) Quelles sont les planètes qui sont concernées par le maximum de collisions possibles ?

Corrigé en 11.6 page 453.

Exercice 11.7 D'après le sujet Centrale 2020

On reprend ici la partie III du problème d'informatique commune Centrale 2020 dans laquelle est définie une base de données répertoriant des photographies. Le schéma de la base (ou modèle physique) est donné par la figure 4 qui est une reproduction partielle du sujet.

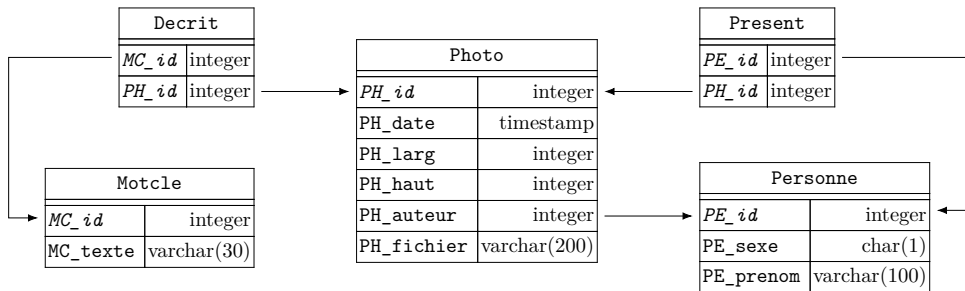


Figure 4 Structure physique de la base de données de photographies.

Cette base de données comporte cinq tables :

- la table Photo qui répertorie les photographies :
 - *PH_id* : identifiant ;
 - *PH_date* : date et heure de prise de vues ;
 - *PH_larg*, *PH_haut* : largeur et hauteur de la photographie en pixels ;
 - *PH_auteur* identifiant de l'auteur de la photographie ;
 - *PH_fichier* nom du fichier contenant la photographie.
- la table Personne des modèles et des photographes ;
- la table MotCle des mots clés utilisés pour décrire une photographie ;
- la table Decrit qui fait le lien entre les photographies et les mots clés qui les décrivent, ses deux colonnes constituent sa clé primaire ;
- la table Present qui fait le lien entre les photographies et les personnes qui y figurent, ses deux colonnes constituent sa clé primaire.

1. Quelques requêtes (voir les indications reproduites en fin d'exercice)

- (a) Écrire une requête SQL donnant les identifiants de toutes les photographies au ratio 4/3 c'est-à-dire dont le rapport largeur sur hauteur est 4/3.
- (b) Écrire une requête qui compte le nombre de photos prises par « Alice » ou « Bernard ».
- (c) Écrire une requête qui fournit l'identifiant et la date des photographies prises avant 2006 et associées au mot clé « surf ».

- (d) Écrire une requête qui donne le prénom de l'auteur et l'identifiant de tous les selfies, c'est-à-dire des photographies sur lesquelles l'auteur est présent.
- (e) Écrire une requête qui sélectionne toutes les photographies sur lesquelles sont présents « Alice » et « Bernard » à l'exception de tout autre personne.

2. Internationalisation des mots clés

Dans le but de gérer les mots-clés dans plusieurs langues, on décide de faire évoluer la structure de la base de données. Le cahier de charges de cette évolution stipule :

- l'ensemble des photographies sélectionnées à l'aide de mots-clés ne doit pas dépendre de la langue utilisée pour exprimer ces mots clés ; autrement dit les photographies décrites par le mot-clé « montagne » en français doivent être les mêmes que celles sélectionnées par les mots-clés « moutain », si la langue choisie est l'anglais, « berg » pour l'allemand « montaña » pour l'espagnol, etc. ;
 - il doit être possible, pour cette nouvelle base de données, d'écrire une requête de recherche de photographies par mot-clef en spécifiant la langue utilisée pour exprimer le mot-clé de telle sorte que changer de langue se fasse en modifiant uniquement des constantes dans la clause WHERE.
- (a) Proposer un nouveau modèle de base de données répondant à cette évolution du cahier des charges en ne détaillant que ce qui change (tables modifiées, nouvelles tables).
 - (b) Avec cette nouvelle base de données, écrire une requête qui permet de sélectionner les identifiants des photographies associées au mot-clé « moutain » exprimé en anglais.

Corrigé en 11.7 page 460.

SQL

- `T1 JOIN T2 USING (c1, c2, ...)` joint les deux tables T1 et T2 sur les colonnes c1, c2... qui doivent exister dans les deux tables ; équivalent à `T1 JOIN T2 ON T1.c1 = T2.c1 AND T1.c2 = T2.c2 AND ...`, sauf que les colonnes c1, c2... n'apparaissent qu'une fois dans le résultat.
- Les requêtes
 - `(SELECT ... FROM ... WHERE ...) INTERSECT (SELECT ... FROM ... WHERE ...)`
 - `(SELECT ... FROM ... WHERE ...) UNION (SELECT ... FROM ... WHERE ...)`
 - `(SELECT ... FROM ... WHERE ...) EXCEPT (SELECT ... FROM ... WHERE ...)`
 sélectionnent respectivement l'intersection, l'union et la différence des résultats des deux requêtes, qui doivent être compatibles : même nombre de colonnes et mêmes types.
- `EXTRACT(part FROM t)` extrait un élément de t, expression de type `date`, `time`, `timestamp` (jour et heure) ou `interval` (durée). *part* peut prendre les valeurs `year`, `month`, `day` (jour dans le mois), `doy` (jour dans l'année), `dow` (jour de la semaine), `hour`, etc.
- Les fonctions d'agrégation `SUM(e)`, `AVG(e)`, `MAX(e)`, `MIN(e)`, `COUNT(e)`, `COUNT(*)` calculent respectivement la somme, la moyenne arithmétique, le maximum, le minimum, le nombre de valeurs non nulles de l'expression e et le nombre de lignes pour chaque groupe de lignes défini par la clause `GROUP BY`. Si la requête ne comporte pas de clause `GROUP BY` le calcul est effectué pour l'ensemble des lignes sélectionnées par la requête.

11.6 SQLite, PostgreSQL, MySQL

Les systèmes de gestion de bases de données

Nous avons défini formellement le modèle de base de données relationnelle à partir des relations entre éléments de divers ensembles et des opérations algébriques sur ces relations. Nous avons vu comment elles permettent de modéliser *formellement* la plupart des interrogations courantes.

Il resterait, à ce stade, à *implémenter ces structures de données* ainsi que les opérations sur les objets informatiques ainsi construits en respectant les contraintes usuelles du génie logiciel :

1. le système doit permettre à un utilisateur de définir ses bases de données, c'est à dire des ensembles de tables ou relations avec leurs schémas relationnels (attributs, domaines, présence de clés...) ;
2. d'ajouter des items à ces tables, d'en modifier ultérieurement la structure ; de définir des droits d'accès...
3. d'interroger ces tables en utilisant de façon apparente ou de préférence transparente les opérateurs de l'algèbre relationnelle... par exemple en utilisant **SQL** ;
4. de permettre une gestion à plusieurs niveaux : l'utilisateur informaticien du système qui crée la base, doit pouvoir permettre à d'autres utilisateurs, situés à un plus haut niveau, de la consulter sans pour autant leur céder tous les droits...
5. les fonctions ou méthodes implémentées doivent être sûres : pérennité des données stockées dans des fichiers ; protection des accès (gestion des autorisations multiples, droits de lecture ou d'écriture...) ; elles doivent être efficaces (complexité des opérations, en particulier les très coûteux accès aux fichiers physiques).

Nous comprenons à ce stade que l'entreprise est au moins du même niveau de complexité que celle qui consiste à définir et implémenter un langage de programmation. Nous n'avons pas écrit nous même Python (ce n'est que partie remise pour ceux qui poursuivront l'aventure) et, de la même façon que nous avons utilisé Python pour programmer des algorithmes, nous utilisons des *systèmes de gestion de base de données relationnelles* (SGBD). Parmi eux les logiciels libres : SQLite, PostgreSQL, MySQL...

Nous ne décrivons pas la syntaxe des fonctions permettant la création, la modification de la structure d'une base ou l'ajout, le retrait, la modification des termes d'une table.

Vous serez par contre amenés en TP à utiliser l'interface d'un système de gestion de base de données pour le faire. A titre d'information on montre ici comment on peut créer la base de l'exercice 11.7 sous SQLite. La première ligne appelle sqlite3 et se place sur la base photos_centrale.db ou la crée si elle n'existe pas déjà. Ici la base est déjà créée mais la description de son schéma correspond aux instructions qui ont permis de construire ses tables.

```
thierry@th-14Z:~$ sqlite3 photos_centrale.db
SQLite version 3.11.0 2016-02-15 17:29:24
Enter ".help" for usage hints.
sqlite> .schema
CREATE TABLE Photo
(
    "PH_id"          integer NOT NULL,
    "PH_date"        timestamp without time zone,
    "PH_larg"        integer NOT NULL,
    "PH_haut"        integer NOT NULL,
    "PH_auteur"      integer,
    "PH_fichier"     character varying(200)
);
CREATE TABLE Personne
(
    "PE_id"          integer NOT NULL,
    "PE_sexe"        char(1),
    "PE_prenom"      character varying(100)
);
CREATE TABLE Present
(
    "PE_id"          integer NOT NULL,
    "PH_id"          integer NOT NULL
);
CREATE TABLE Mot_cle
(
    "MC_id"          integer NOT NULL,
    "MC_texte"       character varying(30)
);
CREATE TABLE Decrit
(
    "MC_id"          integer NOT NULL,
    "PH_id"          integer NOT NULL
);

sqlite> .tables
Decrit      Mot_cle    Personne   Photo      Present
```

11.7 Corrigés des exercices

Corrigé de l'exercice n° 11.1

1. Les schémas relationnels des tables déjà mentionnées en fonction des tables qui coexistent dans la base :

Table Élèves

id	INT (clé primaire)
nom	VARCHAR
prénom	VARCHAR
mél	VARCHAR
naissance	DATE
lv1	INT (clé étrangère)
lv2	INT (clé étrangère)
option	INT (clé étrangère)
groupe	INT
passé	VARCHAR

Table Notes

id	INT (clé primaire)
valeur	Int
élève	INT (clé étrangère)
matière	INT (clé étrangère)
professeur	INT (clé étrangère)
date	DATE

Les attributs lv1, lv2, option, de la table « Élèves » sont des liens vers des lignes de la table « Matières ».

2. On écrira de la même façon des tables de schéma relationnels :

Table Professeurs

id	INT (clé primaire)
nom	VARCHAR
prénom	VARCHAR
mél	VARCHAR
matière 1	INT (clé étrangère)
matière 2	INT (clé étrangère)
passé	VARCHAR (clé)

Table Matières

id	INT (clé primaire)
nom	VARCHAR
professeur	INT (clé étrangère)
adresse web	VARCHAR

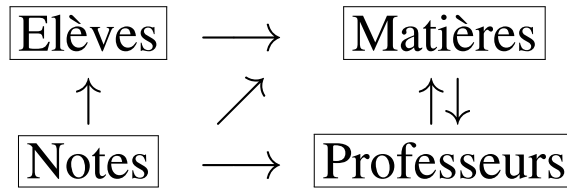
3. Les possibilités de clés : on raisonne en pensant que la structure de la table doit durer et que l'on ne veut pas avoir à la changer d'une année à l'autre.

Table élève : on peut avoir des homonymies de noms et de prénom (surtout si on conserve les données plusieurs années, ce qui est le cas ici). Noms et prénoms ne suffisant pas, on peut choisir comme clé (nom, prénom, mél). Des homonymes ne seront sûrement pas jumeaux et très rarement pacés et logés à même adresse mél. Mais l'identifiant est bien plus pratique et plus sûr comme clé primaire ou étrangère. Son usage doit être systématique.

Table professeur : idem.

Table Notes : pas de clé envisageable, sauf l'identifiant introduit à cet effet...

Une flèche de T_1 vers T_2 indique qu'un attribut de T_1 contient des clés étrangères pointant vers une ligne (un élément) de T_2



Corrigé de l'exercice n° 11.2

1. La table des élèves dont le nom de famille est Germain.

Réponse : c'est une sélection $\sigma_{(nom='Germain')}(Élève)$

2. La table des noms et prénoms des élèves dont le nom de famille est Germain.

Réponse : c'est une sélection suivie d'une projection sur les trois premiers attributs (on gardera l'identifiant, sinon en cas d'homonymie la cohérence de la table est violée) $\Pi_{(id,nom,prénom)} \left(\sigma_{(nom='Germain')}(Élève) \right)$

3. La table des notes de l'élève E1 (donné par son identifiant).

Réponse : sélection $\sigma_{(élève=E1)}(Notes)$ suivie, si on ne veut que la valeur, par une projection : $\Pi_{(valeur)} \left(\sigma_{(élève=E1)}(Notes) \right)$

4. La table des identifiants des élèves dont une note est inférieure à 10 entre le jour J1 et le jour J2 (donnés au format DATE).

Réponse : ces identifiants sont les clés étrangères de la colonne Notes.élève.

On commence donc par sélectionner les notes vérifiant le critère :

$$\sigma_{(valeur \leq 10 \text{ et } date \geq J1 \text{ et } date \leq J2)}(Notes)$$

et on projette ensuite pour obtenir les identifiants :

$$\Pi_{(élève)} \left(\sigma_{(valeur \leq 10 \text{ et } date \geq J1 \text{ et } date \leq J2)}(Notes) \right)$$

5. La table des noms, prénoms et identifiants des élèves dont une note est inférieure à 10 entre le jour J1 et le jour J2 (donnés au format DATE).

Réponse : il nous faudra ici faire une jointure pour rassembler des informations venant de deux tables.

étape 1 : on sélectionne dans la table notes

$$\mathcal{R} = \sigma_{(valeur \leq 10 \text{ et } date \geq J1 \text{ et } date \leq J2)}(Notes)$$

étape 2 : jointure entre la table Élèves et la table que nous venons de construire :

$$Élèves \bowtie_{(Élèves.id=Notes.élève)} \sigma_{(valeur \leq 10 \text{ et } date \geq J1 \text{ et } date \leq J2)}(Notes)$$

étape 3 : et enfin une projection sur les trois attributs demandés

$$\Pi_{(id,nom,prénom)} \left(Élèves \bowtie_{(Élèves.id=Notes.élève)} \mathcal{R} \right)$$

6. La table des élèves (*id*, *nom*, *prénom*) qui n'ont pas de note entre les jours *J1* et *J2*.

Réponse :

Il nous faut rechercher les informations dans deux tables ce qui va demander une jointure.

étape 1 : On sélectionne les notes distribuées entre ces deux dates

$$\sigma_{(date \geq J1 \text{ et } date \leq J2)}(Notes)$$

étape 2 : On réalise la jointure

$$\mathcal{R} = \text{Élève} \bowtie_{(\text{Élèves.id}=\text{Notes.élève})} \sigma_{(date \geq J1 \text{ et } date \leq J2)}(Notes)$$

étape 3 : En projetant sur les attributs (*id*, *nom*, *prénom*) on obtient ceux des élèves ayant eu une note au moins entre ces deux dates :

$$\mathcal{R}_1 = \Pi_{(id, nom, prénom)} \mathcal{R}$$

étape 4 : On fait la différence entre deux tables d'attributs (*id*, *nom*, *prénom*) pour obtenir ceux qui n'ont pas de notes entre ces deux dates.

$$\Pi_{(id, nom, prénom)}(\text{Élève}) - \mathcal{R}_1$$

7. La table des élèves dont **toutes** les notes sont inférieures à 11 entre le jour *J1* et le jour *J2* (donnés au format DATE).

Réponse : L'idée première est peut être de chercher les élèves dont une note entre ces deux jours est supérieure strictement à 11 pour faire une différence, mais il ne faudra pas oublier d'enlever aussi ceux qui n'ont pas de note durant cette période.

étape 1 : Les notes supérieures à 11 attribuées dans la période

$$\mathcal{R} = \sigma_{(valeur > 11 \text{ et } date \geq J1 \text{ et } date \leq J2)}(Notes)$$

étape 3 : Une jointure pour rapprocher ces informations de la table *Élèves*

$$\text{Élèves} \bowtie_{(\text{Élèves.id}=\text{Notes.élève})} \mathcal{R}$$

étape 4 : On projette sur les attributs de la table *élèves*, ce qui nous donne la sous-table de tous les élèves qui ont une note supérieure strictement à 11 dans la période ; on fait la différence

$$\mathcal{R}_c = \text{Élèves} - \Pi_{(id, nom, \dots, passe)} \left(\text{Élèves} \bowtie_{(\text{Élèves.id}=\text{Notes.élève})} \mathcal{R} \right)$$

Mais ce n'est pas fini, en enlevant les élèves qui ont une note supérieur à 11 dans la période, on garde ceux qui n'ont pas eu de note du tout.

Il reste donc une **étape 5 :** $\mathcal{R}_c$ — la table de la question précédente...

Corrigé de l'exercice n° 11.3

1. table des noms, prénoms et notes :


```
SELECT élèves.nom, élèves.prénom, notes.valeur
FROM élèves
JOIN notes ON élèves.id = notes.élève
```

2. table des noms, prénoms, notes et matière précisée par son nom

On doit, dans le cas présent réunir les informations situées dans trois tables (le nom de la matière figure dans la table « Matières » et seul l'identifiant figure comme clé étrangère dans « Notes ». En langage relationnel, nous voulons

$$\left(\text{Élèves} \bowtie_{(\text{élèves}.id=\text{Notes}.élève)} \text{Notes} \right) \bowtie_{(\text{Notes}.matière=\text{Matière}.id)} \text{Matières}$$

```
SELECT élèves.nom, élèves.prénom,
       notes.valeur, matières.nom
FROM (élèves JOIN notes ON élèves.id = notes.élève)
JOIN matières ON notes.matière = matières.id
```

On peut encore faire, la jointure étant associative) :

```
SELECT élèves.nom, élèves.prénom,
       notes.valeur, matières.nom
FROM élèves JOIN
      (notes JOIN matières ON
       notes.matière=matières.id)
ON élèves.id=notes.élève
```

Corrigé de l'exercice n° 11.4

1. La table des élèves dont le nom de famille est Germain

```
SELECT *
FROM élèves
WHERE élèves.nom = 'GERMAIN'
```

2. la table des noms et prénoms des élèves dont le nom de famille est GERMAIN

```
SELECT élèves.id, élèves.nom, élèves.prénom
FROM élèves
WHERE élèves.nom = 'GERMAIN'
```

3. La table des notes de l'élève E1 (donné par son identifiant).

```
SELECT *
FROM notes
WHERE notes.élève = 2
```

4. La table des identifiants des élèves dont une note est inférieure à 10 entre le jour J1 et le jour J2 (donnés au format DATE).

```
SELECT DISTINCT notes.élève
FROM notes
WHERE (notes.date
       BETWEEN '2013-05-15' AND '2013-06-01')
       AND notes.valeur < 10
```

5. La table des noms, prénoms et identifiants des élèves dont une note est inférieure à 10 entre le jour J1 et le jour J2 (donnés au format DATE).

```
SELECT élèves.id, élèves.nom, élèves.prénom
FROM   élèves,   notes

WHERE  élèves.id = notes.élève
      AND
      (notes.date
       BETWEEN '2013-05-15' AND '2013-06-01'
      )
      AND notes.valeur < 10
```

6. La table des élèves (id, nom, prénom) qui n'ont pas de note entre les jours J1 et J2.

```
SELECT id, nom, prénom
FROM élèves
WHERE (élèves.id, nom, prénom) NOT IN (
      SELECT élèves.id, nom, prénom
      FROM élèves, notes
      WHERE élèves.id = notes.élève
            AND (notes.date BETWEEN
                  '2013-05-15' AND '2013-06-15') )
```

7. La table des élèves dont **toutes** les notes sont inférieures à 11 entre le jour J1 et le jour J2 (donnés au format DATE).

```

SELECT élèves.id, nom, prénom
FROM élèves, notes
WHERE élèves.id = notes.élèves
      AND notes.date BETWEEN
        '2013-05-15' AND '2013-06-15')
AND (élèves.id, nom, prénom) NOT IN (
  SELECT élèves.id, nom, prénom
  FROM élèves, notes
  WHERE élèves.id = notes.élève
        AND (notes.date BETWEEN
              '2013-05-15' AND '2013-06-15')
        AND notes.valeur >=11
)

```

Commentaire : nous avons là une jointure puisque l'on appelle les éléments de deux tables avec comme critère la conjonction du prédicat ($\text{élèves.id} = \text{notes.élèves}$), et de deux autres conditions, existence d'une note entre les deux dates, et pas de note supérieure ou égale à 11.

Corrigé de l'exercice n° 11.5

1. $\Pi_{(A_i, A_j, A_k)} \mathcal{R}$.

```

SELECT Ai, Aj, Ak
FROM R

```

2. $\sigma_{(A_i \geq 110 \text{ et } A_i \leq 120)} \mathcal{R}$

```

SELECT *
FROM R
WHERE (Ai >= 110) AND (Ai <= 120)

ou encore,

SELECT *
FROM R
WHERE (Ai BETWEEN 110 AND 120)

```

3. $\Pi_{(nom)} \circ \sigma_{(date \leq J2 \text{ et } date \geq J1)} (\mathcal{R})$

```

SELECT nom
FROM R
WHERE date BETWEEN J1 AND J2

```

4. $\mathcal{R}_1 \bowtie_{(\mathcal{R}_1.A_i = \mathcal{R}_2.B_j)} \mathcal{R}_2$

Avec JOIN ... ON

```
SELECT *
FROM R1 JOIN R2 ON R1.Ai = R2.Bj
```

Sans utiliser JOIN ... ON

```
SELECT DISTINCT *
FROM R1, R2
WHERE R1.Ai = R2.Bj
```

5. $\{t' \in \mathcal{R}_1 / \exists t'' \in \mathcal{R}_2, F(t', t'')\}$ ou $F(t', t'')$ exprime une relation entre les p -uplets t' et t'' .

En algèbre relationnelle, étant entendu que $F(t', t'')$ est une relation entre les attributs (ou composantes) de t' et t'' :

$$\Pi_{A_1, \dots, A_n} (\sigma_{F(t', t'')} (\mathcal{R}_1 \times \mathcal{R}_2)) = \Pi_{A_1, \dots, A_n} (\mathcal{R}_1 \bowtie_{(F(t', t''))} \mathcal{R}_2)$$

En SQL

```
SELECT R1.A1, ..., R1.An
FROM R1, R2
WHERE F(..., ...)
```

```
SELECT R1.A1, ..., R1.An
FROM R1 JOIN R2 ON F(..., ...)
```

6. $\{t' \in \mathcal{R}_1 / \forall t'' \in \mathcal{R}_2, F(t', t'')\}$

C'est le complémentaire dans $\mathcal{R}_1$ de l'ensemble

$$\Pi_{A_1, \dots, A_n} (\sigma_{\text{non}F(t', t'')} (\mathcal{R}_1 \times \mathcal{R}_2)) = \Pi_{A_1, \dots, A_n} (\mathcal{R}_1 \bowtie_{(\text{non}F(t', t''))} \mathcal{R}_2)$$

```
SELECT DISTINCT *
FROM R1
WHERE R1.A1, ..., R1.An NOT
IN (
  SELECT R1.A1, ..., R1.An
  FROM R1 JOIN R2 ON nonF(..., ...)
)
```

Corrigé de l'exercice n° 11.6

Cet exercice présente une base dont le schéma est simple. Il a pour but de vous exercer à construire des requêtes. Le schéma de la table *astre* par exemple, suppose que l'on sait nommer les étoiles sans ambiguïté et que les noms ne changeront pas au cours de la vie de la base, ce qui nous permettra de choisir l'attribut *astre.nomAstre* comme identifiant ou clé primaire dans cette la table. Ce n'est peut être pas (sûrement pas) ce que nous ferions pour une base en taille réelle destinée à durer des décennies.

Nous n'abordons pas non plus les problèmes pratiques liés à la représentation des données numériques dans une base de données².

1. Choix de clés primaires.

— **astre(nomAstre, Diamètre) :**

On choisira `astre.nom`. Le couple (`nomAstre`, `Diamètre`) est une réponse acceptable sur le plan de la syntaxe, mais moins pertinente si nous voulons des clés étrangères vers la table `Astre` dans les autres tables (telles qu'elles sont).

— **planète (nomAstre, nomPlanète, diamètre, masse, tempsRévolution) :**

On choisira pour clé l'attribut `nomPlanète` (sans hésiter, si on sait que, selon certaines conventions des astrophysiciens, c'est un identifiant formé du nom du système stellaire suivi du nom de la planète...). A défaut, le couple (`nomAstre`, `nomPlanète`) est aussi acceptable (mais même remarque pour le choix des clés étrangères).

— **astrophysicien (nom, prénom, pays) :** C'est plus délicat, des homonymies sont fort possibles, on choisira (`astrophysicien.nom`, `astrophysicien.prénom`). Une date de naissance aurait réduit les risque de collision. Mais on ne touche pas au schéma de la table avons nous dit !

— **astéroïde(nomAstéroïde, masse, nom, prénom) :** le nom devrait suffire (si la façon de nommer est bijective, ce que l'on supposera).

— **collision (nomAstre, nomPlanète, nomAstéroïde, date) :** là il est certain que les attributs `nomPlanète`, `nomAstéroïde` sont nécessaires. Peut-il y avoir plusieurs dates de collision ? Si oui la clé primaire (`nomPlanète`, `nomAstéroïde`, `date`) s'impose. Deux réponses acceptables.

2. Clés étrangères :

— **astre(nomAstre, Diamètre) :** pas de clé étrangère ;

— **planète (nomAstre, nomPlanète, diamètre, masse, tempsRévolution) :** clé étrangère vers la table `astre` `nomAstre` ;

— **astrophysicien (nom, prénom, pays) :** pas de clé étrangère ;

— **astéroïde(nomAstéroïde, masse, nom, prénom) :** clé étrangère vers la table `astrophysicien` : (`astéroïde.nom`, `astéroïde.prénom`)

— **collision (nomAstre, nomPlanète, nomAstéroïde, date) :** deux clés étrangères, l'un vers la table `Astre` (c'est `collision.nomAstre`) et vers la table `Planète` (c'est `collision.nomPlanète`).

3. (a) Recherche des noms des astres de diamètre supérieur à $10^6 km$ (remplacé ici par $0.72 \times$ diamètre du soleil : en pratique on aura intérêt à introduire les grandeurs physiques des astres en rayon solaire, masse solaire...) :

```
SELECT nomAstre FROM astre WHERE diamètre > 0.72
```

2. Que l'on rencontrerait vite ici dans MySQL avec les masses des astres qui, comme chacun le sait sont astronomiques Essayez de taper la masse du soleil : $1.9891 \times 10^{30} kg$ sous MySQL.

- (b) Planètes du système solaire et dont le temps de révolution dépasse les 500 jours (terrestres !) :

```
SELECT nomPlanète FROM planète
WHERE nomAstre = 'Soleil' AND tempsRévolution > 500
```

4. (a) La question mériterait d'être précisée (pour une proposition de sujet blanc !).
- C'est une simple sélection projection si on ne veut que les noms des découvreurs :

```
SELECT DISTINCT nom FROM astéroïde
```

- $\bowtie$ Si par contre, on veut toutes leurs coordonnées, il faudra réaliser une jointure entre deux tables :

```
SELECT DISTINCT astrophysicien.nom,
               astrophysicien.prénom, astrophysicien.pays
FROM   astéroïde JOIN astrophysicien
WHERE  astéroïde.nom = astrophysicien.nom
```

Ce que l'on peut encore écrire :

```
SELECT DISTINCT astrophysicien.nom,
               astrophysicien.prénom
FROM   astrophysicien JOIN astéroïde
ON     astéroïde.nom = astrophysicien.nom
```

- (b) On fera une projection-sélection sur la différence entre la table astrophysicien et la sous-table obtenue par la même projection-sélection sur le résultat d'une jointure.

```

SELECT  DISTINCT    astrophysicien.nom,
                    astrophysicien.prénom
FROM      astrophysicien
WHERE     astrophysicien.nom NOT IN (
        SELECT astrophysicien.nom
        FROM   astrophysicien JOIN astéroïde
        ON     astéroïde.nom = astrophysicien.nom)

```

(c) Allons y par étapes sachant que l'on cherche le complémentaire, dans l'ensemble des astres, du sous-ensemble des astres dont une planète est sans collision. Nous chercherons dans l'ordre :

- i. les planètes avec collision(s) ;
- ii. les planètes sans collision ;
- iii. les astres dont une planète est sans collision ;
- iv. les astres qui ont une planète renseignée au moins et dont toutes les planètes ont une collision³ ;

```

SELECT planète.nomPlanète
FROM   planète JOIN collision
      ON planète.nomPlanète = collision.nomPlanète

```

On passe au complémentaire :

```

SELECT planète.nomPlanète FROM planète
WHERE  planète.nomPlanète NOT IN (
    SELECT planète.nomPlanète
    FROM   planète JOIN collision
    ON     (planète.nomPlanète = collision.nomPlanète )
)

```

On observera que l'on aurait pu écrire

```

SELECT planète.nomPlanète FROM planète
WHERE  planète.nomPlanète NOT IN (
    *** placer ici la requête précédente *** )

```

3. Attention à la propriété toujours vraie si $E = \emptyset : \forall x \in E, P(x)$.

Les astres dont une planète est sans collision : Dans le cas où l'on ne veut **que les noms**, il suffit de faire :

```
SELECT planète.nomAstre FROM planète
WHERE planète.nomPlanète NOT IN
  (SELECT planète.nomPlanète
   FROM  planète JOIN collision
   ON    (planète.nomPlanète = collision.nomPlanète)
  )
```

⊠ Les astres dont une planète est sans collision bis : **si nous voulons accéder à la table astre elle même**, il nous faudra joindre les tables astre, planète et collision :

```
SELECT astre.nomAstre, astre.diamètre
FROM astre JOIN planète
ON astre.nomAstre = planète.nomAstre
WHERE planète.nomPlanète NOT IN (
  SELECT planète.nomPlanète
  FROM planète JOIN collision
  ON (planète.nomPlanète = collision.nomPlanète )
)
```

Enfin, le complémentaire :

```
SELECT DISTINCT planète.nomAstre
FROM planète
WHERE planète.nomAstre NOT IN (
  SELECT planète.nomAstre
  FROM planète
  WHERE planète.nomPlanète NOT IN (
    SELECT planète.nomPlanète
    FROM planète JOIN collision
    ON (planète.nomPlanète = collision.nomPlanète)
  )
)
```

⊠ A nouveau le complémentaire dans l'ensemble des astres ayant une planète renseignée au moins, de la table des astres dont une planète est sans collision : là il nous faut trois jointures (pour un astre, toutes les planètes doivent avoir été explorées (au sens de la base de données, cela va

de soi)).

```
SELECT astre.nomAstre
FROM astre JOIN planète
ON astre.nomAstre = planète.nomAstre
WHERE astre.nomAstre NOT IN (
    SELECT astre.nomAstre
    FROM astre JOIN planète
    ON astre.nomAstre = planète.nomAstre
    WHERE planète.nomPlanète NOT IN (
        SELECT planète.nomPlanète
        FROM planète JOIN collision
        ON (planète.nomPlanète
            = collision.nomPlanète)
    )
)
```

Attention : la requête qui suit renvoie aussi les astres sans planète renseignée dans la base :

```
SELECT astre.nomAstre FROM astre
WHERE astre.nomAstre NOT IN (
    SELECT astre.nomAstre FROM astre JOIN planète
    ON astre.nomAstre = planète.nomAstre
    WHERE planète.nomPlanète NOT IN (
        SELECT planète.nomPlanète
        FROM planète JOIN collision
        ON (planète.nomPlanète = collision.nomPlanète)
    )
)
```

5. Fonctions d'agrégation et comptage.

- (a) Pour afficher les astrophysiciens qui ont découvert un astéroïde, la clause **GROUP BY** permet de regrouper en une ligne ceux qui en ont découvert plus d'un :

```
SELECT astrophysicien.nom, astrophysicien.prénom
FROM astrophysicien JOIN astéroïde
ON astéroïde.nom = astrophysicien.nom
GROUP BY astrophysicien.nom, astrophysicien.prénom
```

Pour compter le nombre d'astéroïdes découvert par chacun d'eux ; COUNT(...) permet de dénombrer les nombre de tuples regroupés sur une même ligne dans la requête précédente :

```
SELECT astrophysicien.nom,  
       astrophysicien.prénom, COUNT(astéroïde.nom )  
FROM astrophysicien JOIN astéroïde  
ON astéroïde.nom = astrophysicien.nom  
GROUP BY astrophysicien.nom, astrophysicien.prénom
```

(b) Astres et planètes dans la base. Avançons toujours à petits pas.

Le nombre de planètes par astre (identifié par son nom).

Tout simplement, si on ne veut que le nom qui est renseigné dans collision :

```
SELECT planète.nomAstre, COUNT(*) FROM planète  
GROUP BY planète.nomAstre
```

✕ Si par contre, on souhaite avoir accès à la table astre elle même il nous faudra faire un jointure :

```
SELECT astre.nomAstre, astre.diamètre, COUNT(planète.nomAstre)  
FROM astre JOIN planète  
ON (astre.nomAstre = planète.nomAstre)  
GROUP BY astre.nomAstre
```

Réponse alternative : On trie dans l'ordre DESCendant :

```
SELECT astre.nomAstre, COUNT(planète.nomAstre )  
FROM astre JOIN planète  
ON (astre.nomAstre = planète.nomAstre)  
GROUP BY astre.nomAstre  
ORDER BY COUNT(planète.nomAstre ) DESC
```

On affiche un seul résultat avec LIMIT... (page 434) :

```

SELECT astre.nomAstre, COUNT( planète.nomAstre )
FROM astre
JOIN planète ON ( astre.nomAstre = planète.nomAstre )
GROUP BY astre.nomAstre
ORDER BY COUNT( planète.nomAstre ) DESC
LIMIT 0 , 1

```

Une meilleure réponse.

Arrivés là, nous observons qu'il peut y avoir plusieurs astres correspondant à ce maximum, ; cette façon de faire n'est donc pas pleinement satisfaisante. Pensons donc à la clause HAVING (page 440 ou 333 première édition).

On sélectionne avec HAVING, parmi les astres précédemment sélectionnés ceux dont le COUNT() est plus grand que tous les autres (**ALL**) :

```

SELECT planète.nomAstre, COUNT( planète.nomPlanète )
FROM planète
GROUP BY planète.nomAstre
HAVING COUNT(planète.nomPlanète) >= ALL(
    SELECT COUNT(*) FROM planète
    GROUP BY planète.nomAstre
)

```

✕ On peut faire la même chose avec des jointures si on veut accéder à la table astre elle-même...

(c) Comme précédemment :

```

SELECT nomPlanète, nomAstre
FROM Collision
GROUP BY nomPlanète, nomAstre
HAVING COUNT(*) >= ALL (SELECT COUNT(*)
    FROM Collision
    GROUP BY nomPlanète, nomAstre)

```

Corrigé de l'exercice n° 11.7

1. (a) SELECT PH_id FROM Photo WHERE PH_larg*3=PH_haut*4;

```
(b) SELECT COUNT(*)
FROM Photo JOIN Personne ON Photo.PH_auteur = Personne.PE_id
WHERE Personne.PE_prenom = 'Alice'
    OR Personne.PE_prenom = 'Bernard';
```

Remarque : Comme les noms de colonnes dans les tables sont différents, on peut simplifier l'écriture en :

```
(b) SELECT COUNT(*)
FROM Photo JOIN Personne ON PH_auteur = PE_id
WHERE PE_prenom = 'Alice' OR PE_prenom = 'Bernard';
```

(c) Il nous faudra ici récupérer les informations de 3 tables Photo, Motcle et Decrit. La conception des tables permet d'utiliser JOIN ... USING ce qui simplifie considérablement les écritures.

```
(c) SELECT PH_id, PH_date
FROM (Photo JOIN Decrit USING PH_id) JOIN MotCle USING MC_id
WHERE MC_texte = 'surf' AND EXTRACT( year FROM PH_date) <2006;
```

(d) Nous devons faire une jointure sur Photo, Present et Personne toujours facilitée par JOIN USING

```
(d) SELECT PE_prenom
FROM (Photo JOIN Present USING PH_id) JOIN Personne USING PE_id
WHERE PH_auteur = PE_id;
```

(e) Les photos sur lesquelles 'Bernard' est présent :

```
SELECT PH_id
FROM (Photo JOIN Present USING PH_id) JOIN Personne USING PE_id
WHERE PE_prenom = 'Bernard';
```

Les photos sur lesquelles 'Bernard' et 'Alice' sont présents :

```
SELECT PH_id FROM (
    (Photo JOIN Present USING PH_id) JOIN Personne USING PE_id
    WHERE PE_prenom = 'Bernard'
INTERSECT
    (Photo JOIN Present USING PH_id) JOIN Personne USING PE_id
    WHERE PE_prenom = 'Alice'
);
```

Les photos sur lesquelles 'Bernard' et 'Alice' sont présents et personne d'autre :

```

SELECT PH_id FROM (
  ((Photo JOIN Present USING PH_id) JOIN Personne USING PE_id
   WHERE PE_prenom = 'Bernard'
  INTERSECT
   (Photo JOIN Present USING PH_id) JOIN Personne USING PE_id
   WHERE PE_prenom = 'Alice'
  )
  EXCEPT (
    (Photo JOIN Present USING PH_id) JOIN Personne USING PE_id
    WHERE PE_prenom != 'Bernard' AND PE_prenom != 'Alice' )
  );

```

2. (a) Comme les requêtes ne doivent pas dépendre de la langue utilisée, un mot clé doit être présent dans toutes les langues utilisées une fois et une seule et l'association d'une photo et d'un mot doit être simultanée pour toutes les langues.

On modifie donc la table MotCle et elle seule de la façon suivante :

MC_id	integer
MC_text_fr	varchar(30)
MC_text_en	varchar(30)
MC_text_es	varchar(30)
MC_text_it	varchar(30)
MC_text_ge	varchar(30)

Cela garantit que lorsqu'un mot clé est associé à une photo c'est dans toutes les langues en même temps. Par contre rien ne garantit qu'une même graphie dans une langue n'ait pas deux traductions possibles : deux mots de sens différents mais homographes dans une langue, ne seront pas homographes dans les autres langues⁴. Pour interdire cette situation on procède généralement avec une vérification à chaque insertion, ce qui n'est pas l'objet de la question, ni de notre programme.

(b)

```

SELECT PH_id
FROM (Photo JOIN Decrit USING PH_id) JOIN MotCle USING MC_id
WHERE MC_text_en = 'mountain';

```

4. Ainsi « voile » en français pourra se traduire en anglais tantôt par « sail » (penser à Éric Tabarly), tantôt par « veil » (penser à ceux qu'il semble obséder).

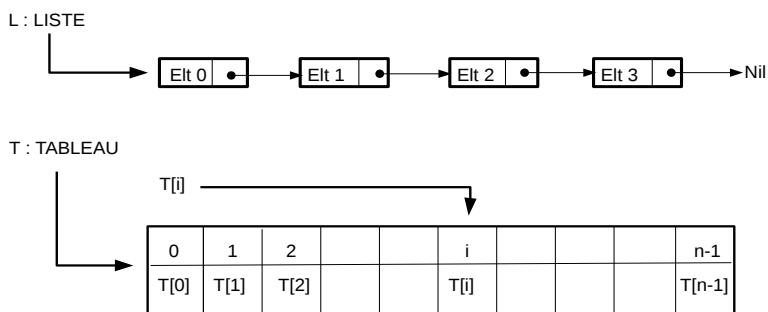
À propos des dictionnaires

12.1 Dictionnaires ou tableaux associatifs

Cette section est destinée à montrer dans les grandes lignes comment sont spécifiés et implémentés les dictionnaires ou tableaux associatifs que nous avons déjà utilisés en boîte noire.

12.1.1 Tableau associatif comme type de données

Nous sommes familiarisés avec les listes, les tableaux, les dictionnaires de Python et nous avons également défini nos propres structures pour représenter/simuler des piles, des files d'attente, des graphes comme nous le ferons encore avec les arbres au chapitre 14. Ces classes implémentent plus ou moins fidèlement les types fondamentaux de l'informatique comme ceux dont nous proposons ici les définitions.



Définition 12.1 liste chaînée

Une **liste chaînée** est un conteneur, éventuellement vide, dont chaque élément ou cellule contient une donnée et l'adresse (mémoire) de la cellule suivante.

Cette structure supporte les opérations minimales suivantes :

- création d'une liste vide ;
- ajout d'un élément dans la liste ;

- recherche d'un élément dans la liste ;
- suppression d'un élément dans la liste ;
- insertion d'un élément dans une position donnée ;
- accès à la cellule de tête ;
- accès au successeur d'une cellule (on note Nil l'absence d'un successeur).

Questions : Avec Python, comme dans beaucoup de langages de haut-niveau, on ne peut accéder aux adresses mémoires. En conséquence, une opération simple décrite ci-dessus n'est pas immédiate. Laquelle ?

Quelle opération fondamentale permise par les listes de Python ne figure pas ici ?

Rappeler l'instruction ou la méthode qui permet de réaliser chacune de ces opérations avec les listes de Python lorsque cela est possible.

Réponses :

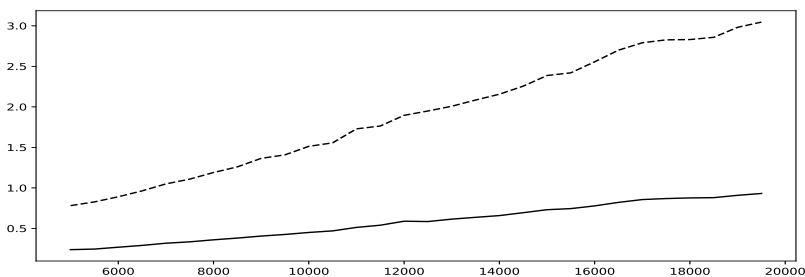
Le cas échéant, on pourra se reporter au tableau de la page 28 pour les méthodes de la classe list. Seul l'accès au successeur d'un élément n'est pas implémenté. D'autre part, l'accès direct à l'élément d'indice i est permis avec les listes de Python (on écrit `L[i]` pour l'appeler), mais n'apparaît pas dans la spécification que nous avons donnée : il faudrait ici partir de la cellule de tête, puis réaliser i fois l'opération successeur pour y accéder.

Définition 12.2 tableau

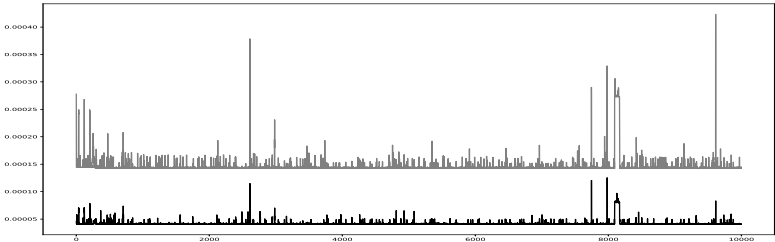
Une structure de **tableau** est un conteneur possédant un nombre fixe d'éléments de même type.

- le nombre et le type des éléments sont définis à la création du tableau (un tableau est donc immuable contrairement à une liste) ;
- chaque composant du tableau est directement accessible en un temps $O(1)$, ce qui suppose que les données soient stockées dans des emplacements mémoire contigus et que l'élément d'adresse $T[i]$ soit directement accessible.

Questions : Quelles sont les opérations sur une liste chaînée qui n'ont pas d'équivalent dans un tableau ? Les tableaux de numpy (numpy.ndarray) correspondent-ils à la spécification ?



Temps cumulés de 1000 instructions `L[i]`, `T[i]` dans des listes (trait continu) et tableaux (- -) dont les tailles sont en abscisses (script en annexe).



Temps cumulés de 1000 instructions $L[i]$, $T[i]$ dans une liste (en noir) et un tableau (en gris) de taille $\text{len}(L) = \text{len}(T) = 10^4$. En abscisses les indices (script en annexe).

Réponses :

L'ajout d'un élément (`L.append(x)`), comme l'insertion (`L.insert(0, x)`), n'a pas de sens dans un tableau immuable (dont la taille et le type des éléments ne peuvent être modifiés). De la même façon l'accès à une cellule ne donne pas accès au successeur. Il est difficile de répondre précisément en ce qui concerne le stockage mémoire sans consulter la documentation de référence. Par contre le graphique de la seconde figure (tracé grisé) suggère que le temps d'accès moyen dans un tableau de taille donnée de Numpy est en $O(1)$ en moyenne. Ce qui est conforme à la spécification donnée. En ce qui concerne les listes, cela n'est pas cohérent avec la spécification des listes chaînées où l'on accéderait à un élément en partant du premier terme et en se déplaçant de successeur en successeur. Comme souvent dans un langage de haut niveau générique, l'implémentation effective est plus complexe que la spécification minimale. Remarquons au passage que l'accès à un élément dans une liste est plus rapide que dans les tableaux de Numpy.

Définition 12.3 *tableau associatif*

Un **tableau associatif** ou **dictionnaire** est un conteneur dont les éléments sont des couples clé-valeur et dans lequel :

- l'association clé-valeur est une application ;
- la recherche d'un élément se fait sur la clé (alors que dans un tableau elle ne se fait pas sur l'indice qui joue le rôle de la clé mais sur les éléments associés aux indices) ;

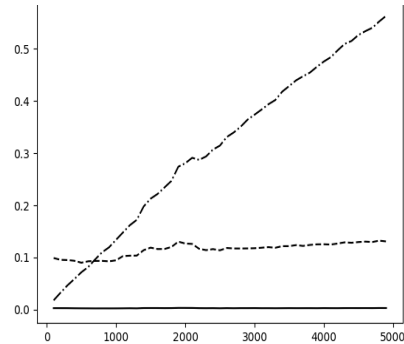
Les tableaux associatifs supportent :

- la création d'une structure vide ;
- l'insertion d'un couple clé-valeur ;
- la suppression d'une clé (et de la valeur associée) ;
- la recherche d'une clé en un temps $O(1)$ en moyenne.

Questions :

Les dictionnaires de Python correspondent-ils à cette spécification ?

Les trois courbes de la figure de droite, déjà présentée page 110 représentent les temps écoulés pour 10 000 recherches (instructions $x \text{ in } L$, $x \text{ in } T$ et $x \text{ in } D$) dans des conteneurs de tailles 100, 200, ..., 4900. Il apparaît clairement que le temps de recherche dans une liste comme fonction de la longueur (---) est linéaire et qu'il est quasi constant dans un dictionnaire (trait continu). La troisième courbe (- -) représente les temps de recherche dans les tableaux de Numpy.



Réponses :

La lecture du graphique montre que les temps moyens de recherche de la présence d'une clé dans le dictionnaire est constant et ne dépend pas de la taille du dictionnaire. Toutes les opérations sont par ailleurs permises par les dictionnaires de Python, par exemple la suppression avec `del D[clé]`. L'implémentation des dictionnaires de Python correspond donc bien à la spécification.

Question : En supposant que les listes de Python sont implémentées comme des listes chaînées, expliquer pourquoi la simple instruction `x in L` a un coup moyen linéaire en la taille de la liste.

Réponse :

La recherche, contrairement à ce qui se passe dans un dictionnaire, suppose un parcours de la liste. Cette opération a un coup moyen en $n/2$ comparaisons. L'accès au successeur se faisant à coup constant.

12.1.2 Tableaux associatifs et tables de hachage

L'implémentation d'une structure de dictionnaire dans laquelle la recherche d'un élément se fait à temps moyen constant procède généralement de la façon suivante :

- On détermine l'ensemble $\mathcal{U}$ des **clés** envisageables (Python par exemple, accepte pour clés les entiers, les flottants, les chaînes de caractères, la plupart des types objets, à l'exception notable des listes).
- On se donne une famille de fonctions $h : \mathcal{U} \rightarrow \mathbb{N}$ (ce sont les **fonctions de hachage**) ; chacune de ces fonctions prend ses valeurs dans un intervalle $\llbracket 0, m_h - 1 \rrbracket$ (ainsi, $h(\mathcal{U}) \subset \llbracket 0, m_h - 1 \rrbracket$).
- Au moment de créer un dictionnaire une fonction de hachage est choisie et un tableau de taille m_h est réservé. Nous parlerons d'**alvéoles** pour désigner les éléments de ce tableau.
- Pour insérer un couple clé-valeur, $h(\text{clé})$ est calculé. C'est un entier $i \in \llbracket 0, m_h - 1 \rrbracket$.

- Une première idée serait de placer le couple dans l'alvéole d'indice i . Mais comme h ne saurait être injective, sauf à choisir $m_h = \mathcal{U}$ ce qui conduirait à

1. Écrire une fonction `chaine_vers_entier(ch)` qui prend une chaîne de caractères en argument et renvoie la valeur $\sum_{k=0}^{n-1} ord(ch[k])256^k$. On pensera à la méthode de Horner pour en réduire la complexité.

Cette fonction renvoie des entiers de grande taille, par exemple :

```
chaine_entiers('exemple de taille modérée') =
639711425003567419095171883946955047862310670182554716698725.
```

2. On souhaite réserver une table de hachage de longueur m la fonction de hachage étant $h : p \rightarrow p \% m$ (reste de la division euclidienne). Comment peut on réécrire la fonction pour qu'elle réalise le pré-traitement et le hachage proposé à moindre coût ?.

Corrigé en 12.1 page 471.

Division

C'est la fonction la plus simple, pour une table avec m alvéoles on choisit

$$h : c \rightarrow c \bmod m = c \% m.$$

La qualité de ce hachage dépend fortement du choix de m si les clés ont une répartition qui n'est pas aléatoire. En pratique on choisit des nombres premiers éloignés des puissances de 2.

Multiplication

Cette méthode consiste à construire des fonctions de hachage de la façon suivante :

- on se donne un réel (ou un flottant) $\theta \in]0, 1[$ et un entier $m > 0$.
- on définit $k : e \in \mathbb{N}^* \rightarrow \lfloor m \times ((e \times \theta) \bmod 1) \rfloor$ (où $x \bmod 1 = x - \lfloor x \rfloor$).

L'exercice qui suit se propose d'en explorer quelques propriétés.

Exercice 12.2 étude empirique pour le choix du paramètre θ

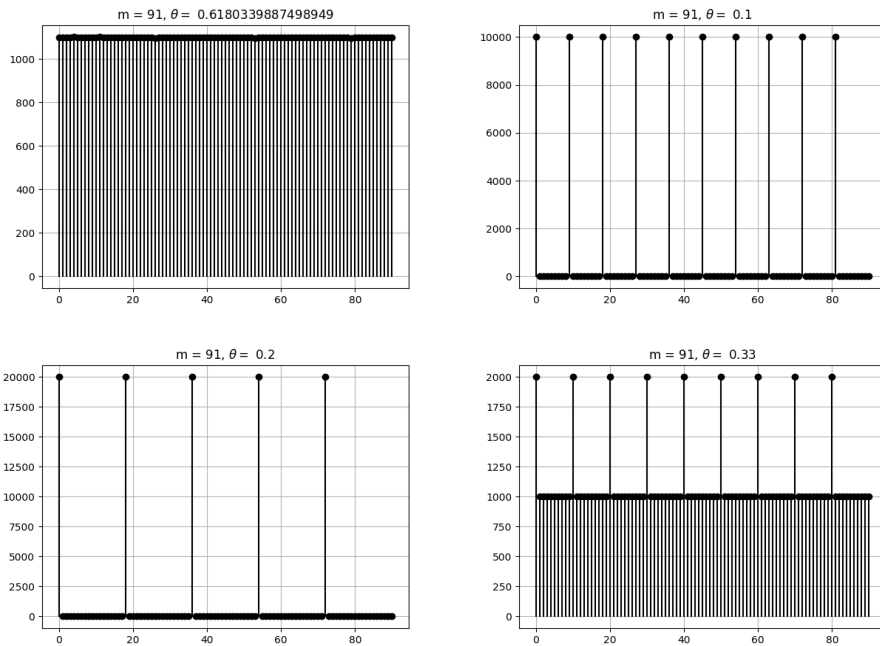
1. Quelle sera la taille d'une table de hachage associée à une fonction multiplicative de cette forme ?
2. On décide de tester la répartition des clés dans la table en fonction de θ .
 - (a) Définir une fonction `hachage(m, theta, c)` qui prend en arguments m un entier strictement positif, $0 < \theta < 1$ un flottant, c entier et qui réalise le hachage de c .
 - (b) Écrire une fonction `test_hachage(m, theta)` qui calcule pour chaque entier i compris entre 0 et $m - 1$ le nombre de valeurs $c \in \llbracket 0, 10^5 \rrbracket$ telles que $h(c) = i$, soit $d_i = |\{c \in \llbracket 0, 10^5 \rrbracket / h(m, \theta, c) = i\}|$. Ces valeurs seront stockées dans un dictionnaire que renverra la fonction.

- (c) Les quatre histogrammes représentent les répartitions obtenues (i en abscisse, et d_i en ordonnée) avec $m = 91$ et $\theta = \frac{\sqrt{5}-1}{2}$, 0.1, 0.2 et 0.33.

Expliquer les résultats lorsque $\theta = 0.1, 0.2$.

- (d) Justifier que $c \rightarrow h(m, a/b, c)$ prend au plus $\max(m, b)$ valeurs. Est-ce cohérent avec ce que l'on observe ?
- (e) **Conseil** : Il pourra être intéressant d'observer la répartition des collisions lorsque $\theta = 0.33$ et pour différentes valeurs de m (plus petites, plus grandes que 100).

Corrigé en 12.2 page 471.



12.2 Compression de texte : LZ78

Cet algorithme de compression de texte (dû à Abraham Lempel et Jacob Ziv, 1978) construit et utilise un dictionnaire pour compresser des données. Il les décompresse à l'aide du dictionnaire inversé.

Algorithme de compression LZ78

On se donne un texte, *texte*, à compresser.

On initialise le code avec *code* = "" (où "" désigne la chaîne vide).

On initialise un dictionnaire : *dc* = {} : 0 ; les clés en seront des chaînes, les valeurs leurs n° d'insertion.

On place une fenêtre de longueur un en position $i = 0$ au dessus de *texte*.

On étend la fenêtre d'observation tant que la chaîne *w* qui y figure est dans le dictionnaire (et tant que l'on n'atteint pas la fin du texte).

On ajoute au code la chaîne 'p*sl' avec *p* tel que *dc*[*p*] = *w* et *s* le caractère suivant.

On insère la nouvelle clé *w* + *s* dans le dictionnaire.

On réitère le procédé à partir de la position *i* qui suit celle de *s*, tant que...

A l'issue du procédé, *code* et *dc* permettent de reconstituer le texte. En pratique, pour des données d'une certaine taille la place prise par *code* et *dc* est nettement inférieure à celle de *texte*.

```
def compressionLZ78(texte):
    '''
    texte: str, une (longue) chaîne à compresser
    '''
    def compresser(code, i):
        assert i < len(texte)
        w = ''
        while i < n and w + texte[i] in dc:
            w += texte[i]
            i += 1
        if i < n:
            ???
        else:
            pass
        return code, i+1

    assert '*' not in texte and '|' not in texte
    n = len(texte)
    i, code, dc = 0, '', {'': 0}

    while i < n:
        code, i = compresser(code, i)
    return code, dc
```

Exercice 12.3 mise en œuvre de LZ78

1. Avec *texte* = 'Les tontons flingueurs ont flingué mes moutons.', l'algorithme construit le code :
`'0*L|0*e|0*s|0*|0*t|0*o|0*n|5*o|7*s|4*f|0*|l|0*i|7*g|0*u|2*u|0*r|3*|6*n|5*|...'`
 Pour chaque ajout d'un segment '*p*s|*' à *code*, il y a un ajout simultané dans le dictionnaire. Préciser les 12 premiers ajouts. Par quelle chaîne faudrait-il remplacer le segment *7*g|* pour reconstituer *texte* ?
2. Dans le schéma d'implémentation de l'algorithme qui est proposé, compléter la sous-fonction `compresser(code, i)` qui gère la fenêtre glissante.
3. Écrire une fonction `decompressionLZ78(code, dc)` qui prend en arguments un code de compression, *code*, et un dictionnaire *dc* renvoyés par `compressionLZ78(texte)` et reconstitue et renvoie le texte d'origine.

Corrigé en 12.3 page 472.

12.3 Corrigés des exercices**Corrigé de l'exercice n° 12.1**

1. Voir ci-dessous.
2. On écrira la ligne `r = (256*r + ord(ch[n-i])) % m` dans le code précédent (ou `% m` à la fin de la ligne d'affectation). Cela permet de ne jamais calculer avec des nombres supérieurs ou égaux à $256 \times m$ alors que $0 \leq r < m$.
 Par exemple :

```
chaine_entiers_modulo('exemple ... modérée', 1023) = 255.
```

```
def chaine_ vers_entier(ch):
    n, r = len(ch), 0
    for i in range(1, n+1):
        r = 256*r + ord(ch[n-i])
    return r
```

Corrigé de l'exercice n° 12.2

1. Il est clair que $h(e) \in \llbracket 0, m-1 \rrbracket$. On pourra donc réserver un tableau de longueur *m*.
2. a & b. On écrit la fonction sans problème (pour les lignes de code concernant les figures, consulter le site associé à cet ouvrage).

```

hachage = lambda m, theta, c : np.floor(m*(theta*c % 1))

def test_hachage(m, theta):
    d = { i : 0 for i in range(0,m+1) }
    for x in range(0, 10**5):
        h = hachage(m, theta, x)
        d[h] += 1
    return d

```

c. On a ici $m = 91$.

- Cas où $\theta = 1/10$. En posant $c = 10q + r$ avec $0 \leq r < 10$, il vient

$$h(c) = \lfloor 91 \times r/10 \rfloor \in \{0, 8, 18, 27, 36, 40, 54, 63, 72, 81\}.$$

La fonction prend donc 10 valeurs comme on peut le remarquer sur le graphique.

- Le cas $\theta = 1/5$ se traite de la même façon.

d. On note $\mu(x) = x - \lfloor x \rfloor$, la mantisse de x . Soit $c = bq + r$ avec $0 \leq r < b$, il vient :

$$\mu\left(\frac{a}{b}c\right) = \mu\left(\frac{a}{b}r\right) \in [0, 1[.$$

Ce qui nous fait au plus b valeurs distinctes. Et comme par ailleurs

$$h(m, a/b, c) = \lfloor m \times \mu\left(\frac{a}{b}r\right) \rfloor,$$

est un entier compris entre 0 et $m - 1$, il prend au plus $\max(m, b)$ valeurs.

e. Une exploration avec différentes valeurs de m et de θ semble montrer que les phénomènes de collisions dépendent de la valeur de θ . Les cas à éviter sont les valeurs $\theta = \frac{p}{q}$ avec q trop petit. Dans tous les cas le nombre d'or semble fournir une répartition quasi-uniforme.

Corrigé de l'exercice n° 12.3

1.

code	ajout	code	ajout	code	ajout
0*Ll	dc[1] = L	0*ol	dc[6] = o	0*ll	dc[11] = l
0*el	dc[2] = e	0*nl	dc[7] = n	0*il	dc[12] = i
0*sl	dc[3] = s	5*ol	dc[8] = to	7*gl	dc[13] = ng
0*	dc[4] =	7*sl	dc[9] = ns	0*ul	dc[14] = u
0*tl	dc[5] = t	4*fl	dc[10] = f	...	...

Dans `dc[10] = f`, que faut-il lire en fait qui n'est pas visible ? (Rép : espace + 'f').

2. On complète la sous-procédure :

```
def compresser(code, i):
    assert i < len(texte)
    w = ''
    while i < n and w + texte[i] in dc:
        w += texte[i]
        i += 1
    if i < n:
        # w est dans le dictionnaire
        # w1 = w + texte[i] est ajouté
        p = len(dc)
        c = texte[i]
        w1 = w + c
        dc[w1] = p
        code += '%s*%s|'%(dc[w], c)
    return code, i+1
```

3. Pour reconstruire le code, il n'y a pas de difficulté après avoir inversé le dictionnaire.

```
def decompressionLZ78(code, dc):
    """
    code: str, code obtenu par LZ78;
    dc : dictionnaire associé
    """
    # inversion du dictionnaire
    dci = { v: k for k,v in dc.items() }
    texte = ''
    if code[-1] == '|':
        code = code[0:-1]
    C = code.split('|')
    for elt in C:
        E = elt.split('*')
        w = dci[int(E[0].strip())]
        s = E[1]
        texte += w + s
    return texte
```


12.4 Annexe

```
def comparaisons_acces_moyen():
    '''
    Temps d'accès moyens à L[i] et T[i] en fonction
    de la taille du conteneur.
    '''
    X = list(range(5000, 20000, 500))
    TL = np.zeros(len(X))
    TT = np.zeros(len(X))

    for i, n in enumerate(X):
        # on construit les conteneurs de taille n
        L = []
        T = np.zeros(n)

        for j in range(0,n):
            v = random.randint(0, 10**4)
            L.append(v)
            T[i] = v
        print( n, len(L), len(T))

    # on réalise 1000 appels de chaque indice
    for k in range(0,10**3):
        t0 = time.time()
        for j in range(0,n-1):
            L[j]
        t1 = time.time()
        for j in range(0,n-1):
            T[j]
        t2 = time.time()
        TL[i] += t1-t0
        TT[i] += t2-t1

    pl.plot(X, TL, color='black')
    pl.plot(X, TT, "--", color='black')
    pl.show()
```

```
def comparaisons_acces(N):  
    '''  
    On veut calculer les temps d'accès à T[i], L[i]  
    en fonction de i, à taille constante N.  
    '''  
    X = list(range(0, N))  
    TL = np.zeros(len(X))  
    TT = np.zeros(len(X))  
    # on remplit les conteneurs de taille n  
    L = []  
    T = np.zeros(N)  
  
    for i in range(0,N):  
        v = random.randint(0, 10**4)  
        L.append(v)  
        T[i] = v  
  
    # on réalise 1000 appels pour chaque indice i  
    for i in range(0,N):  
        t0 = time.time()  
        for j in range(0, 1000):  
            L[i]  
        t1 = time.time()  
        TL[i] += t1-t0  
  
        t0 = time.time()  
        for j in range(0, 1000):  
            T[i]  
        t1 = time.time()  
        TT[i] += t1-t0  
  
    pl.plot(X, TL, color = 'black')  
    pl.plot(X, TT, color = 'gray')  
    pl.show()
```


Programmation dynamique

Le terme *programmation dynamique* désigne une façon de traiter des problèmes d'optimisation combinatoire qui peuvent se ramener à la résolution de sous-problèmes (et en programmant de façon tantôt itérative et ascendante, tantôt récursive). Nous revisitons ainsi une problématique déjà abordée avec les algorithmes gloutons. Ce sera également l'occasion de revenir sur certaines notions abordées avec l'étude de la récursivité¹.

13.1 Premiers exemples

13.1.1 Plus longue sous-suite commune

Le problème que l'on présente ici a des applications importantes dans la comparaison de textes, la recherche de plagats, la comparaison de génomes.

Définition 13.1 sous-suite

Soit $S = (s_0, s_1, \dots, s_{n-1})$ une suite (de caractères, de mots etc...). Une **sous-suite** de S (ou une suite extraite selon une terminologie fréquente dans les cours de maths) est une suite $(z_0, z_1, \dots, z_{p-1})$ (éventuellement vide) de termes de S construite de la façon suivante :

- on se donne une suite **strictement croissante** d'indices $0 \leq i_0 < i_1 < \dots < i_{p-1} \leq n-1$;

- on pose $z_k = s_{i_k}$ pour $0 \leq k \leq p-1$.

En d'autres termes, $Z = (z_k)_k$ est obtenue en prélevant dans l'ordre certains éléments de S .

Formulation du problème :

Soient $S = (s_i)_{0 \leq i < m}$ et $T = (t_j)_{0 \leq j < n}$ deux suites. On souhaite déterminer une suite Z qui soit à la fois une sous-suite de S et une sous-suite de T et de longueur

1. Nous conseillons à ce propos de revoir au moins un des exercices proposés page 156.

maximale. On dira d'une telle sous-suite que c'est une **plus longue sous-suite commune** à S et T (on note PLSC à S et T).

Formalisation du problème :

Soient $S = (s_i)_{0 \leq i < m}$ et $T = (t_j)_{0 \leq j < n}$ deux suites.

L'ensemble des $\mathcal{S}$ des **solutions réalisables** pour la recherche de PLSC à S et T ² est l'ensemble des suites $Z = (z_0, \dots, z_{p-1})$ pour lesquelles il existe un couple $(I, J) \subset \llbracket 0, m-1 \rrbracket \times \llbracket 0, n-1 \rrbracket$ tel que :

- $|I| = |J| = p$;

- en numérotant dans l'ordre croissant les éléments de I et de J : $z_k = s_{i_k} = t_{j_k}$, pour $0 \leq k < p$.

Parmi les solutions réalisables, une **solution optimale** est un couple (I, J) pour lequel p est maximal.

Illustration : On considère $S = \text{'formulation'}$ et $T = \text{'formalisation'}$.

Une solution optimale est donnée avec $p = 10$, $I_{10} = (0, 1, 2, 3, 5, 6, 7, 8, 9, 10)$ et $J_{10} = (0, 1, 2, 3, 5, 8, 9, 10, 11, 12)$. La sous-suite commune correspondante est : $Z = \text{'formulation'}$.

La recherche en force brute d'une sous-suite commune à S et à T pose rapidement un problème de complexité. Il y a, en effet, 2^m sous-suites possibles dans S (autant que de parties dans $\llbracket 0, m-1 \rrbracket$). Nous allons voir qu'une propriété remarquable de ce problème nous permet d'en réduire la complexité.

Sous-structures optimales et plus longues sous-suites communes

Reprenons $S = (s_i)_{0 \leq i < m}$ et $T = (t_j)_{0 \leq j < n}$ nos deux suites génériques et considérons Z une plus longue sous-suite commune à S et T . Notons $p = \text{len}(Z)$.

Théorème 13.1 *Deux cas se présentent :*

1. Soit $s_{m-1} = t_{n-1}$ et dans ce cas : $z_p = s_{m-1} = t_{n-1}$ et $Z[0 : p-1] = (z_0, \dots, z_{p-2})$ est une PLSC à $S[0 : m-1]$ et $T[0 : n-1]$ (attention : notations Python).
2. Soit $s_{m-1} \neq t_{n-1}$ et dans ce cas :
 - (a) si $z_p \neq s_{m-1}$, alors Z est une PLSC à $S[0 : m-1]$ et T ;
 - (b) si $z_p \neq t_{n-1}$, alors Z est une PLSC à S et $T[0 : n-1]$.
 (et en particulier, si $z_p \neq s_{m-1}, t_{n-1}$, alors Z est une PLSC à $S[0 : m-1]$ et $T[0 : n-1]$).

Avant de justifier ce résultat, remarquons qu'il permet de ramener la recherche d'une PLSC à S et T à celles de PLSC dans des sous-chaînes de S et T . C'est dans ce sens que l'on dit que le problème a une propriété de **sous-structure optimale** : une

2. On reprend ici le vocabulaire du chapitre 5.

solution optimale d'une certaine taille se construit à partir de solutions **également optimales** pour un problème de plus petite taille.

Démonstration :

1. Supposons que $s_{m-1} = t_{n-1}$ et considérons $Z = (z_0, \dots, z_{p-1})$ une PLSC à S et T . Si $z_{p-1} \neq s_{m-1}$, alors $Z' = (z_0, \dots, z_{p-1}, s_{m-1})$ est encore une sous-suite de S et de T . Comme elle est de longueur strictement supérieure à celle de Z , nous avons une contradiction.

Donc $z_p = s_{m-1} = t_{n-1}$. $Z[0 : p-1] = (z_0, \dots, z_{p-2})$ est une sous-suite formée d'éléments de $S[0 : m-1]$ et $T[0 : n-1]$. Elle est maximale. Car sinon il existerait une sous-suite commune à $S[0 : m-1]$ et $T[0 : n-1]$ de longueur $q > p-1$. Mais en ajoutant le terme $s_{m-1} = t_{n-1}$ à cette sous-suite nous aurions une sous-suite commune à S et T de longueur $p+1$. Contradiction à nouveau.

2. Supposons maintenant que $s_{m-1} \neq t_{n-1}$.

(a) Si $z_p \neq s_{m-1}$ et alors Z est une sous-suite de longueur p de $S[0 : m-1]$ et de T ; si elle n'est pas maximale comme sous-suite de $S[0 : m-1]$ et de T , c'est qu'il existe une sous-suite commune à $S[0 : m-1]$ et T de longueur strictement supérieure à p . Mais cette sous-suite serait aussi une sous-suite de S et T . Contradiction encore !

(b) Si $z_p \neq t_{n-1}$ par symétrie, Z est une PLSC à S et $T[0 : n-1]$.

□

Les exercices qui suivent montrent comment exploiter cette propriété pour déterminer la **longueur** d'une PLSC. Le premier propose un algorithme récursif, le second donne une version itérative. Dans les deux cas, on calcule les longueurs $c(p, q)$ d'une PLSC à $S[0 : p]$ et $T[0 : q]$ tout en conservant des informations qui permettront de reconstituer une PLSC à S et T comme nous le verrons dans l'exercice 13.3.

Exercice 13.1 calcul de la longueur d'une PLSC (version récursive)

On considère deux chaînes S et T de longueurs m et n , on note $c(m, n)$ la longueur d'une PLSC de S et de T , et d'une façon générale, lorsque $0 \leq p < m$ et $0 \leq q < n$, $c(p, q)$ désignera la longueur d'une PLSC de $S[0 : p]$ et $T[0 : q]$.

1. On suppose que $p = 0$ ou $q = 0$. Que vaut $c(p, q)$?
2. La démonstration du théorème (13.1) permet d'exprimer $c(p, q)$ en fonction de $c(p', q')$ avec $p' + q' < p + q$. Compléter ce qui suit en fonction de l'énoncé de ce théorème :

$$\begin{cases} \text{Si } s_{p-1} = t_{p-1}, \text{ alors, } c(p, q) = ??? \\ \text{Si } s_{p-1} \neq t_{p-1}, \text{ alors, } c(p, q) = \max(???, ???) \end{cases}$$

3. On suppose que $\text{PLSC0}(S, T)$ est une fonction récursive naïve (en particulier écrite sans effort de mémoïsation) qui renvoie la longueur d'une PLSC à S et T en utilisant les relations de récurrence établies dans la question précédente.

- (a) Donner un variant pour les appels d'une telle fonction. Justifier qu'elle termine.
 - (b) Dessiner l'arbre des appels récursifs lorsque $S = \text{'cal'}$ et $T = \text{'col'}$.
4. Écrire une fonction récursive $\text{PLSC1}(S, T, D)$ dont l'appel principal se fera avec $D = \{\}$ (dictionnaire vide) et qui construit et renvoie le dictionnaire (de dictionnaires) avec $D[p][q] = c(p, q)$ pour $0 \leq p < m, 0 \leq q < n$. Cette fonction commencera par :

```
def PLSC1(S, T, D):
    m, n = len(S), len(T)
    if m not in D:
        D[m] = {}
    if m == 0 or n == 0:
        D[m][n] = 0
        return D[m][n][0]
    elif S[-1] == T[-1]:
        if m-1 not in D or n-1 not in D[m-1]:
            r = PLSC1(S[0:m-1], T[0:n-1], D)
        else:
            r = D[m-1][n-1][0]
        D[m][n] = r + 1
        return D[m][n][0]
    else:
        ...
```

Modifier $\text{PLSC1}(S, T, D)$ pour obtenir $\text{PLSC2}(S, T, D)$ qui remplit le dictionnaire avec :

$D[p][q] = c(p, q), (-1, -1)$, dans le cas de figure (1) du théorème,

$D[p][q] = c(p, q), (-1, 0)$ ou $D[p][q] = c(p, q), (0, -1)$ dans les cas de figure (2.a) ou (2.b).

Ces informations supplémentaires permettront de reconstituer une PLSC (voir pour cela l'exercice 13.3).

5. Donner une majoration de la complexité en mémoire et en temps de cette fonction.

Corrigé en 13.1 page 493.

L'exercice 13.2 propose une méthode itérative (et ascendante) pour résoudre le même problème de calcul de la longueur d'une PLSC. Les deux premières questions sont communes avec l'exercice précède.

Exercice 13.2 calcul de la longueur d'une PLSC (version itérative)

On considère deux chaînes S et T de longueurs m et n , on note $c(m, n)$ la longueur d'une PLSC de S et de T . D'une façon générale, lorsque $0 \leq p < m$ et $0 \leq q < n$, $c(p, q)$ désignera la longueur d'une PLSC de $S[0 : p]$ et $T[0 : q]$.

1. On suppose que $p = 0$ ou $q = 0$. Que vaut $c(p, q)$?
2. La démonstration du théorème (13.1) permet d'exprimer $c(p, q)$ en fonction de $c(p', q')$ avec $p' + q' < p + q$. Compléter ce qui suit en fonction de l'énoncé de ce théorème :

$$\begin{cases} \text{Si } s_{p-1} = t_{p-1}, \text{ alors, } c(p, q) = ??? \\ \text{Si } s_{p-1} \neq t_{p-1}, \text{ alors, } c(p, q) = \max(???, ???) \end{cases}$$

3. Écrire une fonction itérative PLSC1(S,T) qui renvoie un dictionnaire D tel que :
 $D[p][q] = c(p, q), (-1, -1)$, dans le cas de figure (1) du théorème,
 $D[p][q] = c(p, q), (-1, 0)$ ou $D[p][q] = c(p, q), (0, -1)$ dans les cas de figure (2.a) ou (2.b).

Cette fonction pourrait avoir la structure :

```
def PLSC1(S, T):
    m, n = len(S), len(T)
    D = { p: {} for p in range(0, m+1)}

    for q in range(0, n+1):
        D[0][q] = 0, (0, 0)
    for p in range(0, m+1):
        D[p][0] = 0, (0, 0)

    for p in range(1, m+1):
        for q in range(1, n+1):
            ???

    return D
```

4. Majorer les complexités en mémoire et en temps de cette fonction.

Corrigé en 13.2 page 494.

Exercice 13.3 reconstitution d'une PLSC

On se propose de déterminer explicitement une PLSC à S et T à partir des informations contenues dans les dictionnaires construits par dans la procédure PLSC2(S,T, D) de l'exercice 13.1 ou par PLSC1(S,T) de l'exercice 13.2. On rappelle que, dans les deux cas, D est un dictionnaire tel que $D[p][q] = c(p, q), (\alpha, \beta)$ où :

$c(p, q)$ est la longueur d'une PLSC à $S[0 : p], T[0 : q]$;
 $(\alpha, \beta) = (-1, 1)$ si $S[p-1] = T[q-1]$;
 $(\alpha, \beta) = (-1, 0)$ si $S[p-1] \neq T[q-1]$ et $c(p-1, q) > c(p, q-1)$;
 $(\alpha, \beta) = (0, -1)$ si $S[p-1] \neq T[q-1]$ et $c(p-1, q) \leq c(p, q-1)$;

Compléter le schéma d'une fonction PLSC(S,T) qui prend deux chaînes en arguments et en renvoie **une** PLSC. Cette fonction appelle PLSC2 ou PLSC1 qui construit ou retourne D, elle dispose d'une sous-procédure récursive reconstruirePLSC(p,q) qui renvoie une PLSC à $S[0 : p]$ et $T[0, q]$.

```
def PLSC(S,T):

    def reconstruirePLSC(p, q):
        if p == 0 or n == 0:
            return ''
        else:
            ???

    #-----
    m, n = len(S), len(T)
    D = {}
    PLSC2(S, T, D)
    return reconstruirePLSC(m, n)
```

Corrigé en 13.3 page 495.

13.1.2 Produit de matrices, parenthésages optimaux

Nous savons que la multiplication matricielle est associative c'est à dire que, lorsque cela a un sens, les produits $A \times (B \times C)$ et $(A \times B) \times C$ sont égaux ce qui fait qu'on note $A \times B \times C$ ou ABC sans préciser la place des parenthèses lorsqu'on ne s'intéresse qu'au produit lui-même.

Par contre, les choses changent quand il s'agit de réaliser le calcul. Lorsque $A \in \mathcal{M}_{p,q}$, $B \in \mathcal{M}_{q,r}$, $C \in \mathcal{M}_{r,s}$, on a toujours $A(BC) = (AB)C \in \mathcal{M}_{p,s}$ mais le premier calcul aura coûté $pqs + qrs = qs(p+r)$ multiplications (de réels, complexes ou flottants) alors que le second en aura coûté $pqr + prs = pr(q+s)$.

$$\begin{bmatrix} * & * & * & * & * & * \\ * & * & * & * & * & * \\ * & * & * & * & * & * \\ * & * & * & * & * & * \\ * & * & * & * & * & * \end{bmatrix} \begin{bmatrix} * & * & * \\ * & * & * \\ * & * & * \\ * & * & * \\ * & * & * \end{bmatrix} \begin{bmatrix} * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \end{bmatrix} = \begin{bmatrix} * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \end{bmatrix}$$

Il y aura dans l'exemple illustré $qs(p+r) = 150$ ou $pr(q+s) = 66$ multiplications selon le parenthésage.

Dans certaines applications, où l'on a à effectuer des produits matriciels en cascade avec des matrices de grande tailles, il y a un grand intérêt à planifier l'organisation des calculs et donc à choisir un parenthésage optimal.

L'exercice 13.4 montre que le nombre de parenthésages possibles pour une suite de n matrices ou de termes quelconques est $c_n = \frac{1}{n+1} \binom{2n}{n} \underset{n \rightarrow +\infty}{\sim} \frac{4^n}{\sqrt{\pi n^{3/2}}}$ ce qui dissuade de tenter une approche en force brute. Il est conseillé de traiter les deux premières questions avant d'aller plus loin.

Exercice 13.4 parenthésages et nombres de Catalan

On cherche à donner une expression du nombre c_n de parenthésages **minimaux** autour d'une suite de $n+1$ termes $A_0, A_1, A_2, \dots, A_n$ qui donnent des expressions distinctes (l'opération n'est pas supposée associative).

Comme $A_0 = (A_0)$ et $A_0 A_1 = (A_0 A_1) = (A_0)(A_1) = ((A_0)(A_1))$, on posera $c_0 = c_1 = 1$, il n'y a dans chaque cas qu'un parenthésage minimal, il est vide.

n	c_{n-1}	parenthésages minimaux
$n = 1$	$c_0 = 1$	A_0
$n = 2$	$c_1 = 1$	$A_0 A_1$
$n = 3$	$c_2 = 2$	$A_0(A_1 A_2), (A_0 A_1)A_2$
$n = 4$	$c_3 = 5$	$(A_0(A_1 A_2))A_3, ((A_0 A_1)A_2)A_3,$ $A_0((A_1 A_2)A_3), A_0(A_1(A_2 A_3)), ???$

1. Il manque une expression dans le tableau. Quelle est-elle ?

2. Justifier la relation de récurrence $c_n = \sum_{i=0}^{n-1} c_i c_{n-1-i}$.

3. A aborder après le cours de maths sur les séries entières :

(a) On pose $G(z) = \sum_{k=0}^{\infty} c_k z^k$. Montrer que $G^2(z) = \sum_{k=0}^{\infty} c_{k+1} z^k$.

(b) En déduire que $zG^2(z) - G(z) + 1 = 0$ puis que $G(z) = \frac{1 - \sqrt{1 - 4z}}{2z}$.

(c) Justifier la formule : $c_n = \frac{1}{n+1} \binom{2n}{n}$.

(d) Justifier l'équivalent donné (remplacer les factorielles avec la formule de Stirling : $n! \sim \left(\frac{n}{e}\right)^n \sqrt{2\pi n}$).

Ce sont là les **nombres de Catalan** qui interviennent dans de nombreux problèmes de dénombrements que l'on rencontre en informatique.

Corrigé en 13.4 page 496.

Sous-structures optimales dans les parenthésages optimaux

Considérons donc une suite de n matrices $(A_0, A_1, \dots, A_{n-1})$ telle que les produits $A_i A_{i+1}$ soient définis. Pour $0 \leq i \leq n-1$, on note ℓ_i le nombre de lignes de A_i et on pose $\ell_n = c_{n-1}$, nombre de colonnes de A_{n-1} ; on a donc $A_i \in \mathcal{M}_{\ell_i, \ell_{i+1}}$.

Nous voulons savoir quels parenthésages permettront d'optimiser le nombre de multiplications pour calculer le produit des A_i . Imaginons qu'un tel parenthésage optimal conduise à effectuer comme dernière opération le produit de deux matrices

$(A_0 \dots A_k) \times (A_{k+1} \dots A_{n-1})$. Ce dernier produit matriciel demandera à lui seul $\ell_0 \times \ell_{k+1} \times \ell_n$ multiplications. Le nombre total des multiplications sera donc égal à la somme de trois termes :

- nombre de multiplications pour le calcul de $(A_0 \dots A_k)$;
- nombre de multiplications pour le calcul de $(A_{k+1} \dots A_{n-1})$;
- $\ell_0 \times \ell_{k+1} \times \ell_n$.

Propriété : Si cette somme de trois termes est optimale, le premier et le second terme sont aussi obtenus avec des parenthésages optimaux.

En effet, si le parenthésage pour calculer $(A_0 \dots A_k)$ n'est pas optimal, on peut le remplacer par un meilleur choix ce qui n'a pas d'incidence sur les façons de calculer $(A_{k+1} \dots A_{n-1})$ ou le dernier produit. On améliore ainsi le score global ce qui contredit le fait que notre parenthésage pour $A_0 \dots A_{n-1}$ est optimal.

Cette propriété est fondamentale, elle va nous permettre d'élaborer une stratégie pour déterminer les parenthésages optimaux.

Notons $m(i, j)$ le **nombre minimal de multiplications** pour calculer $A_i \times \dots \times A_j$ lorsque $0 \leq i < j \leq n-1$, et posons $m(i, i) = 0$. Pour calculer le produit $A_i \dots A_j$ on peut placer une $)$ après A_k pour $i \leq k < j$ et calculer séparément les deux produits $(A_0 \dots A_k)$ et $(A_{k+1} \dots A_j)$. Ce choix étant fait, le nombre minimal de multiplications est $\boxed{m(i, k) + m(k+1, j) + \ell_i \ell_k \ell_{j+1}}$.
 $m(i, j)$ est donc la valeur minimale parmi les $m(i, k) + m(k+1, j) + \ell_i \ell_k \ell_{j+1}$.

L'exercice qui suit propose la mise en place d'un algorithme qui tire profit de cette propriété pour calculer de **façon ascendante** les $m(i, j)$ et relever les indices k pour lesquels les minima de $m(i, k) + m(k+1, j) + \ell_i \ell_k \ell_{j+1}$ sont atteints. Une programmation **réursive avec mémoïsation** est présentée dans l'exercice 13.6.

Exercice 13.5 parenthésage optimal, version ascendante

Soit $L = [\ell_0, \ell_1, \dots, \ell_n]$ une suite de $n+1$ entiers que l'on interprète comme les nombres de lignes et colonnes de n matrices $(A_0, A_1, \dots, A_{n-1})$ avec $A_i \in \mathcal{M}_{\ell_i, \ell_{i+1}}$. On se propose d'écrire un algorithme qui calcule les $m(i, j)$ et les stocke dans un tableau matriciel : $M = \text{np.zeros}((n, n), \text{dtype} = \text{int})$. Cet algorithme remplit parallèlement un tableau S de même taille que le précédent, initialisé avec $S = -\text{np.ones}((n, n), \text{dtype} = \text{int})$ dans lequel $S[i, j]$ est un entier k tel que $m(i, j) = m(i, k) + m(k+1, j) + \ell_i \ell_k \ell_{j+1}$ lorsque $i \leq j$.

1. On suppose les valeurs $m(i, j)$ calculées et stockées. Que dire de M comme matrice ? Où trouvera-t-on le résultat $m(0, n)$ correspondant à l'optimum cherché pour le calcul du produit $A_0 \times A_1 \times \dots \times A_{n-1}$?
2. Pour d donné, à quoi les termes $M[i, i+d]$ correspondent-ils ?
3. **Jeu de test :** On reprend l'exemple du produit de 3 matrices de la page 482. On aura dans ce cas $L = [2, 6, 3, 5]$. Remplir les tableaux S et M en expliquant

dans quel ordre et comment ils peuvent être remplis.

4. Écrire une fonction `choix_parenthesage(L)` qui remplit et renvoie le tuple (S, M) . Donner la complexité en place et en temps de cette fonction.

Conseil :

Remplir dans l'ordre les sur-diagonales $(M[i, i + 1])_i, \dots, M[i, i + d])_i$

5. On écrit un algorithme de calcul du produit :

```
LA = [A0, A1, ...] # liste de n matrices
P = LA[0]
for i in range(1, len(LA)):
    P = P*LA[i]
```

Quelle est sa complexité en nombre de multiplications de deux nombres ? Écrire une fonction `cout_cascade(L)` qui prend en argument une liste comme dans le préambule et renvoie ce coût.

6. On écrit le script suivant avec les fonctions qui précèdent :

```
>>> L = [1200, 2000, 345, 560, 1000, 489]
>>> S, M = choix_parenthesage(L)
>>> c = cout_cascade(L)
>>> S
[[-1  0  1  1  1]
 [-1 -1  1  1  1]
 [-1 -1 -1  2  3]
 [-1 -1 -1 -1  3]
 [-1 -1 -1 -1 -1]]
>>> c, M[0, len(L)-2]/c
2318640000 0.6005033122865128
```

Placer les parenthèses qui donnent l'optimum pour $A_0 \times A_1 \times A_2 \times A_3 \times A_4$

Corrigé en 13.5 page 497.

Exercice 13.6 *parenthesage optimal, version récursive*

On reprend les notations de l'exercice précédent : $L = [\ell_0, \ell_1, \dots, \ell_n]$ est une suite de $n + 1$ entiers que l'on interprète comme les nombres de lignes et colonnes de n matrices $(A_0, A_1, \dots, A_{n-1})$ avec $A_i \in \mathcal{M}_{\ell_i, \ell_{i+1}}$. On rappelle que le nombre optimal de multiplications pour le calcul du produit matriciel $A_i \dots A_j$ vérifie la relation :

$$m(i, j) = \inf\{m(i, k) + m(k + 1, j) + \ell_i \ell_k \ell_{j+1} / i \leq k < j\}.$$

1. On suppose qu'une fonction récursive `choix_parenthesage_rec(L, i, j)` calcule $m(i, j)$. Que serait l'arbre des appels avec $i = 0, j = 4$, avec une programmation naïve sans utiliser de tableau ou dictionnaire pour stocker les résultats intermédiaires lorsqu'ils sont calculés ?

- Écrire une fonction récursive `choix_parenthesage_rec(L, i, j, D)` qui utilise le dictionnaire de dictionnaires `D` pour mémoriser les résultats.

Conseil : Écrire tout d'abord la fonction naïve de calcul des $m(i, j)$ puis l'enrichir ; le dictionnaire pourra contenir à la fois les $m(i, j)$ et une valeur de k pour laquelle $m(i, k) + m(k + 1, j) + \ell_i \ell_k \ell_{j+1}$ sera minimal et donc égal à $m(i, j)$ ($D[i][j] = m[i, j], k$).

Corrigé en 13.6 page 498.

Exercice 13.7 reconstitution d'un parenthésage optimal

On considère une suite de matrices $(A_0, \dots, A_{n-1})$ dont on veut calculer le produit $A_0 \times \dots \times A_{n-1}$. Les deux exercices qui précèdent permettent de déterminer les nombres de multiplications optimaux, $m(i, j)$ pour les calculs $A_i \dots A_j = A_i \times \dots \times A_j$ et pour chacun d'eux relève la position du dernier parenthésage qui permettrait de l'obtenir.

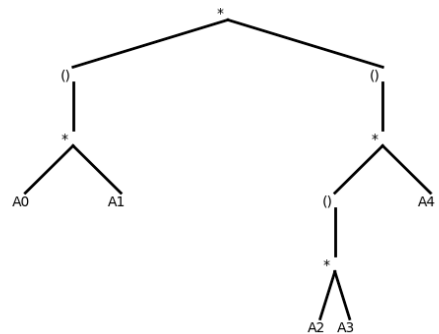
L'objectif est ici de fournir un algorithme qui reconstitue un parenthésage optimal à partir de cette information. La dernière question de l'exercice 13.5 illustre ce problème en proposant une reconstruction papier-crayon.

- On suppose que, dans le tableau S de taille $n \times n$, $S[i, j] = k$ signifie qu'un meilleur parenthésage pour le calcul de $A_i \dots A_j$ est $(A_i \dots A_k)(A_{k+1} \dots A_j)$. Écrire une fonction `parenth_opt(S, i, j)` qui prend un tel tableau en argument, i et j des entiers et renvoie la chaîne représentant l'expression parenthésée correspondante.

Par exemple : `'(A0 A1)((A2 A3)A4)'`

- On se propose de construire l'arbre syntaxique associé à cette expression. Pour cela on utilisera la classe `tree` présentée au chapitre 14, page 535.

Écrire une fonction `arbre_parenth_opt(S, i, j)` susceptible de construire un arbre comme celui de la figure. On « rappelle » que l'instruction pour construire un arbre est `tree(ch, [T1, ...])` où `ch` est une chaîne (l'étiquette du nœud), `[T1, ...]` une liste, éventuellement vide, de sous-arbres.



Corrigé en 13.7 page 500.

13.1.3 Problèmes éligibles à la programmation dynamique

Nous avons avec les exemples qui précèdent abordés des problèmes d'optimisation qu'il nous a été possible de résoudre en les ramenant à des **sous-problèmes de même nature et de tailles strictement inférieures**. Nous avons ainsi défini des algorithmes ascendants, qui construisent les solutions optimales en commençant par les problèmes de petite taille ou, à l'inverse, des algorithmes récursifs. Dans les deux

cas, en faisant usage de dictionnaires (ou de tableaux) pour stocker les résultats partiels nécessaires aux étapes ultérieures et, dans le cas récursif, pour éviter les calculs redondants.

Sous-structure optimale

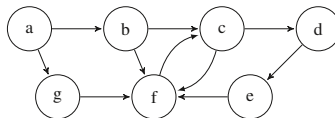
Ces problèmes ont une chose en commun : la solution optimale pour un problème de taille n (par exemple, choisir un parenthésage optimal pour un produit de n matrices) se construit à partir de solutions également optimales pour des problèmes de tailles inférieures. On dit que **les sous-problèmes sont indépendants** : modifier la solution de l'un d'eux n'impose pas de modifier les autres pour conserver la solution globale.

On remarquera que tous les problèmes d'optimisation ne présentent pas cette propriété. Une façon simple d'illustrer cette remarque est de considérer deux problèmes de recherche de chemins dans un graphe orienté $G = (S, A)$:

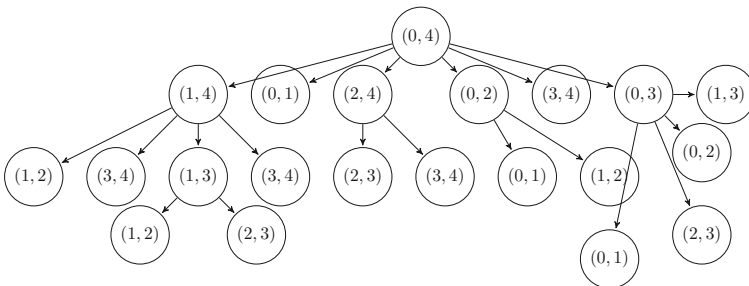
- la recherche d'un chemin avec un **minimum** d'arcs reliant un sommet u à v ;
- la recherche d'un chemin sans boucle avec un **maximum** d'arcs reliant u à v .

Questions :

- Justifier que si un chemin optimal pour le premier de ces problèmes relie u à v en passant par x , alors les sous-chemins reliant u à x et x à v sont optimaux.
- Pour le second problème, donner dans le graphe orienté de la figure un chemin maximal (sans boucle) reliant a à f dont un sous-chemin n'est pas maximal.



Chevauchement de sous-problèmes



Considérons l'arbre des appels récursifs pour une fonction naïve de recherche des coûts optimaux pour la multiplication de $n = 5$ matrices (question 1 de l'exercice 13.5). Il est clair qu'un même sous-problème apparaît dans la résolution de plusieurs problèmes de plus grande taille et que l'on doit s'attendre à une explosion combinatoire. On dit que **les sous-problèmes se chevauchent**. Devant une telle situation, que

nous avons abondamment illustrée dans le chapitre sur la récursivité, une approche descendante ou récursive est rendue efficace par l'utilisation d'un tableau ou de préférence un dictionnaire, qui permet la **mémoïsation** (ou le stockage des résultats déjà calculés).

Diviser (efficacement) pour régner

Dans les algorithmes diviser pour régner bien conçus, les sous-problèmes ne se chevauchent pas. La mémoïsation n'apporte aucun avantage dans un tel cas.

Question : Réétudier les problèmes présentés au chapitre 3 en 3.4 (diviser pour régner). Rencontre-t-on des situations avec chevauchement de sous-problèmes ?

13.2 Autres exemples

13.2.1 Distance d'édition

Définition 13.2 *distance de Levenshtein*

Soient $X = X_0$ et Y deux mots ou chaînes de caractères.

On s'autorise les opérations ou transformations élémentaires $X_p \rightarrow X_{p+1}$ suivantes :

- X_{p+1} est obtenue à partir de X_p par **suppression** d'un caractère ;
- X_{p+1} est obtenue à partir de X_p par **insertion** d'un caractère de Y ;
- X_{p+1} est obtenue à partir de X_p par **remplacement** d'un caractère par un caractère différent provenant de Y .

La **distance de Levenshtein** entre X et Y est égale à la longueur minimale d'une suite d'opérations du type de celles précédemment décrites (on dira de types S , I_Y ou R_Y) qui permet de transformer X en Y . On la note $Lev(X, Y)$.

On admet que toute suite d'opérations de ce type est équivalente à une suite d'opérations de même longueur vérifiant la propriété ★, à savoir : **le caractère modifié à une étape se situe à droite des caractères précédemment supprimés, insérés ou remplacés.**

Exercice 13.8 à propos de la distance de Levenshtein

1. Soient $X = \text{'pirate'}$, $Y = \text{'pilate'}$ et $Z = \text{'pilote'}$.
Donner des suites de transformations de X vers Y , de Y vers Z et enfin de X vers Z .
2. Soient $X = \text{'approximatif'}$ et $Y = \text{'aproksimatifs'}$ deux chaînes de caractères.
Exhiber une suite de transformations de X vers Y puis une suite de transformations de Y vers X de mêmes longueurs.
3. Montrer que s'il existe une suite **minimale** de transformations de types R_Y , I_Y ou S de longueur p qui transforme X en Y , il existe une suite de même longueur qui transforme Y en X . En déduire que $Lev(X, Y) = Lev(Y, X)$.

4. Montrer que la distance de Levenshtein est bien une distance au sens mathématique.

Corrigé en 13.8 page 500.

Calcul effectif de la distance de Levenshtein

On considère deux mots ou chaînes de caractères X et Y de longueurs respectives m et n . On note $d(i, j) = Lev(X[0 : i], Y[0 : j])$. On aura donc $d(m, n) = Lev(X, Y)$. L'exercice qui suit montre comment les propriétés de d permettent de construire un algorithme de calcul de $Lev(X, Y)$.

Exercice 13.9 Vers le calcul de la distance de Levenshtein

On complète les notations de l'encart qui précède en notant $\varepsilon = "$, la chaîne vide (qui est aussi $X[0 : 0]$).

1. Exprimer $d(0, 0)$, $d(0, j)$ et $d(i, 0)$ lorsque $0 \leq i < m$ et $0 \leq j < n$.
2. **Sous-structure optimale :** Une suite de p opérations transforme X en Y . On suppose que c 'est une suite minimale et qu'elle vérifie la propriété ★. Montrer que la suite des $p - 1$ premières opérations transforme soit $X[0 : m - 1]$ en $Y[0 : n - 1]$, soit $X[0 : m]$ en $Y[0 : n - 1]$, soit $X[0 : m - 1]$ en $Y[0 : n]$, et que c 'est une suite minimale pour cette transformation.
3. $d(i, j)$ est le nombre d'opérations dans une suite minimale qui transforme $X[0 : i]$ en $Y[0 : j]$. Montrer que si $X[i - 1] = Y[j - 1]$,

$$d(i, j) = \min(d(i - 1, j - 1), d(i - 1, j) + 1, d(i, j - 1) + 1)$$

et que sinon :

$$d(i, j) = \min(d(i - 1, j - 1) + 1, d(i - 1, j) + 1, d(i, j - 1) + 1)$$

On raisonnera en fonction de l'avant dernier état lors d'une suite de transformations qui transforme $X[0 : i]$ en $Y[0 : j]$.

4. Algorithme de calcul :

Pour calculer $d(m, n) = Lev(X, Y)$ on réserve un tableau de taille $(m + 1) \times (n + 1)$ dans lequel on inscrira $T[0, 0] = d(0, 0) = Lev("", "")$ et $T[i, j] = d(i, j)$. Écrire une fonction `distance_edition(X, Y)` qui construit un tel tableau et renvoie le tuple (d, T) dans lequel $d = Lev(X, Y)$ et T est le tableau des $d(i, j)$.

5. Déterminer le coût mémoire et le coût en temps de cette fonction.

Corrigé en 13.9 page 502.


```
>>> d, T = distance_edition('avion', 'avions'); print(M)
array([[0, 1, 2, 3, 4, 5, 6, 7],
       [1, 0, 1, 2, 3, 4, 5, 6],
       [2, 1, 0, 1, 2, 3, 4, 5],
       [3, 2, 1, 0, 1, 2, 3, 4],
       [4, 3, 2, 1, 1, 1, 2, 3],
       [5, 4, 3, 2, 2, 2, 1, 2]])
```

Ci-dessus le tableau contenant les $d(i, j)$ pour les chaînes 'avion' et 'avions'. $T[5, 7] = 2$ contient la distance entre ces deux mots.

Question : Comment reconstituer les opérations possibles de la dernière étape ?

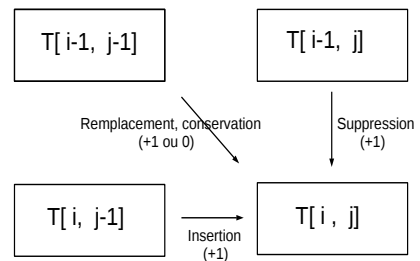
$$T[i-1, j-1] = 2$$

$$T[i-1, j] = 3$$

$$T[i, j-1] = 1$$

$$T[i, j] = 2$$

Réponse : La figure de droite rappelle qu'il y a **au plus 4 façons** d'arriver à un état dans lequel $X[0 : i]$ est transformé en $Y[0 : j]$. Trois ont un coût de +1 et l'une a un coût nul. Dans le cas de figure qui nous intéresse, on voit qu'un chemin minimal ne se termine pas par une suppression de caractère (flèche verticale vers le bas) car le coût serait de $3+1 = 4$. Il reste deux possibilités :



En diagonale, avec un surcoût de 0 : Une suite optimale peut effectivement transformer $X[0 : 4] = \text{'avio'}$ en $Y[0 : 6] = \text{'avions'}$ avec un coût $T[4, 6] = 2$, mais ce ne peut être l'avant dernière étape de notre problème (quelle serait la dernière ?).

Il ne reste plus que le cas d'une insertion, celle du 's', comme dernière étape après que 'avion' ait été transformé en 'aviron' avec un coût de $T[5, 6] = 1$.

13.2.2 Algorithme de Roy-Floyd-Warshall

Nous illustrons ici les méthodes de programmation dynamique avec un algorithme de plus courts chemins dans un graphe orienté et valué.

Mise en place des notations :

Nous noterons $G = (S, A)$ en identifiant les sommets aux entiers $0, \dots, n-1$. Un arc valué sera noté $a = (i, j, w_{i,j})$ et le graphe sera représenté par sa matrice d'adjacence

W définie par
$$\begin{cases} W_{i,j} = +\infty & \text{s'il n'y a pas d'arc entre } i \text{ et } j; \\ W_{i,j} = w_{i,j} & \text{sinon.} \end{cases}$$

On pourra aussi représenter les arcs du graphe par un dictionnaire d'adjacence dont les clés sont les origines des arcs et les valeurs les listes de couples $(j, w_{i,j})$ formés de l'extrémité et du poids de l'arc : $D[i] = [[j_0, w_{i,j_0}], [j_1, w_{i,j_1}], \dots]$.

On note enfin $\mathcal{C}_{i,j}$ l'ensemble des chemins de i à j .

Exercice 13.10 *préalables simples*

1. On considère $G = (S, A)$ un graphe orienté et valué dont certains arcs ont des poids strictement négatifs. Justifier que s'il existe un chemin de poids minimal entre deux sommets quelconques, alors il n'y a pas de cycle de poids strictement négatif dans G .
2. Justifier que si $p = ((i_0, i_1, w_{i_0, i_1}), \dots, (i_{\ell-1}, j, w_{i_{\ell-1}, j}))$ est un chemin de poids minimal entre les sommets i et j , alors pour tout sommet intermédiaire k , par lequel passe le chemin p , les chemins partiels de i à k et de k à j sont eux-mêmes minimaux.

Corrigé en 13.10 page 503.

Une sous-structure optimale pour le problème des plus courts chemins

On peut définir, dans $\mathcal{C}_{i,j}$, une suite $(\mathcal{C}_{i,j,k})_{k \geq 1}$ de sous-ensembles (emboîtés) où chaque $\mathcal{C}_{i,j,k}$ est formé des chemins de i à j dont les sommets intermédiaires, s'ils existent, sont dans $\llbracket 0, k-1 \rrbracket$. On notera $\mathcal{C}_{i,j,0}$ l'ensemble (éventuellement vide) des arcs reliant i à j .

On observe alors que si p est un **chemin de poids minimal** dans $\mathcal{C}_{i,j,k}$,

- soit il « passe » par $k-1$ et p est la concaténation de deux chemins, de i à $k-1$ et de $k-1$ à j qui sont de poids minimaux (raisonner par l'absurde) et respectivement dans $\mathcal{C}_{i,k-1,k-1}$ et dans $\mathcal{C}_{k-1,j,k-1}$;
- soit il ne passe pas par $k-1$ et alors c'est aussi un chemin de poids minimal entre i et j dans $\mathcal{C}_{i,j,k-1}$.

Exercice 13.11 *implémentation de l'algorithme RFW*

Soit $G = (S, A)$, un graphe orienté et valué qui ne contient pas de cycle de poids négatif, seule condition requise pour l'existence de chemins de poids minimaux.

1. On suppose que $S = \llbracket 0, n-1 \rrbracket$. Que représente $\mathcal{C}_{i,j,n}$?
2. On note $\ell(i, j, k)$ le poids minimal d'un chemin de i à j qui appartient à $\mathcal{C}_{i,j,k}$. Ce poids est ∞ si cet ensemble est vide. Justifier les relations suivantes :

$$\begin{cases} \ell(i, j, 0) = W_{i,j} = \begin{cases} = +\infty \text{ s'il n'y a pas d'arc entre } i \text{ et } j; \\ = w_{i,j} \text{ sinon} \end{cases} \\ \ell(i, j, k) = \min(\ell(i, j, k-1), \ell(i, k-1, k-1) + \ell(k-1, j, k-1)) \end{cases}$$

3. Écrire une fonction RFW(W) qui prend en argument la matrice d'adjacence d'un graphe valué et renvoie la matrice contenant les longueurs des chemins de poids minimaux.

Il est conseillé de commencer par préparer les données, par exemple en écrivant une fonction qui construit la matrice d'adjacence à partir de listes d'adjacence.

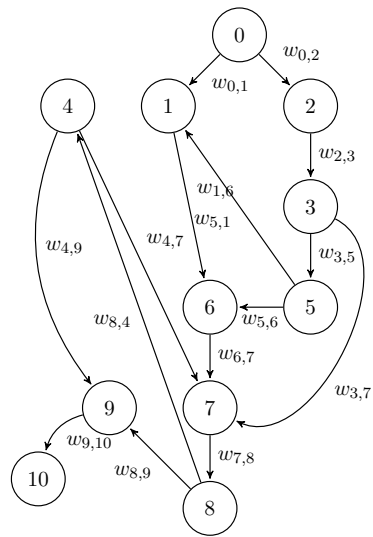
On pourra convenir que l'absence d'arc ou de chemin entre i et j est signalée par $W_{i,j} = n * \max(|w_{u,v}|) + 1$, puisqu'aucun chemin sans cycle n'atteindra ce poids. Nous notons `my_inf` cette valeur dans les scripts du corrigé.

4. Déterminer la complexité en temps et en mémoire de votre algorithme.

Corrigé en 13.11 page 503.

On pourra s'exercer avec le graphe orienté ci-contre. Lorsque les poids sont égaux, la matrice des $\ell(i, j, n)$ est (avec $12 = \text{my_inf}$) :

12	1	1	2	5	3	2	3	4	5	6
12	12	12	12	4	12	1	2	3	4	5
12	3	12	1	4	2	3	2	3	4	5
12	2	12	12	3	1	2	1	2	3	4
12	12	12	12	3	12	12	1	2	1	2
12	1	12	12	4	12	1	2	3	4	5
12	12	12	12	3	12	12	1	2	3	4
12	12	12	12	2	12	12	3	1	2	3
12	12	12	12	1	12	12	2	3	1	2
12	12	12	12	12	12	12	12	12	12	1
12	12	12	12	12	12	12	12	12	12	12




```

def PLSC2(S, T, D):

    m, n = len(S), len(T)

    if m not in D:
        D[m] = {}

    if m == 0 or n == 0:
        D[m][n] = 0, (0,0)
        return D[m][n][0]

    elif S[-1] == T[-1]:
        if m-1 not in D or n-1 not in D[m-1]:
            r = PLSC2(S[0:m-1], T[0:n-1], D)
        else:
            r = D[m-1][n-1][0]
        D[m][n] = r + 1, (-1,-1)
        return D[m][n][0]

    else:
        if m-1 not in D or n not in D[m-1]:
            r1 = PLSC2(S[0:m-1], T[0:n], D)
        else:
            r1 = D[m-1][n][0]

        if n-1 not in D[m]:
            r2 = PLSC2(S[0:m], T[0:n-1], D)
        else:
            r2 = D[m][n-1][0]

        if r1 > r2:
            D[m][n] = r1, (-1,0)
        else:
            D[m][n] = r2, (0,-1)

        return D[m][n][0]

```

5. Le dictionnaire de dictionnaires occupera au plus $(m+1) \times (n+1)$ entrées et chaque valeur prend la place mémoire d'un entier long et de deux entiers dans $\{1, 0, 1\}$.

Le nombre d'appels récur­sifs avec mémo­isation est, lui aussi, majoré par $(m+1) \times (n+1)$.

Corrigé de l'exercice n° 13.2

1. Voir le corrigé 13.1, question 1.
2. Voir le corrigé 13.1, question 2.

3. Il est clair que le calcul de $D[p][q]$ fait appel à des valeurs déjà calculées dans une itération précédente ou lors de l'initialisation.

```
def PLSC1(S,T):
    '''
    S, T : str;
    Renvoie un dictionnaire
    '''
    m, n = len(S), len(T)
    D = { p: {} for p in range(0, m+1)}

    for q in range(0, n+1):
        D[0][q] = 0, (0,0)
    for p in range(0, m+1):
        D[p][0] = 0, (0,0)

    for p in range(1, m+1):
        for q in range(1, n+1):
            if S[p-1] == T[q-1]:
                D[p][q] = D[p-1][q-1][0]+1, (-1,-1)
            else:
                r1 = D[p-1][q][0]
                r2 = D[p][q-1][0]
                if r1 > r2:
                    D[p][q] = r1, (-1,0)
                else:
                    D[p][q] = r2, (0, -1)

    return D
```

4. Le dictionnaire contient au plus $(m+1) \times (n+1)$ paires et le nombre d'itérations est $(m+1) \times (n+1) + m + n + 2 = O(m \times n)$.

Corrigé de l'exercice n° 13.3

```
def reconstruirePLSC(p, q):
    if p == 0 or n == 0:
        return ''
    else:
        if R == (-1,-1):
            return reconstruirePLSC(p-1, q-1) + T[q, n-1]
        elif R == (-1, 0):
            return reconstruirePLSC(p-1, q)
        elif R == (0, -1) :
            return reconstruirePLSC(p, q-1)
```

En ajoutant un caractère c tel que $c = S[p] == T[q]$ à Z en construction, on n'ajoute que des caractères communs aux deux chaînes. Z est donc à tout instant une sous-suite commune.

Il y a exactement autant d'ajouts que de termes $D[p][q][0] = c(p, q), (-1, -1)$ c'est à dire que d'incrémentations qui conduisent à $c(m, m)$. La longueur de Z sera exactement $c(n, m)$ et Z est une PLSC.

Corrigé de l'exercice n° 13.4

1. $(A_0 A_1)(A_2 A_3)$ pour $n = 4$ termes.
2. c_n correspond au nombre de parenthésages minimaux de $A_0, A_1, \dots, A_{n-1}, A_n$. On observe que l'on peut séparer la suite en deux sous-suites $A_0, \dots, A_i$ et $A_{i+1}, \dots, A_n$ de longueurs $i + 1$ et $n - i$ dès lors que $0 \leq i < n$. On peut compléter le parenthésage $(A_0, A_1, \dots, A_i)(A_{i+1}, \dots, A_n)$ en $c_i \times c_{n-1-i}$ parenthésages minimaux. On a bien $c_n = \sum_{i=0}^{n-1} c_i c_{n-1-i}$.
3. On suppose ici que $0 < |z| < 1/4$.

$$(a) \quad G^2(z) = \sum_{n=0}^{\infty} \left(\sum_{i=0}^n c_i c_{n-i} \right) z^n = \sum_{n=0}^{\infty} c_{n+1} z^n.$$

$$(b) \quad G^2(z) = \sum_{n=0}^{\infty} c_{n+1} z^n = \frac{G(z) - c_0}{z}. \text{ Soit } zG^2(z) - G(z) + 1 = 0. \text{ Les}$$

deux solutions de cette équations sont $\frac{1 \pm \sqrt{1-4z}}{2z}$. La solution prolongeable en 0 est donc $G(z) = \frac{1 - \sqrt{1-4z}}{2z}$.

$$(c) \quad \text{On a } (1+x)^\alpha = \sum_{k=0}^{\infty} \binom{\alpha}{k} x^k, \text{ avec } \binom{\alpha}{k} = \frac{\alpha(\alpha-1)\dots(\alpha-k+1)}{k!}.$$

D'où, en substituant :

$$G(z) = \frac{1 - \sqrt{1-4z}}{2z} = \frac{-1}{2} \sum_{k=1}^{\infty} \binom{1/2}{k} (-4)^k z^{k-1}$$

$$\text{Ce qui donne : } c_k = \frac{(-1)^k}{2} \binom{1/2}{k+1} 4^{k+1} = \frac{2^k}{(k+1)!} \prod_{j=0}^k |2j-1|$$

Classiquement, on insère $(2k)(2k-2)\dots(2) = 2^k k!$ dans le produit et on retrouve

$$c_k = \frac{(2k)!}{(k+1)k!k!} = \frac{1}{k+1} \binom{2k}{k}.$$

(d) Équivalence : la formule de Stirling $n! \sim \left(\frac{n}{e}\right)^n \sqrt{2\pi n}$ donne :

$$(2n)! \sim \left(\frac{2n}{e}\right)^{2n} \sqrt{4\pi n}, \text{ d'où } \frac{(2n)!}{(n!)^2} \sim \frac{\left(\frac{2n}{e}\right)^{2n} \sqrt{4\pi n}}{\left(\frac{n}{e}\right)^{2n} 2\pi n} = \frac{4^n}{\sqrt{\pi n}}$$

La formule annoncée est alors immédiate : $c_n = \frac{1}{n+1} \binom{2n}{n} \underset{n \rightarrow +\infty}{\sim} \frac{4^n}{\sqrt{\pi n^{3/2}}}$

Corrigé de l'exercice n° 13.5

1. M sera triangulaire supérieure stricte (les termes diagonaux sont nuls, les termes sous-diagonaux ne sont pas utilisés). Le résultat est $m(0, n-1) = M[0, n-1]$.
2. On attend dans $M[i, i+d] = m(i, i+d)$ le nombre optimal de multiplications de nombres pour obtenir $A_i \times A_{i+1} \times A_{i+d}$, c'est un produit de $d+1$ matrices.
3. On a ici $A_0 \in \mathcal{M}_{2,6}$, $A_1 \in \mathcal{M}_{6,3}$, $A_2 \in \mathcal{M}_{3,5}$. Les matrices M et S seront

$$M = \begin{bmatrix} 0 & 36 & 66 \\ 0 & 0 & 90 \\ 0 & 0 & 0 \end{bmatrix}, S = \begin{bmatrix} -1 & 0 & 1 \\ -1 & -1 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

On commence par remplir $M[0, 1] = 36$, $M[1, 2] = 90$.

Pour calculer $M[0, 2]$ il faudra retenir le minimum des deux nombres

$M[0, 0] + M[1, 2] + 2 \times 6 \times 5 = 0 + 90 + 60 = 150$ pour lequel on aurait $k = 0$, et $M[0, 1] + M[2, 2] + 2 \times 6 \times 5 = 36 + 0 + 30 = 66$ pour lequel $k = 1$. C'est ce dernier que l'on retient.

4. La fonction réserve deux tableaux de taille $n \times n$ (dans lesquels au plus $(n+1)n$ termes sont utiles). Ce qui nous fait une occupation mémoire en $O(n^2)$.

Dans les trois boucles imbriquées, les deux premières conduisent à un passage par terme au dessus de la diagonale. Mais on recherche un minimum ce qui a un coût linéaire. Une majoration grossière mais évidente conduit à évaluer la complexité en $O(n^3)$.

```
def choix_parenthesage(L):
    '''
    L : list;
    [L[i], L[i+1]] représente la taille d'une
    matrice Ai avec 0 <= i <= len(L)-2.
    Renvoie un tuple de tableaux (S, M) ...
    '''
```



```

n = len(L)-1
M = np.zeros((n,n), dtype = int) # contient m[i,j]
S = -np.ones((n,n), dtype = int) # sép. pour m[i,j]

for d in range(1, n):
    for i in range(0, n-d):
        M[i,i+d] = M[i,i] + M[i+1,i+d] + L[i]*L[i+1]*L[i+d+1]
        S[i,i+d] = i # parenthésage après Ai
        for k in range(i, i+d):
            m = M[i,k] + M[k+1,i+d] + L[i]*L[k+1]*L[i+d+1]
            if m < M[i, i+d]:
                M[i, i+d] = m
                S[i, i+d] = i+k # parenth. après A(i+k)

return S, M

```

5. On calcule successivement $A_0 * A_1$, $(A_0 * A_1) * A_2$, $((A_0 * A_1) * A_2) * A_3 \dots$. Les couples de matrices ont pour dimensions (ℓ_0, ℓ_1) , (ℓ_1, ℓ_2) puis (ℓ_0, ℓ_2) , (ℓ_2, ℓ_3) , (ℓ_0, ℓ_3) , $(\ell_3, \ell_4) \dots$

La complexité en nombre de multiplications de deux nombres est donc $\sum_{k=1}^{n-1} \ell_0 \ell_k \ell_{k+1}$.

L'algorithme de calcul des coûts est immédiat :

```

def cout_cascade(L):

    n = len(L)-1 # nombre de matrices
    r = 0
    for k in range(1, n):
        r += L[0]*L[k]*L[k+1]
    return r

```

6. L'opt. $m(0,4)$ est obtenu avec $k = S[0,4] = 1 : (A_0 \times A_1) \times (A_2 \times A_3 \times A_4)$.
 L'opt. $m(2,4)$ est obtenu avec $k = S[2,4] = 3 : (A_0 \times A_1) \times ((A_2 \times A_3) \times A_4)$.
 L'opt. $m(2,3)$ est obtenu avec $k = S[3,4] = 2$ mais on n'a pas besoin de cette information, le parenthésage est déjà complet.

Corrigé de l'exercice n° 13.6

1. Le graphe figure page 487. Les appels avec $i = j$ ou $j = i + 1$ sont terminaux. On n'a fait figurer dans le graphe que les appels avec $j > 1$.
 Une fonction récursive naïve lancera des appels redondants (on dit qu'il y a un **chevauchement des sous-problèmes**) et qu'elle conduira rapidement à une explosion du nombre des appels. Par contre, de par sa simplicité d'écriture, il peut être intéressant de l'écrire pour comparer et tester les autres fonctions que

nous avons écrites. Vous devinez là que cette remarque sent le vécu.

```
def ch_par_rec_brut(L, i, j):
    if i == j:
        return 0
    elif j == i+1:
        return L[i]*L[i+1]*L[i+2]
    else:
        V = []
        for k in range(i, j):
            m = ch_par_rec_brut(L, i, k)
            m += ch_par_rec_brut(L, k+1, j)
            m += L[i]*L[k+1]*L[j+1]
            V.append(m)
        return min(V)
```

2. On introduit donc un dictionnaire (on peut encore améliorer le code en testant la présence dans le dictionnaire AVANT les appels récursifs).

```
def ch_par_rec(L, i, j, D):
    '''
    Appel principal: i=0, j = len(L)-2 et D = {}
    '''
    assert i <= j and j < len(L)-1

    if i in D and j in D[i]:
        return D[i][j]
    elif i not in D:
        D[i] = {}

    if i == j:
        D[i][j] = 0, 0
        return D[i][j]
    elif j == i+1:
        D[i][j] = L[i]*L[i+1]*L[i+2], 0
        return D[i][j]
    else:
        p, s = ch_par_rec(L, i+1, j, D)
        m = p + L[i]*L[i+1]*L[j+1]
        s = i
        for k in range(i+1, j):
            p, s = ch_par_rec(L, i, k, D)
            q, s = ch_par_rec(L, k+1, j, D)
            m1 = p + q + L[i]*L[k+1]*L[j+1]
            if m1 < m:
                m, s = m1, k
        D[i][j] = m, s
        return D[i][j]
```

Corrigé de l'exercice n° 13.7

1. La fonction récursive :

```
def parenth_opt(S, i, j):
    if i == j:
        return 'A'+str(i)
    elif j == i+1:
        return 'A'+str(i) + ' A'+str(j)
    else:
        k = S[i,j]
        G = parenth_opt(S, i, k)
        D = parenth_opt(S, k+1, j)
        if k > i:
            G = '(' + G + ')'
        if j > k+1:
            D = '(' + D + ')'
        return G+D
```

2. La deuxième fonction est une simple adaptation de la première ; elle construit récursivement la même expression en structure arborescente qui se prêtera mieux à l'exécution des calculs à partir d'une liste de matrices.

```
def arbre_parenth_opt(S, i, j):
    if i == j:
        return tree('A'+str(i), [])
    elif j == i+1:
        return tree('*', [tree('A'+str(i), []), \
                             tree('A'+str(j), [])])
    else:
        k = S[i,j]
        G = arbre_parenth_opt(S, i, k)
        D = arbre_parenth_opt(S, k+1, j)
        if k > i:
            G = tree('()', [G])
        if j > k+1:
            D = tree('()', [D])
        return tree('*', [G, D])
```

Corrigé de l'exercice n° 13.8

1. $X = \text{'pirate'} \xrightarrow{R_Y} Y = \text{'pilate'}$;
 $Y = \text{'pilate'} \xrightarrow{R_Y} Z = \text{'pilote'}$;
 $X = \text{'pirate'} \xrightarrow{R_Z} \text{'pilate'} \xrightarrow{R_Z} Z = \text{'pilote'}$;

Les mêmes opérations élémentaires dans un ordre différent auraient donné :
 $X = \text{'pirate'} \xrightarrow{R_Z} \text{'pirote'} \xrightarrow{R_Z} Z = \text{'pilote'}$; mais on n'a plus ici la prop. ★.

2. Une composée de 3 transformations élémentaires permet de transformer $X = \text{'approximatif'}$ en $Y = \text{'aproksimatifs'}$:

$$X = \text{'approximatif'} \xrightarrow{R_Y} \text{'approkimatif'} \xrightarrow{I_Y} \text{'aproksimatif'} \xrightarrow{I_Y} Y.$$

Une transformation en sens inverse se décompose en :

$$Y = \text{'aproksimatifs'} \xrightarrow{S} \text{'approkimatif'} \xrightarrow{S} \text{'approkimatif'} \xrightarrow{R_X} X$$

ou encore en

$$Y = \text{'aproksimatifs'} \xrightarrow{R_X} \text{'approksimatifs'} \xrightarrow{S} \text{'approximatif'} \xrightarrow{S} X \text{ qui, contrairement à la précédente, vérifie la propriété } \star.$$

3. Si $T_{p-1} \circ \dots \circ T_1 \circ T_0(X) = Y$, alors $T_0^{-1} \circ T_1^{-1} \circ \dots \circ T_{p-1}^{-1}(Y) = X$.

Supposons la suite minimale et montrons que les T_i^{-1} sont licites comme transformations de Y vers X . On note $X = Z_0, Z_1 = T_0 Z_0, Z_2 = T_1 Z_1$, etc.

Si T_k est une suppression dans Z_k , $Z_k = T_k^{-1} Z_{k+1}$ est l'insertion d'un caractère de X . En effet si ce n'était pas le cas, le caractère supprimé par T_k ne serait pas un caractère de X et proviendrait donc de Y , donc d'une insertion. En supprimant ces deux opérations (insertion, suppression), on ne change pas le résultat final et la suite des $(T_k)_k$ ne serait pas minimale.

Si T_k est le remplacement dans Z_k d'un caractère par un caractère de Y , T_k^{-1} est le remplacement dans Z_{k+1} d'un caractère par un caractère qui provient de X , toujours pour une raison de minimalité.

Enfin, si T_k est une insertion dans Z_k , $T_k^{-1} Z_{k+1}$ est la suppression du caractère ajouté.

On a donc $Lev(Y, X) \leq p = Lev(X, Y)$. Par symétrie $Lev(Y, X) = Lev(X, Y)$.

4. $Lev(X, Y) \geq 0$.

Il est clair que $Lev(X, X) = 0$ et que $Lev(X, Y) = 0 \Rightarrow X = Y$.

D'après la question précédente, on a $Lev(X, Y) = Lev(Y, X)$.

Il reste à vérifier l'inégalité triangulaire.

Si $p = Lev(X, Y)$ et $q = Lev(Y, Z)$, il existe deux suites **minimales** de transformations telles que :

$$- T_{p-1} \circ \dots \circ T_1 \circ T_0(X) = Y, \text{ avec } T_k \in \{I_Y, R_Y, S\};$$

$$- T_{p+q-1} \circ \dots \circ T_{p+1} \circ T_p(Y) = Z, \text{ avec } T_k \in \{I_Z, R_Z, S\}.$$

$$\text{On a donc } T_{p+q-1} \circ \dots \circ T_{p+1} \circ T_p \circ T_{p-1} \circ \dots \circ T_1 \circ T_0(X) = Z.$$

Considérons les remplacements ou insertions qui font apparaître un caractère de Y qui n'est pas dans Z . Supprimons les de la liste de même que les suppressions qui les concernent et qui existent certainement puisque Cela ne change rien au résultat et les transformations qui restent sont toutes dans $\{I_Z, R_Z, S\}$. On a donc $Lev(X, Z) \leq p + q = Lev(X, Y) + Lev(Y, Z)$.

Corrigé de l'exercice n° 13.9

1. Il n'y a aucune opération à faire pour transformer $X[0 : 0] = Y[0 : 0] = \varepsilon$ en elle-même, donc $d(0, 0) = 0$.

Pour transformer $\varepsilon = X[0 : 0]$ en $Y[0 : j]$ il faut (au minimum) insérer les j caractères de Y . Comme ces ajouts permettent de transformer ε en Y , on a $d(0, j) = j$; de même, $d(i, 0) = i$.

2. On considère donc une suite de p ($p > 1$) opérations qui transforme X en Y , qui est minimale et qui vérifie ★. On note $X = X_0 \longrightarrow X_1 \longrightarrow \dots \longrightarrow X_{p-1} = Y$.

La dernière opération T_{p-1} porte sur le dernier caractère de X_{p-2} (propriété ★). Si c'est une insertion, les étapes précédentes ont transformé $X = X[0 : m]$ en $Y[0 : n - 1]$. Si c'est une suppression, les étapes précédentes ont transformé $X = X[0 : m - 1]$ en $Y[0 : n]$. Si c'est un remplacement, les étapes précédentes ont transformé $X = X[0 : m - 1]$ en $Y[0 : n - 1]$.

Quelque soit l'avant dernier état, la suite des étapes précédentes est **minimale** (si on peut la remplacer par une suite de longueur strictement inférieure, notre suite initiale n'est plus minimale).

3. • Considérons une transformation minimale de $X[0 : i - 1]$ en $Y[0 : j - 1]$. Deux cas se présentent :

- $X'[i - 1] = Y[j - 1]$ et en ne faisant rien on a aussi transformé $X[0 : i]$ en $Y[0 : j]$ avec un coût $d[i - 1, j - 1]$.

- $X'[i - 1] \neq Y[j - 1]$ et remplaçant $X[i - 1]$ par $Y[j - 1]$ on transforme $X[0 : i]$ en $Y[0 : j]$ avec un coût $d[i - 1, j - 1] + 1$.

• Considérons une transformation minimale de $X[0 : i - 1]$ en $Y[0 : j]$. En ajoutant la suppression du terme $X[i - 1]$ on obtient une transformation de $X[0 : i]$ en $Y[0 : j]$ avec un coût $d[i - 1, j] + 1$.

• Considérons une transformation minimale de $X[0 : i]$ vers $Y[0 : j - 1]$. En insérant le terme $Y[j - 1]$ on obtient une transformation de $X[0 : i]$ en $Y[0 : j]$ avec un coût $d[i, j - 1] + 1$.

L'avant dernier état obtenu par une suite minimale conduisant de $X[0 : i]$ à $Y[0 : j]$ est nécessairement une des trois états envisagés, et il a été atteint de façon minimale, **d'où les formules énoncées.**

4. On construit le tableau en fonction de X et de Y . La distance est le terme $T[\text{len}(X), \text{len}(Y)]$.

5. Coûts : place mémoire $(m + 1)(n + 1) = O(n \times m)$.

Nombre d'itérations : $(m + n + 1) + m \times n$ dont $m \times n$ appels à min.

```

def distance_edition(X, Y):
    m, n = len(X), len(Y)
    T = np.zeros((m+1,n+1), dtype=int)

    for i in range(0, m+1):
        T[i,0] = i
    for j in range(0, n+1):
        T[0,j] = j
    for i in range(1,m+1):
        for j in range(1, n+1):
            if X[i-1] == Y[j-1]:
                T[i,j] = min(T[i-1,j-1], T[i-1,j]+1, \
                               T[i,j-1]+1)
            else:
                T[i,j] = min(T[i-1,j-1] + 1, T[i-1,j]+1,
                               T[i,j-1]+1)

    return T[m,n], T

```

Corrigé de l'exercice n° 13.10

1. Soit C un circuit de poids $w < 0$. Si p est un chemin de poids minimal entre deux sommets i et j de C , on peut construire un autre chemin entre i et j en insérant C dans p . Ce chemin sera de poids strictement inférieur à celui de p , ce qui est contradictoire.
2. Soit $k = i_s$, p est une concaténation de $c_1 = ((i_0, i_1, w_{i_0, i_1}), \dots, (i_{s-1}, k, w_{i_{s-1}, k}))$ et de $c_2 = ((i_s, i_{s+1}, w_{i_s, i_{s+1}}), \dots, (i_{\ell-1}, j, w_{i_{\ell-1}, j}))$.
S'il existe un chemin c'_1 de i à k de poids strictement inférieur à celui de c_1 , alors $c'_1 + c_2$ est un chemin de i à j de poids strictement plus petit que celui de p . Contradiction.

Corrigé de l'exercice n° 13.11

1. $\mathcal{C}_{i,j,n} = \mathcal{C}_{i,j}$, est l'ensemble des chemins de G reliant i à j puisque tous les sommets intermédiaires sont dans $\llbracket 0, n-1 \rrbracket$.
2. $C(i, j, 0)$ est l'ensemble des arcs reliant i à j (pas de sommet intermédiaire). On a donc bien $\ell(i, j, 0) = W_{i,j}$.

D'après les propriétés établies dans l'encart, si un chemin minimal de $C(i, j, k)$ ne passe pas par $k-1$ il est dans $C(i, j, k-1) \subset C(i, j, k)$ et reste donc minimal. Dans ce cas, $\ell(i, j, k) = \ell(i, j, k-1)$.

Sinon, sa longueur est la somme des longueurs de deux chemins minimaux de $\mathcal{C}_{i,k-1,k-1}$ et $\mathcal{C}_{k-1,j,k-1}$. Dans ce cas, $\ell(i, j, k) = \ell(i, k-1, k-1) + \ell(k-1, j, k-1)$.

La formule de récurrence de l'énoncé est donc valide. Elle va nous permettre de calculer les distances de tous les couples de sommets.

3. On propose le script calculant la matrice d'adjacence puis une implémentation de l'algorithme RFW qui calcule les poids minimaux des chemins. Ces fonctions définissent ou utilisent une variable `my_inf` contenant un majorant du poids maximal d'un chemin sans cycle et qui joue le rôle de $+\infty$ qui signale l'absence d'arc ou de chemin.

```
def matriceAdjacenceGO(S,A):
    '''
    Renvoie la matrice d'adjacence du graphe orienté et
    valué dont
    - S est la liste des sommets;
    - A la liste des arêtes (implémentées comme triplets).
    '''
    n      = len(S)
    my_inf = n*max([a[2] for a in A])+1
    M      = my_inf*np.ones((n,n), dtype = int)

    for a in A:
        [i,j, w] = a
        M[i,j]   = w
    return M
```

```
def RFW(W):
    '''
    W : matrice d'adjacence; graphe orienté valué.
    '''
    n, m = W.shape
    my_inf = max([ max( [W[i,j] for i in range(0,n)] ) \
                    for j in range(0,m)])

    L = [W]
    for k in range(1, n+1):
        D0 = L[-1]
        D1 = np.zeros((W.shape), dtype = int)
        for i in range(0, n):
            for j in range(0, n):
                D1[i][j] = min(D0[i][j], \
                               D0[i][k-1] + D0[k-1][j], my_inf)
        L.append(D1)

    return D1
```

4. La complexité en temps est clairement $O(n^3)$. L'espace mémoire occupé est en $O(n^3)$ également. Il n'est toutefois pas utile de garder toutes les matrices et le script ci-dessous est en $O(n^2)$ pour ce qui est de la mémoire occupée.

```
def RFW(W) :
    n, m = W.shape
    my_inf = max([ max( [W[i,j] for ...] ) for ...])

    D0 = W
    for k in range(1, n+1):
        D1 = np.zeros((W.shape), dtype = int)
        for i in range(0, n):
            for j in range(0, n):
                D1[i][j] = ...
        D0 = D1.copy()
    return D1
```


Algorithmes pour l'étude des jeux

Nous abordons ici quelques aspects algorithmiques de la théorie des jeux. Notre étude se restreindra aux jeux à deux joueurs qui jouent en alternance, à information complète et à somme nulle que nous modéliserons à l'aide de graphes ou d'arbres.

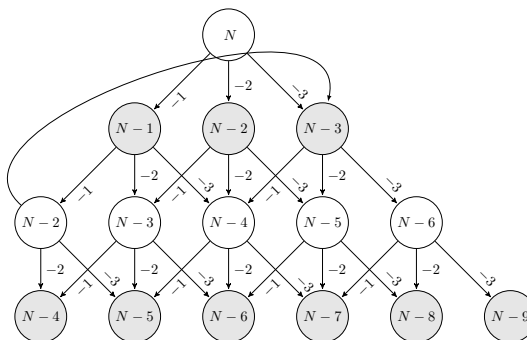
14.1 Jeux sur graphes

14.1.1 Exemples de jeux à deux joueurs

Le jeu de Nim

Rappelons les règles de ce jeu, déjà rencontré dans l'exercice 8.5 page 319.

- On dispose de N jetons identiques posés sur une table ;
- deux joueurs $A = J_0$ et $B = J_1$ jouent tour à tour ;
- chacun d'eux doit, lorsque c'est son tour, prélever soit un, soit deux, soit trois jetons sans vider la table ;
- le joueur qui se trouve devant un jeton seul a perdu.

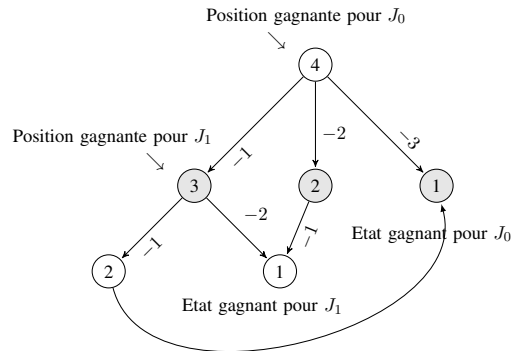


Ce graphe représente partiellement les parties commençant avec N ($N \geq 10$) jetons sur la table, le joueur J_0 étant le premier à jouer. Il peut choisir entre trois coups qui conduisent chacun à un **état du jeu** pour lequel ce sera au joueur J_1 de jouer.

Les sommets en blanc représentent les états dans lesquels c'est le joueur J_0 qui joue. On dit qu'ils sont **contrôlés** par J_0 . Les sommets en gris sont les sommets contrôlés par J_1 . Seuls le nombre de jetons sur la table et le nom ou l'indice du joueur qui doit jouer (qui contrôle le sommet) interviennent. Plusieurs déroulements de parties peuvent donc conduire à un même état, ce qui explique que nous ayons affaire à un graphe qui n'est pas un arbre.

Comme il n'y a pas de partie nulle dans ce jeu, tous les **sommets ou états terminaux** du graphe représentent des **états gagnants** pour J_0 (en gris) ou pour J_1 . Ce sont les sommets de degré sortant égal à 0 dans le graphe.

On observera que certains sommets sont des **positions gagnantes** pour J_0 (ou pour J_1) : c'est une position qui, si elle est atteinte, permet au joueur concerné de gagner à coup sûr s'il ne commet pas d'erreur.



On démontre dans l'exercice 8.5 que le joueur en position de jouer lorsque le nombre de jetons est de la forme $4p$, $4p+2$ ou $4p+3$ peut imposer une succession de coups le conduisant à la victoire quoi que fasse son adversaire. Notre objectif, dans ce chapitre, est de mettre en place un algorithme permettant d'identifier les positions gagnantes puis des stratégies gagnantes pour un joueur dans des jeux de ce type.

Exercice 14.1 construire le graphe du jeu

On se propose de construire le graphe des parties de Nim à deux joueurs J_0 et J_1 qui commencent avec N jetons sur la table. Cela nous servira à tester nos algorithmes.

Les sommets qui représentent les états du jeu sont étiquetés. L'état « Il y a N jetons sur la table et c'est J_0 qui doit jouer » est étiqueté ' N_J0 '. Les sommets portent des n° de 0 à $\text{len}(S)-1$. Ces informations sont stockées dans un dictionnaire, lui aussi noté S , tel que $S['N_Ji']$ est l'indice du sommet.

Le graphe est représenté par le tuple (S, A) où A est sa liste d'adjacence $A = [i, j], \dots$.

Exemple :

$S = \{'4-J0' : 0, '3-J1' : 1, '2-J0' : 2, '1-J1' : 3, '1-J0' : 4, '2-J1' : 5\}$

$A = [[0, 1], [1, 2], [2, 3], [1, 4], [0, 5], [5, 4], [0, 3]]$

1. Dessiner le graphe (S, A) qui sert d'exemple.
2. Compléter la fonction `labelize(N,j)` qui renvoie l'étiquette de l'état « Il y a N jetons sur la table et c'est J_j qui doit jouer ».

`labelize = lambda N, j: ...`

3. On veut écrire une procédure ou fonction récursive `graphe_nim(N, S, A, j)` dont l'appel principal se fera avec $S = \{\}$; $A = []$; $j = 0$; cette fonction

renvoie un tuple S, A qui correspond au graphe étiqueté décrit dans le préambule. Notons que `graphe_nim(N, S, A, j)` **modifie en place** ses arguments S et A . En voici le schéma :

```
def graphe_nim(N, S, A, j):
    '''
    N: int, S: dict, A: list, j = 0|1.
    Renvoie le tuple S, A, graphe orienté et étiqueté.
    '''
    labelize = ... # (0)

    ell_0 = labelize(N, j)
    if ... : # (1)
        S[ell_0] = len(S)

    for i in [1, 2, 3]:
        if ... : # (2)
            ell_1 = ... # (3)
            if ell_1 not in S:
                S[ell_1] = ... # (4)
                A.append([S[...], S[...]]) # (5)
                graphe_nim(?, S, A, ?) # (6)
    return S, A
```

- (a) Le seul appel récursif a lieu en `graphe_nim( ... ) # (6)`, les valeurs retournées ne sont pas affectées à des variables. Sont-elles perdues pour autant ? Expliquer cela.
 - (b) Quel est l'intérêt de placer les étiquettes comme clés du dictionnaire et pas l'inverse ?
 - (c) La correspondance entre les étiquettes et les états est bijective. Quel est le procédé qui assure qu'il en sera de même pour la correspondance entre les états et les indices ?
 - (d) Expliquer la ligne # (5).
 - (e) Compléter le schéma.
4. Dessiner l'arbre des appels principal et récursifs. A quoi correspond-il pour le jeu ?

Corrigé en 14.1 page 525.

L'exercice 14.4 propose la construction de ce même ensemble des parties comme branches d'un arbre.

Le jeu OXO

Dans ce jeu, le joueur J_0 avec les jetons 'O' commence. J_1 place les jetons 'X'. Le premier qui obtient un alignement de trois de ses jetons a gagné.

Le deuxième plateau montre un état dans lequel J_0 a le contrôle, mais le sort de la partie est joué : il y aura match-nul.

Question : Donner un encadrement du nombre de parties possibles. L'exercice 14.5 en permet le décompte en force brute.

2	O	5	X	8	
1		4	O	7	
0		3		6	X

2		5	X	8	O
1	X	4	O	7	O
0	X	3	O	6	X

14.1.2 Représentation par des graphes, vocabulaire

Définition 14.1 prédécesseurs, successeurs

Soit $G = (S, A)$ un graphe orienté. Pour tout $s \in S$, les **successeurs** de s sont les extrémités des arcs dont s est l'origine. Les **prédécesseurs** de s sont les origines des arcs dont s est l'extrémité.

Un sommet est **terminal** s'il n'a pas de successeur (c'est à dire si son **degré sortant** est nul).

Définition 14.2 graphe biparti

Soit $G = (S, A)$ un graphe (orienté ou pas, étiqueté ou pas). On dit que G est **biparti** s'il existe deux sous-ensembles de sommets, S_0 et S_1 , formant une partition de S et tels que pour tout arc ou arête $a \in A$, l'origine et l'extrémité sont dans des S_i différents.

Exemples :

- Le graphe représentant les débuts d'une partie de Nim est un graphe biparti : si on note S_0 et S_1 les ensembles de sommets blancs ou grisés (qui sont les sommets contrôlés par les joueurs J_0 et J_1 respectivement), on constate qu'il n'y a aucun arc de S_i vers lui-même.

Définition 14.3 graphe de jeu ou arène

Un **graphe de jeu à deux joueurs** ou **arène** est une structure $\mathcal{G} = (G, S_0, S_1)$ où $G = (S, A)$ est un graphe fini et biparti, S_0, S_1 sont comme dans la définition 14.2. Par convention, on associe à tout graphe de jeu deux **joueurs** J_0 et J_1 , et on dit que S_i est l'ensemble des **sommets contrôlés** par le joueur J_i .

Exemples :

- Revenons au jeu de Nim : le joueur J_0 joue le premier et contrôle S_0 , l'ensemble des sommets non grisés. J_1 contrôle les autres sommets. Pour J_i , contrôler un sommet ou un état, signifie que c'est à son tour de jouer lorsque la partie est dans cet état.

Les graphes de jeux bipartis tels que nous les avons définis permettent de représenter des jeux dans lesquels deux joueurs jouent en alternance, où chaque coup conduit à une position contrôlée par le joueur adverse.

Maintenant que le déroulement du jeu est en place (l'ensemble des coups permis est égal à A), il nous reste à définir formellement les parties, les conditions de victoire. Nous pourrions alors parler de parties gagnantes, de positions gagnantes et de stratégies.

Comment une partie se déroule-t-elle dans la vraie vie ?

Observons une partie à son début ou à partir d'une position quelconque s . Si $s \in S_i$, c'est le joueur J_i qui a le trait. Deux cas se présentent :

- s est un sommet terminal, il n'y a plus de coup à jouer et la partie est finie (victoire d'un des joueurs ou match nul).
- Sinon, jouer revient à choisir un arc (s, s') d'origine s dans A . Après ce coup, de par la définition de l'arène ou du jeu que nous avons donnée, l'état s' est dans S_{1-i} et il est contrôlé par l'autre joueur.

Ces observations nous conduisent aux définitions formelles qui suivent :

Définition 14.4 parties, parties partielles

- Un chemin dans un graphe orienté G , est **maximal** lorsqu'il est infini ou lorsque l'extrémité de son dernier arc est terminale.
- Soit $\mathcal{G} = (G, S_0, S_1)$ un graphe de jeu. Une **partie** débutant en s_0 est un **chemin maximal** de G et dont le premier arc a pour origine s_0 .
- Une **partie partielle** débutant en s_0 est un chemin fini dont le premier arc a pour origine s_0 .

Observation : Les jeux que nous avons considéré ont des parties finies : les parties aboutissent toutes à des sommets terminaux.

Notations : Nous noterons $\mathcal{P}_0, \mathcal{P}_1$ les ensembles formés des parties partielles qui aboutissent à un sommet/état/position appartenant à S_0 ou S_1 respectivement¹.

On pourra représenter une partie par la succession des sommets visités :

$P = (s_0, s_1, \dots, s_{p-1})$ représente donc le chemin $(s_0, s_1), (s_1, s_2), \dots, (s_{p-2}, s_{p-1})$.

Définition 14.5 stratégies, stratégies sans mémoire

Soit $((S, A), S_0, S_1)$ un graphe de jeu.

- Une **stratégie** pour le joueur J_i est une application $\sigma : \mathcal{P}_i \longrightarrow S_{1-i}$ qui à toute partie partielle $(s_0, s_1, \dots, s_{p-1})$ aboutissant en $s_{p-1} \in S_i$, associe un sommet s_p tel que $(s_{p-1}, s_p) \in A$. (s_{p-1}, s_p) est donc un **coup jouable** pour J_i .
- Une partie $(s_0, s_1, \dots, s_t)$ est **conforme à une stratégie** pour le joueur i , si pour tout k tel que $0 \leq k < t$ et $s_k \in S_i$ $s_{k+1} = \sigma(s_0, s_1, \dots, s_k)$.
- Une **stratégie sans mémoire** est une stratégie $\sigma : \mathcal{P}_i \longrightarrow S_{1-i}$ telle que pour tout couple de parties partielles $(s_0, s_1, \dots, s_k), (v_0, v_1, \dots, v_\ell) \in \mathcal{P}_i^2$,

$$s_k = v_\ell \Rightarrow \sigma((s_0, s_1, \dots, s_k)) = \sigma((v_0, v_1, \dots, v_\ell)).$$

1. On note souvent S^* l'ensemble des chemins finis (donc des parties partielles) ; $S^* S_0$ les parties partielles qui aboutissent en S_0 .

Disposer d'une stratégie σ permet de savoir quel coup (unique, car σ est une application) jouer en toute circonstance du jeu.

Dire qu'une stratégie est sans mémoire c'est dire que pour jouer un coup dans une partie conforme, seul l'état actuel du jeu est à prendre en considération. Il existe une règle aux échecs qui nous dit qu'une partie est nulle si une même position des pièces sur l'échiquier se répète trois fois. Si on considère que les états sont les différentes positions des pièces, il n'y a donc pas de stratégie utile sans mémoire.

Définition 14.6 *condition de gain, position gagnante*

Soit $((S, A), S_0, S_1)$ un graphe de jeu.

- Une **condition de gain** pour le joueur J_i dans ce jeu est un ensemble de parties $\Omega_i \subset S : J_i$ gagne une partie π ssi $\pi \in \Omega_i$.
- Une **stratégie est gagnante** pour le joueur J_i si toute partie conforme à la stratégie est gagnée par J_i .
- Une **stratégie est gagnante à partir d'une position** d pour le joueur J_i si toute partie conforme à la stratégie et qui débute en d est gagnée par J_i (appartient à Ω_i). On dit que d est une **position gagnante** pour J_i .

Un jeu est donc pleinement défini si on dispose d'un graphe de jeu (ou arène), d'une condition de gain pour chaque joueur et si le sommet de départ est choisi. On considère alors les parties débutant sur ce sommet.

Exemples :

- Pour le jeu de Nim, une condition de gain pour J_i est l'ensemble des parties pour lesquelles le sommet terminal représente un état appartenant au joueur adverse avec un jeton sur la table. Ce que nous avons étiqueté ' $1 - J_{1-i}$ ' dans l'exercice 14.1. Des positions gagnantes sont indiquées sur le graphe de la page 508.
- Pour le jeu de OXO, une condition de gain pour J_i est l'ensemble des parties pour lesquelles le sommet terminal représente un plateau dans lequel trois de ses pions sont alignés. Notons que dans ce jeu, $\Omega_0 \cup \Omega_1 \neq S$. Il y a des parties nulles.
- Pour le jeu de Hex, une condition de gain pour J_i est l'ensemble des parties pour lesquelles le sommet terminal représente un plateau sur lequel des pions tracent une ligne continue joignant deux bords opposés. On démontre qu'il n'y a pas de partie nulle dans un tel jeu.

14.2 Calcul des attracteurs dans les jeux d'accessibilité

14.2.1 Jeu d'accessibilité

Définition 14.7 *jeu d'accessibilité*

Soit $\mathcal{G} = ((S, A), S_0, S_1)$ un graphe de jeu à deux joueurs, et Ω_i une condition de

gain sur $\mathcal{G}$ pour le joueur J_i .

On dit que le jeu ainsi défini est un **jeu d'accessibilité** lorsque

- il existe un ensemble de sommets terminaux F_i (ensemble cible) tel que les parties gagnantes pour J_i sont celles qui se terminent sur un sommet de F_i , on a donc $\Omega_i = \{(s_0, \dots, s_t) \in \mathcal{P} / s_t \in F_i\}$.
- il n'y a pas de match nul (c'est à dire que $\Omega_{1-i} = \mathcal{P} \setminus \Omega_i$).

Question : les jeux de Nim, de OXO sont-ils des jeux d'accessibilité. Peut on caractériser F_i dans chaque cas ?

14.2.2 Attracteurs et pièges

Considérons un jeu d'accessibilité dans lequel la condition de victoire pour le joueur J_0 est que la partie aboutisse à F_0 . Pour une position donnée, J_0 est à un coup de gagner avec certitude, soit si c'est à son tour de jouer et s'il **peut choisir** un coup qui le conduit à F_0 , soit si c'est à son adversaire de jouer et que **tous** les coups jouables mènent à F_0 .

Cela nous incite à construire l'ensemble des positions gagnantes dans un tel jeu de la façon suivante :

1. On définit deux applications de $\mathcal{P}(S)$ dans lui-même, en posant pour tout ensemble de sommets, $X \subset S$:

$$Pr(X) = \{s \in V_0 / \exists (s, s') \in A, s' \in X\} \cup \{s \in V_1 / \forall (s, s') \in A, s' \in X\}$$

et $\mathcal{F}(X) = X \cup Pr(X)$. $Pr(X)$ est l'ensemble des positions qui **permettent** au joueur J_0 ou qui **imposent** au joueur J_1 d'amener le jeu en X .

2. On définit alors par récurrence une suite d'ensembles en posant :

$$(a) \quad A_0 = F_0;$$

$$(b) \quad A_{n+1} = A_n \cup Pr(A_n) = \mathcal{F}(A_n).$$

Observons que, lorsque $s \in A_i$, le joueur J_0 est à i coups au plus de la victoire.

La suite $(A_i)_i$ est croissante pour l'inclusion et comme S est fini, elle est **stationnaire** : il existe un rang n_0 à partir duquel $A_{n_0} = A_{n_0+p}$. On note $Attr(F_0, J_0) = A_{n_0}$.

Définition 14.8 attracteur et rang

1. L'ensemble $Attr(F_0, J_0)$ est le **bassin d'attraction**² de F_0 pour le joueur J_0 .
2. Pour tout $s \in Attr(F_0, J_0)$, on définit le **rang** de s : $rg(s) = \min\{i / s \in A_i\}$. Lorsque $s \notin Attr(F_0, J_0)$, on pose : $rg(s) = \infty$.

2. La littérature parle souvent d'attracteur.

Théorème 14.1

Soit $\mathcal{G} = (S, A, S_0, S_1)$ un graphe de jeu d'accessibilité à deux joueurs pour lequel F_0 est la condition de gain pour le joueur J_0 . Alors,

- $W_0 = \text{Attr}(F_0, J_0)$ est l'ensemble des positions gagnantes pour le joueur J_0 .
- $W_1 = S \setminus \text{Attr}(F_0, J_0)$ est l'ensemble des positions gagnantes pour le joueur J_1 .

Ce théorème, dont la démonstration est proposée dans l'exercice 14.3, nous dit que le bassin d'attraction $\text{Attr}(F_0, J_0)$ est l'ensemble des positions gagnantes pour J_0 . Un **algorithme**, avec une complexité linéaire en $|S| + |A|$, permet de le calculer.

Exercice 14.2 *Calcul des positions gagnantes*

On se propose d'implémenter un algorithme de calcul du bassin d'attraction dans un jeu d'accessibilité. Nous allons procéder par étapes.

La fonction que nous voulons construire aura l'en-tête suivante :

```
def bassin_attraction(S, A, S0, F0):
    '''
        S : list of int,          sommets ou positions;
        A : list of [int, int],   liste d'adjacence;
        S0 : list of int,        positions contrôlées par J0;
        F0 : list of int,        condition de gain pour J0.

        Retourne Attr: liste des positions gagnantes.

        Pré-conditions:
        (S, A) graphe orienté;
        A inclus dans S0 x S1 union S1 x S0 (S1 = S\S0);
        (donc (S, A) est biparti, associé à la partition S0, S1);
        F sommets terminaux de (S, A), condition de gain pour J0.
    '''
```

On considère le graphe produit par la fonction de l'exercice 14.1, dont les sommets et les arcs sont représentés par le dictionnaire et la liste :

$DS = \{ '5-J0' : 0, '4-J1' : 1, '3-J0' : 2, '2-J1' : 3, '1-J0' : 4, '1-J1' : 5, '2-J0' : 6, '3-J1' : 7 \}$

$A = [[0, 1], [1, 2], [2, 3], [3, 4], [2, 5], [1, 6], [6, 5], [1, 4], [0, 7], [7, 6], [7, 4], [0, 3]]$

1. Écrire des lignes de code qui construiront la liste S des sommets, la liste S_0 des sommets contrôlés par J_0 et la condition de gain F_0 qui pourront être données en arguments lors d'un appel `bassin_attraction(S, A, S0, F0)`.
2. Nous voulons que notre fonction renvoie une liste des sommets de $W_0 = \text{Attr}(J_0, F_0)$. Elle devra disposer, pour chaque sommet de l'ensemble de ses prédécesseurs, de son degré sortant.
Écrire les lignes de codes qui construisent les dictionnaires `pred` et `hmr`, tels que `pred[u]` et `hmr[u]` contiennent respectivement la liste des prédécesseurs de u et

son degré sortant (hmr pour how-many-remain, car ses valeurs sont appelées à être décrémentées).

Tester la fonction en construction avec le graphe donné en exemple qui correspond à un jeu de Nim dans lequel J_0 démarre avec 5 jetons (et vérifier à l'aide de la figure).

3. Pour terminer on définira une sous-procédure récursive `marquer_poursuivre(s)` qui prend en argument une **position gagnante**, s , l'ajoute à la liste `Attr`, et recherche dans les prédécesseurs de s d'éventuelles positions gagnantes.

Les appels principaux se feront dans la fonction `bassin_attraction(...)` avec ses dernières lignes :

```
for x in F0:
    marquer_propager(x)
return Attr
```

Indications :

- `marquer_poursuivre(s)` ne fait rien si s a déjà été ajouté à `Attr` ;
- un sommet u est dans `Attr` s'il est dans S_0 et possède un successeur dans `Attr`, ou bien si tous ses successeurs sont dans `Attr` ; pour s'assurer de cela on décrémente `hmr[u]` chaque fois que l'on rencontre un successeur de u dans `Attr`...

4. Montrer que la liste `Attr` renvoyée par la fonction correspond bien au bassin d'attraction (ou attracteur !) de F_0 et J_0 .

Indications : on prouvera les deux inclusions ensembliste $Attr \subset W_0$ et $W_0 \subset Attr$. Pour la première on pourra procéder par récurrence sur la profondeur des appels récursifs et pour la seconde, considérer la suite $(A_i)_i$ définie page 513 et montrer que $A_i \subset Attr$ par récurrence sur i .

5. Montrer que cette fonction a une complexité linéaire en $|S| + |A|$.

Corrigé en 14.2 page 526.

Ci-dessous, le script permettant d'appeler la fonction `bassin_attraction(S,A,S0,F0)` suivi du résultat. On a placé en début de la sous-procédure `marquer_poursuivre()` une instruction `print(...)`

On observe que le sommet de départ, $5 - J_0$ n'est pas une position gagnante pour J_0 ce qui est conforme à ce nous démontrons dans l'exercice 8.5 en recherchant des invariants de boucle dans un programme de simulation du jeu. L'algorithme procède autrement, par exploration du graphe.

```
Attr = bassin_attraction(S.values(), A, S0, F0)
Sinv = {v: k for k, v in S.items()}
for r in Attr:
    print(Sinv[r], r)
```

```

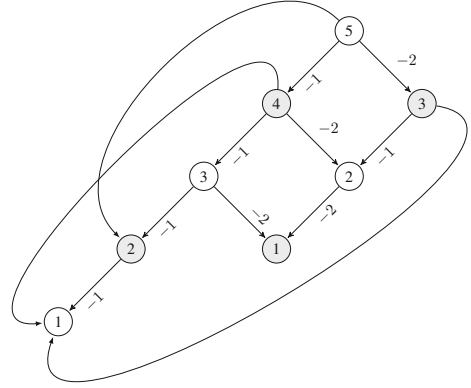
marquer_propager(5)
marquer_propager(2)
marquer_propager(6)

```

```

1-J1 5
3-J0 2
2-J0 6

```



Question :

Qui gagne cette partie ? Et comment ?

Exercice 14.3 démonstration du théorème 14.1

La démonstration repose sur les définitions du bassin d'attraction et du rang (14.8). On suppose que les sommets de S sont numérotés.

1. Parmi les assertions (a), (b), (c), (d) certaines sont vraies, d'autres sont fausses.

(a) Pour tout $s \in Attr(F_0, J_0)$, on est dans un des trois cas suivants :

- i. $s \in F_0$;
- ii. $s \in S_0 \setminus F_0$ et s admet un prédécesseur t tel que $\text{rang}(t) < \text{rang}(s)$;
- iii. $s \in S_1 \setminus F_0$ et, pour tout prédécesseur t de s , $\text{rang}(t) < \text{rang}(s)$;

(b) Pour tout $s \in Attr(F_0, J_0)$, on est dans un des trois cas suivants :

- i. $s \in F_0$;
- ii. $s \in S_0 \setminus F_0$ et s admet un successeur t tel que $\text{rang}(t) < \text{rang}(s)$;
- iii. $s \in S_1 \setminus F_0$ et, pour tout successeur t de s , $\text{rang}(t) < \text{rang}(s)$;

(c) Pour tout $s \in S \setminus Attr(F_0, J_0)$, on est dans un des deux cas :

- i. $s \in S_0$ et il existe un successeur t de s tel que $\text{rang}(t) = \infty$;
- ii. $s \in S_1$ et pour tout successeur t de s , $\text{rang}(t) = \infty$;

(d) Pour tout $s \in S \setminus Attr(F_0, J_0)$, on est dans un des deux cas :

- i. $s \in S_0$ et pour tout successeur t de s , $\text{rang}(t) = \infty$;
- ii. $s \in S_1$ et il existe un successeur t de s tel que $\text{rang}(t) = \infty$.

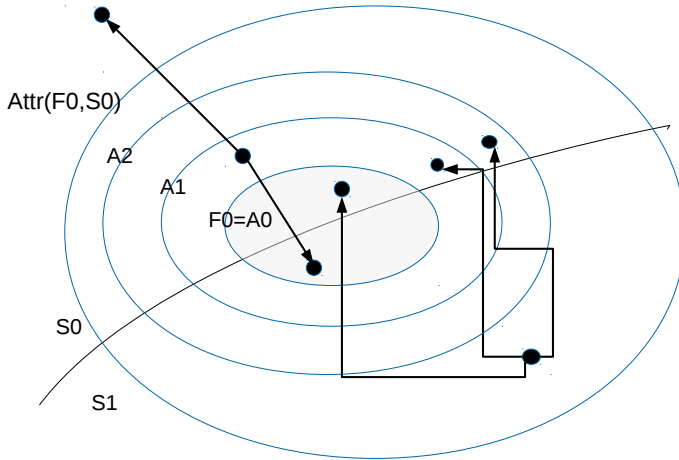
2. On définit une fonction $f : Attr(F_0, J_0) \setminus F_0 \rightarrow Attr(F_0, J_0)$ en posant :

- si $s \in S_0 \cap Attr(F_0, J_0)$, $f(s)$ est le successeur de rang strictement inférieur à celui de s et de plus petit indice ;
- si $s \in S_1 \cap Attr(F_0, J_0)$, $f(s)$ est le successeur de plus petit indice.

Soit $s_0 \in Attr(F_0, J_0)$. Construire une stratégie gagnante pour J_0 à partir de cette position. Montrer qu'il existe des parties conformes à cette stratégie. En déduire que s_0 est une position gagnante.

3. On considère $s_0 \notin \text{Attr}(F_0, J_0)$. Construire une stratégie gagnante pour J_1 à partir de cette position.

Corrigé 14.3 page 528.



Ce schéma est-il conforme à la construction des $(A_i)_i$?

14.3 Algorithme du minimax, heuristiques

Nous envisageons maintenant l'étude des jeux en déployant l'ensemble des parties possible comme un arbre. Un nœud représentant un état, les branchements, les différents coups permis au joueur qui contrôle cet état.

14.3.1 Arbres

Nous donnons deux définitions équivalentes de la structure d'arbre. La première les présente comme des graphes particuliers, la seconde les définit ex-nihilo.

Définition 14.9

Un arbre est un graphe orienté dans lequel

- *il existe un sommet et un seul, appelé **racine**, n'ayant pas d'antécédent ;*
- *tous les autres sommets admettent un prédécesseur et un seul.*

Définition 14.10

*On appelle arbre un ensemble $\mathcal{A}$ non vide, dont les éléments seront appelés **nœuds**, sur lequel est définie une relation notée xPy (« x est père-mère de y ») telle que :*

- *il existe un nœud, r , et un seul, appelé **racine**, n'ayant pas de père-mère (c'est-à-dire, tel que $\forall x \in \mathcal{A}, \text{non}(xPr)$);*
- *tous les autres nœuds admettent un(e) père-mère et un(e) seul(e) ($\forall y \in \mathcal{A}, \exists! x \in \mathcal{A}, (xPy)$);*

Propriété fondamentale : il existe un unique chemin qui nous permet de remonter d'un nœud quelconque jusqu'à la racine. Formellement :

pour tout nœud $x \in \mathcal{A} \setminus \{r\}$, il existe une suite finie $x_0 = r, \dots, x_p = x$ telle que pour tout i tel que $0 \leq i \leq p - 1$, $(x_{i-1} \mathcal{P} x_i)$ (on dit que les x_i sont les ascendants de x).

Nous serons amenés à utiliser le vocabulaire suivant :

- les nœuds sans descendant sont les *feuilles* de l'arbre ou nœuds terminaux ;
- le *degré* d'un nœud est le nombre de ses descendants ;
- la *profondeur* d'un nœud est le nombre de ses ascendants stricts ;
- la *hauteur* d'un arbre est la profondeur maximale de ses nœuds ;
- un arbre est *étiqueté* lorsqu'à chaque nœud on associe une information, appelée *étiquette*.

Questions : si un graphe $G = (S, A)$ est un arbre,

- comment définit on la relation $x \mathcal{P} y$?
- que sont les fils-filles d'un nœud ?
- qu'est-ce que le père-mère d'un nœud ?

Représentation des arbres finis

Nous pouvons implémenter les arbres comme nous l'avons fait pour les graphes, par listes d'adjacence, en précisant la liste des nœuds et en donnant la liste des arcs. Mais il est souvent plus intuitif d'adopter une définition récursive :

- une feuille est un arbre ayant un seul nœud : (racine, []);
- un arbre est un tuple (racine, liste de sous-arbres).

La classe tree définie en 14.5 page 535 implémente cette structure.

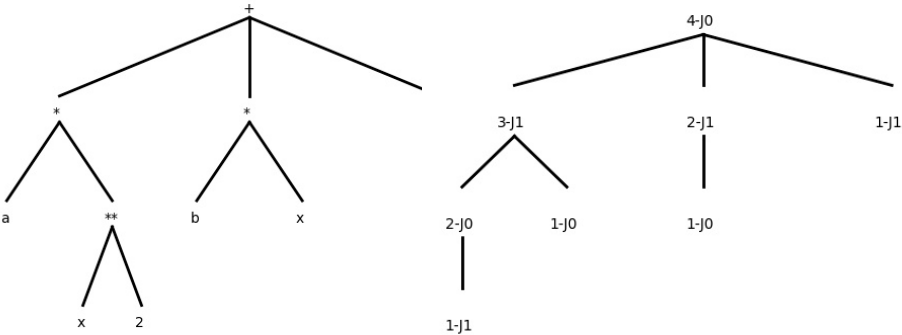
Elle est disponible sur le site accompagnant ce livre (et aussi sur www.univnligne.fr). Elle servira à implémenter les algorithmes de cette section.

Appel du constructeur : `tree(root: str, subtrees:list)`, `root` est l'étiquette de la racine, `sub_trees` est une liste de sous-arbres ou d'étiquettes qui seront transformées en arbres avant d'être ajoutées à la liste des sous-arbres.

attributs	
root	str (étiquette de la racine)
sub_trees	liste des sous arbres
méthodes	
add_sub_tree(s :tree)	ajoute s à sub_trees
add_sub_tree(s : string)	ajoute tree('s',[]) à sub_trees
is_leaf()	teste si l'arbre est une feuille
copy()	renvoie une copie de l'arbre (et des sous-arbres)

Montrons comment construire un arbre pour l'expression algébrique $ax^2 + bx + c$:

```
from arbres_jeux import tree
T1 = tree('*', ['a', tree('**', ['x', '2'])])
T2 = tree('*', ['b', 'x'])
T0 = tree('+', [])
T0.add_sub_tree(T1)
T0.add_sub_tree(T2)
T0.add_sub_tree('c')
```



Exercice 14.4 *arbre des parties pour le jeu de Nim*

Écrire une fonction récursive `arbre_Nim(N, j)` qui prend en arguments un entier $N > 1$ et $j \in \{0, 1\}$ et renvoie l'**arbre** des parties possibles dans le jeu de Nim débutant avec N jetons sur la table et le joueur j . On étiquettera les nœuds comme dans l'exercice 14.1 (ce qui donne l'arbre de la figure de droite).

Corrigé en 14.4 page 529.

Exercice 14.5 *arbres des parties pour oxo*

On veut ici construire un arbre de jeu pour le jeu d'oxo présenté page 510. On numérote les 9 cases du plateau de 0 à 8 comme sur la figure.

Une partie partielle est représentée par une chaîne dans laquelle les coups joués sont séparés par une ',' et représentés par des sous-chaînes 'O :i' ou 'X :i' (signifiant que J_0 ou J_1 a posé son jeton sur la case i).

Ainsi 'O :4, X :5, O :6, X :1,O :2' est **une** partie conduisant au plateau de la figure. On convient que le plateau vide est représenté par la chaîne vide.

2 O	5 X	8
1 X	4 O	7
0	3	6 O

1. Combien y a-t-il de parties conduisant à ce même état (ou plateau) ?
2. On décide de représenter un état par un dictionnaire dont les clés sont les indices de 0 à 8 et les valeurs un des trois caractères ' ', 'O' ou 'X'. Écrire une

fonction `partie_etat(partie ; string)` qui prend une chaîne représentant une partie partielle et renvoie le dictionnaire représentant l'état ou le plateau atteint.

On pourra construire le plateau vide avec

`d = {i : ' ' for i in range(0, 9)}`, puis le remplir.

3. On représente les 8 alignements possibles par les 8 listes des positions :

```
alignements = [[0, 3, 6], [1, 4, 7], [2, 5, 8]]
alignements += [[0, 1, 2], [3, 4, 5], [6, 7, 8]]
alignements += [[0, 4, 8], [2, 4, 6]]
```

```
def est_final(plateau):
    '''
    val = {' ': 0, 'O': 1, 'X': 4}
    for L in alignements:
        s = sum([val[???] for e in L])
        if s == 3:
            return ???
        elif s == 12:
            return ???
    # plateau plein?
    return ???
```

Compléter la fonction `est_final(plateau : dict)` qui prend en argument un dictionnaire plateau et renvoie :

- True, 0 si la partie est finie sur une victoire de J_0 ;
- True, 1 si la partie est finie sur une victoire de J_1 ;
- True, None si la partie est finie et nulle ;
- False, None si la partie n'est pas terminée.

4. Écrire une fonction récursive `arbre_oxo(label = "", j = 0)` qui construit en place et renvoie l'arbre du jeu. L'appel principal se fait avec les arguments par défaut, les nœuds seront étiquetés avec la chaîne représentant la partie partielle.

Ajouter un compteur pour dénombrer les feuilles de l'arbre, à savoir les parties finies (il y a 255 168 feuilles pour 549 946 nœuds).

5. Modifier cette fonction et écrire `arbre_oxo_1(label = "", d = partie_etat(""), j = 0)` qui prend en arguments une chaîne label représentant le dernier coup joué, d, un dictionnaire représentant le plateau courant, j le joueur qui contrôle, et renvoie un arbre du jeu dont les feuilles sont étiquetées avec le résultat de la partie : 'J0' ou 'J1' ou 'nul', les autres nœuds par le dernier coup joué avec indication du joueur qui contrôle le coup à jouer('0 :X-J1' par exemple).

Corrigé en 14.5 page 529.

Algorithme de parcours en profondeur

L'algorithme qui suit permet de parcourir l'arbre en profondeur d'abord (on appelle récursivement les fils du nœud traité). Trois types de traitement sont envisageables : **préfixe** (un nœud est traité avant le parcours des branches dont il est racine), **postfixe** ou **suffixe** (le traitement suit celui des fils), ou **terminal** (les feuilles sont traitées). A la lecture de l'algorithme il est clair qu'ils peuvent figurer en même temps et sont gérés par les fonctions Pref, Suff et Term.

Parcours des arbres finis

```
silence = lambda x : None

def parcours_arbre(T, Pref = silence,
                  Suff = silence,
                  Term = silence):
    assert isinstance(T, tree)
    Pref(T.root)
    for s in T.sub_trees:
        parcours_arbre(s, Pref, Suff, Term)
    Suff(T.root)
    if T.is_leaf():
        Term(T.root)
```

Exercice 14.6 méthodes pour les arbres

En vous inspirant de la procédure *parcours* écrire les fonctions suivantes, prouver leur correction, calculer leur complexité.

1. Une fonction qui calcule la *profondeur* de chaque nœud d'un arbre et ajoute à son étiquette la chaîne '(p)' où p est la profondeur..
2. Une fonction qui calcule la *taille* (le nombre de nœuds) d'un arbre.
3. Une fonction qui calcule la *hauteur* d'un arbre.
4. Une fonction qui retourne le *maximum* (ou le minimum) si les nœuds ont des valeurs numériques.
5. Une fonction qui teste l'*appartenance* à un arbre.

Corrigé en 14.6 page 531.

14.3.2 L'algorithme du minimax

L'algorithme du minimax

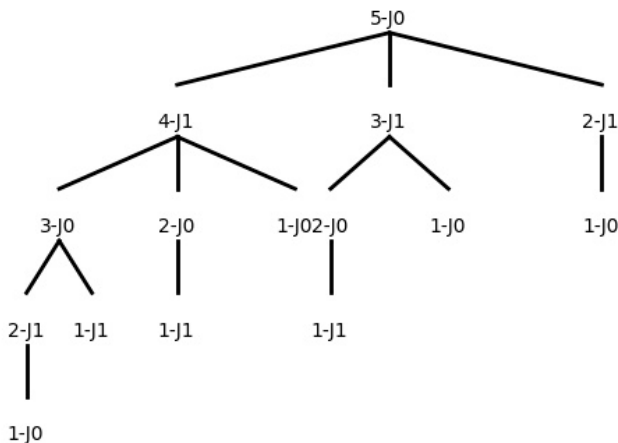
Avec cet algorithme, qui date de la première moitié du XX^{ième} siècle, on considère, lorsque la complexité du jeu le permet, l'ensemble des parties possibles.

Les feuilles d'un **arbre de jeu** représentent le résultat d'une partie (match gagné par J0, perdu par J0 ou nul), on leur attribue donc un score ou une valeur (+1, -1 ou 0 dans les cas où l'on ne considère que l'issue, +g, -g, 0 si les gains sont variables).

A partir de là, on définit récursivement **une valeur pour chaque nœud de l'arbre** :

- S'il est contrôlé par J0, ce sera le **maximum** des valeurs des sous-arbres ;
- S'il est contrôlé par J1, ce sera le **minimum** des valeurs des sous-arbres ;

Lorsqu'un nœud a une valeur positive pour le joueur J0, celui-ci peut imposer une partie conduisant à la victoire avec ce gain. Le score est calculé en supposant que l'adversaire joue tous ses coups de façon à minimiser ses pertes (ce sont les hypothèses d'information complète et de rationalité du joueur adverse : pas de ruse de guerre ou de proposition commerciale).



Exercice 14.7 mise en place, à la main

Ci-dessus, l'arbre de jeux pour une partie de Nim débutant avec 5 jetons, J0 jouant le premier.

1. Attribuer les valeurs aux nœuds en commençant par les feuilles (du point de vue de J0).

2. Si J_1 joue correctement (c'est l'hypothèse « joueur rationnel »), que se passera-t-il ?
3. On suppose J_1 novice, quels sont ses premiers coups qui permettraient à J_0 de gagner ?

Corrigé en 14.7 page 532.

Exercice 14.8 une fonction minimax pour le jeu de Nim

On suppose que l'arbre des parties possibles pour un jeu de Nim est déjà construit et implémenté avec la classe `tree` comme dans l'exercice 14.4. On le note T .

1. Les étiquettes des nœuds sont de la forme $'n - J_0', 'n - J_1'$ où n est le nombre de jetons et représente un état du jeu, J_j le joueur qui contrôle l'état. Quel critère choisir pour déterminer la valeur d'un nœud terminal pour le joueur J_j ?
2. Écrire une fonction `minimax_nim(T : tree, j : int)` qui prend en arguments un arbre de jeu, T , un entier $j \in \{0, 1\}$ qui représente un joueur et ajoute à l'étiquette de chaque nœud sa valeur donnée par l'algorithme minimax et renvoie la valeur de la racine de l'arbre. On prendra garde au fait que l'argument T est modifié en place.
3. Justifier que cette fonction termine (ce doit être la cas). Quelle est sa complexité ?

Corrigé en 14.8 page 532.

Exercice 14.9 mise en place plus générale

On suppose que les arbres de jeux considérés ici sont implémentés avec la classe `tree`. On veut écrire une fonction `minimax(T : tree, j : int, f : function)` implémentant l'algorithme minimax pour un arbre de jeu quelconque, T . $j \in \{0, 1\}$, un joueur, $f(\text{ch} : \text{string}, j : \text{int})$ une fonction de deux arguments qui permet d'évaluer les feuilles de l'arbre.

Préconditions :

- l'étiquette d'un nœud indique le joueur qui contrôle (contient `'J0'` ou `'J1'`);
- $f(T.\text{root}, j)$ détermine le gain pour le joueur j lorsque T est une feuille.

1. Écrire f dans le cas du jeu de Nim.
2. Écrire f dans le cas du jeu OXO (on suppose que les **feuilles** sont étiquetées `'J0'`, `'J1'`, **pour désigner le gagnant**, ou `'nul'`).
3. Écrire `minimax(T : tree, j : int, f : function)`

Ci-dessous, les nœuds de profondeur 1 pour l'arbre de jeu de oxo, réétiquetés par l'algorithme minimax. `'O :0-J1-(0)'` signifie que J_0 a placé son jeton `'O'` sur la case 0, que J_1 contrôle le plateau et que le score calculé par minimax est de 0.

```

>>> T = arbre_oxo_1(label = '', j = 0)
>>> T1 = T.copy()
>>> minimax(T1, 0, f_oxo)
>>> for t in T1.sub_trees:
        print(t.root)

O:0-J1-(0)
O:1-J1-(0)
O:2-J1-(0)
O:3-J1-(0)
O:4-J1-(0)
O:5-J1-(0)
O:6-J1-(0)
O:7-J1-(0)
O:8-J1-(0)

```

Ce résultat (très partiel) pour le jeu oxo alias tic-tac-toe ou morpion nous indique toutefois **que le joueur qui commence ne peut espérer qu'un match nul si son adversaire ne commet pas d'erreur**. Signalons toutefois que sur les 255 168 parties possibles, 131184 conduisent à la victoire de J_0 et 77904 à celle de J_1 , il reste 46 080 possibilités de nul (on pourra placer des compteurs pour vérifier cela).

Corrigé en 14.9 page 533.

14.3.3 L'algorithme du minimax avec heuristiques

Nous avons vu qu'avec le jeu oxo et ses 9 cases à remplir, plus de 250 000 parties sont possibles et que l'arbre de jeux comporte plus de 550 000 nœuds que l'algorithme du minimax devra tous traiter. Pour certains jeux, comme les échecs, le jeu de Go, construire l'arbre du jeu et le parcourir entièrement est inenvisageable. L'algorithme minimax tel quel n'est donc plus opérationnel.

Une première idée est de ne construire ou de n'explorer qu'une partie de l'arbre. On ne s'intéresse qu'à des positions ou états du jeu qui semblent plus favorables, on se limite par ailleurs à une exploration de profondeur limitée.

Il s'agit là de méthodes approchées qui ne garantissent pas le choix optimal mais qui fournissent des solutions qui à l'usage se révèlent acceptables, voire très efficaces. On parle de **méthodes heuristiques** ou, par abus d'**heuristiques** lorsque (la preuve de) leur optimalité n'est pas établie.

L'algorithme du minimax avec fonction d'évaluation

On construit ou explore l'arbre du jeu à partir d'une position donnée en limitant la profondeur des sous-arbres à explorer (3 coups, 5 coups d'avance...). L'algorithme explore donc un **arbre de jeu partiel** dont les feuilles ne sont plus nécessairement des fins de parties.

On se donne une **fonction d'évaluation** qui est susceptible d'attribuer une valeur à tous les états du jeu. Aux échecs, elle dépendra de la nature des différentes pièces, de leurs positions (l'occupation du centre de l'échiquier ayant une meilleure valeur stratégique), etc. L'algorithme n'est pas modifié dans sa conception : il opère sur un arbre de jeu partiel et sa fonction d'évaluation est heuristique.

Voir par exemple la référence [16].

Exercice 14.10 *minimax avec profondeur limitée et heuristique*

On suppose comme nous l'avons toujours fait que les étiquettes de l'arbre de jeu sont de la forme 'état-J0' ou 'état-J1'.

Quelles sont les modifications mineures à apporter à l'algorithme du minimax pour explorer à hauteur limitée ? Les mettre en place avec une procédure récursive `minimax_profondeur(T, j, f, pmax)`.

Corrigé en 14.10 page 534.

14.4 Corrigés des exercices

Corrigé de l'exercice n° 14.1

1. Ce graphe est clairement celui de la page 508. L'étiquette contenant J0 correspond aux sommets non grisés.
2. Voir la ligne correspondante dans le script complet ci-dessous..
3. (a) Les dictionnaires et listes qui sont envoyés en arguments d'une fonction ou procédure sont modifiés par la fonction. Après les appels récursifs, les variables *S* et *A* de l'appel principal contiennent donc l'ensemble des sommets et des arcs.
 - (b) Chaque état est **caractérisé** par le couple (N, J_i) donc par son étiquette. Placer ces étiquettes comme clés d'un dictionnaire permet de vérifier facilement que l'état a déjà été rencontré ou pas.
 - (c) Chaque étiquette n'est ajoutée qu'une fois au dictionnaire *S* et si l'indice qui leur est attribué est la longueur de *S* à cet instant là, dans # (4) comme dans l'instruction conditionnelle # (1), la correspondance restera bijective.


```

# préparation
Attr, pred, hmr = [], {}, {}
for s in S:
    pred[s] = []
    hmr[s] = 0

for a in A:
    u, v = tuple(a)
    hmr[u] += 1
    pred[v].append(u)

for x in F0:
    marquer_propager(x)
return Attr

```

4. Preuve de correction : Ce qui est clair :

- les éléments de F_0 sont ajoutés à $Attr$ au moment des appels principaux ;
- les sommets sont ajoutés dans `marquer_propager()` exclusivement ;
- un élément n'est ajouté qu'une fois à la liste $Attr$.

- $Attr \subset W_0$. Il suffit de montrer que pour tout appel `marquer_propager(u)`, u est dans W_0 . On procède par récurrence sur la profondeur h de l'appel récursif. Pour les appels principaux ($h = 0$), les sommets sont dans F_0 , donc dans W_0 . Supposons qu'après un appel de hauteur h on ait $Attr \subset W_0$. Pour les appels récursifs induits,

- soit u est un sommet contrôlé par J_0 et prédécesseur d'un élément de $Attr$, c'est bien un élément de W_0 ;
- soit $hmr[u] == 0$. Or la décrémentation de $hmr[u]$ n'a lieu que lorsqu'on rencontre **un successeur dans $Attr$** . La condition $hmr[u] == 0$ signifie que **tous** les successeurs ont été rencontrés et que tous étaient dans $Attr$.

- $W_0 \subset Attr$ Considérons à nouveau la suite $(A_i)_i$ définie page 513 et qui converge vers l'attracteur. Montrons par récurrence sur i que $A_i \subset Attr$. C'est clair lorsque $i = 0$.

Supposons que $A_i \subset Attr$. Chaque $u \in A_i$ a été ajouté lors d'un appel `marquer_poursuivre(u)` lors duquel tous ses prédécesseurs ont été examinés. Donc tous les éléments de A_{i+1} qui sont des prédécesseurs d'éléments de A_i auront été examinés au moins une fois.

5. Complexité :

Les deux boucles dans la fonction conduisent à $|S| + |A|$ itérations. Le nombre des appels récursifs `marquer_propager(u)` qui ne renvoient pas `None` immédiatement est majoré par $|W_0| \leq |S|$.

Le nombre d'itérations dans la boucle de `marquer_propager(s)` est le **degré entrant** de s . Chaque $s \in W_0$ conduit à une boucle au plus, les arcs (u, s) correspondants

sont visités au plus une fois. Le nombre total de ces itérations est donc majoré par $|A|$.

Corrigé de l'exercice n° 14.3

1. Les propositions (b) et (d) sont seules vraies.
2. • On construit une stratégie sans mémoire pour $J_0 : \sigma : \mathcal{P}_0 \longrightarrow S_1$ de la façon suivante :

Pour toute partie partielle $\pi = (s_0, s_1, \dots, s_{p-1})$ telle que $s_{p-1} \in S_0$, position jouable pour J_0 ,

- Si s_{p-1} est dans l'attracteur, on pose $\sigma(\pi) = f(s_{p-1})$.
- Sinon, on choisit pour $\sigma(\pi)$ un successeur quelconque.

• Supposons alors que $s_0 \in \text{Attr}(F_0, J_0)$. On définit une suite en posant $s_{k+1} = f(s_k)$ tant que s_k est dans le domaine de f . Cette suite a tous ses éléments dans l'attracteur. Elle définit une partie partielle conforme à la stratégie σ . Son dernier terme s'il existe est dans F_0 .

• Dans une partie conforme à la stratégie σ la suite des rangs des s_k est strictement décroissante. En effet, lorsque $s_k \in S_1$ **tous** ses successeurs ont un rang strictement inférieur d'après (b)-(iii). Lorsque $s_k \in S_1$, $f(s_k)$ est de rang strictement inférieur par définition de f .

La suite des rangs atteint donc 0 et la partie termine sur un sommet de rang zéro, c'est à dire dans F_0 . **Toutes les parties conformes partant de s_0 sont donc gagnantes et il en existe au moins une.**

3. Les éléments qui ne sont pas dans l'attracteur W_0 ont un rang infini.

On définit $g : S \setminus W_0 \longrightarrow S \setminus W_0$ en posant :

- si $s \in S_0$, $g(s)$ est son successeur de plus petit indice ;
- si $s \in S_1$, $g(s)$ est son successeur de rang infini et de plus petit indice ;

Cela a un sens d'après (d).

On définit une stratégie ν pour J_1 en posant $\nu((s_0, s_1, \dots, s_{k-1})) = g(s_{k-1})$ lorsque $s_{k-1} \in S_1$.

On démontre comme précédemment qu'il existe une partie conforme au moins. Il est clair par ailleurs que toute partie conforme reste en dehors de F_0 . Donc J_0 ne gagne pas et c'est J_1 qui gagne.

Corrigé de l'exercice n° 14.4

```
def arbre_Nim(N, j):
    '''
    N :int, j = 0 ou 1.
    '''
    label = '%s-J%s'%(N,j)
    T      = tree(label, [])

    for i in [1,2,3]:
        if N-i > 0:
            t = arbre_Nim(N-i, 1-j)
            T.add_sub_tree(t)
    return T
```

Corrigé de l'exercice n° 14.5

1. Observons que pour atteindre cet état, 3 coups ont été joués par 'O' et deux par 'X', ainsi $3! \times 2! = 12$ parties peuvent y conduire.

2. On représente les parties par des chaînes de caractères, les états ou plateaux par des dictionnaires.

```
def partie_etat(ch):
    ''' ch = 'O: i0, X: i1, ...' '''

    ch = ch.strip()
    d = {i : ' ' for i in range(0,9)}
    if ch == '':
        return d
    CH = ch.split(',')
    for e in CH:
        e = e.strip()
        E = e.split(':')
        i = int(E[1])
        if i in d and d[i] == ' ' and E[0] in ['O', 'X']:
            d[i] = E[0]
        else:
            return None
    return d
```

3. Sur une ligne, les sommes possibles sont données par le tableau :

OOO	XXX	—	OOX	OO-	XXO	XX-	-O	-X
12	3	0	9	8	6	2	4	1


```
def est_final(plateau):
    val = {' ': 0, 'O': 1, 'X': 4}
    for L in alignements:
        s = sum([val[plateau[e]] for e in L])
        if s == 3:
            return True, 0
        elif s == 12:
            return True, 1
    # plateau plein?
    return len([i for i in plateau if plateau[i] != ' ']) == 9, \
        None
```

4. Avec un compteur pour les feuilles, c'est à dire le nombre de parties :

```
def arbre_oxo(label = '', j = 0):
    ''' label: string, représente une partie partielle '''
    global compteur
    jeton = {0: 'O', 1: 'X'}
    plateau = partie_etat(label)
    T = tree(label, [])
    r, w = est_final(plateau)
    if r:
        compteur += 1
    for i in plateau:
        if plateau[i].strip() == '':
            if label == '':
                suiv = label + '%s:%s'%(jeton[j], i)
            else:
                suiv = label + ', %s:%s'%(jeton[j], i)
            T.add_sub_tree(arbre_oxo(suiv, 1-j))
    return T
```

5.

```
def arbre_oxo_1(label = '', d = partie_etat('') , j = 0):
    '''
    label: string, dernier coup joué;
    d : dict représente un plateau.
    '''
    jeton = {0: 'O', 1: 'X'}
    r, w = est_final(d)

    if r and w == 0:
        return tree('J0', [])
    elif r and w == 1:
        return tree('J1', [])
    elif r:
        return tree('nul', [])
```

```

else:
    T = tree(label + '-J'+ str(j), [])
    for i in d:
        if d[i].strip() == '':
            last = '%s:%s'%(jeton[j], i)
            d1 = d.copy()
            d1[i] = jeton[j]
            T.add_sub_tree(arbre_oxo_1(last, d1, 1-j))
return T

```

Corrigé de l'exercice n° 14.6

Dans l'ordre, les 5 scripts demandés. Certaines de ces fonctions ont vocation à être implémentées comme méthodes de la classe tree. Voir le script arbres_jeux.py.

```

def profondeurs(T, p = 0):
    T.root += ' (%s)'%p
    for s in T.sub_trees:
        profondeurs(s, p+1)

def taille(T):
    s = 0
    for S in T.sub_trees:
        s += taille(S)
    return s + 1

def hauteur(T):
    if T.is_leaf():
        return 0
    else:
        return 1+ max([hauteur(s) for s in T.sub_trees])

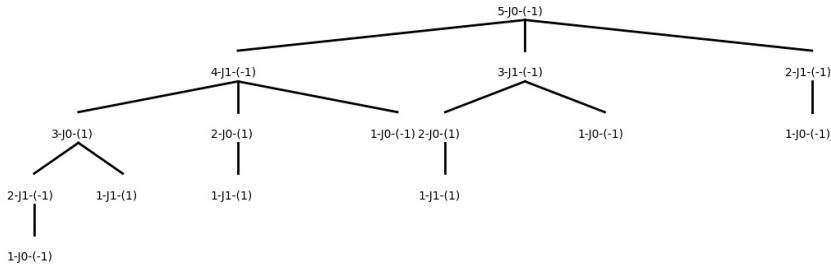
def noeud_max(T):
    if T.is_leaf():
        return T.root
    else:
        return max([noeud_max(s) for s in T.sub_trees])

def est_un_noeud(T,x):
    if x == T.root:
        return True
    else:
        for S in T.sub_trees:
            if est_un_noeud(S,x):
                return True
        return False

```

Corrigé de l'exercice n° 14.7

1. Le résultat (avec le programme de l'exercice qui suit).



2. Il choisira systématiquement les états de score -1 (dont il sait qu'ils le conduiront à coup sûr à d'autres possibilités de choisir des états de même score) et gagnera.

3. J1 ne peut commettre d'erreurs que dans les branches 4-J1(-1) et 3-J1(-1) en jouant les coups correspondants aux sous-branches de gauche.

Corrigé de l'exercice n° 14.8

1. Un nœud terminal au jeu de Nim correspond à un état gagnant pour J si son adversaire le contrôle. On testera donc la présence de la chaîne `jc = '-J%s'%(j)` ('J0' ou 'J1') selon la valeur de `j`.

2. Ce qui se complète avec :

```
def minimax_nim(T, j):
    '''
    T : tree, arbre de jeu;
    j : 0|1, joueur;
    Précondition: les noeuds sont étiquetés par
    une chaîne donnant l'état et le joueur qui
    contrôle.
    '''
    jc = '-J%s'%(j) #---

    if T.is_leaf() and jc in T.root:
        v = -1

    elif T.is_leaf() and jc not in T.root:
        v = +1

    elif jc in T.root:
        #c'est J qui contrôle le noeud
        v = max([minimax_nim(S, j) for S in T.sub_trees])

    else:
        #c'est l'autre joueur qui contrôle le noeud
        v = min([minimax_nim(S, j) for S in T.sub_trees])

    T.root += '-(%s)'%(v)
    return v
```

3. La fonction termine : on le prouve sur la hauteur de l'arbre T .

- C'est clair si $h = 0$ (feuilles) ;
- Si tout appel avec des arbres de hauteur $\leq h$ termine, il en sera de même avec un arbre de hauteur $h+1$ dans lequel tous les appels récursifs se font avec des sous-arbres de hauteurs $\leq h$.

Chaque nœud est visité une fois et une seule. Le nombre de calculs de score est (feuilles comprises) égal au nombre de nœuds.

Corrigé de l'exercice n° 14.9

Les scripts pour les questions 1., 2. et 3. :

```
def f_nim(ch, j):
    '''
    ch : string;
    j = 0|1.
    Pré condition : ch = 'n-Ji' avec n:int et j = 0 ou 1.
    '''
    assert '-' in ch
    CH = ch.split('-')
    assert CH[1] == 'J0' or CH[1] == 'J1'

    n = int(CH[0])
    if n==1 and 'J'+ str(j) == CH[1]:
        return -1
    elif n==1 and 'J'+ str(1-j) == CH[1]:
        return 1
```

```
def f_oxo(label, j):
    Jc = '-J'+str(j) # joueur courant
    Ja = '-J'+str(1-j) # adversaire
    if Jc in label:
        return 1
    elif Ja in label:
        return -1
    else:
        # match nul
        return 0
```

```
def minimax(T, J, f):
    jc = '-J'+ str(j)

    if T.is_leaf():
        v = f(T.root, J)
    elif jc in T.root:
        #c'est J qui contrôle le noeud
        v = max([minimax(S, J, f) for S in T.sub_trees])
    else:
        #c'est l'autre joueur qui contrôle le noeud
        v = min([minimax(S, J, f) for S in T.sub_trees])
    T.root += '-(%s)'%v
    return v
```

Corrigé de l'exercice n° 14.10

La condition d'arrêt portera sur l'argument pmx, décrémenté à chaque appel récursif. Il donne ainsi la hauteur du nœud à considérer.

```
def minimax_profondeur(T, j, f, pmax):
    '''
    T      : tree, arbre de jeu;
    j      : 0|1, joueur;
    f      : fonction f:(label, j)--> int;
    pmax   : int, profondeur max.

    Précondition: les noeuds sont étiquetés par
    une chaîne donnant l'état et le joueur
    qui contrôle le noeud ('xxx-J0', 'xxx-J1')
    '''
    jc = '-J'+ str(j)

    if T.is_leaf() or pmax == 0:
        v = f(T.root, j)

    elif jc in T.root:
        #c'est J qui contrôle le noeud
        v = max([minimax_profondeur(S, j, f, pmax-1)\
                  for S in T.sub_trees])

    else:
        #c'est l'autre joueur qui contrôle le noeud
        v = min([minimax_profondeur(S, j, f, pmax-1)\
                  for S in T.sub_trees])

    T.root += '-(%s)'%v
    return v
```

14.5 Une classe pour représenter les arbres

```

class tree:

    def __init__(self, root, sub_trees):
        '''
        root      : string (étiquette de la racine);
        sub_trees : list of tres (sous-arbres);
        '''
        self.my_type = 'tree'
        self.root     = root
        self.sub_trees = []
        for sub_tree in sub_trees:
            self.add_sub_tree(sub_tree)

    def add_sub_tree(self, s):
        '''
        s : string | tree.
        Ajoute tree(s,[]) ou une copie de s comme sous-arbre.
        Renvoie None.
        '''
        if isinstance(s, tree):
            self.sub_trees.append(s.copy())
        else:
            self.sub_trees.append(tree(s, []))

    def is_leaf(self):
        return len(self.sub_trees) == 0

    def hight(self):
        if self.is_leaf():
            return 0
        else:
            return 1+max([s.hight() for s in self.sub_trees])

    def copy(self):
        '''
        Renvoie une copie distincte.
        '''
        my_copy = tree(self.root, [])
        for s in self.sub_trees:
            my_copy.add_sub_tree(s.copy())
        return my_copy

```


Algorithmes pour l'étiquetage et la classification

15.1 Vocabulaire et définitions

Nous allons, pour l'essentiel, dans ce chapitre exposer des méthodes de classement et de classification automatique. Nous commençons par en préciser la problématique et les notions sous-jacentes.

15.1.1 Classement et classification automatique

On rappelle la définition d'une partition qui intervient dans les questions de classement et de classification.

Définition 15.1 *partitions*

Soit E un ensemble.

1. Une **partition** de E est la donnée d'une famille $(A_i)_i$ de parties ou sous-ensembles **non-vides** de E , telle que :
$$\begin{cases} \bigcup_i A_i &= E \\ A_i \cap A_j &= \emptyset, \text{ pour tous } i \neq j \end{cases}$$
2. Soient $(A_i)_{i \in I}$ et $(B_j)_{j \in J}$ deux partitions du même ensemble E . On dit que $(B_j)_j$ **est plus fine** que $(A_i)_i$ ssi $\forall j \in J, \exists i \in I, B_j \subset A_i$.

• Relations d'équivalence

Il est souhaitable, lorsqu'on aborde ces problèmes d'avoir en tête le lien qui existe entre partitions et relations d'équivalence :

- Si R est une relation d'équivalence sur un ensemble E , ses classes d'équivalence forment une partition de E .
- Pour toute partition P de E il existe une relation d'équivalence sur E telle que P est l'ensemble des classes d'équivalence de R . Elle peut trivialement se définir par « $R(x, y)$ ssi x et y appartiennent à un même élément de P », mais c'est évidemment

une autre formulation qui fera sens en pratique.

• **Classer, étiqueter** les éléments d'un ensemble (documents écrits, images, individus, lieux géographiques, événements ou données quelconques) revient à placer ces éléments dans des classes qui forment une **partition** de cet ensemble. Dans une autre formulation, cela revient à associer à chaque élément une **étiquette** appartenant à une liste prédéfinie (les éléments d'une même classe se voyant affublés d'une même étiquette).

Les propriétés des partitions assurent que l'opération de classement d'un élément e est toujours définie de façon unique. En d'autres termes, l'association $e \rightarrow C$ est une application.

• **Classifier** est l'opération qui vise à définir une partition ou un système de partitions pour un ensemble de données. On imagine sans peine qu'en pratique on visera à obtenir une classification dans laquelle les individus appartenant à une même classe seront le plus ressemblants/proches possibles, les individus appartenant à des classes distinctes devront être le moins ressemblants/proches possible.

Cette notion de **ressemblance** ou de **proximité** dépendra du type des données et nous serons amenés à considérer des distances ou des mesures de similarité adaptées à chaque problème.

Exercice 15.1 reconnaître une partition

Nous allons, dans ce chapitre, présenter un algorithme de classification qui aura à produire des partitions d'un ensemble. On souhaite construire au préalable un outil de débogage qui vérifie qu'une structure est une partition d'un ensemble (fini) E .

On veut écrire une fonction `is_partition(P, E)` qui prend en arguments une liste de listes P , une liste (ou un ensemble) E , et qui renvoie `True` ssi P est une partition de E , `False` sinon.

On suppose que les éléments de E et des sous-listes de P peuvent être clés dans un dictionnaire (*ie* sont **hashables**) et on propose le schéma de fonction suivant :

1. Réserver un dictionnaire D .
2. Pour chaque élément P_k de P (P_k est donc lui-même une liste) :
 - Si P_k est vide, afficher un message d'erreur approprié et en tirer les conséquences.
 - Sinon, pour $i \in P_k$,
 - (a) Si i n'appartient pas à E , afficher le message d'erreur approprié et en tirer les conséquences.
 - (b) Si i ne figure pas déjà dans le dictionnaire, faire $D[i] = k$.
 - (c) Sinon, si i figure déjà dans le dictionnaire, il y a deux possibilités. Afficher le message d'erreur approprié dans chaque cas et en tirer les conséquences.

Écrire la fonction, déterminer sa complexité, prouver sa correction.
Corrigé en 15.1 page 557.

15.1.2 Distances, similarités

On propose quelques exemples de mesures de similarité entre éléments d'un ensemble en vue de leur classification. On commence par rappeler ce qu'est une distance en mathématiques.

Définition 15.2 *distance (mathématiques)*

Soit E un ensemble. Une distance sur E est une application de $E \times E$ dans $\mathbb{R}^+$ telle que

1. $d(x, x) = 0$ et $d(x, y) = 0 \Rightarrow x = y$ (axiome de séparation) ;
2. $d(x, y) = d(y, x)$ (symétrie) ;
3. $d(x, y) \leq d(x, z) + d(z, y)$ (inégalité triangulaire).

On peut en particulier définir une distance sur (une partie d')un espace vectoriel à partir d'une norme en posant $d(X, Y) = \mathcal{N}(X - Y)$. Ainsi, dans $\mathbb{R}^n$,

$$d(X, Y) = \|X - Y\|_1 = \sum_{i=1}^n |x_i - y_i| \quad (15.1.1)$$

$$d(X, Y) = \|X - Y\|_2 = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (15.1.2)$$

$$d(X, Y) = \|X - Y\|_\infty = \max_{1 \leq i \leq n} |x_i - y_i| \quad (15.1.3)$$

Mesures de similarité

Nous utilisons ces distances usuelles dans les problèmes numériques ou qui ont une modélisation géométrique, mais elles ne sont pas adaptées à toutes les situations. Nous ferons alors intervenir d'autres distances ou, à défaut, des mesures ou indices de similarité dont nous donnons ici quelques exemples.

• Indice et distance de Jaccard

On définit *a priori* l'**indice de Jaccard** comme une mesure de similarité entre deux ensembles finis A et B (l'un d'eux étant non vide) en posant

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \text{ qui a les propriétés } \begin{cases} 0 \leq J(A, B) \leq 1 \\ J(A, B) = 1 \Leftrightarrow A = B \\ J(A, B) = 0 \Leftrightarrow A \cap B = \emptyset \end{cases}$$

Nous pouvons lui associer la **distance de Jaccard**, définie par

$$d_J(A, B) = 1 - J(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|} = \frac{|A \Delta B|}{|A \cup B|}.$$

où $A \Delta B$ désigne la différence symétrique entre A et B : $A \Delta B = (A \cup B) \setminus (A \cap B)$.

Cet indice intervient lorsque nous voulons comparer des **éléments** caractérisés par des attributs booléens/binaires (présence ou non de certaines propriétés). Supposons que nous voulions comparer des individus selon qu'ils possèdent ou non des propriétés P_i . Chaque individu est représenté par un vecteur de $\{0, 1\}^n$ avec $x_i = 1$ ssi $P_i(x)$:

$$X = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} \leftarrow \text{ne vérifie pas } P_2, Y = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} \leftarrow \text{vérifie } P_2.$$

La distance de Jaccard est nulle lorsque les vecteurs X et Y sont identiques, égale à 1 lorsque $\forall i, x_i \neq y_i$.

L'exercice 15.9 page 553 montre comment utiliser la distance de Jaccard pour comparer deux partitions d'un ensemble de données.

• Comparaisons de textes, matrices termes-documents

Pour évaluer la proximité entre documents textes issus d'un même corpus on commence par en donner une représentation sommaire en repérant les **mots utiles** du corpus et en associant à chaque document D_j un vecteur colonne X^j dont le terme $x_{i,j}$ dépend de la fréquence du mot ou du terme t_i dans le document et dans le corpus $\mathcal{C}$. Le nombre d'occurrences du terme pourrait servir à comparer grossièrement des documents de même taille, mais on préfère le plus souvent le codage tf-idf (tf : fréquence ou plutôt nombre d'occurrences des termes dans les documents, idf : inverse du nombre de documents contenant un terme) défini par les formules

$$x_{i,j} = tf_{t_i,d_j} \times \ln \frac{|\mathcal{C}|}{df_{t_i}} \text{ avec } \begin{cases} tf_{t_i,d_j} & : \text{fréquence de } t_i \text{ dans } D_j \\ df_{t_i} & : \text{nombre de docs. contenant } t_i \end{cases} \quad (15.1.4)$$

Un indice de similarité entre deux documents est alors donné par la **similarité cosinus** définie comme le cosinus de l'angle entre X^j et X^k comme vecteurs de $\mathbb{R}^n$ avec n taille du lexique retenu (de quelques milliers à quelques dizaines de milliers de termes en pratique) :

$$\text{sim}(D_j, D_k) = \frac{\sum x_{i,j} x_{i,k}}{\|X^j\|_2 \|X^k\|_2} = \frac{(X_j | X_k)}{\|X^j\|_2 \|X^k\|_2} \quad (15.1.5)$$

où $(X_j | X_k)$ désigne le produit scalaire canonique dans $\mathbb{R}^n$.

L'exercice qui suit permet de vérifier que deux documents similaires ont un indice de similitude proche de 1, que deux documents très différents ont un indice de similitude proche de 0.

Exercice 15.2 *similarité cosinus*

On suppose que les documents d'un corpus sont représentés comme en (15.1.4) par des vecteurs X^j tels que $x_{i,j} = tf_{t_i,d_j} \times \ln \frac{|C|}{df_{t_i}}$.

1. On suppose que le mot ou le terme t_i est présent dans tous les documents. Que vaut alors $x_{i,j}$?
2. On suppose que le mot ou le terme t_i est présent dans le seul document D_j . Que vaut alors $x_{i,j}$?
3. On pose $\delta(D_j, D_k) = 1 - \text{sim}(D_j, D_k)$.
Que dire des documents D_j et D_k tels que $\delta(D_j, D_k) = 0$? Penser au cas d'égalité dans la formule de Cauchy-Schwarz de votre cours de maths par exemple. S'agit-il d'une distance ?
4. δ vérifie-t-elle l'inégalité triangulaire ?

Indication : on pourra réécrire une relation $\delta(X, Z) \leq \delta(X, Y) + \delta(Y, Z)$ en introduisant les notations

$$\cos \alpha = \frac{(X|Y)}{\|X\|_2, \|Y\|_2}, \cos \beta = \frac{(Y|Z)}{\|Y\|_2, \|Z\|_2}, \cos \gamma = \dots$$

puis s'aider d'un dessin (on se souviendra que trois vecteurs de $\mathbb{R}^{+,n}$, même avec $n \geq 10^5$, sont coplanaires).

5. Que peut on dire de deux documents pour lesquels $\delta(D_j, D_k) = 1$?

Corrigé en 15.2 page 557.

En pratique vous serez amenés à définir vos propres mesures de similarité/dissimilarité. Si par exemple vous souhaitez comparer des cv professionnels, des profils de réseaux sociaux, vous pourrez commencer par construire un modèle avec des attributs booléens, puis définir une « distance » en panachant indice de Jaccard portant sur ces attributs, similarité cosinus sur le contenu textuel du cv ou des tweets.

15.1.3 Inertie d'une partition

On définit l'inertie d'une partie finie de $\mathbb{R}^n$, puis d'une partition d'un ensemble, notion essentielle dans l'**évaluation des algorithmes de classification**.

Définition 15.3 *variance ou inertie, cas d'une partition*

1. Si $A = (X_j)_{0 \leq j \leq m-1}$ est une partie finie de $\mathbb{R}^n$, le **barycentre** et l'**inertie** de

A sont définis par :

$$G_A = \frac{1}{m} \sum_{j=0}^{m-1} X_j \quad (\text{le barycentre de la famille}) \quad (15.1.6)$$

$$I_A = \sum_{j=0}^{m-1} \|X_j - G_A\|_2^2 \quad (\text{l'inertie de la famille}) \quad (15.1.7)$$

On appelle aussi variance de A , la moyenne $\frac{I_A}{|A|}$.

2. Soit $(A_k)_{0 \leq k \leq K-1}$ une partition d'un ensemble fini E contenu dans $\mathbb{R}^n$, on appelle **inertie totale de la partition** la somme des inerties de chacune de ses classes :

$$I = \sum_{k=0}^{K-1} I_{A_k} = \sum_{k=0}^{K-1} \sum_{X_j \in A_k} \|X_j - G_{A_k}\|^2$$

Exercice 15.3 calcul de l'inertie d'une partition

Dans la même optique que dans l'exercice 15.1, à savoir tester des algorithmes de classification, on souhaite écrire une fonction qui calcule l'inertie d'une partition d'un ensemble fini de $\mathbb{R}^n$.

Les données à traiter seront stockées dans une matrice $X \in \mathcal{M}_{N,M}(\mathbb{R})$, la colonne X^j de X représentant l'élément j de l'ensemble E . Cette matrice est définie dans numpy par `X = numpy.matrix((N,M), ...)`

1. Écrire une fonction `iso_barycentre(X)` qui prend en argument une matrice X et renvoie la matrice colonne dont le terme $G[i, 0]$ est la moyenne des termes $X[i, :]$.

indication : `np.sum(M)` renvoie la somme des termes de M .

2. Écrire une fonction `inertie(P,X)` qui prend en argument P , une liste de listes d'entiers et X , une matrice de flottants.

Pré-condition : P est une une partition des indices des colonnes de X .

Post-condition : `inertie(P,X)` est égal à l'inertie de P vue comme partition de E .

On rappelle que l'on peut extraire les colonnes d'une matrice de numpy de la façon suivante :

```
>>> X = np.matrix(np.ones((4,4))); X[2,:] = [0,2,4,6]
>>> X[:, [1,2] ]
[[1. 1.]
 [1. 1.]
 [2. 4.]
 [1. 1.]]
```

Corrigé en 15.3 page 558.

Un étude détaillée de l'inertie est proposée dans l'exercice 15.7 page 551 où l'on démontre une propriété fondamentale de l'algorithme des k-moyennes. Le script de l'exercice qui précède intervient pour sa part dans les tests et comparaisons présentés page 555.

15.1.4 A propos d'intelligence artificielle

Qu'est-ce que l'intelligence artificielle ?

Nous ne connaissons pas de définition satisfaisante de l'intelligence artificielle. Il existe toutefois un consensus pour rattacher certains algorithmes, méthodes ou programmes à ce domaine en considérant tantôt le problème que l'on tente de résoudre, tantôt les techniques mises en œuvre. Le problème peut être d'une complexité telle qu'il est inenvisageable d'appliquer une méthode exacte de bout en bout ou, tout simplement, ne pas admettre de formulation précise.

Caractérisation par les problèmes

- reconnaissance de formes ;
- traitement du langage ;
- **classement, étiquetage automatique ;**
- **classification automatique, clustering ;**
- certains programmes de jeux dans lesquels interviennent des **heuristiques...**

Caractérisation par les méthodes

- systèmes experts ;
- programmation logique ;
- calculs bayésiens ;
- **apprentissage supervisé ;**
- **apprentissage non supervisé ;**
- apprentissage profond (réseaux de neurones) ;

Les problèmes et méthodes en gras sont ceux que nous abordons dans ce chapitre. Nous avons déjà expliqué ce qu'étaient le classement-étiquetage et la classification (ou clustering en anglais). Précisons les notions d'apprentissage automatique-statistique (machine learning) supervisé ou non supervisé.

Apprentissage supervisé

La problématique de l'apprentissage supervisé est la suivante :

- On dispose de données déjà classées ou étiquetées (s'il s'agit d'un problème de classement) ou de données sous la forme d'entrées-sorties. C'est l'**ensemble d'apprentissage**.
- On souhaite pour des données nouvelles du même type, attribuer une étiquette ou une valeur de sortie en se basant sur ce que l'on sait déjà grâce à l'ensemble d'apprentissage.

D'une façon générale, on se donne un modèle **pour la fonction de prédiction** f : entrées $\rightarrow$ sorties, et on cherche à approcher (les paramètres de) f à partir des couples d'entrées-sorties fournis par l'ensemble d'apprentissage. Les différentes méthodes reposent sur des raisonnements probabilistes ou statistiques. Nous illustrons cela dans le paragraphe qui suit.

Apprentissage non supervisé

On parle d'apprentissage non supervisé lorsqu'il s'agit de **découvrir** les paramètres d'un **modèle de structure sous-jacente** à un ensemble de données **en l'absence de connaissance *a priori***. Nous illustrons cela en 15.3.

15.2 Classement supervisé, k-plus proches voisins

15.2.1 L'algorithme

Algorithme des k-plus proches voisins

Problématique :

On veut classer ou étiqueter des données appartenant à un ensemble muni d'une distance ou d'une mesure de similarité δ . Les étiquettes sont déterminées et on dispose d'un ensemble d'apprentissage A dans lequel chaque donnée est étiquetée ou classée.

L'idée de l'algorithme est la suivante :

Pour prédire l'étiquette d'un élément X'

1. On calcule les k éléments de l'ensemble d'apprentissage qui sont les plus proches de X' (au sens de δ);
2. On attribue à X' l'étiquette la plus fréquente dans ce voisinage.

Exercice 15.4 une implémentation de l'algorithme des k-plus proches voisins

Les données que nous voulons étiqueter/classer sont représentées par des vecteurs de $\mathbb{R}^n$. On note X l'ensemble des données potentielles et $dist$ une distance sur X .

On identifie les éléments de l'ensemble d'apprentissage aux colonnes d'une matrice que nous notons A . Chaque colonne $A[:, j]$ est donc un élément de X dont l'étiquette est connue. L'étiquetage des éléments de A est implémenté dans un dictionnaire E tel que $E[j]$ est l'étiquette (en pratique un indice de classe) de la colonne $A[:, j]$.

La spécification de la fonction principale figure en fin d'exercice.

1. Cette fonction principale construit une liste triée D , dans laquelle chaque terme est un tuple composé de l'étiquette d'une colonne de A et de sa distance à x :

$$D = [(0, 2.0), (0, 2.236...), (1, 3.162...), \dots]$$

Écrire une fonction `choisir_label(D, tol = 10**(-3))` qui prend en argument une liste D comme celle que nous venons de décrire, un flottant tol et qui renvoie la première étiquette si la distance associée est nulle (ou $\leq tol$) et une des plus fréquentes sinon. Si plusieurs étiquettes ont la même fréquence on fera un choix aléatoire (avec `random.choice(R)` par exemple).

Pré-condition : D est triée dans l'ordre croissant de la distance à x .

On pourra utiliser un dictionnaire pour associer à chaque étiquette rencontrée son nombre d'occurrences dans D .

2. Compléter la fonction `k_ppv(A, E, k, x, dist)` qui termine avec `return choisir_label(D, tol = 10**(-3))`.
3. Déterminer la complexité de cette fonction. Nombre d'appels à `dist`, nombre de comparaisons.

```
def k_ppv(A, E, k, x, dist):
    '''
    A      : np.matrix, données de l'ens. d'apprentissage;
    E      : dict, {indice de colonne de A : étiquette};
    k      : int, nombre de voisins à prendre en compte;
    x      : np.matrix, matrice colonne, élt. à classer;
    dist   : distance ou mesure de similarité.
    Renvoie une étiquette pour x.
    '''
```

Corrigé en 15.4 page 559.

15.2.2 Les tests

En pratique on teste de façon systématique un algorithme supervisé. On procède en partant d'un ensemble de données étiquetées ou pour lesquelles les sorties sont connues, X . On sélectionne un sous-ensemble des données qui sera l'ensemble d'apprentissage, $A \subset X$. On compare alors les prédictions que fournit l'algorithme pour des éléments $x \in X \setminus A$ aux étiquettes connues.

La matrice de confusion de ces deux étiquetages rassemble tous ces éléments de comparaison.

Définition 15.4 matrice de confusion de deux étiquetages

Soient X un ensemble de données, $\mathcal{L} = \{e_0, e_1, \dots\}$ un ensemble fini d'étiquettes et f_1, f_2 deux applications $f_i : X \rightarrow \mathcal{L}$. **La matrice de confusion** de f_1, f_2 est la matrice M de taille $|\mathcal{L}| \times |\mathcal{L}|$ telle que $M[i, j] = |\{x \in X / f_1(x) = e_i, f_2(x) = e_j\}|$.

Nous allons tester notre algorithme sur deux ensembles de données. Des données artificielles (ensembles de points de $\mathbb{N}^2$) qui nous permettront page 547 d'en donner une illustration graphique. Nous le reprenons ensuite en 15.4.2 avec des données fournies par la bibliothèque `scikit-learn`.

On commence par une fonction réalisant le calcul des matrices de confusion de deux partitions.

Exercice 15.5 *matrice de confusion*

Notations :

- X est un ensemble de données étiquetées représenté par une matrice X (chaque colonne X^j est un élément de l'ensemble). EX est un dictionnaire tel que $EX[j]$ est l'étiquette de X^j .
- dist est une distance sur X .
- $A \subset X$, est l'ensemble d'apprentissage représenté par une sous-matrice de X . Le dictionnaire EA est tel que $EA[j]$ est l'étiquette de X^j .
- Pour chaque élément de $X1 = X \setminus A$, l'**étiquette apprise** est

$$r = \text{k_ppv}(A, EA, k, x, \text{dist})$$
- On suppose que les étiquettes sont les entiers notés de 0 à p .

1. Écrire une fonction `matrice_confusion(X, EX, A, EA, k, dist)` qui prend des arguments tels que nous les avons décrits (k : int) et renvoie la matrice de confusion des étiquetages initial et appris de X . Cette fonction appelle pour toute colonne $x = X^j$, $r = \text{k_ppv}(A, EA, k, x, \text{dist})$.

Indication : initialiser la matrice avec

```
n_labels = len(set(EX.values()))
Mat       = np.matrix(np.zeros((n_labels, n_labels)))
```

2. On suppose que l'ensemble d'apprentissage a été construit à partir de la matrice X et du dictionnaire EX avec le script :

```
N, M = X.shape
S     = random.sample([i for i in range(0,M)], M//3)
A     = X[:, S]
EA    = {}
for i, j in enumerate(S):
    EA[i] = EX[j]
```

Modifier votre fonction pour qu'elle ne compare ces mêmes étiquetages que pour les éléments de $X1$. Pourquoi fait-on cela ?

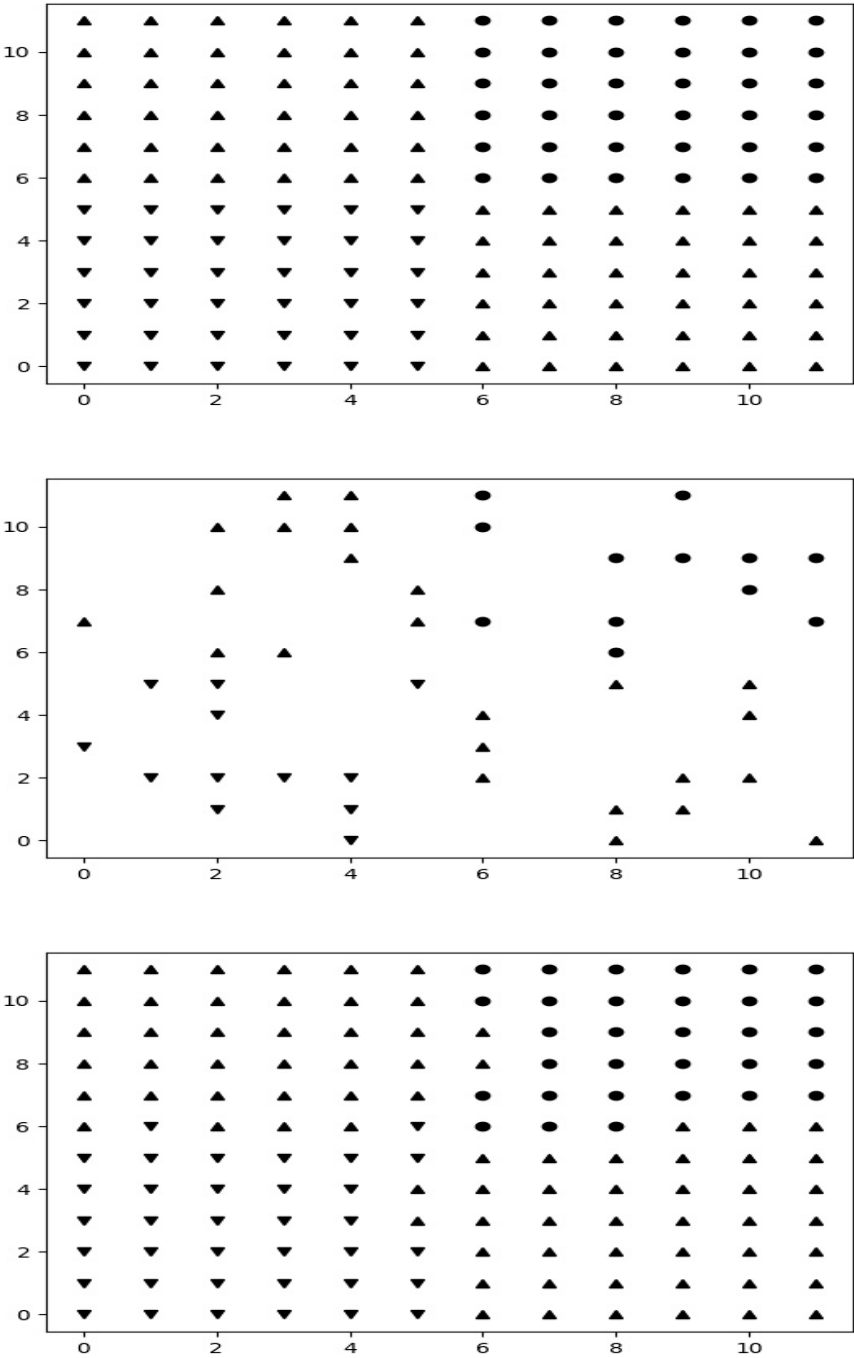
Corrigé en 15.5 page 560.

Les trois figures présentent respectivement, un ensemble X de points de $\mathbb{R}^2$ de coordonnées entières, A l'ensemble obtenu aléatoirement et enfin les prédictions fournies par `k_ppv(X, EX, A, EA, 3, dist)`. La matrice de confusion des deux étiquetages sur

les éléments de $X \setminus A$ est $\begin{bmatrix} 8 & 1 & 0 \\ 2 & 56 & 0 \\ 0 & 6 & 30 \end{bmatrix}$. On y lit que seuls 9 éléments de $X \setminus A$,

qui en compte 103, sont mal étiquetés et on peut rapidement calculer les probabilités

conditionnelles $P(pred = j|val = i)$ et $P(val = i|pred = j)$. Les scripts sont disponibles en ligne.



15.3 Classification non supervisée, algorithme des k-moyennes

L'algorithme de classification que nous présentons ici est un algorithme non supervisé : aucune information préalable sur une partition, aucun exemple de partie ne sont fournis en donnée. Seul le nombre de classes dans la partition que l'on veut obtenir est précisé.

L'algorithme des k-moyennes (ou des centres mobiles)

Les données

- On dispose de M documents notés $(D_0, D_1, \dots, D_{M-1})$; ces documents peuvent être des textes, des images, des relevés statistiques provenant de mesures biologiques comme de comportements d'achats, etc.
- Chaque document est représenté par un vecteur $X_j \in \mathbb{R}^N$;
- On suppose que l'on dispose d'une fonction d'écart entre les documents $dist(D_i, D_j)$ **qui vérifie l'inégalité triangulaire**.

Spécification

$k_moyennes(X, k)$ prend en arguments un ensemble de documents $X = (X_j)_j$, un entier $k \geq 1$ et renvoie un tuple (P, C) où P est une liste de k parties formant une **partition** de l'ensemble des documents et C est la liste des k centres de gravité de ces parties.

Une version basique de l'algorithme des k-moyennes

- On se donne ou on choisit aléatoirement k documents dans $(D_j)_j$ pour initialiser la liste C . Ils sont notés $C_0, \dots, C_{k-1}$.
- On réserve un ensemble P de k listes vides ;
- Pour chaque document représenté par X_j on calcule les k distances $dist(D_j, C_\ell), \ell = 0, \dots, k-1$. Si la distance minimale est $dist(D_j, C_{\ell_{min}})$, on place D_j dans la classe ℓ_{min} (on place donc j dans la liste $P[\ell_{min}]$).
- On dispose d'une première partition, on calcule le centre de gravité de chaque classe. La liste C est mise à jour avec les k centres de gravité : $C = [C_0, \dots, C_{k-1}]$.
- On itère le processus jusqu'à ce que les classes ne soient plus modifiées ou ...

Dans la variante que nous proposons dans l'exercice qui suit, les centres initiaux sont choisis de façon aléatoire. Mais ce choix influe sur le résultat (il en serait de même avec des choix imposés par l'utilisateur). On procède en pratique en faisant tourner la méthode plusieurs fois avec des initialisations différentes et en comparant les résultats.

Un squelette à compléter en exercice

```
def k_means(X, L0, dist, tmax, p =0.01):
    """
    X      : matrice des données;
    L0     : list (indices de col. des centres initiaux);
    dist   : fonction distance ou similitude;
    tmax   : nombre maximum d'iterations.
    """
    N, M   = X.shape
    K       = len(L0)
    # initialisation
    P, C, Wh = partition_0(X, L0, dist)
    t, stable = 0, False

    # itérations
    while not stable and t <= tmax:
        t += 1
        c = 0 # changements de classe
        for j in range(0,M):
            Vj = ???
            Dj = [dist(C[k], Vj) for k in range(0,K)]
            kj = ???
            # kj : indice d'un des centres les plus proches
            # j change-t-il de place?
            if kj != Wh[j] and len(P[Wh[j]]) > 1:
                P[Wh[j]].???
                P[kj].???
                # on note que j est dans la partie kj
                Wh[j] = ???
                c += 1
            for k, Pk in enumerate(P):
                if len(Pk) == 1:
                    C[k] = ???
                else:
                    C[k] = ???
            stable = ???
    return P, C
```

Exercice 15.6 Implémentation de l'algorithme des k -moyennes

On se propose de construire un programme implémentant l'algorithme des k -moyennes.

Les données à traiter seront stockées dans une matrice $X \in \mathcal{M}_{N,M}(\mathbb{R})$, la colonne X^j de X représentant le document D_j . Cette matrice est définie dans numpy par

```
X = numpy.matrix((N,M), ...)
```

Un « squelette » du script est présenté page 549. Il sera complété au fil des questions.

1. Petits outils préalables.

- (a) Écrire une fonction `iso_barycentre(X)` qui prend en argument une matrice X et renvoie la matrice colonne dont le terme $G[i, 0]$ est la moyenne des termes $X[i, :]$.
indication : `np.sum(M)` renvoie la somme des termes de M .
- (b) Écrire une fonction `pos_min(L)` qui retourne la position de la première occurrence du min de L (liste de nombres entiers ou flottants).
- (c) Modifier cette fonction pour obtenir `positions_min(L)` qui retourne la **liste** des positions du min de L .
- (d) Pour initialiser on choisira K centres parmi les colonnes de X . Écrire la ligne de code qui permet de construire aléatoirement la liste des indices.
Indication : `random.sample(population, k)`

2. On se propose de compléter `k_means(X, L, d, tmax)` à partir de la structure de la page 549.

Le nombre de classes à déterminer est donné par `len(L0)` où $L0$ est la liste des indices des centres initiaux.

La ligne `P, C, Wh = partition_0(X, L0, dist)` définit P , une première partition **de l'ensemble des indices** de X , C la liste des centres de gravité des classes de P , et Wh (pour *where is*,) un dictionnaire tel que pour tout indice de colonne j , on ait $Wh[j] = kj$, indice de la classe de P dans laquelle se trouve l'élément j .

- (a) Compléter la ligne `Vj = ???` dans la boucle `for` imbriquée.
 - (b) Compléter `kj = ???`. Il y a deux façons de faire suggérées par les questions qui précèdent. On pensera éventuellement à `rd.choice()`.
 - (c) A quelle partie de l'algorithme la deuxième boucle `for` correspond-elle ? Compléter.
 - (d) A quoi sert le dictionnaire `Wh` ? Compléter les lignes qui le concernent.
 - (e) Quels choix peut-on faire pour compléter la ligne `stable = ???` ?
3. **Complexité** : On note t le nombre d'itérations effectives lors d'un appel `k_means(X, L, d, tmax)` et $K = \text{len}(L)$ (le nombre de classes).
- (a) Majorer le nombre d'appels à la fonction `iso_barycentre()`.
 - (b) Majorer le nombre d'appels à la fonction `dist()`.

Corrigé en 15.6 page 561

Nous démontrons dans l'exercice 15.7 qu'avec l'algorithme des k -moyennes, **l'inertie des partitions décroît à chaque itération**. Cela signifie que **le classement s'améliore** : les classes sont de plus en plus regroupées et les centres de plus en plus éloignés les uns des autres. Mais cela reste une méthode approchée et heuristique : rien ne garantit que la partition obtenue réalise un minimum pour l'inertie. Ce n'est d'ailleurs

pas souvent le cas. Nous livrerons une illustration expérimentale de ces propriétés avec le script de la page 555.

La démonstration de ce résultat repose sur des outils usuels en statistique ou en mécanique : barycentres, produits scalaires, théorème de Huygens et sur un raisonnement algorithmique plus fin. Elle n'est pas au programme de l'enseignement de l'informatique de tronc commun mais ne fait appel qu'à des connaissances de base en math et info.

Exercice 15.7 Une propriété de l'algorithme des k -moyennes

On veut montrer qu'à chaque itération de l'algorithme des k -moyennes l'inertie totale de la partition décroît.

On notera $\| \cdot \|$ la norme euclidienne canonique de $\mathbb{R}^n$. L'ensemble E des données est représenté par une matrice dont chacune des M colonnes X^j représente un document. Soit $P = (P_k)_{0 \leq k \leq K-1}$ une partition de E (ou de l'ensemble $\llbracket 0, M-1 \rrbracket$ des indices dans un script). On note G_k le barycentre de la partie P_k et $m_k = |P_k|$ et G le barycentre de E .

1. Montrer que pour tout $k \in \llbracket 0, K-1 \rrbracket$,

$$\sum_{j \in P_k} \|X_j - G\|^2 = \sum_{j \in P_k} (\|X_j - G_k\|^2 + \|G_k - G\|^2)$$

2. En déduire que l'inertie de E vérifie

$$I_E = \sum_{k=0}^{K-1} \sum_{j \in P_k} \|X_j - G_k\|^2 + \sum_{k=0}^{K-1} m_k \|G_k - G\|^2$$

Que représentent $\sum_{j \in P_k} \|X_j - G_k\|^2$ et $\sum_{k=0}^{K-1} \sum_{j \in P_k} \|X_j - G_k\|^2$?

C'est ce que l'on souhaite minimiser avec un algorithme de classification dont l'objectif est de construire des classes les plus homogènes possible.

3. Notons, lors de l'itération t dans la fonction `k_means` :

- $(G_0^t = C^t[0], \dots, G_{K-1}^t = C^t[K-1])$ les centres de gravité des classes ;
- $(P_0^t, \dots, P_{K-1}^t)$ les parties ;

- $I^t = \sum_{k=0}^{K-1} \sum_{j \in P_k^t} \|X_j - G_k^t\|^2$ la somme des inerties de chaque classe à la fin de l'itération t , c'est-à-dire, selon la définition 15.3, l'inertie totale de la partition P^t .

On introduit alors $Q^t = \sum_{k=0}^{K-1} \sum_{j \in P_k^t} \|X_j - G_k^{t-1}\|^2$. Les G_k^{t-1} ne sont donc plus les centres de gravité des classes P_k^t mais des P_k^{t-1} .

- (a) Montrer par un calcul analogue au précédent que $Q^t \geq I^t$.

- (b) Montrer par un raisonnement sur les distances aux centres de gravité que $I^t \geq Q^{t+1}$. On comparera les termes contenant un même X_j dans les deux expressions et l'on envisagera deux cas selon que l'élément X_j change de classe ou pas entre t et $t + 1$.
- (c) En déduire que la suite $(I^t)_t$ décroît.

Corrigé en 15.7 page 563.

15.4 Bibliothèque scikit-learn

Cette bibliothèque de Python implémente les algorithmes et les outils (distances, mesures de similitude...) fondamentaux pour la fouille des données et l'apprentissage automatique. Elle met aussi à notre disposition, dans le sous-module **datasets**, des **ensembles de données** qui permettent de tester rapidement les algorithmes, ceux de la bibliothèque comme les nôtres.

La documentation : <https://scikit-learn.org/stable/index.html>

Les ensembles de données : https://scikit-learn.org/stable/datasets/toy_dataset.html

15.4.1 scikit-learn.datasets

Pour tester nos fonctions `k_ppv()` et `k_means()`, nous chargeons depuis le module `scikit-learn.datasets`, un relevé de 150 mesures de longueurs/largeurs de sépales et de pétales d'iris appartenant à trois espèces : Iris-Setosa, Iris-Versicolour et Iris-Virginica. Les données sont stockées dans un tableau `iris.data` de dimensions 150×4 ; la partition d'origine comporte 3 classes de même cardinal 50, les étiquettes sont dans `iris.target`.

```
>>> from sklearn import datasets
>>> iris = datasets.load_iris()
>>> X = np.matrix(iris.data.transpose())
>>> X.shape
(4, 150)
>>> X[:, 0:7]
[[5.1 4.9 4.7 4.6 5. 5.4 4.6]
 [3.5 3. 3.2 3.1 3.6 3.9 3.4]
 [1.4 1.4 1.3 1.5 1.4 1.7 1.4]
 [0.2 0.2 0.2 0.2 0.2 0.4 0.3]]
>>> type(iris.target); iris.target.shape; iris.target
<class 'numpy.ndarray'>
(150,)
[0 0 0 0 0 0 ... 1 1 1 1 1 1... 2 2 2 2 2]
```

15.4.2 k-plus proches voisins avec scikit-learn

```

y      = iris.target
N, M   = X.shape
EX     = {} # étiquettes
for j, e in enumerate(y):
    EX[j] = e
# --- ensemble d'apprentissage
S      = rd.sample([i for i in range(0,M)], M//3)
A      = X[:, S]
EA     = {}
for i, j in enumerate(S):
    EA[i] = y[j]
# --- matrice de confusion
MC = matrix_confusion(X, EX, S, A, EA, 3)
print(MC)

[[35.  0.  0.]
 [ 0. 32.  2.]
 [ 0.  1. 35.]]

```

Exercice 15.8 *comprendre le script*

iris, X et y sont définis dans le paragraphe précédent.

1. Pourquoi avons-nous transposé et transformé iris.data en matrice (script précédent)? Pourquoi avons-nous choisi la facilité de stocker nos données dans des matrices?
2. Lecture de la matrice de confusion :
 - (a) Quel est l'effectif exact de l'ensemble d'apprentissage?
 - (b) Quelle est la classe pour laquelle l'apprentissage donne pleinement satisfaction? Comment expliquer cela?
 - (c) On note 0, 1 et 2 les classes. Préciser les probabilités conditionnelles :

$$p_{i,j} = P(\text{classe}(X) = i | \text{prédiction}(X) = j)$$

Corrigé en 15.8 page 565.

15.4.3 k-moyennes avec scikit-learn

Exercice 15.9 *test de k_moyennes()*

Le script de la page 552 permet de charger l'ensemble de données iris.dataset avec lequel nous allons tester notre fonction k_moyennes().

1. Pourquoi avons-nous choisi de transposer et de transformer en matrice le tableau iris.data?

2. Que représente la colonne $X[:, 0]$?
3. Comment peut-on interpréter le tableau `iris.target`? En déduire la partition Q des données initiales en trois espèces. Calculer l'inertie de cette partition avec la fonction de l'exercice 15.3.
4. Écrire une fonction `comparer(E,P,Q)` qui prend en arguments E un ensemble ou une liste, P et Q deux listes de listes, avec la précondition P et Q sont des partitions de même longueur de E , et qui renvoie une matrice M de taille $(K + 1) \times (K + 1)$ avec $K = \text{len}(P)$, avec $M[K, j] = |P[j]|$, $M[i, K] = |Q[i]|$, $M[i, j] = |Q[i] \cap P[j]|$ pour $0 \leq i, j \leq K - 1$ et $M[k, k] = |E|$.
On pourra utiliser la fonction `is_partition(E,P)` pour vérifier les pré-conditions.
5. On écrit le script suivant avec nos fonctions (Q est la partition obtenue avec `iris.target`) :

```
N, M = X.shape
dist = lambda X, Y: norm(X-Y, 2)
L0 = rd.sample([i for i in range(0, M)], 3)
P, C = k_means(X, L0, dist, 200)
M = comparer([i for i in range(0,150)], P, Q)
print(M)
print(inertie([y0, y1, y2], X))
print(inertie(P, X))
```

```
-----
[[ 0  0 50 50]
 [47  3  0 50]
 [14 36  0 50]
 [61 39 50 150]]
[[1.         1.         0. ]
 [0.265625  0.96511628 1. ]
 [0.8556701  0.32075472 1. ]]
89.29740000000001
78.8556658259773
```

Comment interpréter la matrice M obtenue? Comment expliquer que la partition que notre algorithme a obtenue ait une **inertie plus faible** que la partition d'origine (qui est donc la partition réelle selon les espèces)?

Attention : si vous répétez le script, il arrivera que les résultats soient moins bons. Rappelons que cet algorithme doit être exécuté plusieurs fois !

6. Montrer en écrivant une fonction `matrice_jaccard(P,Q)` comment la matrice `comparer(E,P,Q)` permet d'obtenir la matrice des distances de Jaccard entre les classes de P et les classes de Q .

Corrigé en 15.9 page 565.

Le script de la page 555 est obtenu en ajoutant à notre fonction `k_means()` une ligne permettant l'affichage du n° de l'itération et de l'inertie totale de la partition. Comme démontré dans l'exercice 15.7, elle décroît.

```
from    scipy.linalg import norm
from    sklearn          import datasets

iris    = datasets.load_iris()
X        = np.matrix(iris.data.transpose())

N, M    = X.shape
dist    = lambda X, Y: norm(X-Y, 2)
L0      = rd.sample([i for i in range(0, M)], 3)
P, C    = k_means(X, L0, dist, 200 )

t = 1; inertie = 83.46996551724138
t = 2; inertie = 82.31399516908215
t = 3; inertie = 81.277800000000004
t = 4; inertie = 80.226346215781
t = 5; inertie = 79.59232190942474
t = 6; inertie = 79.026166666666667
t = 7; inertie = 78.85566582597731
t = 8; inertie = 78.85566582597731
```

• La classe `Kmeans` de `scikit-learn`

```
from    sklearn          import datasets
from    sklearn.cluster  import KMeans

iris    = datasets.load_iris()
X        = iris.data
R        = KMeans(n_clusters=3, random_state=0).fit(X)
P        = R.predict(X)
c0 = len([x for x in P if x == 0])
c1 = len([x for x in P if x == 1])
c2 = len([x for x in P if x == 2])

print(type(R))
print(R.inertia_)
print(c0,c1,c2)
-----
<class 'sklearn.cluster._kmeans.KMeans'>
78.85144142614601
38 50 62
```

Nous faisons maintenant tourner la méthode Kmeans de scikit-learn ¹ Nous affichons les classes obtenues sur le même ensemble de données et l’inertie de la partition obtenue. Le résultat sera toujours le même, l’algorithme tourne plusieurs fois et affiche la partition dont l’inertie est optimale.

15.4.4 Lexique français anglais (US)

intelligence artificielle	artificial intelligence
apprentissage automatique	machine learning
apprentissage supervisé	supervised learning
apprentissage non supervisé	unsupervised learning
classement, étiquetage	classification *
classification *	clustering
classe	cluster
k-plus proches voisins (kppv)	k-nearest neighbors (knn)
k-moyennes, k-centres	k-means, k-centroids
étiqueter	to label
segmenter	to tokenize

1. L’objectif n’est pas ici de fournir une aide pour scikit-learn. On se contente de comparer.

15.5 Corrigés des exercices

Corrigé de l'exercice n° 15.1

Les commentaires répondent aux questions :

```
def is_partition(P, E):
    '''
    P: list (of lists);
    E : list|set
    '''
    D = {}
    for k, Pk in enumerate(P):
        if len(Pk) == 0: # (1)
            print('la classe P%s est vide'%(k))
            return False
        for i in Pk:
            if i not in E: # (2)
                print('P%s non inclue dans E'%(k))
                return False
            if i not in D:
                D[i] = k
            elif D[i] != k: # (3)
                print('P%s et P%s ont %s en commun'\
                      %(k, D[i], i))
                return False
            else: # (4)
                print('P%s contient deux fois %s'\
                      %(k, i))
                return False
    # Reste à vérifier que la réunion des Pk = E
    for i in E:
        if i not in D: # (5)
            print('%s ne figure dans aucune classe'%(i))
            return False
    return True
```

P est une partition ssi les classes sont non vides (vérifié en # (1)), d'intersections deux à deux vides (vérifié en # (3)), et si $\cup P_k = E$ ce qui est vérifié en # (2) pour l'inclusion $\subset$ et en # (5) pour $\supset$. En # (4), on s'assure que P_k est bien un ensemble. Par ailleurs, si P est une partition de E il y a $2|E|$ itérations en tout.

Corrigé de l'exercice n° 15.2

1. On a alors $df_{t_i} = |\mathcal{C}|$, puisque t_i est présent partout, et $x_{i,j} = 0$. Ce serait le cas avec les mots courants de la langue, articles, prépositions, auxiliaires que l'on

évite systématiquement de prendre en compte (mots-vides ou stop-words en anglais).

- On a $df_{t_i} = 1$, puisque t_i est présent dans le seul document D_j . Ainsi le coefficient de tf_{t_i,d_j} dans $x_{i,j}$ prend la **valeur maximale** et $x_{i,j} = tf_{t_i,d_j} \times \ln |\mathcal{C}|$.
- On aura $\delta(D_j, D_k) = 1 - \text{sim}(D_j, D_k) = 0$ ssi $\text{sim}(D_j, D_k) = 1$ et avec le cas d'égalité dans Cauchy-Schwarz, les vecteurs X^j et X^k sont proportionnels. Le coefficient de proportionnalité k est positif (pourquoi ?).

Donc pour tout indice i qui correspond au même terme t_i , on a

$$k = \frac{x_{i,j}}{x_{i,k}} = \frac{tf_{t_i,d_j} \times \ln \frac{|\mathcal{C}|}{df_{t_i}}}{tf_{t_i,d_k} \times \ln \frac{|\mathcal{C}|}{df_{t_i}}} = \frac{tf_{t_i,d_j}}{tf_{t_i,d_k}}$$

Ce qui signifie les occurrences des mots dans les deux documents sont proportionnelles. Par exemple :

D_j = « Bonjour à toi. » et D_k = « Bonjour à toi. À toi, Bonjour ! »

Distance : $\delta(D, D) = 0$, δ est clairement symétrique, mais il n'y a pas la séparabilité (des docs différents peuvent être à δ nul). δ n'est donc pas une distance au sens mathématique strict.

- La relation $\delta(X, Z) \leq \delta(X, Y) + \delta(Y, Z)$ est équivalente à $\cos \gamma \geq \cos \alpha + \cos \beta - 1$. On a un contre-exemple immédiat en choisissant dans $\mathbb{R}^{+n}$ comme c'est toujours possible (pourquoi ?), trois vecteurs tels que $\alpha = \beta = \frac{\pi}{4}$ et $\gamma = \frac{\pi}{2}$. δ ne vérifie donc pas l'inégalité triangulaire.
- On aura $\delta(D_j, D_k) = 1$ ssi $\text{sim}(D_j, D_k) = 0$ soit, $\frac{\sum x_{i,j}x_{i,k}}{\|X^j\|_2\|X^k\|_2} = 0$. Dans ce cas, puisque les x_i sont positifs, chaque terme est nul et aucun terme n'est présent dans les deux documents à la fois.

Corrigé de l'exercice n° 15.3

```
def iso_barycentre(X):
    assert isinstance(X, np.matrix)
    N, M = X.shape
    G = np.zeros((N,1))
    for i in range(0,N):
        G[i, 0] = np.sum(X[i, :])/M
    return G
```

```

from numpy.linalg import norm

def inertie(P, X):
    assert isinstance(X, np.matrix)
    N, M = X.shape
    assert is_partition(P, [j for j in range(0,M)])
    T = 0
    for Pk in P:
        V = 0
        C = iso_barycentre(X[:, Pk])
        for j in Pk:
            V += norm(X[:,j]-C, 2)**2
        T += V # inertie dans Pk
    return T

```

Corrigé de l'exercice n° 15.4

1. C'est dans cette fonction que résident les variantes de l'algorithme.

```

def choisir_label(D, tol = 10**(-3)):
    '''
    D : list of tuple; [(label, distance), ...]
    '''
    assert isinstance(D, list) and len(D) > 0
    if D[0][1] <= tol:
        return D[0][0]
    else:
        V = {} # fréquence des étiquettes
        for e in D:
            ell = e[0]
            if ell in V:
                V[ell] += 1
            else:
                V[ell] = 1
        fm = V[ell] # recherche de la fréq. max
        for ell in V:
            if V[ell] > fm:
                fm = V[ell]
        R = [ell for ell in V if V[ell] == fm]
        return rd.choice(R)

```

2. Quatre lignes suffisent

```
def k_ppv(A, E, k, x, dist = dist2):
    '''
    A      : np.matrix; données de l'ens. d'apprentissage;
    E      : dict; {j (colonne de A): étiquette};
    k      : int; nombre de voisins à prendre en compte;
    x      : np.matrix (matrice colonne, élt. à classer);
    dist   : distance ou mesure de similarité.
    Renvoie une étiquette pour x.
    '''
    assert isinstance(A, np.matrix)
    N, M = A.shape
    D      = [(E[j], dist(x, A[:, j])) for j in range(0, M)]
    D.sort(key = lambda t: t[1])
    return choisir_label(D[0:k])
```

3. Complexité

Les appels à `dist` se font dans la construction de D . Il y en a $M = |A|$.

Les comparaisons ont lieu dans le tri : $O(M \ln M)$ et dans `choisir_label(D)` il y en a au plus autant que d'étiquettes. Donc $O(1)$.

Corrigé de l'exercice n° 15.5

On ne donne que le deuxième script, il y a peu de différences entre les deux. En ne comparant que les étiquettes apprises on a une meilleure mesure de la prédiction car **les données de A sont reproduites à l'identique** (on garde les étiquettes à distance 0 !) ce qui biaise l'efficacité réelle de l'algorithme.

```
def matrice_confusion(X, EX, S, A, EA, k, dist = dist2):
    '''
    E : dict , {int (indice col) : int (label)}
    '''
    assert isinstance(X, np.matrix)
    N, M      = X.shape

    n_labels = len(set(EX.values()))
    Mat      = np.matrix(np.zeros((n_labels, n_labels)))
    for j in range(0, M):
        if j not in S:
            x      = X[:, j]
            v      = EX[j]
            r      = k_ppv(A, EA, k, x, dist)
            Mat[v, r] += 1

    return Mat
```

Corrigé de l'exercice n° 15.6

1. (a) : Voir le corrigé de l'exercice 15.3 page 558.
 (b) et (c) :

```
def pos_min(L):
    pos, min = 0, L[0]
    for i in range(1, len(L)):
        if L[i] < min:
            pos, min = i, L[i]
    return pos

def positions_min(L):
    m, Pm = L[0], [0]
    for i in range(1, len(L)):
        if L[i] < m:
            m, Pm = L[i], [i]
        elif L[i] == m:
            Pm.append(i)
    return Pm
```

- (d): `L0 = rd.sample([i for i in range(0, M)], 3)`
2. (a) `Vj = X[:, j]` on place ici la colonne `j` et on cherche la liste des distances de `Vj` aux `k` centres des classes avec :
- `Dj = [dist(C[k], Vj) for k in range(0, K)]`
- (b) Si on choisit une classe parmi celles dont les centres sont les plus proches on fera : `kj = rd.choice(positions_min(Dj))`. Par contre il sera plus rapide de choisir le premier indice (ou la première classe) avec `kj = pos_min(Dj)`. C'est ce dernier choix **qui devra être fait** pour de gros documents sans risque de collision.
- (c) Il s'agit de déterminer les centres de gravité. On appelle donc `iso_barycentre(X[:, Pk])` si la classe contient plus d'un élément, on garde l'élément unique sinon.
- (d) `Wh` sert à repérer les changements de classe avec une recherche à coût constant.
- (e) On pourrait donner un critère d'arrêt avec `stable = c==0` (pas de migration) ou avec `stable = c < M*p` lorsque le nombre de migrations passe en-dessous d'un certain seuil.
3. (a) Le nombre d'appels à la fonction `iso_barycentre()` est majoré par $K \times (t + 1)$.
- (b) Le nombre d'appels à la fonction `dist()` est égal à $M \times K \times (t + 1)$.

L'algorithme avec la fonction d'initialisation

```
def k_means(X, L0, dist, tmax):
    '''
    X      : matrice des données;
    L0     : list (indices de col. des centres initiaux);
    dist   : fonction distance ou similitude;
    tmax   : nombre maximum d'iterations.
    '''
    assert isinstance(X, np.matrix) and isinstance(L0, list)
    N, M   = X.shape
    K      = len(L0)
    assert 2 <= K and K < M

    # initialisation
    P, C, Wh = partition_0(X, L0, dist)
    assert is_partition(P, [j for j in range(0,M)] )
    assert check(P, Wh)

    t, stable = 0, False
    # itérations
    while not stable and t <= tmax:
        t += 1
        c = 0 # changements de classe
        for j in range(0,M):
            Vj = X[:, j] # colonne j
            Dj = [dist(C[k], Vj) for k in range(0,K)]
            kj = rd.choice(positions_min(Dj))
            # kj est l'indice d'un centre le plus proche
            # j change-t-il de place?
            if kj != Wh[j] and len(P[Wh[j]]) > 1:
                P[Wh[j]].remove(j)
                P[kj].append(j)
                # on place j dans la partie kj après...
                Wh[j] = kj
                c += 1
            # actualiser position, incrémenter...

        assert check(P, Wh) #
        assert is_partition(P, [i for i in range(0,M)])

        for k, Pk in enumerate(P):
            if len(Pk) == 1:
                C[k] = X[:, Pk[0]]
            else:
                C[k] = iso_barycentre(X[:, Pk])

        stable = c == 0 # (c/M <= p)
        print('t = %s; Inertie = %s'%(t, inertie(P, X)))
    return P, C
```

La fonction `is_partition()` est définie dans l'exercice 15.1 et `check(P, Wh)` renvoie `True` ssi pour tout `elt. j` dans `P[k]`, `W[j] = k`.

```
def partition_0(X, L0, dist):
    """
    X      : matrice des données;
    L0     : list (indices de col. des centres initiaux);
    dist   : fonction distance ou similarité.
    """
    assert isinstance(X, np.matrix)
    N, M   = X.shape
    K      = len(L0)
    assert 2 <= K and K < M
    Sk
    P = [[j]      for j in L0]
    C = [X[:, j]  for j in L0]
    Wh = {} # Wh(ere is?)
    for k in range(0, K):
        Wh[L0[k]] = k
    assert check(P, Wh)

    # répartition
    for j in range(0, M):
        if j not in L0:
            Vj = X[:, j] # colonne j
            Dj = [dist(C[k], Vj) for k in range(0, K)]
            # liste des distances
            kj = rd.choice(positions_min(Dj))
            # kj est l'indice d'un centre le plus proche
            P[kj].append(j)
            # on place j dans la classe/partie P[kj]
            Wh[j] = kj
            # on le note dans le dictionnaire
    for k in range(0, K):
        C[k] = iso_barycentre(X[:, P[k]])

    return P, C, Wh
```

Corrigé de l'exercice n° 15.7

1. Soit $k \in \llbracket 0, K-1 \rrbracket$.

$$\begin{aligned}
 \sum_{j \in P_k} \|X_j - G\|^2 &= \sum_{j \in P_k} \|X_j - G_k + G_k - G\|^2 \\
 &= \sum_{j \in P_k} (\|X_j - G_k\|^2 + \|G_k - G\|^2 + 2 \langle X_j - G_k, G_k - G \rangle)
 \end{aligned}$$

Or,

$$\sum_{j \in P_k} \langle X_j - G_k | G_k - G \rangle = \left\langle \sum_{j \in P_k} (X_j - G_k) | (G_k - G) \right\rangle$$

et comme $\sum_{j \in P_k} (X_j - G_k) = 0$, on obtient la formule attendue. C'est là le théorème de Huygens.

2. On a $\sum_{j \in P_k} \|G_k - G\|^2 = m_k \|G_k - G\|^2$. En sommant les égalités obtenues dans la question 1,

$$\begin{aligned} I_E &= \sum_{j=0}^{M-1} \|X_j - G\|^2 = \sum_{k=0}^{K-1} \sum_{j \in P_k} \|X_j - G\|^2 \\ &= \sum_{k=0}^{K-1} \sum_{j \in P_k} \|X_j - G_k\|^2 + \sum_{k=0}^{K-1} m_k \|G_k - G\|^2 \end{aligned}$$

I_E est l'inertie totale du nuage de points, $\sum_{k=0}^{K-1} \sum_{j \in P_k} \|X_j - G_k\|^2$ est l'**inertie de la partition**.

3. t est le n° de l'itération (on notera $t = 0$ pour la phase d'initialisation : première partition, premiers centres de gravité). Écrivons les quantités à comparer avant de nous lancer :

$$\begin{aligned} Q^t &= \sum_{k=0}^{K-1} \sum_{j \in P_k^t} \|X_j - G_k^{t-1}\|^2 = \sum_{k=0}^{K-1} \sum_{j \in P_k^t} \|X_j - G_K^t + G_K^t - G_k^{t-1}\|^2 \\ &= \sum_{k=0}^{K-1} \sum_{j \in P_k^t} \|X_j - G_K^t\|^2 + \|G_K^t - G_k^{t-1}\|^2 + 2 \langle X_j - G_K^t | G_K^t - G_k^{t-1} \rangle \end{aligned}$$

- (a) Comme précédemment, pour k fixé,

$$\sum_{j \in P_k^t} \langle X_j - G_K^t | G_K^t - G_k^{t-1} \rangle = \left\langle \sum_{j \in P_k^t} (X_j - G_K^t) | G_K^t - G_k^{t-1} \right\rangle = 0.$$

Il reste donc $Q^t = I^t + \sum_{k=0}^{K-1} \sum_{j \in P_k^t} \|G_K^t - G_k^{t-1}\|^2 \geq I^t$.

- (b) Montrons que $I^t \geq Q^{t+1}$. Regardons les deux sommes :

$$I^t = \sum_{k=0}^{K-1} \sum_{j \in P_k^t} \|X_j - G_k^t\|^2 \text{ et } Q^{t+1} = \sum_{k=0}^{K-1} \sum_{j \in P_k^{t+1}} \|X_j - G_k^t\|^2$$

Chaque indice j est présent une fois et une seule dans chacune de ces deux sommes.

Comparons donc les termes de ces sommes dans lesquels figure j_0 ou, de façon équivalente, X_{j_0} .

Dans la première somme, X_{j_0} apparaît avec le carré $\|X_{j_0} - G_k^t\|^2$ de sa distance au centre de la classe P_k^t à laquelle il appartient à l'instant t .

Qu'en est-il dans la deuxième somme ?

- Soit j_0 n'a pas migré entre t et $t + 1$, et ces nombres sont égaux car les valeurs de k sont les mêmes.

- Si j_0 a migré dans la classe $P_{k'}^{t+1}$, c'est parce qu'à l'itération t , $\|X_{j_0} - G_{k'}^t\|^2$ était minimale. On a alors $\|X_{j_0} - G_{k'}^t\|^2 \leq \|X_{j_0} - G_k^t\|^2$.

(c) Ainsi, $\boxed{Q^{t+1} \leq I^t \leq Q^t}$ d'où $\boxed{I^{t+1} \leq Q^{t+1} \leq I^t \leq Q^t}$.

La suite des inerties des partitions est bien décroissante.

Corrigé de l'exercice n° 15.8

1. Notre fonction `k_ppv` travaille avec des matrices dont les colonnes sont des vecteurs de numpy. <ce qui est pratique pour utiliser les `np.norm` sans travail supplémentaire.
2. (a) On a en tout $30 + 32 + 2 + 1 + 35 = 100$ éléments dans $X1 = X \setminus A$ donc $|A| = 50$.
(b) Les éléments de la première classe (Iris-Setosa) sont parfaitement reconnus. Les éléments de cette classe ont des caractéristiques morphologiques bien distinctes de celles des éléments des deux autres classes.
(c) On a $p_{i,j} = \frac{P(\text{classe}(X) = i \cap \text{prédiction}(X) = j)}{P(\text{prédiction}(X) = j)} = \frac{M[i, j]}{\sum M[:, j]}$

$$p_{0,0} = 1, p_{0,i} = 0 \text{ si } i = 1, 2.$$

$$p_{1,0} = 0, p_{1,1} = 32/33, p_{2,1} = 2/37$$

$$p_{2,0} = 0, p_{2,1} = 1/33, p_{1,2} = 35/37.$$

Corrigé de l'exercice n° 15.9

1. Notre fonction est implémentée avec des matrices dont les colonnes représentent les données. Scikit-learn travaille avec des tableaux dont les lignes représentent les données.
2. C'est un iris avec un sépale de longueur 5.1cm et de largeur 3.5cm, de pétale de longueur 1.4cm et de largeur 0.2cm.
3. `iris.target[j] = k` ssi la donnée X^j est dans la classe k . On obtient la partition des indices (comme nous le faisons toujours) avec :

```
>>> y = iris.target
>>> y0 = [i for i in range(0, len(y)) if y[i] == 0]
>>> y1 = [i for i in range(0, len(y)) if y[i] == 1]
>>> y2 = [i for i in range(0, len(y)) if y[i] == 2]

>>> inertie([y0, y1, y2], X)
1.7859479999999999
```

4. On vérifie tout puisque nous sommes en phase de test. Les lignes avec assert sont appelées à disparaître une fois les preuves de correction et les tests faits.

```
def comparer(E, P, Q):
    '''
    E : list|set;
    P : list of lists (partition de E);
    Q : list of lists (partition de E);
    P et Q sont de même longueur.
    Renvoie une matrice ...
    '''
    assert is_partition(P,E) and is_partition(Q,E)
    assert len(P) == len(Q)
    K = len(P)
    M = np.matrix(np.zeros((K+1,K+1), dtype = int))
    for k, Pk in enumerate(P):
        M[K, k] = len(Pk)
        for j, Qj in enumerate(Q):
            M[j,k] = len([e for e in Qj if e in Pk])
    for j, Qj in enumerate(Q):
        M[j,K] = len(Qj)
        M[K,K] += len(Qj)
    return M
```

5. La classe $Q[0]$ de l'ensemble cible coïncide parfaitement avec la partie $P[1]$ donnée par notre fonction. les choses sont moins lisibles dans les autres cas, mais le calcul des distances de Jaccard en facilitera l'interprétation.

Pour ce qui est de l'inertie, elle s'explique par le fait que la variabilité entre espèces ne suffit pas à les séparer complètement. Notre algorithme classe en fonction des mesures observées et obtient une partition de plus petite inertie que celle obtenue avec le seul critère de l'espèce (qui n'est pas explicité ici).

Ainsi en allait-il de Sapiens et de Néandertal : certains sapiens pouvaient se promener incognito chez leurs cousins et réciproquement.

6. On rappelle que $d_J(A, B) = 1 - J(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|} = \frac{|A \Delta B|}{|A \cup B|}$.

Le calcul qui suit montre qu'une classe $P[2]$ coïncide avec $Q[0]$ ce que confirme la distance de Jaccard égale à 0 entre ces deux classes. Ensuite $Q[1]$ est proche

de $P[0]$ alors que $Q[2]$ est plus proche de $P[1]$.

```
def matrice_jaccard(E, P, Q):

    M      = comparer(E, P,Q)
    n, m   = M.shape
    K      = n-1
    J      = np.matrix(np.zeros((K,K), dtype = float))
    for i in range(0, K):
        for j in range(0, K):
            # J[i,j] = d_J(Q[i], P[k])
            J[i,j] = (M[i,K]+M[K,j]- 2*M[i,j]) \
                    / (M[i,K] + M[K,j]- M[i,j])

    return J
```

```
M = comparer([i for i in range(0,150)], P, [y0, y1, y2])
print(M)
J = matrice_jaccard([i for i in range(0,150)], P, \
                    [y0, y1, y2])

print(J)
print(inertia([y0, y1, y2], X))
print(inertia(P, X))

-----
[[ 0  0  50  50]
 [47  3  0  50]
 [14 36  0  50]
 [61 39  50 150]]

[[1.         1.         0.    ]
 [0.265625   0.96511628 1.    ]
 [0.8556701  0.32075472 1.    ]]

89.297400000000001
78.85566582597731
```

Attention : d'autres appels renverront des résultats pour lesquels l'inertie sera supérieure. L'idée reste d'utiliser plusieurs fois la méthode et de garder les meilleurs résultats. On peut aussi rechercher les éléments qui se retrouvent réunis dans une même partition à chaque expérimentation et qui forment donc les noyaux durs des classes.

Glossaire de l'informatique générale

Dans ce glossaire vous trouverez le vocabulaire parfois fluctuant de l'informatique générale. Bien souvent une entrée en éclairera une autre et il faudra faire votre cheminement : l'ordre alphabétique n'est pas une implémentation du programme d'Euclide.

accumulateur

Pour exécuter des instructions, un processeur accède à des données qui sont stockées dans des petites mémoires rapides (de 16, 32 ou 64 bits) que l'on appelle des registres. L'accumulateur est le registre principal d'un processeur ou micro processeur. C'est celui qui contient la valeur en cours de calcul. On y accède avec la variable ans dans vos calculettes, avec l'underscore _ dans une console Python.

arité

L'arité d'un opérateur ou d'une fonction est le nombre d'arguments qu'il ou elle prend en charge. Les opérateurs de l'arithmétique élémentaire sont souvent binaires (d'arité 2) comme +, ×, -(soustraction), / , parfois unaires (d'arité 1) comme - (signe), ! (factorielle).

assembleur, langage assembleur, langage binaire/langage machine

Le processeur ou les processeurs installés sur un ordinateur interprètent des instructions écrites en langage machine. Ces instructions sont codées sous forme binaire. Un assembleur est le programme qui permet de coder directement ces instructions ou des regroupement utiles de ces instructions sous une forme plus accessible et condensée.

Le langage Python est le même sur toutes les machines (c'est ce qui fait de lui un langage de haut niveau) bien que l'implémentation de Python change avec le système d'exploitation, à l'opposé, un programme assembleur est entièrement dédié au processeur pour lequel il est écrit ! L'assembleur pour le processeur 6502 par exemple est différent de tous les autres ; c'est ce qui a permis aux programmeurs de reconnaître à la sortie du film Terminator le processeur dont le personnage-titre était doté.

On programme encore de nos jours directement en assembleur pour gérer des

processeurs élémentaires embarqués : machines outils, robotique rudimentaire et autres machines à laver le linge...

Les instructions en langage binaire sont des séquences particulières de symboles (ou **bits** pour **binarydigits**) notés 0 et 1 ; un **mot** a une taille d'un **octet** s'il est formé de 8 bits. Il y a $2^8 = 256$ mots de 8 bits possibles et $2^{64} = 18446744073709551616$ mots de 64 bits possibles.

bibliothèque standard

La bibliothèque standard d'un langage de programmation est la bibliothèque logicielle qui est utilisée dans toute implémentation de ce langage. Elle peut, comme dans le cas de Python, comporter différents modules qu'il faudra ajouter au cœur du langage si on souhaite les utiliser.

bit

Abréviation de *binary digit*. C'est la plus petite unité d'information traitée par un processeur. Un bit représente 0 ou 1 en numération binaire.

byte

Un byte est la taille de la plus petite zone adressable par un processeur. Un byte égale un octet. A ne confondre, ni avec bit, ni avec un *mot mémoire* qui peut être formé de n octets, $n = 1$ (d'où la confusion), 2, 2^2 , ...

calcul scientifique

On appelle calcul scientifique l'ensemble des méthodes visant à implémenter les algorithmes de calcul ou de résolution de problèmes numériques. Cette discipline se démarque de l'**analyse numérique**, branche des mathématiques qui vise à l'étude d'algorithmes de résolution exacte ou, le plus souvent, approchée de ces problèmes (méthodes de calcul approché d'intégrales, de résolution de systèmes d'équations, d'équations différentielles...) en ce sens que l'implémentation effective doit tenir compte de la différence de comportement entre les nombres réels, ceux des mathématiciens, et les flottants.

calcul formel

C'est la branche des mathématiques qui vise à l'étude d'algorithmes de traitement exacts des objets mathématiques (factorisations, simplifications d'expressions exactes, expression exacte d'une intégrale, de l'image d'un réel par une fonction, expressions exactes des solutions d'une équation) dans l'optique de leur implémentation effective. Le calcul formel inclut la conception des structures informatiques qui permettent de représenter les objets mathématiques.

Le tableau suivant illustre la différence entre le calcul scientifique en général et le calcul formel. A gauche les calculs approchés avec numpy. A droite les calculs formels avec sympy. Comme on ne connaît pas d'expression exacte sous quelque forme que ce soit de $\sin(\pi/7)$, un logiciel de calcul formel laissera l'expression inchangée.

<pre>import numpy print(numpy.sin(numpy.pi/3)) print(numpy.sin(numpy.pi/7)) 0.8660254037844386 0.4338837391175581</pre>	<pre>import sympy print(sympy.sin(sympy.pi/3)) print(sympy.sin(sympy.pi/7)) sqrt(3)/2 sin(pi/7)</pre>
--	--

compilateur

Un compilateur est un programme informatique qui traduit un programme source (au format ASCII) en code binaire (langage machine) directement exécutable par le processeur. Exemples : compilateurs C, C++, OCaml, Fortran etc.

Voir : (langage) compilé.

compilé (langage)

Un langage est compilé lorsque le code est traduit en un langage de plus bas niveau (en langage assembleur, lui-même proche des instructions du processeur) préalablement à l'exécution du programme. C'est le code compilé qui est ensuite directement exécuté par la machine. Ainsi C, C++, OCaml, Fortran sont des langages compilés. Java a un comportement intermédiaire : le code Java est compilé pour une machine virtuelle (jvm : java virtual machine) qui exécute le programme sur la machine physique. Ainsi le code java est en principe universel alors que le code de la machine virtuelle dépend d'ordinateur et de l'OS sur lequel elle tourne. En pratique on écrit le code ou programme dans un éditeur (ou dans un EDI), on compile ce premier programme (le code), on exécute ce logiciel (l'exécutable). Il n'y a donc pas comme dans Python de shell ou console.

Voir : assembleur, compilateur, exécutable, (langage) interprété, machine virtuelle.

éditeur de texte

Un éditeur est un logiciel dans lequel on peut créer et éditer du texte. S'il est sophistiqué et dédié à l'écriture de code avec des fonctions annexes facilitant la programmation, l'exécution de scripts ou la compilation de programmes et leur débogage cela devient un EDI/IDE (environnement de développement intégré). Exemples : Edit sous Windows, vi ou emacs sous Unix... *Voir IDE*

éditeur hexadécimal

Un éditeur hexadécimal est un logiciel qui permet visualiser ou modifier un fichier binaire. Chaque octet y est représenté de manière concise en base 16 (donc par un des symboles ou chiffres hexadécimaux : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F (alias 15))

espace de nom

Commençons par poser le problème : la fonction sinus n'est pas définie dans

le cœur du langage Python mais elle est disponible dans le module **math** de la bibliothèque standard (installée avec le langage) et dans le module **numpy**. Vous importez les deux modules et avez besoin d'appeler `sinus` : laquelle des deux fonctions `sinus` va répondre à l'appel ?

La notion d'**espace de noms** est un moyen pour éviter de tels conflits de noms. On préfixe le nom utilisé par celui de son module d'origine ; ainsi, la fonction `sin` du module `maths` s'appelle `math.sin(...)` et celle du module `numpy` s'appelle `numpy.sin(...)`. Comme, au sein d'un même espace de noms, il n'y ne peut y avoir d'homonymes les instructions `math.sin(...)` et `numpy.sin(...)` sont sans ambiguïté.

```
>>> import numpy
>>> import math
>>> math.sin(10**7)
0.4205477931907825
>>> numpy.sin(10**7)
0.4205477931907825
```

exécutable

Un fichier exécutable est un fichier contenant un programme en langage machine directement exécutable. Cela signifie que le système d'exploitation le reconnaît en tant que tel, le charge en mémoire et l'exécute sans intervention d'un autre programme. Un tel programme est dédié à un processeur particulier et ne peut être exécuté sur une machine construite autour d'un autre type de processeur. Un exécutable est le plus souvent obtenu par compilation, parfois écrit directement en langage assembleur.

Contrairement à Wikipedia, nous ne dirons pas qu'un script Python est un exécutable, puisque pour que le code produise un effet, l'interpréteur (en l'occurrence le programme Python) est appelé.

Voir langage compilé, interprété, assembleur, langage machine.

garbage collector *voir ramasse-miettes*

IDE ou EDI

Un environnement de développement intégré est un éditeur dédié à l'écriture de code, disposant pour cela de fonctions annexes facilitant l'écriture, l'exécution de scripts ou la compilation de programmes ainsi que leur débogage. Avec Python à côté des consoles vous disposez de tels éditeurs spécifiques (avec coloration des mots clés, signalement d'erreurs de syntaxe, menus pour l'exécution des scripts) comme **IDLE**. Il existe des EDI plus universels. Par exemple **Eclipse** logiciel libre dans lequel on peut écrire du code Python, C, C++, PHP, Java.

IEEE

Association professionnelle d'ingénieurs des domaines de l'électronique et de l'informatique. Elle compte plusieurs dizaines de milliers de membres à titre personnel, un grand nombre de sociétés savantes. Elle est à l'origine de nombreuses publications : revues, documentation électronique... Elle joue un rôle majeur dans la définition de normes. Consultez sa page web francophone par exemple.

implémentation, implémenter

On dit que l'on implémente un algorithme lorsqu'on le traduit dans un langage de programmation ou dans un logiciel dans lequel il admet une réalisation effective. Par exemple l'algorithme de transformation de Fourier rapide est implémenté dans la bibliothèque `scipy`. Un langage de programmation est lui-même un algorithme défini par une longue famille de règles de réécriture ; une implémentation de ce langage en est une traduction en un programme effectif : votre logiciel Python par exemple, est une **implémentation** probablement écrite en C++ (il en existe implémentées en java). C lui même a été implémenté en langage machine ou en assembleur.

ingénierie inverse

L'ingénierie inverse (on dit aussi rétro-ingénierie) est le processus qui, à partir d'un objet (artefact ou logiciel) existant, consiste à imaginer ses composants cachés, son processus de fabrication. Une telle recherche peut trouver des motivations diverses : veille technologique, surveillance de violation de brevet, protection contre les virus informatiques, espionnage industriel ou tout simplement d'ordre didactique : « comment auriez-vous fait cela ? » demanda le professeur, qui savait par avance que sa question posée à un si brillant public, serait à l'origine de nombreux brevets.

intelligence artificielle

On regroupe sous ce vocable un ensemble de problèmes et de techniques de l'informatique qui ont en commun d'avoir fait fantasmer dans les années 50 un groupe de mathématiciens, physiciens, chimistes, psychologues et autres auteurs de science fiction qui se sont réunis l'été 1956 au Dartmouth College à Hanover, New Hampshire, pensant, dirons nous en exagérant à peine, que l'heure était proche où l'on saurait modéliser le fonctionnement du cerveau.

Aujourd'hui sont considérées comme faisant partie de l'intelligence artificielle les techniques permettant la classification automatique de documents, la reconnaissance de formes, d'images, le traitement du langage, la recherche de thèmes latents dans des documents et d'une façon générale ce qui relève de l'apprentissage automatique.

interprété (langage)

Un langage est interprété lorsque le code source est exécuté pas à pas à la différence d'un langage compilé dont le code est préalablement traduit en un code directement exécutable sur la machine.

Ainsi, vous pouvez écrire un code Python dans l'éditeur de votre choix, chaque fois que vous voudrez l'exécuter il vous faudra appeler un interpréteur Python

pour le lire et l'exécuter. Lorsque vous compilez un code C vous faites appel à un compilateur qui construit un programme directement exécutable par la machine. Vous pourrez donc distribuer cet exécutable qui fonctionnera y compris sur une machine ne disposant pas d'une implémentation de C.

mémoire

C'est un organe qui permet l'enregistrement, la conservation et la restitution des données. Les mémoires et leur propriétés évoluent rapidement avec les avancées technologiques : circuits électroniques, supports optiques, magnétiques, moléculaires ou autre, l'information y est stockée sous forme binaire, de façon volatile ou rémanente.

mémoire centrale, mémoire vive ou RAM

C'est la mémoire contenant les instructions et les données lors de leur traitement direct par le processeur. C'est donc une mémoire accessible en lecture et en écriture qui se doit d'être d'accès rapide. On parle aussi de **RAM** (*random access memory*). La mémoire centrale est aujourd'hui encore une mémoire volatile.

mémoire de masse

Terme désignant les mémoires de « grande » capacité et rémanentes. Actuellement associé au stockage de données sur disque, disque optique, disque flash ou bande magnétique.

mémoire morte

C'est une mémoire dans laquelle les informations ne peuvent pas être modifiées. On ne peut que les lire (c'est pourquoi on parle aussi de **ROM** pour *read only memory*). Sert aujourd'hui quasi exclusivement pour stocker des **logiciels embarqués**.

mémoire rémanente

C'est une mémoire contenant des données de façon permanente, par opposition aux mémoires volatiles. Par exemple :

- la mémoire contenant le **BIOS** qui permet le démarrage d'un ordinateur par la lecture des premières instructions (comme rechercher le système d'exploitation à une adresse fixe sur un disque dur (mémoire de masse)) ;
 - les mémoires de masses (disque dur en général) contenant le système d'exploitation, les fichiers de données, les programmes installés sur un ordinateur ;
 - les mémoires flash des clés usb,
- sont des mémoires rémanentes.

mémoire volatile

C'est une mémoire dont les données sont perdues (désorganisées) dès lors que l'alimentation électrique est coupée. Par exemple la mémoire centrale d'un ordinateur.

module, package (ou paquet), bibliothèque (ou library)

Un module est un fichier (un script) que l'on peut importer à partir du shell

ou à partir d'un autre script et qui étend les possibilités du langage. Une bibliothèque ou paquet (librairie or package) est un dossier contenant des (sous-) modules. Ce qui fait qu'un script devient un module, c'est sa place dans l'arborescence qui permet de le charger par une instruction simple (comme **import** avec Python par exemple).

octet

Voir byte.

OS, système d'exploitation

Un système d'exploitation (ou OS pour operating system) est le logiciel installé sur une machine physique qui sert de couche intermédiaire entre le matériel (processeurs, mémoires, cartes vidéo, écrans, interfaces usb, lecteurs de cartes etc.) et les autres logiciels : traitements de texte, navigateurs, gestionnaires d'image et de vidéos, langages de programmation... Les logiciels (y compris les langages de programmation) doivent être implémentés en fonction de cette sous-couche.

Clarifions :

Le script Python que vous écrivez est identique d'une machine à l'autre (ce qui fait de Python un langage de haut niveau - plusieurs niveaux au dessus du langage machine). Mais le programme Python qui interprète ce script en réalisant l'exécution pas à pas des instructions qu'il contient, dépend quant à lui, de votre machine. En clair, si vous téléchargez Python pour l'installer sur votre ordinateur, vous devez prendre garde à télécharger la version correspondant au **couple** machine physique-OS.

De la même façon un OS dépend de la machine sur lequel il fonctionne : le code Unix sur un PC avec processeur Intel ne sera pas le même que le code Unix implémenté sur une machine avec processeur Motorola (même différence entre un Intel 32 bits et un Intel 64 bits). Pour éviter trop de problèmes, les différents processeurs ont des jeux d'instructions compatibles pendant plusieurs générations, de la même façon les instructions des OS restent compatibles d'une version à l'autre ... jusqu'au jour où ça s'arrête. On rachète alors tout et on recommence !

paradigmes de programmation

Le mot paradigme admet plusieurs sens qui présentent des analogies.

- Au XIX^{ième} siècle, le Littré (dictionnaire) le définit comme suit : *Terme de grammaire. Exemple, modèle de déclinaison, de conjugaison. Le paradigme d'une conjugaison, la série des formes d'un verbe présentée en tableau.*
- Depuis les débuts du XX^{ième} siècle, il désigne en **épistémologie** des modèles de pensée systématisés dans des disciplines scientifiques.
- En informatique, c'est dans ce sens que l'on parle de paradigme de programmation fonctionnelle, impérative, logique, objet (ou orientée objet)...

paradigme en sciences expérimentales

Karl Popper considère qu'un paradigme des sciences expérimentales (et nous ajoutons des statistiques) est de construire des théories ou des modèles pour lesquels on peut imaginer et réaliser des expériences destinées à prouver qu'elles sont fausses (modèles falsifiables). Tant que ces expériences échouent, on ne rejette pas la théorie ou le modèle ; mais on se garde de croire que l'on a prouvé quoi que ce soit. En algorithmique, en faisant des preuves de programme, c'est-à-dire des démonstrations comme en mathématiques, on peut et **on doit** s'affranchir de ce paradigme.

pointeur

Un pointeur est un objet ou une variable qui contient l'**adresse mémoire** d'une donnée ou d'une fonction.

programmation fonctionnelle

C'est le paradigme de programmation dans lequel on s'impose de n'évaluer que des "fonctions pures" en rejetant les changements d'états ou l'utilisation de variables globales. Elle assure une sécurité accrue du code, une meilleure lisibilité des programmes.

Un langage fonctionnel est un langage dans lequel ce style de programmation est encouragé sinon le seul possible. Exemples : **LISP**, **Caml**.

programmation impérative

En programmation impérative, un programme est une suite d'instructions. La quasi-totalité des processeurs sont dotés d'un langage machine impératif. Les assembleurs et langages impératifs de plus haut niveau admettent comme instructions

- l'affectation ;
- le branchement conditionnel, (**if then else**) ;
- le bouclage avec la possibilité de programmer des itérations prédéfinies (boucle **for**) et conditionnelles (boucle **while**) ;
- le branchement inconditionnel (appel d'une fonction ou procédure et **goto** dans les assembleurs)

Exemples de langages impératifs : **FORTRAN**, **C**

programmation logique

En programmation logique, un programme est un ensemble de faits élémentaires et de règles de logique. Ces faits et règles sont exploités par un moteur d'inférence qui déduit de nouveaux faits... Un premier langage de programmation logique est **Prolog**, développé à Marseille-Luminy dans les années 70.

programmation objet

Dans les années 70, Niklaus Wirth écrivait qu'un programme était l'association de structures de données et d'algorithmes adaptés à ces structures².

Dans un langage qu'il écrit à cette époque et destiné à donner de bonnes habitudes de programmation dans l'enseignement³, l'utilisateur peut définir ses

2. Le titre de l'ouvrage qu'il fait paraître en 1976 est *Algorithms + Data Structures = Programs*

3. Le langage Pascal qui a très largement fait ses preuves, y compris dans l'industrie

propres types structurés, les enregistrements. Un enregistrement peut contenir plusieurs champs de types différents et permet de représenter des objets composites. Cette façon de regrouper dans le corps du programme les données qui sont sémantiquement liées permet de mieux spécifier les fonctions et les algorithmes qui assurent leur traitement.

A ce niveau, les opérations sur ces objets informatiques sont encore gérées par des fonctions ou programmes qui sont écrits séparément. En programmation objet on va plus loin dans l'intégration en définissant pour chaque type de données (que l'on appelle alors une **classe**), non seulement les champs qui le composent (que l'on appelle **attributs**), mais aussi les fonctions ou programmes qui opèrent sur les objets de la classe ou gèrent la création de nouveaux objets (on parle alors de **méthodes**). Dans l'idéal, seules les méthodes donnent accès ou permettent de modifier les attributs d'un objet. On parle alors de **l'encapsulation** des données et des méthodes.

Ainsi,

- les **objets** d'une classe donnée possèdent des champs ou **attributs** qui spécifient leurs structures de données (ils sont en nombre quelconque et de type quelconque). Ces attributs sont renseignés pour tous les objets de cette classe.
- La définition de la classe, inclut des **méthodes** qui sont les fonctions ou procédures opérant sur les objets de la classe ou définissant les relations avec les objets d'autres classes. On définit en particulier les règles de création (on dit d'**instanciation**) d'un objet appartenant à la classe. La méthode qui assure cela est un **constructeur**.
- Les changements de valeur des attributs sont gérés par les seules méthodes définies avec la classe de l'objet ; en particulier seul un appel du constructeur permet la création d'un nouvel objet et l'affectation de valeurs à ses champs.
- L'organisation entre les classes peut être gérée par l'**héritage** qui permet de définir des sous-classes de classes existantes. Une sous-classe gardera attributs et méthodes de la classe parente et possédera en plus des attributs et méthodes propres. C'est à la fois une économie puisqu'on ne réécrit pas plusieurs fois les algorithmes, et une sécurité accrue pour le code. On ne revient plus sur une méthode ou une structure bien spécifiée.
- On s'interdit (comme dans toute bonne programmation) l'usage de variables globales dans les méthodes et les interactions entre les objets sont entièrement définies par les méthodes.

Ce type de programmation permet de structurer des logiciels complexes en les organisant comme des ensembles d'entités distinctes (objets appartenant à des classes) dont les règles d'interaction sont définies de la façon la plus **modulaire** possible. Un objectif est de permettre à plusieurs équipes de (bons) programmeurs de travailler sur un même programme, chacune s'occupant d'une famille de classes, les interactions entre les classes étant gérées comme des méthodes de classe, une équipe ne connaissant que la spécification des méthodes définies

par les autres.

Il est tout à fait possible de faire de la programmation objet dans un langage quelconque mais la syntaxe de certains langages (Python, C++, Ocaml), que l'on dit alors **orientés objet**, offre des outils spécifiques permettant de définir les classes, de créer (ou instancier) les objets appartenant à ces classes, d'appeler les attributs et les méthodes...

ramasse-miettes (ou garbage collector)

Pour gérer l'espace mémoire, il y a deux manières de faire :

- soit on laisse l'utilisateur (ici un programmeur) s'en occuper et le langage met à sa disposition des primitives d'allocation et de restitution de blocs mémoire ;
- soit le langage s'en occupe en boîte noire : il met alors en œuvre un *garbage collector* dont le rôle est
 - de trouver les objets ou les blocs mémoires inutilisés ;
 - de rendre accessible l'espace qu'ils occupent.

Les algorithmes pour ces tâches sont sophistiqués et se doivent d'être performants (efficacité, rapidité...).

En C,C++, c'est la première solution qui est adoptée, avec Python c'est la seconde (avec toutefois la possibilité de désactiver le GC)...

registre

Un registre est un espace mémoire situé dans le processeur. Ces mémoires sont celles qui ont le meilleur temps d'accès (parce que situées dans le processeur) chacune d'elle est dédiée à des opérations particulières. On pourra trouver dans une architecture classique :

nom du registre	contenu du registre
compteur ordinal	adresse de la prochaine instruction
registre d'instruction	code de l'instruction en cours
pointeur de pile	adresse du prochain emplacement libre en mémoire
accumulateur	résultat de l'opération courante
registre de données	communication avec le bus de données
...	...

En dire plus reviendrait à entrer en détail dans l'architecture des processeurs et nous mènerait loin de nos objectifs du moment.

script

C'est un programme écrit dans un langage interprété. Il ne nécessite aucune compilation.

syntaxe et sémantique

Qu'il s'agisse des langues naturelles ou des langages de programmation, la syntaxe d'un langage est l'ensemble des règles de formation des phrases ou expressions correctes du langage, sa grammaire. La sémantique d'un texte a trait au sens des phrases ou expressions. La sémantique d'un programme a trait à ce qu'il produit lors de son exécution. Lorsqu'il exécute un script l'interpréteur (ou le compilateur) analyse la syntaxe du programme. Lorsque vous

construisez une preuve de correction de programme, vous en faites une étude sémantique. Les commentateurs d'un discours politique analysent en général sa sémantique, même si parfois la syntaxe peut, par ses dérives ou sa qualité, retenir leur attention.

virtuelle

Une machine virtuelle est un programme qui simule le fonctionnement d'un ordinateur (machine physique + OS) sur une autre machine. L'ordinateur simulé est la **machine virtuelle**, l'ordinateur sur lequel elle est installée est l'**hôte**.

Illustrons cela : Votre machine est un PC (processeur Intel ou AMD) qui fonctionne sous Windows 10.

Vous voulez utiliser d'anciens périphériques qui ne fonctionnent que sous XP. Vous installez donc un logiciel qui crée des machines virtuelles comme la **VirtualBox** de Oracle (gratuite et sous licence libre). Vous créez alors, grâce à ce logiciel, un programme qui simule (on dit dans ce cas **émule**) un ordinateur sur lequel les logiciels ne sont pas encore installés (c'est l'état de votre machine physique quand vous formatez le disque système avant de réinstaller un OS). Sur cette machine virtuelle, qui gère le lecteur de CD Rom, les bus usb..., vous pouvez installer Windows XP à partir de son disque original. Vous disposez alors d'une (émulation de) machine XP dans votre machine W7. Vous pouvez aussi faire tourner Windows sous Linux et Linux sous Windows.

Voir aussi : OS

Bibliographie

- [1] T. BALABONSKI, S. CONCHON, J-C FILLIÂTRE, KIM NGUYEN
Numérique et sciences informatiques - classe de 1^{ère}
Numérique et sciences informatiques - classe de T^{ale}
Ellipse, 2019
La spécialité NSI au lycée : vous pourrez y trouver bien des informations et des idées.

- [2] MICHAËL BAUDIN
Scilab is not naive
Polycopié en ligne, consortium Scilab, sur les calculs avec les flottants (consulté 03/2021)
<https://forge.scilab.org/index.php/p/docscilabisnotnaive/downloads/178/>

- [3] MAÏTINE BERGOUNIOUX
Quelques méthodes de filtrage en Traitement d'Images (2011)
<https://hal.archives-ouvertes.fr/hal-00512280v2/document>

- [4] DIETMAR BERWANGER
Graph games with perfect information
<http://www.lsv.fr/~dwb/graph-games-pi.pdf>

- [5] ALAIN CAZES ET JOELLE DELACROIX
Architecture des machines et des systèmes informatiques
Dunod, 2011
Pour ceux qui ne veulent pas attendre d'être en école d'ingénieur pour comprendre le fonctionnement de leur ordinateur !

- [6] THOMAS H. CORMEN, CHARLES E. LEISERSON, RONALD L. RIVEST, CLIFFORD STEIN
Algorithmique
Dunod, 2009
Une référence fondamentale.

- [7] MICHEL CRUCIANU, MEZIANE YACCOUB
Classification automatique
Cours Cnam RCP208 en ligne, 2019
<https://cedric.cnam.fr/vertigo/Cours/ml/coursClassificationAutomatique.html#figgroupescompa>
- [8] CHRISTINE FROIDEVAUX, MARIE-CLAUDE GAUDEL ET MICHÈLE SORIA
Types de données et algorithmes
McGraw-Hill, coll. « Informatique », 1990, 575 p
Les fondements de l'algorithmique. Très clair.
L'ouvrage semble épuisé, on en trouvera une version scannée sur le site :
<https://www.lri.fr/~chris/LivreAlgorithmique/FroidevauxGaudelSoria.pdf>
- [9] PIERRE GILLES DE GENNES
La percolation : un concept unificateur
La Recherche. Mensuel n° 99, Avril 2000
- [10] GIMP, DOCUMENTATION
Matrice de convolution
<https://docs.gimp.org/fr/gimp-filter-convolution-matrix.html>
Les filtres de convolution dans le mode d'emploi de Gimp.
- [11] PIERRE-ANTOINE GUIHÉNEUF
Rotations discrètes, mode d'emploi pour faire tourner une image
Images des mathématiques - cnrs - 2018
<http://images.math.cnrs.fr/Rotations-discretes.html>
Pour en savoir plus sur les problèmes sous-jacents à la rotation des images.
Vulgarisation avec des ouvertures plus poussées.
- [12] HARRY KESTEN
What is ... Percolation.
Notices of the AMS, vol. 53, number 5
<http://www.ams.org/notices/200605/what-is-kesten.pdf>
Deux pages sur la percolation comme problème mathématique.
- [13] DEXTER KOZEN, SHMUEL ZAKS
Optimal Bounds for the Change-Making Problem.
Theoretical Computer Science, 123 (1994)
<https://www.cs.cornell.edu/~kozen/Papers/change.pdf>
Les systèmes de monnaies.
- [14] FRÉDÉRIC LE ROUX
Le jeu de Hex

interstices INRIA (2012)
<http://images.math.cnrs.fr/Le-jeu-de-Hex.html>
Le jeu de Hex.

[15] RÉMY MALGOUYRES

Algorithmes pour la synthèse d'images et l'animation
Dunod, 2002

Abordable et complet : les algorithmes permettant de dessiner en 2D en 3D à l'écran. Si votre TIPE vous conduit à vous intéresser à ces questions.

[16] OLIVIER THÉTAUD

Programmation des échecs et d'autres jeux
interstices INRIA (2012)

<https://interstices.info/programmation-des-echecs-et-dautres-jeux/>
Échecs et minimax.

[17] FABIEN TORRE

L'Intelligence Artificielle et les Jeux

<https://fabien-torre.fr/Enseignement/Cours/Intelligence-Artificielle/jeux.php>
Jeux et minimax.

[18] STEPHEN WOLFRAM

Cellular Automata and Complexity
Westview Press, 1994

Le point de vue d'un Physicien sur les automates cellulaires (ou celui d'un informaticien sur la Physique).

Index

- _ (underscore)
 - dernière expression évaluée, 14
- accessibilité
 - jeu, 512
- accumulateur, 569
- adjacence
 - matrice (graphe), 369
- adjacent
 - sommet, 365
- affectation, 13
 - dans une autre instruction, 15
 - multiple, 15
- agenda, 208, 216
- algorithme
 - attracteur (jeux), 514
 - calcul PLSC, 481
 - composantes connexes, 374, 395
 - d'Euclide
 - étendu, 331
 - v.réursive, 144
 - de Bresenham (point milieu), 242
 - de détection de contours, 260
 - de Dijkstra, 383
 - de Floyd, 320
 - de Hörner, 329
 - de la médiane, 200
 - de parcours d'arbre, 521
 - de rech. de points les plus proches, 161
 - de recherche de motif, 116
 - de recherche dichotomique
 - v.itérative, 117
 - v.réursive, 164
 - de recherche du maximum, 106
 - de recherche séquentielle, 103
 - de rotation d'image, 247
 - de Roy-Floyd-Warshall, 490
 - de tri
 - par comptage, 113
 - par fusion, 194
 - par insertion, 190
 - par insertion dichotomique, 196
 - par sélection, 109
 - rapide, 192
 - des k-moyennes, 548
 - des k-plus proches voisins, 544
 - distance de Levenshtein, 489
 - division euclidienne, 41, 96
 - du minimax, 522
 - exponentiation rapide
 - v. itérative, 121
 - v.réursive, 163
 - glouton, 210
 - attrib. de salles, 210
 - rendu de monnaie, 210, 211
 - sélect.activ, 213
 - LZ78 (compression), 469
 - parcours largeur d'un graphe, 375
 - parcours profondeur d'un graphe
 - v. itérative, 378, 397
 - v. réursive, 370
 - parenthésage opt. (v. ascendante), 484
 - parenthésage opt. (v. réursive), 485
 - pour la convolution, 255
- allocation
 - de salles, 208, 216
- alvéole
 - table de hachage, 467
- annulation
 - phénomène d', 296
- ans

- dernière expression évaluée, 14
- appel
 - principal, 143
 - récuratif, 143
 - terminal, 143, 152
- apprentissage
 - automatique, 543
 - non supervisé, 544
 - supervisé, 543, 544
- arête
 - dans un graphe, 365
- arbre, 517
 - binaire, 17
 - de jeu (Nim), 519
 - de jeu (oxo), 519
 - des appels récuratifs, 194
 - syntactique, 16, 17
- arc
 - dans un graphe, 366
- arene
 - de jeu, 510
- argument
 - effectif, 48
 - formel, 48
 - par défaut, 156
- arité
 - opérateur, 16
 - opérateur, fonction, 569
- arrêt
 - d'un programme (control-c), 41
- array
 - numpy.array, 56
- arrondi, 291
 - flottants, 290
- artificielle
 - intelligence, 543
- as
 - module, 55
- assembleur, 569
- attracteur
 - jeux d'accessibilité, 513
- attribut
 - d'une classe, 405
 - programmation objet, 577
- auxiliaire
 - fonction, 153
- AVG
 - SQL, 439
- Avogadro
 - nombre d', 321
- barycentre, 541
- base
 - de données, 425
 - relationnelles, 425
 - de numération, 98
- bassin
 - d'attraction, 513
- Bezout
 - identité de, 332
- bibliothèque, 574
 - numpy, 56
 - standard, 54
- bibliothèque standard, 570
- bin()
 - fonction native, 99
- binaire
 - opérateur, 569
- binomiaux
 - coefficients, 157
- biparti
 - graphe, 510
- bit, 233, 570
- boîte noire
 - dictionnaire, 111
- booléen
 - type, 21
- boucle
 - for, 35
 - invariant de, 318
 - programme qui, 41
 - variant de, 318
 - while, 40
- break
 - mot réservé, 44
- Bresenham
 - algorithme (point milieu), 242
- brute

- force, 210
- byte, 570
- calcul
 - formel, 570
 - formel (logiciel), 285
 - formel (SymPy), 298
 - scientifique, 570
- cancellation, 296
 - phénomène de, 296
- Catalan
 - nombres de, 483
- centre
 - de gravité, 541
- chaîne
 - dans un graphe, 365
- chargement
 - module, 54
- chemin
 - dans un graphe, 366
 - maximal, 511
- chevauchement
 - sous-problèmes, 488, 498
- chiffres
 - binaires, 98
- choix
 - local, 211
- circuit
 - graphe orienté, 366
- clé
 - d'un tri, 189
 - dans un tri, 199
 - dictionnaire, 465
- classe
 - constructeur, 407
 - d'équivalence, 366, 538
 - en programmation objet, 405
 - programmation objet, 577
- classement, 537
- classer, 537
- classification, 537
- classifier, 537
- coût
 - cumulé d'un chemin, 381
- codage
 - entiers uint8, 236
 - RGB, 234
- coefficients
 - binomiaux, 157
- coercition, 236
- Collatz
 - Lothar, 317
- collision
 - hachage, 467
- combinatoire
 - optimisation, 207
- compilé
 - langage, 571
- compilateur, 571
- complément
 - à deux, 284
- complex
 - attribut
 - conjugate, 20
 - imag, 20
 - real, 20
 - fonction
 - numpy.absolute, 57
 - numpy.conjugate, 57
 - numpy.exp, 57
 - numpy.image, 57
 - numpy.real, 57
 - type, 20
- complexité, 324
 - d'un algorithme, 95
 - exponentielle, 170
 - quadratique, 136
- composante
 - connexe, 365
 - fortement connexe, 366
- compréhension
 - liste en, 39
- compression
 - algorithme LZ78, 469
- comptage, 113
- compteur, 327
- condition
 - d'arrêt

- prog. récursif, 145
- d'arrêt (while), 41
- dans une instruction cond., 32
- de gain (jeu), 512
- configuration
 - géométrique, 147
- confusion
 - matrice de, 545
- connecteurs
 - logique, 21
- connexe
 - composante, 365
 - graphe, 365
 - graphe fortement, 366
- constante, 13
 - d'Euler (maths), 322
- constructeur
 - classe, 407
 - programmation objet, 577
- continue, 38
- contours
 - détection, 260
- contrôle
 - d'un sommet (gr. de jeu), 510
- contrastes
 - image, 256
- convention
 - de la valeur signée, 283
 - du complément à 2, 283
- conversion
 - de type, 236
- convolution
 - implémentation, 255
 - produit de, 251
- copy, 26
 - module, 26
- correction
 - partielle, 318
 - totale, 318
- cosinus
 - similarité, 540
- couleurs
 - fondamentales, 234
- COUNT
 - SQL, 439
- courbe
 - de Von Koch (pr. récursif), 147
 - de VonKoch (pr. itérative), 65
- décimaux
 - nombres, 280
- décomposition
 - base 2, 98
 - d'un entier, 158
- définition
 - fonction, 45
- dérivée
 - discrète, 250
- De Gennes
 - Pierre Gilles, 386
- deepcopy, 26
- def
 - définition d'une fonction, 45
- degré
 - d'un sommet dans un graphe, 365
- dénormalisé
 - nombre, 289
- dictionnaire, 27
 - comptage, 113
 - en boîte noire, 111
 - implémentation, 466
 - recherche dans, 111
 - spécification, 465
- Dijkstra
 - algorithme, 383
- discret
 - mathématique, 240
- dissimilarité, 539
- distance, 539
 - de Jaccard, 539
 - de Levenshtein, 488
- diviser
 - pour régner, 160
- division
 - bases de données, 431
 - euclidienne, 96
 - algorithme, 41
- document

- texte, 540
- domination
 - relation de, 325
- dyadiques
 - nombres, 280
- dynamique
 - typage, 312
- écran, 233
- EDI/IDE, 572
- éditeur, 571
- en place
 - tri, 189, 198
- encapsulation
 - programmation objet, 410, 577
- ensemble
 - d'apprentissage, 543
- ensemble (set), 26
- entier
 - multi-précision, 285
 - représentation (64bits), 282
 - sur 8 bits (uint8), 236
- enumerate, 37
- équation
 - de droite, 240
- équivalence
 - classe d', 366, 538
 - relation d', 366, 538
 - relation de comparaison, 325
- erreur
 - d'absorption, 294
 - d'annulation ou cancellation, 296
 - d'arrondi, 293
- espace de nom, 571
- etiqueter
 - étiqueter, 538
- Euclide
 - Algorithme, 330
 - Algorithme étendu, 331
- Euler
 - constante d', 322
- évaluation
 - fonction (minimax), 525
 - paresseuse, 137
- exécutable
 - fichier, 572
- exact
 - alg. glouton, 219
- explosion
 - combinatoire, 151
- exponentiation
 - rapide (v. iter.), 121
 - rapide (v. récursive) , 163
- exponentiation rapide
 - matricielle, 154
- expression, 13, 16
 - arbre syntaxique, 16
- externe
 - tri, 189
- extrémité
 - d'un arc, 366
 - initiale, 366
 - terminale, 366
- facteur
 - de remplissage, 467
- False, 21
- fenêtre, 249
 - glissante, 116, 153
- Fibonacci
 - récurrence matricielle, 154
 - suite de, 155, 332
- FIFO
 - pile, 375
- file
 - LILO, file d'attente, 375
- filtre
 - de Sobel, 260
 - linéaire, 251
- fine
 - partition plus fine que..., 537, 541
- finesse
 - des partitions, 537, 541
- flottant, 288
 - représentation, 288
 - type, 19
- floutage
 - image, 255

- Floyd
 - algorithme, 320
 - Roy-Floyd-Warshall, 490
- fonction, 45, 49
 - auxiliaire, 153
 - Fibonacci, 154
 - d'évaluation (minimax), 525
 - de Cantor-Lebesgue, 145
 - de prédiction, 543
 - de terminaison, 318
 - lambda, 49
- force
 - brute, 210
- formule
 - de Huygens, 100
 - de Stirling, 326, 483
- fortement connexe
 - composante, 366
- fractale, 65, 147
- FROM
 - SQL, 433
- géométrie
 - algorithmique, 161
- gagnante
 - position, 512
 - stratégie, 512
- gain
 - condition de, 512
- garbage collector, 572
- glouton
 - algorithme, 210
- gradient, 259, 260
 - discret, 259
- graphe, 365
 - biparti, 510
 - de jeu, 510
 - orienté, 366
 - pondéré ou valué, 367
- GROUP BY
 - SQL, 439
- Hörner
 - algorithme de, 329
- hachage
 - fonction de, 466, 467
 - table de, 467
- hauteur
 - d'un arbre, 194
- HAVING
 - SQL, 439
- heuristique
 - méthode, 524
- Huygens
 - formule de, 100
- IDE, 572
- identifiant
 - base de données, 424
- identificateur, 13
- IEEE, 572
- IEEE 754, 288
- image, 233
 - couleur, 234
 - niveaux de gris, 233
 - noir et blanc, 233
- immuable, 24
- implémentation, 573
 - de la convolution, 255
- import
 - module, 54, 571, 574
- incrémentatation, 15
- indépendants
 - sous-problèmes, 487
- indentation, 31
- indexable, 22
- indice
 - de Jaccard, 539
 - de similarité, 539, 540
- induit
 - sous-graphe, 365
- inertie
 - d'une famille, 541
 - d'une partition, 541
- ingénierie
 - inverse, 573
- instance
 - d'une classe, 405

- programmation objet, 577
- instanciation
 - programmation objet, 577
- instruction
 - conditionnelle if, 32
- int
 - type, 19
- intelligence
 - artificielle, 543, 573
- interne
 - tri, 189
- interprété
 - langage, 13, 573
- invariant
 - de boucle, 97, 318
- instance, 312
- itérable
 - objet, 35
- itérables, 27
- Jaccard
 - distance de, 539
 - indice de, 539
- jeu
 - d'accessibilité, 512
 - de Nim, 319, 507
 - graphe de, 510
 - OXO, 510
- JOIN ON
 - SQL, 433
- jointure
 - bases de données, 429
- k-means
 - algorithme (k-moyennes), 548
- k-moyennes
 - algorithme, 548
- k-plus proches voisins
 - algorithme, 544
- k-ppv
 - k-plus proches voisins, 544
- Kesten
 - Harry, 386
- key
 - argument de sort(), 199
 - dans sort, 199
- Lamé
 - théorème de, 334
- lambda, 49
- langage
 - assembleur, 569
 - machine, 569
- Levenshtein
 - distance de, 488
- lexicographique
 - ordre, 162
- library, 574
- ligne
 - polygonale, 147
- LILO
 - file d'attente, 375
- LIMIT
 - SQL, 434
- lissage
 - image, 255
- list (liste), 25
- liste
 - chaînée
 - définition, 463
 - d'adjacence, 367
 - en compréhension, 39
- liste commentée
 - des mots réservés, 18
- local
 - meilleur choix, 211
 - optimum, 211
- mémoïsation
 - en progr.dynamique, 488
- matplotlib.image, 234
- matrice
 - de confusion, 545
- MAX
 - SQL, 439
- médiane, 200
- mémoire, 574
 - centrale, 574

- de masse, 574
- flash, 574
- morte, 189, 574
- rémanente, 574
- RAM, 574
- ROM, 574
- vive, 189
- volatile, 574
- mémoïsation, 152, 334, 335
 - suite de Fibonacci, dictionnaires, 156
 - suite de Fibonacci, tableau, 155
- méthode
 - approchée, 219
 - d'une classe, 405
 - des trapèzes, 102
 - exacte, 219
 - gloutonne, 210
 - programmation objet, 577
- MIN
 - SQL, 439
- module, 54, 574
 - matplotlib.image, 234
 - sympy, 364
- monnaie
 - rendu de, 207
- motif, 116
- mots
 - utiles, 540
 - vides (stop-words), 540
- mots réservés
 - and, 21
 - as, 55
 - break, 44
 - def, 45
 - elif, 31
 - else, 31
 - False, 21
 - from, 54
 - global, 47
 - if, 31
 - import, 54
 - in, 21
 - is, 21
 - lambda, 49
 - liste des, 18
 - None, 45
 - not, 21
 - or, 21
 - return, 45
 - True, 21
 - while, 40
- moyenne
 - calcul de la, 100
- muable, 25
- multi-précision
 - entiers, 285
- MySQL
 - bases de données, 433
- nœud
 - arbre, 517
- Nim
 - jeu de, 319, 507
- nombres
 - de Catalan, 483
 - premiers, 44
- nonlocal, 50
- normalisé
 - nombre, 288
 - noyau, 255
- norme
 - IEEE 754, 288
- noyau
 - de convolution, 251
 - de Sobel, 260
 - normalisé, 255
- numération
 - binaire, 280
 - décimale, 280
 - hexadécimale, 280
- O
 - relation de domination, 325
- occurrences
 - nombre d', 37
- octet, 233, 575
- opérateur
 - linéaire, 251

- python
 - `+, -, *, **`, 19
 - / division , 19
 - // division entiers, 19
 - % reste, 19
- opérateurs
 - logiques, 21
- opération
 - fondamentale, 324
- optimale
 - solution, 209
 - sous-structure, 478, 487
- optimisation, 207
 - combinatoire, 207
- optimum
 - global, 207
 - local, 207, 211
- ordre
 - lexicographique, 189
 - total, 165, 189, 199
- OXO
 - jeu de, 510
- package, 574
- paquet, 574
- paradigme, 575
- paramètre
 - effectif, 48
 - formel, 48
- parenthésage
 - optimal (pr. de matrices), 482
- parenthésages
 - nombres de, 483
- paresseuse
 - évaluation, 137
- partage
 - d'un entier, 158
- partie
 - dans un jeu, 511
 - partielle dans un jeu, 511
- parties
 - d'une liste, 335
- partition, 537
 - d'un ensemble, 158
- pente
 - d'une droite, 240
 - discrète, 250
- percolation, 386, 387
- permutation, 146
- pile, 375
- pivotement
 - image, 247
- pixel
 - écran, 233
- place
 - tri en, 189
- PLSC
 - plus longue..., 477
- plus longue
 - sous-suite commune, 477
- point milieu
 - algorithme de Bresenham, 242
- pointeur, 467, 576
- points
 - les plus rapprochés, 161
- polynômes
 - de Rump, 298
- pondéré
 - graphe, 367
- Popper
 - Karl, 576
- position
 - gagnante, 512
- post-condition, 311
- pré-condition, 311
- pré-ordre
 - total, 189, 199
- prédécesseur, 510
- prédiction
 - apprentissage, 543
 - fonction de, 543
- premiers
 - nombres, 37, 44
- problème
 - optimisation
 - combinatoire, 209
- procédure, 49
- produit

- cartésien (Bases de D.), 428
- de convolution, 251
- produit cartésien
 - bases de données, 428
- programmation
 - fonctionnelle, 576
 - impérative, 576
 - logique, 576
 - objet, 576
- quicksort, 192
- racine
 - d'un arbre, 517
- RAM
 - mémoire, 574
- ramasse-miettes, 578
- random (module), 43
- randrange, 43
- rang
 - d'une position (jeux), 513
- range, 35
- réalisable
 - solution, 209
- recherche
 - de motif dans une chaîne, 116
 - des deux valeurs les plus proches, 114
 - dichotomique, 117, 164
 - du maximum, 106
 - séquentielle, 103
- récuratif
 - programme, 143
- récursive
 - fonction, 143
- rékursivité, 143
- registre, 578
- relation
 - d'équivalence, 366, 538
 - de comparaison (maths), 325
 - théorie des ensembles, 422
- remplissage
 - taux, facteur de, 467
- rendu
 - de monnaie, 207
- renommage cartésien
 - bases de données, 430
- représentation
 - des arbres, 518
 - des entiers, 282
 - images, 233
- ressemblance, 538
- ressource
 - unique, 213
- return
 - mot réservé, 45
- RGB
 - codage couleurs, 234
- ROM
 - mémoire, 574
- rotation
 - image, 247
- Roy
 - Roy-Floyd-Warshall, 490
- Rump
 - polynômes de, 298
- sélection
 - tri, 109
- sémantique, 578
- séparateurs
 - syntaxe Python, 17
- série
 - harmonique, 321
 - harmonique, 322
- salles
 - allocation de, 208, 216
- scikit-learn
 - bibliothèque Python, 552
 - datasets, 552
 - Kmeans, 555
- script, 578
- SELECT
 - SQL, 433
- sélection
 - activités, 213
- set (ensemble), 26
- seuil de percolation, 387
- SGBD, 425, 445

- signal
 - discret, 251
- signature
 - d'une fonction, 311
- similarité, 539
 - cosinus, 540
- Sobel
 - filtre de, 260
 - noyau de, 260
- solution
 - optimale (pb. opt. comb.), 209
 - réalisable (pb. opt. comb.), 209
- sommet
 - d'un graphe, 365
- sort
 - méthode, 199
- sous-graphe, 365
- sous-listes
 - d'une liste, 335
- sous-problèmes
 - indépendants, 487
 - qui se chevauchent, 488
- sous-structure
 - optimale, 478, 487
- sous-suite
 - définition, 477
 - plus longue... commune, 477
- spécification
 - d'un algorithme, 95, 311
- SQL
 - bases de données, 433
- SQLite, 446
- stable
 - tri, 189
- stationnaire
 - suite, 513
- Stirling
 - formule de, 326, 483
- stratégie
 - dans un jeu, 511
 - gagnante, 512
 - gloutonne, 210
 - praticable, 511
 - sans mémoire, 511
- substitution
 - alg.glouton, 219
- successeur, 510
- suite
 - de Collatz, 317
 - de Fibonacci, 151, 154, 155
 - de Syracuse, 317
 - double, 252
 - stationnaire, 513
- SUM
 - SQL, 439
- supervisé
 - apprentissage, 543
- support
 - fini, 251
- Sympy
 - calcul formel, 298
- sympy
 - module, 364
- syntaxe, 578
- Syracuse
 - suite de, 317
- système
 - d'exploitation, 575
- table
 - de hachage, 467
- tableau, 56
 - associatif (dictionnaire), 111
 - définition, 464
- tableau associatif
 - spécification, 465
- taille
 - des données, 324
- taux
 - de remplissage, 467
- terminaison
 - d'un algorithme, 318
 - fonction de, 318
- terminal
 - appel, 143, 152
- théorème
 - de Lamé, 334
- time, 328

- module, 328
- total
 - ordre, 189
 - pré-ordre, 189
- trapèzes
 - méthode des, 102
- tree
 - classe (arbres), 518
- tri
 - clé d'un, 189
 - dichotomique, 196
 - en place, 189
 - externe, 189
 - fusion, 166, 194
 - interne, 189
 - par comptage, 113
 - par insertion, 190
 - par sélection, 109
 - rapide, 166, 192
 - stable, 189
- True, 21
- tuple, 25
- typage
 - dynamique, 312
- type, 312
 - booléen, 21
 - complex, 20
 - flottant, 19
 - int, 19
- uint8
 - numpy, 236
- unaire
 - opérateur, 569
- valeur
 - de hachage, 467
 - dictionnaire, 465
- variable, 13
- variance
 - calcul de la, 100
 - d'une famille, 541
 - d'une partition, 541
- variant
 - de boucle, 97, 318
- vectorialisation
 - Python, 57
- virgule flottante, 288
- virtuelle
 - machine, 579
- Von Koch
 - courbe de, 65
 - courbes, 147
- Warshall
 - Roy-Floyd-Warshall, 490
- WHERE
 - SQL, 433
- while
 - boucle, 40

Classes préparatoires scientifiques
1^{re} et 2^e années

**NOUVEAUX
PROGRAMMES**

INFORMATIQUE

TRONC COMMUN

Ce cours couvre le programme d'informatique du tronc commun des classes préparatoires scientifiques de 1^{re} et 2^e années (MPSI, PCSI, PTSI, MP, PC, PSI et PT) mis en place en 2021.

Il est décomposé en trois parties, chacune correspondant à un semestre d'enseignement :

- Rappels sur le langage Python – Méthodes itératives – Récursivité – Tris – Algorithmes gloutons – Traitement de l'image.
- Représentation des nombres en machine – Preuves et complexité – Graphes – Aperçu de la POO.
- Bases de données et SQL – Dictionnaires – Programmation dynamique – Algorithmes et jeux – Algorithmes pour l'IA.

Il contient plus de 150 exercices, tous corrigés. Les scripts et des compléments sont disponibles sur le site des éditions Ellipses.

Thierry Audibert est professeur en classe préparatoire au lycée Paul-Cézanne à Aix-en-Provence.

Amar Oussalah a été maître de conférences en informatique à l'université de Provence.

www.editions-ellipses.fr



9 782340 048706